Learn Being Forward

COURSES · CAMPUS PARTNERSHIPS · PLACEMENTS

COURSE NOTES · BASIC TO ADVANCED

JavaScript Programming

Complete notes for the Learn Being Forward JavaScript Programming syllabus: every module, topic and subtopic explained step by step, with diagrams, verified examples and a practice checkpoint for each module.

25	3	69	572
modules	levels	minimum days	JavaScript programs

How to use these notes

These notes follow the **Learn Being Forward JavaScript Programming syllabus** exactly: the same 25 modules, in the same order, with every topic and subtopic explained step by step. Each subtopic number (for example **3.7.2**) matches the syllabus, so you can always find where you are.

Level	Modules	Minimum	Outcome
1. Beginner	1 to 10	18 days	Write JavaScript programs with variables, operators, control flow, functions, arrays, strings, objects, numbers and dates
2. Intermediate	11 to 18	26 days	Master closures and this, modern ES6+ features, prototypes and classes, the DOM, events, asynchronous code and APIs
3. Advanced	19 to 25	25 days	Modules and tooling, engine internals and patterns, advanced browser APIs, Node.js with Express, TypeScript, testing, frameworks and interview readiness
Total	1 to 25	69 days	About 12 weeks at 6 study days per week

Read
Read the concept for each subtopic and note the key points in your own words.

Type & run
Type out and run every example. Do not copy and paste code. Compare with the OUTPUT box.

Checkpoint
Finish the practice checkpoint at the end of each module before moving on.

Revise
At the end of each level, revise its modules and build one small project.

Every program that runs outside a web page was executed with **Node.js 22** to produce the output shown. Browser examples (the DOM, events, storage and other web APIs) were run against a simulated browser document where possible; where an example needs a real browser, a server or a network, its box says so and shows what the browser console or page displays. Values that depend on the moment or the machine, such as dates, timings and random numbers, will differ when you run the programs.

HOW TO READ THE BOXES
Dark boxes with a gold top line are **JavaScript code**. Dashed cream boxes labelled **OUTPUT** show what the program prints. Gold-bordered boxes are notes and tips, red boxes warn about common mistakes, and navy boxes hold the key idea to remember.

Course plan at a glance

Module	Topic	Level	Min days
01	Introduction to JavaScript	Beginner	1
02	Setup and your first program	Beginner	1
03	Variables and data types	Beginner	2
04	Operators and expressions	Beginner	1
05	Control flow	Beginner	2
06	Functions	Beginner	3

Module	Topic	Level	Min days
07	Arrays	Beginner	3
08	Strings	Beginner	2
09	Objects	Beginner	2
10	Numbers, Math and Date	Beginner	1
11	Scope, closures and the this keyword	Intermediate	4
12	Modern JavaScript (ES6 and beyond)	Intermediate	3
13	Object-oriented JavaScript: prototypes and classes	Intermediate	4
14	Error handling and debugging	Intermediate	1
15	The DOM (Document Object Model)	Intermediate	4
16	Events	Intermediate	3
17	Asynchronous JavaScript	Intermediate	4
18	Working with APIs and browser storage	Intermediate	3
19	Modules and the tooling ecosystem	Advanced	2
20	Advanced JavaScript internals and patterns	Advanced	3
21	Advanced browser APIs	Advanced	3
22	Back-end JavaScript with Node.js and Express	Advanced	5
23	TypeScript essentials	Advanced	4
24	Testing, security and best practices	Advanced	3
25	Frameworks, data structures, projects and interviews	Advanced	5

Table of contents

LEVEL 1 · BEGINNER LEVEL · MODULES 1 TO 10

Module 01 · Introduction to JavaScript 9

1.1 What is JavaScript? 9
1.2 History of JavaScript 12
1.3 Why do we need JavaScript? 14
1.4 Features of JavaScript 15
1.5 Where is JavaScript used? 16
1.6 JavaScript engines and runtimes 18
1.7 JavaScript compared with other languages 19

Module 02 · Setup and your first program 21

2.1 Tools 21
2.2 Running JavaScript 22
2.3 Hello World 25
2.4 Basic syntax 26
2.5 Comments 29
2.6 Strict mode 30
2.7 Console methods 31
2.8 Errors 32

Module 03 · Variables and data types 36

3.1 Declaring variables 36
3.2 Scope (introduction) 39
3.3 Data types overview 41
3.4 Primitive types in depth 43
3.5 Primitives vs references 46
3.6 Type conversion and coercion 47

Module 04 · Operators and expressions 52

4.1 Arithmetic operators 52
4.2 Assignment operators 54
4.3 Comparison operators 55
4.4 Logical operators 56
4.5 Nullish and optional operators 58
4.6 Other operators 59
4.7 Bitwise operators 60
4.8 Precedence and associativity 61

Module 05 · Control flow 64

5.1 Conditional statements 64
5.2 switch statement 67
5.3 Loops 68
5.4 Jump statements 71
5.5 Nested loops and patterns 73
5.6 Classic problems 74

Module 06 · Functions 80

6.1 Defining functions 80
6.2 Parameters and arguments 83

6.3 Return values 86
6.4 Functions as values 87
6.5 Arrow functions vs regular functions 89
6.6 Recursion 90
6.7 Pure functions and side effects (introduction) 92

Module 07 · Arrays 95

7.1 Array basics 95
7.2 Adding and removing 97
7.3 Searching 99
7.4 Iteration methods 101
7.5 Sorting and reordering 103
7.6 Other methods 105
7.7 Mutating vs non-mutating methods 105
7.8 Copying arrays 106
7.9 Array problems 107

Module 08 · Strings 111

8.1 Creating strings 111
8.2 Accessing characters 113
8.3 String methods 114
8.4 Regular expressions 118
8.5 Classic string problems 121

Module 09 · Objects 125

9.1 Creating objects 125
9.2 Working with properties 127
9.3 Methods 129
9.4 Iterating objects 130
9.5 Copying and merging 131
9.6 Protecting objects 133
9.7 Arrays of objects 134
9.8 JSON 135

Module 10 · Numbers, Math and Date 139

10.1 Number 139
10.2 Math 141
10.3 BigInt 143
10.4 Date 144
10.5 Timers (introduction) 148

LEVEL 2 · INTERMEDIATE LEVEL · MODULES 11 TO 18

Module 11 · Scope, closures and the this keyword 152

11.1 Execution context 152
11.2 Scope 154
11.3 Hoisting 156
11.4 Closures 158
11.5 The this keyword 162
11.6 Explicit binding 166
11.7 Functional programming techniques 167

Module 12 · Modern JavaScript (ES6 and beyond) 172

12.1 Destructuring 172

12.2 Spread and rest 177
12.3 Enhanced object literals 179
12.4 Map and Set 181
12.5 Symbols 187
12.6 Iterators and generators 189
12.7 Other modern features 196

Module 13 · Object-oriented JavaScript: prototypes and classes 201

13.1 Constructor functions 201
13.2 Prototypes 203
13.3 ES6 classes 209
13.4 Inheritance with classes 216
13.5 OOP principles in JavaScript 220
13.6 Property descriptors 225

Module 14 · Error handling and debugging 229

14.1 Error types 229
14.2 Handling errors 231
14.3 Debugging 238
14.4 Defensive programming 242

Module 15 · The DOM (Document Object Model) 247

15.1 DOM basics 247
15.2 Selecting elements 251
15.3 Traversing the DOM 254
15.4 Changing content 256
15.5 Changing attributes and styles 258
15.6 Creating and removing elements 262
15.7 Forms 267
15.8 Browser Object Model (BOM) 269
15.9 DOM performance 271

Module 16 · Events 275

16.1 Event basics 275
16.2 The event object 277
16.3 Event types 279
16.4 Event flow 286
16.5 Event delegation 290
16.6 Custom events 291
16.7 Performance 293

Module 17 · Asynchronous JavaScript 297

17.1 Synchronous vs asynchronous code 297
17.2 The event loop 299
17.3 Callbacks 302
17.4 Promises 303
17.5 async and await 310
17.6 Cancellation and timeouts 316

Module 18 · Working with APIs and browser storage 321

18.1 HTTP basics 321
18.2 Fetch API 324
18.3 Other request tools 329
18.4 CORS 331
18.5 Authentication basics 332

18.6 Browser storage 334

LEVEL 3 · ADVANCED LEVEL · MODULES 19 TO 25

Module 19 · Modules and the tooling ecosystem 341

19.1 Modules 341
19.2 npm 348
19.3 Build tools 354
19.4 Code quality tools 357
19.5 Version control 360

Module 20 · Advanced JavaScript internals and patterns 366

20.1 How engines run JavaScript 366
20.2 Memory management 369
20.3 Metaprogramming 374
20.4 Functional programming 378
20.5 Design patterns in JavaScript 380
20.6 Performance 386
20.7 Writing polyfills 389

Module 21 · Advanced browser APIs 394

21.1 Observers 394
21.2 Graphics and media 398
21.3 Background work 403
21.4 Progressive Web Apps (PWA) 405
21.5 Device and system APIs 408
21.6 Real-time communication 413
21.7 Web Components 416

Module 22 · Back-end JavaScript with Node.js and Express 423

22.1 Node.js fundamentals 423
22.2 Core modules 427
22.3 Express.js 436
22.4 Building a REST API 443
22.5 Databases 449
22.6 Authentication and security 453
22.7 Deployment 459

Module 23 · TypeScript essentials 466

23.1 Introduction 466
23.2 Basic types 469
23.3 Object types 473
23.4 Functions 479
23.5 Classes in TypeScript 480
23.6 Generics 482
23.7 Narrowing 484
23.8 Utility types 486
23.9 Using TypeScript with Node.js and Vite 488

Module 24 · Testing, security and best practices 493

24.1 Testing 493
24.2 Web security 503
24.3 Clean code 507
24.4 Accessibility basics 510

Module 25 · Frameworks, data structures, projects and interviews 515

25.1 Front-end frameworks (overview) 515
25.2 Data structures and algorithms in JavaScript 522
25.3 Capstone projects 532
25.4 Interview preparation 534
25.5 Career paths with JavaScript 542

LEVEL 1

Beginner level

Outcome: write JavaScript programs with variables, operators, control flow, functions, arrays, strings, objects, numbers and dates.

10 MODULES · 18 DAYS MINIMUM

Module 01 · Introduction to JavaScript	**1 day**
Module 02 · Setup and your first program	**1 day**
Module 03 · Variables and data types	**2 days**
Module 04 · Operators and expressions	**1 day**
Module 05 · Control flow	**2 days**
Module 06 · Functions	**3 days**
Module 07 · Arrays	**3 days**
Module 08 · Strings	**2 days**
Module 09 · Objects	**2 days**
Module 10 · Numbers, Math and Date	**1 day**

MODULE 01 · BEGINNER

Introduction to JavaScript

Minimum duration **1 day** | Level **Beginner**

This module explains what JavaScript is, how it became the language of the web, and where it runs today, before any code is written.

IN THIS MODULE

1.1 What is JavaScript?
1.2 History of JavaScript
1.3 Why do we need JavaScript?
1.4 Features of JavaScript
1.5 Where is JavaScript used?
1.6 JavaScript engines and runtimes
1.7 JavaScript compared with other languages

1.1 What is JavaScript?

JavaScript is the programming language of the web. Every time a web page does something more than display static text and pictures (a menu slides open, a form warns you that your email address is invalid before you submit it, a map scrolls smoothly, new messages appear without reloading the page) JavaScript is almost certainly doing the work. Today it also runs on servers, phones, desktops and even microcontrollers, which makes it one of the most widely used programming languages in the world.

1.1.1 Definition: a high-level, dynamically typed, interpreted (JIT-compiled) programming language

High-level

You write in human-friendly terms (variables, functions, objects); memory is managed for you by a **garbage collector**.

Dynamically typed

A variable has no fixed type: `let x = 5;` can later hold `"five"`. Types belong to **values** and are checked while the program runs.

Interpreted, JIT-compiled

You do not run a separate compiler. The engine reads your source code directly and compiles hot code to machine code **just in time** while it runs.

Older books call JavaScript an "interpreted scripting language". That was accurate in 1995; modern engines such as V8 combine an interpreter with optimising just-in-time (JIT) compilers, which is why today's JavaScript can be surprisingly fast.

A FIRST TASTE OF JAVASCRIPT

main.js

```
let message = "Hello";        // a string now
console.log(message, typeof message);
message = 42;                 // the same variable can hold a number later (dynamic typing)
console.log(message, typeof message);
console.log(0.1 * 3, 2 ** 10); // numbers, operators and output: no compile step needed
```

OUTPUT

```
Hello string
42 number
0.30000000000000004 1024
```

1.1.2 Multi-paradigm: procedural, object-oriented (prototype-based) and functional

JavaScript does not force one style of programming. You can write step-by-step **procedural** code, model the world with **objects** (JavaScript's objects are based on **prototypes**, with the `class` syntax added in 2015 on top), or use a **functional** style in which functions are values that are passed around, combined and returned. Real programs mix all three.

ONE PROBLEM, THREE STYLES

main.js

```
// procedural: a sequence of steps
let total = 0;
for (const price of [120, 80, 50]) total += price;
console.log("procedural:", total);

// object-oriented: data and behaviour together
class Cart {
  constructor() { this.items = []; }
  add(price) { this.items.push(price); return this; }
  total() { return this.items.reduce((sum, p) => sum + p, 0); }
}
console.log("object-oriented:", new Cart().add(120).add(80).add(50).total());

// functional: functions as values, no changing state
const sum = (list) => list.reduce((a, b) => a + b, 0);
console.log("functional:", sum([120, 80, 50]));
```

OUTPUT

```
procedural: 250
object-oriented: 250
functional: 250
```

1.1.3 Single-threaded with an event loop

A JavaScript program runs on **one thread**: only one piece of your code executes at any moment. Yet a web page can download data, run a timer and respond to clicks "at the same time". The trick is the **event loop**. Slow operations (network requests, timers, file reads) are handed to the browser or Node.js, which perform them in the background; when one finishes, a callback is placed in a **queue**, and the event loop runs queued callbacks one by one whenever the call stack is empty. Module 17 explains this in depth; the example below already shows its effect.

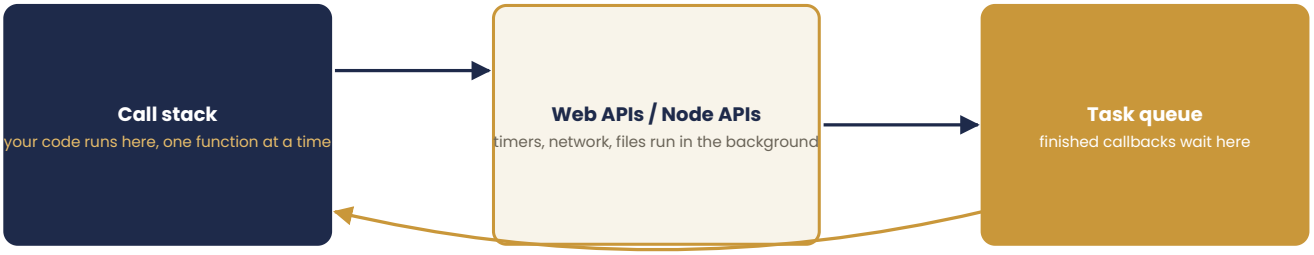
EVEN A 0 MS TIMER WAITS FOR THE CURRENT CODE TO FINISH

main.js

```
console.log("1. start");
setTimeout(() => console.log("3. timer callback (runs after the current code finishes)"), 0);
console.log("2. end of the script");
```

OUTPUT

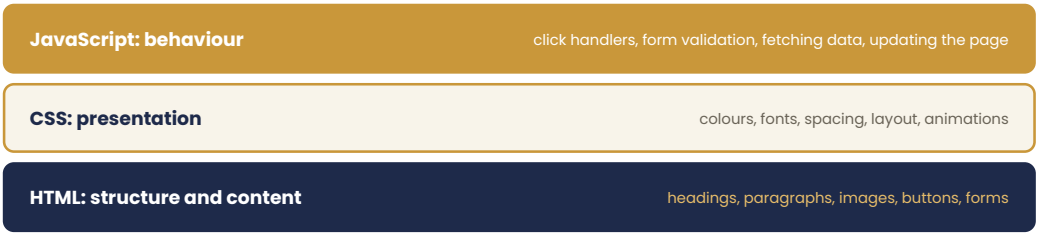
```
1. start
2. end of the script
3. timer callback (runs after the current code finishes)
```



THE EVENT LOOP IN ONE PICTURE

1.1.4 The three layers of the web: HTML (structure), CSS (style), JavaScript (behaviour)

A web page is built from three languages that work together. **HTML** describes the content and its structure (headings, paragraphs, images, buttons, forms). **CSS** describes how it looks (colours, fonts, layout, animation). **JavaScript** describes how it behaves: it reacts to the user, changes the page, talks to servers and stores data. A useful picture is a house: HTML is the walls and rooms, CSS is the paint and furniture, and JavaScript is the electricity and plumbing that make things work.



THE THREE LAYERS OF EVERY WEB PAGE

ALL THREE LAYERS IN ONE SMALL PAGE

HTML

```
<!-- HTML: structure -->
<button id="like">Like</button>
<span id="count">0</span> likes

<!-- CSS: style -->
<style>
  #like { background: #C9973A; color: white; border: none; padding: 6px 14px; border-radius: 6px; }
</style>

<!-- JavaScript: behaviour -->
<script>
  let likes = 0;
  document.getElementById("like").addEventListener("click", () => {
    likes++;
    document.getElementById("count").textContent = likes;
  });
</script>
```

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```html
<button id="like">Like</button> <span id="count">0</span> likes
```

THE BEHAVIOUR LAYER RUNNING IN A BROWSER main.js

```javascript
let likes = 0;
const button = document.getElementById("like");
button.addEventListener("click", () => {
  likes++;
  document.getElementById("count").textContent = likes;
  console.log(`clicked, likes = ${likes}`);
});
button.click(); button.click(); button.click();          // simulate three clicks
```

BROWSER CONSOLE

```
clicked, likes = 1
clicked, likes = 2
clicked, likes = 3
--- page shows ---
Like 3 likes
```

1.2 History of JavaScript



MILESTONES IN THE HISTORY OF JAVASCRIPT

1.2.1 Created by Brendan Eich at Netscape in 10 days (1995)

In 1995 the Netscape Navigator browser dominated the young web, and Netscape wanted a lightweight language that web designers and part-time programmers could embed directly in pages. They hired **Brendan Eich**, who wrote the first version in about **ten days** in May 1995. It borrowed its syntax from Java and C, first-class functions from Scheme and prototype-based objects from Self. The rush explains some quirks that JavaScript still carries for backward compatibility, such as `typeof null === "object"`.

1.2.2 Names: Mocha → LiveScript → JavaScript

The language was first called **Mocha**, renamed **LiveScript** when it shipped in a Netscape beta in September 1995, and renamed again to **JavaScript** in December 1995. The final name was a marketing decision: Java was the most talked-about technology of the time, and Netscape had a partnership with Sun Microsystems, which owned Java. "JavaScript" is still a trademark (now owned by Oracle), which is why the official standard has a different name.

1.2.3 JavaScript vs Java (unrelated languages)

Despite the name, JavaScript and Java are completely different languages, designed by different people for different purposes. The usual joke: "Java is to JavaScript as car is to carpet." They share some C-style syntax (braces, `if`, `for`) and little else; section 1.7.1 compares them in detail.

1.2.4 ECMAScript standard and TC39

In 1996 Netscape submitted JavaScript to **Ecma International**, a standards organisation, so that all browsers could implement the same language. The standard is called **ECMAScript** (document ECMA-262), and JavaScript is its best-known implementation. The language evolves through **TC39** (Technical Committee 39), a group of browser makers, companies and invited experts. New features pass through a public process with stages 0 to 4; only proposals that reach stage 4 (two shipping implementations and tests) enter the yearly standard.

Stage	Name	Meaning
0	Strawperson	an idea submitted for discussion
1	Proposal	the committee agrees the problem is worth solving
2	Draft	the syntax and semantics are described precisely
2.7	Approved in principle	the design is complete; tests are being written
3	Candidate	implementations and feedback from real use
4	Finished	included in the next yearly edition of ECMAScript

1.2.5 ES5 (2009), ES6 / ES2015 (the big update), then yearly releases (ES2016 onwards)

ES3 (1999) was the version the web ran on for a decade; ES4 was abandoned as too ambitious. **ES5** (2009) added strict mode, JSON support and array methods such as `forEach`, `map` and `filter`. **ES6**, officially **ES2015**, was the biggest change in the language's history: `let` and `const`, arrow functions, classes, modules, promises, template literals, destructuring, default parameters, Map and Set and much more. Since then a smaller edition has been published **every year** (ES2016, ES2017 …), and browsers ship features continuously.

Edition	Notable additions
ES2015 (ES6)	let/const, arrow functions, classes, modules, promises, template literals, destructuring, spread, Map/Set, Symbol
ES2016	Array.prototype.includes, exponent operator **
ES2017	async/await, Object.entries/values, string padding
ES2018	rest/spread for objects, async iteration, Promise.finally
ES2019	Array.flat/flatMap, Object.fromEntries, optional catch binding
ES2020	BigInt, optional chaining ?., nullish coalescing ??, Promise.allSettled, globalThis
ES2021	replaceAll, logical assignment ||= &&= ??=, numeric separators 1_000
ES2022	class fields and private #fields, top-level await, .at(), Object.hasOwn, Error cause
ES2023	findLast, toSorted/toReversed/toSpliced/with (copying array methods)
ES2024	Object.groupBy, Promise.withResolvers, resizable ArrayBuffer
ES2025	Iterator helpers, new Set methods (union, intersection …), RegExp.escape, Promise.try, JSON modules

1.2.6 Node.js (2009) takes JavaScript to the server

For its first fourteen years JavaScript lived only in browsers. In 2009 **Ryan Dahl** released **Node.js**, which combined Google's fast V8 engine (2008) with an event-driven, non-blocking I/O library, so JavaScript could run on servers and handle thousands of simultaneous connections efficiently. Its package manager **npm** (2010) grew into the largest software registry in the world, and "JavaScript everywhere" became reality: the same language for the browser, the server, build tools and command-line programs.

JAVASCRIPT OUTSIDE THE BROWSER main.js

```
// This book runs most examples with Node.js:
console.log(`Node.js major version: ${process.versions.node.split(".")[0]}`);
console.log(`V8 engine version starts with: ${process.versions.v8.split(".")[0]}`);
console.log(`running on platform: ${process.platform}`);
```

OUTPUT

```
Node.js major version: 22
V8 engine version starts with: 12
running on platform: linux
```

1.3 Why do we need JavaScript?

1.3.1 The only language that runs natively in every browser

Browsers understand exactly three languages natively: HTML, CSS and JavaScript (plus WebAssembly, a low-level compilation target). If you want code to run inside a web page on every device, from phones to laptops to smart TVs, without installing anything, it has to be JavaScript or something compiled to it (TypeScript, for example, is compiled to JavaScript).

1.3.2 Interactive and dynamic web pages

Without JavaScript, every interaction requires sending a request to the server and reloading the whole page. With it, pages respond instantly: form validation as you type, image galleries, drag and drop, live search suggestions, infinite scrolling, chat, dashboards that update themselves. Whole applications such as Gmail, Google Docs, Figma and Visual Studio Code for the Web run in the browser as JavaScript.

1.3.3 Full-stack development with one language

With Node.js on the server, one developer or team can write the front end, the back end, the build scripts and even the database queries (MongoDB uses JavaScript-like syntax) in the same language. Code such as validation rules can be shared between client and server, and moving between parts of the project is easy. Stacks such as **MERN** (MongoDB, Express, React, Node.js) are built entirely on JavaScript.

1.3.4 Huge ecosystem (npm) and community

The **npm registry** hosts millions of packages: frameworks, utilities, UI components, testing tools. Almost any common problem already has a well-tested library. JavaScript also has one of the largest developer communities, so tutorials, answers and help are easy to find.

1.3.5 High job demand

JavaScript has been the most commonly used programming language in the Stack Overflow Developer Survey for more than ten consecutive years. Front-end, back-end (Node.js), full-stack and mobile (React Native) roles all require it, and TypeScript, its typed superset, is increasingly expected as well.

1.4 Features of JavaScript

1.4.1 Lightweight and interpreted

JavaScript needs no compilation step, no project files and no installation in a browser: write a line in the console and it runs. This makes experimenting fast and the learning curve gentle, while JIT compilation keeps real programs fast.

1.4.2 Dynamically and weakly typed

Dynamic typing means types are checked at run time and variables can hold any type. **Weak typing** means JavaScript converts between types automatically (**coercion**) instead of reporting an error, which is convenient but produces famous surprises. Module 3 covers coercion in detail; TypeScript (Module 23) adds static types for large projects.

WEAK TYPING: AUTOMATIC CONVERSIONS main.js

```
console.log("5" + 2);      // string concatenation: "52"
console.log("5" - 2);      // numeric subtraction: 3
console.log("5" * "2");    // 10
console.log(true + 1);     // true becomes 1: 2
console.log([] + {});      // both become strings: "[object Object]"
console.log(1 < 2 < 3, 3 > 2 > 1);   // (true < 3) is (1 < 3); (true > 1) is (1 > 1)
```

OUTPUT

```
52
3
10
2
[object Object]
true false
```

1.4.3 First-class functions

Functions are **values**: they can be stored in variables, kept in arrays and objects, passed as arguments to other functions and returned from functions. This makes callbacks, event handlers and functional programming natural, and it is the reason methods such as `map`, `filter` and `addEventListener` exist.

FUNCTIONS ARE VALUES main.js

```
const greet = function (name) { return `Hello, ${name}`; };   // a function stored in a variable
const shout = (text) => text.toUpperCase() + "!";

function twice(fn, value) { return fn(fn(value)); }             // a function receiving a function
console.log(twice(shout, "hi"));

const tools = [greet, shout];                                   // functions inside an array
console.log(tools.map((f) => f("asha")));

function makeMultiplier(n) { return (x) => x * n; }             // a function returning a function
const triple = makeMultiplier(3);
console.log(triple(7));
```

OUTPUT

```
HI!!
[ 'Hello, asha', 'ASHA!' ]
21
```

1.4.4 Prototype-based objects

In class-based languages such as Java, objects are created from classes. In JavaScript, objects inherit directly from **other objects**: every object has a hidden link to a **prototype** object, and when a property is not found on the object itself, JavaScript looks it up along this prototype chain. The `class` keyword (ES2015) is a convenient syntax over this mechanism (Module 13).

OBJECTS INHERITING FROM OBJECTS main.js

```
const animal = { speak() { return `${this.name} makes a sound`; } };
const dog = Object.create(animal);        // dog's prototype is animal
dog.name = "Bruno";
console.log(dog.speak());                  // speak is found on the prototype
console.log(Object.getPrototypeOf(dog) === animal, Object.hasOwn(dog, "speak"));
```

OUTPUT

```
Bruno makes a sound
true false
```

1.4.5 Event-driven and asynchronous

JavaScript programs mostly **wait for events** (a click, a key press, a message from a server, a timer) and react to them. Long operations do not block the program: they are started, and a callback, a **promise** or an `await` continues the work when the result arrives (Module 17).

ASYNCHRONOUS OPERATIONS FINISH LATER, IN THEIR OWN TIME main.js

```
function download(file, ms) {
  return new Promise((resolve) => setTimeout(() => resolve(`${file} downloaded`), ms));
}
console.log("requesting two files...");
download("photo.jpg", 200).then(console.log);
download("notes.pdf", 100).then(console.log);
console.log("the program keeps running while it waits");
```

OUTPUT

```
requesting two files...
the program keeps running while it waits
notes.pdf downloaded
photo.jpg downloaded
```

1.4.6 Platform independent

The same JavaScript code runs on Windows, macOS, Linux, Android and iOS, in any modern browser or JavaScript runtime, without changes or recompilation. Differences are in the **environment** rather than the language: a browser offers `document` and `window`, Node.js offers `fs` and `process`.

1.5 Where is JavaScript used?

Area	Typical tools	Examples
Front-end web development	React, Angular, Vue, Svelte, Next.js	Facebook, Instagram web, Netflix UI, YouTube
Back-end development	Node.js, Express, NestJS, Fastify, Deno, Bun	APIs at PayPal, LinkedIn, Uber, Walmart

Area	Typical tools	Examples
Mobile apps	React Native, Ionic, NativeScript	Instagram, Discord and Shopify mobile apps (parts)
Desktop apps	Electron, Tauri (JavaScript UI)	VS Code, Slack, Discord, Figma desktop
Browser games and graphics	Canvas, WebGL, Three.js, Phaser, Babylon.js	2D/3D games, data visualisations, product configurators
Serverless and cloud functions	AWS Lambda, Cloudflare Workers, Vercel and Netlify functions	event-driven back ends that scale automatically
Browser extensions	WebExtensions API (Chrome, Edge, Firefox)	ad blockers, password managers, Grammarly
Machine learning in the browser	TensorFlow.js, ONNX Runtime Web, Transformers.js	face detection, pose estimation, on-device AI models

1.5.1 Front-end web development (React, Angular, Vue)

Front-end frameworks organise large web interfaces into reusable **components** that update efficiently when data changes. **React** (Meta) is the most widely used library, **Angular** (Google) is a complete framework popular in enterprises, and **Vue** is known for its gentle learning curve. Module 25 introduces them.

1.5.2 Back-end development (Node.js, Express, NestJS)

On the server, Node.js runs JavaScript with access to files, networks and databases. **Express** is the classic minimal web framework, **NestJS** a structured framework inspired by Angular, and **Fastify** a fast alternative. Module 22 builds a REST API with Express.

1.5.3 Mobile apps (React Native, Ionic)

React Native uses React and JavaScript to build truly native iOS and Android apps from one code base; **Ionic** and Capacitor wrap web apps in a native shell.

1.5.4 Desktop apps (Electron: VS Code, Slack, Discord)

Electron packages a web app together with Chromium and Node.js into a desktop application for Windows, macOS and Linux. Visual Studio Code, Slack, Discord and many others are built this way.

1.5.5 Browser games and graphics (Canvas, WebGL, Three.js)

The **Canvas** API draws 2D graphics pixel by pixel, **WebGL** and the newer **WebGPU** give access to the graphics card, and libraries such as **Three.js** and **Phaser** make 3D scenes and 2D games approachable (Module 21).

1.5.6 Serverless and cloud functions

With serverless platforms you write small JavaScript functions that run in the cloud in response to events (an HTTP request, a file upload, a schedule), and the provider handles servers and scaling automatically.

1.5.7 Browser extensions

Chrome, Edge and Firefox extensions are written in HTML, CSS and JavaScript using the WebExtensions APIs, which let them modify pages, add toolbar buttons and react to browser events.

1.5.8 Machine learning in the browser (TensorFlow.js)

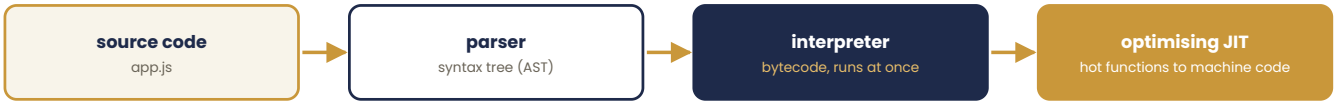
TensorFlow.js and similar libraries train and run machine-learning models directly in the browser or in Node.js, using the GPU through WebGL or WebGPU. Data can stay on the user's device, which helps privacy.

1.6 JavaScript engines and runtimes

Two terms are often confused. A JavaScript **engine** is the program that parses and executes JavaScript code: the language itself. A **runtime** is an engine plus the environment around it: the APIs a program can use (the DOM in a browser, the file system in Node.js) and the event loop that runs callbacks.

1.6.1 Engines: V8 (Chrome, Node), SpiderMonkey (Firefox), JavaScriptCore (Safari)

Engine	Developer	Used in
V8	Google (open source, C++)	Chrome, Edge, Opera, Brave, Node.js, Deno, Electron
SpiderMonkey	Mozilla	Firefox (it was the very first JavaScript engine)
JavaScriptCore (Nitro)	Apple	Safari and every browser on iOS, Bun



HOW A MODERN ENGINE SUCH AS V8 RUNS CODE

1.6.2 Runtimes: browsers, Node.js, Deno, Bun

Runtime	Engine	Provides	Notes
Browser	V8 / SpiderMonkey / JavaScriptCore	window, document (DOM), fetch, localStorage, events	sandboxed: no direct file-system access
Node.js	V8	fs, http, process, path, npm ecosystem	the standard server-side runtime since 2009
Deno	V8	web-standard APIs, built-in TypeScript, secure by default (permissions)	created by Node.js author Ryan Dahl (2018)
Bun	JavaScriptCore	very fast start-up, bundler, test runner and package manager built in	Node.js-compatible (2022)

DETECTING THE RUNTIME main.js

```
// Code can check which environment it is running in:
const inBrowser = typeof window !== "undefined" && typeof document !== "undefined";
const inNode = typeof process !== "undefined" && !!process.versions?.node;
console.log(`browser: ${inBrowser}, Node.js: ${inNode}`);
console.log(`globalThis works everywhere: ${typeof globalThis}`);
```

OUTPUT

```
browser: false, Node.js: true
globalThis works everywhere: object
```

1.7 JavaScript compared with other languages

1.7.1 JavaScript vs Java

Aspect	JavaScript	Java
Typing	dynamic and weak	static and strong
Execution	interpreted + JIT in an engine; no build step needed	compiled to bytecode, run on the JVM
Objects	prototype-based; class syntax on top	class-based
Concurrency	single thread + event loop; workers for parallelism	multi-threaded
Main home	browsers, Node.js	enterprise back ends, Android
Hello world	`console.log("Hi")`	a class with a main method

1.7.2 JavaScript vs Python

Aspect	JavaScript	Python
Typing	dynamic, weak (implicit coercion)	dynamic, strong (no implicit "5" + 2)
Blocks	braces { } and semicolons	indentation
Runs in browsers	yes, natively	no (only via WebAssembly projects such as Pyodide)
Strengths	web front end and back end, real-time apps	data science, AI/ML, scripting, automation
Async model	built around the event loop and promises	asyncio as one option among threads and processes

1.7.3 JavaScript vs TypeScript

TypeScript, created by Microsoft in 2012, is a **superset** of JavaScript: every JavaScript program is valid TypeScript, and TypeScript adds optional **static types** that are checked by the compiler and then removed, producing plain JavaScript. Types catch mistakes before the code runs and make editors much smarter, which is why most large projects use it. TypeScript does not replace knowing JavaScript; it builds on it (Module 23).

TYPESCRIPT ADDS TYPES TO JAVASCRIPT TypeScript

```
// TypeScript: types are checked before the program runs
function area(width: number, height: number): number {
  return width * height;
}
area(5, "3");   // error TS2345: Argument of type 'string' is not assignable to parameter of type
'number'.

// The JavaScript it compiles to: the types are simply removed
function area(width, height) {
  return width * height;
}
```

JavaScript is a high-level, dynamically and weakly typed, JIT-compiled, multi-paradigm language that runs on a single thread with an event loop. Created by Brendan Eich in 1995 and standardised as ECMAScript by TC39, it changed most with ES2015 and now evolves yearly. It is the behaviour layer of the web next to HTML and CSS, and through Node.js, Deno and Bun it also runs servers, tools, mobile and desktop apps. Engines (V8, SpiderMonkey, JavaScriptCore) execute the language; runtimes add APIs such as the DOM or the file system.

PRACTICE CHECKPOINT

Explain in your own words what **HTML**, **CSS** and **JavaScript** each do on a web page, and name **five apps or sites** built with JavaScript. (Suggested answer: HTML gives a page its structure and content, CSS its appearance and layout, and JavaScript its behaviour and interactivity. Five examples: Gmail, Google Maps, Visual Studio Code (Electron), Netflix's web interface (React), Slack desktop (Electron); Facebook, Discord and Figma are also correct answers.)

MODULE 02 · BEGINNER

Setup and your first program

Minimum duration **1 day** | Level **Beginner**

By the end of this module you can run JavaScript in the browser and in Node.js, and you know the basic syntax rules: statements, blocks, keywords, naming, comments, strict mode, the console and how to read error messages.

IN THIS MODULE

2.1 Tools
2.2 Running JavaScript
2.3 Hello World
2.4 Basic syntax
2.5 Comments
2.6 Strict mode
2.7 Console methods
2.8 Errors

2.1 Tools

JavaScript needs very little to get started: a browser to run code in web pages, an editor to write it, and Node.js to run it outside the browser. All three are free and available for Windows, macOS and Linux.

2.1.1 A modern browser (Chrome, Edge or Firefox)

Any up-to-date browser runs modern JavaScript. Chrome and Edge (both based on Chromium) and Firefox include excellent **developer tools** (DevTools): a console to run code, an inspector for the page, a debugger, network and performance panels. Keep your browser updated: new JavaScript features arrive with browser updates every few weeks.

2.1.2 VS Code with the Live Server and Prettier extensions

Visual Studio Code is the most popular editor for JavaScript (it is itself written in TypeScript and runs on Electron). Useful extensions for this course:

Extension	What it does
Live Server	serves your folder on a local web server and reloads the browser automatically every time you save
Prettier	formats code consistently on save (indentation, quotes, semicolons)
ESLint	highlights likely bugs and bad practices as you type (Module 19)
Error Lens	shows errors and warnings directly on the line where they occur

2.1.3 Installing Node.js (LTS) and checking node -v and npm -v

Download the **LTS** (Long-Term Support) version from nodejs.org, or install it with a version manager such as nvm or fnm, which makes switching versions easy. Node.js comes with **npm**, the package manager. Check the installation in a terminal:

Checking Node.js and npm (your version numbers may differ)

```
$ node -v
v22.22.2
$ npm -v
10.9.4
$ node                    # interactive REPL: type JavaScript, see results immediately
Welcome to Node.js v22.22.2.
Type ".help" for more information.
> 2 + 3
5
> "js".toUpperCase()
'JS'
> .exit
```

2.2 Running JavaScript

2.2.1 Browser DevTools console

Open DevTools with **F12** or **Ctrl+Shift+J** (Cmd+Option+J on a Mac) and choose the **Console** tab. Type an expression and press Enter: the result appears immediately. The console runs code in the context of the current page, so you can inspect and change it. It is the quickest place to experiment, and where your `console.log` messages and errors appear.

CODE TYPED INTO THE CONSOLE OF A PAGE TITLED "DEMO"

main.js

```
console.log(10 * 3);
console.log(document.title);
document.body.style.background = "#F8F4EA";         // change the page live
console.log("page background changed");
```

BROWSER CONSOLE

```
30
Demo
page background changed
```

2.2.2 Inline <script> in HTML

JavaScript can be written directly inside an HTML file between `<script>` and `</script>` tags. The browser runs it when it reaches the tag while reading the page. Inline scripts are fine for tiny experiments, but real projects keep JavaScript in separate files.

INDEX.HTML WITH AN INLINE SCRIPT HTML

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Inline script</title>
</head>
<body>
  <h1>My first JavaScript page</h1>
  <script>
    console.log("Hello from an inline script!");
    document.querySelector("h1").textContent += " (updated by JavaScript)";
  </script>
</body>
</html>
```

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```html
<h1>My first JavaScript page</h1>
```

WHAT THE BROWSER SHOWS main.js

```
console.log("Hello from an inline script!");
document.querySelector("h1").textContent += " (updated by JavaScript)";
```

BROWSER CONSOLE

```
Hello from an inline script!
--- page shows ---
My first JavaScript page (updated by JavaScript)
```

2.2.3 External script file <script src="app.js">

Normally JavaScript lives in its own `.js` file, linked with the `src` attribute. The same file can be used by many pages and cached by the browser, HTML and JavaScript stay separate, and the editor gives full support. A `<script>` tag with `src` must be empty.

LINKING AN EXTERNAL FILE HTML

```html
<!-- index.html -->
<body>
  <h1>External script</h1>
  <p id="out"></p>
  <script src="app.js"></script>
</body>
```

APP.JS LOADED FROM A SEPARATE FILE main.js

```
// app.js would contain exactly this; here it is loaded through <script src="app.js">
```

BROWSER CONSOLE

```
app.js is running
--- page shows ---
External script

Loaded from app.js at Mon Mar 09 2026
```

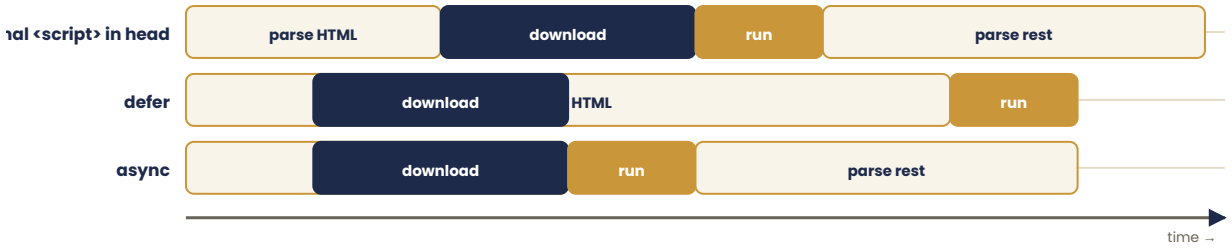
APP.JS

JavaScript

```
// app.js
const out = document.getElementById("out");
out.textContent = "Loaded from app.js at " + new Date(2026, 2, 9).toDateString();
console.log("app.js is running");
```

2.2.4 Script placement: head vs end of body, defer and async

When the browser meets a classic `<script>` tag, it **stops building the page**, downloads and runs the script, and only then continues. A script in the `<head>` therefore cannot see elements that appear later in the body, and a slow script delays the whole page. Three solutions: put scripts at the **end of the body**; add `defer`, which downloads in parallel and runs after the page is parsed, in order; or add `async`, which downloads in parallel and runs as soon as it arrives, in any order (good for independent scripts such as analytics). Module scripts (`type="module"`) are deferred automatically.



HOW NORMAL, DEFER AND ASYNC SCRIPTS AFFECT PAGE LOADING

A SCRIPT IN THE HEAD RUNS TOO EARLY

main.js

```
// This script is in the <head> WITHOUT defer: the <p> below does not exist yet
console.log("element found:", document.getElementById("msg"));
```

BROWSER CONSOLE

```
element found: null
with defer, element found: Hello
```

RECOMMENDED PLACEMENT

HTML

```
<head>
  <script src="app.js" defer></script>      <!-- recommended for most scripts -->
  <script src="analytics.js" async></script> <!-- independent script, order does not matter -->
  <script type="module" src="main.js"></script>  <!-- modules are deferred by default -->
</head>
```

2.2.5 Running files with node app.js

To run JavaScript outside the browser, save it in a file and pass the file name to `node`. Most examples in this book are run this way; their output boxes show exactly what Node.js printed. `node --watch app.js` re-runs the file automatically whenever you save it.

NODE HELLO.JS

hello.js

```
// hello.js
const name = "Learner";
const year = 2026;
console.log(`Hello, ${name}! Welcome to JavaScript in ${year}.`);
console.log("2 + 2 =", 2 + 2);
```

OUTPUT

```
Hello, Learner! Welcome to JavaScript in 2026.
2 + 2 = 4
```

Running files in a terminal

```
$ node hello.js
Hello, Learner! Welcome to JavaScript in 2026.
2 + 2 = 4
$ node --watch hello.js          # re-run on every save
$ node -e "console.log(7 * 6)"   # run a one-liner
42
```

2.2.6 Online playgrounds (CodePen, JSFiddle)

Online editors run HTML, CSS and JavaScript in the browser with nothing to install, and give you a link to share: ideal for quick experiments, asking for help and showing work. Popular ones: **CodePen** and **JSFiddle** (front-end snippets), **StackBlitz** and **CodeSandbox** (full projects, including Node.js and frameworks), and the **TypeScript Playground**.

2.3 Hello World

2.3.1 console.log("Hello, World!")

`console.log` prints values to the console: the browser's DevTools console or the terminal for Node.js. It accepts any number of values separated by commas and prints them with spaces between. It is the main tool for seeing what a program is doing.

CONSOLE.LOG

main.js

```
console.log("Hello, World!");
console.log("Several values:", 42, true, null, [1, 2, 3], { course: "JavaScript" });
console.log();                                  // an empty line
console.log("Total: " + 250 + " rupees");      // join strings with +
console.log(`Total: ${200 + 50} rupees`);       // or with a template literal
```

OUTPUT

```
Hello, World!
Several values: 42 true null [ 1, 2, 3 ] { course: 'JavaScript' }

Total: 250 rupees
Total: 250 rupees
```

2.3.2 alert(), prompt(), confirm()

Browsers provide three simple built-in dialog boxes. `alert(message)` shows a message and waits for OK. `prompt(message, default)` asks the user to type something and returns the text (or `null` if they press Cancel). `confirm(message)` asks a yes/no question and returns `true` or `false`. They **block** the page until

answered and look different in every browser, so real applications build their own dialogs in HTML; they remain handy while learning. They do not exist in Node.js.

THE THREE DIALOG BOXES main.js

```
const name = prompt("What is your name?", "Guest");
const age = Number(prompt("How old are you?"));
if (confirm(`Is ${name}, aged ${age}, correct?`)) {
  alert(`Welcome, ${name}! Next year you will be ${age + 1}.`);
} else {
  alert("Please reload and try again.");
}
console.log("typeof prompt result:", typeof name);
```

CONSOLE (user input in gold)

```
[prompt] What is your name? Asha
[prompt] How old are you? 20
[confirm] Is Asha, aged 20, correct? -> OK
[alert] Welcome, Asha! Next year you will be 21.
typeof prompt result: string
```

NOTE

The output lines starting with [prompt], [confirm] and [alert] stand for the dialog boxes; the gold text is what the user typed.

2.3.3 document.write() and why to avoid it

`document.write(html)` inserts HTML into the page at the position where the script runs. Old tutorials use it, but it is a bad practice: called after the page has loaded, it **erases the entire page**; it blocks parsing; it does not work with defer, async or modules; and browsers may ignore it on slow connections. Use DOM methods such as `textContent`, `append` or `insertAdjacentHTML` instead (Module 15).

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<h1>Title</h1><p>Original content</p>
```

DOCUMENT.WRITE AFTER LOADING WIPES THE PAGE main.js

```
document.write("<p>written while the page loads</p>");   // works only during loading
setTimeout(() => {
  document.write("<p>written later</p>");                 // replaces EVERYTHING on the page
  console.log("paragraphs left:", document.querySelectorAll("p").length);
}, 50);
```

BROWSER CONSOLE

```
paragraphs left: 1
--- page shows ---
written later
```

2.4 Basic syntax

2.4.1 Statements and semicolons (and automatic semicolon insertion)

A program is a sequence of **statements**, each an instruction such as a declaration, an assignment or a function call, normally written one per line and ended with a semicolon. If you leave semicolons out,

Automatic Semicolon Insertion (ASI) adds them at line breaks in most cases, so many code bases omit them. ASI has traps, though: a line break after `return` ends the statement, and a line starting with `(` or `[` continues the previous one. This book uses semicolons consistently, as most style guides recommend.

ASI AND ITS MOST FAMOUS TRAP main.js

```
let a = 1; let b = 2           // several statements on one line need semicolons between them
console.log(a + b)              // no semicolon: ASI inserts one at the line break

function getConfig() {
  return                        // ASI inserts a semicolon right here!
  { debug: true };
}
console.log("getConfig() returns:", getConfig());
```

OUTPUT

```
3
getConfig() returns: undefined
```

2.4.2 Case sensitivity

JavaScript is **case-sensitive**: `name`, `Name` and `NAME` are three different identifiers, and keywords must be lowercase (`let`, not `Let`). Built-in names have exact capitalisation: `console.log`, `Math.max`, `document.getElementById`, `JSON.parse`.

DIFFERENT CASE, DIFFERENT NAME main.js

```
let course = "JavaScript";
let Course = "Python";
console.log(course, Course);
try {
  console.Log("wrong case");
} catch (e) {
  console.log(e.name + ": " + e.message);
}
```

OUTPUT

```
JavaScript Python
TypeError: console.Log is not a function
```

2.4.3 Code blocks { }

Curly braces group statements into a **block**, used as the body of `if`, loops and functions. Blocks also create a **scope** for variables declared with `let` and `const` (section 3.2). Indent the contents of a block (usually two spaces in JavaScript) so the structure is visible.

BLOCKS GROUP STATEMENTS

main.js

```
let marks = 82;
if (marks >= 40) {
  const result = "pass";                // exists only inside this block
  console.log(`marks ${marks}: ${result}`);
  if (marks >= 75) {
    console.log("distinction");
  }
}
console.log(typeof result);             // result is not visible here
```

OUTPUT

```
marks 82: pass
distinction
undefined
```

2.4.4 Reserved keywords

Reserved words have special meaning and cannot be used as variable or function names.

Group	Keywords
Declarations	`var let const function class`
Control flow	`if else switch case default for while do break continue return`
Errors	`try catch finally throw`
Objects and types	`new this super extends typeof instanceof in of delete void`
Values	`true false null` (and the property `undefined`)
Modules and async	`import export async await yield`
Other / reserved	`debugger with enum implements interface package private protected public static`

USING A KEYWORD AS A NAME IS A SYNTAX ERROR

main.js

```
try {
  eval("let class = 5;");          // eval runs code from a string: used here only to show the error
} catch (e) {
  console.log(`${e.name}: ${e.message}`);
}
```

OUTPUT

```
SyntaxError: Unexpected token 'class'
```

2.4.5 Identifiers and naming conventions (camelCase, PascalCase, UPPER_SNAKE_CASE)

An **identifier** (a name for a variable, function or class) may contain letters, digits, `$` and `_`, must not start with a digit, and must not be a reserved word. Choose descriptive names (`totalPrice`, not `tp`). The community conventions are:

Convention	Used for	Examples
camelCase	variables, functions, methods, properties	`firstName`, `getTotal()`, `isLoggedIn`
PascalCase	classes and constructor functions, React components	`Student`, `ShoppingCart`, `UserCard`
UPPER_SNAKE_CASE	constants known before the program runs	`MAX_USERS`, `API_URL`, `TAX_RATE`
kebab-case	file names, CSS classes, HTML attributes (not identifiers)	`user-profile.js`, `data-user-id`
_leading underscore	by convention "internal" (true privacy uses #fields, Module 13)	`_cache`

VALID IDENTIFIERS FOLLOWING THE CONVENTIONS main.js

```
const MAX_SEATS = 60;
let studentName = "Asha";
let $price = 499, _count = 3, café = "ok";       // $, _ and Unicode letters are allowed
function calculateTotal(price, qty) { return price * qty; }
class StudentRecord {}
console.log(MAX_SEATS, studentName, calculateTotal($price, _count), typeof StudentRecord, café);
```

OUTPUT
```
60 Asha 1497 function ok
```

2.5 Comments

2.5.1 Single-line // and multi-line /* */

Comments are ignored by the engine and exist for human readers. `//` comments out the rest of a line; `/* ... */` can span several lines. Good comments explain **why** something is done, not what the code obviously does. In VS Code, Ctrl+/ toggles comments on the selected lines.

SINGLE-LINE AND MULTI-LINE COMMENTS main.js

```
// Calculate the final price including GST
const price = 1000;
const GST_RATE = 0.18;   // 18% for this product category
/*
  Discounts are applied before tax, as required by the
  shop's billing policy.
*/
const discount = 100;
console.log((price - discount) * (1 + GST_RATE));
```

OUTPUT
```
1062
```

2.5.2 JSDoc comments /** */

A comment starting with `/**` placed before a function or class is a **JSDoc** comment. Tags such as `@param`, `@returns` and `@example` document the code; editors show the text on hover and use the types for autocompletion and checking, and tools can generate documentation websites from them.

DOCUMENTING A FUNCTION WITH JSDOC main.js

```
/**
 * Calculates simple interest.
 * @param {number} principal - the amount invested
 * @param {number} rate - the yearly interest rate in percent
 * @param {number} years - the duration in years
 * @returns {number} the interest earned
 * @example simpleInterest(10000, 7.5, 2); // 1500
 */
function simpleInterest(principal, rate, years) {
  return (principal * rate * years) / 100;
}
console.log(simpleInterest(10000, 7.5, 2));
```

OUTPUT

```
1500
```

2.6 Strict mode

2.6.1 "use strict" and what it changes

Strict mode, enabled by the directive `"use strict";` at the top of a file or function, turns several silent mistakes into errors and disables some confusing old features. Modules (`import` / `export`, Module 19) and class bodies are always strict, so most modern code runs in strict mode automatically. Use it in every classic script.

Without strict mode	With strict mode
assigning to an undeclared variable silently creates a global	ReferenceError
assigning to a read-only property fails silently	TypeError
duplicate parameter names allowed	SyntaxError
`this` is the global object in plain function calls	`this` is `undefined`
`with` statement and octal literals like 010 allowed	SyntaxError
deleting a variable fails silently	SyntaxError

STRICT MODE CATCHES ACCIDENTAL GLOBALS

main.js

```
function sloppy() {
  totl = 100;                      // typo: silently creates a global variable
  return typeof globalThis.totl;
}
console.log("sloppy mode:", sloppy());

function strict() {
  "use strict";
  try {
    totalCount = 100;              // same typo
  } catch (e) {
    return `${e.name}: ${e.message}`;
  }
}
console.log("strict mode:", strict());
```

OUTPUT

```
sloppy mode: number
strict mode: ReferenceError: totalCount is not defined
```

2.7 Console methods

2.7.1 console.log, info, warn, error, table, group, time

The `console` object offers more than `log`. Browsers display warnings and errors with icons and colours and let you filter by level; in Node.js, `warn` and `error` go to the standard error stream.

Method	Use
`console.log / info / debug`	general messages (info and debug are filterable levels)
`console.warn`	warnings: yellow in the browser
`console.error`	errors: red, with a stack trace in browsers
`console.table(data)`	arrays and objects as a table
`console.group / groupEnd`	indent related messages (groupCollapsed starts folded)
`console.time / timeEnd`	measure elapsed time between two calls with the same label
`console.count(label)`	count how many times a line runs
`console.assert(cond, msg)`	print msg only if the condition is false
`console.dir(obj)`	show an object's properties interactively

CONSOLE METHODS IN NODE.JS (THE TIMING VARIES) main.js

```
console.info("server started");
console.warn("disk space is low");
console.error("could not connect to the database");
console.table([{ name: "Asha", marks: 91 }, { name: "Ravi", marks: 78 }]);
console.group("Order #101");
console.log("item: notebook");
console.log("qty: 3");
console.groupEnd();
for (const fruit of ["apple", "kiwi", "apple"]) console.count(fruit);
console.assert(2 + 2 === 5, "maths is broken!");
console.time("loop");
let s = 0; for (let i = 0; i < 1e6; i++) s += i;
console.timeEnd("loop");
```

OUTPUT

```
server started
disk space is low
could not connect to the database
┌─────────┬─────────┬─────────┐
│ (index) │ name    │ marks   │
├─────────┼─────────┼─────────┤
│ 0       │ 'Asha'  │ 91      │
│ 1       │ 'Ravi'  │ 78      │
└─────────┴─────────┴─────────┘
Order #101
  item: notebook
  qty: 3
apple: 1
kiwi: 1
apple: 2
Assertion failed: maths is broken!
loop: 13.783ms
```

NOTE

The time printed by `console.timeEnd` differs on every run. In Node.js, warn, error and assert messages are written to the error stream, shown here together with normal output; in a browser they appear with yellow and red styling.

2.8 Errors

2.8.1 SyntaxError, ReferenceError, TypeError

When something goes wrong, JavaScript **throws an error object** describing it. Three kinds account for most beginner errors.

Error	When	Example
SyntaxError	the code is not valid JavaScript; detected when the script is parsed, so nothing in it runs	missing bracket or quote, `let let = 1`
ReferenceError	using a name that does not exist (or a let/const before its declaration)	a typo such as `conosle.log`
TypeError	a value is used in a way its type does not allow	calling something that is not a function, reading a property of `undefined`

Error	When	Example
RangeError	a number is outside the allowed range	`new Array(-1)`, too much recursion
URIError, EvalError	rare: bad URI encoding, legacy eval issues	`decodeURIComponent("%")`

TYPICAL ERROR MESSAGES

main.js

```
const tests = [
  () => JSON.parse("{ bad json }"),
  () => undefinedVariable + 1,
  () => { const user = undefined; return user.name; },
  () => { const n = 5; n(); },
  () => { const c = 1; c = 2; },
  () => new Array(-1),
];
for (const t of tests) {
  try { t(); } catch (e) { console.log(`${e.name}: ${e.message}`); }
}
```

OUTPUT

```
SyntaxError: Expected property name or '}' in JSON at position 2 (line 1 column 3)
ReferenceError: undefinedVariable is not defined
TypeError: Cannot read properties of undefined (reading 'name')
TypeError: n is not a function
TypeError: Assignment to constant variable.
RangeError: Invalid array length
```

2.8.2 Reading errors in the console

An uncaught error stops the current script and prints a message with three useful parts: the **error type**, the **message**, and the **location** (file, line and column), followed by a **stack trace** listing the functions that were running. In the browser, click the file link to jump to the line in the Sources panel. Read the message carefully, go to the first line of the stack that is in your own code, and check the values used on that line.

AN UNCAUGHT TYPEERROR WITH ITS STACK TRACE

shop.js

```
function getDiscount(customer) {
  return customer.membership.level * 5;      // fails when membership is missing
}
function checkout(customer) {
  return 1000 - getDiscount(customer);
}
console.log(checkout({ name: "Asha", membership: { level: 2 } }));
console.log(checkout({ name: "Ravi" }));        // no membership
console.log("this line never runs");
```

OUTPUT

```
990
shop.js:2
  return customer.membership.level * 5;      // fails when membership is missing
                             ^

TypeError: Cannot read properties of undefined (reading 'level')
    at getDiscount (shop.js:2:30)
    at checkout (shop.js:5:17)
    at Object.<anonymous> (shop.js:8:13)
```

Reading it from the top: the TypeError says `membership` was `undefined` when the code tried to read `level`; the first stack line points to `getDiscount` in shop.js line 2; below it, `checkout` called it from line 5, and that was called from line 8. The fix is to handle customers without a membership, for example with `customer.membership?.level ?? 0` (Module 4).

Run JavaScript in the browser console, in HTML pages with inline or external scripts (use defer, or put scripts at the end of the body), or with node file.js. console.log prints values; alert, prompt and confirm are simple browser dialogs; avoid document.write. Statements end with semicolons (ASI adds most of them, with traps), the language is case-sensitive, blocks use braces, keywords are reserved, and names follow camelCase, PascalCase and UPPER_SNAKE_CASE conventions. Write // and /* */ comments and JSDoc for functions, enable strict mode, use the richer console methods, and read error type, message, location and stack trace to fix bugs.

PRACTICE CHECKPOINT

Create an **HTML page with an external script** that asks the user's name with **prompt** and greets them **in the console and on the page**, then run a **separate script with Node.js**.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```html
<h1 id="greeting">Loading...</h1>
```

GREET.JS RUNNING IN THE PAGE main.js

```
// greet.js (linked from index.html with a deferred script tag)
"use strict";
const name = prompt("What is your name?")?.trim() || "friend";
const hour = 10;                                    // new Date().getHours() in a real page
const part = hour < 12 ? "morning" : hour < 17 ? "afternoon" : "evening";
const message = `Good ${part}, ${name}! Welcome to JavaScript.`;
console.log(message);
document.getElementById("greeting").textContent = message;
```

CONSOLE (user input in gold)

```
[prompt] What is your name? Meera
Good morning, Meera! Welcome to JavaScript.
--- page shows ---
Good morning, Meera! Welcome to JavaScript.
```

INDEX.HTML HTML

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Greeting</title>
  <script src="greet.js" defer></script>
</head>
<body>
  <h1 id="greeting">Loading...</h1>
</body>
</html>
```

NODE NODE-GREET.JS MEERA

node-greet.js

```
// node-greet.js  (run with: node node-greet.js)
const name = process.argv[2] ?? "friend";          // name passed on the command line
console.log(`Hello, ${name}! This script runs in Node.js ${process.versions.node.split(".")[0]}, not
in a browser.`);
console.log(`typeof document here: ${typeof document}`);
```

OUTPUT

```
Hello, Meera! This script runs in Node.js 22, not in a browser.
typeof document here: undefined
```

MODULE 03 · BEGINNER

Variables and data types

Minimum duration **2 days** | Level **Beginner**

JavaScript variables hold values of any type and can change type at run time. This module covers declaring variables with var, let and const, an introduction to scope and hoisting, all eight data types, how primitives differ from objects, and how JavaScript converts between types, explicitly and implicitly.

IN THIS MODULE

3.1 Declaring variables
3.2 Scope (introduction)
3.3 Data types overview
3.4 Primitive types in depth
3.5 Primitives vs references
3.6 Type conversion and coercion

3.1 Declaring variables

A **variable** is a named container for a value. In JavaScript the container has no fixed type: the value decides the type. Variables are declared with one of three keywords. Modern code uses `const` by default and `let` when the value must change; `var` belongs to older code.



THE THREE WAYS TO DECLARE A VARIABLE

3.1.1 var

`var` was the only declaration keyword until 2015. A var variable belongs to the whole enclosing **function** (or the global scope), not to the block it is written in; it can be declared again without error; and it is **hoisted**: usable, with the value `undefined`, before the line that declares it. These behaviours cause bugs, which is why let and const were introduced.

VAR: FUNCTION-SCOPED, RE-DECLARABLE, HOISTED

main.js

```
var city = "Pune";
var city = "Delhi";              // re-declaration allowed, silently
console.log(city);

if (true) {
  var inside = "visible outside the block";
}
console.log(inside);             // var ignores block boundaries

console.log(early);              // hoisted: undefined, no error
var early = 5;
```

OUTPUT

```
Delhi
visible outside the block
undefined
```

3.1.2 let

`let` (ES2015) declares a variable whose value can be changed later. It is **block-scoped**: it exists only inside the nearest `{ }`. Declaring the same name twice in one scope is a SyntaxError, and using it before its declaration is a ReferenceError.

LET: BLOCK-SCOPED AND REASSIGNABLE

main.js

```
let score = 10;
score = score + 5;               // reassignment is fine
score += 5;
console.log(score);

let total;                       // declared without a value: undefined
console.log(total);

for (let i = 0; i < 3; i++) {    // i exists only inside the loop
  console.log("i =", i);
}
console.log(typeof i);
```

OUTPUT

```
20
undefined
i = 0
i = 1
i = 2
undefined
```

3.1.3 const (constant binding, not constant value)

`const` declares a variable that **cannot be reassigned**; it must be initialised immediately. Note what is constant: the **binding** (which value the name refers to), not the value itself. If the value is an object or array, its contents can still change. Use `Object.freeze` (Module 9) to make an object itself unchangeable.

CONST PREVENTS REASSIGNMENT, NOT MUTATION

main.js

```
const PI = 3.14159;
const user = { name: "Asha", roles: ["student"] };

user.name = "Asha Rao";          // allowed: changing the object, not the binding
user.roles.push("mentor");
console.log(user);

try {
  user = { name: "Ravi" };       // not allowed: reassigning the binding
} catch (e) {
  console.log(`${e.name}: ${e.message}`);
}
try {
  eval("const RATE;");           // const must be initialised
} catch (e) {
  console.log(`${e.name}: ${e.message}`);
}
```

OUTPUT

```
{ name: 'Asha Rao', roles: [ 'student', 'mentor' ] }
TypeError: Assignment to constant variable.
SyntaxError: Missing initializer in const declaration
```

3.1.4 var vs let vs const comparison

	`var`	`let`	`const`
Scope	function (or global)	block	block
Reassign	yes	yes	no
Re-declare in same scope	yes	no (SyntaxError)	no (SyntaxError)
Hoisted	yes, initialised to undefined	yes, but in the Temporal Dead Zone	yes, but in the Temporal Dead Zone
Must be initialised	no	no	yes
Global declaration creates window property	yes	no	no
Recommendation	avoid in new code	when the value changes	default choice

TIP

Start every declaration with `const`. Change it to `let` only when you find you need to reassign the variable. This makes code easier to read: a reader knows that a const name always refers to the same value.

3.1.5 Naming rules

Variable names follow the identifier rules of section 2.4.5: letters, digits, `$` and `_`; not starting with a digit; not a reserved word; case-sensitive. Good names describe the content: nouns for values (`price`, `studentList`), `is` / `has` / `can` prefixes for booleans (`isValid`, `hasAccess`), and plural names for arrays.

VALID AND INVALID NAMES

main.js

```
const firstName = "Meera";        // camelCase
const isLoggedIn = true;          // boolean prefix
const students = ["Asha", "Ravi"];// plural for a collection
const MAX_RETRIES = 3;            // fixed configuration constant
const _internal = 1, $el = "x", user2 = "ok";
const tests = ["let 2user = 1", "let my-name = 1", "let new = 1", "let user name = 1"];
for (const t of tests) {
  try { eval(t); } catch (e) { console.log(`${t.padEnd(18)} -> ${e.name}`); }
}
```

OUTPUT

```
let 2user = 1      -> SyntaxError
let my-name = 1    -> SyntaxError
let new = 1        -> SyntaxError
let user name = 1  -> SyntaxError
```

3.2 Scope (introduction)

Scope is the region of the program where a variable can be used. Module 11 covers scope and closures in depth; the basics are needed now.

3.2.1 Global, function and block scope

A variable declared outside any function or block has **global scope** and is visible everywhere. A variable declared inside a function has **function scope**: visible only inside that function. A `let` or `const` declared inside a block (if, for, or plain braces) has **block scope**. Inner scopes can read variables of outer scopes, never the other way round.

NESTED SCOPES

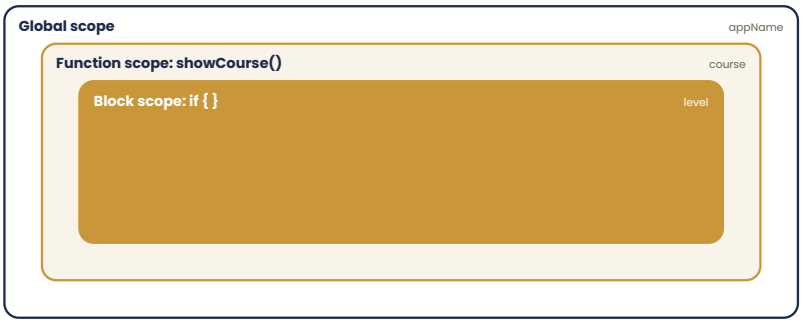
main.js

```
const appName = "LBF";                // global scope

function showCourse() {
  const course = "JavaScript";        // function scope
  if (course.length > 5) {
    const level = "Beginner";         // block scope
    console.log(appName, course, level);   // inner code sees outer variables
  }
  console.log(typeof level);          // the block variable is gone
}
showCourse();
console.log(typeof course);           // the function variable is not visible here
```

OUTPUT

```
LBF JavaScript Beginner
undefined
undefined
```



INNER SCOPES CAN SEE OUTER VARIABLES; OUTER SCOPES CANNOT SEE INNER ONES

3.2.2 Re-declaration and re-assignment rules

Within one scope, `let` and `const` names can be declared only once. A variable in an inner scope may, however, have the same name as one in an outer scope: it **shadows** the outer one inside that block. Shadowing is legal but can be confusing, so use it sparingly.

SHADOWING VERSUS RE-DECLARING main.js

```
let level = "global";
{
  let level = "inner block";     // a NEW variable that shadows the outer one
  console.log(level);
}
console.log(level);              // the outer variable is unchanged

let count = 1;
count = 2;                        // reassignment: fine for let
try { eval("let count = 3;"); } catch (e) { console.log(e.name, "-", e.message); }
```

OUTPUT

```
inner block
global
```

NOTE

The eval call above runs in the same scope as count. Re-declaring with let is reported as "Identifier 'count' has already been declared"; in a normal file this error is reported before any code runs.

3.2.3 Hoisting and the Temporal Dead Zone (introduced; detailed in Module 11)

Before running a scope, the engine registers all its declarations: this is called **hoisting**, as if declarations were moved to the top. A `var` is registered and initialised to `undefined`, so using it early gives undefined. A `let` or `const` is registered but **not initialised**; from the start of the block until the declaration line, it is in the **Temporal Dead Zone** (TDZ), and touching it throws a ReferenceError. Function declarations are hoisted completely, so they can be called before they appear.

HOISTING WITH VAR, LET AND FUNCTIONS

main.js

```
console.log(sayHi());                // function declarations are fully hoisted
function sayHi() { return "hi from a hoisted function"; }

console.log(a);                      // var: undefined
var a = 1;

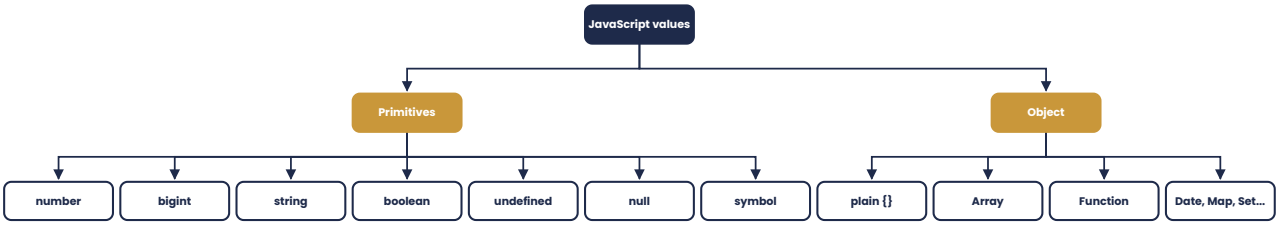
try {
  console.log(b);                    // let: in the Temporal Dead Zone
} catch (e) {
  console.log(`${e.name}: ${e.message}`);
}
let b = 2;
console.log(b);
```

OUTPUT

```
hi from a hoisted function
undefined
ReferenceError: Cannot access 'b' before initialization
2
```

3.3 Data types overview

Every value in JavaScript has one of **eight types**: seven **primitive** types and the **object** type. Primitives are simple, immutable values; objects are collections of properties and include arrays, functions, dates and everything else that is not a primitive.



THE EIGHT DATA TYPES

3.3.1 Primitive types: number, bigint, string, boolean, undefined, null, symbol

Type	Example values	Used for
number	`42`, `-3.5`, `1e6`, `Infinity`, `NaN`	all ordinary numbers, integers and decimals alike
bigint	`123n`, `2n ** 64n`	integers larger than $2^{53} - 1$
string	`"hi"`, `'hi'`, `` `hi ${name}` ``	text
boolean	`true`, `false`	yes/no conditions
undefined	`undefined`	a variable or property that has no value yet
null	`null`	an intentional "no value"
symbol	`Symbol("id")`	unique keys for object properties

3.3.2 Reference type: object (including arrays and functions)

Everything else is an **object**: a collection of key-value pairs. Arrays are objects with numeric keys and a length; functions are callable objects; dates, regular expressions, maps and sets are built-in object types. Objects are handled **by reference** (section 3.5).

DIFFERENT KINDS OF OBJECTS main.js

```
const student = { name: "Asha", marks: 91 };   // plain object
const marks = [91, 78, 85];                     // array (an object)
function add(a, b) { return a + b; }            // function (a callable object)
const today = new Date(2026, 2, 9);             // date object
add.description = "adds two numbers";           // functions can even have properties
console.log(student.name, marks.length, add(2, 3), today.getFullYear(), add.description);
```

OUTPUT

```
Asha 3 5 2026 adds two numbers
```

3.3.3 typeof operator and its quirks (typeof null)

The `typeof` operator returns a string naming a value's type. Two results are surprising: `typeof null` is `"object"` (a bug from the first version of JavaScript that can never be fixed without breaking the web), and functions report `"function"` although they are objects. Arrays report `"object"`; use `Array.isArray()` to detect them.

TYPEOF FOR EVERY KIND OF VALUE main.js

```
const values = [42, 3.14, 10n, "text", true, undefined, null, Symbol("id"), {}, [1, 2], function ()
{}, new Date(0)];
for (const v of values) {
  const shown = typeof v === "symbol" ? v.toString() : typeof v === "bigint" ? `${v}n` :
JSON.stringify(v) ?? String(v);
  console.log(`${shown.padEnd(26)} typeof -> ${typeof v}`);
}
console.log("Array.isArray([1, 2]):", Array.isArray([1, 2]));
console.log("null check:", null === null);
```

OUTPUT

```
42                         typeof -> number
3.14                       typeof -> number
10n                        typeof -> bigint
"text"                     typeof -> string
true                       typeof -> boolean
undefined                  typeof -> undefined
null                       typeof -> object
Symbol(id)                 typeof -> symbol
{}                         typeof -> object
[1,2]                      typeof -> object
function () {}             typeof -> function
"1970-01-01T00:00:00.000Z" typeof -> object
Array.isArray([1, 2]): true
null check: true
```

3.4 Primitive types in depth

3.4.1 Number: integers and floats in one type, Infinity, NaN

JavaScript has a single `number` type for both integers and decimals, stored as a 64-bit IEEE 754 double-precision floating-point value. There are special values: `Infinity` and `-Infinity` (results that overflow, or division by zero) and `NaN`, "Not a Number", the result of an invalid numeric operation. NaN is the only value that is not equal to itself. Numbers can be written in decimal, scientific (`1.5e3`), hexadecimal (`0xff`), octal (`0o17`) and binary (`0b1010`) notation, with `_` as a digit separator.

NUMBERS, INFINITY AND NAN main.js

```
console.log(7 / 2, 10 / 5, typeof 42, typeof 4.2);   // one type for both
console.log(1 / 0, -1 / 0, 0 / 0);                   // Infinity, -Infinity, NaN
console.log("abc" * 3, Math.sqrt(-1));               // invalid operations give NaN
console.log(NaN === NaN, Number.isNaN(NaN));         // NaN is not equal to itself
console.log(1.5e3, 0xff, 0o17, 0b1010, 1_000_000);   // number literals
console.log(Number.MAX_VALUE, Number.MIN_VALUE, Number.EPSILON);
```

OUTPUT

```
3.5 2 number number
Infinity -Infinity NaN
NaN NaN
false true
1500 255 15 10 1000000
1.7976931348623157e+308 5e-324 2.220446049250313e-16
```

3.4.2 Floating-point precision (0.1 + 0.2)

Binary floating point cannot represent most decimal fractions exactly, just as 1/3 cannot be written exactly in decimal. 0.1 is stored as the nearest binary fraction, so small rounding errors appear: `0.1 + 0.2` is `0.30000000000000004`. This is not a JavaScript bug; every language using IEEE 754 doubles behaves the same. Compare decimals with a tolerance, round for display, and for money calculate in whole **paise or cents** (integers).

FLOATING-POINT SURPRISES AND HOW TO HANDLE THEM main.js

```
console.log(0.1 + 0.2);
console.log(0.1 + 0.2 === 0.3);
console.log(Math.abs(0.1 + 0.2 - 0.3) < Number.EPSILON);    // compare with a tolerance
console.log((0.1 + 0.2).toFixed(2));                        // round for display
const pricePaise = 1999 + 2999;                              // money in integer paise
console.log(`Rs ${(pricePaise / 100).toFixed(2)}`);
console.log(1.005 * 1000, 9.95 * 100);
```

OUTPUT

```
0.30000000000000004
false
true
0.30
Rs 49.98
1004.9999999999999 994.9999999999999
```

3.4.3 Number.MAX_SAFE_INTEGER and BigInt 123n

A double has 53 bits of precision, so integers are exact only up to `Number.MAX_SAFE_INTEGER` = $2^{53} - 1$ = 9,007,199,254,740,991. Beyond that, consecutive integers can no longer be told apart. **BigInt** (ES2020)

represents integers of any size: write `n` after the literal or call `BigInt()`. BigInts cannot be mixed with numbers in arithmetic without explicit conversion, and they have no decimals.

SAFE INTEGERS AND BIGINT main.js

```
console.log(Number.MAX_SAFE_INTEGER);
console.log(9007199254740992 === 9007199254740993);     // beyond the safe range: equal!
console.log(Number.isSafeInteger(2 ** 53));

const big = 9007199254740993n;
console.log(big + 2n, typeof big);
console.log(2n ** 100n);
console.log(7n / 2n);                                    // integer division: decimals are dropped
try { console.log(big + 1); } catch (e) { console.log(`${e.name}: ${e.message}`); }
console.log(Number(10n) + 1, BigInt(25) * 4n);           // explicit conversion
```

OUTPUT

```
9007199254740991
true
false
9007199254740995n bigint
126765060022822940149670320537 6n
3n
TypeError: Cannot mix BigInt and other types, use explicit conversions
11 100n
```

3.4.4 String: single, double and template literals

Strings hold text and can be written with single quotes, double quotes or **backticks**. Single and double quotes are equivalent (pick one style and be consistent). Backticks create **template literals**, which allow embedded expressions `${...}` and multiple lines (Module 8 covers strings in depth).

THREE KINDS OF STRING LITERAL main.js

```
const single = 'It\'s a single-quoted string';
const double = "She said \"hello\"";
const name = "Asha", marks = 91;
const template = `${name} scored ${marks} (${marks >= 85 ? "distinction" : "pass"})`;
const multiLine = `Line one
Line two`;
console.log(single);
console.log(double);
console.log(template);
console.log(multiLine);
console.log(typeof template, template.length);
```

OUTPUT

```
It's a single-quoted string
She said "hello"
Asha scored 91 (distinction)
Line one
Line two
string 28
```

3.4.5 Boolean

The boolean type has exactly two values, `true` and `false`. Booleans result from comparisons and logical operators and drive conditions and loops. `Boolean(value)` converts any value to a boolean (section 3.6.3).

BOOLEAN VALUES

main.js

```
const isAdult = 21 >= 18;
const hasTicket = false;
console.log(isAdult, hasTicket, typeof isAdult);
console.log(isAdult && !hasTicket ? "can buy a ticket" : "cannot enter");
console.log(Boolean(1), Boolean(0), Boolean("text"), Boolean(""));
```

OUTPUT

```
true false boolean
can buy a ticket
true false true false
```

3.4.6 undefined vs null

Both mean "no value", with a difference in intent. `undefined` is what JavaScript gives you automatically: a declared variable with no value, a missing object property, a missing function argument, a function without return. `null` is a value **you** assign deliberately to say "empty on purpose", for example "no user is logged in". `null == undefined` is true, but `null === undefined` is false.

UNDEFINED AND NULL

main.js

```
let notAssigned;
const user = { name: "Ravi" };
function noReturn() {}
function show(x) { return x; }
console.log(notAssigned, user.email, noReturn(), show());     // all undefined

let selectedCourse = null;                                     // deliberately empty
console.log(selectedCourse, typeof selectedCourse, typeof undefined);
console.log(null == undefined, null === undefined);
console.log(Number(null), Number(undefined));                  // 0 versus NaN
```

OUTPUT

```
undefined undefined undefined undefined
null object undefined
true false
0 NaN
```

3.4.7 Symbol (introduced; detailed in Module 12)

A **symbol** (ES2015) is a unique, immutable identifier: every call to `Symbol()` creates a new value different from all others, even with the same description. Symbols are used as object property keys that cannot clash with other keys, and the language uses well-known symbols such as `Symbol.iterator` to define behaviour.

SYMBOLS ARE ALWAYS UNIQUE main.js

```
const id1 = Symbol("id");
const id2 = Symbol("id");
console.log(id1 === id2, typeof id1, id1.description);

const user = { name: "Asha", [id1]: 101 };      // a symbol as a property key
console.log(user[id1], Object.keys(user));      // symbol keys are hidden from Object.keys
console.log(JSON.stringify(user));
```

OUTPUT

```
false symbol id
101 [ 'name' ]
{"name":"Asha"}
```

3.5 Primitives vs references

3.5.1 Copy by value vs copy by reference

Assigning a primitive copies the **value**: the two variables are independent. Assigning an object copies the **reference** (the address of the object): both variables then point to the same object, so a change made through one is visible through the other. The same rule applies when passing arguments to functions. Comparing objects with `===` compares references, not contents.

VALUE COPIES VERSUS SHARED REFERENCES main.js

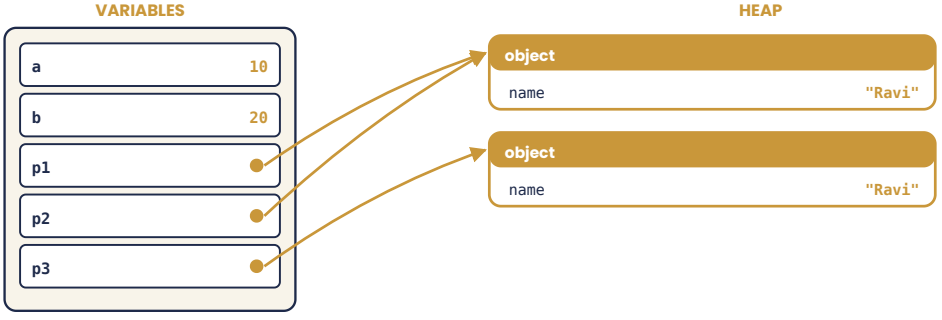
```
let a = 10;
let b = a;          // copy of the value
b = 20;
console.log(a, b);

const p1 = { name: "Asha" };
const p2 = p1;      // copy of the reference: same object
p2.name = "Ravi";
console.log(p1.name, p2.name, p1 === p2);

const p3 = { name: "Ravi" };
console.log(p1 === p3);           // different objects with equal contents are not ===
```

OUTPUT

```
10 20
Ravi Ravi true
false
```



P1 AND P2 REFER TO THE SAME OBJECT; P3 TO ANOTHER WITH EQUAL CONTENTS

3.5.2 Immutability of primitives

Primitive values can never be changed. String methods such as `toUpperCase` return a **new** string; writing to `str[0]` has no effect (and throws in strict mode). What can change is which value a `let` variable holds. Objects, by contrast, are mutable.

PRIMITIVES CANNOT BE MODIFIED main.js

```
"use strict";
let word = "hello";
word.toUpperCase();              // returns a new string; word is unchanged
console.log(word);
word = word.toUpperCase();       // reassign to keep the new value
console.log(word);
try {
  word[0] = "J";                 // cannot modify a primitive
} catch (e) {
  console.log(`${e.name}: ${e.message}`);
}
```

OUTPUT

```
hello
HELLO
TypeError: Cannot assign to read only property '0' of string 'HELLO'
```

3.6 Type conversion and coercion

JavaScript often needs a value of a different type: a number from text typed by the user, a string for display, a boolean for a condition. **Conversion** can be **explicit**, when you call a function, or **implicit** (**coercion**), when an operator does it automatically.

3.6.1 Explicit conversion: Number(), String(), Boolean(), parseInt(), parseFloat()

`Number(value)` converts the whole value and gives `NaN` if it is not a valid number (empty or whitespace-only strings give 0). `parseInt(text, radix)` and `parseFloat(text)` read digits from the **start** of a string and stop at the first invalid character, which suits values like "42px". Always pass the radix (usually 10) to parseInt. `String(value)` and template literals convert to text; `Boolean(value)` converts to true or false.

EXPLICIT CONVERSIONS

main.js

```
const inputs = ["42", "3.14", "  7  ", "", "12px", "abc", "0x1A", null, undefined, true, [], [5]];
for (const v of inputs) {
  const label = JSON.stringify(v) ?? "undefined";
  console.log(`${label.padEnd(10)} Number: ${String(Number(v)).padEnd(5)} parseInt:
${String(parseInt(v, 10)).padEnd(4)} parseFloat: ${parseFloat(v)}`);
}
console.log(String(123), String(null), String([1, 2]), `${true}`);
console.log(parseInt("ff", 16), parseInt("1010", 2), (255).toString(16));
```

OUTPUT

```
"42"       Number: 42    parseInt: 42   parseFloat: 42
"3.14"     Number: 3.14  parseInt: 3    parseFloat: 3.14
"  7  "    Number: 7     parseInt: 7    parseFloat: 7
""         Number: 0     parseInt: NaN  parseFloat: NaN
"12px"     Number: NaN   parseInt: 12   parseFloat: 12
"abc"      Number: NaN   parseInt: NaN  parseFloat: NaN
"0x1A"     Number: 26    parseInt: 0    parseFloat: 0
null       Number: 0     parseInt: NaN  parseFloat: NaN
undefined  Number: NaN   parseInt: NaN  parseFloat: NaN
true       Number: 1     parseInt: NaN  parseFloat: NaN
[]         Number: 0     parseInt: NaN  parseFloat: NaN
[5]        Number: 5     parseInt: 5    parseFloat: 5
123 null 1,2 true
255 10 ff
```

3.6.2 Implicit coercion: "5" + 2, "5" - 2

Operators coerce their operands. The `+` operator is special: if either operand is a string, it performs **string concatenation**. All other arithmetic operators (`-`, `*`, `/`, `%`, `**`) convert both operands to numbers. Comparisons with `==` also coerce (Module 4). Objects are first converted to primitives, usually to strings.

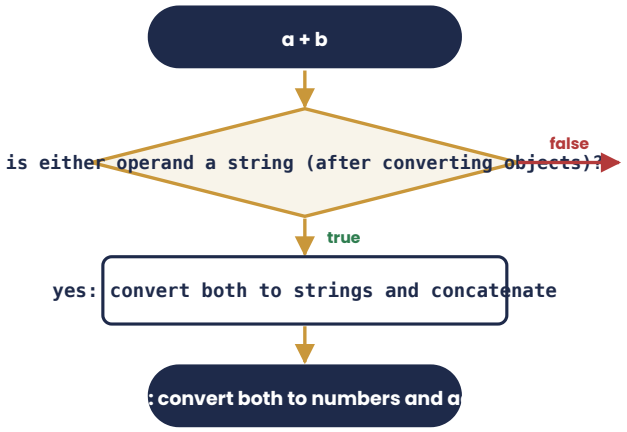
IMPLICIT COERCION BY OPERATORS

main.js

```
console.log("5" + 2);          // "52": concatenation
console.log("5" - 2);          // 3: numeric
console.log("5" * "4");        // 20
console.log(5 + 2 + "1");      // "71": left to right, 7 then "71"
console.log("1" + 5 + 2);      // "152"
console.log(+"42" + 1);        // unary plus converts: 43
console.log(true + true, null + 1, undefined + 1);
console.log([1, 2] + [3], {} + "", [] + []);
console.log("10" / "4", "ten" * 2);
```

OUTPUT

```
52
3
20
71
152
43
2 1 NaN
1,23 [object Object]
2.5 NaN
```



HOW THE + OPERATOR DECIDES

3.6.3 Truthy and falsy values

When a value is used as a condition (in `if`, `while`, `!`, `&&`, `||`), it is converted to a boolean. Exactly **eight values are falsy**: `false`, `0`, `-0`, `0n`, `""` (empty string), `null`, `undefined` and `NaN`. **Everything else is truthy**, including `"0"`, `"false"`, `" "`, empty arrays `[]` and empty objects `{}`.

THE EIGHT FALSY VALUES main.js

```
const falsy = [false, 0, -0, 0n, "", null, undefined, NaN];
const truthy = ["0", "false", " ", [], {}, -1, Infinity, function () {}];
console.log("falsy :", falsy.map((v) => Boolean(v)).join(" "));
console.log("truthy:", truthy.map((v) => Boolean(v)).join(" "));

const cart = [];
if (cart) console.log("an empty array is truthy: check cart.length instead");
if (!cart.length) console.log("cart is empty");
```

OUTPUT

```
falsy : false false false false false false false false
truthy: true true true true true true true true
an empty array is truthy: check cart.length instead
cart is empty
```

3.6.4 isNaN() vs Number.isNaN()

Since `NaN !== NaN`, you cannot test for it with `===`. The old global `isNaN(value)` first **converts** its argument to a number, so it returns true for anything that is not numeric, such as `"hello"` or `undefined`. `Number.isNaN(value)` (ES2015) returns true only if the value really is NaN. Prefer Number.isNaN, and use `Number.isFinite` to check for a usable number.

TESTING FOR NAN

main.js

```
const samples = [NaN, "hello", undefined, "42", 42, {}];
for (const v of samples) {
  const label = typeof v === "string" ? `"${v}"` : String(v === undefined ? "undefined" : (typeof v
=== "object" ? "{}" : v));
  console.log(`${label.padEnd(10)} isNaN: ${String(isNaN(v)).padEnd(6)} Number.isNaN:
${Number.isNaN(v)}`);
}
const input = "12a";
const n = Number(input);
console.log(Number.isNaN(n) ? `"${input}" is not a valid number` : n);
```

OUTPUT

```
NaN        isNaN: true   Number.isNaN: true
"hello"    isNaN: true   Number.isNaN: false
undefined  isNaN: true   Number.isNaN: false
"42"       isNaN: false  Number.isNaN: false
42         isNaN: false  Number.isNaN: false
{}         isNaN: true   Number.isNaN: false
"12a" is not a valid number
```

Declare variables with const by default, let when they change, and avoid var (function-scoped, re-declarable, hoisted as undefined). let and const are block-scoped and sit in the Temporal Dead Zone until declared. JavaScript has seven primitive types (number, bigint, string, boolean, undefined, null, symbol) and objects (including arrays and functions); typeof identifies them, with the quirk typeof null === "object". Numbers are 64-bit floats (beware 0.1 + 0.2 and the safe-integer limit; use BigInt for huge integers). Primitives are copied by value and immutable; objects are shared by reference. Convert explicitly with Number, String, Boolean, parseInt and parseFloat; understand coercion by + and other operators, the eight falsy values, and test with Number.isNaN.

PRACTICE CHECKPOINT

Declare variables of **every primitive type**, print each value with **typeof**, and **predict then verify** the output of ten coercion expressions.

SAMPLE SOLUTION

main.js

```
const values = {
  number: 42.5,
  bigint: 12345678901234567890n,
  string: "JavaScript",
  boolean: true,
  undefined: undefined,
  null: null,
  symbol: Symbol("key"),
};
for (const [label, value] of Object.entries(values)) {
  const shown = typeof value === "bigint" ? `${value}n` : String(value === null ? "null" :
value?.toString?.() ?? value);
  console.log(`${label.padEnd(10)} ${shown.padEnd(24)} typeof: ${typeof value}`);
}

const predictions = [
  ['"3" + 4', "3" + 4, "34"],
  ['"3" - 4', "3" - 4, -1],
  ['"3" * "4"', "3" * "4", 12],
  ['true + 2', true + 2, 3],
  ['null + 5', null + 5, 5],
  ['undefined + 5', undefined + 5, NaN],
  ['[] + 1', [] + 1, "1"],
  ['"5" == 5', "5" == 5, true],
  ['!!"0"', !!"0", true],
  ['1 + "2" + 3', 1 + "2" + 3, "123"],
];
let correct = 0;
for (const [expr, actual, guess] of predictions) {
  const ok = Object.is(actual, guess);
  if (ok) correct++;
  console.log(`${expr.padEnd(14)} = ${typeof actual === "string" ? JSON.stringify(actual) :
String(actual)}  ${ok ? "as predicted" : "SURPRISE"}`);
}
console.log(`${correct}/10 predictions correct`);
```

OUTPUT

```
number     42.5                    typeof: number
bigint     12345678901234567890n   typeof: bigint
string     JavaScript              typeof: string
boolean    true                    typeof: boolean
undefined  undefined               typeof: undefined
null       null                    typeof: object
symbol     Symbol(key)             typeof: symbol
"3" + 4        = "34"  as predicted
"3" - 4        = -1  as predicted
"3" * "4"      = 12  as predicted
true + 2       = 3  as predicted
null + 5       = 5  as predicted
undefined + 5  = NaN  as predicted
[] + 1         = "1"  as predicted
"5" == 5       = true  as predicted
!!"0"          = true  as predicted
1 + "2" + 3    = "123"  as predicted
10/10 predictions correct
```

MODULE 04 · BEGINNER

Operators and expressions

Minimum duration **1 day** | Level **Beginner**

An expression is any piece of code that produces a value; operators combine values into expressions. Besides the usual arithmetic and comparison operators, JavaScript has several unique ones: strict and loose equality, logical operators that return operands, nullish coalescing, optional chaining and logical assignment. This module covers them all, plus bitwise operators and precedence.

IN THIS MODULE

4.1 Arithmetic operators
4.2 Assignment operators
4.3 Comparison operators
4.4 Logical operators
4.5 Nullish and optional operators
4.6 Other operators
4.7 Bitwise operators
4.8 Precedence and associativity

4.1 Arithmetic operators

4.1.1 + - * / % and exponent **

The arithmetic operators work on numbers (and BigInts). Division always produces a decimal result when needed: `7 / 2` is 3.5 (use `Math.trunc` or `Math.floor` for integer division). `%` is the **remainder**, whose sign follows the left operand. `**` (ES2016) raises to a power and is right-associative. Remember that `+` concatenates when a string is involved.

Operator	Name	Example	Result
`+`	addition / concatenation	`7 + 2`, `"7" + 2`	`9`, `"72"`
`-`	subtraction	`7 - 2`	`5`
`*`	multiplication	`7 * 2`	`14`
`/`	division	`7 / 2`	`3.5`
`%`	remainder	`7 % 2`, `-7 % 2`	`1`, `-1`
`**`	exponent	`2 ** 10`	`1024`

ARITHMETIC IN PRACTICE main.js

```
const a = 17, b = 5;
console.log(a + b, a - b, a * b, a / b, a % b, a ** 2);
console.log(Math.trunc(a / b), Math.floor(-17 / 5), Math.trunc(-17 / 5));   // integer division
console.log(-17 % 5, 17 % -5);                    // remainder takes the sign of the left operand
console.log(2 ** 3 ** 2);                         // right-associative: 2 ** 9
console.log(10 % 2 === 0 ? "even" : "odd", 7.5 % 2);
const totalSeconds = 3725;
console.log(`${Math.floor(totalSeconds / 3600)}h ${Math.floor(totalSeconds % 3600 / 60)}m
${totalSeconds % 60}s`);
```

OUTPUT

```
22 12 85 3.4 2 289
3 -4 -3
-2 2
512
even 1.5
1h 2m 5s
```

4.1.2 Increment and decrement (prefix and postfix)

`++` adds 1 and `--` subtracts 1 from a variable. In **prefix** form (`++x`) the variable changes first and the expression gives the new value; in **postfix** form (`x++`) the expression gives the old value and then the variable changes. The difference matters only when the result is used inside a larger expression, which is best avoided for readability.

PREFIX VERSUS POSTFIX main.js

```
let x = 5;
console.log(x++);   // prints 5, then x becomes 6
console.log(x);     // 6
console.log(++x);   // x becomes 7, prints 7
let y = 10;
const z = y-- + --y;   // 10 + 8 (y goes 10 -> 9 -> 8): legal but confusing
console.log(z, y);
```

OUTPUT

```
5
6
7
18 8
```

4.1.3 Unary plus for conversion

A `+` in front of a single value converts it to a number, exactly like `Number(value)`. It is a common shorthand when reading numeric input; unary `-` converts and negates.

CONVERTING WITH UNARY PLUS main.js

```
const qty = "3", price = "250";
console.log(qty + price);          // "3250": concatenation
console.log(+qty * +price);        // 750
console.log(+"", +" 42 ", +"4a", +true, +null, +undefined, -"5");
```

OUTPUT

```
3250
750
0 42 NaN 1 0 NaN -5
```

4.2 Assignment operators

4.2.1 =, compound assignment += -= *= /= %= **=

`=` assigns the value on the right to the variable on the left and is itself an expression whose value is the assigned value (so `a = b = 0` assigns right to left). **Compound assignment** combines an operation with assignment: `x += 5` means `x = x + 5`. `+=` also appends to strings.

COMPOUND ASSIGNMENT

main.js

```
let total = 100;
total += 50;   console.log("+=", total);
total -= 30;   console.log("-=", total);
total *= 2;    console.log("*=", total);
total /= 4;    console.log("/=", total);
total %= 7;    console.log("%=", total);
total **= 3;   console.log("**=", total);
let msg = "Hello";
msg += ", World";
console.log(msg);
let a, b;
a = b = 0;     // right to left
console.log(a, b);
```

OUTPUT

```
+= 150
-= 120
*= 240
/= 60
%= 4
**= 64
Hello, World
0 0
```

4.2.2 Logical assignment &&=, ||=, ??=

ES2021 added three **logical assignment** operators that assign only under a condition and short-circuit otherwise. `a ||= b` assigns if a is falsy; `a &&= b` assigns if a is truthy; `a ??= b` assigns if a is `null` or `undefined`. They are ideal for setting defaults and updating optional values.

LOGICAL ASSIGNMENT OPERATORS

main.js

```
const settings = { theme: "", fontSize: 0, language: null };
settings.theme ||= "light";        // "" is falsy -> replaced
settings.fontSize ??= 16;          // 0 is not null/undefined -> kept
settings.language ??= "en";        // null -> replaced
console.log(settings);

let user = { name: "Asha" };
user &&= { ...user, lastSeen: "today" };   // only if user exists
let guest = null;
guest &&= { ...guest, lastSeen: "today" }; // stays null
console.log(user, guest);
```

OUTPUT

```
{ theme: 'light', fontSize: 0, language: 'en' }
{ name: 'Asha', lastSeen: 'today' } null
```

4.3 Comparison operators

4.3.1 Loose equality == vs strict equality ===

`===` (**strict equality**) is true only if both values have the same type and the same value; no conversion happens. `==` (**loose equality**) first converts the operands to a common type using complicated rules, producing surprises such as `"" == 0` and `"0" == false` being true. **Always use `===` and `!==`**. The one common exception is `value == null`, a short way to test for both null and undefined.

== VERSUS === FOR TEN PAIRS OF VALUES main.js

```
const pairs = [[5, "5"], [0, ""], [0, "0"], ["", "0"], [false, "0"], [null, undefined], [null, 0],
[NaN, NaN], [[1], 1], [true, 1]];
console.log("a".padEnd(11) + "b".padEnd(11) + "==     ===");
for (const [a, b] of pairs) {
  const show = (v) => (typeof v === "string" ? `"${v}"` : Array.isArray(v) ? `[${v}]` :
String(v)).padEnd(11);
  console.log(show(a) + show(b) + String(a == b).padEnd(7) + (a === b));
}
```

OUTPUT

```
a          b          ==     ===
5          "5"        true   false
0          ""         true   false
0          "0"        true   false
""         "0"        false  false
false      "0"        true   false
null       undefined  true   false
null       0          false  false
NaN        NaN        false  false
[1]        1          true   false
true       1          true   false
```

The loose-equality rules in short: null and undefined are equal only to each other; if one side is a number and the other a string or boolean, both are converted to numbers; objects are converted to primitives first; NaN equals nothing. Rather than memorising the table, avoid `==`.

4.3.2 != vs !==

`!==` is the negation of `===` (strict inequality) and `!=` the negation of `==`, with the same coercion problems. Use `!==`.

INEQUALITY main.js

```
console.log(5 != "5", 5 !== "5");
console.log(null != undefined, null !== undefined);
const input = "0";
if (input !== 0) console.log("strict: the string '0' is not the number 0");
```

OUTPUT

```
false true
false true
strict: the string '0' is not the number 0
```

4.3.3 > < >= <= with numbers and strings

Relational operators compare numbers numerically. Two **strings** are compared character by character by their UTF-16 code values ("lexicographically"), so uppercase letters come before lowercase and `"10" <`

`"9"` is true. If one operand is a number, the other is converted to a number. For human-friendly text ordering use `localeCompare` (Module 8).

COMPARING NUMBERS AND STRINGS main.js

```
console.log(10 > 9, "10" > "9", "10" > 9);       // number, string, mixed
console.log("apple" < "banana", "Zebra" < "apple", "a".charCodeAt(0), "Z".charCodeAt(0));
console.log("abc" < "abd", "abc" < "ab");
console.log(null >= 0, undefined >= 0, NaN > 1);  // null -> 0; undefined/NaN -> false
console.log(["b", "a", "C"].sort(), ["b", "a", "C"].sort((x, y) => x.localeCompare(y)));
```

OUTPUT

```
true false true
true true 97 90
true false
true false false
[ 'C', 'a', 'b' ] [ 'a', 'b', 'C' ]
```

4.3.4 Object.is()

`Object.is(a, b)` is like `===` with two differences: `Object.is(NaN, NaN)` is true and `Object.is(0, -0)` is false. It implements "same value" equality, used internally by React state comparisons and Array.includes-like checks.

OBJECT.IS EDGE CASES main.js

```
console.log(NaN === NaN, Object.is(NaN, NaN));
console.log(0 === -0, Object.is(0, -0));
console.log(Object.is("a", "a"), Object.is({}, {}));
console.log([NaN].includes(NaN), [NaN].indexOf(NaN));   // includes uses SameValueZero, indexOf uses
===
```

OUTPUT

```
false true
true false
true false
true -1
```

4.4 Logical operators

4.4.1 &&, ||, !

`&&` (AND) is true when both conditions are true; `||` (OR) when at least one is; `!` (NOT) inverts a value after converting it to boolean.

a	b	a && b	a || b	!a
true	true	true	true	false
true	false	false	true	false
false	true	false	true	true
false	false	false	false	true

COMBINING CONDITIONS

main.js

```
const age = 22, hasId = true, isBanned = false;
console.log(age >= 18 && hasId);            // both true
console.log(age < 13 || age > 60);          // neither
console.log(!isBanned);
const canEnter = age >= 18 && hasId && !isBanned;
console.log(`can enter: ${canEnter}`);
```

OUTPUT

```
true
false
true
can enter: true
```

4.4.2 Short-circuit evaluation and returning operand values

JavaScript's logical operators do not necessarily return true or false: they return **one of their operands**. `a || b` returns a if a is truthy, otherwise b. `a && b` returns a if a is falsy, otherwise b. Evaluation stops as soon as the result is known (**short-circuit**), so the right side may never run. This enables idioms for defaults and conditional execution.

SHORT-CIRCUITING AND OPERAND VALUES

main.js

```
console.log("Asha" || "Guest", "" || "Guest", 0 || 100);      // first truthy (or the last)
console.log("Asha" && "Welcome", "" && "Welcome", null && 5);  // first falsy (or the last)

function log(msg) { console.log("  log called:", msg); return true; }
false && log("never runs");                 // short-circuit: the right side is skipped
true || log("never runs either");
true && log("runs");

const user = { name: "Ravi", isAdmin: true };
user.isAdmin && console.log("show admin panel");   // conditional execution
const display = user.nickname || user.name;        // default value
console.log(display);
```

OUTPUT

```
Asha Guest 100
Welcome  null
  log called: runs
show admin panel
Ravi
```

4.4.3 Double NOT !!

Applying `!` twice converts any value to its boolean equivalent: the first `!` converts and inverts, the second inverts back. `!!value` is the same as `Boolean(value)`.

CONVERTING TO BOOLEAN WITH !!

main.js

```
const values = ["text", "", 0, 42, null, [], {}];
console.log(values.map((v) => !!v).join(" "));
const name = "   ";
console.log(`has name: ${!!name.trim()}`);
```

OUTPUT

```
true false false true false true true
has name: false
```

4.5 Nullish and optional operators

4.5.1 Nullish coalescing ?? vs ||

`a ?? b` (ES2020) returns b only if a is **null or undefined** ("nullish"); otherwise it returns a, even if a is falsy like 0, "" or false. `||` replaces every falsy value, which is a bug when 0 or an empty string are valid values (a volume of 0, a quantity of 0, an empty optional note). Use `??` for defaults.

?? KEEPS VALID FALSY VALUES main.js

```
const config = { volume: 0, title: "", retries: null };
console.log("volume  ||:", config.volume || 50, "  ??:", config.volume ?? 50);
console.log("title   ||:", config.title || "Untitled", "  ??:", JSON.stringify(config.title ??
"Untitled"));
console.log("retries ||:", config.retries || 3, "  ??:", config.retries ?? 3);
console.log("missing ??:", config.timeout ?? 30);
```

OUTPUT

```
volume  ||: 50   ??: 0
title   ||: Untitled   ??: ""
retries ||: 3   ??: 3
missing ??: 30
```

NOTE

`??` cannot be mixed with `&&` or `||` without parentheses: `a || b ?? c` is a SyntaxError; write `(a || b) ?? c`.

4.5.2 Optional chaining ?.

Reading a property of `undefined` or `null` throws a TypeError, a very common bug with nested data from APIs. **Optional chaining** `?.` (ES2020) stops and returns `undefined` if the value before it is null or undefined. It works for properties (`obj?.prop`), computed keys (`obj?.[key]`) and method calls (`obj.method?.()`). Combined with `??` it gives safe access with a default.

SAFE ACCESS WITH ?. main.js

```
const student = { name: "Asha", address: { city: "Pune" }, getGrade() { return "A"; } };
const other = { name: "Ravi" };

console.log(student.address.city, student.address?.city);
console.log(other.address?.city);                  // undefined instead of an error
console.log(other.address?.city ?? "city not given");
console.log(student.getGrade?.(), other.getGrade?.());
const key = "name";
console.log(other?.[key]);
const nobody = null;
console.log(nobody?.name, nobody?.address?.city);
try { console.log(other.address.city); } catch (e) { console.log(`${e.name}: ${e.message}`); }
```

OUTPUT

```
Pune Pune
undefined
city not given
A undefined
Ravi
undefined undefined
TypeError: Cannot read properties of undefined (reading 'city')
```

TIP

Do not scatter `?.` everywhere: use it where a value can legitimately be missing. If a value should always exist, a TypeError is useful because it reveals a bug early.

4.6 Other operators

4.6.1 Ternary ?:

The **conditional (ternary) operator** `condition ? valueIfTrue : valueIfFalse` is the only operator with three operands. It is an expression, so it can be used inside template literals, assignments and return statements. Keep it for simple choices; nested ternaries quickly become unreadable.

CHOOSING A VALUE WITH ?: main.js

```
const marks = 72;
const result = marks >= 40 ? "pass" : "fail";
console.log(result);
console.log(`You have ${3} item${3 === 1 ? "" : "s"} in your cart`);
const grade = marks >= 85 ? "A" : marks >= 70 ? "B" : marks >= 40 ? "C" : "F";   // readable only when formatted like this
console.log(grade);
```

OUTPUT

```
pass
You have 3 items in your cart
B
```

4.6.2 typeof, instanceof, in, delete

`typeof` returns the type of a value as a string (Module 3). `obj instanceof Constructor` checks whether an object was created by a constructor (it walks the prototype chain). `"key" in obj` checks whether a property exists, including inherited ones. `delete obj.key` removes a property from an object and returns true.

TYPEOF, INSTANCEOF, IN AND DELETE main.js

```
const today = new Date();
const list = [1, 2, 3];
console.log(typeof today, today instanceof Date, list instanceof Array, list instanceof Object);

const car = { brand: "Tata", model: "Nexon", color: undefined };
console.log("model" in car, "color" in car, "price" in car, "toString" in car);
delete car.model;
console.log(car, "model" in car);
```

OUTPUT

```
object true true true
true true false true
{ brand: 'Tata', color: undefined } false
```

4.6.3 Comma operator

The **comma operator** evaluates several expressions from left to right and returns the value of the last one. It appears mostly in `for` loop headers that update two variables; elsewhere it hurts readability.

(Commas that separate function arguments, array elements or declarations are not the comma operator.)

THE COMMA OPERATOR main.js

```
for (let i = 0, j = 4; i < j; i++, j--) {      // comma operator in the update part
  console.log(i, j);
}
const value = (console.log("first"), console.log("second"), 42);
console.log(value);
```

OUTPUT

```
0 4
1 3
first
second
42
```

4.6.4 Spread ... (introduced; detailed in Module 12)

The **spread** syntax `...` expands an iterable (such as an array or string) into individual elements, or an object into its properties. It is used to copy and combine arrays and objects and to pass array elements as separate arguments.

SPREAD AT A GLANCE main.js

```
const a = [1, 2, 3], b = [4, 5];
console.log([...a, ...b, 6]);                // combine arrays
console.log(Math.max(...a));                 // array elements as arguments
const base = { theme: "light", size: 14 };
console.log({ ...base, size: 16 });          // copy and override
console.log([..."hey"]);                     // a string spread into characters
```

OUTPUT

```
[ 1, 2, 3, 4, 5, 6 ]
3
{ theme: 'light', size: 16 }
[ 'h', 'e', 'y' ]
```

4.7 Bitwise operators

4.7.1 & | ^ ~ << >> >>>

Bitwise operators work on the binary representation of numbers, converted to **32-bit signed integers**. They are used for flags and permissions, low-level graphics and binary protocols, hashing, and some performance tricks; ordinary application code rarely needs them.

Operator	Name	Example (5 = 0101, 3 = 0011)	Result
`&`	AND	`5 & 3`	1 (0001)
`|`	OR	`5 | 3`	7 (0111)
`^`	XOR	`5 ^ 3`	6 (0110)
`~`	NOT	`~5`	-6

Operator	Name	Example (5 = 0101, 3 = 0011)	Result
`<<`	left shift	`5 << 1`	10
`>>`	sign-propagating right shift	`-20 >> 2`	-5
`>>>`	zero-fill right shift	`-1 >>> 28`	15

BITWISE OPERATORS AND PERMISSION FLAGS

main.js

```
const bin = (n) => (n >>> 0).toString(2).slice(-8).padStart(8, "0");
console.log(5 & 3, 5 | 3, 5 ^ 3, ~5, 5 << 1, -20 >> 2, -1 >>> 28);
console.log(bin(5), bin(3), bin(5 & 3));

const READ = 1, WRITE = 2, EXECUTE = 4;          // permission flags: one bit each
let perms = READ | WRITE;
console.log("can write:", (perms & WRITE) !== 0, " can execute:", (perms & EXECUTE) !== 0);
perms |= EXECUTE;                                 // add a flag
perms &= ~WRITE;                                  // remove a flag
console.log("perms now:", bin(perms));
console.log("is odd 7:", (7 & 1) === 1, " fast floor of 7.9:", 7.9 | 0);
```

OUTPUT

```
1 7 6 -6 10 -5 15
00000101 00000011 00000001
can write: true  can execute: false
perms now: 00000101
is odd 7: true  fast floor of 7.9: 7
```

4.8 Precedence and associativity

Precedence decides which operator applies first when several appear in one expression (`*` before `+`); **associativity** decides the order among operators of equal precedence: most are left-to-right, while assignment, `**` and the ternary are right-to-left. When in doubt, add parentheses: they cost nothing and make intent clear.

Precedence (high to low)	Operators	Associativity
grouping	`( ... )`	n/a
member access, call, optional chaining	`. [] () ?. new`	left to right
postfix	`x++ x--`	n/a
unary	`! ~ + - ++x --x typeof void delete await`	right to left
exponent	`**`	right to left
multiplicative	`* / %`	left to right
additive	`+ -`	left to right
shift	`<< >> >>>`	left to right
relational	`< <= > >= in instanceof`	left to right
equality	`== != === !==`	left to right
bitwise AND, XOR, OR	`&` then `^` then `|`	left to right

Precedence (high to low)	Operators	Associativity
logical AND, OR, nullish	`&&` then `||` / `??`	left to right
conditional	`? :`	right to left
assignment	`= += -= ... &&= ||= ??=`	right to left
comma	`,`	left to right

PRECEDENCE SURPRISES

main.js

```
console.log(2 + 3 * 4, (2 + 3) * 4);            // * before +
console.log(10 - 4 - 3, 2 ** 3 ** 2);            // left-assoc -, right-assoc **
console.log(!true || true, !(true || true));     // ! binds tighter than ||
console.log(1 + 2 + "3", "1" + 2 + 3);           // + is left-to-right
console.log(3 > 2 > 1);                          // (3 > 2) > 1  ->  true > 1  ->  false
console.log(typeof 1 + 1);                       // (typeof 1) + 1  ->  "number1"
console.log(true || false && false, (true || false) && false);   // && before ||
```

OUTPUT

```
14 20
3 512
true false
33 123
false
number1
true false
```

Arithmetic operators include % (remainder) and ** (exponent); + concatenates when a string is involved, and unary + converts to number. Prefix and postfix ++/-- differ in the value they produce. Compound (+=) and logical assignment (||=, &&=, ??=) shorten updates. Always compare with === and !== (== coerces), know how strings compare, and Object.is for NaN and -0. && and || short-circuit and return operands, !! converts to boolean, ?? defaults only null/undefined, and ?. reads nested values safely. The ternary chooses values; typeof, instanceof, in and delete inspect and modify objects; spread expands collections; bitwise operators work on 32-bit integers. Use parentheses when precedence is not obvious.

PRACTICE CHECKPOINT

Write a **tip calculator**, a **demonstration table** comparing **==** and **===** for ten pairs of values, and **safe access** to a nested object property with **?.** and **??**.

SAMPLE SOLUTION

main.js

```
// 1. Tip calculator
function splitBill(amount, tipPercent = 10, people = 1) {
  const tip = amount * tipPercent / 100;
  const total = amount + tip;
  return { tip: tip.toFixed(2), total: total.toFixed(2), each: (total / people).toFixed(2) };
}
console.log(splitBill(1850, 15, 4));
console.log(splitBill(500));

// 2. == versus === table
const pairs = [[1, "1"], [0, false], ["", false], [null, undefined], [null, false], [NaN, NaN],
["1,2", [1, 2]], [" \t", 0], [true, "true"], [undefined, 0]];
const show = (v) => (typeof v === "string" ? JSON.stringify(v) : Array.isArray(v) ? `[${v}]` :
String(v));
console.log("left".padEnd(11) + "right".padEnd(11) + "==".padEnd(7) + "===");
for (const [a, b] of pairs) console.log(show(a).padEnd(11) + show(b).padEnd(11) + String(a ==
b).padEnd(7) + String(a === b));

// 3. Safe nested access
const orders = [
  { id: 1, customer: { name: "Asha", address: { city: "Pune", pin: "411001" } } },
  { id: 2, customer: { name: "Ravi" } },
  { id: 3 },
];
for (const o of orders) {
  const city = o.customer?.address?.city ?? "unknown city";
  const name = o.customer?.name ?? "guest";
  console.log(`order ${o.id}: ${name}, ${city}, pin ${o.customer?.address?.pin ?? "-"}`);
}
```

OUTPUT

```
{ tip: '277.50', total: '2127.50', each: '531.88' }
{ tip: '50.00', total: '550.00', each: '550.00' }
left       right      ==     ===
1          "1"        true   false
0          false      true   false
""         false      true   false
null       undefined  true   false
null       false      false  false
NaN        NaN        false  false
"1,2"      [1,2]      true   false
" \t"      0          true   false
true       "true"     false  false
undefined  0          false  false
order 1: Asha, Pune, pin 411001
order 2: Ravi, unknown city, pin -
order 3: guest, unknown city, pin -
```

MODULE 05 · BEGINNER

Control flow

Minimum duration **2 days** | Level **Beginner**

Control flow decides which statements run and how many times. This module covers conditions (if, else-if ladders, the ternary, guard clauses), the switch statement, every kind of loop including for...of and for...in, break, continue and labels, nested loops for printing patterns, and the classic beginner problems up to a number-guessing game.

IN THIS MODULE

5.1 Conditional statements
5.2 switch statement
5.3 Loops
5.4 Jump statements
5.5 Nested loops and patterns
5.6 Classic problems

5.1 Conditional statements

5.1.1 if, if-else, else-if ladder, nested if

An `if` statement runs a block only when its condition is truthy. `else` gives an alternative block. An **else-if ladder** tests conditions in order and runs only the first block whose condition is true. Conditions can be **nested**, though deep nesting hurts readability. Always use braces, even for single statements: it prevents bugs when lines are added later.

IF, ELSE-IF LADDER AND NESTED IF

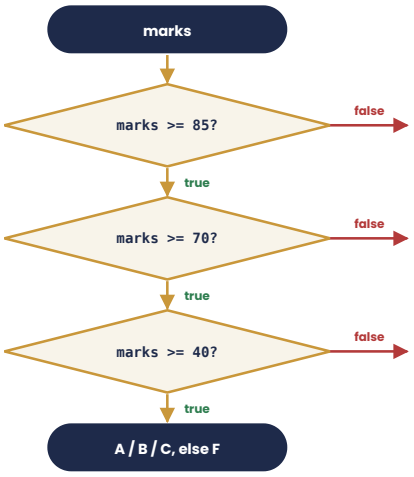
main.js

```
function describe(marks) {
  if (marks < 0 || marks > 100) {
    return "invalid marks";
  } else if (marks >= 85) {
    return "A: distinction";
  } else if (marks >= 70) {
    return "B: first class";
  } else if (marks >= 40) {
    return "C: pass";
  } else {
    return "F: fail";
  }
}
for (const m of [92, 75, 40, 12, 105]) console.log(m, "->", describe(m));

const age = 20, hasLicence = false;
if (age >= 18) {
  if (hasLicence) {
    console.log("can drive");
  } else {
    console.log("adult, but needs a licence");      // nested if
  }
}
```

OUTPUT

```
92 -> A: distinction
75 -> B: first class
40 -> C: pass
12 -> F: fail
105 -> invalid marks
adult, but needs a licence
```



AN ELSE-IF LADDER TESTS CONDITIONS FROM TOP TO BOTTOM

5.1.2 Ternary for simple conditions

When an `if/else` only chooses between two values, the ternary operator (Module 4) is shorter. Use if/else when the branches contain statements or more than one action.

THE TERNARY AS A COMPACT IF/ELSE main.js

```
const stock = 0;
const label = stock > 0 ? "In stock" : "Out of stock";      // choose a value
console.log(label);
const items = 1;
console.log(`${items} ${items === 1 ? "item" : "items"}`);
const isMember = true, price = 1000;
const finalPrice = isMember ? price * 0.9 : price;
console.log(finalPrice);
```

OUTPUT

```
Out of stock
1 item
900
```

5.1.3 Guard clauses and early returns

A **guard clause** checks for a special or invalid case at the top of a function and returns (or throws) immediately. The main logic then sits at the outer level instead of being buried in nested `if` blocks. Code with early returns is flatter and easier to read.

FLATTENING NESTED IFS WITH GUARD CLAUSES main.js

```
// Nested version: the real work is three levels deep
function shippingNested(order) {
  if (order) {
    if (order.items.length > 0) {
      if (order.total >= 500) {
        return 0;
      } else {
        return 50;
      }
    } else {
      return "empty order";
    }
  } else {
    return "no order";
  }
}

// Guard clauses: handle special cases first, then the normal case
function shipping(order) {
  if (!order) return "no order";
  if (order.items.length === 0) return "empty order";
  if (order.total >= 500) return 0;
  return 50;
}

const orders = [null, { items: [], total: 0 }, { items: ["book"], total: 650 }, { items: ["pen"],
total: 120 }];
for (const o of orders) console.log(shippingNested(o), shipping(o));
```

OUTPUT

```
no order no order
empty order empty order
0 0
50 50
```

5.2 switch statement

5.2.1 switch, case, break, default

A `switch` compares one value against a list of `case` values and jumps to the first match. Each case normally ends with `break` to leave the switch; `default` runs when no case matches. switch is clearer than a long else-if ladder when you compare one variable with many fixed values.

A BASIC SWITCH main.js

```
function dayName(n) {
  let name;
  switch (n) {
    case 1: name = "Monday"; break;
    case 2: name = "Tuesday"; break;
    case 3: name = "Wednesday"; break;
    case 4: name = "Thursday"; break;
    case 5: name = "Friday"; break;
    case 6: name = "Saturday"; break;
    case 7: name = "Sunday"; break;
    default: name = "invalid day";
  }
  return name;
}
console.log(dayName(3), dayName(7), dayName(9));
```

OUTPUT

```
Wednesday Sunday invalid day
```

5.2.2 Strict comparison in switch

switch compares with **strict equality** (`===`): the string "1" does not match `case 1`. Convert input to the right type first. To switch on conditions instead of values, a common idiom is `switch (true)` with boolean expressions as cases.

SWITCH USES === main.js

```
const input = "2";              // e.g. from prompt() or a form: always a string
switch (input) {
  case 2: console.log("number 2 matched"); break;
  case "2": console.log('string "2" matched'); break;
}
switch (Number(input)) {
  case 2: console.log("after Number(), the number 2 matches"); break;
}

const temp = 31;
switch (true) {                  // switch on conditions
  case temp < 0: console.log("freezing"); break;
  case temp < 20: console.log("cool"); break;
  case temp < 30: console.log("warm"); break;
  default: console.log("hot");
}
```

OUTPUT

```
string "2" matched
after Number(), the number 2 matches
hot
```

5.2.3 Fall-through and grouping cases

Without `break`, execution **falls through** into the next case. This is a classic bug when forgotten, but it is useful deliberately to group several cases that share one block. Mark intentional fall-through with a comment.

GROUPING CASES, AND ACCIDENTAL FALL-THROUGH main.js

```
function daysInMonth(month, year) {
  switch (month) {
    case "Apr": case "Jun": case "Sep": case "Nov":      // grouped cases
      return 30;
    case "Feb":
      return (year % 4 === 0 && year % 100 !== 0) || year % 400 === 0 ? 29 : 28;
    default:
      return 31;
  }
}
console.log(daysInMonth("Jun", 2026), daysInMonth("Feb", 2024), daysInMonth("Feb", 2026),
daysInMonth("Jan", 2026));

const level = 2;
switch (level) {                 // forgotten breaks: falls through every following case
  case 1: console.log("level 1 perks");
  case 2: console.log("level 2 perks");
  case 3: console.log("level 3 perks");
}
```

```
OUTPUT
30 29 28 31
level 2 perks
level 3 perks
```

5.3 Loops

Loops repeat a block of code. Choose the loop that states your intent most clearly: `for` for counting, `while` when the number of repetitions is unknown, `do-while` when the body must run at least once, `for...of` to visit the values of an array or other iterable, and `for...in` to visit the keys of an object.

5.3.1 for

A `for` loop has three parts in its header: an **initialisation** that runs once, a **condition** checked before every iteration, and an **update** that runs after every iteration. Declare the counter with `let` so it exists only inside the loop.

COUNTING WITH FOR main.js

```
for (let i = 1; i <= 5; i++) {
  console.log(`5 x ${i} = ${5 * i}`);
}
let sum = 0;
for (let i = 2; i <= 20; i += 2) sum += i;            // even numbers 2..20
console.log("sum of evens:", sum);
const colors = ["red", "green", "blue"];
for (let i = colors.length - 1; i >= 0; i--) console.log(i, colors[i]);   // counting down
```

OUTPUT

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
sum of evens: 110
2 blue
1 green
0 red
```



THE ORDER OF A FOR LOOP: INIT, THEN CONDITION, BODY, UPDATE, CONDITION, ...

5.3.2 while

A `while` loop checks its condition before each iteration and repeats while it is truthy. Use it when you do not know in advance how many iterations are needed. Make sure something inside the loop eventually makes the condition false.

REPEATING UNTIL A CONDITION CHANGES main.js

```
let balance = 10000;
let years = 0;
while (balance < 20000) {          // how long until the money doubles at 8%?
  balance *= 1.08;
  years++;
}
console.log(`${years} years, balance Rs ${balance.toFixed(2)}`);

let n = 1234567, digits = 0;
while (n > 0) { n = Math.floor(n / 10); digits++; }
console.log("digits:", digits);
```

OUTPUT

```
10 years, balance Rs 21589.25
digits: 7
```

5.3.3 do-while

A `do...while` loop checks its condition **after** the body, so the body always runs at least once. It suits menus and input validation, where you must ask before you can check.

DO-WHILE RUNS AT LEAST ONCE main.js

```
let attempts = 0;
let code;
const tries = ["12", "abc", "4821"];          // simulated user inputs
do {
  code = tries[attempts];
  attempts++;
  console.log(`attempt ${attempts}: entered "${code}"`);
} while (!/^\d{4}$/.test(code));              // repeat until a 4-digit code is entered
console.log("accepted:", code);

let x = 100;
do { console.log("runs once even though x > 10"); } while (x < 10);
```

OUTPUT

```
attempt 1: entered "12"
attempt 2: entered "abc"
attempt 3: entered "4821"
accepted: 4821
runs once even though x > 10
```

5.3.4 for...of (iterables)

`for...of` (ES2015) loops over the **values** of any **iterable**: arrays, strings, Maps, Sets, NodeLists and more. It is the clearest way to visit every element when you do not need the index; use `entries()` when you need both.

FOR...OF OVER DIFFERENT ITERABLES main.js

```
const fruits = ["mango", "apple", "kiwi"];
for (const fruit of fruits) console.log(fruit);
for (const [index, fruit] of fruits.entries()) console.log(index, fruit);
for (const ch of "Hi!") console.log(ch);
const prices = new Map([["pen", 10], ["book", 250]]);
for (const [item, price] of prices) console.log(`${item}: Rs ${price}`);
for (const n of new Set([3, 1, 3, 2])) console.log("unique", n);
```

OUTPUT

```
mango
apple
kiwi
0 mango
1 apple
2 kiwi
H
i
!
pen: Rs 10
book: Rs 250
unique 3
unique 1
unique 2
```

5.3.5 for...in (object keys) and why not to use it on arrays

`for...in` loops over the **enumerable property names** (keys) of an object, including inherited ones. It is meant for objects. On arrays it is a poor choice: keys are strings, not numbers; any extra properties added to the array (or its prototype) are visited too; and the order is not guaranteed in all cases. Use `for...of`, `forEach` or a `for` loop for arrays, and `Object.keys/entries` (Module 9) for objects.

FOR...IN ON AN OBJECT AND ON AN ARRAY

main.js

```
const student = { name: "Asha", marks: 91, city: "Pune" };
for (const key in student) {
  console.log(`${key} = ${student[key]}`);
}

const arr = [10, 20, 30];
arr.note = "extra property";
for (const i in arr) console.log("for...in:", i, typeof i);     // string keys, extra property
included
for (const v of arr) console.log("for...of:", v);                // just the values
```

OUTPUT

```
name = Asha
marks = 91
city = Pune
for...in: 0 string
for...in: 1 string
for...in: 2 string
for...in: note string
for...of: 10
for...of: 20
for...of: 30
```

5.3.6 Infinite loops

A loop whose condition never becomes false runs forever, freezing the browser tab or the Node.js process (a browser eventually offers to stop the page). Common causes: forgetting the update, updating the wrong variable, or a condition that can never be met (for example comparing floating-point values with `!==`). Intentional infinite loops, `while (true)`, are fine when they contain a `break` or `return`.

INFINITE LOOPS: ACCIDENTAL AND INTENTIONAL

main.js

```
// while (i < 5) { console.log(i); }        // BUG: i never changes, runs forever

let i = 0;
while (true) {                              // intentional: exits with break
  i++;
  if (i * i > 50) break;
}
console.log("first number whose square exceeds 50:", i);

for (let x = 0.1, steps = 0; steps < 100; x += 0.1, steps++) {
  if (Math.abs(x - 1) < 1e-9) { console.log(`reached 1.0 after ${steps + 1} additions (x !== 1 would
never stop)`); break; }
}
```

OUTPUT

```
first number whose square exceeds 50: 8
reached 1.0 after 10 additions (x !== 1 would never stop)
```

5.4 Jump statements

5.4.1 break and continue

`break` leaves the innermost loop (or switch) immediately. `continue` skips the rest of the current iteration and goes on with the next one. Use them to stop searching once a result is found, or to skip items that

should not be processed.

BREAK AND CONTINUE

main.js

```
const numbers = [4, 9, 15, 22, 7, 30];
for (const n of numbers) {
  if (n > 20) {
    console.log("first number above 20:", n);
    break;                                  // stop the loop
  }
}
for (const n of numbers) {
  if (n % 2 === 0) continue;                // skip even numbers
  console.log("odd:", n);
}
```

OUTPUT

```
first number above 20: 22
odd: 9
odd: 15
odd: 7
```

5.4.2 Labels

A **label** names a loop (`outer: for (...)`) so that `break outer` or `continue outer` can act on that outer loop from inside a nested one. Labels are rarely needed; extracting the nested loops into a function and using `return` is often clearer.

BREAKING OUT OF NESTED LOOPS WITH A LABEL

main.js

```
const grid = [
  [1, 2, 3],
  [4, 99, 6],
  [7, 8, 9],
];
outer: for (let r = 0; r < grid.length; r++) {
  for (let c = 0; c < grid[r].length; c++) {
    if (grid[r][c] === 99) {
      console.log(`found 99 at row ${r}, column ${c}`);
      break outer;                          // leaves BOTH loops
    }
  }
}
rows: for (const row of grid) {
  for (const v of row) {
    if (v % 2 === 0) continue rows;         // skip to the next row at the first even number
    console.log("checked", v);
  }
}
```

OUTPUT

```
found 99 at row 1, column 1
checked 1
checked 7
```

5.5 Nested loops and patterns

5.5.1 Star, number and character patterns

Printing patterns is the classic exercise for nested loops: the **outer loop** controls the rows, the **inner loop** builds the content of each row. In JavaScript, build each row as a string (with `+=` or `repeat()`) and print it once, since every `console.log` starts a new line.

STAR PATTERNS

main.js

```
const n = 5;
for (let i = 1; i <= n; i++) {                  // right-angled triangle
  let row = "";
  for (let j = 1; j <= i; j++) row += "* ";
  console.log(row);
}
for (let i = 1; i <= n; i++) {                  // pyramid with repeat()
  console.log(" ".repeat(n - i) + "*".repeat(2 * i - 1));
}
```

OUTPUT

```
*
* *
* * *
* * * *
* * * * *
    *
   ***
  *****
 *******
*********
```

NUMBER AND CHARACTER PATTERNS

main.js

```
for (let i = 1; i <= 4; i++) {                  // number triangle
  let row = "";
  for (let j = 1; j <= i; j++) row += j + " ";
  console.log(row);
}
let k = 1;
for (let i = 1; i <= 4; i++) {                  // Floyd's triangle
  let row = "";
  for (let j = 1; j <= i; j++) row += String(k++).padStart(3);
  console.log(row);
}
for (let i = 0; i < 5; i++) {                   // character pattern
  let row = "";
  for (let j = 0; j <= i; j++) row += String.fromCharCode(65 + j);
  console.log(row);
}
for (let i = 1; i <= 3; i++) {                  // multiplication table grid
  let row = "";
  for (let j = 1; j <= 5; j++) row += String(i * j).padStart(4);
  console.log(row);
}
```

OUTPUT

```
1
1 2
1 2 3
1 2 3 4
  1
  2  3
  4  5  6
  7  8  9 10
A
AB
ABC
ABCD
ABCDE
   1   2   3   4   5
   2   4   6   8  10
   3   6   9  12  15
```

5.6 Classic problems

5.6.1 Sum of digits, reverse a number, palindrome

These problems all peel off the last digit with `% 10` and remove it with `Math.floor(n / 10)`, repeating until the number is 0.

WORKING DIGIT BY DIGIT

main.js

```
function sumOfDigits(n) {
  n = Math.abs(n);
  let sum = 0;
  while (n > 0) { sum += n % 10; n = Math.floor(n / 10); }
  return sum;
}
function reverseNumber(n) {
  let rev = 0;
  while (n > 0) { rev = rev * 10 + (n % 10); n = Math.floor(n / 10); }
  return rev;
}
const isPalindrome = (n) => n === reverseNumber(n);
console.log(sumOfDigits(9045), reverseNumber(12340), reverseNumber(1234));
for (const n of [121, 1331, 123, 7]) console.log(n, isPalindrome(n) ? "is a palindrome" : "is not a
palindrome");
console.log("string way:", String(98765).split("").reverse().join(""));
```

OUTPUT

```
18 4321 4321
121 is a palindrome
1331 is a palindrome
123 is not a palindrome
7 is a palindrome
string way: 56789
```

5.6.2 Factorial and Fibonacci

FACTORIAL AND FIBONACCI WITH LOOPS

main.js

```
function factorial(n) {
  let result = 1;
  for (let i = 2; i <= n; i++) result *= i;
  return result;
}
console.log("5! =", factorial(5), " 20! =", factorial(20));
console.log("25! =", factorial(25), "(no longer exact)");
let big = 1n;
for (let i = 2n; i <= 25n; i++) big *= i;
console.log("25! with BigInt =", big);

const fib = [0, 1];
while (fib.length < 15) fib.push(fib.at(-1) + fib.at(-2));
console.log("Fibonacci:", fib.join(", "));
```

OUTPUT

```
5! = 120  20! = 2432902008176640000
25! = 1.5511210043330986e+25 (no longer exact)
25! with BigInt = 15511210043330985984000000n
Fibonacci: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377
```

5.6.3 Prime check, FizzBuzz

A number n > 1 is **prime** if no number from 2 up to √n divides it; checking only up to the square root makes the test fast. **FizzBuzz** is the famous screening question: print 1 to n, but "Fizz" for multiples of 3, "Buzz" for multiples of 5 and "FizzBuzz" for multiples of both. The trick is to test the combined case first.

PRIME CHECK AND FIZZBUZZ main.js

```
function isPrime(n) {
  if (n < 2) return false;
  for (let d = 2; d * d <= n; d++) {
    if (n % d === 0) return false;
  }
  return true;
}
const primes = [];
for (let n = 1; n <= 50; n++) if (isPrime(n)) primes.push(n);
console.log("primes up to 50:", primes.join(" "));
console.log("97 prime?", isPrime(97), " 91 prime?", isPrime(91));

let line = [];
for (let i = 1; i <= 15; i++) {
  if (i % 15 === 0) line.push("FizzBuzz");      // both: check first
  else if (i % 3 === 0) line.push("Fizz");
  else if (i % 5 === 0) line.push("Buzz");
  else line.push(i);
}
console.log(line.join(" "));
```

OUTPUT

```
primes up to 50: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
97 prime? true  91 prime? false
1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz
```

5.6.4 Number guessing game

The number guessing game combines everything: a random secret number, a loop that repeats until the guess is right, input conversion and validation, and conditions that give hints. In the browser it uses `prompt`; in Node.js, input comes from the terminal through the `readline` module (shown with the simulated answers typed by a player).

GUESSING GAME IN THE BROWSER

main.js

```
const secret = 37;                              // Math.floor(Math.random() * 100) + 1 in a real game
let attempts = 0;
let guess;
do {
  const text = prompt("Guess a number between 1 and 100:");
  if (text === null) { alert("Game cancelled"); break; }
  guess = Number(text);
  attempts++;
  if (!Number.isInteger(guess) || guess < 1 || guess > 100) alert("Please enter a whole number from 1
to 100.");
  else if (guess < secret) alert("Too low!");
  else if (guess > secret) alert("Too high!");
  else alert(`Correct! ${secret} in ${attempts} attempts.`);
} while (guess !== secret);
```

CONSOLE (user input in gold)

```
[prompt] Guess a number between 1 and 100: 50
[alert] Too high!
[prompt] Guess a number between 1 and 100: abc
[alert] Please enter a whole number from 1 to 100.
[prompt] Guess a number between 1 and 100: 25
[alert] Too low!
[prompt] Guess a number between 1 and 100: 37
[alert] Correct! 37 in 4 attempts.
```

GUESSING GAME IN NODE.JS

main.js

```
const readline = require("node:readline/promises");
const rl = readline.createInterface({ input: process.stdin, output: process.stdout, terminal: false
});

async function play() {
  const secret = 64;                    // Math.floor(Math.random() * 100) + 1
  for (let attempt = 1; attempt <= 7; attempt++) {
    const guess = Number(await rl.question(`Attempt ${attempt}/7, your guess: `));
    if (guess === secret) { console.log(`You win in ${attempt} attempts!`); rl.close(); return; }
    console.log(guess < secret ? "Higher..." : "Lower...");
  }
  console.log(`Out of attempts. The number was ${secret}.`);
  rl.close();
}
play();
```

CONSOLE (user input in gold)

```
Attempt 1/7, your guess: 50
Higher...
Attempt 2/7, your guess: 75
Lower...
Attempt 3/7, your guess: 62
Higher...
Attempt 4/7, your guess: 68
Lower...
Attempt 5/7, your guess: 64
You win in 5 attempts!
```

if, else-if ladders and nested ifs choose between blocks; the ternary chooses between values; guard clauses keep functions flat. switch compares one value with ===, needs break (fall-through is useful only for grouping cases) and has a default. for counts, while repeats while a condition holds, do-while runs at least once, for…of visits iterable values, and for…in visits object keys (not for arrays). break leaves a loop, continue skips an iteration, and labels target outer loops. Nested loops build patterns row by row; digit loops with % 10, prime checks up to √n, FizzBuzz and a guessing game practise all of it.

PRACTICE CHECKPOINT

Write **FizzBuzz**, a **grade calculator with a switch**, and a **number-guessing game** using prompt and a loop.

FIZZBUZZ, GRADES AND A GUESSING GAME

main.js

```
// FizzBuzz 1..20
const out = [];
for (let i = 1; i <= 20; i++) out.push(i % 15 === 0 ? "FizzBuzz" : i % 3 === 0 ? "Fizz" : i % 5 === 0
? "Buzz" : i);
console.log(out.join(" "));

// Grade calculator with switch
function grade(marks) {
  switch (true) {
    case marks > 100 || marks < 0: return "invalid";
    case marks >= 90: return "O";
    case marks >= 80: return "A+";
    case marks >= 70: return "A";
    case marks >= 60: return "B";
    case marks >= 40: return "C";
    default: return "F";
  }
}
for (const m of [95, 83, 71, 64, 45, 20, 120]) console.log(`${m} -> ${grade(m)}`);

// Guessing game
const secret = 7;                       // Math.floor(Math.random() * 10) + 1
let tries = 0, guess = null;
while (guess !== secret) {
  const answer = prompt("Guess the number (1-10):");
  if (answer === null) break;
  guess = parseInt(answer, 10);
  tries++;
  console.log(guess === secret ? `Got it in ${tries} tries!` : guess < secret ? `${guess} is too low`
: `${guess} is too high`);
}
```

CONSOLE (user input in gold)

```
1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz 16 17 Fizz 19 Buzz
95 -> O
83 -> A+
71 -> A
64 -> B
45 -> C
20 -> F
120 -> invalid
[prompt] Guess the number (1-10): 5
5 is too low
[prompt] Guess the number (1-10): 9
9 is too high
[prompt] Guess the number (1-10): 7
Got it in 3 tries!
```

MODULE 06 · BEGINNER

Functions

Minimum duration **3 days** | Level **Beginner**

Functions are the building blocks of JavaScript programs, and in JavaScript they are first-class values. This module covers every way to define them (declarations, expressions, arrow functions, IIFEs), parameters and arguments (defaults, rest parameters, the legacy arguments object, passing objects), return values, callbacks and higher-order functions, the differences between arrow and regular functions, recursion, and an introduction to pure functions.

IN THIS MODULE

6.1 Defining functions
6.2 Parameters and arguments
6.3 Return values
6.4 Functions as values
6.5 Arrow functions vs regular functions
6.6 Recursion
6.7 Pure functions and side effects (introduction)

6.1 Defining functions

A **function** is a reusable block of code with a name (usually), inputs called **parameters** and an output called the **return value**. Functions remove repetition, give names to ideas and split programs into small, testable pieces. JavaScript offers several ways to create one.

6.1.1 Function declarations

A **function declaration** starts with the `function` keyword and a name. Declarations are **hoisted** completely: you can call them before they appear in the code, which allows a "main logic first, helpers below" layout.

FUNCTION DECLARATIONS — main.js

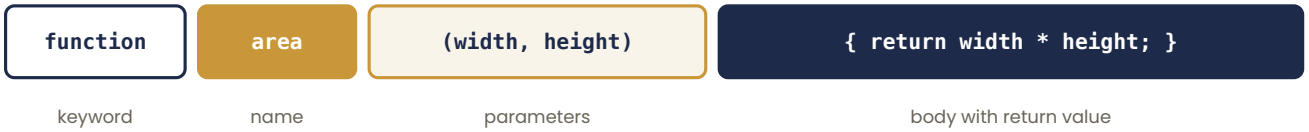
```
console.log(area(5, 3));            // works: declarations are hoisted

function area(width, height) {       // declaration: name, parameters, body
  return width * height;
}

function greet(name) {
  console.log(`Hello, ${name}!`);
}
greet("Asha");
greet("Ravi");
```

OUTPUT

```
15
Hello, Asha!
Hello, Ravi!
```



THE PARTS OF A FUNCTION DECLARATION

6.1.2 Function expressions

A **function expression** creates a function as part of an expression, usually assigned to a variable. It is not hoisted like a declaration: the variable exists, but the function is assigned only when that line runs, so calling it earlier fails. Function expressions are natural when a function is a value: stored in a variable, passed as an argument or created conditionally.

FUNCTION EXPRESSIONS ARE VALUES ASSIGNED AT RUN TIME main.js

```
try {
  console.log(square(4));                // too early: square is in the TDZ
} catch (e) {
  console.log(`${e.name}: ${e.message}`);
}
const square = function (x) {            // function expression
  return x * x;
};
console.log(square(4));

const isWeekend = true;
const getGreeting = isWeekend
  ? function () { return "Enjoy your weekend!"; }
  : function () { return "Have a productive day!"; };
console.log(getGreeting());
```

OUTPUT

```
ReferenceError: Cannot access 'square' before initialization
16
Enjoy your weekend!
```

6.1.3 Arrow functions and implicit return

Arrow functions (ES2015) are a shorter syntax: parameters, then `=>`, then the body. If the body is a single expression, it is returned automatically (**implicit return**) without braces or `return`. One parameter needs no parentheses; zero or several do. To return an object literal implicitly, wrap it in parentheses. Arrow functions also differ in how `this` works (6.5).

ARROW FUNCTION FORMS

main.js

```
const add = (a, b) => a + b;                 // implicit return
const double = n => n * 2;                    // one parameter: parentheses optional
const sayHi = () => "hi";                     // no parameters
const makeUser = (name, age) => ({ name, age });   // object literal in parentheses
const describe = (n) => {                     // block body: needs return
  const kind = n % 2 === 0 ? "even" : "odd";
  return `${n} is ${kind}`;
};
console.log(add(2, 3), double(21), sayHi(), makeUser("Asha", 21), describe(7));
const broken = () => { name: "x" };           // braces are read as a block: returns undefined
console.log(broken());
```

OUTPUT

```
5 42 hi { name: 'Asha', age: 21 } 7 is odd
undefined
```

6.1.4 Anonymous and named function expressions

A function expression can be **anonymous** (`function () {...}`) or **named** (`function fact() {...}`). The name of a named function expression is visible only inside the function itself, which allows recursion that keeps working even if the outer variable is reassigned, and it appears in stack traces. Anonymous functions assigned to variables still get an inferred `name` property.

ANONYMOUS AND NAMED FUNCTION EXPRESSIONS

main.js

```
const anon = function () {};
const named = function calculate() {};
const arrow = () => {};
console.log(anon.name, named.name, arrow.name);

const countdown = function tick(n) {          // 'tick' is usable inside for recursion
  if (n === 0) return "liftoff";
  return `${n} ` + tick(n - 1);
};
console.log(countdown(3));
console.log(typeof tick);                     // not visible outside

setTimeout(function () { console.log("anonymous callback ran"); }, 0);
```

OUTPUT

```
anon calculate arrow
3 2 1 liftoff
undefined
anonymous callback ran
```

6.1.5 Immediately Invoked Function Expressions (IIFE)

An **IIFE** is a function expression that is called immediately after it is created: `(function () { ... })()`. Before block scope and modules existed, IIFEs were the main way to create a private scope so that variables did not leak into the global scope; you will still see them in older code and library bundles. Today block scope, modules and top-level `await` cover most uses, but an async IIFE remains handy in scripts.

IMMEDIATELY INVOKED FUNCTION EXPRESSIONS

main.js

```
(function () {
  const secret = "hidden inside the IIFE";
  console.log("IIFE ran:", secret);
})();
console.log(typeof secret);

const counter = (() => {              // an IIFE returning an object: the module pattern
  let count = 0;                       // private state
  return { increment: () => ++count, value: () => count };
})();
counter.increment(); counter.increment();
console.log("counter:", counter.value());

(async () => {                         // async IIFE
  const result = await Promise.resolve(42);
  console.log("async IIFE got", result);
})();
```

OUTPUT

```
IIFE ran: hidden inside the IIFE
undefined
counter: 2
async IIFE got 42
```

6.2 Parameters and arguments

6.2.1 Parameters vs arguments

Parameters are the names listed in the function definition; **arguments** are the actual values passed in a call. JavaScript does not check the count: missing arguments make their parameters `undefined`, and extra arguments are ignored (but still available through rest parameters or `arguments`). `fn.length` gives the number of declared parameters.

PARAMETERS AND ARGUMENTS

main.js

```
function introduce(name, city) {          // parameters: name, city
  return `${name} from ${city}`;
}
console.log(introduce("Asha", "Pune"));    // arguments: "Asha", "Pune"
console.log(introduce("Ravi"));            // missing argument -> undefined
console.log(introduce("Meera", "Delhi", "extra"));   // extra argument ignored
console.log("declared parameters:", introduce.length);
```

OUTPUT

```
Asha from Pune
Ravi from undefined
Meera from Delhi
declared parameters: 2
```

6.2.2 Default parameters

A parameter can have a **default value** (ES2015), used when the argument is missing or explicitly `undefined` (but not when it is `null`). Defaults are evaluated at call time and may refer to earlier parameters or call functions.

DEFAULT PARAMETER VALUES main.js

```
function createOrder(item, qty = 1, price = qty > 10 ? 9 : 10) {
  return `${qty} x ${item} @ Rs ${price} = Rs ${qty * price}`;
}
console.log(createOrder("pen"));
console.log(createOrder("pen", 5));
console.log(createOrder("pen", 20));
console.log(createOrder("pen", undefined, 12));   // undefined -> default used
console.log(createOrder("pen", null));            // null is a value: qty = null -> 0

function log(message, time = new Date(2026, 2, 9, 10, 30).toTimeString().slice(0, 5)) {   // evaluated
at each call
  console.log(`[${time}] ${message}`);
}
log("server started");
```

OUTPUT

```
1 x pen @ Rs 10 = Rs 10
5 x pen @ Rs 10 = Rs 50
20 x pen @ Rs 9 = Rs 180
1 x pen @ Rs 12 = Rs 12
null x pen @ Rs 10 = Rs 0
[10:30] server started
```

6.2.3 Rest parameters ...args

A **rest parameter** `...name` collects all remaining arguments into a real **array**. It must be the last parameter. It lets a function accept any number of arguments.

COLLECTING ARGUMENTS WITH REST PARAMETERS main.js

```
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
console.log(sum(), sum(5), sum(1, 2, 3, 4));

function tag(label, ...items) {
  console.log(`${label}: ${items.join(", ")} (${items.length} items, is array:
${Array.isArray(items)})`);
}
tag("fruits", "mango", "kiwi", "apple");

const values = [7, 3, 9];
console.log(sum(...values));              // spread an array into separate arguments
```

OUTPUT

```
0 5 10
fruits: mango, kiwi, apple (3 items, is array: true)
19
```

6.2.4 The arguments object (legacy)

Inside a regular (non-arrow) function, `arguments` is an array-like object holding all passed arguments. It was the only way to handle a variable number of arguments before rest parameters. It is not a real array (no map, filter, reduce), it does not exist in arrow functions, and it makes code harder to read. Use rest parameters in new code.

THE OLD ARGUMENTS OBJECT

main.js

```
function oldStyleSum() {
  let total = 0;
  for (let i = 0; i < arguments.length; i++) total += arguments[i];
  console.log(`arguments is an array? ${Array.isArray(arguments)}, length ${arguments.length}`);
  return total;
}
console.log(oldStyleSum(10, 20, 30));
function toArray() { return Array.from(arguments); }
console.log(toArray("a", "b"));
const arrow = () => typeof arguments;     // arrow functions have no own arguments (here: Node's
module wrapper's)
console.log("arrow functions see their outer arguments, not their own:", arrow() === "object");
```

OUTPUT

```
arguments is an array? false, length 3
60
[ 'a', 'b' ]
arrow functions see their outer arguments, not their own: true
```

6.2.5 Passing primitives vs objects

Arguments are passed **by value**, but for objects the value is a **reference**. Reassigning a parameter inside the function never affects the caller's variable. Changing the **contents** of an object parameter (its properties, an array's elements) does affect the caller, because both refer to the same object. To avoid surprising callers, copy objects before modifying them.

PRIMITIVES VERSUS OBJECTS AS ARGUMENTS

main.js

```
function addBonus(marks) { marks += 10; return marks; }
let score = 70;
addBonus(score);
console.log("primitive unchanged:", score);

function promote(student) { student.level += 1; }       // mutates the caller's object
const s = { name: "Asha", level: 1 };
promote(s);
console.log("object changed:", s);

function replace(student) { student = { name: "new object" }; }   // reassigns only the parameter
replace(s);
console.log("after reassigning the parameter:", s.name);

function promoteSafely(student) { return { ...student, level: student.level + 1 }; }   // copy instead
const promoted = promoteSafely(s);
console.log(s.level, promoted.level);
```

OUTPUT

```
primitive unchanged: 70
object changed: { name: 'Asha', level: 2 }
after reassigning the parameter: Asha
2 3
```

6.3 Return values

6.3.1 return and undefined by default

`return value` ends the function immediately and sends the value back to the caller. A function without a `return` statement, or with a bare `return;`, returns `undefined`. Code after a return in the same block never runs. Remember the ASI trap: the returned expression must start on the same line as `return`.

RETURN STATEMENTS main.js

```
function check(age) {
  if (age < 0) return "invalid";          // early return
  if (age >= 18) return "adult";
  return "minor";
  console.log("never runs");              // unreachable
}
function noReturn() { const x = 5; }
function bareReturn() { return; }
console.log(check(-1), check(30), check(12));
console.log(noReturn(), bareReturn());
```

OUTPUT

```
invalid adult minor
undefined undefined
```

6.3.2 Returning objects and arrays

A function returns a single value, but that value can be an object or array holding several results. With destructuring (Module 12) the caller can unpack them in one line.

RETURNING SEVERAL VALUES main.js

```
function stats(numbers) {
  const total = numbers.reduce((a, b) => a + b, 0);
  return {
    min: Math.min(...numbers),
    max: Math.max(...numbers),
    average: total / numbers.length,
  };
}
const result = stats([72, 85, 91, 64]);
console.log(result);
const { min, max } = stats([5, 9, 2]);              // destructuring the returned object
console.log(min, max);

function divide(a, b) { return [Math.floor(a / b), a % b]; }
const [quotient, remainder] = divide(17, 5);         // returning an array
console.log(`17 / 5 = ${quotient} remainder ${remainder}`);
```

OUTPUT

```
{ min: 64, max: 91, average: 78 }
2 9
17 / 5 = 3 remainder 2
```

6.4 Functions as values

6.4.1 Assigning functions to variables

Because functions are values, a function can be stored in a variable, an array element or an object property, copied to another variable and compared. The variable holds a reference to the function object; adding parentheses calls it.

FUNCTIONS STORED LIKE ANY OTHER VALUE main.js

```
function shout(text) { return text.toUpperCase() + "!"; }
const yell = shout;                       // no parentheses: the function itself
console.log(yell("hello"), yell === shout);
const ops = { add: (a, b) => a + b, sub: (a, b) => a - b };
const pipeline = [(x) => x + 1, (x) => x * 10, String];
console.log(ops.add(2, 3), ops["sub"](9, 4));
let value = 4;
for (const step of pipeline) value = step(value);
console.log(value, typeof value);
console.log(typeof shout, shout instanceof Object);
```

OUTPUT

```
HELLO! true
5 5
50 string
function true
```

6.4.2 Callback functions

A **callback** is a function passed to another function, to be called later by it: when an event happens, when a timer fires, for each element of an array, or when data arrives. Callbacks are everywhere in JavaScript.

PASSING CALLBACKS main.js

```
function processOrder(orderId, onSuccess, onError) {
  if (orderId > 0) onSuccess(`order ${orderId} confirmed`);
  else onError("invalid order id");
}
processOrder(101, (msg) => console.log("success:", msg), (err) => console.log("error:", err));
processOrder(-5, (msg) => console.log("success:", msg), (err) => console.log("error:", err));

[3, 1, 2].forEach((n, i) => console.log(`element ${i} is ${n}`));    // array callback
setTimeout(() => console.log("timer callback after 10 ms"), 10);      // timer callback
```

OUTPUT

```
success: order 101 confirmed
error: invalid order id
element 0 is 3
element 1 is 1
element 2 is 2
timer callback after 10 ms
```

6.4.3 Higher-order functions

A **higher-order function** takes a function as an argument or returns a function (or both). Array methods such as `map`, `filter` and `reduce` are higher-order functions. Writing your own lets you separate the general mechanism (repeat, retry, measure, transform) from the specific action.

HIGHER-ORDER FUNCTIONS

main.js

```
function repeat(times, action) {          // takes a function
  for (let i = 1; i <= times; i++) action(i);
}
repeat(3, (i) => console.log(`round ${i}`));

function applyToAll(list, transform) {
  const result = [];
  for (const item of list) result.push(transform(item));
  return result;
}
console.log(applyToAll([1, 2, 3], (n) => n * n));
console.log(applyToAll(["a", "b"], (s) => s.toUpperCase()));

const prices = [120, 450, 80, 999];
console.log(prices.filter((p) => p > 100).map((p) => p * 1.18).map(Math.round));
```

OUTPUT

```
round 1
round 2
round 3
[ 1, 4, 9 ]
[ 'A', 'B' ]
[ 142, 531, 1179 ]
```

6.4.4 Returning functions from functions

A function can create and return another function. The returned function remembers the variables of the function that created it, even after that function has returned: this is a **closure** (Module 11). Function factories build customised functions from a template.

FUNCTION FACTORIES

main.js

```
function makeMultiplier(factor) {
  return (n) => n * factor;               // remembers factor
}
const double = makeMultiplier(2);
const triple = makeMultiplier(3);
console.log(double(10), triple(10));

function makeGreeter(greeting) {
  return function (name) { return `${greeting}, ${name}!`; };
}
const hello = makeGreeter("Hello"), namaste = makeGreeter("Namaste");
console.log(hello("Asha"), namaste("Ravi"));

const add = (a) => (b) => a + b;          // curried function
console.log(add(5)(3), [1, 2, 3].map(add(10)));
```

OUTPUT

```
20 30
Hello, Asha! Namaste, Ravi!
8 [ 11, 12, 13 ]
```

6.5 Arrow functions vs regular functions

6.5.1 Syntax differences

	Regular function	Arrow function
Syntax	`function f(a) { return a; }`	`const f = (a) => a;`
Implicit return	no	yes, for expression bodies
Own `this`	yes: depends on how it is called	no: uses `this` of the surrounding code
Own `arguments`	yes	no
Can be used with `new`	yes (constructor)	no
Hoisted (declarations)	yes	no (it is an expression)
Good for	methods, constructors, functions needing their own this	callbacks, short helpers, keeping outer this

6.5.2 No own this or arguments (detailed in Module 11)

A regular function gets its own `this`, decided by how it is called: as a method, `this` is the object before the dot. An arrow function has no `this` of its own; it uses the `this` of the code around it. That makes arrows perfect for callbacks inside methods, and wrong for methods themselves.

THIS IN REGULAR AND ARROW FUNCTIONS

main.js

```
const timer = {
  name: "pomodoro",
  seconds: 0,
  regularMethod() { return `regular: this.name = ${this.name}`; },
  arrowMethod: () => `arrow: this.name = ${this.name} (this is the surrounding this, not timer)`,
  start() {
    [1, 2, 3].forEach(() => this.seconds++);          // arrow callback: uses start's this
    return `after start: ${this.seconds} seconds`;
  },
};
console.log(timer.regularMethod());
console.log(timer.arrowMethod());
console.log(timer.start());

try {
  const Arrow = () => {};
  new Arrow();
} catch (e) {
  console.log(`${e.name}: ${e.message}`);
}
```

OUTPUT

```
regular: this.name = pomodoro
arrow: this.name = undefined (this is the surrounding this, not timer)
after start: 3 seconds
TypeError: Arrow is not a constructor
```

6.6 Recursion

6.6.1 Base case and recursive case

A **recursive** function calls itself. It needs a **base case** that returns directly without recursion, and a **recursive case** that calls the function on a smaller problem, moving towards the base case. Recursion fits problems defined in terms of smaller versions of themselves: nested data, trees, divide and conquer.

BASE CASE AND RECURSIVE CASE main.js

```
function countdown(n) {
  if (n === 0) {                    // base case
    console.log("Go!");
    return;
  }
  console.log(n);
  countdown(n - 1);                 // recursive case: a smaller problem
}
countdown(3);

function sumTo(n) {
  return n <= 0 ? 0 : n + sumTo(n - 1);
}
console.log("sum 1..100 =", sumTo(100));
```

OUTPUT

```
3
2
1
Go!
sum 1..100 = 5050
```

6.6.2 Call stack and "Maximum call stack size exceeded"

Each function call pushes a frame onto the **call stack**; returning pops it. A recursive call to `factorial(4)` stacks frames for 4, 3, 2 and 1 before any of them returns. The stack is limited (around ten thousand frames in V8, depending on frame size); recursion without a reachable base case, or very deep recursion, throws **RangeError: Maximum call stack size exceeded**. Use a loop for deep linear recursion.

WATCHING THE CALL STACK, AND OVERFLOWING IT

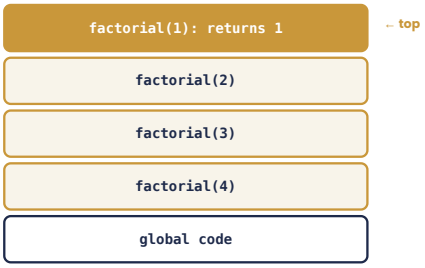
main.js

```
function factorial(n, depth = 0) {
  console.log(`${"  ".repeat(depth)}factorial(${n})`);
  const result = n <= 1 ? 1 : n * factorial(n - 1, depth + 1);
  console.log(`${"  ".repeat(depth)}returns ${result}`);
  return result;
}
factorial(4);

function infinite(n) { return infinite(n + 1); }      // no base case
try {
  infinite(0);
} catch (e) {
  console.log(`${e.name}: ${e.message}`);
}
```

OUTPUT

```
factorial(4)
  factorial(3)
    factorial(2)
      factorial(1)
      returns 1
    returns 2
  returns 6
returns 24
RangeError: Maximum call stack size exceeded
```



THE DEEPEST CALL (THE BASE CASE) RETURNS FIRST

6.6.3 Problems: factorial, Fibonacci, sum of an array, flatten a nested array, reverse a string

CLASSIC RECURSIVE PROBLEMS

main.js

```
const factorial = (n) => (n <= 1 ? 1 : n * factorial(n - 1));
const fib = (n) => (n < 2 ? n : fib(n - 1) + fib(n - 2));
const sumArray = (arr) => (arr.length === 0 ? 0 : arr[0] + sumArray(arr.slice(1)));
const reverse = (s) => (s === "" ? "" : reverse(s.slice(1)) + s[0]);

function flatten(arr) {
  const result = [];
  for (const item of arr) {
    if (Array.isArray(item)) result.push(...flatten(item));   // recurse into nested arrays
    else result.push(item);
  }
  return result;
}

console.log("6! =", factorial(6));
console.log("fib(0..10):", Array.from({ length: 11 }, (_, i) => fib(i)).join(" "));
console.log("sum:", sumArray([4, 8, 15, 16, 23, 42]));
console.log("reverse:", reverse("recursion"));
console.log("flatten:", flatten([1, [2, [3, [4, [5]]]], 6]));
console.log("built-in flat:", [1, [2, [3, [4, [5]]]], 6].flat(Infinity));

const memo = new Map();
function fastFib(n) {                                   // memoised: each value computed once
  if (n < 2) return n;
  if (memo.has(n)) return memo.get(n);
  const v = fastFib(n - 1) + fastFib(n - 2);
  memo.set(n, v);
  return v;
}
console.log("fib(78) =", fastFib(78), "(the largest Fibonacci number below MAX_SAFE_INTEGER)");
```

OUTPUT

```
6! = 720
fib(0..10): 0 1 1 2 3 5 8 13 21 34 55
sum: 108
reverse: noisrucer
flatten: [ 1, 2, 3, 4, 5, 6 ]
built-in flat: [ 1, 2, 3, 4, 5, 6 ]
fib(78) = 8944394323791464 (the largest Fibonacci number below MAX_SAFE_INTEGER)
```

NOTE

Naive recursive Fibonacci recomputes the same values over and over (fib(40) makes more than 300 million calls); memoising with a Map makes it linear.

6.7 Pure functions and side effects (introduction)

A **pure function** (1) always returns the same output for the same inputs and (2) has no **side effects**: it does not modify its arguments, global variables, the DOM or files, and does not depend on anything that changes (the clock, random numbers, external state). Pure functions are easy to test, reason about, cache and reuse. Real programs need side effects (printing, saving, updating the page), but keeping most logic pure and pushing side effects to the edges makes code more reliable. Functional programming builds on this idea (Module 20).

PURE VERSUS IMPURE FUNCTIONS

main.js

```
// pure: output depends only on input, nothing else changes
const addTax = (price, rate) => price * (1 + rate);

// impure: depends on and modifies outside state
let total = 0;
function addToTotal(price) { total += price; return total; }

// impure: mutates its argument
function addItemImpure(cart, item) { cart.push(item); return cart; }
// pure alternative: returns a new array
const addItem = (cart, item) => [...cart, item];

console.log(addTax(100, 0.18), addTax(100, 0.18));          // always the same
console.log(addToTotal(100), addToTotal(100));              // different results for the same input
const cart = ["pen"];
const newCart = addItem(cart, "book");
console.log(cart, newCart);
addItemImpure(cart, "bag");
console.log(cart);
```

OUTPUT

```
118 118
100 200
[ 'pen' ] [ 'pen', 'book' ]
[ 'pen', 'bag' ]
```

Define functions with declarations (hoisted), expressions (values assigned at run time), arrow functions (short, implicit return, no own this) or IIFEs (run immediately, private scope). Parameters receive arguments; missing ones are undefined, defaults fill them, rest parameters collect extras (prefer them to arguments). Objects are passed as references, so mutating them affects the caller. return ends a function (undefined by default) and can return objects or arrays for several results. Functions are values: store them, pass them as callbacks, write higher-order functions and return functions (closures). Recursion needs a base case and is limited by the call stack. Prefer pure functions without side effects.

PRACTICE CHECKPOINT

Write a set of **utility functions** (isPrime, factorial, capitalize) as **declarations, expressions and arrow functions**, a function that **accepts a callback**, and a **recursive array flattener**.

SAMPLE SOLUTION

main.js

```
// declaration
function isPrime(n) {
  if (!Number.isInteger(n) || n < 2) return false;
  for (let d = 2; d * d <= n; d++) if (n % d === 0) return false;
  return true;
}
// function expression
const factorial = function fact(n) {
  return n <= 1 ? 1 : n * fact(n - 1);
};
// arrow function
const capitalize = (text = "") => text.charAt(0).toUpperCase() + text.slice(1).toLowerCase();

// higher-order function that accepts a callback
function processAll(items, callback, label = "result") {
  return items.map((item, index) => `${label} ${index + 1}: ${callback(item)}`);
}

// recursive flattener with an optional depth
function flatten(arr, depth = Infinity) {
  return arr.reduce((flat, item) =>
    Array.isArray(item) && depth > 0 ? flat.concat(flatten(item, depth - 1)) : flat.concat([item]),
[]);
}

console.log([2, 15, 17, 21, 97].map(isPrime).join(" "));
console.log([0, 5, 10].map(factorial).join(" "));
console.log(["jAVAsCRIPT", "nODE", ""].map((t) => capitalize(t)));
processAll([7, 8, 9], isPrime, "is prime").forEach((line) => console.log(line));
processAll(["asha", "ravi"], capitalize, "name").forEach((line) => console.log(line));
console.log(flatten([1, [2, [3, [4, [5, [6]]]]]]));
console.log(flatten([1, [2, [3, [4]]]], 1));
```

OUTPUT

```
true false true false true
1 120 3628800
[ 'Javascript', 'Node', '' ]
is prime 1: true
is prime 2: false
is prime 3: false
name 1: Asha
name 2: Ravi
[ 1, 2, 3, 4, 5, 6 ]
[ 1, 2, [ 3, [ 4 ] ] ]
```

MODULE 07 · BEGINNER

Arrays

Minimum duration **3 days** | Level **Beginner**

Arrays are ordered lists that can hold values of any type, and their built-in methods are used constantly in real JavaScript code. This module covers creating and indexing arrays, adding and removing elements, searching, the iteration methods map, filter and reduce, sorting (and its famous trap), the newer non-mutating methods, grouping, copying, and classic array problems.

IN THIS MODULE

7.1 Array basics
7.2 Adding and removing
7.3 Searching
7.4 Iteration methods
7.5 Sorting and reordering
7.6 Other methods
7.7 Mutating vs non-mutating methods
7.8 Copying arrays
7.9 Array problems

7.1 Array basics

An **array** stores an ordered list of values, called **elements**, each identified by an **index** starting at 0. JavaScript arrays are dynamic (they grow and shrink automatically) and can mix types, although in practice an array usually holds values of one kind.



AN ARRAY OF FOUR STRINGS; STUDENTS[2] IS "MEERA" AND STUDENTS.LENGTH IS 4

7.1.1 Creating arrays: literal, Array(), Array.of(), Array.from()

The **array literal** `[ ]` is the normal way to create an array. The `Array()` constructor has a trap: with a single number it creates an empty array of that **length** (with "holes"), not an array containing the number. `Array.of()` always creates an array of its arguments. `Array.from()` builds an array from any iterable or array-like object (strings, Sets, NodeLists, `{ length: n }`), optionally mapping each element.

WAYS TO CREATE AN ARRAY

main.js

```
const fruits = ["mango", "apple", "kiwi"];          // literal
const mixed = [42, "text", true, null, { id: 1 }, [1, 2]];
const empty = [];
console.log(fruits, mixed.length, empty.length);

console.log(Array(3), Array(3).length);              // trap: 3 empty slots
console.log(Array(1, 2, 3), Array.of(3));            // several arguments vs Array.of

console.log(Array.from("hello"));                    // from a string
console.log(Array.from(new Set([1, 2, 2, 3])));      // from a Set
console.log(Array.from({ length: 5 }, (_, i) => i * i));   // generate with a mapping function
console.log(new Array(4).fill(0));
```

OUTPUT

```
[ 'mango', 'apple', 'kiwi' ] 6 0
[ <3 empty items> ] 3
[ 1, 2, 3 ] [ 3 ]
[ 'h', 'e', 'l', 'l', 'o' ]
[ 1, 2, 3 ]
[ 0, 1, 4, 9, 16 ]
[ 0, 0, 0, 0 ]
```

7.1.2 Indexing, length and at() with negative indexes

Read or write an element with `arr[index]`. `length` is one more than the highest index; reading beyond the end gives `undefined` (no error). Writing at an index beyond the end extends the array (leaving holes), and setting `length` truncates it. `at(index)` (ES2022) accepts **negative** indexes counting from the end: `arr.at(-1)` is the last element.

INDEXING, LENGTH AND AT()

main.js

```
const marks = [72, 85, 91, 64, 78];
console.log(marks[0], marks[2], marks[marks.length - 1], marks[10]);
console.log(marks.at(-1), marks.at(-2), marks.at(0));
marks[1] = 88;                    // update
console.log(marks);
marks[7] = 50;                    // beyond the end: holes at 5 and 6
console.log(marks, marks.length);
marks.length = 3;                 // truncate
console.log(marks);
```

OUTPUT

```
72 91 78 undefined
78 64 72
[ 72, 88, 91, 64, 78 ]
[ 72, 88, 91, 64, 78, <2 empty items>, 50 ] 8
[ 72, 88, 91 ]
```

7.1.3 Arrays are objects; Array.isArray()

Arrays are a special kind of **object** whose keys are numeric strings ("0", "1", …) and which maintain a `length`. Hence `typeof []` is `"object"`, arrays are copied by reference, and two arrays with the same elements are not `===`. Use `Array.isArray()` to test whether a value is an array.

ARRAYS ARE OBJECTS

main.js

```
const a = [1, 2, 3];
console.log(typeof a, Array.isArray(a), Array.isArray({ length: 3 }));
console.log(Object.keys(a));
const b = a;                        // same array
b.push(4);
console.log(a, a === b, [1, 2] === [1, 2]);
```

OUTPUT

```
object true false
[ '0', '1', '2' ]
[ 1, 2, 3, 4 ] true false
```

7.1.4 Multi-dimensional arrays

JavaScript has no special matrix type, but an array can contain arrays: `grid[row][col]`. Rows can have different lengths (a "jagged" array). Build an empty grid with `Array.from`, not `Array(n).fill([])`, which would make every row the same array.

ARRAYS OF ARRAYS

main.js

```
const matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9],
];
console.log(matrix[1][2], matrix.length, matrix[0].length);
for (const row of matrix) console.log(row.join(" "));

const transposed = matrix[0].map((_, c) => matrix.map((row) => row[c]));
console.log(transposed);

const wrong = Array(3).fill([]);          // three references to ONE array
wrong[0].push("x");
console.log(wrong);
const right = Array.from({ length: 3 }, () => []);
right[0].push("x");
console.log(right);
```

OUTPUT

```
6 3 3
1 2 3
4 5 6
7 8 9
[ [ 1, 4, 7 ], [ 2, 5, 8 ], [ 3, 6, 9 ] ]
[ [ 'x' ], [ 'x' ], [ 'x' ] ]
[ [ 'x' ], [], [] ]
```

7.2 Adding and removing

7.2.1 push(), pop(), unshift(), shift()

`push` adds elements at the end and returns the new length; `pop` removes and returns the last element. `unshift` adds at the start and `shift` removes from the start. Working at the end is fast (O(1)); working at the start must move every element (O(n)). push/pop make an array a **stack**; push/shift make it a **queue**.

ADDING AND REMOVING AT THE ENDS main.js

```
const queue = ["Asha", "Ravi"];
console.log(queue.push("Meera", "Kiran"), queue);   // returns the new length
console.log(queue.pop(), queue);                    // removes the last
console.log(queue.unshift("Zoya"), queue);          // adds at the start
console.log(queue.shift(), queue);                  // removes the first
console.log([].pop(), [].shift());                  // empty array: undefined
```

OUTPUT

```
4 [ 'Asha', 'Ravi', 'Meera', 'Kiran' ]
Kiran [ 'Asha', 'Ravi', 'Meera' ]
4 [ 'Zoya', 'Asha', 'Ravi', 'Meera' ]
Zoya [ 'Asha', 'Ravi', 'Meera' ]
undefined undefined
```



PUSH/POP WORK AT THE END, UNSHIFT/SHIFT AT THE START

7.2.2 splice()

`splice(start, deleteCount, ...items)` is the Swiss-army knife: it removes `deleteCount` elements from position `start`, optionally inserts new items there, **modifies the array in place** and returns an array of the removed elements. A negative start counts from the end.

REMOVING, INSERTING AND REPLACING WITH SPLICE main.js

```
const days = ["Mon", "Tue", "Wed", "Thu", "Fri"];
const removed = days.splice(1, 2);                  // remove 2 from index 1
console.log(removed, days);
days.splice(1, 0, "Tue", "Wed");                    // insert without removing
console.log(days);
days.splice(2, 1, "WEDNESDAY");                     // replace one element
console.log(days);
days.splice(-1, 1);                                 // remove the last
console.log(days);
```

OUTPUT

```
[ 'Tue', 'Wed' ] [ 'Mon', 'Thu', 'Fri' ]
[ 'Mon', 'Tue', 'Wed', 'Thu', 'Fri' ]
[ 'Mon', 'Tue', 'WEDNESDAY', 'Thu', 'Fri' ]
[ 'Mon', 'Tue', 'WEDNESDAY', 'Thu' ]
```

7.2.3 slice()

`slice(start, end)` returns a **new array** with the elements from start up to (not including) end, leaving the original unchanged. Negative indexes count from the end, and `slice()` with no arguments copies the whole array. Do not confuse s**l**ice (copy a part) with s**p**lice (change in place).

EXTRACTING PART OF AN ARRAY WITH SLICE main.js

```
const letters = ["a", "b", "c", "d", "e", "f"];
console.log(letters.slice(1, 4));       // b c d
console.log(letters.slice(3));          // from index 3 to the end
console.log(letters.slice(-2));         // the last two
console.log(letters.slice(1, -1));      // drop first and last
console.log(letters);                   // unchanged
```

OUTPUT

```
[ 'b', 'c', 'd' ]
[ 'd', 'e', 'f' ]
[ 'e', 'f' ]
[ 'b', 'c', 'd', 'e' ]
[ 'a', 'b', 'c', 'd', 'e', 'f' ]
```

7.2.4 concat() and spread

`concat` returns a new array joining the array with other arrays or values. The spread syntax `[...a, ...b]` does the same more flexibly, and is the modern way to combine arrays or insert elements at any position.

COMBINING ARRAYS main.js

```
const frontEnd = ["HTML", "CSS"], backEnd = ["Node", "SQL"];
console.log(frontEnd.concat(backEnd, "Git"));
console.log([...frontEnd, "JavaScript", ...backEnd]);
console.log(["start", ...frontEnd, "end"]);
console.log(frontEnd);                   // originals unchanged
```

OUTPUT

```
[ 'HTML', 'CSS', 'Node', 'SQL', 'Git' ]
[ 'HTML', 'CSS', 'JavaScript', 'Node', 'SQL' ]
[ 'start', 'HTML', 'CSS', 'end' ]
[ 'HTML', 'CSS' ]
```

7.3 Searching

7.3.1 indexOf(), lastIndexOf(), includes()

`indexOf(value)` returns the first index of a value (compared with `===`) or -1; `lastIndexOf` searches from the end; `includes(value)` returns true or false and, unlike indexOf, finds `NaN`. These compare values, so they cannot find objects by their contents; use `find` for that.

SEARCHING BY VALUE

main.js

```
const nums = [5, 12, 8, 12, NaN, 3];
console.log(nums.indexOf(12), nums.lastIndexOf(12), nums.indexOf(99));
console.log(nums.includes(8), nums.includes("8"), nums.includes(NaN), nums.indexOf(NaN));
console.log(nums.indexOf(12, 2));                  // start searching at index 2
const users = [{ id: 1 }, { id: 2 }];
console.log(users.includes({ id: 1 }));            // a different object: false
```

OUTPUT

```
1 3 -1
true false true -1
3
false
```

7.3.2 find(), findIndex(), findLast(), findLastIndex()

These take a **callback** that tests each element. `find` returns the first element for which the callback returns truthy (or `undefined`); `findIndex` returns its index (or -1). `findLast` and `findLastIndex` (ES2023) search from the end.

SEARCHING WITH A TEST FUNCTION

main.js

```
const products = [
  { name: "Pen", price: 10, stock: 100 },
  { name: "Notebook", price: 45, stock: 0 },
  { name: "Bag", price: 899, stock: 12 },
  { name: "Ruler", price: 15, stock: 0 },
];
console.log(products.find((p) => p.price > 40));
console.log(products.findIndex((p) => p.stock === 0));
console.log(products.findLast((p) => p.stock === 0).name, products.findLastIndex((p) => p.stock ===
0));
console.log(products.find((p) => p.price > 5000));
```

OUTPUT

```
{ name: 'Notebook', price: 45, stock: 0 }
1
Ruler 3
undefined
```

7.3.3 some(), every()

`some(callback)` returns true if **at least one** element passes the test; `every(callback)` returns true if **all** do. Both stop as soon as the answer is known. On an empty array, some returns false and every returns true.

SOME AND EVERY

main.js

```
const marks = [72, 85, 38, 91];
console.log("anyone failed?", marks.some((m) => m < 40));
console.log("all passed?", marks.every((m) => m >= 40));
console.log("all numbers?", marks.every((m) => typeof m === "number"));
console.log([].some(() => true), [].every(() => false));
```

OUTPUT

```
anyone failed? true
all passed? false
all numbers? true
false true
```

7.4 Iteration methods

The iteration methods take a callback that receives `(element, index, array)`. They express **what** you want (transform, select, combine) rather than how to loop, and they are the heart of modern JavaScript data processing.

7.4.1 forEach()

`forEach` calls the callback for every element, for its side effects; it returns `undefined`. You cannot stop it early with break (use `for...of`, `some` or `find` when you need to), and it does not wait for async callbacks.

FOREACH

main.js

```
const cities = ["Pune", "Delhi", "Chennai"];
cities.forEach((city, index) => console.log(`${index + 1}. ${city}`));
const result = cities.forEach((c) => c);
console.log("forEach returns:", result);
```

OUTPUT

```
1. Pune
2. Delhi
3. Chennai
forEach returns: undefined
```

7.4.2 map()

`map` creates a **new array** of the same length by transforming each element with the callback. It never changes the original. Use it whenever you want "the same list, but each item changed": prices with tax, names in uppercase, objects to strings, data to HTML.

TRANSFORMING EVERY ELEMENT WITH MAP

main.js

```
const prices = [100, 250, 80];
console.log(prices.map((p) => p * 1.18));
console.log(prices.map((p, i) => `#${i + 1}: Rs ${p}`));
const students = [{ name: "Asha", marks: 91 }, { name: "Ravi", marks: 78 }];
console.log(students.map((s) => s.name));
console.log(students.map((s) => ({ ...s, passed: s.marks >= 40 })));
console.log(["1", "2", "3"].map(Number), ["1", "2", "3"].map(parseInt));   // the parseInt trap
```

OUTPUT

```
[ 118, 295, 94.39999999999999 ]
[ '#1: Rs 100', '#2: Rs 250', '#3: Rs 80' ]
[ 'Asha', 'Ravi' ]
[
  { name: 'Asha', marks: 91, passed: true },
  { name: 'Ravi', marks: 78, passed: true }
]
[ 1, 2, 3 ] [ 1, NaN, NaN ]
```

NOTE

`["1","2","3"].map(parseInt)` gives `[1, NaN, NaN]` because map passes the index as the second argument, which parseInt treats as the radix. Use `map(Number)` or `map((s) => parseInt(s, 10))`.

7.4.3 filter()

`filter` creates a new array with only the elements for which the callback returns truthy. The result may be shorter (or empty); the original is unchanged.

SELECTING ELEMENTS WITH FILTER main.js

```
const nums = [3, 8, 12, 5, 20, 7];
console.log(nums.filter((n) => n % 2 === 0));
console.log(nums.filter((n) => n > 100));
const words = ["", "apple", null, "kiwi", undefined, 0];
console.log(words.filter(Boolean));                          // remove falsy values
const products = [{ name: "Pen", stock: 100 }, { name: "Notebook", stock: 0 }, { name: "Bag", stock:
12 }];
console.log(products.filter((p) => p.stock > 0).map((p) => p.name));
```

OUTPUT

```
[ 8, 12, 20 ]
[]
[ 'apple', 'kiwi' ]
[ 'Pen', 'Bag' ]
```

7.4.4 reduce() and reduceRight()

`reduce(callback, initialValue)` combines all elements into a **single value**: a sum, a maximum, an object, a string, even another array. The callback receives the **accumulator** (the result so far) and the current element, and returns the new accumulator. Always pass an initial value: without it, reduce uses the first element and throws on an empty array. `reduceRight` works from the last element to the first.

COMBINING ELEMENTS WITH REDUCE main.js

```
const nums = [5, 10, 15, 20];
const sum = nums.reduce((acc, n) => acc + n, 0);
const max = nums.reduce((acc, n) => (n > acc ? n : acc), -Infinity);
console.log(sum, max);

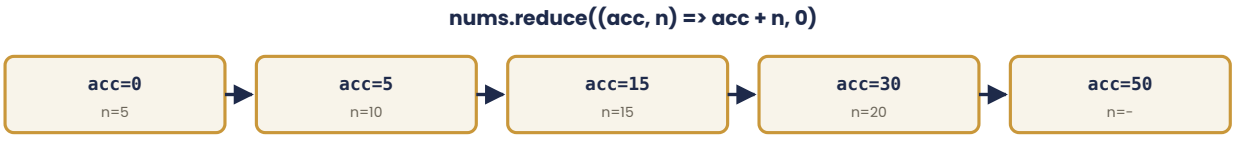
const cart = [{ item: "Pen", price: 10, qty: 3 }, { item: "Bag", price: 899, qty: 1 }];
console.log("total:", cart.reduce((total, line) => total + line.price * line.qty, 0));

const fruits = ["apple", "kiwi", "apple", "mango", "kiwi", "apple"];
const counts = fruits.reduce((acc, f) => { acc[f] = (acc[f] ?? 0) + 1; return acc; }, {});
console.log(counts);

console.log(["a", "b", "c"].reduceRight((acc, ch) => acc + ch, ""));
try { [].reduce((a, b) => a + b); } catch (e) { console.log(`${e.name}: ${e.message}`); }
```

OUTPUT

```
50 20
total: 929
{ apple: 3, kiwi: 2, mango: 1 }
cba
TypeError: Reduce of empty array with no initial value
```



HOW REDUCE CARRIES THE ACCUMULATOR THROUGH THE ARRAY

7.4.5 Chaining methods

Because map, filter, slice and similar methods return arrays, they can be **chained** into a readable pipeline. Put filter before map when possible, so less work is done, and format chains one method per line.

A DATA PIPELINE WITH CHAINED METHODS main.js

```
const orders = [
  { id: 1, customer: "Asha", amount: 1200, status: "delivered" },
  { id: 2, customer: "Ravi", amount: 450, status: "cancelled" },
  { id: 3, customer: "Meera", amount: 3100, status: "delivered" },
  { id: 4, customer: "Kiran", amount: 800, status: "delivered" },
];
const report = orders
  .filter((o) => o.status === "delivered")
  .map((o) => ({ ...o, amount: o.amount * 1.18 }))
  .sort((a, b) => b.amount - a.amount)
  .map((o) => `${o.customer}: Rs ${o.amount.toFixed(2)}`);
console.log(report);
const revenue = orders.filter((o) => o.status === "delivered").reduce((s, o) => s + o.amount, 0);
console.log("revenue:", revenue);
```

OUTPUT

```
[ 'Meera: Rs 3658.00', 'Asha: Rs 1416.00', 'Kiran: Rs 944.00' ]
revenue: 5100
```

7.4.6 flat() and flatMap()

`flat(depth)` (ES2019) returns a new array with nested arrays flattened to the given depth (default 1; `Infinity` for all levels). `flatMap(callback)` maps each element to an array and flattens one level, useful when each input produces zero, one or several outputs.

FLATTENING NESTED ARRAYS main.js

```
const nested = [1, [2, 3], [4, [5, [6]]]];
console.log(nested.flat(), nested.flat(2), nested.flat(Infinity));
const sentences = ["learn javascript", "build projects"];
console.log(sentences.map((s) => s.split(" ")));
console.log(sentences.flatMap((s) => s.split(" ")));
const items = [{ name: "kit", parts: ["a", "b"] }, { name: "solo", parts: [] }];
console.log(items.flatMap((i) => i.parts));
```

OUTPUT

```
[ 1, 2, 3, 4, [ 5, [ 6 ] ] ] [ 1, 2, 3, 4, 5, [ 6 ] ] [ 1, 2, 3, 4, 5, 6 ]
[ [ 'learn', 'javascript' ], [ 'build', 'projects' ] ]
[ 'learn', 'javascript', 'build', 'projects' ]
[ 'a', 'b' ]
```

7.5 Sorting and reordering

7.5.1 sort() and comparator functions (the number sort trap)

`sort()` sorts an array **in place** and returns it. Without arguments it converts elements to **strings** and compares their UTF-16 codes, which sorts numbers wrongly: `[10, 9, 1, 100].sort()` gives `[1, 10, 100, 9]`. Pass a **comparator** `(a, b) => ...` that returns a negative number if a should come first, positive if b

should, and 0 if equal: `(a, b) => a - b` sorts numbers ascending. Use `localeCompare` for text. Sorting is stable since ES2019.

SORTING WITH COMPARATORS main.js

```
console.log([10, 9, 1, 100, 25].sort());                    // the trap: string order
console.log([10, 9, 1, 100, 25].sort((a, b) => a - b));     // ascending
console.log([10, 9, 1, 100, 25].sort((a, b) => b - a));     // descending

const names = ["Zoya", "asha", "Ravi", "meera"];
console.log([...names].sort());                             // uppercase first
console.log([...names].sort((a, b) => a.localeCompare(b))); // natural order

const students = [{ name: "Asha", marks: 85 }, { name: "Ravi", marks: 92 }, { name: "Meera", marks: 85
}];
students.sort((a, b) => b.marks - a.marks || a.name.localeCompare(b.name));   // marks desc, then name
console.log(students.map((s) => `${s.name}:${s.marks}`).join(" "));
```

OUTPUT

```
[ 1, 10, 100, 25, 9 ]
[ 1, 9, 10, 25, 100 ]
[ 100, 25, 10, 9, 1 ]
[ 'Ravi', 'Zoya', 'asha', 'meera' ]
[ 'asha', 'meera', 'Ravi', 'Zoya' ]
Ravi:92 Asha:85 Meera:85
```

7.5.2 reverse()

`reverse()` reverses the order of the elements **in place** and returns the array.

REVERSING IN PLACE main.js

```
const steps = [1, 2, 3, 4];
steps.reverse();
console.log(steps);
console.log("palindrome?", [..."level"].reverse().join("") === "level");
```

OUTPUT

```
[ 4, 3, 2, 1 ]
palindrome? true
```

7.5.3 Non-mutating versions: toSorted(), toReversed(), toSpliced(), with()

ES2023 added **copying** versions of the mutating methods: `toSorted`, `toReversed`, `toSpliced` and `with(index, value)` return a new array and leave the original untouched. They are ideal where data must not be changed, such as React state.

COPYING ARRAY METHODS main.js

```
const original = [3, 1, 2];
const sorted = original.toSorted((a, b) => a - b);
const reversed = original.toReversed();
const spliced = original.toSpliced(1, 1, "x", "y");
const replaced = original.with(0, 99);
console.log(original, sorted, reversed, spliced, replaced);
```

OUTPUT

```
[ 3, 1, 2 ] [ 1, 2, 3 ] [ 2, 1, 3 ] [ 3, 'x', 'y', 2 ] [ 99, 1, 2 ]
```

7.6 Other methods

7.6.1 join(), fill(), keys(), values(), entries()

`join(separator)` combines elements into a string (default separator ","). `fill(value, start, end)` overwrites elements with a value in place. `keys()`, `values()` and `entries()` return iterators over indexes, elements and [index, element] pairs, used with for...of or spread.

JOIN, FILL AND ITERATORS main.js

```
const words = ["learn", "being", "forward"];
console.log(words.join(" "), "|", words.join("-"), "|", words.join());
console.log(new Array(5).fill("-").join(""), [1, 2, 3, 4, 5].fill(0, 1, 3));
console.log([...words.keys()], [...words.values()]);
for (const [i, w] of words.entries()) console.log(i, w);
```

OUTPUT

```
learn being forward | learn-being-forward | learn,being,forward
----- [ 1, 0, 0, 4, 5 ]
[ 0, 1, 2 ] [ 'learn', 'being', 'forward' ]
0 learn
1 being
2 forward
```

7.6.2 Object.groupBy()

`Object.groupBy(array, callback)` (ES2024) groups elements into an object whose keys are the values returned by the callback. `Map.groupBy` does the same into a Map, allowing any key type.

GROUPING WITH OBJECT.GROUPBY AND MAP.GROUPBY main.js

```
const students = [
  { name: "Asha", dept: "CSE", marks: 91 },
  { name: "Ravi", dept: "ECE", marks: 72 },
  { name: "Meera", dept: "CSE", marks: 85 },
  { name: "Kiran", dept: "MECH", marks: 38 },
];
const byDept = Object.groupBy(students, (s) => s.dept);
console.log(Object.keys(byDept), byDept.CSE.map((s) => s.name));
const byResult = Object.groupBy(students, (s) => (s.marks >= 40 ? "pass" : "fail"));
console.log({ pass: byResult.pass.length, fail: byResult.fail.length });
const map = Map.groupBy([1, 2, 3, 4, 5, 6], (n) => n % 3);
console.log(map);
```

OUTPUT

```
[ 'CSE', 'ECE', 'MECH' ] [ 'Asha', 'Meera' ]
{ pass: 3, fail: 1 }
Map(3) { 1 => [ 1, 4 ], 2 => [ 2, 5 ], 0 => [ 3, 6 ] }
```

7.7 Mutating vs non-mutating methods

Some array methods change the original array (**mutating**), others return a new array or value (**non-mutating**). Mixing them up causes bugs where data changes unexpectedly, for example when an array is shared between parts of a program or stored in a UI framework's state.

Mutating (change the array)	Non-mutating (return something new)
`push pop shift unshift`	`concat slice`, spread `[...a]`
`splice`	`toSpliced`
`sort reverse`	`toSorted toReversed`
`fill copyWithin`	`with`
assignment `arr[i] = x`, `length = n`	`map filter reduce flat flatMap find some every join includes indexOf at`

MUTATION CAN SURPRISE YOU

main.js

```
const scores = [40, 10, 30];
const sortedCopy = scores.toSorted((a, b) => a - b);   // safe
console.log(scores, sortedCopy);
const sortedInPlace = scores.sort((a, b) => a - b);    // changes scores too!
console.log(scores, sortedInPlace === scores);
```

OUTPUT

```
[ 40, 10, 30 ] [ 10, 30, 40 ]
[ 10, 30, 40 ] true
```

7.8 Copying arrays

7.8.1 Shallow copy with spread and slice

`[...arr]`, `arr.slice()` and `Array.from(arr)` create a **shallow copy**: a new array whose elements are the same values. For primitives that is a full copy, but objects inside the array are still **shared** between the original and the copy.

A SHALLOW COPY SHARES NESTED OBJECTS

main.js

```
const original = [1, 2, { name: "Asha" }];
const copy = [...original];
copy[0] = 100;                       // changing a primitive element: independent
copy[2].name = "Changed";            // changing a nested object: shared!
console.log(original, copy, original[2] === copy[2]);
```

OUTPUT

```
[ 1, 2, { name: 'Changed' } ] [ 100, 2, { name: 'Changed' } ] true
```

7.8.2 Deep copy with structuredClone()

`structuredClone(value)` (available in browsers and Node.js 17+) creates a **deep copy**: nested arrays and objects are copied too, so nothing is shared. It also handles Dates, Maps, Sets and circular references, but not functions, DOM nodes or class instances' prototypes. The old trick `JSON.parse(JSON.stringify(x))` loses Dates, undefined, Infinity and functions.

DEEP COPY WITH STRUCTUREDCLONE

main.js

```
const data = { tags: ["js", "web"], created: new Date(2026, 2, 9), meta: { views: 10 } };
const deep = structuredClone(data);
deep.tags.push("node");
deep.meta.views = 99;
console.log(data.tags, data.meta.views, deep.created instanceof Date);
const viaJson = JSON.parse(JSON.stringify(data));
console.log(typeof viaJson.created);       // the Date became a string
```

OUTPUT

```
[ 'js', 'web' ] 10 true
string
```

7.9 Array problems

7.9.1 Max/min, second largest, remove duplicates, frequency count

MAX/MIN, SECOND LARGEST, DUPLICATES AND FREQUENCY

main.js

```
const a = [12, 45, 7, 45, 23, 7, 89, 12, 3];
console.log("max", Math.max(...a), "min", Math.min(...a));

let first = -Infinity, second = -Infinity;           // second largest in one pass
for (const x of a) {
  if (x > first) { second = first; first = x; }
  else if (x > second && x !== first) second = x;
}
console.log("second largest", second);

console.log("unique (Set)", [...new Set(a)]);
console.log("unique (filter)", a.filter((x, i) => a.indexOf(x) === i));

const freq = new Map();
for (const x of a) freq.set(x, (freq.get(x) ?? 0) + 1);
console.log("repeated", [...freq].filter(([, c]) => c > 1).map(([v, c]) => `${v}x${c}`).join(" "));
```

OUTPUT

```
max 89 min 3
second largest 45
unique (Set) [ 12, 45, 7, 23, 89, 3 ]
unique (filter) [ 12, 45, 7, 23, 89, 3 ]
repeated 12x2 45x2 7x2
```

7.9.2 Rotate, chunk, intersection and union of arrays

ROTATE, CHUNK AND SET OPERATIONS

main.js

```
const rotateLeft = (arr, k) => { k %= arr.length; return [...arr.slice(k), ...arr.slice(0, k)]; };
const rotateRight = (arr, k) => rotateLeft(arr, arr.length - (k % arr.length));
console.log(rotateLeft([1, 2, 3, 4, 5], 2), rotateRight([1, 2, 3, 4, 5], 1));

function chunk(arr, size) {
  const out = [];
  for (let i = 0; i < arr.length; i += size) out.push(arr.slice(i, i + size));
  return out;
}
console.log(chunk([1, 2, 3, 4, 5, 6, 7], 3));

const x = [1, 2, 3, 4], y = [3, 4, 5, 6];
console.log("intersection", x.filter((v) => y.includes(v)));
console.log("union", [...new Set([...x, ...y])]);
console.log("difference", x.filter((v) => !y.includes(v)));
const sx = new Set(x), sy = new Set(y);                    // ES2025 Set methods
console.log([...sx.intersection(sy)], [...sx.union(sy)], [...sx.difference(sy)]);
```

OUTPUT

```
[ 3, 4, 5, 1, 2 ] [ 5, 1, 2, 3, 4 ]
[ [ 1, 2, 3 ], [ 4, 5, 6 ], [ 7 ] ]
intersection [ 3, 4 ]
union [ 1, 2, 3, 4, 5, 6 ]
difference [ 1, 2 ]
[ 3, 4 ] [ 1, 2, 3, 4, 5, 6 ] [ 1, 2 ]
```

7.9.3 Linear search, binary search, bubble sort

SEARCHING AND SORTING BY HAND

main.js

```
function linearSearch(arr, target) {
  for (let i = 0; i < arr.length; i++) if (arr[i] === target) return i;
  return -1;
}
function binarySearch(sorted, target) {                    // O(log n): the array must be sorted
  let lo = 0, hi = sorted.length - 1, steps = 0;
  while (lo <= hi) {
    steps++;
    const mid = Math.floor((lo + hi) / 2);
    if (sorted[mid] === target) return { index: mid, steps };
    if (sorted[mid] < target) lo = mid + 1; else hi = mid - 1;
  }
  return { index: -1, steps };
}
function bubbleSort(input) {
  const arr = [...input];
  for (let pass = 0; pass < arr.length - 1; pass++) {
    let swapped = false;
    for (let j = 0; j < arr.length - 1 - pass; j++) {
      if (arr[j] > arr[j + 1]) { [arr[j], arr[j + 1]] = [arr[j + 1], arr[j]]; swapped = true; }
    }
    console.log(`pass ${pass + 1}: ${arr.join(" ")}`);
    if (!swapped) break;
  }
  return arr;
}
const sorted = [3, 8, 12, 17, 21, 30, 42, 55, 61];
console.log(linearSearch(sorted, 42), binarySearch(sorted, 42), binarySearch(sorted, 5));
console.log(bubbleSort([29, 10, 14, 37, 13]));
```

OUTPUT

```
6 { index: 6, steps: 2 } { index: -1, steps: 3 }
pass 1: 10 14 29 13 37
pass 2: 10 14 13 29 37
pass 3: 10 13 14 29 37
pass 4: 10 13 14 29 37
[ 10, 13, 14, 29, 37 ]
```

Create arrays with literals (beware Array(n)), Array.of and Array.from; index from 0, use at(-1) for the end, and Array.isArray to test. push/pop/unshift/shift work at the ends, splice changes in place, slice copies a part, concat and spread combine. Search with indexOf/includes (values) or find/findIndex/findLast (tests), some and every. forEach runs side effects, map transforms, filter selects, reduce combines (always give an initial value), and these chain into pipelines; flat and flatMap flatten. sort converts to strings unless given a comparator such as (a, b) => a - b; toSorted, toReversed, toSpliced and with copy instead of mutating. Object.groupBy groups; spread makes shallow copies and structuredClone deep ones.

PRACTICE CHECKPOINT

Given an array of **product objects**, use **map, filter, reduce and sort** to find the **in-stock items**, the **total inventory value**, and the **three most expensive products**.

SAMPLE SOLUTION

main.js

```
const products = [
  { id: 1, name: "Wireless Mouse", category: "Electronics", price: 799, stock: 25 },
  { id: 2, name: "Notebook (A5)", category: "Stationery", price: 60, stock: 0 },
  { id: 3, name: "Backpack", category: "Bags", price: 1499, stock: 8 },
  { id: 4, name: "USB-C Cable", category: "Electronics", price: 349, stock: 40 },
  { id: 5, name: "Desk Lamp", category: "Home", price: 1299, stock: 0 },
  { id: 6, name: "Water Bottle", category: "Home", price: 450, stock: 15 },
  { id: 7, name: "Headphones", category: "Electronics", price: 2499, stock: 5 },
];

const inStock = products.filter((p) => p.stock > 0);
console.log("In stock:", inStock.map((p) => p.name).join(", "));

const totalValue = inStock.reduce((sum, p) => sum + p.price * p.stock, 0);
console.log(`Total inventory value: Rs ${totalValue.toLocaleString("en-IN")}`);

const top3 = products.toSorted((a, b) => b.price - a.price).slice(0, 3);
top3.forEach((p, i) => console.log(`${i + 1}. ${p.name.padEnd(15)} Rs ${p.price}`));

const valueByCategory = inStock.reduce((acc, p) => {
  acc[p.category] = (acc[p.category] ?? 0) + p.price * p.stock;
  return acc;
}, {});
console.log(valueByCategory);
```

OUTPUT

```
In stock: Wireless Mouse, Backpack, USB-C Cable, Water Bottle, Headphones
Total inventory value: Rs 65,172
1. Headphones      Rs 2499
2. Backpack        Rs 1499
3. Desk Lamp       Rs 1299
{ Electronics: 46430, Bags: 11992, Home: 6750 }
```

MODULE 08 · BEGINNER

Strings

Minimum duration **2 days** | Level **Beginner**

Strings are immutable sequences of UTF-16 code units with a rich set of built-in methods, and template literals make building them easy. This module covers the three kinds of string literals, escape characters and tagged templates, accessing characters, every important string method, regular expressions, and the classic string problems asked in interviews.

IN THIS MODULE

8.1 Creating strings
8.2 Accessing characters
8.3 String methods
8.4 Regular expressions
8.5 Classic string problems

8.1 Creating strings

8.1.1 Single quotes, double quotes and backticks

A string literal can be written with `'single'` or `"double"` quotes, which behave identically, or with `` `backticks` ``, which create a **template literal** with extra powers. Use one quote style consistently (Prettier enforces one); choose the other kind when the text contains your usual quote character, or use a template literal.

THREE WAYS TO WRITE A STRING — main.js

```
const a = 'single quotes';
const b = "double quotes";
const c = `backticks`;
const quote = "It's easy";                // apostrophe inside double quotes
const html = '<a href="/home">Home</a>';  // double quotes inside single quotes
console.log(a, "|", b, "|", c, "|", quote, "|", html);
console.log(typeof c, String(42), (3.5).toString(), `${null}`);
```

OUTPUT

```
single quotes | double quotes | backticks | It's easy | <a href="/home">Home</a>
string 42 3.5 null
```

8.1.2 Template literals with ${} and multi-line strings

Template literals (ES2015) embed any expression inside `${ }`: variables, arithmetic, function calls, ternaries. They can also span **several lines** directly, keeping the line breaks and indentation. They are the most readable way to build text from values.

EMBEDDING EXPRESSIONS AND MULTIPLE LINES main.js

```
const name = "Asha", course = "JavaScript", fee = 4999, discount = 0.1;
console.log(`Hello ${name}, welcome to ${course}!`);
console.log(`Fee after discount: Rs ${(fee * (1 - discount)).toFixed(2)}`);
console.log(`You ${fee > 3000 ? "qualify" : "do not qualify"} for EMI`);
console.log(`Uppercase: ${course.toUpperCase()}, length ${course.length}`);
const card = `
  Name   : ${name}
  Course : ${course}
  Status : enrolled`;
console.log(card);
console.log("Old way: " + name + " pays Rs " + fee + ".");
```

OUTPUT

```
Hello Asha, welcome to JavaScript!
Fee after discount: Rs 4499.10
You qualify for EMI
Uppercase: JAVASCRIPT, length 10

  Name   : Asha
  Course : JavaScript
  Status : enrolled
Old way: Asha pays Rs 4999.
```

8.1.3 Escape characters

Inside a string, a backslash starts an **escape sequence** for characters that are hard to type or would end the string.

Escape	Meaning	Escape	Meaning
`\n`	new line	`\\`	backslash
`\t`	tab	`\'` `\"` `\``	quotes and backtick
`\r`	carriage return	`\u00e9`	Unicode code unit (é)
`\0`	null character	`\u{1F600}`	any Unicode code point
`\$` or `$\{`	a literal ${ in a template literal	`\x41`	Latin-1 character (A)

ESCAPE SEQUENCES main.js

```
console.log("Line 1\nLine 2");
console.log("Name:\tAsha\nCity:\tPune");
console.log('It\'s', "a \"quoted\" word", `a \`backtick\``);
console.log("Path: C:\\Users\\asha");
console.log("caf\u00e9", "\u{1F600}".length, "\x41");
console.log(`Price: \${amount}`, String.raw`C:\new\table`);
```

OUTPUT

```
Line 1
Line 2
Name:   Asha
City:   Pune
It's a "quoted" word a `backtick`
Path: C:\Users\asha
café 2 A
Price: ${amount} C:\new\table
```

NOTE

`String.raw` returns a template literal with backslashes left as they are, which is handy for Windows paths and regular expressions. The smiley emoji has length 2 because it needs two UTF-16 code units (8.2.2).

8.1.4 Tagged templates (overview)

A **tagged template** is a template literal preceded by a function name: `tag`text ${value}``. The function receives the literal text parts as an array and the values as further arguments, and returns anything it likes. Libraries use tags for safe HTML escaping, SQL query builders (with parameters instead of string concatenation), styled components (CSS-in-JS) and GraphQL queries.

TAG FUNCTIONS PROCESS TEMPLATE LITERALS main.js

```
function highlight(strings, ...values) {
  return strings.reduce((out, text, i) => out + text + (i < values.length ? `[${values[i]}]` : ""),
"");
}
const user = "Asha", items = 3;
console.log(highlight`User ${user} has ${items} items in the cart`);

function safeHtml(strings, ...values) {              // escape user input before inserting it into
HTML
  const esc = (s) => String(s).replace(/[&<>"']/g, (c) => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;",
'"': "&quot;", "'": "&#39;" })[c]);
  return strings.reduce((out, text, i) => out + text + (i < values.length ? esc(values[i]) : ""), "");
}
const comment = '<img src=x onerror="alert(1)">';
console.log(safeHtml`<p>${comment}</p>`);
```

OUTPUT

```
User [Asha] has [3] items in the cart
<p>&lt;img src=x onerror=&quot;alert(1)&quot;&gt;</p>
```

8.2 Accessing characters

8.2.1 Indexing, charAt(), at()

Characters are read by index from 0, with `str[i]` or `str.charAt(i)`. Out of range, `str[i]` gives `undefined` while `charAt` gives an empty string. `at(i)` (ES2022) also accepts negative indexes from the end. Strings are iterable, so `for...of` and spread work character by character.

READING CHARACTERS main.js

```
const word = "JavaScript";
console.log(word[0], word[4], word.charAt(4), word.at(-1), word.at(-6));
console.log(word[50], JSON.stringify(word.charAt(50)));
for (const ch of "abc") console.log(ch, ch.charCodeAt(0));
console.log([..."hello"], String.fromCharCode(72, 105));
```

OUTPUT

```
J S S t S
undefined ""
a 97
b 98
c 99
[ 'h', 'e', 'l', 'l', 'o' ] Hi
```

8.2.2 length

`length` is the number of UTF-16 **code units**. For ordinary text this equals the number of characters, but characters outside the Basic Multilingual Plane, such as most emoji, take two code units, and some visible characters are combinations of several code points. Spreading a string or using `Array.from` splits by code points; `Intl.Segmenter` splits by user-perceived characters.

LENGTH COUNTS UTF-16 CODE UNITS main.js

```
console.log("hello".length, "".length, "नमस्ते".length);
const emoji = "I \u2764\uFE0F JS \u{1F600}";
console.log(emoji, emoji.length, [...emoji].length);
const family = "\u{1F468}\u200D\u{1F469}\u200D\u{1F467}";
const graphemes = [...new Intl.Segmenter("en", { granularity: "grapheme" }).segment(family)].length;
console.log(family.length, [...family].length, graphemes);
```

OUTPUT

```
5 0 6
I ❤️ JS 😀 10 9
8 5 1
```

8.2.3 Immutability

Strings are **immutable**: no method changes a string in place, and assigning to an index has no effect (an error in strict mode). Every "modifying" method returns a new string; to keep the result, assign it.

STRINGS CANNOT BE CHANGED IN PLACE main.js

```
"use strict";
let city = "pune";
city.toUpperCase();
console.log(city);                    // unchanged
city = city[0].toUpperCase() + city.slice(1);
console.log(city);
try { city[0] = "X"; } catch (e) { console.log(`${e.name}: ${e.message}`); }
```

OUTPUT

```
pune
Pune
TypeError: Cannot assign to read only property '0' of string 'Pune'
```

8.3 String methods

Strings have dozens of methods, all returning new values. The table groups the most useful ones, covered in the subsections that follow.

Group	Methods
Case	`toUpperCase toLowerCase toLocaleUpperCase`
Searching	`indexOf lastIndexOf includes startsWith endsWith search`
Extracting	`slice substring at charAt`
Cleaning	`trim trimStart trimEnd`
Padding / repeating	`padStart padEnd repeat`

Group	Methods
Replacing	`replace` `replaceAll`
Splitting / joining	`split` `concat` (and array `join`)
Comparing	`localeCompare`, `normalize`

8.3.1 Case: toUpperCase(), toLowerCase()

`toUpperCase()` and `toLowerCase()` change case. Converting both sides to one case is the simple way to compare case-insensitively; for languages with special rules (Turkish dotted i), use the locale-aware versions.

CHANGING CASE main.js

```
const email = "Asha.Rao@Example.COM";
console.log(email.toLowerCase(), "LBF".toLowerCase(), "pune".toUpperCase());
console.log("YES".toLowerCase() === "yes", "Straße".toUpperCase());
const answer = "  Yes ";
console.log(answer.trim().toLowerCase() === "yes" ? "confirmed" : "not confirmed");
```

OUTPUT

```
asha.rao@example.com lbf PUNE
true STRASSE
confirmed
```

8.3.2 Searching: indexOf(), lastIndexOf(), includes(), startsWith(), endsWith(), search()

`indexOf(sub, from)` returns the position of the first occurrence or -1; `lastIndexOf` searches backwards. `includes`, `startsWith` and `endsWith` return booleans and are clearer for yes/no questions. `search(regex)` returns the index of the first regular-expression match. All are case-sensitive.

SEARCHING INSIDE STRINGS main.js

```
const text = "the quick brown fox jumps over the lazy dog";
console.log(text.indexOf("the"), text.lastIndexOf("the"), text.indexOf("cat"), text.indexOf("o", 15));
console.log(text.includes("fox"), text.includes("Fox"), text.startsWith("the"), text.endsWith("dog"));
console.log(text.search(/[aeiou]{2}/), text.startsWith("quick", 4));
let count = 0, pos = text.indexOf("o");
while (pos !== -1) { count++; pos = text.indexOf("o", pos + 1); }
console.log(`"o" occurs ${count} times`);
const file = "report-2026.PDF";
console.log(file.toLowerCase().endsWith(".pdf") ? "PDF file" : "other file");
```

OUTPUT

```
0 31 -1 17
true false true true
5 true
"o" occurs 4 times
PDF file
```

8.3.3 Extracting: slice(), substring()

`slice(start, end)` returns the characters from start up to (not including) end; negative values count from the end. `substring(start, end)` is similar but treats negative values as 0 and swaps the arguments if start > end. Prefer `slice`: it behaves like the array method of the same name. (The old `substr` is deprecated.)

SLICE AND SUBSTRING

main.js

```
const s = "JavaScript";
console.log(s.slice(0, 4), s.slice(4), s.slice(-6), s.slice(-6, -3), s.slice(2, -2));
console.log(s.substring(0, 4), s.substring(4, 0), s.substring(-3));
const email = "asha.rao@example.com";
const at = email.indexOf("@");
console.log(`user: ${email.slice(0, at)}, domain: ${email.slice(at + 1)}`);
const card = "4111222233334444";
console.log(`card ending ${card.slice(-4)}`);
```

OUTPUT

```
Java Script Script Scr vaScri
Java Java JavaScript
user: asha.rao, domain: example.com
card ending 4444
```

8.3.4 Cleaning: trim(), trimStart(), trimEnd()

`trim()` removes whitespace (spaces, tabs, line breaks, non-breaking spaces) from both ends; `trimStart()` and `trimEnd()` from one end. Always trim text typed by users before validating or storing it.

REMOVING SURROUNDING WHITESPACE

main.js

```
const raw = "   Asha Rao \n\t";
console.log(`[${raw.trim()}]`, `[${raw.trimStart()}]`.length, `[${raw.trimEnd()}]`);
const username = "  ravi  ";
console.log(username.trim().length === 0 ? "empty" : `hello ${username.trim()}`);
console.log(`[${"  multiple   spaces   inside  ".trim().replace(/\s+/g, " ")}]`);
```

OUTPUT

```
[Asha Rao] 13 [   Asha Rao]
hello ravi
[multiple spaces inside]
```

8.3.5 Padding and repeating: padStart(), padEnd(), repeat()

`padStart(length, fill)` and `padEnd(length, fill)` add characters until the string reaches a given length: ideal for aligned columns, leading zeros and masking. `repeat(n)` repeats a string n times.

PADDING AND REPEATING

main.js

```
console.log("7".padStart(3, "0"), "42".padStart(6), "|" + "Asha".padEnd(10) + "|");
const items = [["Pen", 10], ["Notebook", 45], ["Bag", 899]];
for (const [name, price] of items) console.log(name.padEnd(10, ".") + String(price).padStart(6));
console.log("-".repeat(16), "ha".repeat(3));
const mobile = "9876543210";
console.log(mobile.slice(-4).padStart(mobile.length, "*"));
const time = (h, m) => `${String(h).padStart(2, "0")}:${String(m).padStart(2, "0")}`;
console.log(time(9, 5), time(14, 30));
```

OUTPUT

```
007     42 |Asha      |
Pen.......    10
Notebook..    45
Bag.......   899
---------------- hahaha
******3210
09:05 14:30
```

8.3.6 Replacing: replace(), replaceAll()

`replace(pattern, replacement)` replaces only the **first** match when the pattern is a string, or all matches when it is a regular expression with the `g` flag. `replaceAll` (ES2021) replaces every occurrence of a string. The replacement can be a function that computes each replacement from the match.

REPLACING TEXT

main.js

```
const s = "I like Java. Java is popular.";
console.log(s.replace("Java", "JavaScript"));       // first only
console.log(s.replaceAll("Java", "JavaScript"));    // all
console.log(s.replace(/java/gi, "JS"));             // regex: global, case-insensitive
console.log("2026-03-09".replace(/(\d+)-(\d+)-(\d+)/, "$3/$2/$1"));   // groups in the replacement
console.log("price: 100, tax: 18".replace(/\d+/g, (n) => Number(n) * 2));  // replacement function
```

OUTPUT

```
I like JavaScript. Java is popular.
I like JavaScript. JavaScript is popular.
I like JS. JS is popular.
09/03/2026
price: 200, tax: 36
```

8.3.7 Splitting and joining: split(), concat()

`split(separator, limit)` breaks a string into an array of parts; `split("")` gives characters and a regex separator handles several delimiters. Array `join(separator)` does the reverse. `concat` joins strings, but `+` or template literals are more common.

SPLIT AND JOIN

main.js

```
const csv = "Asha,21,Pune,8.7";
const [name, age, city] = csv.split(",");
console.log(csv.split(","), name, Number(age) + 1, city);
console.log("a-b_c d".split(/[-_ ]/), "word".split(""), "one two three".split(" ", 2));
console.log("  extra   spaces  here ".trim().split(/\s+/));
console.log(["2026", "03", "09"].join("-"), "Hello".concat(", ", "World", "!"));
const slug = "Learn JavaScript Fast".toLowerCase().split(" ").join("-");
console.log(slug);
```

OUTPUT

```
[ 'Asha', '21', 'Pune', '8.7' ] Asha 22 Pune
[ 'a', 'b', 'c', 'd' ] [ 'w', 'o', 'r', 'd' ] [ 'one', 'two' ]
[ 'extra', 'spaces', 'here' ]
2026-03-09 Hello, World!
learn-javascript-fast
```

8.3.8 Comparing with localeCompare()

`a.localeCompare(b)` compares strings according to the rules of a human language and returns a negative number, 0 or a positive number, which is exactly what `sort` expects. Options handle case and accents (`sensitivity: "base"` treats "a", "A" and "á" as equal) and numbers inside text (`numeric: true` sorts "file2" before "file10"). `Intl.Collator` is the faster reusable version.

LANGUAGE-AWARE COMPARISON

main.js

```
console.log("apple".localeCompare("banana"), "b".localeCompare("a"), "a".localeCompare("a"));
console.log("a".localeCompare("A", undefined, { sensitivity: "base" }),
"résumé".localeCompare("resume", "en", { sensitivity: "base" }));
const files = ["file10.txt", "File2.txt", "file1.txt", "file20.txt"];
console.log([...files].sort());
console.log([...files].sort((a, b) => a.localeCompare(b, undefined, { numeric: true, sensitivity:
"base" })));
const collator = new Intl.Collator("en", { numeric: true });
console.log(["v1.10", "v1.9", "v1.2"].sort(collator.compare));
```

OUTPUT

```
-1 1 0
0 0
[ 'File2.txt', 'file1.txt', 'file10.txt', 'file20.txt' ]
[ 'file1.txt', 'File2.txt', 'file10.txt', 'file20.txt' ]
[ 'v1.2', 'v1.9', 'v1.10' ]
```

8.4 Regular expressions

A **regular expression** (regex) is a pattern that describes text: "three digits", "something that looks like an email", "one or more spaces". JavaScript regexes test whether text matches, find and extract parts, and drive replace and split.

8.4.1 Creating regex: literal /.../ and RegExp

Write a regex as a **literal** between slashes, `/pattern/flags`, when the pattern is fixed. Use the `RegExp` constructor, `new RegExp("pattern", "flags")`, when the pattern is built from a string at run time (remember to escape backslashes in the string, and to escape user input with `RegExp.escape` in ES2025 or by hand).

TWO WAYS TO CREATE A REGULAR EXPRESSION

main.js

```
const pin = /^\d{6}$/;                      // literal
console.log(pin.test("411001"), pin.test("41100"));
const word = "java";
const dynamic = new RegExp(`\\b${word}\\b`, "i");   // built from a variable
console.log(dynamic, dynamic.test("I like Java"), dynamic.test("javascript"));
console.log(pin.source, pin.flags === "", typeof pin);
```

OUTPUT

```
true false
/\bjava\b/i true false
^\d{6}$ true object
```

8.4.2 Flags: g, i, m, s, u, y

Flag	Name	Effect
`g`	global	find all matches (matchAll, replace all), not just the first
`i`	ignore case	"A" matches "a"
`m`	multiline	`^` and `$` match at each line start and end
`s`	dotAll	`.` also matches line breaks

Flag	Name	Effect
u	unicode	treat the pattern as code points; enables \p{L} and similar
y	sticky	match only at lastIndex (for tokenizers)
d	indices	report start/end positions of groups
v	unicodeSets	an upgraded u mode with set operations (ES2024)

THE EFFECT OF FLAGS

main.js

```
const text = "Cat cat CAT\nsecond line";
console.log(text.match(/cat/g), text.match(/cat/gi));
console.log("multi:\nline".match(/^line$/m)?.[0], /a.b/.test("a\nb"), /a.b/s.test("a\nb"));
console.log("café naïve".match(/\p{L}+/gu));
```

```
OUTPUT
[ 'cat' ] [ 'Cat', 'cat', 'CAT' ]
line false true
[ 'café', 'naïve' ]
```

8.4.3 test(), match(), matchAll(), exec()

`regex.test(str)` returns true or false. `str.match(regex)` returns the first match with its groups (or, with `g`, an array of all matched strings), or `null`. `str.matchAll(regex)` (requires `g`) returns an iterator of full match objects with groups and positions. `regex.exec(str)` is the lower-level method behind them; with `g` it continues from `lastIndex` on each call.

MATCHING METHODS

main.js

```
const log = "ERROR 2026-03-09 disk full; WARN 2026-03-10 slow; ERROR 2026-03-11 timeout";
console.log(/ERROR/.test(log));
const first = log.match(/(\w+) (\d{4}-\d{2}-\d{2})/);
console.log(first[0], "|", first[1], "|", first[2], "| index", first.index);
console.log(log.match(/\d{4}-\d{2}-\d{2}/g));
for (const m of log.matchAll(/(ERROR|WARN) (\S+) ([^;]+)/g)) console.log(`${m[1].padEnd(5)} on
${m[2]}: ${m[3]}`);
const re = /\d+/g;
let m;
while ((m = re.exec("a1b22c333")) !== null) console.log(`found ${m[0]} at ${m.index}, next search from
${re.lastIndex}`);
```

```
OUTPUT
true
ERROR 2026-03-09 | ERROR | 2026-03-09 | index 0
[ '2026-03-09', '2026-03-10', '2026-03-11' ]
ERROR on 2026-03-09: disk full
WARN  on 2026-03-10: slow
ERROR on 2026-03-11: timeout
found 1 at 1, next search from 2
found 22 at 3, next search from 5
found 333 at 6, next search from 9
```

8.4.4 Character classes, quantifiers, groups and named groups

Pattern	Matches	Pattern	Matches
`\d` / `\D`	a digit / a non-digit	`*` `+` `?`	0 or more / 1 or more / 0 or 1
`\w` / `\W`	letter, digit or _ / anything else	`{3}` `{2,4}` `{2,}`	exactly 3 / 2 to 4 / 2 or more
`\s` / `\S`	whitespace / non-whitespace	`*?` `+?`	lazy: as few as possible
`.`	any character except newline	`^` `$`	start / end of the text
`[aeiou]` `[^0-9]` `[a-z]`	one of / none of / a range	`\b`	word boundary
`(abc)`	capturing group	`(?:abc)`	non-capturing group
`(?<name>...)`	named group	`a|b`	a or b
`(?=...)` `(?!...)`	lookahead / negative lookahead	`\1`, `\k<name>`	back-reference to a group

GROUPS, LAZINESS, BACK-REFERENCES AND LOOKAHEAD

main.js

```
const date = "Exam on 2026-03-09 at 10:30";
const m = date.match(/(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})/);
console.log(m.groups, `${m.groups.day}/${m.groups.month}/${m.groups.year}`);
console.log("<b>bold</b><i>it</i>".match(/<.+>/)[0], "|", "<b>bold</b><i>it</i>".match(/<.+?>/)[0]);
// greedy vs lazy
console.log("hello hello world".match(/\b(\w+) \1\b/)[0]);                                    //
repeated word
console.log("price: 499 rupees, 20 items".match(/\d+(?= rupees)/)[0]);                          //
lookahead
console.log("2026-03-09".replace(/(?<y>\d+)-(?<m>\d+)-(?<d>\d+)/, "$<d>.$<m>.$<y>"));
```

OUTPUT

```
[Object: null prototype] { year: '2026', month: '03', day: '09' } 09/03/2026
<b>bold</b><i>it</i> | <b>
hello hello
499
09.03.2026
```

8.4.5 Common patterns: email, phone, password strength

A few patterns cover most everyday validation. Keep them simple and readable: a practical email check only needs "something@something.something" (real validation sends a confirmation email), and complex rules are easier to express as several small tests than as one huge regex.

PRACTICAL VALIDATION PATTERNS

main.js

```
const patterns = {
  email: /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/,
  indianMobile: /^(?:\+91[- ]?)?[6-9]\d{9}$/,
  pinCode: /^[1-9]\d{5}$/,
  hexColor: /^#(?:[0-9a-f]{3}){1,2}$/i,
};
const samples = { email: ["asha@example.com", "bad@", "a b@x.io"], indianMobile: ["9876543210", "+91
9876543210", "1234567890"], pinCode: ["411001", "011001"], hexColor: ["#C9973A", "#fff", "#12345"] };
for (const [kind, values] of Object.entries(samples))
  console.log(kind.padEnd(13), values.map((v) => `${v}:${patterns[kind].test(v)}`).join("  "));

function passwordStrength(pw) {
  const checks = [/.{8,}/, /[a-z]/, /[A-Z]/, /\d/, /[^A-Za-z0-9]/];
  const score = checks.filter((re) => re.test(pw)).length;
  return ["very weak", "weak", "weak", "medium", "strong", "very strong"][score];
}
for (const pw of ["abc", "password1", "Password1", "Pa$$w0rd!2026"]) console.log(pw.padEnd(14),
passwordStrength(pw));
```

OUTPUT

```
email         asha@example.com:true  bad@:false  a b@x.io:false
indianMobile  9876543210:true  +91 9876543210:true  1234567890:false
pinCode       411001:true  011001:false
hexColor      #C9973A:true  #fff:true  #12345:false
abc            weak
password1      medium
Password1      strong
Pa$$w0rd!2026  very strong
```

8.5 Classic string problems

8.5.1 Reverse a string, palindrome, anagram

REVERSE, PALINDROME AND ANAGRAM

main.js

```
const reverse = (s) => [...s].reverse().join("");
function isPalindrome(s) {
  const clean = s.toLowerCase().replace(/[^a-z0-9]/g, "");
  return clean === reverse(clean);
}
const normalize = (s) => s.toLowerCase().replace(/[^a-z]/g, "").split("").sort().join("");
const isAnagram = (a, b) => normalize(a) === normalize(b);

console.log(reverse("JavaScript"), reverse("12345"));
for (const s of ["racecar", "A man, a plan, a canal: Panama", "hello"]) console.log(`"${s}"
palindrome: ${isPalindrome(s)}`);
console.log(isAnagram("Listen", "Silent"), isAnagram("Dormitory", "Dirty room"), isAnagram("hello",
"world"));
```

OUTPUT

```
tpircSavaJ 54321
"racecar" palindrome: true
"A man, a plan, a canal: Panama" palindrome: true
"hello" palindrome: false
true true false
```

8.5.2 Count vowels and words

COUNTING VOWELS, CONSONANTS AND WORDS

main.js

```
function analyse(text) {
  const vowels = (text.match(/[aeiou]/gi) ?? []).length;
  const letters = (text.match(/[a-z]/gi) ?? []).length;
  const words = text.trim().split(/\s+/).filter(Boolean);
  return { vowels, consonants: letters - vowels, words: words.length, longest: words.reduce((a, w) =>
(w.length > a.length ? w : a), "") };
}
console.log(analyse("Learning  JavaScript is fun and practical"));
console.log(analyse("   "));
```

OUTPUT

```
{ vowels: 12, consonants: 23, words: 6, longest: 'JavaScript' }
{ vowels: 0, consonants: 0, words: 0, longest: '' }
```

8.5.3 Capitalise each word, title case

CAPITALISING WORDS

main.js

```
const capitalizeWords = (s) => s.toLowerCase().replace(/\b\w/g, (c) => c.toUpperCase());
function titleCase(s) {
  const small = new Set(["a", "an", "the", "and", "or", "of", "in", "on", "to", "for"]);
  return s.toLowerCase().split(" ").map((w, i) => (i > 0 && small.has(w) ? w : w[0].toUpperCase() +
w.slice(1))).join(" ");
}
console.log(capitalizeWords("the QUICK brown fox"));
console.log(titleCase("the lord of the rings and the return of the king"));
```

OUTPUT

```
The Quick Brown Fox
The Lord of the Rings and the Return of the King
```

8.5.4 Character frequency, first non-repeating character

FREQUENCY COUNTING

main.js

```
function frequency(s) {
  const map = new Map();
  for (const ch of s) map.set(ch, (map.get(ch) ?? 0) + 1);
  return map;
}
function firstUnique(s) {
  const f = frequency(s);
  for (const ch of s) if (f.get(ch) === 1) return ch;      // second pass keeps the original order
  return null;
}
console.log(Object.fromEntries(frequency("programming")));
for (const s of ["programming", "swiss", "aabbcc"]) console.log(s, "->", firstUnique(s));
```

OUTPUT

```
{ p: 1, r: 2, o: 1, g: 2, a: 1, m: 2, i: 1, n: 1 }
programming -> p
swiss -> w
aabbcc -> null
```

8.5.5 Truncate text with an ellipsis

SHORTENING TEXT FOR PREVIEWS

main.js

```
function truncate(text, max) {
  if (text.length <= max) return text;
  const cut = text.slice(0, max - 1);
  const lastSpace = cut.lastIndexOf(" ");
  return (lastSpace > max * 0.6 ? cut.slice(0, lastSpace) : cut).trimEnd() + "\u2026";   // break at a
word if reasonable
}
const desc = "JavaScript is the programming language of the web and runs everywhere";
for (const n of [80, 30, 12]) console.log(`${String(n).padStart(2)}: ${truncate(desc, n)}`);
```

OUTPUT

```
80: JavaScript is the programming language of the web and runs everywhere
30: JavaScript is the programming…
12: JavaScript…
```

Strings use single or double quotes or backtick template literals (with ${} interpolation and multiple lines); escapes write special characters and tagged templates process literals with a function. Index with [] or at(), length counts UTF-16 code units, and strings are immutable. Use toUpperCase/toLowerCase, indexOf/includes/startsWith/endsWith, slice, trim, padStart/padEnd/repeat, replace/replaceAll, split/join and localeCompare. Regular expressions (literals or RegExp) with flags g, i, m, s, u test, match, extract named groups and power replace; keep validation patterns simple. Practise reversing, palindromes, anagrams, counting, title case, frequency and truncation.

PRACTICE CHECKPOINT

Write a **password-strength checker** using regular expressions and a function that converts any sentence to **title case** and to a **URL slug**.

SAMPLE SOLUTION

main.js

```
function checkPassword(pw) {
  const rules = [
    { test: /.{10,}/, tip: "use at least 10 characters" },
    { test: /[a-z]/, tip: "add a lowercase letter" },
    { test: /[A-Z]/, tip: "add an uppercase letter" },
    { test: /\d/, tip: "add a digit" },
    { test: /[^\w\s]/, tip: "add a symbol" },
    { test: /^(?!.*(.)\1\1)/, tip: "avoid three identical characters in a row" },
  ];
  const failed = rules.filter((r) => !r.test.test(pw));
  const score = rules.length - failed.length;
  const label = score === 6 ? "strong" : score >= 4 ? "medium" : "weak";
  return { label, score, tips: failed.map((r) => r.tip) };
}

const SMALL = new Set(["a", "an", "and", "as", "at", "but", "by", "for", "in", "of", "on", "or",
"the", "to", "with"]);
function toTitleCase(sentence) {
  const words = sentence.trim().toLowerCase().split(/\s+/);
  return words.map((w, i) => (i !== 0 && i !== words.length - 1 && SMALL.has(w) ? w :
w.charAt(0).toUpperCase() + w.slice(1))).join(" ");
}
function toSlug(sentence) {
  return sentence
    .normalize("NFD").replace(/[\u0300-\u036f]/g, "")   // remove accents
    .toLowerCase()
    .replace(/[^a-z0-9\s-]/g, "")                       // drop punctuation
    .trim()
    .replace(/[\s-]+/g, "-");                           // spaces and dashes to single dashes
}

for (const pw of ["hello", "Hello123", "Hello123!!", "Str0ng#Passw0rd", "Aaa111!!!bbb"]) {
  const r = checkPassword(pw);
  console.log(`${pw.padEnd(16)} ${r.label.padEnd(7)} ${r.score}/6 ${r.tips.length ? "-> " +
r.tips.join(", ") : ""}`);
}
for (const s of ["  the art OF   war in the modern age ", "Café crème: a guide to French coffee!", "10
tips for learning JavaScript -- fast"]) {
  console.log(toTitleCase(s), "|", toSlug(s));
}
```

OUTPUT

```
hello            weak    2/6 -> use at least 10 characters, add an uppercase letter, add a digit, add
a symbol
Hello123         medium  4/6 -> use at least 10 characters, add a symbol
Hello123!!       strong  6/6
Str0ng#Passw0rd  strong  6/6
Aaa111!!!bbb     medium  5/6 -> avoid three identical characters in a row
The Art of War in the Modern Age | the-art-of-war-in-the-modern-age
Café Crème: a Guide to French Coffee! | cafe-creme-a-guide-to-french-coffee
10 Tips for Learning Javascript -- Fast | 10-tips-for-learning-javascript-fast
```

MODULE 09 · BEGINNER

Objects

Minimum duration **2 days** | Level **Beginner**

Objects store data as key-value pairs and are the building blocks of almost everything in JavaScript: arrays, functions, dates and the DOM are all objects. This module covers creating objects, reading, adding and deleting properties, methods and an introduction to this, iterating with Object.keys, values and entries, copying and merging, freezing, working with arrays of objects, and JSON.

IN THIS MODULE

9.1 Creating objects
9.2 Working with properties
9.3 Methods
9.4 Iterating objects
9.5 Copying and merging
9.6 Protecting objects
9.7 Arrays of objects
9.8 JSON

9.1 Creating objects

An **object** is a collection of **properties**, each a **key** (a string or symbol) paired with a **value** of any type, including other objects and functions. Where an array answers "what is at position 2?", an object answers "what is the student's name?".



AN OBJECT MAPS KEYS TO VALUES

9.1.1 Object literals

The **object literal** `{ key: value, ... }` is the usual way to create an object. Keys that are valid identifiers need no quotes; others ("first-name", "2nd") must be quoted. Values can be any expression. A trailing comma after the last property is allowed and makes diffs cleaner.

AN OBJECT LITERAL

main.js

```
const student = {
  name: "Asha Rao",
  age: 21,
  isEnrolled: true,
  courses: ["JavaScript", "SQL"],
  address: { city: "Pune", pin: "411001" },
  "first-language": "Marathi",          // a key that is not a valid identifier needs quotes
};
console.log(student);
console.log(student.name, student.courses[0], student.address.city, student["first-language"]);
const empty = {};
console.log(empty, Object.keys(empty).length);
```

OUTPUT

```
{
  name: 'Asha Rao',
  age: 21,
  isEnrolled: true,
  courses: [ 'JavaScript', 'SQL' ],
  address: { city: 'Pune', pin: '411001' },
  'first-language': 'Marathi'
}
Asha Rao JavaScript Pune Marathi
{} 0
```

9.1.2 new Object() and Object.create()

`new Object()` creates an empty object, identical to `{}` but longer, so it is rarely used. `Object.create(proto)` creates a new object whose **prototype** is `proto`, so it inherits proto's properties (Module 13). `Object.create(null)` creates an object with no prototype at all, a clean dictionary without inherited keys such as `toString`.

NEW OBJECT AND OBJECT.CREATE

main.js

```
const a = new Object();
a.title = "created with new Object()";
const defaults = { theme: "light", greet() { return `hi from ${this.name}`; } };
const user = Object.create(defaults);       // inherits from defaults
user.name = "Ravi";
console.log(a, user.theme, user.greet(), Object.keys(user));
const dict = Object.create(null);
dict.apple = 3;
console.log("toString" in {}, "toString" in dict, dict);
```

OUTPUT

```
{ title: 'created with new Object()' } light hi from Ravi [ 'name' ]
true false [Object: null prototype] { apple: 3 }
```

9.1.3 Property shorthand and computed property names

When a variable has the same name as the key, ES2015 **shorthand** lets you write it once: `{ name, age }` instead of `{ name: name, age: age }`. **Computed property names** `[expression]: value` use the result of an expression as the key.

SHORTHAND AND COMPUTED KEYS

main.js

```
const name = "Meera", age = 22, city = "Delhi";
const person = { name, age, city };              // shorthand
console.log(person);

const field = "email";
const lang = "en";
const record = {
  [field]: "meera@example.com",                  // computed key
  [`label_${lang}`]: "Email address",
  [`${field}Verified`]: false,
};
console.log(record);
```

OUTPUT

```
{ name: 'Meera', age: 22, city: 'Delhi' }
{
  email: 'meera@example.com',
  label_en: 'Email address',
  emailVerified: false
}
```

9.2 Working with properties

9.2.1 Dot vs bracket notation

Dot notation (`obj.key`) is short and clear and works when the key is a valid identifier known when you write the code. **Bracket notation** (`obj["key"]`) accepts any string, including keys with spaces or dashes, and any expression, so it is required when the key is in a variable. Reading a missing property gives `undefined`.

DOT AND BRACKET NOTATION

main.js

```
const product = { name: "Pen", price: 10, "in stock": true };
console.log(product.name, product["price"], product["in stock"]);
const key = "price";
console.log(product[key], product.key);          // product.key looks for a property literally named
"key"
for (const k of ["name", "price", "color"]) console.log(k, "->", product[k]);
```

OUTPUT

```
Pen 10 true
10 undefined
name -> Pen
price -> 10
color -> undefined
```

9.2.2 Adding, updating and deleting properties

Objects are mutable: assigning to a new key adds a property, assigning to an existing key updates it, and the `delete` operator removes it. This works even on objects declared with `const`, because const only fixes the binding.

CHANGING AN OBJECT

main.js

```
const car = { brand: "Tata", model: "Nexon" };
car.year = 2026;                  // add
car["color"] = "blue";            // add with brackets
car.model = "Nexon EV";           // update
delete car.color;                 // delete
console.log(car);
console.log(delete car.nonexistent, Object.keys(car).length);
```

OUTPUT

```
{ brand: 'Tata', model: 'Nexon EV', year: 2026 }
true 3
```

9.2.3 Checking properties: in, hasOwnProperty(), Object.hasOwn()

Checking `obj.key !== undefined` is unreliable, because a property can exist with the value `undefined`. The `in` operator checks whether a key exists on the object **or its prototype chain**. `Object.hasOwn(obj, key)` (ES2022) checks only the object's **own** properties, the modern replacement for `obj.hasOwnProperty(key)`, which fails for objects without a prototype.

TESTING WHETHER A PROPERTY EXISTS

main.js

```
const config = { debug: false, timeout: undefined };
console.log(config.timeout !== undefined, "timeout" in config);      // exists, with the value
undefined
console.log("debug" in config, "retries" in config);
console.log("toString" in config, Object.hasOwn(config, "toString"));  // inherited vs own
console.log(config.hasOwnProperty("debug"), Object.hasOwn(config, "debug"));
const bare = Object.create(null);
bare.x = 1;
console.log(Object.hasOwn(bare, "x"), typeof bare.hasOwnProperty);
```

OUTPUT

```
false true
true false
true false
true true
true undefined
```

9.2.4 Nested objects and optional chaining

Objects can contain objects to any depth, which is how JSON data from APIs usually looks. Read deep values with chained dots, and use **optional chaining** `?.` with `??` where a level might be missing (Module 4).

READING AND WRITING NESTED DATA

main.js

```
const order = {
  id: 501,
  customer: { name: "Asha", contact: { email: "asha@example.com", phone: null } },
  items: [{ name: "Pen", qty: 3 }, { name: "Bag", qty: 1 }],
};
console.log(order.customer.contact.email, order.items[1].name);
console.log(order.customer.contact.phone ?? "no phone", order.shipping?.address?.city ?? "address
pending");
order.shipping = { address: { city: "Pune" } };     // add a nested object
console.log(order.shipping.address.city, order.items.reduce((n, i) => n + i.qty, 0));
```

OUTPUT

```
asha@example.com Bag
no phone address pending
Pune 4
```

9.3 Methods

9.3.1 Functions as object properties

A property whose value is a function is called a **method**. Methods give objects behaviour: an account can deposit, a cart can calculate its total. Call a method with `obj.method()`.

METHODS ARE FUNCTIONS STORED IN PROPERTIES

main.js

```
const calculator = {
  add: function (a, b) { return a + b; },
  subtract: (a, b) => a - b,
};
console.log(calculator.add(4, 5), calculator.subtract(10, 3), typeof calculator.add);
```

OUTPUT

```
9 7 function
```

9.3.2 Method shorthand

ES2015 **method shorthand** drops the colon and the `function` keyword: `{ greet() { ... } }`. It is the standard way to write methods in object literals and classes.

METHOD SHORTHAND

main.js

```
const account = {
  owner: "Ravi",
  balance: 1000,
  deposit(amount) { this.balance += amount; return this.balance; },
  withdraw(amount) {
    if (amount > this.balance) return "insufficient funds";
    this.balance -= amount;
    return this.balance;
  },
};
console.log(account.deposit(500), account.withdraw(2000), account.withdraw(300));
```

OUTPUT

```
1500 insufficient funds 1200
```

9.3.3 this inside methods (introduction)

Inside a method called as `obj.method()`, the keyword `this` refers to `obj`, the object before the dot, so the method can read and change the object's other properties. `this` is decided by **how the function is called**, not where it is written: a method copied into a variable and called alone loses its object. Arrow functions have no `this` of their own, so they are a poor choice for methods. Module 11 covers all the rules.

THIS REFERS TO THE OBJECT BEFORE THE DOT main.js

```
const cart = {
  items: [{ price: 100, qty: 2 }, { price: 50, qty: 1 }],
  total() { return this.items.reduce((sum, i) => sum + i.price * i.qty, 0); },   // this = cart
  arrowTotal: () => typeof this,                                                 // arrow: no own this
};
console.log(cart.total());
const detached = cart.total;
try { detached(); } catch (e) { console.log(`called alone: ${e.name}: ${e.message}`); }
console.log(cart.total.call({ items: [{ price: 10, qty: 3 }] }));              // explicit this
console.log("arrow method sees this as:", cart.arrowTotal());
```

```
OUTPUT
250
called alone: TypeError: Cannot read properties of undefined (reading 'reduce')
30
arrow method sees this as: object
```

9.4 Iterating objects

9.4.1 for...in

`for...in` visits every enumerable string key of an object, including inherited ones. Combine it with `Object.hasOwn` to skip inherited keys, or prefer `Object.keys` / `entries`, which return only own keys.

LOOPING OVER KEYS WITH FOR...IN main.js

```
const base = { inherited: true };
const scores = Object.create(base);
Object.assign(scores, { maths: 91, physics: 78, chemistry: 85 });
for (const subject in scores) console.log(subject, scores[subject]);
console.log("--- own keys only ---");
for (const subject in scores) if (Object.hasOwn(scores, subject)) console.log(subject);
```

```
OUTPUT
maths 91
physics 78
chemistry 85
inherited true
--- own keys only ---
maths
physics
chemistry
```

9.4.2 Object.keys(), Object.values(), Object.entries()

These return arrays of an object's own enumerable keys, values, or `[key, value]` pairs. Being arrays, they work with for...of and every array method. Integer-like keys come first in ascending order, then string keys in insertion order.

KEYS, VALUES AND ENTRIES

main.js

```
const marks = { maths: 91, physics: 78, chemistry: 85 };
console.log(Object.keys(marks), Object.values(marks));
console.log(Object.entries(marks));
for (const [subject, mark] of Object.entries(marks)) console.log(`${subject.padEnd(10)} ${mark}`);
const total = Object.values(marks).reduce((a, b) => a + b, 0);
const best = Object.entries(marks).sort(([, a], [, b]) => b - a)[0][0];
console.log(`total ${total}, best subject ${best}, subjects ${Object.keys(marks).length}`);
console.log(Object.keys({ b: 1, 2: "x", a: 2, 1: "y" }));     // integer keys first
```

OUTPUT

```
[ 'maths', 'physics', 'chemistry' ] [ 91, 78, 85 ]
[ [ 'maths', 91 ], [ 'physics', 78 ], [ 'chemistry', 85 ] ]
maths      91
physics    78
chemistry  85
total 254, best subject maths, subjects 3
[ '1', '2', 'b', 'a' ]
```

9.4.3 Object.fromEntries()

`Object.fromEntries(pairs)` (ES2019) is the inverse of `Object.entries`: it builds an object from an array (or any iterable, such as a Map) of [key, value] pairs. The pattern entries, then array methods, then fromEntries transforms objects like arrays.

BUILDING OBJECTS FROM PAIRS

main.js

```
const prices = { pen: 10, notebook: 45, bag: 899 };
const withTax = Object.fromEntries(Object.entries(prices).map(([item, p]) => [item, Math.round(p *
1.18)]));
const cheap = Object.fromEntries(Object.entries(prices).filter(([, p]) => p < 100));
console.log(withTax, cheap);
console.log(Object.fromEntries(new Map([["a", 1], ["b", 2]])));
const query = Object.fromEntries(new URLSearchParams("page=2&sort=price&dir=asc"));
console.log(query);
```

OUTPUT

```
{ pen: 12, notebook: 53, bag: 1061 } { pen: 10, notebook: 45 }
{ a: 1, b: 2 }
{ page: '2', sort: 'price', dir: 'asc' }
```

9.5 Copying and merging

9.5.1 Reference behaviour

Like arrays, objects are assigned and passed **by reference**: `const b = a` makes two names for one object. To get an independent object you must copy it.

TWO NAMES, ONE OBJECT main.js

```
const original = { name: "Asha", marks: 91 };
const alias = original;
alias.marks = 50;
console.log(original.marks, alias === original);
console.log({ a: 1 } === { a: 1 }, JSON.stringify({ a: 1 }) === JSON.stringify({ a: 1 }));
```

OUTPUT

```
50 true
false true
```

9.5.2 Object.assign() and spread {...obj}

`Object.assign(target, ...sources)` copies the own enumerable properties of the sources into the target (later sources win) and returns the target. The **object spread** syntax `{ ...a, ...b }` (ES2018) creates a new object in the same way and is the modern idiom for copying, merging and updating objects without mutating them.

MERGING OBJECTS main.js

```
const defaults = { theme: "light", fontSize: 14, language: "en" };
const userPrefs = { fontSize: 18, language: "hi" };
console.log(Object.assign({}, defaults, userPrefs));
const settings = { ...defaults, ...userPrefs, updated: true };   // later properties override earlier
ones
console.log(settings, defaults);
const student = { name: "Asha", marks: 91 };
const updated = { ...student, marks: 95 };                       // copy with one change
console.log(student, updated);
const { marks, ...withoutMarks } = updated;                      // remove a property without mutating
console.log(withoutMarks);
```

OUTPUT

```
{ theme: 'light', fontSize: 18, language: 'hi' }
{ theme: 'light', fontSize: 18, language: 'hi', updated: true } { theme: 'light', fontSize: 14,
language: 'en' }
{ name: 'Asha', marks: 91 } { name: 'Asha', marks: 95 }
{ name: 'Asha' }
```

9.5.3 Shallow vs deep copy and structuredClone()

Spread and `Object.assign` make **shallow** copies: nested objects and arrays are still shared. `structuredClone` makes a **deep** copy, including nested structures, Dates, Maps and Sets (but not functions).

SHALLOW AND DEEP COPIES

main.js

```
const user = { name: "Ravi", address: { city: "Delhi" }, tags: ["js"] };
const shallow = { ...user };
shallow.address.city = "Mumbai";          // shared nested object!
shallow.tags.push("node");
console.log(user.address.city, user.tags);

const deep = structuredClone(user);
deep.address.city = "Chennai";
deep.tags.push("react");
console.log(user.address.city, user.tags.length, deep.address.city, deep.tags.length);
try { structuredClone({ fn() {} }); } catch (e) { console.log(`${e.name}: functions cannot be
cloned`); }
```

OUTPUT

```
Mumbai [ 'js', 'node' ]
Mumbai 2 Chennai 3
DataCloneError: functions cannot be cloned
```

9.6 Protecting objects

9.6.1 Object.freeze(), Object.seal(), Object.preventExtensions()

Three levels of protection, each **shallow** (nested objects stay unprotected unless frozen themselves). In strict mode (classes and modules), forbidden changes throw a TypeError; in sloppy mode they fail silently.

Method	Add properties	Delete properties	Change values	Check with
`Object.preventExtensions`	no	yes	yes	`Object.isExtensible`
`Object.seal`	no	no	yes	`Object.isSealed`
`Object.freeze`	no	no	no	`Object.isFrozen`

FREEZING, SEALING AND PREVENTING EXTENSIONS

main.js

```
"use strict";
const tryIt = (label, fn) => { try { fn(); console.log(`${label}: ok`); } catch (e) {
console.log(`${label}: ${e.constructor.name}`); } };

const frozen = Object.freeze({ rate: 0.18, nested: { level: 1 } });
tryIt("frozen: change", () => { frozen.rate = 0.2; });
tryIt("frozen: add", () => { frozen.extra = 1; });
tryIt("frozen: nested change", () => { frozen.nested.level = 2; });   // shallow: nested object is not
frozen

const sealed = Object.seal({ name: "Asha" });
tryIt("sealed: change", () => { sealed.name = "Ravi"; });
tryIt("sealed: delete", () => { delete sealed.name; });

const fixed = Object.preventExtensions({ a: 1 });
tryIt("non-extensible: add", () => { fixed.b = 2; });
tryIt("non-extensible: delete", () => { delete fixed.a; });
console.log(Object.isFrozen(frozen), Object.isSealed(sealed), Object.isExtensible(fixed));
```

OUTPUT

```
frozen: change: TypeError
frozen: add: TypeError
frozen: nested change: ok
sealed: change: ok
sealed: delete: TypeError
non-extensible: add: TypeError
non-extensible: delete: ok
true true false
```

9.7 Arrays of objects

9.7.1 Searching, filtering, sorting and grouping records

Most real data is an **array of objects**: rows from a database, items from an API, records in a list. The array methods from Module 7 combined with object access cover nearly every operation: find a record, filter by a condition, sort by a field, compute totals, group by a category, and transform records for display.

WORKING WITH RECORDS

main.js

```
const students = [
  { id: 1, name: "Asha", dept: "CSE", marks: 91 },
  { id: 2, name: "Ravi", dept: "ECE", marks: 72 },
  { id: 3, name: "Meera", dept: "CSE", marks: 85 },
  { id: 4, name: "Kiran", dept: "MECH", marks: 38 },
  { id: 5, name: "Zoya", dept: "ECE", marks: 88 },
];
console.log(students.find((s) => s.id === 3).name);
console.log(students.filter((s) => s.marks >= 80).map((s) => s.name));
console.log(students.toSorted((a, b) => b.marks - a.marks).map((s) => `${s.name}:${s.marks}`).join("
"));
const avg = students.reduce((sum, s) => sum + s.marks, 0) / students.length;
console.log(`average ${avg.toFixed(1)}`);
const byDept = Object.groupBy(students, (s) => s.dept);
for (const [dept, list] of Object.entries(byDept)) console.log(dept, list.map((s) => s.name).join(",
"));
const lookup = Object.fromEntries(students.map((s) => [s.id, s.name]));   // id -> name index
console.log(lookup[5]);
```

OUTPUT

```
Meera
[ 'Asha', 'Meera', 'Zoya' ]
Asha:91 Zoya:88 Meera:85 Ravi:72 Kiran:38
average 74.8
CSE Asha, Meera
ECE Ravi, Zoya
MECH Kiran
Zoya
```

9.8 JSON

JSON (JavaScript Object Notation) is a text format for data, based on JavaScript's object and array literals. It is the standard format for web APIs, configuration files (package.json) and data storage such as localStorage.

9.8.1 JSON format rules

JSON looks like JavaScript but is stricter: keys must be **double-quoted strings**; strings use double quotes only; values can only be strings, numbers, `true`, `false`, `null`, objects and arrays; no functions, `undefined`, comments, trailing commas, NaN or Infinity.

Valid JSON	Invalid JSON
`{"name": "Asha", "age": 21}`	`{name: "Asha"}` (unquoted key)
`["a", 1, true, null]`	`['a']` (single quotes)
`{"nested": {"list": [1, 2]}}`	`{"a": 1,}` (trailing comma)
`"just a string"`, `42`	`{"f": function () {}}`, `undefined`, `NaN`

9.8.2 JSON.stringify() and JSON.parse()

`JSON.stringify(value, replacer, indent)` converts a value to a JSON string. Functions, `undefined` and symbols are skipped in objects (and become null in arrays), Dates become ISO strings, and an indent

argument pretty-prints. `JSON.parse(text, reviver)` converts JSON text back into values and throws a SyntaxError for invalid JSON, so wrap untrusted input in try/catch.

CONVERTING TO AND FROM JSON

main.js

```
const student = { name: "Asha", marks: [91, 85], active: true, address: null, greet() {}, nickname:
undefined, joined: new Date(Date.UTC(2026, 2, 9)) };
const json = JSON.stringify(student);
console.log(json);
console.log(JSON.stringify(student, ["name", "marks"], 2));          // only some keys, indented

const back = JSON.parse(json);
console.log(back.name, back.marks[1], typeof back.joined);
const revived = JSON.parse(json, (key, value) => (key === "joined" ? new Date(value) : value));
console.log(revived.joined instanceof Date, revived.joined.getUTCFullYear());

for (const bad of ["{name: 'Asha'}", '{"a": 1,}', ""]) {
  try { JSON.parse(bad); } catch (e) { console.log(`${e.name}: ${e.message}`); }
}
```

OUTPUT

```
{"name":"Asha","marks":[91,85],"active":true,"address":null,"joined":"2026-03-09T00:00:00.000Z"}
{
  "name": "Asha",
  "marks": [
    91,
    85
  ]
}
Asha 85 string
true 2026
SyntaxError: Expected property name or '}' in JSON at position 1 (line 1 column 2)
SyntaxError: Expected double-quoted property name in JSON at position 8 (line 1 column 9)
SyntaxError: Unexpected end of JSON input
```

Create objects with literals (shorthand and computed keys), Object.create for a chosen prototype, or Object.create(null) for clean dictionaries. Read with dot or bracket notation; add, update and delete freely; test existence with in or Object.hasOwn; use ?. and ?? for nested data. Methods are function properties (use shorthand), and this refers to the object before the dot. Iterate with Object.keys/values/entries (for...in includes inherited keys) and rebuild with Object.fromEntries. Objects are references: copy with spread or Object.assign (shallow) or structuredClone (deep). freeze, seal and preventExtensions protect objects shallowly. Arrays of objects are processed with find, filter, sort, reduce and groupBy, and JSON.stringify/parse convert data to and from text.

PRACTICE CHECKPOINT

Write a **student records** program that stores students as an **array of objects**, **adds, updates and deletes** records, **sorts by marks**, **groups by grade** and saves the result as a **JSON string**.

SAMPLE SOLUTION

main.js

```
let students = [
  { id: 1, name: "Asha", marks: 91 },
  { id: 2, name: "Ravi", marks: 72 },
  { id: 3, name: "Meera", marks: 64 },
];
let nextId = 4;

const addStudent = (name, marks) => { students = [...students, { id: nextId++, name, marks }]; };
const updateStudent = (id, changes) => { students = students.map((s) => (s.id === id ? { ...s,
...changes } : s)); };
const deleteStudent = (id) => { students = students.filter((s) => s.id !== id); };
const grade = (m) => (m >= 85 ? "A" : m >= 70 ? "B" : m >= 50 ? "C" : "F");

addStudent("Kiran", 45);
addStudent("Zoya", 88);
updateStudent(3, { marks: 79 });
deleteStudent(2);

const sorted = students.toSorted((a, b) => b.marks - a.marks);
console.log("Ranked:");
sorted.forEach((s, i) => console.log(`  ${i + 1}. ${s.name.padEnd(6)} ${s.marks}
${grade(s.marks)}`));

const byGrade = Object.groupBy(students, (s) => grade(s.marks));
const summary = Object.fromEntries(Object.entries(byGrade).map(([g, list]) => [g, list.map((s) =>
s.name)]));
console.log("By grade:", summary);

const saved = JSON.stringify({ savedAt: "2026-03-09", students: sorted, byGrade: summary }, null, 2);
console.log(saved);
console.log("restored count:", JSON.parse(saved).students.length);
```

OUTPUT

```
Ranked:
  1. Asha   91  A
  2. Zoya   88  A
  3. Meera  79  B
  4. Kiran  45  F
By grade: { A: [ 'Asha', 'Zoya' ], B: [ 'Meera' ], F: [ 'Kiran' ] }
{
  "savedAt": "2026-03-09",
  "students": [
    {
      "id": 1,
      "name": "Asha",
      "marks": 91
    },
    {
      "id": 5,
      "name": "Zoya",
      "marks": 88
    },
    {
      "id": 3,
      "name": "Meera",
      "marks": 79
    },
    {
      "id": 4,
      "name": "Kiran",
      "marks": 45
    }
  ],
  "byGrade": {
    "A": [
      "Asha",
      "Zoya"
    ],
    "B": [
      "Meera"
    ],
    "F": [
      "Kiran"
    ]
  }
}
restored count: 4
```

MODULE 10 · BEGINNER

Numbers, Math and Date

Minimum duration **1 day** | Level **Beginner**

This module covers the built-in objects for numeric work and dates: Number methods and formatting with Intl, the Math object including random numbers, BigInt, the Date object (creating, reading, changing, calculating, formatting, time zones), an overview of the new Temporal API, and an introduction to timers.

IN THIS MODULE

10.1 Number
10.2 Math
10.3 BigInt
10.4 Date
10.5 Timers (introduction)

NOTE

Examples that create dates use fixed values so that the output is repeatable; examples that read the current date or time say so, and their output will differ when you run them. Output that depends on the time zone was produced with the zone set to Asia/Kolkata (IST, UTC+05:30).

10.1 Number

Every number primitive can use the methods of `Number.prototype`, and the `Number` constructor itself has useful static methods and constants. To call a method directly on a number literal, wrap it in parentheses or use two dots: `(255).toString(16)`.

10.1.1 toFixed(), toPrecision(), toString(base)

`toFixed(digits)` returns a **string** with exactly that many digits after the decimal point, rounding as needed (it is the usual way to display money). `toPrecision(digits)` rounds to a total number of significant digits. `toString(base)` converts to another base from 2 to 36. All of them return strings; convert back with `Number()` if you need to calculate.

FORMATTING NUMBERS AS STRINGS

main.js

```
const price = 1234.5678;
console.log(price.toFixed(2), price.toFixed(0), (0.5).toFixed(0), (2.5).toFixed(0),
(1.005).toFixed(2));
console.log(price.toPrecision(6), price.toPrecision(2), (0.000123).toPrecision(2));
console.log((255).toString(2), (255).toString(16), (255).toString(8), (35).toString(36));
console.log(typeof price.toFixed(2), Number(price.toFixed(2)) + 1);
```

OUTPUT

```
1234.57 1235 1 3 1.00
1234.57 1.2e+3 0.00012
11111111 ff 377 z
string 1235.57
```

NOTE

`(1.005).toFixed(2)` gives "1.00", not "1.01", because 1.005 is stored as 1.00499999999999989… in binary. For money, calculate in integer paise or use `Intl.NumberFormat` for display.

10.1.2 Number.isInteger(), Number.isFinite(), Number.parseFloat()

The static methods of `Number` test values **without converting** them, unlike the old global `isFinite` and `isNaN`. `Number.parseFloat` and `Number.parseInt` are the same functions as the globals, grouped under Number in ES2015.

TESTING NUMBERS WITHOUT COERCION — main.js

```
console.log(Number.isInteger(5), Number.isInteger(5.0), Number.isInteger(5.5), Number.isInteger("5"));
console.log(Number.isFinite(42), Number.isFinite(Infinity), Number.isFinite("42"), isFinite("42"));
console.log(Number.isSafeInteger(2 ** 53 - 1), Number.isSafeInteger(2 ** 53));
console.log(Number.parseFloat("3.75kg"), Number.parseInt("08", 10), Number.parseFloat === parseFloat);
console.log(Number.MAX_SAFE_INTEGER, Number.MIN_SAFE_INTEGER, Number.MAX_VALUE > 1e308);
```

OUTPUT

```
true true false false
true false false true
true false
3.75 8 true
9007199254740991 -9007199254740991 true
```

10.1.3 Number.EPSILON and comparing floats

`Number.EPSILON` is the difference between 1 and the next representable number, about 2.22×10^{-16}. Because of rounding errors, compare floating-point results with a **tolerance** instead of `===`. A relative tolerance works for numbers of any size.

COMPARING FLOATING-POINT NUMBERS SAFELY — main.js

```
const a = 0.1 + 0.2, b = 0.3;
console.log(a === b, Math.abs(a - b), Number.EPSILON);
const nearlyEqual = (x, y, tol = Number.EPSILON) => Math.abs(x - y) <= tol * Math.max(1, Math.abs(x),
Math.abs(y));
console.log(nearlyEqual(a, b), nearlyEqual(1e10 + 0.00001, 1e10), nearlyEqual(1, 1.0001));
let total = 0;
for (let i = 0; i < 10; i++) total += 0.1;
console.log(total, total === 1, nearlyEqual(total, 1));
```

OUTPUT

```
false 5.551115123125783e-17 2.220446049250313e-16
true false false
0.9999999999999999 false true
```

10.1.4 Formatting with toLocaleString() and Intl.NumberFormat (currency, percent)

`toLocaleString(locale, options)` formats a number for humans in a given language and region: digit grouping, decimal separator, currency symbol. Indian English (`"en-IN"`) groups digits as lakhs and crores. `Intl.NumberFormat` creates a reusable formatter, which is faster when formatting many numbers, and supports styles such as currency, percent, unit and compact notation.

LOCALE-AWARE NUMBER FORMATTING main.js

```
const amount = 12345678.9;
console.log(amount.toLocaleString("en-US"), amount.toLocaleString("en-IN"), amount.toLocaleString("de-DE"));
const inr = new Intl.NumberFormat("en-IN", { style: "currency", currency: "INR" });
const usd = new Intl.NumberFormat("en-US", { style: "currency", currency: "USD" });
console.log(inr.format(amount), usd.format(1999.5), inr.format(-250));
const pct = new Intl.NumberFormat("en-IN", { style: "percent", maximumFractionDigits: 1 });
console.log(pct.format(0.1825), pct.format(1.2));
console.log(new Intl.NumberFormat("en", { notation: "compact" }).format(1520000), new Intl.NumberFormat("en-IN", { notation: "compact" }).format(1520000));
console.log(new Intl.NumberFormat("en", { style: "unit", unit: "kilometer-per-hour" }).format(80));
console.log((7).toLocaleString("en", { minimumIntegerDigits: 2 }), (3.14159).toLocaleString("en", { maximumFractionDigits: 2 }));
```

OUTPUT

```
12,345,678.9 1,23,45,678.9 12.345.678,9
₹1,23,45,678.90 $1,999.50 -₹250.00
18.3% 120%
1.5M 15L
80 km/h
07 3.14
```

10.2 Math

`Math` is a built-in object (not a constructor) holding mathematical constants and functions. All its methods are static: `Math.round(x)`, never `new Math()`.

10.2.1 Math.round, floor, ceil, trunc, abs, sign

`Math.round` rounds to the nearest integer (halves round **up**, towards +∞, so -2.5 becomes -2). `floor` rounds down towards -∞, `ceil` rounds up towards +∞, and `trunc` removes the fractional part (towards 0). `abs` gives the absolute value and `sign` returns -1, 0 or 1. To round to n decimals, multiply, round and divide.

ROUNDING FUNCTIONS main.js

```
const values = [4.4, 4.5, 4.6, -4.4, -4.5, -4.6];
console.log("x     round floor ceil trunc");
for (const v of values) console.log(String(v).padEnd(6) + [Math.round(v), Math.floor(v), Math.ceil(v), Math.trunc(v)].map((n) => String(n).padEnd(6)).join(""));
console.log(Math.abs(-7.5), Math.sign(-3), Math.sign(0), Math.sign(42));
const roundTo = (x, d) => Math.round(x * 10 ** d) / 10 ** d;
console.log(roundTo(3.14159, 2), roundTo(1234.5678, 1), Math.ceil(23 / 5) + " pages needed for 23 items at 5 per page");
```

OUTPUT

```
x     round floor ceil trunc
4.4   4     4     5     4
4.5   5     4     5     4
4.6   5     4     5     4
-4.4  -4    -5    -4    -4
-4.5  -4    -5    -4    -4
-4.6  -5    -5    -4    -4
7.5 -1 0 1
3.14 1234.6 5 pages needed for 23 items at 5 per page
```

10.2.2 Math.max, min, pow, sqrt, cbrt

`Math.max` and `Math.min` take any number of arguments (spread an array into them); with no arguments they return -Infinity and Infinity. `Math.pow(x, y)` equals `x ** y`. `sqrt` and `cbrt` give square and cube roots; there are also `hypot`, `log`, `log10`, `log2`, `exp` and more.

MAX, MIN, POWERS AND ROOTS

main.js

```
const marks = [72, 85, 91, 64];
console.log(Math.max(...marks), Math.min(...marks), Math.max(), Math.min());
console.log(Math.pow(2, 10), 2 ** 10, Math.sqrt(144), Math.cbrt(27), Math.sqrt(-1));
console.log(Math.hypot(3, 4), Math.log10(1000), Math.log2(1024), Math.round(Math.log(Math.E) * 100) /
100);
const clamp = (x, lo, hi) => Math.min(Math.max(x, lo), hi);
console.log(clamp(150, 0, 100), clamp(-5, 0, 100), clamp(42, 0, 100));
```

OUTPUT

```
91 64 -Infinity Infinity
1024 1024 12 3 NaN
5 3 10 1
100 0 42
```

10.2.3 Math.random() and random integers in a range

`Math.random()` returns a pseudo-random decimal number from 0 (inclusive) to 1 (exclusive). Scale and floor it to get integers in a range: `Math.floor(Math.random() * (max - min + 1)) + min` gives an integer from min to max inclusive. Math.random is **not** suitable for security (passwords, OTPs, tokens); use `crypto.getRandomValues` or `crypto.randomUUID` for that.

RANDOM NUMBERS (THE CHECKS PRINT THE SAME EVERY TIME)

main.js

```
const randomInt = (min, max) => Math.floor(Math.random() * (max - min + 1)) + min;
const r = Math.random();
console.log(r >= 0 && r < 1);
const rolls = Array.from({ length: 6000 }, () => randomInt(1, 6));
const counts = rolls.reduce((acc, n) => { acc[n] = (acc[n] ?? 0) + 1; return acc; }, {});
console.log("all faces 1-6 appear:", Object.keys(counts).join(","), " each roughly 1000 times:",
Object.values(counts).every((c) => c > 850 && c < 1150));
const pick = (arr) => arr[Math.floor(Math.random() * arr.length)];
console.log("picked a valid colour:", ["red", "green", "blue"].includes(pick(["red", "green",
"blue"])));
function shuffle(arr) {                          // Fisher-Yates shuffle
  const a = [...arr];
  for (let i = a.length - 1; i > 0; i--) { const j = Math.floor(Math.random() * (i + 1)); [a[i], a[j]]
= [a[j], a[i]]; }
  return a;
}
console.log("shuffled keeps the same items:", shuffle([1, 2, 3, 4, 5]).toSorted().join(",") ===
"1,2,3,4,5");
console.log("secure UUID format:", /^[0-9a-f-]{36}$/.test(crypto.randomUUID()));
```

OUTPUT

```
true
all faces 1-6 appear: 1,2,3,4,5,6  each roughly 1000 times: true
picked a valid colour: true
shuffled keeps the same items: true
secure UUID format: true
```

10.2.4 Math.PI and trigonometry

`Math.PI` and `Math.E` are the familiar constants. The trigonometric functions (`sin`, `cos`, `tan`, `atan2` and their inverses) work in **radians**; convert degrees with `deg * Math.PI / 180`. They are used in graphics, animation, games and geometry.

CONSTANTS AND TRIGONOMETRY main.js

```
const toRad = (deg) => (deg * Math.PI) / 180;
const toDeg = (rad) => (rad * 180) / Math.PI;
console.log(Math.PI, Math.E);
console.log(Math.sin(toRad(30)).toFixed(3), Math.cos(toRad(60)).toFixed(3),
Math.tan(toRad(45)).toFixed(3));
console.log(toDeg(Math.atan2(1, 1)), toDeg(Math.asin(0.5)).toFixed(1));
const r = 7;
console.log(`circle r=7: area ${(Math.PI * r ** 2).toFixed(2)}, circumference ${(2 * Math.PI *
r).toFixed(2)}`);
for (let deg = 0; deg < 360; deg += 90) console.log(`point at ${deg}°:
(${Math.round(Math.cos(toRad(deg)) * 10)}, ${Math.round(Math.sin(toRad(deg)) * 10)})`);
```

OUTPUT

```
3.141592653589793 2.718281828459045
0.500 0.500 1.000
45 30.0
circle r=7: area 153.94, circumference 43.98
point at 0°: (10, 0)
point at 90°: (0, 10)
point at 180°: (-10, 0)
point at 270°: (0, -10)
```

10.3 BigInt

10.3.1 Creating and using BigInt

`BigInt` (ES2020) represents integers of arbitrary size, beyond `Number.MAX_SAFE_INTEGER`. Create one with the `n` suffix or `BigInt(value)`. BigInts support `+ - * ** %` and `/` (which truncates), comparisons and bitwise operators, but not Math methods or decimals. Typical uses: large IDs (such as Twitter/X snowflake IDs), cryptography, exact factorials and money in the smallest unit across huge totals.

WORKING WITH BIGINT main.js

```
const big = 12345678901234567890123456789n;
const fromNumber = BigInt(Number.MAX_SAFE_INTEGER) + 2n;
console.log(big * 2n, fromNumber, typeof big);
console.log(2n ** 128n);
console.log(17n / 5n, 17n % 5n, -7n / 2n);
let f = 1n;
for (let i = 1n; i <= 30n; i++) f *= i;
console.log("30! =", f.toString(), f.toLocaleString("en-IN"));
console.log(BigInt("9007199254740993"), BigInt.asUintN(8, 257n));
```

OUTPUT

```
24691357802469135780246913578024691357802469135780n 9007199254740993n bigint
340282366920938463463374607431768211456n
3n 2n -3n
30! = 265252859812191058636308480000000 26,52,52,85,98,12,19,10,58,63,63,08,48,00,00,000
9007199254740993n 1n
```

10.3.2 Mixing BigInt and Number

BigInt and Number cannot be mixed in arithmetic: `1n + 1` throws a TypeError, so convert explicitly with `BigInt(n)` or `Number(b)` (which may lose precision). Comparisons do work across types (`1n < 2`, `1n == 1`), but `1n === 1` is false because the types differ. JSON.stringify cannot serialize BigInt directly.

BIGINT AND NUMBER DO NOT MIX

main.js

```
try { console.log(10n + 5); } catch (e) { console.log(`${e.name}: ${e.message}`); }
console.log(10n + BigInt(5), Number(10n) + 5);
console.log(Number(2n ** 60n), 2n ** 60n);                  // converting back may lose precision
console.log(1n < 2, 2n > 1.5, 1n == 1, 1n === 1);
try { JSON.stringify({ id: 1n }); } catch (e) { console.log(`${e.name}: ${e.message}`); }
console.log(JSON.stringify({ id: 1n }, (k, v) => (typeof v === "bigint" ? v.toString() : v)));
```

OUTPUT

```
TypeError: Cannot mix BigInt and other types, use explicit conversions
15n 15
1152921504606847000 1152921504606846976n
true true true false
TypeError: Do not know how to serialize a BigInt
{"id":"1"}
```

10.4 Date

The `Date` object represents a single moment in time, stored internally as the number of **milliseconds since 1 January 1970 00:00:00 UTC** (the Unix epoch). A Date has no time zone of its own: its methods show that moment either in the computer's local time zone or in UTC. The API has well-known pitfalls (months count from 0, objects are mutable), which the new Temporal API fixes (10.4.6).

10.4.1 Creating dates

`new Date()` is the current moment. `new Date(year, monthIndex, day, hours, minutes, seconds, ms)` builds a local date, where **months run from 0 (January) to 11 (December)**. `new Date(milliseconds)` uses a timestamp. `new Date(string)` parses a string; only the ISO format `"2026-03-09T10:30:00"` is reliable (a date-only ISO string is treated as UTC, a date-time string without zone as local). `Date.now()` returns the current timestamp as a number.

CREATING DATE OBJECTS

main.js

```
const d1 = new Date(2026, 2, 9);                  // 9 March 2026, local midnight (month 2 = March)
const d2 = new Date(2026, 2, 9, 14, 30, 0);        // with a time
const d3 = new Date(0);                            // the epoch
const d4 = new Date("2026-03-09T14:30:00");        // ISO date-time without zone: local time
const d5 = new Date("2026-03-09T09:00:00Z");       // Z = UTC
const d6 = new Date(Date.UTC(2026, 11, 25));       // a UTC date
console.log(d1.toString());
console.log(d2.toISOString(), "|", d4.getTime() === d2.getTime());
console.log(d3.toISOString(), d5.toISOString(), d6.toISOString());
console.log(new Date("not a date").toString(), Number.isNaN(new Date("x").getTime()));
const now = new Date();                            // current moment: differs every run
console.log("now is a Date after 2025:", now instanceof Date && now.getFullYear() > 2025, typeof
Date.now());
```

OUTPUT

```
Mon Mar 09 2026 00:00:00 GMT+0530 (India Standard Time)
2026-03-09T09:00:00.000Z | true
1970-01-01T00:00:00.000Z 2026-03-09T09:00:00.000Z 2026-12-25T00:00:00.000Z
Invalid Date true
now is a Date after 2025: true number
```

10.4.2 Getting and setting components

Getters read parts of a date in local time: `getFullYear`, `getMonth` (0 to 11), `getDate` (day of the month, 1 to 31), `getDay` (day of the week, 0 = Sunday), `getHours`, `getMinutes`, `getSeconds`, `getMilliseconds`, `getTime` (timestamp). Each has a `getUTC...` twin and a matching setter. Setters **modify the Date** and automatically roll over out-of-range values, which is handy for arithmetic.

READING AND CHANGING DATE PARTS

main.js

```
const d = new Date(2026, 2, 9, 14, 30, 45);
const days = ["Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"];
console.log(d.getFullYear(), d.getMonth(), d.getDate(), d.getDay(), days[d.getDay()]);
console.log(d.getHours(), d.getMinutes(), d.getSeconds(), d.getUTCHours(), d.getTimezoneOffset());
d.setFullYear(2027);
d.setMonth(11);            // December
d.setDate(31);
console.log(d.toDateString());
d.setDate(d.getDate() + 1); // rolls over into the next year
console.log(d.toDateString());
const jan31 = new Date(2026, 0, 31);
jan31.setMonth(1);         // 31 February does not exist: rolls over to 3 March
console.log(jan31.toDateString());
```

OUTPUT

```
2026 2 9 1 Monday
14 30 45 9 -330
Fri Dec 31 2027
Sat Jan 01 2028
Tue Mar 03 2026
```

10.4.3 Date arithmetic and differences

Subtracting two Dates gives the difference in **milliseconds** (they convert to timestamps). Divide by 1000 × 60 × 60 × 24 for days. To add days, months or years, use the setters, which handle month lengths and leap years; adding a fixed number of milliseconds per day can go wrong across daylight-saving changes in zones that use them.

DATE ARITHMETIC

main.js

```
const DAY = 24 * 60 * 60 * 1000;
const start = new Date(2026, 2, 9), end = new Date(2026, 11, 25);
console.log("days until 25 Dec:", Math.round((end - start) / DAY));

function addDays(date, n) { const d = new Date(date); d.setDate(d.getDate() + n); return d; }
function addMonths(date, n) { const d = new Date(date); d.setMonth(d.getMonth() + n); return d; }
console.log(addDays(start, 30).toDateString(), "|", addMonths(start, 10).toDateString());

const isLeap = (y) => new Date(y, 1, 29).getDate() === 29;
const daysInMonth = (y, m) => new Date(y, m + 1, 0).getDate();     // day 0 of next month = last day
of this one
console.log(isLeap(2024), isLeap(2026), daysInMonth(2024, 1), daysInMonth(2026, 3));

function age(birth, on) {
  let years = on.getFullYear() - birth.getFullYear();
  const hadBirthday = on.getMonth() > birth.getMonth() || (on.getMonth() === birth.getMonth() &&
on.getDate() >= birth.getDate());
  return hadBirthday ? years : years - 1;
}
console.log("age:", age(new Date(2004, 7, 15), new Date(2026, 2, 9)));
const t0 = performance.now(); for (let i = 0; i < 1e6; i++); const ms = performance.now() - t0;
console.log("timing works:", ms >= 0);
```

OUTPUT

```
days until 25 Dec: 291
Wed Apr 08 2026 | Sat Jan 09 2027
true false 29 30
age: 21
timing works: true
```

10.4.4 Formatting with toLocaleDateString() and Intl.DateTimeFormat

`toLocaleDateString`, `toLocaleTimeString` and `toLocaleString` format a Date for a given locale, with options to choose which parts appear and how. `Intl.DateTimeFormat` is the reusable formatter; `Intl.RelativeTimeFormat` produces phrases like "in 3 days". `toISOString` gives the standard machine format in UTC, ideal for storage and APIs.

FORMATTING DATES FOR PEOPLE main.js

```
const d = new Date(2026, 2, 9, 14, 5);
console.log(d.toLocaleDateString("en-IN"), "|", d.toLocaleDateString("en-US"), "|",
d.toLocaleDateString("de-DE"));
console.log(d.toLocaleDateString("en-IN", { weekday: "long", day: "numeric", month: "long", year:
"numeric" }));
console.log(d.toLocaleTimeString("en-IN", { hour: "2-digit", minute: "2-digit" }), "|",
d.toLocaleTimeString("en-GB"));
const fmt = new Intl.DateTimeFormat("en-IN", { dateStyle: "medium", timeStyle: "short" });
console.log(fmt.format(d));
console.log(new Intl.DateTimeFormat("hi-IN", { dateStyle: "full" }).format(d));
const rtf = new Intl.RelativeTimeFormat("en", { numeric: "auto" });
console.log(rtf.format(3, "day"), "|", rtf.format(-1, "day"), "|", rtf.format(-2, "week"));
console.log(d.toISOString(), "|", `${d.getDate()}/${d.getMonth() + 1}/${d.getFullYear()}`);
```

OUTPUT

```
9/3/2026 | 3/9/2026 | 9.3.2026
Monday, 9 March 2026
02:05 pm | 14:05:00
9 Mar 2026, 2:05 pm
सोमवार, 9 मार्च 2026
in 3 days | yesterday | 2 weeks ago
2026-03-09T08:35:00.000Z | 9/3/2026
```

10.4.5 Time zones and timestamps

A Date is a **timestamp**: the same instant everywhere. What differs is how it is displayed. Methods without UTC in their name use the time zone of the computer (or browser) running the code; `getTimezoneOffset()` returns the difference from UTC in minutes (negative east of Greenwich: -330 for India). To display in another zone, pass `timeZone` to the Intl formatters. Store and exchange times as UTC ISO strings or timestamps, and convert to local time only for display.

ONE INSTANT IN SEVERAL TIME ZONES main.js

```
const meeting = new Date("2026-03-09T09:00:00Z");          // 09:00 UTC
const zones = ["UTC", "Asia/Kolkata", "Europe/London", "America/New_York", "Asia/Tokyo"];
for (const tz of zones) {
  console.log(tz.padEnd(18), meeting.toLocaleString("en-GB", { timeZone: tz, dateStyle: "medium",
timeStyle: "short" }));
}
console.log("timestamp:", meeting.getTime(), " seconds:", Math.floor(meeting.getTime() / 1000));
console.log("local offset in minutes:", meeting.getTimezoneOffset());
console.log("same instant:", new Date(meeting.getTime()).toISOString() === meeting.toISOString());
```

OUTPUT

```
UTC                9 Mar 2026, 09:00
Asia/Kolkata       9 Mar 2026, 14:30
Europe/London      9 Mar 2026, 09:00
America/New_York   9 Mar 2026, 05:00
Asia/Tokyo         9 Mar 2026, 18:00
timestamp: 1773046800000  seconds: 1773046800
local offset in minutes: -330
same instant: true
```

10.4.6 Temporal API (the modern date/time API, overview)

Temporal is the new date and time API designed to replace Date. It reached TC39 stage 3, shipped in Firefox in 2025 and is rolling out to other engines (use the `@js-temporal/polyfill` package meanwhile). It

fixes Date's problems: objects are **immutable**, months count from 1, there are separate types for different needs, time zones and calendars are first-class, and arithmetic and differences are explicit and correct.

Temporal type	Represents	Example
`Temporal.PlainDate`	a calendar date without time or zone	a birthday, a holiday
`Temporal.PlainTime`	a wall-clock time	a shop opening time
`Temporal.PlainDateTime`	date and time without zone	a local appointment
`Temporal.ZonedDateTime`	an exact time in a specific time zone	a flight departure
`Temporal.Instant`	an exact moment (like a timestamp)	a log entry
`Temporal.Duration`	a length of time	2 hours 30 minutes

TEMPORAL AT A GLANCE (NEEDS AN ENGINE WITH TEMPORAL OR THE POLYFILL)

main.js

```
const birthday = Temporal.PlainDate.from("2004-08-15");
const today = Temporal.PlainDate.from("2026-03-09");
console.log(birthday.month);                                    // 8: months start at 1
console.log(today.add({ months: 1, days: 10 }).toString());    // returns a NEW object
console.log(birthday.until(today, { largestUnit: "years" }).toString());   // a Duration

const flight = Temporal.ZonedDateTime.from("2026-03-09T23:30[Asia/Kolkata]");
console.log(flight.withTimeZone("Europe/London").toString());
```

OUTPUT

```
8
2026-04-19
P21Y6M22D
2026-03-09T18:00:00+00:00[Europe/London]
```

10.5 Timers (introduction)

10.5.1 setTimeout, setInterval, clearTimeout, clearInterval

`setTimeout(callback, ms, ...args)` runs a function once after at least `ms` milliseconds and returns a timer id. `setInterval(callback, ms)` runs it repeatedly every `ms` milliseconds. `clearTimeout(id)` and `clearInterval(id)` cancel them. The delay is a **minimum**: the callback runs only when the event loop is free (Module 17), so timers are not precise and never interrupt running code.

ONE-OFF AND REPEATING TIMERS

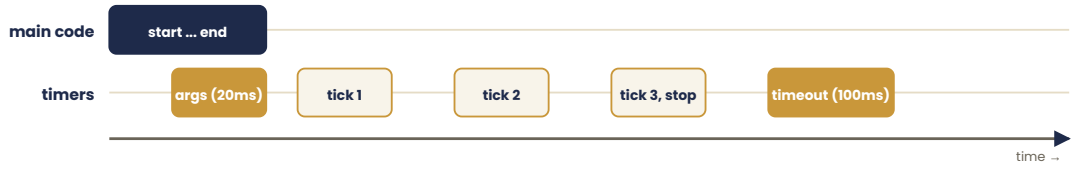
main.js

```
console.log("start");
const id1 = setTimeout(() => console.log("timeout after 100 ms"), 100);
const id2 = setTimeout(() => console.log("this one is cancelled"), 50);
clearTimeout(id2);
setTimeout((name, n) => console.log(`arguments passed: ${name}, ${n}`), 20, "Asha", 3);

let ticks = 0;
const interval = setInterval(() => {
  ticks++;
  console.log(`tick ${ticks}`);
  if (ticks === 3) {
    clearInterval(interval);
    console.log("interval stopped");
  }
}, 30);
console.log("end of synchronous code");
```

OUTPUT

```
start
end of synchronous code
arguments passed: Asha, 3
tick 1
tick 2
tick 3
interval stopped
timeout after 100 ms
```



CALLBACKS RUN AFTER THE SYNCHRONOUS CODE, IN ORDER OF THEIR DUE TIMES

Number methods format (toFixed, toPrecision, toString(base)) and test values without coercion (Number.isInteger, isFinite, isSafeInteger); compare floats with a tolerance based on Number.EPSILON; format for people with toLocaleString and Intl.NumberFormat (currency, percent, compact). Math provides rounding (round, floor, ceil, trunc), max/min, powers, roots, constants, trigonometry in radians and Math.random (not for security; use crypto). BigInt handles huge integers but does not mix with Number. Date stores milliseconds since the 1970 epoch; months start at 0, setters roll over, differences are in milliseconds, Intl.DateTimeFormat formats and timeZone converts for display; Temporal is the modern replacement. setTimeout and setInterval schedule callbacks that clearTimeout and clearInterval cancel.

PRACTICE CHECKPOINT

Build an **age calculator** from a birth date, a **countdown timer** to a chosen date, and a **random OTP generator**.

SAMPLE SOLUTION

main.js

```
// 1. Age calculator (years, months and days) as of a fixed "today" so the output is repeatable
function ageBetween(birth, today) {
  let years = today.getFullYear() - birth.getFullYear();
  let months = today.getMonth() - birth.getMonth();
  let days = today.getDate() - birth.getDate();
```

```
  if (days < 0) {
    months--;
    days += new Date(today.getFullYear(), today.getMonth(), 0).getDate();   // days in the previous
month
  }
  if (months < 0) { years--; months += 12; }
  return { years, months, days };
}
const today = new Date(2026, 2, 9);
const birth = new Date(2004, 7, 15);
const a = ageBetween(birth, today);
console.log(`Born ${birth.toLocaleDateString("en-IN", { dateStyle: "long" })}: ${a.years} years,
${a.months} months, ${a.days} days`);

// 2. Countdown to a chosen date (simulated clock: three ticks one second apart)
const target = new Date(2026, 11, 31, 23, 59, 59);
function remaining(from) {
  let s = Math.max(0, Math.floor((target - from) / 1000));
  const d = Math.floor(s / 86400); s %= 86400;
  const h = Math.floor(s / 3600); s %= 3600;
  const m = Math.floor(s / 60);
  return `${d}d ${String(h).padStart(2, "0")}h ${String(m).padStart(2, "0")}m ${String(s %
60).padStart(2, "0")}s`;
}
let fakeNow = new Date(2026, 2, 9, 10, 0, 0).getTime();
let ticks = 0;
const timer = setInterval(() => {
  console.log("Time left to New Year's Eve:", remaining(new Date(fakeNow)));
  fakeNow += 1000;
  if (++ticks === 3) clearInterval(timer);
}, 10);

// 3. OTP generator using the cryptographically secure generator
function generateOtp(length = 6) {
  const digits = new Uint32Array(length);
  crypto.getRandomValues(digits);
  return Array.from(digits, (n) => n % 10).join("");
}
const otps = Array.from({ length: 5 }, () => generateOtp());
console.log("5 OTPs generated, all 6 digits:", otps.every((o) => /^\d{6}$/.test(o)));
console.log("8-digit OTP has length:", generateOtp(8).length);
```

OUTPUT

```
Born 15 August 2004: 21 years, 6 months, 22 days
5 OTPs generated, all 6 digits: true
8-digit OTP has length: 8
Time left to New Year's Eve: 297d 13h 59m 59s
Time left to New Year's Eve: 297d 13h 59m 58s
Time left to New Year's Eve: 297d 13h 59m 57s
```

LEVEL 2

Intermediate level

Outcome: master closures and this, modern ES6+ features, prototypes and classes, the DOM, events, asynchronous code and APIs.

8 MODULES · 26 DAYS MINIMUM

Module 11 · Scope, closures and the this keyword	**4 days**
Module 12 · Modern JavaScript (ES6 and beyond)	**3 days**
Module 13 · Object-oriented JavaScript: prototypes and classes	**4 days**
Module 14 · Error handling and debugging	**1 day**
Module 15 · The DOM (Document Object Model)	**4 days**
Module 16 · Events	**3 days**
Module 17 · Asynchronous JavaScript	**4 days**
Module 18 · Working with APIs and browser storage	**3 days**

MODULE 11 · INTERMEDIATE

Scope, closures and the this keyword

Minimum duration **4 days** | Level **Intermediate**

These are the concepts that separate beginners from confident JavaScript developers, and they appear in almost every interview. This module explains execution contexts and the call stack, scope and the scope chain, hoisting in detail, closures and their practical uses, every rule that decides the value of this, explicit binding with call, apply and bind, and functional techniques built on closures: currying, composition, debounce and throttle.

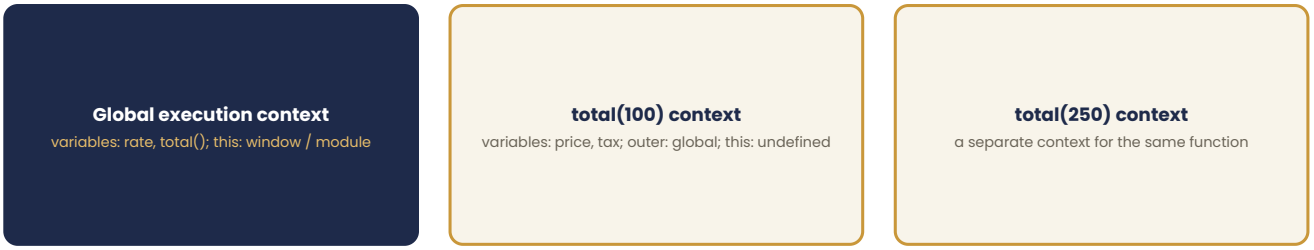
IN THIS MODULE

11.1 Execution context
11.2 Scope
11.3 Hoisting
11.4 Closures
11.5 The this keyword
11.6 Explicit binding
11.7 Functional programming techniques

11.1 Execution context

11.1.1 Global and function execution contexts

Whenever JavaScript runs code, it does so inside an **execution context**: an internal record of everything the code needs, namely its variables and functions (the **variable environment**), a link to the outer scope (for the **scope chain**), and the value of `this`. When a script starts, the engine creates the **global execution context**. Every function **call** creates a new **function execution context** for that call, with its own parameters and local variables, even when the same function is called many times (each recursive call has its own).



EACH CALL GETS ITS OWN EXECUTION CONTEXT

11.1.2 Creation phase and execution phase

Each execution context is set up in two phases. In the **creation phase**, before any line runs, the engine scans the code: it creates the scope, registers `var` variables with the value `undefined`, registers `let`/`const` without initialising them (the Temporal Dead Zone), stores complete function declarations, and determines `this`. In the **execution phase**, the code runs line by line, assigning values and calling functions. The creation phase is what we call **hoisting** (11.3).

WHAT EXISTS BEFORE THE CODE RUNS main.js

```
console.log("before declarations:");
console.log("  var a =", a);                      // registered as undefined in the creation phase
console.log("  typeof greet =", typeof greet);     // function declarations are stored completely
try { console.log(b); } catch (e) { console.log("  let b ->", e.message); }

var a = 1;
let b = 2;
function greet() { return "hi"; }

console.log("after the execution phase reached them:", a, b, greet());
```

OUTPUT

```
before declarations:
  var a = undefined
  typeof greet = function
  let b -> Cannot access 'b' before initialization
after the execution phase reached them: 1 2 hi
```

11.1.3 The call stack

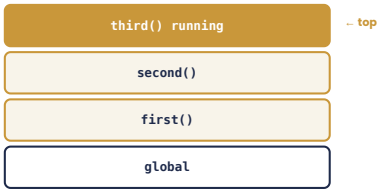
The engine keeps execution contexts on the **call stack**. The global context is at the bottom; each function call pushes its context on top, and returning pops it, resuming the context below. Only the context on top is running, which is what "single-threaded" means. The stack trace in an error message is a snapshot of this stack.

CONTEXTS PUSHED AND POPPED ON THE CALL STACK main.js

```
function first() {
  console.log("first starts");
  second();
  console.log("first ends");
}
function second() {
  console.log("second starts");
  third();
  console.log("second ends");
}
function third() {
  console.log("third runs; current stack:");
  console.log(new Error().stack.split("\n").slice(1, 4).map((l) => "    " + l.trim().split(" (")
[0]).join("\n"));
}
first();
```

OUTPUT

```
first starts
second starts
third runs; current stack:
   at third
   at second
   at first
second ends
first ends
```



THE CALL STACK WHILE THIRD() RUNS

11.2 Scope

11.2.1 Global, function, block and module scope

Scope decides where a variable is visible. JavaScript has four kinds: **global** scope (visible everywhere; in browsers, top-level `var` and function declarations also become properties of `window`); **function** scope (variables declared anywhere in a function, including all `var`s); **block** scope (`let`, `const` and `class` inside `{ }`); and **module** scope (top-level variables of an ES module are private to that module unless exported).

THE KINDS OF SCOPE — main.js

```
var globalVar = "global (var)";
let globalLet = "global (let)";

function demo() {
  var fnVar = "function scope";
  if (true) {
    var alsoFnScope = "var ignores the block";
    let blockLet = "block scope";
    const blockConst = "block scope too";
    console.log(globalVar, "|", fnVar, "|", blockLet);
  }
  console.log(alsoFnScope, "|", typeof blockLet);
}
demo();
console.log(typeof fnVar, typeof globalLet);
```

OUTPUT

```
global (var) | function scope | block scope
var ignores the block | undefined
undefined string
```

TOP-LEVEL VAR AND FUNCTIONS BECOME WINDOW PROPERTIES IN A CLASSIC BROWSER SCRIPT — main.js

```
var oldStyle = "on window";
let modern = "not on window";
function helper() {}
console.log(window.oldStyle, window.modern, typeof window.helper);
```

BROWSER CONSOLE

```
on window undefined function
```

11.2.2 Lexical scope and the scope chain

JavaScript uses **lexical** (static) scope: which variables a function can see is decided by **where the function is written** in the source code, not where it is called from. When code uses a name, the engine looks in the current scope, then in the enclosing scope, and so on outwards up to the global scope: this is

the **scope chain**. If the name is not found anywhere, a ReferenceError is thrown. Inner scopes can see outer variables; outer scopes never see inner ones.

LEXICAL SCOPE: WHERE A FUNCTION IS WRITTEN MATTERS

main.js

```
const level = "global";

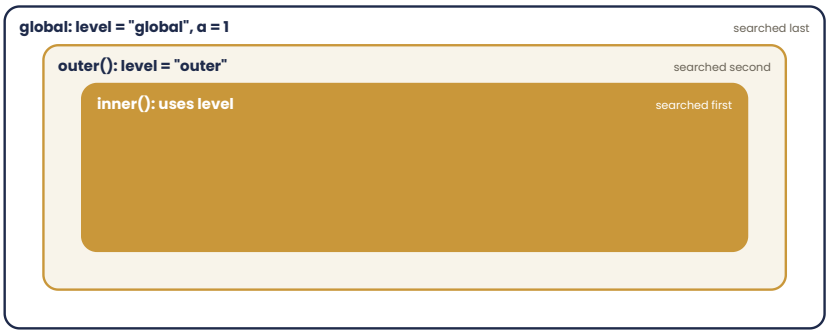
function outer() {
  const level = "outer";
  function inner() {
    console.log("inner sees:", level);          // found in outer: the nearest enclosing scope
  }
  return inner;
}

function caller() {
  const level = "caller";                       // irrelevant: scope is decided where inner was
WRITTEN
  const fn = outer();
  fn();
}
caller();

const a = 1;
function f1() { const b = 2; function f2() { const c = 3; console.log(a + b + c); } f2(); }
f1();                                            // f2 reaches c, b and a through the scope chain
```

OUTPUT

```
inner sees: outer
6
```



THE SCOPE CHAIN IS SEARCHED FROM THE INSIDE OUT

11.2.3 Shadowing

A variable in an inner scope with the same name as an outer one **shadows** it: inside the inner scope, the name refers to the inner variable and the outer one is unreachable by that name. Shadowing a parameter or a widely used name can cause confusing bugs; linters can warn about it (the ESLint rule no-shadow).

INNER NAMES HIDE OUTER ONES

main.js

```
let count = 10;
function increment(count) {        // the parameter shadows the outer count
  count++;
  return count;
}
console.log(increment(1), count);  // the outer count is untouched

const user = "Asha";
{
  const user = "Ravi";             // shadows inside this block only
  console.log("inside:", user);
}
console.log("outside:", user);
```

OUTPUT

```
2 10
inside: Ravi
outside: Asha
```

11.3 Hoisting

Hoisting is the effect of the creation phase: declarations are processed before the code runs, so they behave as if they had been moved to the top of their scope. Only the **declaration** is hoisted, never the assignment of a value; and the various declaration kinds are hoisted differently.

Declaration	Hoisted?	Value before the declaration line
`var x = 1`	yes	`undefined`
`let x`, `const x`	yes, but uninitialised	ReferenceError (Temporal Dead Zone)
`class X {}`	yes, but uninitialised	ReferenceError (TDZ)
`function f() {}`	yes, with its body	the complete function: can be called
`const f = function () {}` / arrow	only the variable (as const/let/var)	TDZ error, or undefined with var (calling it is a TypeError)

11.3.1 Hoisting of var declarations

A `var` declaration is hoisted to the top of its **function** (not its block) and initialised to `undefined`. The assignment stays where it is written. This explains classic puzzles where a variable seems to exist "too early" or an outer variable seems to vanish.

VAR IS HOISTED AS UNDEFINED main.js

```
console.log(score);      // undefined: the declaration was hoisted, not the value
var score = 90;
console.log(score);

var name = "global";
function show() {
  console.log(name);      // undefined, not "global": the local var below is hoisted to here
  var name = "local";
  console.log(name);
}
show();
```

OUTPUT

```
undefined
90
undefined
local
```

THE HOISTED VIEW OF SHOW() JavaScript

```
// How the engine effectively sees show():
function show() {
  var name;               // declaration hoisted to the top, value undefined
  console.log(name);
  name = "local";         // assignment stays in place
  console.log(name);
}
```

11.3.2 Hoisting of function declarations vs function expressions

A function **declaration** is hoisted together with its body, so it can be called before it appears. A function **expression** (or arrow function) assigned to a variable is only as hoisted as that variable: with `var` it is `undefined` (calling it throws "is not a function"), with `let` / `const` it is in the TDZ.

DECLARATIONS VERSUS EXPRESSIONS main.js

```
console.log(declared());                  // works
function declared() { return "declaration: hoisted with its body"; }

try { expressedVar(); } catch (e) { console.log("var expression:", e.message); }
var expressedVar = function () { return "too late"; };

try { arrowConst(); } catch (e) { console.log("const arrow:", e.message); }
const arrowConst = () => "too late";
```

OUTPUT

```
declaration: hoisted with its body
var expression: expressedVar is not a function
const arrow: Cannot access 'arrowConst' before initialization
```

11.3.3 let, const and the Temporal Dead Zone

`let`, `const` and `class` declarations are hoisted to the top of their **block** but stay **uninitialised** until the declaration line runs. The period between the start of the block and the declaration is the **Temporal Dead Zone** (TDZ); any access in it throws a ReferenceError. The TDZ is "temporal" because it is about **when** code

runs, not where it is written: a function declared above a let can use it, as long as it is called after the declaration has executed.

THE TEMPORAL DEAD ZONE main.js

```
let x = "outer";
{
  // TDZ for the inner x starts here
  try { console.log(x); } catch (e) { console.log(e.message); }   // not "outer": the inner x shadows
it already
  let x = "inner";                                                // TDZ ends
  console.log(x);
}

function useLater() { return config.mode; }   // refers to config, declared below
const config = { mode: "production" };
console.log(useLater());                        // fine: called after config was initialised
try { typeof notYet; let notYet = 1; } catch (e) { console.log("even typeof throws in the TDZ:",
e.name); }
```

OUTPUT

```
Cannot access 'x' before initialization
inner
production
even typeof throws in the TDZ: ReferenceError
```

11.4 Closures

11.4.1 What a closure is

A **closure** is a function together with the scope in which it was created. Because of lexical scope, a function can use variables of its enclosing function; a closure means it can keep doing so **even after the enclosing function has returned**. Every function in JavaScript is a closure; the term matters when an inner function outlives its outer function, typically because it is returned, stored, or passed as a callback.

A FUNCTION THAT REMEMBERS ITS BIRTHPLACE main.js

```
function makeGreeter(greeting) {
  const punctuation = "!";
  return function (name) {                    // this inner function closes over greeting and
punctuation
    return `${greeting}, ${name}${punctuation}`;
  };
}
const hello = makeGreeter("Hello");           // makeGreeter has finished...
const namaste = makeGreeter("Namaste");
console.log(hello("Asha"));                   // ...but its variables are still available
console.log(namaste("Ravi"));
```

OUTPUT

```
Hello, Asha!
Namaste, Ravi!
```

11.4.2 How closures remember outer variables

A closure keeps a **reference** to the outer variables themselves, not a copy of their values. If the variable changes, the closure sees the new value; if the closure changes it, every other closure sharing that scope

sees the change. Each call of the outer function creates a **new** scope, so closures from different calls are independent. As long as a closure is reachable, its captured variables stay in memory.

CLOSURES SHARE LIVE VARIABLES

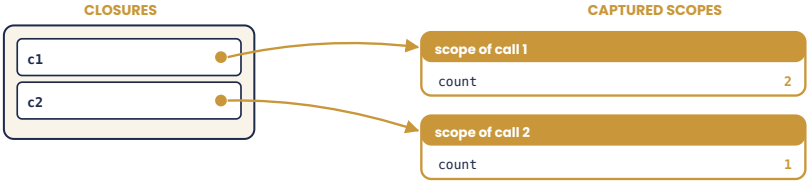
main.js

```
function makeCounter() {
  let count = 0;                              // lives on after makeCounter returns
  return {
    increment: () => ++count,
    decrement: () => --count,
    value: () => count,
  };
}
const c1 = makeCounter();
const c2 = makeCounter();                     // a separate scope with its own count
c1.increment(); c1.increment(); c1.increment(); c1.decrement();
c2.increment();
console.log("c1:", c1.value(), " c2:", c2.value());

let message = "first";
const show = () => message;                   // captures the variable, not its current value
message = "changed later";
console.log(show());
```

OUTPUT

```
c1: 2  c2: 1
changed later
```



EACH COUNTER KEEPS ITS OWN CAPTURED SCOPE ALIVE

11.4.3 Practical uses: data privacy, counters, function factories, memoization

Closures are behind many everyday patterns. **Data privacy**: variables in the outer function cannot be reached from outside, only through the functions returned. **Stateful functions** such as counters and ID generators. **Function factories** that produce customised functions. **Memoization**: caching results in a variable that only the function can see. Event handlers, timers and callbacks also rely on closures to remember the data they need.

CLOSURES IN PRACTICE

main.js

```
function createAccount(initial) {                // data privacy
  let balance = initial;                          // cannot be changed directly from outside
  return {
    deposit(amount) { if (amount > 0) balance += amount; return balance; },
    getBalance: () => balance,
  };
}
const acc = createAccount(1000);
acc.deposit(500);
acc.balance = 1_000_000;                          // adds an unrelated property; the real balance is
safe
console.log(acc.getBalance());

const nextId = (() => { let id = 100; return () => `ID-${++id}`; })();   // generator of unique ids
console.log(nextId(), nextId(), nextId());

const withTax = (rate) => (price) => +(price * (1 + rate)).toFixed(2);  // factory
const gst18 = withTax(0.18), gst5 = withTax(0.05);
console.log(gst18(1000), gst5(1000));

function memoize(fn) {                            // memoization
  const cache = new Map();
  return (n) => {
    if (cache.has(n)) return `${cache.get(n)} (cached)`;
    const result = fn(n);
    cache.set(n, result);
    return result;
  };
}
const slowSquare = (n) => { for (let i = 0; i < 1e6; i++); return n * n; };
const fastSquare = memoize(slowSquare);
console.log(fastSquare(12), fastSquare(12), fastSquare(13));
```

OUTPUT

```
1500
ID-101 ID-102 ID-103
1180 1050
144 144 (cached) 169
```

11.4.4 Closures in loops (var vs let)

The most famous closure puzzle: functions created in a loop with `var` all see the **same** variable, because var is function-scoped, so after the loop they all report its final value. With `let`, each iteration gets a **new** binding, so each function remembers its own value. Before ES2015, the fix was an IIFE that captured the current value as a parameter.

THE LOOP CLOSURE PUZZLE

main.js

```
var callbacksVar = [];
for (var i = 0; i < 3; i++) callbacksVar.push(() => i);
console.log("var:", callbacksVar.map((f) => f()));            // all see the one i, now 3

const callbacksLet = [];
for (let j = 0; j < 3; j++) callbacksLet.push(() => j);
console.log("let:", callbacksLet.map((f) => f()));            // each iteration has its own j

var callbacksIife = [];
for (var k = 0; k < 3; k++) callbacksIife.push(((captured) => () => captured)(k));   // old fix
console.log("IIFE:", callbacksIife.map((f) => f()));

for (var t = 1; t <= 3; t++) setTimeout(() => console.log("timeout with var sees", t), 0);
```

OUTPUT

```
var: [ 3, 3, 3 ]
let: [ 0, 1, 2 ]
IIFE: [ 0, 1, 2 ]
timeout with var sees 4
timeout with var sees 4
timeout with var sees 4
```

11.4.5 Module pattern with closures

Before ES modules, the **module pattern** organised code into self-contained units: an IIFE holds private variables and functions and returns an object exposing a public API. The **revealing module pattern** defines everything privately and returns references to the parts that should be public. ES modules (Module 19) now provide privacy natively, but the pattern still appears in older code and shows clearly how closures create encapsulation.

THE REVEALING MODULE PATTERN

main.js

```
const ShoppingCart = (function () {
  const items = [];                                  // private state
  const TAX = 0.18;                                  // private constant
  function subtotal() { return items.reduce((s, i) => s + i.price * i.qty, 0); }   // private helper

  function add(name, price, qty = 1) { items.push({ name, price, qty }); }
  function total() { return +(subtotal() * (1 + TAX)).toFixed(2); }
  function count() { return items.reduce((n, i) => n + i.qty, 0); }

  return { add, total, count };                      // revealing module: public API
})();

ShoppingCart.add("Notebook", 60, 3);
ShoppingCart.add("Pen", 10, 5);
console.log(ShoppingCart.count(), ShoppingCart.total());
console.log(ShoppingCart.items, typeof ShoppingCart.subtotal);    // private parts are not reachable
```

OUTPUT

```
8 271.4
undefined undefined
```

11.5 The this keyword

`this` is a special value available inside functions. Unlike ordinary variables, it is **not** determined by where the function is written but, for regular functions, by **how the function is called**. Arrow functions are the exception: they take `this` from their surrounding scope. The rules below, applied in order, decide its value.

Call form	this is	Example
`new Fn()`	the newly created object	`new Student("Asha")`
`fn.call(obj)`, `apply`, `bind`	the object passed explicitly	`greet.call(user)`
`obj.method()`	the object before the dot	`cart.total()`
plain call `fn()`	`undefined` in strict mode; the global object in sloppy mode	`const f = obj.m; f()`
arrow function	the `this` of the enclosing scope (lexical)	`() => this.x`
DOM event handler (regular function)	the element the listener is attached to	`btn.addEventListener("click", function () {...})`

11.5.1 this in the global scope

At the top level of a classic browser script, `this` is the global object `window`. At the top level of an ES module it is `undefined`. In a Node.js CommonJS file it is `module.exports` (an empty object at first). `globalThis` (ES2020) always refers to the global object in every environment.

TOP-LEVEL THIS IN NODE.JS main.js

```
console.log("Node.js CommonJS top level:", this === module.exports, JSON.stringify(this));
console.log("globalThis is the global object:", typeof globalThis.setTimeout === "function");
```

OUTPUT

```
Node.js CommonJS top level: true {}
globalThis is the global object: true
```

TOP-LEVEL THIS IN A BROWSER SCRIPT main.js

```
console.log("classic script:", this === window);
console.log("globalThis === window:", globalThis === window);
```

BROWSER CONSOLE

```
classic script: true
globalThis === window: true
```

TOP-LEVEL THIS IN A BROWSER MODULE main.js

```
console.log("module script:", this);
```

BROWSER CONSOLE

```
module script: undefined
```

11.5.2 this in regular functions (strict vs non-strict)

When a regular function is called on its own (`fn()`, no object before it), `this` is `undefined` in strict mode, and the global object in non-strict ("sloppy") mode. Strict mode is the safer behaviour: code that wrongly relies on `this` fails loudly instead of silently reading or creating global variables. Classes and modules are always strict.

PLAIN FUNCTION CALLS main.js

```
function sloppy() { return this === globalThis; }
function strict() { "use strict"; return this; }
console.log("sloppy plain call: this is the global object?", sloppy());
console.log("strict plain call: this is", strict());
```

OUTPUT

```
sloppy plain call: this is the global object? true
strict plain call: this is undefined
```

11.5.3 this in object methods

When a function is called as a method, `obj.method()`, `this` is the object before the dot, at the moment of the call. The same function can be attached to several objects and will see a different `this` for each. Only the last object in a chain counts: in `a.b.c()`, this is `a.b`.

THE OBJECT BEFORE THE DOT main.js

```
function describe() { return `${this.name} costs Rs ${this.price}`; }
const pen = { name: "Pen", price: 10, describe };
const bag = { name: "Bag", price: 899, describe };
console.log(pen.describe());
console.log(bag.describe());

const shop = {
  name: "LBF Store",
  owner: { name: "Asha", whoAmI() { return this.name; } },
};
console.log(shop.owner.whoAmI());              // this is shop.owner, not shop
```

OUTPUT

```
Pen costs Rs 10
Bag costs Rs 899
Asha
```

11.5.4 this in arrow functions (lexical this)

Arrow functions do not have their own `this`; they use the `this` of the scope where they are **defined**, exactly like any other outer variable. Neither the call form nor `call` / `apply` / `bind` can change it. This makes arrows ideal for callbacks inside methods (they keep the method's `this`), and unsuitable as object methods or constructors.

ARROW FUNCTIONS INHERIT THIS main.js

```
const team = {
  name: "Blue",
  members: ["Asha", "Ravi"],
  listWithArrow() {
    return this.members.map((m) => `${m} is in team ${this.name}`);       // arrow: uses
listWithArrow's this
  },
  listWithFunction() {
    return this.members.map(function (m) { return `${m} is in team ${this?.name}`; });   // own this:
undefined here
  },
  arrowMethod: () => typeof this,                                          // this from the module
scope
};
console.log(team.listWithArrow());
console.log(team.listWithFunction());
console.log(team.arrowMethod.call(team));                                  // call cannot change an
arrow's this
```

OUTPUT

```
[ 'Asha is in team Blue', 'Ravi is in team Blue' ]
[ 'Asha is in team undefined', 'Ravi is in team undefined' ]
object
```

11.5.5 this in constructors and classes

When a function is called with `new`, or a class is instantiated, `this` is the brand-new object being created, so the constructor can set its properties. Inside class methods, `this` follows the method rules: it is the instance when called as `instance.method()`. Class bodies are strict mode, so a detached method gets `this === undefined`.

THIS IN CONSTRUCTORS AND CLASSES main.js

```
function Student(name) {          // constructor function
  this.name = name;                // this = the new object
  this.createdBy = "constructor";
}
const s = new Student("Asha");
console.log(s);

class Counter {
  count = 0;
  increment() { this.count++; return this; }   // this = the instance; returning it allows chaining
}
const c = new Counter();
c.increment().increment().increment();
console.log(c.count);
const detached = c.increment;
try { detached(); } catch (e) { console.log("detached class method:", e.message); }
```

OUTPUT

```
Student { name: 'Asha', createdBy: 'constructor' }
3
detached class method: Cannot read properties of undefined (reading 'count')
```

11.5.6 this in event handlers

Inside a DOM event listener written as a **regular function**, `this` is the element the listener is attached to (the same as `event.currentTarget`). With an **arrow function**, `this` comes from the surrounding code instead, so use `event.currentTarget` when you need the element.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) — HTML

```
<button id="save">Save</button> <button id="cancel">Cancel</button>
```

THIS INSIDE EVENT LISTENERS — main.js

```
const buttons = document.querySelectorAll("button");
buttons[0].addEventListener("click", function (event) {
  console.log("regular function: this is", this.id, "| same as currentTarget:", this ===
event.currentTarget);
});
buttons[1].addEventListener("click", (event) => {
  console.log("arrow function: this is window?", this === window, "| use currentTarget:",
event.currentTarget.id);
});
buttons[0].click();
buttons[1].click();
```

BROWSER CONSOLE

```
regular function: this is save | same as currentTarget: true
arrow function: this is window? true | use currentTarget: cancel
```

11.5.7 Losing this in callbacks

The most common `this` bug: passing a method as a callback (`setTimeout(obj.method, 100)`, `button.addEventListener("click", obj.method)`, `arr.map(obj.method)`) passes only the function. It is later called without its object, so `this` is lost. Fix it with an arrow wrapper, `bind`, or a class field holding an arrow function.

LOSING THIS, AND THREE FIXES — main.js

```
class Timer {
  seconds = 0;
  label = "my timer";
  tick() { this.seconds++; return `${this.label}: ${this.seconds}`; }
  tickArrow = () => { this.seconds++; return `${this.label} (arrow field): ${this.seconds}`; };
}
const t = new Timer();
const run = (callback) => { try { return callback(); } catch (e) { return `lost this -> ${e.message}`;
} };

console.log(run(t.tick));                    // passed alone: this is lost
console.log(run(() => t.tick()));            // fix 1: arrow wrapper keeps the call form
console.log(run(t.tick.bind(t)));            // fix 2: bind
console.log(run(t.tickArrow));               // fix 3: arrow class field
```

OUTPUT

```
lost this -> Cannot read properties of undefined (reading 'seconds')
my timer: 1
my timer: 2
my timer (arrow field): 3
```

11.6 Explicit binding

Every function has three methods to set `this` explicitly. `call` and `apply` call the function immediately with a given `this`; `bind` returns a new function with `this` fixed permanently.

11.6.1 call()

`fn.call(thisArg, arg1, arg2, ...)` calls fn with `this` set to thisArg and the remaining arguments passed individually.

CALL SETS THIS AND PASSES ARGUMENTS ONE BY ONE main.js

```
function introduce(greeting, punctuation) {
  return `${greeting}, I am ${this.name} from ${this.city}${punctuation}`;
}
const asha = { name: "Asha", city: "Pune" };
const ravi = { name: "Ravi", city: "Delhi" };
console.log(introduce.call(asha, "Hello", "!"));
console.log(introduce.call(ravi, "Namaste", "."));
```

OUTPUT

```
Hello, I am Asha from Pune!
Namaste, I am Ravi from Delhi.
```

11.6.2 apply()

`fn.apply(thisArg, [args])` is the same as call but takes the arguments as an **array**. Before spread syntax it was the way to pass an array as separate arguments (`Math.max.apply(null, arr)`); today `fn.call(thisArg, ...arr)` or `fn(...arr)` does the same.

APPLY TAKES AN ARRAY OF ARGUMENTS main.js

```
function introduce(greeting, punctuation) { return `${greeting}, I am ${this.name}${punctuation}`; }
console.log(introduce.apply({ name: "Meera" }, ["Hi", "!"]));
const marks = [72, 91, 64];
console.log(Math.max.apply(null, marks), Math.max(...marks));
```

OUTPUT

```
Hi, I am Meera!
91 91
```

11.6.3 bind()

`fn.bind(thisArg, ...presetArgs)` returns a **new function** whose `this` is permanently thisArg (it cannot be rebound, even with call). Any preset arguments are fixed too. bind is the classic fix for passing methods as callbacks.

BIND FIXES THIS PERMANENTLY

main.js

```
const user = {
  name: "Kiran",
  greet() { return `Hi, I am ${this.name}`; },
};
const loose = user.greet;
const bound = user.greet.bind(user);
console.log(typeof loose.call(undefined) === "string" ? "?" : "", bound());
console.log(bound.call({ name: "Someone else" }));        // bound this cannot be changed
setTimeout(user.greet.bind(user), 0);                     // safe as a callback
setTimeout(() => console.log("timer:", bound()), 5);
```

OUTPUT

```
? Hi, I am Kiran
Hi, I am Kiran
timer: Hi, I am Kiran
```

11.6.4 Function borrowing and partial application

Function borrowing uses a method of one object on another with call or apply, for example array methods on array-like objects such as `arguments` or a NodeList. **Partial application** fixes some arguments of a function in advance, producing a more specific function; bind does it by passing arguments after thisArg.

BORROWING METHODS AND FIXING ARGUMENTS

main.js

```
const arrayLike = { 0: "a", 1: "b", 2: "c", length: 3 };
console.log(Array.prototype.map.call(arrayLike, (x) => x.toUpperCase()));   // borrowing map
console.log(Array.prototype.join.call(arrayLike, "-"));

const person = { name: "Asha", describe() { return `${this.name}`; } };
console.log(person.describe.call({ name: "Borrowed" }));

function multiply(a, b) { return a * b; }
const double = multiply.bind(null, 2);          // partial application: a = 2
const triple = multiply.bind(null, 3);
console.log(double(21), triple(7));
const logLevel = (level, msg) => `[${level}] ${msg}`;
const warn = (msg) => logLevel("WARN", msg);       // partial application with a closure
console.log(warn("disk almost full"));
```

OUTPUT

```
[ 'A', 'B', 'C' ]
a-b-c
Borrowed
42 21
[WARN] disk almost full
```

11.7 Functional programming techniques

11.7.1 Currying

Currying transforms a function of several arguments into a chain of functions of one argument each: `f(a, b, c)` becomes `f(a)(b)(c)`. Each step returns a closure remembering the arguments so far. Curried functions are easy to specialise and to compose. A generic `curry` helper converts any function.

CURRYING

main.js

```
const add3 = (a) => (b) => (c) => a + b + c;          // manually curried
console.log(add3(1)(2)(3));

function curry(fn) {
  return function curried(...args) {
    return args.length >= fn.length ? fn(...args) : (...more) => curried(...args, ...more);
  };
}
const price = (taxRate, discount, amount) => +(amount * (1 - discount) * (1 + taxRate)).toFixed(2);
const curriedPrice = curry(price);
const withGst = curriedPrice(0.18);                   // fix the tax rate
const saleWithGst = withGst(0.1);                     // fix the discount
console.log(saleWithGst(1000), saleWithGst(250), curriedPrice(0.05, 0, 100), curriedPrice(0.05)(0)
(100));
```

OUTPUT

```
6
1062 265.5 105 105
```

11.7.2 Function composition and pipe

Composition builds a new function from small ones: `compose(f, g)(x)` is `f(g(x))`, applying functions from right to left. **pipe** applies them left to right, which reads like a sequence of steps. Both are a one-line `reduce`.

COMPOSE AND PIPE

main.js

```
const compose = (...fns) => (x) => fns.reduceRight((acc, fn) => fn(acc), x);
const pipe = (...fns) => (x) => fns.reduce((acc, fn) => fn(acc), x);

const trim = (s) => s.trim();
const lower = (s) => s.toLowerCase();
const dashes = (s) => s.replace(/\s+/g, "-");
const noPunct = (s) => s.replace(/[^a-z0-9-]/g, "");

const slugify = pipe(trim, lower, dashes, noPunct);
console.log(slugify("  Learn JavaScript: Closures & This!  "));

const inc = (n) => n + 1, dbl = (n) => n * 2;
console.log(compose(inc, dbl)(5), pipe(inc, dbl)(5));     // inc(dbl(5)) versus dbl(inc(5))
```

OUTPUT

```
learn-javascript-closures--this
11 12
```

11.7.3 Debounce and throttle

Some events fire very often: typing in a search box, resizing the window, scrolling. **Debounce** delays a function until the events have **stopped** for a given time, so it runs once after the burst (search suggestions after the user pauses typing). **Throttle** runs the function **at most once** per interval during the burst (update a scroll position indicator every 200 ms). Both are closures holding a timer or a timestamp.

DEBOUNCE AND THROTTLE

main.js

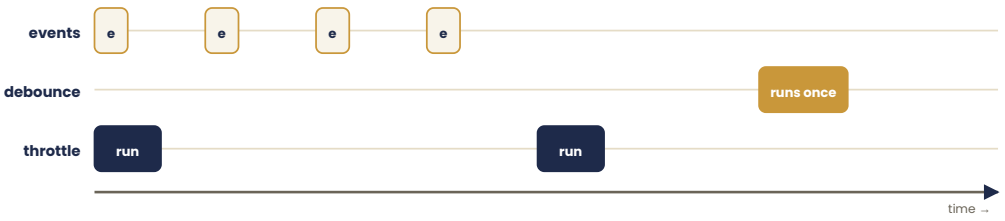
```
function debounce(fn, delay) {
  let timer;
  return (...args) => {
    clearTimeout(timer);                        // every new call restarts the wait
    timer = setTimeout(() => fn(...args), delay);
  };
}
function throttle(fn, interval) {
  let last = -Infinity;
  return (...args) => {
    const now = Date.now();
    if (now - last >= interval) { last = now; fn(...args); }   // ignore calls inside the interval
  };
}

const search = debounce((q) => console.log(`debounced search for "${q}"`), 50);
["j", "ja", "jav", "java"].forEach((q, i) => setTimeout(() => search(q), i * 10));   // fast typing

const onScroll = throttle((pos) => console.log(`throttled scroll handler at ${pos}px`), 40);
for (let i = 0; i < 10; i++) setTimeout(() => onScroll(i * 100), 200 + i * 10);      // 10 events in
90 ms
```

OUTPUT

```
debounced search for "java"
throttled scroll handler at 0px
throttled scroll handler at 500px
throttled scroll handler at 900px
```



DEBOUNCE WAITS FOR SILENCE; THROTTLE LIMITS THE RATE

Code runs in execution contexts (global, one per function call) created in a creation phase (hoisting) and run in an execution phase, stacked on the call stack. Scope is lexical: global, function, block and module scope, searched outwards along the scope chain, with inner names shadowing outer ones. var is hoisted as undefined, function declarations with their bodies, let/const/class into the TDZ. A closure is a function that keeps its outer variables alive; it enables privacy, counters, factories, memoization and the module pattern (use let in loops). this depends on the call: new, explicit call/apply/bind, method call, plain call (undefined in strict mode), arrow (lexical), event handler (the element); passing methods as callbacks loses it. Currying, compose/pipe, debounce and throttle are closure-based techniques.

PRACTICE CHECKPOINT

Write a **counter factory** using closures, a **memoize** function, a **debounce** function, and answer **ten output-prediction questions** on hoisting and this.

SAMPLE SOLUTION

main.js

```
// 1. Counter factory
function createCounter({ start = 0, step = 1 } = {}) {
```

```
  let value = start;
  return {
    next: () => (value += step),
    reset: () => (value = start),
    get current() { return value; },
  };
}
const byTwo = createCounter({ start: 10, step: 2 });
byTwo.next(); byTwo.next();
console.log("counter:", byTwo.current, byTwo.reset(), byTwo.next());

// 2. Memoize for any number of arguments
function memoize(fn) {
  const cache = new Map();
  return function (...args) {
    const key = JSON.stringify(args);
    if (!cache.has(key)) cache.set(key, fn.apply(this, args));
    return cache.get(key);
  };
}
let calls = 0;
const slowAdd = (a, b) => { calls++; return a + b; };
const fastAdd = memoize(slowAdd);
fastAdd(2, 3); fastAdd(2, 3); fastAdd(4, 5);
console.log("memoize: real calls =", calls);

// 3. Debounce
function debounce(fn, ms) {
  let id;
  return (...args) => { clearTimeout(id); id = setTimeout(() => fn(...args), ms); };
}
const save = debounce((text) => console.log(`debounced save: "${text}"`), 30);
save("H"); save("He"); save("Hel"); save("Hello");

// 4. Ten predictions
const answers = [];
answers.push(typeof hoistedFn);                          // Q1
function hoistedFn() {}
answers.push(String(hoistedVar));                        // Q2
var hoistedVar = 5;
try { answers.push(tdz); } catch { answers.push("ReferenceError"); }   // Q3
let tdz = 1;
const obj = { n: "obj", f() { return this.n; }, g: () => typeof this };
answers.push(obj.f());                                   // Q4
const f = obj.f;
try { answers.push(String(f())); } catch { answers.push("TypeError"); }   // Q5 (sloppy module: this
is globalThis, n undefined)
answers.push(obj.g());                                   // Q6
answers.push(obj.f.call({ n: "called" }));              // Q7
answers.push(obj.f.bind({ n: "bound" }).call({ n: "x" }));   // Q8
answers.push((function () { "use strict"; return typeof this; })());   // Q9
var fns = []; for (var i = 0; i < 2; i++) fns.push(() => i);
answers.push(fns.map((fn) => fn()).join(","));           // Q10
answers.forEach((a, i) => console.log(`Q${i + 1}: ${a}`));
```

OUTPUT

```
counter: 14 10 12
memoize: real calls = 2
Q1: function
Q2: undefined
Q3: ReferenceError
Q4: obj
Q5: undefined
Q6: object
Q7: called
Q8: bound
Q9: undefined
Q10: 2,2
debounced save: "Hello"
```

MODULE 12 · INTERMEDIATE

Modern JavaScript (ES6 and beyond)

Minimum duration **3 days** | Level **Intermediate**

ES2015 (ES6) and the yearly editions after it changed how JavaScript is written. This module covers the features found in every modern codebase: destructuring, spread and rest, enhanced object literals, the Map and Set collections (including the ES2025 Set methods, WeakMap, WeakSet and Map.groupBy), symbols, the iteration protocols, generators and the ES2025 iterator helpers, and a set of smaller features such as numeric separators and globalThis.

IN THIS MODULE

12.1 Destructuring
12.2 Spread and rest
12.3 Enhanced object literals
12.4 Map and Set
12.5 Symbols
12.6 Iterators and generators
12.7 Other modern features

12.1 Destructuring

Destructuring unpacks values from arrays, or properties from objects, into separate variables in one statement. The pattern on the left of `=` mirrors the shape of the data on the right: square brackets for arrays (matched by **position**) and braces for objects (matched by **property name**). It replaces many lines of `const x = data[0]` or `const name = user.name` and is used everywhere in modern code: function parameters, loops, imports and React hooks.

12.1.1 Array destructuring and skipping items

Variables in an array pattern take the values in order. To skip an item, leave its slot empty with an extra comma. Extra variables that have no matching item get `undefined`; extra items are simply ignored. Any **iterable** can be destructured this way, including strings, Sets and Maps (12.6).

ARRAY DESTRUCTURING BY POSITION

main.js

```
const scores = [91, 78, 85, 64];

const [first, second] = scores;            // by position
console.log("first:", first, "second:", second);

const [, , third] = scores;                // skip the first two with empty slots
console.log("third:", third);

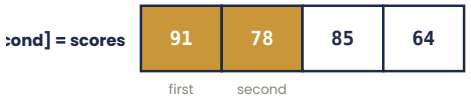
const [a, b, c, d, e] = scores;            // e has no matching item
console.log("e:", e);

const [x, y] = "Hi";                       // strings are iterable
console.log("from a string:", x, y);

const [p, q] = new Set(["red", "green", "blue"]);
console.log("from a Set:", p, q);
```

OUTPUT

```
first: 91 second: 78
third: 85
e: undefined
from a string: H i
from a Set: red green
```



ARRAY PATTERNS MATCH BY POSITION

12.1.2 Object destructuring and renaming

An object pattern lists property names; each becomes a variable with the same name. The order does not matter, and properties that are not listed are ignored. To store a property in a variable with a **different name**, write `property: newName`; read it as "take *property* and put it in *newName*". A missing property gives `undefined`.

OBJECT DESTRUCTURING AND RENAMING main.js

```
const user = { id: 7, name: "Asha", city: "Bengaluru", role: "admin" };

const { name, city } = user;               // by name; order does not matter
console.log(name, "lives in", city);

const { id: userId, role: userRole } = user;   // rename: property -> new variable
console.log("userId:", userId, "userRole:", userRole);

const { email } = user;                    // not present
console.log("email:", email);

// destructuring an existing variable requires parentheses (otherwise { starts a block)
let total;
({ id: total } = user);
console.log("assigned later:", total);
```

OUTPUT

```
Asha lives in Bengaluru
userId: 7 userRole: admin
email: undefined
assigned later: 7
```

WATCH OUT

When assigning to **already declared** variables, wrap the whole statement in parentheses: `({ a, b } = obj);`. Without them, the `{` at the start of the line is read as a block and you get a SyntaxError.

12.1.3 Default values

A default is written with `=` inside the pattern and is used only when the value is `undefined` (missing). Unlike `||`, it is **not** used for `null`, `0`, `""` or `false`. Defaults can be combined with renaming: `{ theme: t = "light" }`. A default expression is only evaluated when it is needed.

DEFAULTS APPLY ONLY TO UNDEFINED main.js

```
const settings = { theme: undefined, fontSize: 0, lang: null };

const { theme = "light", fontSize = 16, lang = "en", volume = 50 } = settings;
console.log({ theme, fontSize, lang, volume });   // 0 and null are kept

const [r = 255, g = 255, b = 255] = [10];
console.log("rgb:", r, g, b);

const { theme: t = "dark" } = {};                  // rename + default
console.log("t:", t);

function expensive() { console.log("  (default computed)"); return 99; }
const [k = expensive()] = [5];                     // not called: value present
const [m = expensive()] = [];                      // called: value missing
console.log("k:", k, "m:", m);
```

OUTPUT

```
{ theme: 'light', fontSize: 0, lang: null, volume: 50 }
rgb: 10 255 255
t: dark
  (default computed)
k: 5 m: 99
```

12.1.4 Nested destructuring

Patterns can be nested to reach inside objects within objects, or arrays within objects, in one step. The intermediate names (here `address` and `marks`) are only part of the pattern: they are **not** created as variables. Nested patterns throw a TypeError if an intermediate value is `undefined`, so give the nested part a default (`address: { city } = {}`) when it may be missing.

REACHING INTO NESTED DATA main.js

```
const student = {
  name: "Ravi",
  address: { city: "Mysuru", pin: "570001" },
  marks: [88, 92, 79],
};

const { name, address: { city, pin }, marks: [maths, science] } = student;
console.log(name, city, pin, maths, science);

try {
  console.log(address);                    // the pattern name is not a variable
} catch (e) {
  console.log("address ->", e.name);
}

const noAddress = { name: "Meera" };
const { address: { city: c2 } = {} } = noAddress;   // default protects the nested pattern
console.log("city with default:", c2);

try {
  const { address: { city: c3 } } = noAddress;
} catch (e) {
  console.log("without default ->", e.name);
}
```

OUTPUT

```
Ravi Mysuru 570001 88 92
address -> ReferenceError
city with default: undefined
without default -> TypeError
```



A NESTED PATTERN FOLLOWS THE SHAPE OF THE DATA

12.1.5 Destructuring function parameters

A parameter can itself be a pattern. This is the standard way to accept an **options object**: callers pass named properties in any order, the function lists exactly what it uses, and each option can have a default. Add `= {}` after the pattern so the function can also be called with no argument at all.

OPTIONS OBJECTS AND DESTRUCTURED CALLBACK PARAMETERS

main.js

```
function createButton({ label, color = "navy", size = "md", disabled = false } = {}) {
  return `<button class="${color} ${size}"${disabled ? " disabled" : ""}>${label ?? "OK"}</button>`;
}
console.log(createButton({ label: "Save", size: "lg" }));
console.log(createButton({ disabled: true, label: "Wait" }));
console.log(createButton());                        // works thanks to = {}

// arrays of pairs destructured in callbacks and loops
const prices = { pen: 10, book: 250, bag: 899 };
for (const [item, price] of Object.entries(prices)) {
  console.log(`${item.padEnd(5)} Rs ${price}`);
}
const totals = [["Asha", 91], ["Ravi", 78]].map(([n, s]) => `${n}:${s >= 80 ? "A" : "B"}`);
console.log(totals);
```

OUTPUT

```
<button class="navy lg">Save</button>
<button class="navy md" disabled>Wait</button>
<button class="navy md">OK</button>
pen   Rs 10
book  Rs 250
bag   Rs 899
[ 'Asha:A', 'Ravi:B' ]
```

TIP

Prefer an options object over long positional parameter lists. `createUser({ name, admin: true })` is self-describing; `createUser("Asha", null, true, false)` is not.

12.1.6 Swapping variables

Array destructuring swaps two variables without a temporary variable: the right side builds a new array from the current values, and the left side unpacks it in the opposite order. The same trick rotates three or more values, and is used inside sorting algorithms.

SWAPPING WITHOUT A TEMPORARY VARIABLE

main.js

```
let a = 1, b = 2;
[a, b] = [b, a];
console.log("swapped:", a, b);

let x = "x", y = "y", z = "z";
[x, y, z] = [y, z, x];                      // rotate left
console.log("rotated:", x, y, z);

const arr = [40, 10, 30];
[arr[0], arr[1]] = [arr[1], arr[0]];        // swap elements inside an array
console.log("array:", arr);
```

OUTPUT

```
swapped: 2 1
rotated: y z x
array: [ 10, 40, 30 ]
```

NOTE

Put a semicolon at the end of the line **before** a statement that starts with `[`. Without it, JavaScript may join the two lines into one expression (`b = 2[a, b] = ...`).

12.2 Spread and rest

The same three dots `...` have two opposite meanings depending on where they appear. **Spread** expands an iterable or object into individual elements or properties (on the right side, in calls and literals). **Rest** collects several elements or properties into one array or object (on the left side, in parameters and destructuring patterns).



SPREAD EXPANDS; REST COLLECTS

12.2.1 Spread with arrays, objects and function calls

In an **array literal**, spread copies the elements of any iterable, which makes it the usual way to copy, concatenate and insert. In a **function call**, it passes array elements as separate arguments (replacing the old `fn.apply(null, arr)`). In an **object literal** (ES2018), it copies an object's own enumerable properties; later properties overwrite earlier ones, which is how settings are merged and objects are updated immutably.

SPREAD IN ARRAYS, CALLS AND OBJECTS main.js

```
// arrays
const a = [1, 2], b = [3, 4];
const joined = [...a, ...b];
const inserted = [0, ...a, 2.5, ...b, 5];
const copy = [...a];
copy.push(99);
console.log(joined, inserted, "original a:", a);
console.log([..."hello"], [...new Set([1, 1, 2])]);

// function calls
const marks = [72, 95, 88];
console.log("max:", Math.max(...marks));
const parts = [2026, 2, 9];
console.log(new Date(...parts).toDateString());

// objects: copy, merge and override
const defaults = { theme: "light", fontSize: 14, lang: "en" };
const userPrefs = { fontSize: 18, lang: "kn" };
const settings = { ...defaults, ...userPrefs };     // later wins
console.log(settings);
const updated = { ...settings, theme: "dark" };     // immutable update
console.log(updated.theme, "| settings still", settings.theme);
```

OUTPUT

```
[ 1, 2, 3, 4 ] [
  0, 1, 2, 2.5,
  3, 4, 5
] original a: [ 1, 2 ]
[ 'h', 'e', 'l', 'l', 'o' ] [ 1, 2 ]
max: 95
Mon Mar 09 2026
{ theme: 'light', fontSize: 18, lang: 'kn' }
dark | settings still light
```

WATCH OUT

Spread makes a **shallow** copy: nested arrays and objects are still shared. For a full independent copy of plain data, use `structuredClone(obj)` (Module 9).

SPREAD COPIES ONLY THE TOP LEVEL main.js

```
const original = { name: "Asha", tags: ["js"] };
const shallow = { ...original };
shallow.tags.push("css");                    // shared nested array
console.log("original tags:", original.tags);

const deep = structuredClone(original);
deep.tags.push("html");
console.log("after deep copy change:", original.tags, deep.tags);
```

OUTPUT

```
original tags: [ 'js', 'css' ]
after deep copy change: [ 'js', 'css' ] [ 'js', 'css', 'html' ]
```

12.2.2 Rest in parameters and destructuring

A **rest parameter** `...name` gathers all remaining arguments into a real array (unlike the old `arguments` object, which is not an array and does not exist in arrow functions). In destructuring, a rest element collects the remaining array items, and a rest property collects the remaining object properties, which is a clean way to remove a property without mutating the object. A rest element must always come **last**.

REST PARAMETERS AND REST IN PATTERNS main.js

```
function sum(...numbers) {                   // a real array
  return numbers.reduce((t, n) => t + n, 0);
}
console.log(sum(), sum(5), sum(1, 2, 3, 4));

function log(level, ...messages) {
  console.log(`[${level.toUpperCase()}]`, messages.join(" | "));
}
log("info", "server started", "port 3000");

const [head, ...tail] = [10, 20, 30, 40];
console.log("head:", head, "tail:", tail);

const user = { id: 1, name: "Ravi", password: "s3cret", email: "ravi@x.in" };
const { password, ...safeUser } = user;          // remove password without mutating
console.log(safeUser);
console.log("user still has password:", "password" in user);
```

OUTPUT

```
0 5 10
[INFO] server started | port 3000
head: 10 tail: [ 20, 30, 40 ]
{ id: 1, name: 'Ravi', email: 'ravi@x.in' }
user still has password: true
```

Where the dots appear	Meaning	Example
function call argument	spread an iterable into arguments	`Math.max(...arr)`
array literal	spread elements into a new array	`[...a, ...b]`

Where the dots appear	Meaning	Example
object literal	copy own properties into a new object	`{ ...obj, x: 1 }`
function parameter	rest: gather remaining arguments	`function f(a, ...rest)`
array pattern	rest: gather remaining items	`const [h, ...t] = arr`
object pattern	rest: gather remaining properties	`const { pw, ...safe } = u`

12.3 Enhanced object literals

ES6 added four shortcuts to object literals. **Shorthand properties**: `{ name }` means `{ name: name }` when a variable has the same name as the property. **Shorthand methods**: `greet() { }` instead of `greet: function () { }` (these methods can also use `super`). **Computed property names**: `[expression]: value` uses the result of any expression as the key. And getters/setters, already met in Module 9, sit naturally alongside them.

SHORTHAND PROPERTIES, METHODS AND COMPUTED KEYS

main.js

```
const name = "Asha", age = 21;
const field = "score";
let n = 0;

const student = {
  name,                                 // shorthand for name: name
  age,
  [field]: 91,                          // computed key -> "score"
  [`${field}Grade`]: "A",               // -> "scoreGrade"
  [`item${++n}`]: "first",              // any expression
  greet() {                             // shorthand method
    return `Hi, I am ${this.name}`;
  },
  get isAdult() { return this.age >= 18; },
};

console.log(student);
console.log(student.greet(), "| adult:", student.isAdult);

// a common pattern: build an object from variables to return or send
function makePoint(x, y) { return { x, y, dist: Math.hypot(x, y) }; }
console.log(makePoint(3, 4));

// computed keys inside reduce
const counts = ["a", "b", "a"].reduce((acc, ch) => ({ ...acc, [ch]: (acc[ch] ?? 0) + 1 }), {});
console.log(counts);
```

OUTPUT

```
{
  name: 'Asha',
  age: 21,
  score: 91,
  scoreGrade: 'A',
  item1: 'first',
  greet: [Function: greet],
  isAdult: [Getter]
}
Hi, I am Asha | adult: true
{ x: 3, y: 4, dist: 5 }
{ a: 2, b: 1 }
```

OLD STYLE VERSUS ENHANCED LITERALS

main.js

```
// Before ES6
var student = {
  name: name,
  age: age,
  greet: function () { return "Hi, I am " + this.name; }
};
student[field] = 91;          // computed keys needed a separate statement

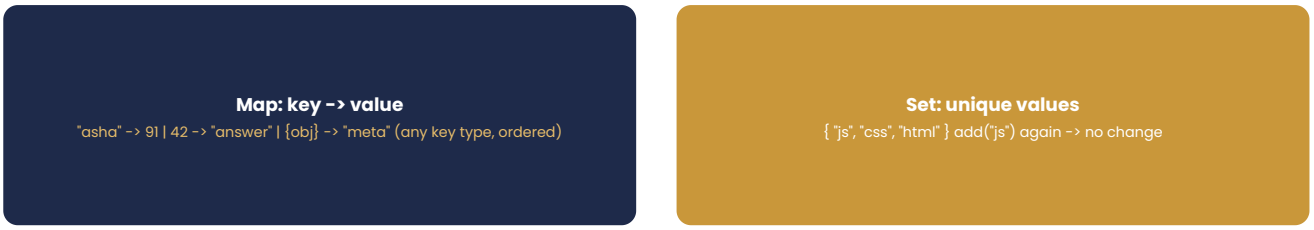
// ES6 and later
const student = { name, age, greet() { return `Hi, I am ${this.name}`; }, [field]: 91 };
```

NOTE

Both create the same object; the modern form is shorter and keeps everything in one literal.

12.4 Map and Set

Before ES6, plain objects were used for every key-value store and arrays for every collection. ES6 added two purpose-built collections. A **Map** stores key-value pairs where the keys can be **any** value (objects, functions, numbers) and remembers insertion order. A **Set** stores **unique** values, so adding a duplicate has no effect. Both are iterable, have a `size` property and offer fast `has` lookups.



THE TWO ES6 KEYED COLLECTIONS

12.4.1 Map: set, get, has, delete, size, iteration

Create a Map with `new Map()`, optionally from an iterable of `[key, value]` pairs. `set(key, value)` adds or replaces (and returns the map, so calls can be chained); `get(key)` returns the value or `undefined`; `has`, `delete` and `clear` do what their names say; `size` is the count. Iterate with `for...of` (each item is a `[key, value]` pair), `forEach`, or the `keys()`, `values()` and `entries()` iterators. Keys are compared with SameValueZero: like `===` except that `NaN` equals `NaN`.

WORKING WITH A MAP main.js

```
const stock = new Map([["pen", 120], ["book", 45]]);

stock.set("bag", 12).set("pen", 100);          // add + replace, chained
console.log("size:", stock.size);
console.log("pen:", stock.get("pen"), "| eraser:", stock.get("eraser"));
console.log("has book?", stock.has("book"));

stock.delete("book");
for (const [item, qty] of stock) {
  console.log(`  ${item}: ${qty}`);
}
console.log("keys:", [...stock.keys()], "values:", [...stock.values()]);
stock.forEach((qty, item) => { if (qty < 50) console.log("low stock:", item); });

// any value can be a key
const user = { name: "Asha" };
const lastSeen = new Map();
lastSeen.set(user, "09:30").set(42, "number key").set(NaN, "NaN key");
console.log(lastSeen.get(user), "|", lastSeen.get(42), "|", lastSeen.get(NaN));
console.log("a different object with the same content:", lastSeen.get({ name: "Asha" }));

// converting between Map and object
const asObject = Object.fromEntries(stock);
const backToMap = new Map(Object.entries(asObject));
console.log(asObject, backToMap);
```

OUTPUT

```
size: 3
pen: 100 | eraser: undefined
has book? true
  pen: 100
  bag: 12
keys: [ 'pen', 'bag' ] values: [ 100, 12 ]
low stock: bag
09:30 | number key | NaN key
a different object with the same content: undefined
{ pen: 100, bag: 12 } Map(2) { 'pen' => 100, 'bag' => 12 }
```

12.4.2 Map vs Object

Objects remain the right choice for fixed records with known property names (a user, a config, JSON data). A Map is better as a **dictionary** whose keys are added and removed at run time, especially when the keys are not strings or come from users. Object keys are always strings or symbols (a number key becomes a string, and an object key becomes `"[object Object]"`); an object also inherits keys from its prototype, so a user-supplied key like `"constructor"` or `"__proto__"` can collide or misbehave.

WHERE PLAIN OBJECTS FALL SHORT

main.js

```
const obj = {};
const k1 = { id: 1 }, k2 = { id: 2 };
obj[k1] = "first";
obj[k2] = "second";                          // both become "[object Object]"
obj[1] = "number";
console.log(Object.keys(obj), obj[k1]);

const map = new Map([[k1, "first"], [k2, "second"], [1, "number"]]);
console.log("map keys keep their types:", [...map.keys()].map((k) => typeof k));

const counts = {};
console.log("'constructor' in empty object:", "constructor" in counts, typeof counts.constructor);
const safe = new Map();
console.log("map.has('constructor'):", safe.has("constructor"));
```

OUTPUT

```
[ '1', '[object Object]' ] second
map keys keep their types: [ 'object', 'object', 'number' ]
'constructor' in empty object: true function
map.has('constructor'): false
```

Feature	Object	Map
Key types	strings and symbols only	any value, including objects and functions
Order	mostly insertion order, but integer-like keys first	always insertion order
Size	`Object.keys(o).length`	`map.size`
Iteration	needs Object.keys/entries	directly iterable
Inherited keys	yes (from the prototype) unless `Object.create(null)`	none
Frequent add/delete	not optimised for it	designed for it
JSON support	JSON.stringify works	convert first with Object.fromEntries
Best for	records with known fields	dictionaries, caches, counting, lookups by object

12.4.3 Set: add, has, delete, removing duplicates

A Set keeps each value at most once (again using SameValueZero). `add` returns the set so it can be chained, `has` checks membership very quickly (much faster than `array.includes` for large collections), and `delete`, `clear` and `size` work like Map's. The most famous one-liner in JavaScript removes duplicates from an array: `[...new Set(array)]`.

SETS AND DUPLICATE REMOVAL main.js

```
const tags = new Set();
tags.add("js").add("css").add("js").add("html");     // second "js" ignored
console.log(tags, "size:", tags.size);
console.log("has css?", tags.has("css"));
tags.delete("css");
for (const t of tags) console.log("  tag:", t);

const nums = [3, 1, 3, 2, 1, 3];
const unique = [...new Set(nums)];
console.log("unique:", unique);

const words = "the cat and the hat and the bat".split(" ");
console.log("distinct words:", new Set(words).size, "of", words.length);

// objects are compared by reference
const s = new Set([{ id: 1 }, { id: 1 }]);
console.log("two equal-looking objects:", s.size);

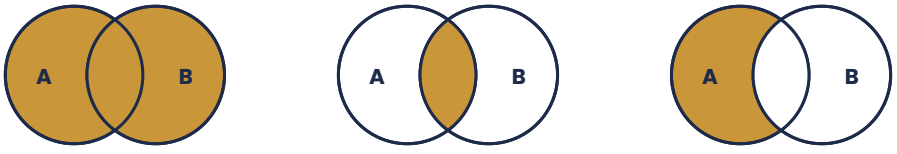
// unique by a property: keep the first object per email
const users = [{ email: "a@x.in", n: 1 }, { email: "b@x.in", n: 2 }, { email: "a@x.in", n: 3 }];
const seen = new Set();
const firstPerEmail = users.filter((u) => !seen.has(u.email) && seen.add(u.email));
console.log(firstPerEmail.map((u) => u.n));
```

OUTPUT

```
Set(3) { 'js', 'css', 'html' } size: 3
has css? true
  tag: js
  tag: html
unique: [ 3, 1, 2 ]
distinct words: 5 of 8
two equal-looking objects: 2
[ 1, 2 ]
```

12.4.4 Set methods: union, intersection, difference, isSubsetOf (ES2025)

ES2025 added mathematical set operations directly to `Set.prototype`. Each returns a **new** Set (or a boolean) and leaves the original unchanged. The argument can be a Set or any "set-like" object (with `size`, `has` and `keys`), such as a Map. They are available in all current browsers and in Node.js 22+.



THE THREE CORE SET OPERATIONS

ES2025 SET METHODS main.js

```
const frontend = new Set(["html", "css", "js", "react"]);
const backend = new Set(["js", "node", "sql"]);

console.log("union:               ", frontend.union(backend));
console.log("intersection:        ", frontend.intersection(backend));
console.log("difference (F - B):  ", frontend.difference(backend));
console.log("symmetricDifference: ", frontend.symmetricDifference(backend));

const basics = new Set(["html", "css"]);
console.log("basics isSubsetOf frontend?  ", basics.isSubsetOf(frontend));
console.log("frontend isSupersetOf basics?", frontend.isSupersetOf(basics));
console.log("basics isDisjointFrom backend?", basics.isDisjointFrom(backend));
console.log("originals unchanged:", frontend.size, backend.size);
```

OUTPUT

```
union:               Set(6) { 'html', 'css', 'js', 'react', 'node', 'sql' }
intersection:        Set(1) { 'js' }
difference (F - B):  Set(3) { 'html', 'css', 'react' }
symmetricDifference: Set(5) { 'html', 'css', 'react', 'node', 'sql' }
basics isSubsetOf frontend?   true
frontend isSupersetOf basics? true
basics isDisjointFrom backend? true
originals unchanged: 4 3
```

Method	Returns	Contains / tests
`a.union(b)`	Set	everything in a or b
`a.intersection(b)`	Set	only values in both
`a.difference(b)`	Set	values in a but not in b
`a.symmetricDifference(b)`	Set	values in exactly one of them
`a.isSubsetOf(b)`	boolean	every value of a is in b
`a.isSupersetOf(b)`	boolean	every value of b is in a
`a.isDisjointFrom(b)`	boolean	no value in common

Before ES2025 (or for older browsers), the same operations are written with spread and `filter`:

THE PRE-ES2025 WAY main.js

```
const a = new Set([1, 2, 3, 4]), b = new Set([3, 4, 5]);
const union = new Set([...a, ...b]);
const inter = new Set([...a].filter((x) => b.has(x)));
const diff = new Set([...a].filter((x) => !b.has(x)));
console.log([...union], [...inter], [...diff]);
```

OUTPUT

```
[ 1, 2, 3, 4, 5 ] [ 3, 4 ] [ 1, 2 ]
```

12.4.5 WeakMap and WeakSet

A **WeakMap** is a Map whose keys must be objects (or non-registered symbols) and are held **weakly**: if nothing else in the program refers to a key object, the garbage collector may remove it together with its entry. A **WeakSet** is the same idea for a set of objects. Because entries can vanish at any time, they are

not iterable and have no `size` or `clear`; you can only `set/get/has/delete` (or `add/has/delete`). They are used to attach private or cached data to objects you do not own (for example DOM elements) without causing memory leaks.

WEAKMAP AND WEAKSET

main.js

```
// Private data per object, freed automatically with the object
const secrets = new WeakMap();
class Account {
  constructor(owner, pin) {
    this.owner = owner;
    secrets.set(this, { pin });            // not visible as a property
  }
  check(pin) { return secrets.get(this).pin === pin; }
}
const acc = new Account("Asha", "4321");
console.log(acc, "| pin ok?", acc.check("4321"), "| wrong?", acc.check("0000"));

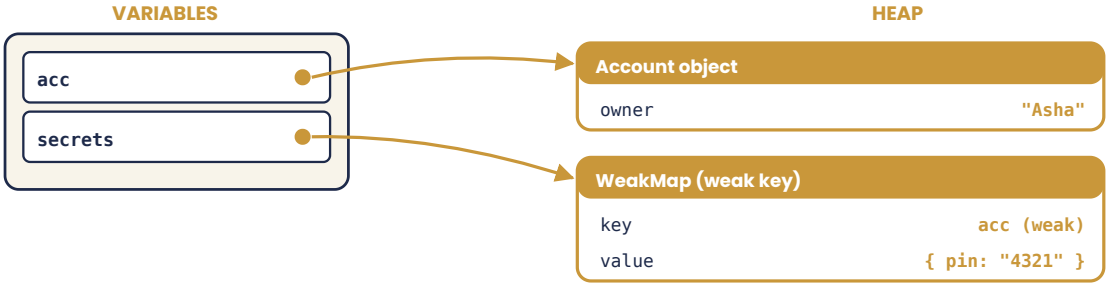
// Cache results per object
const cache = new WeakMap();
function area(shape) {
  if (cache.has(shape)) return cache.get(shape) + " (cached)";
  const a = shape.w * shape.h;
  cache.set(shape, a);
  return a;
}
const rect = { w: 3, h: 4 };
console.log(area(rect), "|", area(rect));

// Mark objects without modifying them
const visited = new WeakSet();
const pageA = { url: "/home" }, pageB = { url: "/about" };
visited.add(pageA);
console.log("visited A?", visited.has(pageA), "| visited B?", visited.has(pageB));

try { new WeakMap().set("text", 1); } catch (e) { console.log("primitive key ->", e.constructor.name);
}
console.log("iterable?", typeof new WeakMap()[Symbol.iterator]);
```

OUTPUT

```
Account { owner: 'Asha' } | pin ok? true | wrong? false
12 | 12 (cached)
visited A? true | visited B? false
primitive key -> TypeError
iterable? undefined
```



WHEN ACC IS NO LONGER REFERENCED, THE WEAKMAP ENTRY CAN BE COLLECTED TOO

12.4.6 Map.groupBy()

ES2024 added `Object.groupBy(items, fn)` and `Map.groupBy(items, fn)`. Both call `fn` for each item and put the item into a group named by the returned value. `Object.groupBy` produces a null-prototype object whose keys are strings; `Map.groupBy` produces a Map, so the group keys can be **any value** (numbers, booleans, objects) and keep their type and order.

GROUPING WITH MAP.GROUPBY AND OBJECT.GROUPBY main.js

```
const orders = [
  { id: 1, city: "Mysuru", amount: 450 },
  { id: 2, city: "Pune", amount: 1200 },
  { id: 3, city: "Mysuru", amount: 2300 },
  { id: 4, city: "Delhi", amount: 90 },
];

const byCity = Map.groupBy(orders, (o) => o.city);
for (const [city, list] of byCity) {
  console.log(city.padEnd(7), list.map((o) => o.id));
}

const bySize = Map.groupBy(orders, (o) => o.amount >= 1000);   // boolean keys
console.log("big orders:", bySize.get(true).map((o) => o.id), "| small:", bySize.get(false).map((o) =>
o.id));

const asObject = Object.groupBy(orders, (o) => (o.amount >= 1000 ? "big" : "small"));
console.log(Object.keys(asObject), asObject.big.length);

// grouping by an object key: only possible with a Map
const premium = { tier: "premium" }, basic = { tier: "basic" };
const tierOf = (o) => (o.amount > 1000 ? premium : basic);
const byTier = Map.groupBy(orders, tierOf);
console.log("premium ids:", byTier.get(premium).map((o) => o.id));
```

OUTPUT

```
Mysuru  [ 1, 3 ]
Pune    [ 2 ]
Delhi   [ 4 ]
big orders: [ 2, 3 ] | small: [ 1, 4 ]
[ 'small', 'big' ] 2
premium ids: [ 2, 3 ]
```

12.5 Symbols

12.5.1 Creating unique symbols

A **Symbol** is a primitive whose every value is **unique**. `Symbol("description")` creates a new symbol; the description is only a label for debugging, so two symbols with the same description are still different. Symbols are used as property keys that can never collide with other keys, including keys added later by other code or libraries. Symbol-keyed properties are skipped by `for...in`, `Object.keys` and `JSON.stringify`, but they are **not** private: `Object.getOwnPropertySymbols` finds them. `Symbol.for("key")` instead uses a global registry, returning the same symbol for the same key everywhere.

UNIQUE SYMBOLS AS PROPERTY KEYS

main.js

```
const id1 = Symbol("id");
const id2 = Symbol("id");
console.log(typeof id1, id1.toString(), "| description:", id1.description);
console.log("same description, equal?", id1 === id2);

const ID = Symbol("id");
const user = { name: "Asha", [ID]: 101 };
user.id = "a library's own id";           // no collision with the symbol key
console.log(user[ID], "|", user.id);

console.log("keys:", Object.keys(user));
console.log("JSON:", JSON.stringify(user));
console.log("symbols:", Object.getOwnPropertySymbols(user));

const g1 = Symbol.for("app.theme");
const g2 = Symbol.for("app.theme");
console.log("Symbol.for shares:", g1 === g2, "| key:", Symbol.keyFor(g1));

try { new Symbol(); } catch (e) { console.log("new Symbol() ->", e.name); }
```

OUTPUT

```
symbol Symbol(id) | description: id
same description, equal? false
101 | a library's own id
keys: [ 'name', 'id' ]
JSON: {"name":"Asha","id":"a library's own id"}
symbols: [ Symbol(id) ]
Symbol.for shares: true | key: app.theme
new Symbol() -> TypeError
```

12.5.2 Well-known symbols (Symbol.iterator)

JavaScript defines a set of built-in **well-known symbols** that let your objects hook into language behaviour. The most important is `Symbol.iterator`: an object with a method under this key is **iterable** and works with `for...of`, spread and destructuring (12.6). Others customise conversion to primitives, the tag shown by `Object.prototype.toString`, and `instanceof`.

HOOKING INTO LANGUAGE BEHAVIOUR

main.js

```
const temperature = {
  celsius: 30,
  [Symbol.toPrimitive](hint) {            // how the object converts
    if (hint === "number") return this.celsius;
    if (hint === "string") return `${this.celsius}°C`;
    return this.celsius;                  // "default", e.g. for +
  },
  get [Symbol.toStringTag]() { return "Temperature"; },
};
console.log(`${temperature}`, "|", temperature * 2, "|", temperature + 5);
console.log(Object.prototype.toString.call(temperature));

const range = {
  from: 1, to: 4,
  *[Symbol.iterator]() { for (let i = this.from; i <= this.to; i++) yield i; },
};
console.log("iterable:", [...range]);

class Even { static [Symbol.hasInstance](n) { return n % 2 === 0; } }
console.log("4 instanceof Even:", 4 instanceof Even, "| 7:", 7 instanceof Even);
```

OUTPUT

```
30°C | 60 | 35
[object Temperature]
iterable: [ 1, 2, 3, 4 ]
4 instanceof Even: true | 7: false
```

Well-known symbol	Used by	Lets you customise
`Symbol.iterator`	for…of, spread, destructuring, Array.from	how the object is iterated
`Symbol.asyncIterator`	for await…of	asynchronous iteration (Module 17)
`Symbol.toPrimitive`	conversion to number or string	the primitive value of the object
`Symbol.toStringTag`	Object.prototype.toString	the "[object Tag]" name
`Symbol.hasInstance`	instanceof	what counts as an instance
`Symbol.species`	map, filter on subclasses	which constructor creates derived objects

12.6 Iterators and generators

12.6.1 The iterable and iterator protocols

Two small protocols power `for...of`, spread, destructuring, `Array.from`, `new Map(iterable)` and `Promise.all`. An **iterable** is any object with a `[Symbol.iterator]()` method that returns an **iterator**. An **iterator** is an object with a `next()` method that returns `{ value, done }` each time it is called; `done: true` means the sequence has ended. Arrays, strings, Maps, Sets, `arguments`, NodeLists and typed arrays are all built-in iterables; plain objects are not.

CALLING NEXT() BY HAND main.js

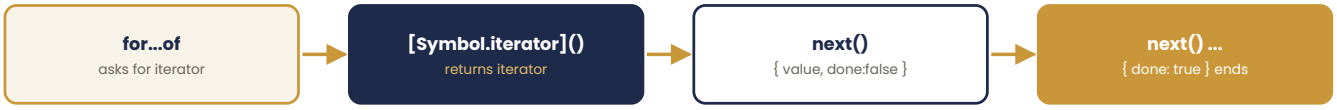
```
const colours = ["red", "green"];
const it = colours[Symbol.iterator]();     // ask the iterable for an iterator
console.log(it.next());
console.log(it.next());
console.log(it.next());                    // finished
console.log(it.next());                    // stays finished

// what for...of does behind the scenes
const it2 = "ab"[Symbol.iterator]();
let step;
while (!(step = it2.next()).done) console.log("manual loop got:", step.value);

console.log("plain object iterable?", typeof {}[Symbol.iterator]);
try { for (const x of { a: 1 }) {} } catch (e) { console.log(e.message); }
```

OUTPUT

```
{ value: 'red', done: false }
{ value: 'green', done: false }
{ value: undefined, done: true }
{ value: undefined, done: true }
manual loop got: a
manual loop got: b
plain object iterable? undefined
{(intermediate value)} is not iterable
```



HOW FOR...OF USES THE ITERATION PROTOCOLS

12.6.2 Making a custom iterable

To make your own object iterable, give it a `[Symbol.iterator]` method that returns a fresh object with a `next()` method. Returning a new iterator each time allows the object to be looped over more than once, or by two loops at the same time. An iterator can also have an optional `return()` method, which is called when a loop exits early (via `break` or an error) so that resources can be released.

A CLASS WHOSE INSTANCES ARE ITERABLE

main.js

```
class Countdown {
  constructor(start) { this.start = start; }
  [Symbol.iterator]() {
    let current = this.start;
    return {
      next: () => (current >= 0 ? { value: current--, done: false } : { value: undefined, done: true
}),
      return: () => { console.log("  (loop stopped early, cleaning up)"); return { done: true }; },
    };
  }
}

const c = new Countdown(3);
console.log([...c]);                       // spread
console.log(Array.from(c, (n) => n * 10)); // Array.from with a map function
const [first, second] = c;                 // destructuring stops after two (calls return)
console.log(first, second);
for (const n of c) {
  if (n === 1) break;                      // triggers return()
  console.log("loop:", n);
}
```

OUTPUT

```
[ 3, 2, 1, 0 ]
[ 30, 20, 10, 0 ]
  (loop stopped early, cleaning up)
3 2
loop: 3
loop: 2
  (loop stopped early, cleaning up)
```

12.6.3 Generator functions function* and yield

Writing iterators by hand is verbose. A **generator function**, declared with `function*`, does it for you. Calling it does **not** run its body; it returns a **generator object**, which is both an iterator and an iterable. Each `next()` runs the body until the next `yield`, which hands out a value and **pauses** the function, keeping all its local variables. When the body returns, the generator is done. `yield*` delegates to another iterable, and a value passed to `next(value)` becomes the result of the paused `yield` expression.

GENERATORS PAUSE AT EACH YIELD

main.js

```
function* steps() {
  console.log("  start");
  yield "one";
  console.log("  resumed after one");
  yield "two";
  console.log("  resumed after two");
  return "finished";
}

const gen = steps();                    // nothing printed yet
console.log("created generator");
console.log(gen.next());
console.log(gen.next());
console.log(gen.next());
console.log(gen.next());
```

OUTPUT

```
created generator
  start
{ value: 'one', done: false }
  resumed after one
{ value: 'two', done: false }
  resumed after two
{ value: 'finished', done: true }
{ value: undefined, done: true }
```



GENERATOR STATES

CUSTOM ITERABLES, YIELD* AND NEXT(VALUE) main.js

```
// Generators make custom iterables short
class Range {
  constructor(from, to, step = 1) { Object.assign(this, { from, to, step }); }
  *[Symbol.iterator]() {
    for (let i = this.from; i <= this.to; i += this.step) yield i;
  }
}
console.log([...new Range(0, 10, 2)]);

// yield* delegates to another iterable
function* letters() { yield "a"; yield* ["b", "c"]; yield* "de"; }
console.log([...letters()]);

// two-way communication: next(value) resumes the paused yield with value
function* conversation() {
  const name = yield "What is your name?";
  const city = yield `Hello ${name}! Which city?`;
  return `${name} from ${city}`;
}
const chat = conversation();
console.log(chat.next().value);
console.log(chat.next("Asha").value);
console.log(chat.next("Mysuru").value);
```

OUTPUT

```
[ 0, 2, 4, 6, 8, 10 ]
[ 'a', 'b', 'c', 'd', 'e' ]
What is your name?
Hello Asha! Which city?
Asha from Mysuru
```

12.6.4 Infinite and lazy sequences

Because a generator computes each value only when `next()` asks for it, it can describe an **infinite** sequence without running forever or using infinite memory. This is called **lazy evaluation**: work is done on demand. Infinite generators must be consumed carefully, by taking a fixed number of values or stopping with `break`; spreading one (`[...gen]`) would never finish.

INFINITE SEQUENCES CONSUMED LAZILY

main.js

```
function* naturals(start = 1) {
  let n = start;
  while (true) yield n++;                  // infinite, but lazy
}

function* fibonacci() {
  let [a, b] = [0, 1];
  while (true) {
    yield a;
    [a, b] = [b, a + b];
  }
}

function take(iterable, count) {           // helper: first count values
  const out = [];
  if (count <= 0) return out;
  for (const v of iterable) {
    out.push(v);
    if (out.length === count) break;       // stop before pulling one value too many
  }
  return out;
}

console.log("naturals:", take(naturals(), 5));
console.log("fibonacci:", take(fibonacci(), 10));

const ids = naturals(1001);
console.log("next ids:", ids.next().value, ids.next().value, ids.next().value);

// lazy pipeline: only as many squares are computed as needed
let computed = 0;
function* squares(src) { for (const n of src) { computed++; yield n * n; } }
console.log("first 3 squares:", take(squares(naturals()), 3), "| computed:", computed);
```

OUTPUT

```
naturals: [ 1, 2, 3, 4, 5 ]
fibonacci: [
  0, 1,  1,  2,  3,
  5, 8, 13, 21, 34
]
next ids: 1001 1002 1003
first 3 squares: [ 1, 4, 9 ] | computed: 3
```

12.6.5 Iterator helpers: map, filter, take, drop, toArray (ES2025)

ES2025 added **iterator helpers**: methods on every built-in iterator (including generator objects and the iterators from `map.keys()` or `array.values()`). `map`, `filter`, `take`, `drop` and `flatMap` return new **lazy** iterators; `toArray`, `reduce`, `forEach`, `some`, `every` and `find` consume them. Unlike array methods, nothing is computed until values are pulled, so they work on infinite sequences and avoid building large intermediate arrays. `Iterator.from(obj)` wraps any iterable or iterator so the helpers can be used on it.

ES2025 ITERATOR HELPERS

main.js

```
function* naturals() { let n = 1; while (true) yield n++; }

const firstEvenSquares = naturals()
  .filter((n) => n % 2 === 0)
  .map((n) => n * n)
  .take(5)
  .toArray();
console.log("first 5 even squares:", firstEvenSquares);

console.log("skip 3, take 4:", naturals().drop(3).take(4).toArray());
console.log("sum of 1..100:", naturals().take(100).reduce((t, n) => t + n, 0));
console.log("first > 50 divisible by 7:", naturals().find((n) => n > 50 && n % 7 === 0));

// helpers on built-in iterators
const marks = new Map([["Asha", 91], ["Ravi", 64], ["Meera", 85]]);
console.log("toppers:", marks.entries().filter(([, m]) => m >= 80).map(([n]) => n).toArray());

// laziness: each value flows through the whole chain before the next is made
const traced = naturals()
  .map((n) => { console.log("  map", n); return n * 10; })
  .filter((n) => { console.log("  filter", n); return n > 10; })
  .take(2);
console.log("nothing has run yet");
console.log("result:", traced.toArray());

// Iterator.from works with any iterable
console.log(Iterator.from("hello").map((c) => c.toUpperCase()).toArray().join(""));
```

OUTPUT

```
first 5 even squares: [ 4, 16, 36, 64, 100 ]
skip 3, take 4: [ 4, 5, 6, 7 ]
sum of 1..100: 5050
first > 50 divisible by 7: 56
toppers: [ 'Asha', 'Meera' ]
nothing has run yet
  map 1
  filter 10
  map 2
  filter 20
  map 3
  filter 30
result: [ 20, 30 ]
HELLO
```

Array method	Iterator helper	Difference
`arr.map(f)`	`it.map(f)`	array: eager, builds a new array; iterator: lazy, one value at a time
`arr.filter(f)`	`it.filter(f)`	same: eager vs lazy
`arr.slice(0, n)`	`it.take(n)`	take also stops an infinite source
`arr.slice(n)`	`it.drop(n)`	skips the first n values
(already an array)	`it.toArray()`	collects the remaining values
`arr.reduce / some / every / find`	same names on iterators	consume the iterator

12.7 Other modern features

12.7.1 Optional chaining and nullish coalescing (recap)

Introduced in ES2020 and covered in Module 4, these two operators combine especially well with destructuring and API data. `a?.b` stops and returns `undefined` if `a` is `null` or `undefined`; it also works for calls (`fn?.()`) and brackets (`arr?.[0]`). `x ?? y` uses `y` only when `x` is `null` or `undefined`, keeping valid falsy values such as `0` and `""`. `??=` assigns only when the target is nullish.

SAFE ACCESS TO UNCERTAIN DATA

main.js

```
const response = {
  user: { name: "Asha", settings: { volume: 0 }, tags: [] },
  meta: null,
};

console.log(response.user?.settings?.volume ?? 50);      // 0 kept
console.log(response.user.settings.volume || 50);          // 0 lost with ||
console.log(response.meta?.page ?? 1);                     // meta is null
console.log(response.user.tags?.[0] ?? "no tags");
console.log(response.user.greet?.() ?? "no greet method");

const config = { retries: 0 };
config.retries ??= 3;            // stays 0
config.timeout ??= 5000;         // added
console.log(config);
```

OUTPUT

```
0
50
1
no tags
no greet method
{ retries: 0, timeout: 5000 }
```

12.7.2 Numeric separators 1_000_000

Since ES2021, an underscore can be placed between digits of a numeric literal to make it readable, for example `1_000_000` or the Indian grouping `1_00_00_000`. The underscores are ignored by the engine: the value is exactly the same. They work in decimal, binary, hex and BigInt literals, but not at the start or end, next to the decimal point, doubled, or inside strings passed to `Number()`.

READABLE NUMBER LITERALS main.js

```
const population = 1_40_00_00_000;          // 140 crore (Indian grouping)
const budget = 2_500_000;
const fee = 1_999.50;
const mask = 0b1111_0000;
const colour = 0xFF_C1_07;
const big = 9_007_199_254_740_993n;

console.log(population, budget, fee, mask, colour, big);
console.log("same value:", 1_000 === 1000);
console.log("Number('1_000') ->", Number("1_000"));     // separators only work in code
console.log("parseInt('1_000') ->", parseInt("1_000"));
```

OUTPUT

```
1400000000 2500000 1999.5 240 16761095 9007199254740993n
same value: true
Number('1_000') -> NaN
parseInt('1_000') -> 1
```

12.7.3 globalThis

The global object has different names in different environments: `window` (and `self`, `frames`) in browsers, `self` in web workers, `global` in Node.js. ES2020's `globalThis` refers to the global object everywhere, so library code no longer needs to detect the environment. Avoid adding your own globals; use it mainly for feature detection.

GLOBALTHIS IN NODE.JS main.js

```
console.log("globalThis exists:", typeof globalThis);
console.log("in Node.js, globalThis === global:", globalThis === global);
console.log("window here:", typeof window);

// feature detection that works in any environment
const hasFetch = typeof globalThis.fetch === "function";
const hasStructuredClone = "structuredClone" in globalThis;
console.log({ hasFetch, hasStructuredClone });
```

OUTPUT

```
globalThis exists: object
in Node.js, globalThis === global: true
window here: undefined
{ hasFetch: true, hasStructuredClone: true }
```

GLOBALTHIS IN THE BROWSER main.js

```
console.log("globalThis === window:", globalThis === window);
console.log("self === window:", self === window);
console.log("typeof global:", typeof global);
console.log("document via globalThis:", globalThis.document.title);
```

BROWSER CONSOLE

```
globalThis === window: true
self === window: true
typeof global: undefined
document via globalThis: Demo
```

12.7.4 Following new features through the TC39 proposal stages

New JavaScript features are designed by **TC39**, the Ecma International committee of browser vendors, companies and invited experts. Each idea moves through numbered stages (see the table in Module 1): 0 strawperson, 1 proposal, 2 draft, 2.7 approved in principle, 3 candidate (browsers start shipping), and 4 finished, after which it is added to the next yearly ECMAScript edition published each June. A feature is safe to use in production when it reaches stage 4 and ships in all the browsers and Node.js versions you support; until then, **feature-detect** it and provide a fallback, as the example below does. (Its output shows that Node.js 22 already has the ES2025 Set methods and iterator helpers but not yet Promise.try or RegExp.escape, which arrived in later versions.)

Where to follow

The proposals list on github.com/tc39/proposals, the TC39 meeting notes, and the "What's new in ES20xx" posts on MDN and the V8 blog.

Checking support

caniuse.com and the MDN compatibility tables show which browser versions support each feature; node.green does the same for Node.js.

Using it early

Transpilers such as Babel and TypeScript, and polyfills such as core-js, let you use newer syntax and APIs in older environments (Module 19).

Recently finished or advanced	What it adds
Temporal (Module 10)	a modern date and time API that replaces Date
`Promise.try` (ES2025)	runs a function and always returns a promise
`RegExp.escape` (ES2025)	escapes user text for safe use in a regular expression
Import attributes / JSON modules (ES2025)	`import data from "./x.json" with { type: "json" }`
`Error.isError`, `Math.sumPrecise`, `Array.fromAsync`	smaller utilities reaching stage 4 / ES2026
Explicit resource management	`using` declarations that dispose resources automatically

DETECTING FEATURES IN NODE.JS 22

main.js

```
// Feature detection: check before using a feature your runtime may not have yet
const features = {
  "Set.prototype.union (ES2025)": typeof Set.prototype.union === "function",
  "Iterator helpers (ES2025)": typeof Iterator === "function" && typeof Iterator.prototype.take ===
"function",
  "Map.groupBy (ES2024)": typeof Map.groupBy === "function",
  "Promise.try (ES2025)": typeof Promise.try === "function",
  "RegExp.escape (ES2025)": typeof RegExp.escape === "function",
  "Temporal": typeof globalThis.Temporal === "object",
};
for (const [name, ok] of Object.entries(features)) console.log(ok ? "yes" : "no ", " ", name);

// a fallback (polyfill-style) when a feature is missing
const tryCall = Promise.try ? (fn) => Promise.try(fn) : (fn) => new Promise((resolve) =>
resolve(fn()));
tryCall(() => JSON.parse('{"ok":true}')).then((v) => console.log("result:", v));
```

OUTPUT

```
yes   Set.prototype.union (ES2025)
yes   Iterator helpers (ES2025)
yes   Map.groupBy (ES2024)
no    Promise.try (ES2025)
no    RegExp.escape (ES2025)
no    Temporal
result: { ok: true }
```

Destructuring unpacks arrays by position and objects by name, with renaming (prop: name), defaults (used only for undefined), nesting, parameter patterns for options objects and one-line swaps. Spread (...) expands iterables and objects in calls and literals (shallow copies); rest (...) gathers remaining arguments, items or properties. Enhanced literals add shorthand properties and methods and computed keys. Map stores ordered key-value pairs with any key type; Set stores unique values, and ES2025 adds union, intersection, difference, symmetricDifference and subset tests. WeakMap and WeakSet hold object keys weakly and are not iterable. Map.groupBy and Object.groupBy group items. Symbols are unique keys; well-known symbols such as Symbol.iterator customise language behaviour. The iterable/iterator protocols drive for...of and spread; generators (function*, yield) produce lazy, even infinite, sequences, and ES2025 iterator helpers (map, filter, take, drop, toArray) process them lazily. Numeric separators and globalThis round out the modern toolkit, and new features arrive through the TC39 stages.

PRACTICE CHECKPOINT

Remove duplicates and compute **set operations** on two lists with Set, count **word frequencies** with Map, and write a **generator** that yields an infinite ID sequence.

SAMPLE SOLUTION

main.js

```
// 1. Duplicates and set operations
const batchA = ["Asha", "Ravi", "Meera", "Asha", "John", "Ravi"];
const batchB = ["Ravi", "Zoya", "John", "Kiran", "Zoya"];
const setA = new Set(batchA), setB = new Set(batchB);
console.log("unique A:", [...setA]);
console.log("unique B:", [...setB]);
console.log("in both:     ", [...setA.intersection(setB)]);
console.log("either:      ", [...setA.union(setB)]);
console.log("only in A:   ", [...setA.difference(setB)]);
console.log("only in one: ", [...setA.symmetricDifference(setB)]);

// 2. Word frequencies with Map
const text = "the quick brown fox jumps over the lazy dog the fox sleeps";
const freq = new Map();
for (const word of text.split(" ")) {
  freq.set(word, (freq.get(word) ?? 0) + 1);
}
const top3 = [...freq].sort((a, b) => b[1] - a[1] || a[0].localeCompare(b[0])).slice(0, 3);
console.log("distinct words:", freq.size);
for (const [word, count] of top3) console.log(`  ${word.padEnd(5)} ${"*".repeat(count)} ${count}`);

// 3. Infinite ID generator
function* idGenerator(prefix = "LBF", start = 1) {
  let n = start;
  while (true) {
    const reset = yield `${prefix}-${String(n).padStart(4, "0")}`;
    n = reset ?? n + 1;               // next(value) can reset the counter
  }
}
const ids = idGenerator();
console.log(ids.next().value, ids.next().value, ids.next().value);
console.log("after reset to 500:", ids.next(500).value, ids.next().value);
console.log("five more with helpers:", idGenerator("STU", 21).take(5).toArray());
```

OUTPUT

```
unique A: [ 'Asha', 'Ravi', 'Meera', 'John' ]
unique B: [ 'Ravi', 'Zoya', 'John', 'Kiran' ]
in both:      [ 'Ravi', 'John' ]
either:       [ 'Asha', 'Ravi', 'Meera', 'John', 'Zoya', 'Kiran' ]
only in A:    [ 'Asha', 'Meera' ]
only in one:  [ 'Asha', 'Meera', 'Zoya', 'Kiran' ]
distinct words: 9
  the   *** 3
  fox   ** 2
  brown * 1
LBF-0001 LBF-0002 LBF-0003
after reset to 500: LBF-0500 LBF-0501
five more with helpers: [ 'STU-0021', 'STU-0022', 'STU-0023', 'STU-0024', 'STU-0025' ]
```

MODULE 13 · INTERMEDIATE

Object-oriented JavaScript: prototypes and classes

Minimum duration **4 days** | Level **Intermediate**

JavaScript objects inherit directly from other objects through prototypes. This module starts with constructor functions and what the new operator really does, explains prototypes and the prototype chain, and then shows the ES6 class syntax that sits on top of this model: constructors, methods, getters and setters, static members, public and private fields, and static blocks. It continues with inheritance (extends, super, extending built-ins), the four OOP principles in JavaScript style, composition and mixins, and property descriptors.

IN THIS MODULE

13.1 Constructor functions
13.2 Prototypes
13.3 ES6 classes
13.4 Inheritance with classes
13.5 OOP principles in JavaScript
13.6 Property descriptors

13.1 Constructor functions

Object literals are perfect for a single object, but when you need many objects with the same shape (100 students, 50 products), you want a reusable **blueprint**. Before ES6, the blueprint was a **constructor function**: an ordinary function, named in PascalCase by convention, that is called with the `new` keyword and fills in `this`. ES6 classes (13.3) are built on exactly this mechanism, so understanding it explains how classes really work.

13.1.1 Creating objects with new

A CONSTRUCTOR FUNCTION USED WITH NEW

main.js

```
function Student(name, marks) {
  this.name = name;              // "this" is the new, empty object
  this.marks = marks;
  this.passed = marks >= 40;
}

const s1 = new Student("Asha", 91);
const s2 = new Student("Ravi", 35);
console.log(s1);
console.log(s2);
console.log("s1 instanceof Student:", s1 instanceof Student);
console.log("s1.constructor === Student:", s1.constructor === Student);

// forgetting new (sloppy mode): "this" is the global object and nothing is returned
const oops = Student("Meera", 70);
console.log("without new returns:", oops, "| leaked global:", globalThis.marks);

function StrictStudent(name) { "use strict"; this.name = name; }
try { StrictStudent("Meera"); } catch (e) { console.log("strict, without new ->", e.constructor.name +
":", e.message); }
```

OUTPUT

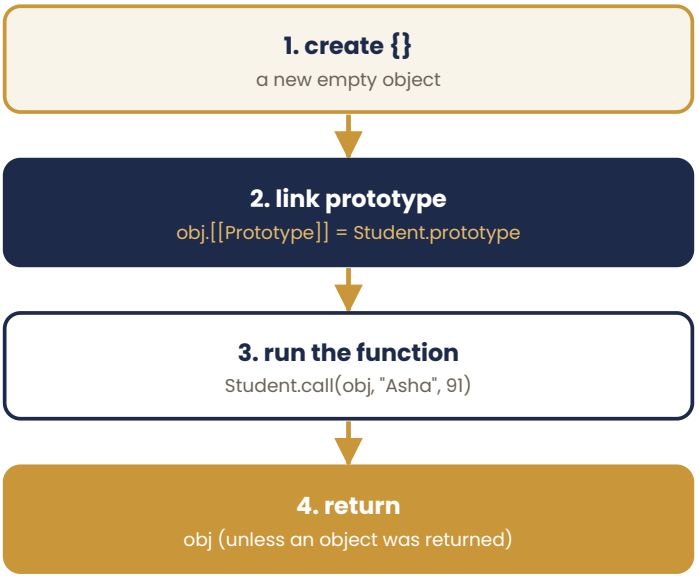
```
Student { name: 'Asha', marks: 91, passed: true }
Student { name: 'Ravi', marks: 35, passed: false }
s1 instanceof Student: true
s1.constructor === Student: true
without new returns: undefined | leaked global: 70
strict, without new -> TypeError: Cannot set properties of undefined (setting 'name')
```

WATCH OUT

Calling a constructor **without** `new` is a classic bug: in sloppy mode `this` is the global object, so the "properties" become global variables; in strict mode `this` is `undefined` and the first assignment throws. ES6 classes prevent this by refusing to be called without `new`.

13.1.2 What new does step by step

`new Student("Asha", 91)` performs four steps: (1) create a brand-new empty object; (2) set that object's hidden **[[Prototype]]** link to `Student.prototype`; (3) call `Student` with `this` bound to the new object, so the function body fills it in; (4) return the new object automatically, unless the function explicitly returns some other **object** (returning a primitive is ignored). Writing these steps by hand makes `new` much less mysterious.



THE FOUR STEPS OF NEW

RE-IMPLEMENTING NEW main.js

```
function Student(name, marks) {
  this.name = name;
  this.marks = marks;
}
Student.prototype.grade = function () { return this.marks >= 80 ? "A" : "B"; };

function myNew(Constructor, ...args) {
  const obj = {};                                        // 1
  Object.setPrototypeOf(obj, Constructor.prototype);     // 2
  const result = Constructor.apply(obj, args);           // 3
  return result !== null && typeof result === "object" ? result : obj;   // 4
}

const a = myNew(Student, "Asha", 91);
const b = new Student("Asha", 91);
console.log(a, a.grade(), a instanceof Student);
console.log(b, b.grade(), b instanceof Student);

function Weird() { this.x = 1; return { replaced: true }; }   // returning an object wins
function Fine() { this.x = 1; return 42; }                    // primitive return ignored
console.log(new Weird(), new Fine());
```

OUTPUT

```
Student { name: 'Asha', marks: 91 } A true
Student { name: 'Asha', marks: 91 } A true
{ replaced: true } Fine { x: 1 }
```

13.2 Prototypes

13.2.1 prototype property and [[Prototype]] (__proto__)

Two similar names cause most of the confusion here. Every object has an internal, hidden link called **[[Prototype]]** that points to another object (or `null`); it is read with `Object.getPrototypeOf(obj)` and is also exposed by the old accessor `obj.__proto__`. Separately, every ordinary **function** has a property named `prototype`: an object that becomes the [[Prototype]] of everything created by calling that

function with `new`. So `Object.getPrototypeOf(s1) === Student.prototype`, and `Student.prototype.constructor === Student`.

PROTOTYPE VS [[PROTOTYPE]]

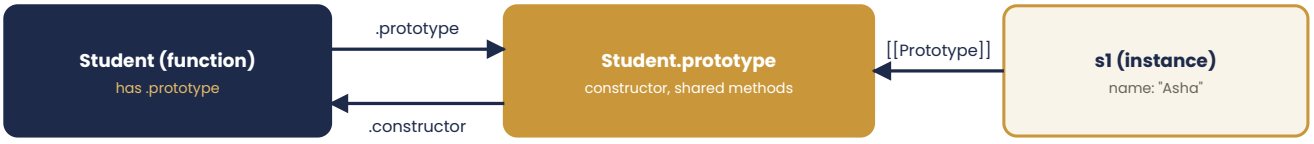
main.js

```
function Student(name) { this.name = name; }
const s1 = new Student("Asha");

console.log("Student.prototype:", Student.prototype);
console.log("[[Prototype]] of s1 is Student.prototype:", Object.getPrototypeOf(s1) ===
Student.prototype);
console.log("same via __proto__:", s1.__proto__ === Student.prototype);
console.log("Student.prototype.constructor === Student:", Student.prototype.constructor === Student);
console.log("s1 has its own 'prototype' property?", "prototype" in s1);    // only functions have it
console.log("arrow functions have prototype?", (() => {}).prototype);
```

OUTPUT

```
Student.prototype: {}
[[Prototype]] of s1 is Student.prototype: true
same via __proto__: true
Student.prototype.constructor === Student: true
s1 has its own 'prototype' property? false
arrow functions have prototype? undefined
```



HOW A CONSTRUCTOR, ITS PROTOTYPE OBJECT AND AN INSTANCE ARE LINKED

NOTE

`__proto__` is a legacy accessor kept for compatibility. In new code, use `Object.getPrototypeOf`, `Object.setPrototypeOf` (rarely, it is slow) and `Object.create`.

13.2.2 The prototype chain

When you read a property, JavaScript first looks on the object itself (an **own** property). If it is not there, it follows the [[Prototype]] link and looks there, then that object's prototype, and so on until it finds the property or reaches `null`, in which case the result is `undefined`. This sequence of linked objects is the **prototype chain**; it is how every object gets `toString` and `hasOwnProperty` from `Object.prototype`, and how every array gets `map` from `Array.prototype`. Writing a property, by contrast, always creates or changes an own property on the object itself (this is called **shadowing**).

LOOKING UP PROPERTIES ALONG THE CHAIN

main.js

```
function Student(name) { this.name = name; }
Student.prototype.school = "LBF Academy";
const s1 = new Student("Asha");

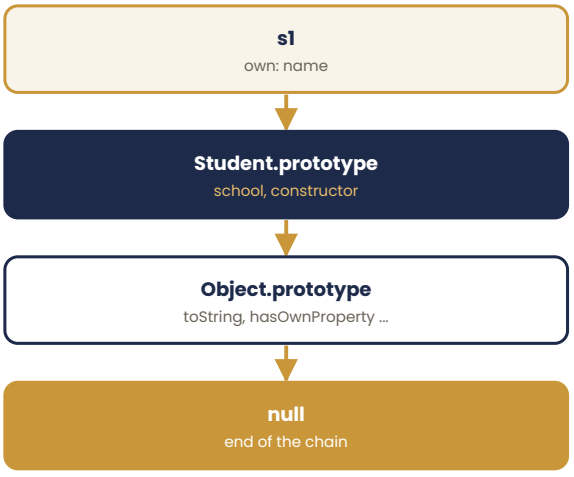
console.log(s1.name, "| own?", Object.hasOwn(s1, "name"));
console.log(s1.school, "| own?", Object.hasOwn(s1, "school"));     // found one level up
console.log(typeof s1.toString, "| from Object.prototype");       // two levels up
console.log(s1.missing);                                          // reached null

// walk the chain
let obj = s1, level = 0;
while (obj) {
  console.log(`  level ${level++}:`, obj === s1 ? "s1" : obj.constructor.name + ".prototype");
  obj = Object.getPrototypeOf(obj);
}
console.log(`  level ${level}: null (end of chain)`);

// writing shadows instead of changing the prototype
s1.school = "Own School";
const s2 = new Student("Ravi");
console.log(s1.school, "|", s2.school);
```

OUTPUT

```
Asha | own? true
LBF Academy | own? false
function | from Object.prototype
undefined
  level 0: s1
  level 1: Student.prototype
  level 2: Object.prototype
  level 3: null (end of chain)
Own School | LBF Academy
```



THE PROTOTYPE CHAIN OF S1

13.2.3 Adding methods to the prototype

Methods defined inside a constructor (`this.greet = function () {...}`) are created again for every object: 1,000 students means 1,000 identical function objects. Methods placed on the constructor's `prototype` exist **once** and are shared by all instances through the chain, while each instance keeps only its own data. Methods added to the prototype later are immediately available to objects that already exist.

SHARED METHODS ON THE PROTOTYPE main.js

```
function Slow(name) {
  this.name = name;
  this.greet = function () { return "Hi " + this.name; };   // a new function per object
}
function Fast(name) { this.name = name; }
Fast.prototype.greet = function () { return "Hi " + this.name; };   // one shared function

const s1 = new Slow("A"), s2 = new Slow("B");
const f1 = new Fast("A"), f2 = new Fast("B");
console.log("Slow: same function?", s1.greet === s2.greet);
console.log("Fast: same function?", f1.greet === f2.greet);
console.log(f1.greet(), "|", f2.greet());

Fast.prototype.shout = function () { return this.greet().toUpperCase() + "!"; };   // added later
console.log(f1.shout(), "(existing object sees the new method)");
console.log("own keys of f1:", Object.keys(f1));
```

OUTPUT

```
Slow: same function? false
Fast: same function? true
Hi A | Hi B
HI A! (existing object sees the new method)
own keys of f1: [ 'name' ]
```

13.2.4 Object.getPrototypeOf() and Object.create()

`Object.getPrototypeOf(obj)` returns an object's prototype. `Object.create(proto, descriptors?)` creates a new object whose prototype is `proto`, without any constructor at all; this is the purest form of prototypal inheritance ("objects linked to other objects"). `Object.create(null)` creates an object with **no** prototype, a clean dictionary that inherits nothing. `isPrototypeOf` tests whether an object appears anywhere in another's chain.

LINKING OBJECTS DIRECTLY

main.js

```
const animal = {
  eats: true,
  describe() { return `${this.name} eats: ${this.eats}, sound: ${this.sound()}`; },
  sound() { return "..."; },
};

const dog = Object.create(animal);         // dog -> animal -> Object.prototype
dog.name = "Bruno";
dog.sound = function () { return "Woof"; }; // overrides animal.sound

console.log(dog.describe());
console.log("prototype of dog is animal:", Object.getPrototypeOf(dog) === animal);
console.log("animal.isPrototypeOf(dog):", animal.isPrototypeOf(dog));
console.log("own keys:", Object.keys(dog), "| inherited eats:", dog.eats);

const dict = Object.create(null);          // no prototype at all
dict.constructor = "just a key";
console.log("prototype:", Object.getPrototypeOf(dict), "| toString?", typeof dict.toString, "|",
dict.constructor);
```

OUTPUT

```
Bruno eats: true, sound: Woof
prototype of dog is animal: true
animal.isPrototypeOf(dog): true
own keys: [ 'name', 'sound' ] | inherited eats: true
prototype: null | toString? undefined | just a key
```

13.2.5 Built-in prototypes (Array.prototype, String.prototype)

Built-in types work the same way. `[1, 2]` is linked to `Array.prototype` (where `map`, `push` and the rest live), which is linked to `Object.prototype`. Even primitives use prototypes: when you call `"hi".toUpperCase()`, JavaScript temporarily wraps the string in a String object so that `String.prototype` methods can be found. It is technically possible to add methods to built-in prototypes, but this is called **monkey-patching** and should be avoided: it can clash with future standard methods (this happened with `Array.prototype.flatten`, which is why the standard method is called `flat`) and with other libraries.

BUILT-IN PROTOTYPES

main.js

```
const nums = [1, 2, 3];
console.log(Object.getPrototypeOf(nums) === Array.prototype);
console.log(Object.getPrototypeOf(Array.prototype) === Object.prototype);
console.log("map is own on nums?", Object.hasOwn(nums, "map"), "| on Array.prototype?",
Object.hasOwn(Array.prototype, "map"));

console.log(Object.getPrototypeOf("hi") === String.prototype, Object.getPrototypeOf(42) ===
Number.prototype);
console.log("some String.prototype methods:", Object.getOwnPropertyNames(String.prototype).filter((m)
=> m.startsWith("to")).slice(0, 5));

// Monkey-patching works, but avoid it in real code
Array.prototype.last = function () { return this[this.length - 1]; };
console.log("patched last():", nums.last());
for (const key in nums) console.log("  for...in now sees:", key);   // the patch leaks into for...in
delete Array.prototype.last;
```

OUTPUT

```
true
true
map is own on nums? false | on Array.prototype? true
true true
some String.prototype methods: [
  'toString',
  'toWellFormed',
  'toLocaleLowerCase',
  'toLocaleUpperCase',
  'toLowerCase'
]
patched last(): 3
  for...in now sees: 0
  for...in now sees: 1
  for...in now sees: 2
  for...in now sees: last
```

WATCH OUT

Never modify built-in prototypes in application or library code. If you need a helper, write a normal function (`last(arr)`). The only accepted use is a **polyfill** that adds a standard method missing from an older environment, after checking that it does not already exist.

13.2.6 Prototypal inheritance with constructor functions

To make one constructor inherit from another before ES6, three steps were needed: call the parent constructor inside the child (`Parent.call(this, ...)`) to set up the inherited own properties; link the child's prototype to the parent's with `Object.create(Parent.prototype)` so methods are inherited; and restore the `constructor` property that this replacement overwrote. ES6 `extends` (13.4) does all of this for you, but you will still meet this pattern in older code.

INHERITANCE THE PRE-ES6 WAY

main.js

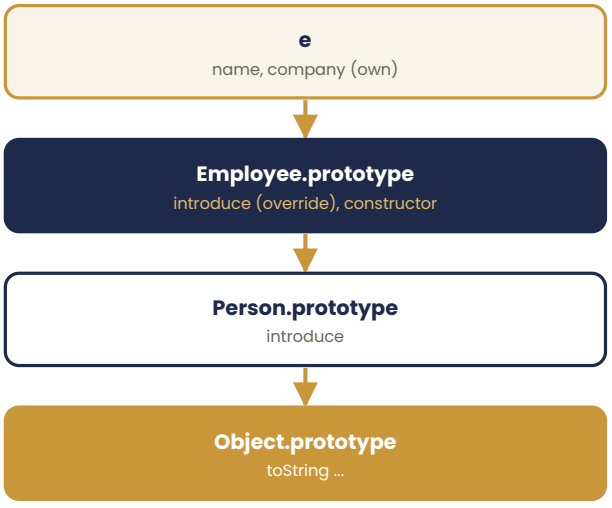
```
function Person(name) {
  this.name = name;
}
Person.prototype.introduce = function () { return `I am ${this.name}`; };

function Employee(name, company) {
  Person.call(this, name);                          // 1. parent's own properties
  this.company = company;
}
Employee.prototype = Object.create(Person.prototype);    // 2. inherit methods
Employee.prototype.constructor = Employee;               // 3. repair constructor
Employee.prototype.introduce = function () {             // override + reuse parent
  return Person.prototype.introduce.call(this) + `, I work at ${this.company}`;
};

const e = new Employee("Asha", "Infosys");
console.log(e.introduce());
console.log(e instanceof Employee, e instanceof Person, e.constructor.name);
console.log("chain:", [Object.getPrototypeOf(e) === Employee.prototype,
Object.getPrototypeOf(Employee.prototype) === Person.prototype]);
```

OUTPUT

```
I am Asha, I work at Infosys
true true Employee
chain: [ true, true ]
```



THE CHAIN BUILT BY OBJECT.CREATE(PERSON.PROTOTYPE)

13.3 ES6 classes

The `class` keyword (ES2015) is a cleaner syntax for the constructor-and-prototype pattern, with extra features added since: fields, private members and static blocks. A class is still a function underneath (`typeof MyClass === "function"`) and its methods still live on `MyClass.prototype`. Classes differ from constructor functions in a few safe ways: they must be called with `new`, their bodies always run in strict mode, their methods are non-enumerable, and class declarations are not usable before the line that defines them (they are in the TDZ like `let`).

13.3.1 class syntax and constructor

DECLARING A CLASS

main.js

```
class Student {
  constructor(name, marks) {     // runs on every new Student(...)
    this.name = name;
    this.marks = marks;
  }
}

const s = new Student("Asha", 91);
console.log(s, typeof Student);
console.log("prototype link as before:", Object.getPrototypeOf(s) === Student.prototype);

try { Student("Ravi", 50); } catch (e) { console.log("without new ->", e.message); }

try { new Later(); } catch (e) { console.log("before declaration ->", e.name); }
class Later {}

// a class without a constructor gets an empty default one
class Empty {}
console.log(new Empty());
```

OUTPUT

```
Student { name: 'Asha', marks: 91 } function
prototype link as before: true
without new -> Class constructor Student cannot be invoked without 'new'
before declaration -> ReferenceError
Empty {}
```

A class has at most one `constructor`. JavaScript does not support overloading (several constructors with different parameters); use default parameters, an options object or static factory methods (13.3.4) instead. Classes can also be written as expressions (`const Point = class { ... }`) and passed around like any value.

13.3.2 Instance methods

Methods written in the class body are placed on the prototype, shared by every instance, and called on an instance with `this` referring to it. Returning `this` from a method allows **method chaining**. Remember from Module 11 that a method loses its `this` when passed as a bare callback; class fields with arrow functions (13.3.5) or `bind` solve that.

METHODS AND CHAINING

main.js

```
class Cart {
  constructor() { this.items = []; }
  add(name, price, qty = 1) {
    this.items.push({ name, price, qty });
    return this;                              // enables chaining
  }
  total() { return this.items.reduce((t, i) => t + i.price * i.qty, 0); }
  summary() { return `${this.items.length} lines, total Rs ${this.total()}`; }
}

const cart = new Cart().add("Pen", 10, 5).add("Book", 250).add("Bag", 899);
console.log(cart.summary());
console.log("methods shared:", new Cart().total === cart.total);
console.log("methods are non-enumerable:", Object.keys(Cart.prototype),
Object.getOwnPropertyNames(Cart.prototype));
```

OUTPUT

```
3 lines, total Rs 1199
methods shared: true
methods are non-enumerable: [] [ 'constructor', 'add', 'total', 'summary' ]
```

13.3.3 Getters and setters

`get` and `set` define **accessor properties**: code that runs when a property is read or assigned, while callers use plain property syntax. Getters are ideal for computed values (`fullName`, `area`); setters can validate or convert input. A setter usually stores the value in another property (often a private field, 13.3.6) to avoid calling itself endlessly.

ACCESSOR PROPERTIES WITH VALIDATION main.js

```
class Temperature {
  #celsius = 0;
  get celsius() { return this.#celsius; }
  set celsius(value) {
    if (typeof value !== "number" || value < -273.15) throw new RangeError(`Invalid temperature:
${value}`);
    this.#celsius = value;
  }
  get fahrenheit() { return this.#celsius * 9 / 5 + 32; }       // computed
  set fahrenheit(f) { this.celsius = (f - 32) * 5 / 9; }        // converts, reuses validation
}

const t = new Temperature();
t.celsius = 37;
console.log(t.celsius, "C =", t.fahrenheit, "F");
t.fahrenheit = 212;
console.log(t.celsius, "C after setting 212 F");
try { t.celsius = -300; } catch (e) { console.log(e.name + ":", e.message); }

class Square {
  constructor(side) { this.side = side; }
  get area() { return this.side ** 2; }        // getter only: read-only
}
const sq = new Square(4);
sq.area = 100;                                  // ignored (throws in strict mode code)
console.log("area:", sq.area);
```

OUTPUT

```
37 C = 98.6 F
100 C after setting 212 F
RangeError: Invalid temperature: -300
area: 16
```

13.3.4 Static methods and properties

Members marked `static` belong to the **class itself**, not to instances: you call `MathUtil.max()`, not `obj.max()`. Use them for utility functions related to the class, constants, counters shared by all instances, and **factory methods** that create instances in different ways (a common replacement for constructor overloading). Inside a static method, `this` is the class.

STATIC MEMBERS AND A FACTORY METHOD main.js

```
class User {
  static #count = 0;                          // private static counter
  static ROLES = ["admin", "editor", "viewer"];

  constructor(name, role = "viewer") {
    if (!User.ROLES.includes(role)) throw new Error(`Unknown role ${role}`);
    this.name = name;
    this.role = role;
    this.id = ++User.#count;
  }
  static fromEmail(email) {                   // factory method
    const name = email.split("@")[0];
    return new this(name[0].toUpperCase() + name.slice(1));
  }
  static get count() { return User.#count; }
  static compareByName(a, b) { return a.name.localeCompare(b.name); }
}

const users = [new User("Ravi", "admin"), User.fromEmail("asha@lbf.in"), new User("Meera", "editor")];
console.log(users.sort(User.compareByName).map((u) => `${u.id}:${u.name}(${u.role})`));
console.log("created:", User.count, "| roles:", User.ROLES);
console.log("instances do not see static members:", users[0].fromEmail);
```

```
OUTPUT
[ '2:Asha(viewer)', '3:Meera(editor)', '1:Ravi(admin)' ]
created: 3 | roles: [ 'admin', 'editor', 'viewer' ]
instances do not see static members: undefined
```

13.3.5 Public class fields

Class fields (ES2022) declare instance properties directly in the class body, with optional initial values: `count = 0;`. They are created on each instance before the constructor body runs, make the shape of the object visible at a glance, and reduce constructor boilerplate. A field initialised with an **arrow function** captures `this` permanently, which makes it safe to pass as an event handler or callback (at the cost of one function per instance).

FIELDS AND ARROW-FUNCTION FIELDS

main.js

```
class Counter {
  count = 0;                                  // public field
  step = 1;
  label;                                      // declared, starts as undefined
  createdAt = "2026-03-09";

  constructor(label) { this.label = label; }

  increment() { this.count += this.step; return this.count; }
  handleClick = () => {                       // arrow field: this is always the instance
    this.count++;
    return `${this.label}: ${this.count}`;
  };
}

const c = new Counter("likes");
console.log(c);
const { handleClick } = c;                    // detached, still works
console.log(handleClick(), handleClick());
const { increment } = c;
try { increment(); } catch (e) { console.log("detached normal method ->", e.constructor.name); }
```

OUTPUT

```
Counter {
  count: 0,
  step: 1,
  label: 'likes',
  createdAt: '2026-03-09',
  handleClick: [Function: handleClick]
}
likes: 1 likes: 2
detached normal method -> TypeError
```

13.3.6 Private fields and methods #field

Names starting with `#` are **private** (ES2022): they can only be accessed by code inside the class body. This is true privacy enforced by the language, not a naming convention like `_balance`: outside code gets a SyntaxError, the field does not appear in `Object.keys`, `JSON.stringify` or `console.log` of plain objects, and it cannot be added from outside. Private **methods** (`#validate()`), private getters and private static members work the same way. `#field in obj` checks whether an object has the private field (a reliable "is this really one of mine" test).

REAL PRIVACY WITH # MEMBERS main.js

```
class BankAccount {
  #balance = 0;                          // private field
  #history = [];

  constructor(owner, opening = 0) {
    this.owner = owner;
    this.#deposit(opening, "opening");
  }
  #deposit(amount, note) {               // private method
    this.#validate(amount);
    this.#balance += amount;
    this.#history.push(`+${amount} (${note})`);
  }
  #validate(amount) {
    if (!(amount > 0)) throw new RangeError("Amount must be positive");
  }
  deposit(amount) { this.#deposit(amount, "deposit"); return this; }
  get balance() { return this.#balance; }
  get history() { return [...this.#history]; }   // a copy, so callers cannot change it
  static isAccount(obj) { return #balance in obj; }
}

const acc = new BankAccount("Asha", 1000).deposit(500);
console.log(acc.owner, acc.balance, acc.history);
console.log("keys:", Object.keys(acc), "| JSON:", JSON.stringify(acc));
acc.balance = 1_000_000;                          // no setter: ignored
console.log("after trying to set balance:", acc.balance);
acc.history.push("+999999 (hack)");
console.log("history unchanged:", acc.history.length);
try { acc.deposit(-5); } catch (e) { console.log(e.name + ":", e.message); }
console.log("isAccount:", BankAccount.isAccount(acc), BankAccount.isAccount({ balance: 5 }));
```

OUTPUT

```
Asha 1500 [ '+1000 (opening)', '+500 (deposit)' ]
keys: [ 'owner' ] | JSON: {"owner":"Asha"}
after trying to set balance: 1500
history unchanged: 2
RangeError: Amount must be positive
isAccount: true false
```

ACCESSING A PRIVATE FIELD FROM OUTSIDE main.js

```
console.log(acc.#balance);
```

ERROR

```
SyntaxError: Private field '#balance' must be declared in an enclosing class
```

The error is reported when the file is parsed, so not a single line of the file runs.

13.3.7 Static initialization blocks

A `static { ... }` block (ES2022) runs **once**, when the class is defined, with `this` referring to the class. It is used for static set-up that needs more than a single expression, such as try/catch, loops or computing several static fields together, and it can access private static members.

ONE-TIME STATIC SET-UP main.js

```
class Config {
  static settings;
  static #secretKey;
  static {
    console.log("static block runs once, while the class is being defined");
    const raw = '{"theme":"dark","pageSize":"20"}';
    try {
      const parsed = JSON.parse(raw);
      this.settings = { ...parsed, pageSize: Number(parsed.pageSize) };
    } catch {
      this.settings = { theme: "light", pageSize: 10 };
    }
    this.#secretKey = "K-" + this.settings.theme.length;
  }
  static keyHint() { return Config.#secretKey.slice(0, 2) + "***"; }
}

console.log("after the class definition");
console.log(Config.settings, Config.keyHint());
new Config(); new Config();                  // no further block runs
```

OUTPUT

```
static block runs once, while the class is being defined
after the class definition
{ theme: 'dark', pageSize: 20 } K-***
```

Member	Syntax	Belongs to	Visible outside?
constructor	`constructor(a) { }`	runs per instance	n/a
method	`m() { }`	prototype (shared)	yes
getter / setter	`get x() { } set x(v) { }`	prototype	yes
public field	`x = 1;`	each instance	yes
private field / method	`#x = 1; #m() { }`	each instance	no
static method / field	`static m() { } static X = 1;`	the class	yes
private static	`static #x;`	the class	no
static block	`static { }`	runs once at definition	n/a

13.4 Inheritance with classes

13.4.1 extends and super()

`class Child extends Parent` sets up both links that 13.2.6 built by hand: `Child.prototype` inherits from `Parent.prototype` (instance methods), and `Child` itself inherits from `Parent` (static methods). If the child has a constructor, it **must** call `super(...)` before using `this`, because in a derived class it is the parent constructor that actually creates the object. A child with no constructor gets one that passes all arguments to the parent.

EXTENDING A CLASS

main.js

```
class Person {
  constructor(name) { this.name = name; }
  introduce() { return `I am ${this.name}`; }
  static create(name) { return new this(name); }
}

class Student extends Person {
  constructor(name, course) {
    super(name);                // must come before "this"
    this.course = course;
  }
  study() { return `${this.name} studies ${this.course}`; }
}

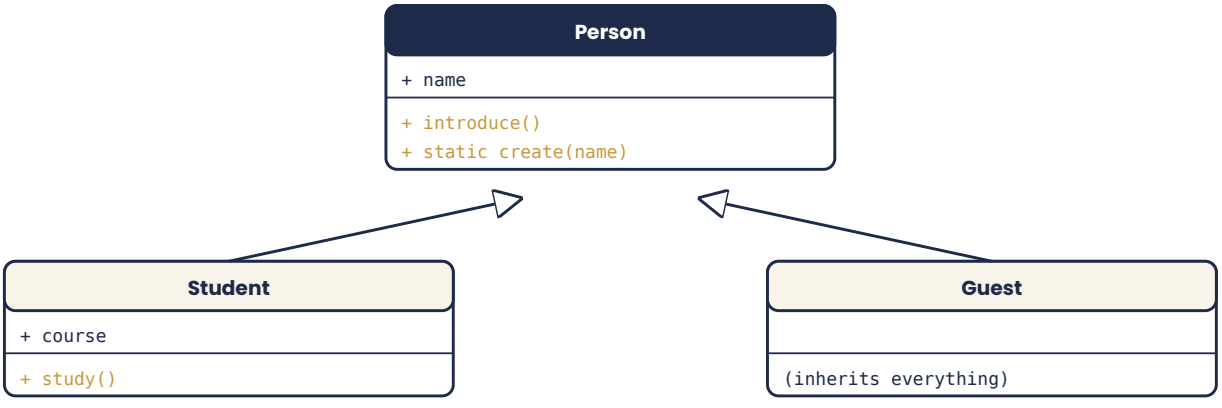
class Guest extends Person {}    // default constructor: (...args) => super(...args)

const s = new Student("Asha", "JavaScript");
console.log(s, "|", s.introduce(), "|", s.study());
console.log(new Guest("Ravi").introduce());
console.log("static inherited:", Student.create("Meera") instanceof Student);
console.log("links:", Object.getPrototypeOf(Student.prototype) === Person.prototype,
Object.getPrototypeOf(Student) === Person);

class Broken extends Person {
  constructor(name) { this.name = name; }   // forgot super()
}
try { new Broken("X"); } catch (e) { console.log(e.name + ":", e.message); }
```

OUTPUT

```
Student { name: 'Asha', course: 'JavaScript' } | I am Asha | Asha studies JavaScript
I am Ravi
static inherited: true
links: true true
ReferenceError: Must call super constructor in derived class before accessing 'this' or returning from
derived constructor
```



STUDENT AND GUEST EXTEND PERSON

13.4.2 Method overriding and super.method()

A child class **overrides** a parent method by defining a method with the same name; calls on child instances now find the child's version first in the prototype chain. Inside the override, `super.method(...)` calls the parent's version, so the child can **extend** behaviour instead of copying it. `super` works in methods, getters, static methods and object-literal methods.

OVERRIDING AND CALLING THE PARENT VERSION

main.js

```
class Shape {
  constructor(name) { this.name = name; }
  area() { return 0; }
  describe() { return `${this.name} with area ${this.area().toFixed(2)}`; }
  toString() { return `[Shape ${this.name}]`; }
}

class Circle extends Shape {
  constructor(r) { super("circle"); this.r = r; }
  area() { return Math.PI * this.r ** 2; }                  // override
}

class Rectangle extends Shape {
  constructor(w, h) { super("rectangle"); Object.assign(this, { w, h }); }
  area() { return this.w * this.h; }
  describe() { return super.describe() + ` (${this.w} x ${this.h})`; }   // extend
}

class Square extends Rectangle {
  constructor(side) { super(side, side); this.name = "square"; }
  toString() { return super.toString().replace("Shape", "Square"); }
}

for (const s of [new Circle(1), new Rectangle(2, 3), new Square(4)]) console.log(s.describe());
console.log(String(new Square(2)));
```

OUTPUT

```
circle with area 3.14
rectangle with area 6.00 (2 x 3)
square with area 16.00 (4 x 4)
[Square square]
```

13.4.3 Extending built-in classes (Array, Error)

Classes can extend built-ins such as `Array`, `Map` and `Error`, inheriting all their behaviour. A subclass of Array keeps the special `length` behaviour, and methods like `map` and `filter` return instances of the subclass. Extending `Error` is the standard way to create custom error types (Module 14): set `name` so that messages and logs show the right type.

SUBCLASSING ARRAY, MAP AND ERROR main.js

```
class Stack extends Array {
  peek() { return this[this.length - 1]; }
  get isEmpty() { return this.length === 0; }
}
const st = new Stack();
st.push(10, 20, 30);
console.log(st, "peek:", st.peek(), "length:", st.length, "empty:", st.isEmpty);
const doubled = st.map((x) => x * 2);
console.log("map returns a Stack too:", doubled instanceof Stack, doubled.peek());

class DefaultMap extends Map {
  constructor(makeDefault) { super(); this.makeDefault = makeDefault; }
  get(key) {
    if (!this.has(key)) this.set(key, this.makeDefault());
    return super.get(key);
  }
}
const groups = new DefaultMap(() => []);
for (const w of ["apple", "avocado", "banana"]) groups.get(w[0]).push(w);
console.log(groups);

class InsufficientFundsError extends Error {
  constructor(needed, available) {
    super(`Need Rs ${needed}, only Rs ${available} available`);
    this.name = "InsufficientFundsError";
    this.shortBy = needed - available;
  }
}
try {
  throw new InsufficientFundsError(500, 320);
} catch (e) {
  console.log(e.name, "|", e.message, "| short by", e.shortBy, "| is Error:", e instanceof Error);
}
```

OUTPUT

```
Stack(3) [ 10, 20, 30 ] peek: 30 length: 3 empty: false
map returns a Stack too: true 60
DefaultMap(2) {
  'a' => [ 'apple', 'avocado' ],
  'b' => [ 'banana' ],
  makeDefault: [Function (anonymous)]
}
InsufficientFundsError | Need Rs 500, only Rs 320 available | short by 180 | is Error: true
```

13.4.4 instanceof

`obj instanceof C` returns `true` if `C.prototype` appears anywhere in `obj`'s prototype chain. It therefore recognises parent classes as well, and it can be fooled by changing prototypes, or fail for objects created in another window or iframe (another set of built-ins). For arrays use `Array.isArray`, and for duck typing check for the method you need (`typeof x.render === "function"`).

HOW INSTANCEOF DECIDES

main.js

```
class Animal {}
class Dog extends Animal {}
const d = new Dog();

console.log("d instanceof Dog:", d instanceof Dog);
console.log("d instanceof Animal:", d instanceof Animal);
console.log("d instanceof Object:", d instanceof Object);
console.log("primitive 'hi' instanceof String:", "hi" instanceof String);
console.log("Object.create(null) instanceof Object:", Object.create(null) instanceof Object);

// what instanceof really checks
function myInstanceOf(obj, C) {
  for (let p = Object.getPrototypeOf(obj); p; p = Object.getPrototypeOf(p)) if (p === C.prototype)
return true;
  return false;
}
console.log("myInstanceOf(d, Animal):", myInstanceOf(d, Animal));

// duck typing: check the capability, not the class
const looksLikeDog = { bark() { return "Woof"; } };
console.log("has bark?", typeof looksLikeDog.bark === "function", "| instanceof Dog?", looksLikeDog
instanceof Dog);
```

OUTPUT

```
d instanceof Dog: true
d instanceof Animal: true
d instanceof Object: true
primitive 'hi' instanceof String: false
Object.create(null) instanceof Object: false
myInstanceOf(d, Animal): true
has bark? true | instanceof Dog? false
```

13.5 OOP principles in JavaScript

Encapsulation
Keep data and the code that changes it together, and hide internal state (#private fields, closures).

Abstraction
Expose a simple interface and hide the complex details behind it.

Inheritance
Reuse and specialise behaviour through extends and the prototype chain.

Polymorphism
Different objects respond to the same method call in their own way.

13.5.1 Encapsulation with private fields and closures

Encapsulation means an object controls its own state: outside code changes it only through methods that keep it valid. JavaScript offers two strong tools. **Private fields** (13.3.6) in classes, and **closures** (Module 11) in factory functions, where the state is a local variable that only the returned methods can see. Both make illegal states impossible to create from outside.

ENCAPSULATION WITH A CLOSURE main.js

```
// Closure-based encapsulation: a factory function
function createWallet(owner) {
  let balance = 0;                          // truly hidden: only the methods below see it
  const log = [];
  return {
    owner,
    pay(amount) {
      if (amount > balance) return `Declined: balance ${balance}`;
      balance -= amount;
      log.push(-amount);
      return `Paid ${amount}, left ${balance}`;
    },
    topUp(amount) { balance += amount; log.push(+amount); return balance; },
    get balance() { return balance; },
    statement: () => log.join(", "),
  };
}

const w = createWallet("Asha");
w.topUp(500);
console.log(w.pay(120));
console.log(w.pay(1000));
w.balance = 1_000_000;                      // getter only: no effect
console.log("balance:", w.balance, "| statement:", w.statement());
console.log("keys visible:", Object.keys(w));
```

OUTPUT

```
Paid 120, left 380
Declined: balance 380
balance: 380 | statement: 500, -120
keys visible: [ 'owner', 'pay', 'topUp', 'balance', 'statement' ]
```

	Private #fields (class)	Closure (factory function)
Privacy	enforced by the language	enforced by scope
Methods	shared on the prototype (memory efficient)	new functions per object
instanceof	works	no class to check
this	needed; can be lost in callbacks	not needed; methods can be passed freely
Typical use	many objects, inheritance	modules, small objects, functional style

13.5.2 Abstraction

Abstraction means users of an object only need to know **what** it does, not **how**. A `PaymentGateway` exposes `pay(amount)`; whether it talks to UPI, a card network or a test stub is hidden. JavaScript has no `abstract` keyword (TypeScript adds one, Module 23), but the idea is expressed with a base class whose methods throw "not implemented", and by preventing the base class itself from being instantiated with `new.target`.

AN ABSTRACT BASE CLASS

main.js

```
class PaymentMethod {
  constructor() {
    if (new.target === PaymentMethod) throw new TypeError("PaymentMethod is abstract");
  }
  pay(amount) {                                  // the template: same steps for everyone
    this.validate(amount);
    const ref = this.process(amount);            // the part each subclass supplies
    return `Rs ${amount} paid via ${this.label} (ref ${ref})`;
  }
  validate(amount) { if (amount <= 0) throw new RangeError("Invalid amount"); }
  process() { throw new Error(`${this.constructor.name} must implement process()`); }
}

class Upi extends PaymentMethod {
  label = "UPI";
  constructor(vpa) { super(); this.vpa = vpa; }
  process(amount) { return `UPI-${this.vpa.split("@")[0].toUpperCase()}-${amount}`; }
}
class Card extends PaymentMethod {
  label = "card";
  constructor(last4) { super(); this.last4 = last4; }
  process(amount) { return `CARD-${this.last4}-${amount * 7 % 1000}`; }
}
class Wallet extends PaymentMethod { label = "wallet"; }   // forgot process()

console.log(new Upi("asha@okbank").pay(499));
console.log(new Card("4242").pay(1200));
try { new PaymentMethod(); } catch (e) { console.log(e.message); }
try { new Wallet().pay(10); } catch (e) { console.log(e.message); }
```

OUTPUT

```
Rs 499 paid via UPI (ref UPI-ASHA-499)
Rs 1200 paid via card (ref CARD-4242-400)
PaymentMethod is abstract
Wallet must implement process()
```

13.5.3 Polymorphism

Polymorphism ("many forms") lets code call the same method on different kinds of objects and get behaviour appropriate to each, without `if` / `switch` on the type. In JavaScript it comes from method overriding in subclasses and also from **duck typing**: any object that has the right method can be used, even if it is unrelated by inheritance ("if it walks like a duck and quacks like a duck…").

ONE CALL, MANY BEHAVIOURS

main.js

```
class Notification {
  constructor(to) { this.to = to; }
  send(msg) { return `generic message to ${this.to}: ${msg}`; }
}
class Email extends Notification { send(msg) { return `Email to ${this.to}: "${msg}" (with subject)`;
} }
class Sms extends Notification { send(msg) { return `SMS to ${this.to}: ${msg.slice(0, 20)}...`; } }
class Push extends Notification { send(msg) { return `Push to device ${this.to}:
${msg.toUpperCase()}`; } }

// unrelated class, same method name: duck typing
const slackBot = { to: "#general", send: (msg) => `Slack ${"#general"}: ${msg}` };

const channels = [new Email("asha@lbf.in"), new Sms("+91 98450 12345"), new Push("pixel-7"),
slackBot];
for (const ch of channels) console.log(ch.send("Your class starts at 10 am tomorrow"));
```

OUTPUT

```
Email to asha@lbf.in: "Your class starts at 10 am tomorrow" (with subject)
SMS to +91 98450 12345: Your class starts at...
Push to device pixel-7: YOUR CLASS STARTS AT 10 AM TOMORROW
Slack #general: Your class starts at 10 am tomorrow
```

13.5.4 Composition over inheritance and mixins

Deep inheritance trees become rigid: a `FlyingSwimmingRobotDog` cannot inherit from four parents, and changes to a base class ripple everywhere. **Composition** builds objects from small, independent pieces of behaviour instead ("has-a" rather than "is-a"). In JavaScript this is done by combining plain objects of functions with `Object.assign`, by holding other objects as properties, or by **mixins**: functions that take a class and return a subclass with extra methods, so several can be applied in a chain.

COMPOSITION AND MIXINS

main.js

```
// 1. Composition with behaviour objects
const canWalk = (self) => ({ walk: () => `${self.name} walks` });
const canSwim = (self) => ({ swim: () => `${self.name} swims` });
const canFly = (self) => ({ fly: () => `${self.name} flies` });

function createDuck(name) {
  const self = { name };
  return Object.assign(self, canWalk(self), canSwim(self), canFly(self));
}
function createFish(name) {
  const self = { name };
  return Object.assign(self, canSwim(self));
}
const duck = createDuck("Donald"), fish = createFish("Nemo");
console.log(duck.walk(), "|", duck.swim(), "|", duck.fly());
console.log(fish.swim(), "| fish can fly?", "fly" in fish);

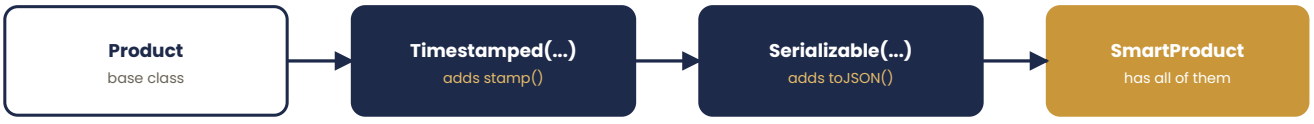
// 2. Class mixins: functions that extend any base class
const Serializable = (Base) => class extends Base {
  toJSON() { return { type: this.constructor.name, ...this }; }
};
const Timestamped = (Base) => class extends Base {
  stamp(date) { this.updatedAt = date; return this; }
};

class Product { constructor(name, price) { this.name = name; this.price = price; } }
class SmartProduct extends Serializable(Timestamped(Product)) {}

const p = new SmartProduct("Laptop", 54999).stamp("2026-03-09");
console.log(JSON.stringify(p));
console.log("still a Product:", p instanceof Product);
```

OUTPUT

```
Donald walks | Donald swims | Donald flies
Nemo swims | fish can fly? false
{"type":"SmartProduct","name":"Laptop","price":54999,"updatedAt":"2026-03-09"}
still a Product: true
```



MIXINS STACK BEHAVIOUR ONTO A BASE CLASS

TIP

Use inheritance for a genuine, stable "is-a" relationship with one or two levels (an `InsufficientFundsError` is an `Error`). Use composition for capabilities that can be mixed freely (can-fly, can-log, can-serialize). When unsure, prefer composition.

13.6 Property descriptors

13.6.1 Object.defineProperty(): writable, enumerable, configurable

Every property has hidden settings described by a **property descriptor**. A data property has a `value` and three flags: **writable** (can the value change?), **enumerable** (does it show in `for...in`, `Object.keys` and `JSON.stringify`?) and **configurable** (can the property be deleted or its flags changed?). An accessor property has `get` / `set` instead of value/writable. Properties created normally have all flags `true`; properties created with `Object.defineProperty` default every omitted flag to **false**.

INSPECTING AND SETTING PROPERTY FLAGS

main.js

```
const user = { name: "Asha" };
console.log(Object.getOwnPropertyDescriptor(user, "name"));

Object.defineProperty(user, "id", { value: 101, enumerable: true });    // not writable, not
configurable
Object.defineProperty(user, "secret", { value: "x9", writable: true });  // hidden from keys
Object.defineProperty(user, "initials", {                                // accessor property
  get() { return this.name.split(" ").map((w) => w[0]).join(""); },
  enumerable: false,
});

user.id = 999;                        // silently ignored (TypeError in strict mode)
delete user.id;                       // also ignored: not configurable
console.log("id:", user.id);
console.log("keys:", Object.keys(user), "| JSON:", JSON.stringify(user));
console.log("secret still readable:", user.secret, "| initials:", user.initials);
console.log(Object.getOwnPropertyDescriptor(user, "id"));
const strictCheck = (() => { "use strict"; try { user.id = 5; } catch (e) { return e.message; } })();
console.log("strict mode:", strictCheck);
```

OUTPUT

```
{ value: 'Asha', writable: true, enumerable: true, configurable: true }
id: 101
keys: [ 'name', 'id' ] | JSON: {"name":"Asha","id":101}
secret still readable: x9 | initials: A
{ value: 101, writable: false, enumerable: true, configurable: false }
strict mode: Cannot assign to read only property 'id' of object '#<Object>'
```

Flag	When false	Default with literal / defineProperty
`writable`	assignment is ignored (TypeError in strict mode)	true / false
`enumerable`	hidden from for...in, Object.keys, spread, JSON	true / false
`configurable`	cannot delete or redefine (except writable true to false)	true / false

Built-in methods use these flags: that is why `Array.prototype.map` does not show up in `for...in`, and why class methods are non-enumerable. `Object.defineProperties` sets several at once, and `Object.freeze`, `seal` and `preventExtensions` (Module 9) are shortcuts that change these flags for every property.

DEFINEPROPERTIES AND WHAT FREEZE DOES UNDERNEATH

main.js

```
const product = {};
Object.defineProperties(product, {
  sku: { value: "LBF-101", enumerable: true },
  price: { value: 499, writable: true, enumerable: true },
  _cost: { value: 300 },                                    // hidden internal value
  margin: { get() { return this.price - this._cost; }, enumerable: true },
});
product.price = 599;
product.sku = "HACK";
console.log(product, "| margin:", product.margin);

const frozen = Object.freeze({ a: 1 });
console.log(Object.getOwnPropertyDescriptor(frozen, "a"));
console.log("map enumerable on Array.prototype?", Object.getOwnPropertyDescriptor(Array.prototype,
"map").enumerable);
```

OUTPUT

```
{ sku: 'LBF-101', price: 599, margin: [Getter] } | margin: 299
{ value: 1, writable: false, enumerable: true, configurable: false }
map enumerable on Array.prototype? false
```

Constructor functions called with new create objects: new makes an object, links its [[Prototype]] to Constructor.prototype, runs the function with this bound to it and returns it. Property lookups follow the prototype chain up to Object.prototype and null; methods on the prototype are shared by all instances. Object.create links objects directly, and built-ins like Array.prototype work the same way (do not monkey-patch them). ES6 classes are syntax over this model, adding methods, getters/setters, static members, public fields, truly private #fields and methods, and static blocks. extends and super() set up inheritance; overriding with super.method() extends parent behaviour; built-ins such as Array, Map and Error can be subclassed; instanceof checks the prototype chain. Encapsulation (private fields, closures), abstraction (base classes, new.target), polymorphism (overriding, duck typing) and composition/mixins are the OOP principles in JavaScript. Property descriptors (writable, enumerable, configurable) control how each property behaves.

PRACTICE CHECKPOINT

Build a **BankAccount** class with a private balance, a **SavingsAccount** subclass, static helpers, and then rebuild the same design with **constructor functions and prototypes** to compare.

SAMPLE SOLUTION

main.js

```
// ---------- Version 1: ES6 classes ----------
class BankAccount {
  static #nextNumber = 1001;
  #balance = 0;
  constructor(owner, opening = 0) {
    this.owner = owner;
    this.number = BankAccount.#nextNumber++;
    if (opening) this.deposit(opening);
  }
  deposit(amount) {
    BankAccount.#check(amount);
    this.#balance += amount;
    return this;
  }
  withdraw(amount) {
```

```
    BankAccount.#check(amount);
    if (amount > this.#balance) throw new Error(`Insufficient funds in ${this.number}`);
    this.#balance -= amount;
    return this;
  }
  get balance() { return this.#balance; }
  toString() { return `${this.constructor.name} #${this.number} (${this.owner}): Rs
${this.#balance.toFixed(2)}`; }
  static #check(amount) { if (!(amount > 0)) throw new RangeError("Amount must be positive"); }
  static formatInr(n) { return new Intl.NumberFormat("en-IN", { style: "currency", currency: "INR"
}).format(n); }
  static transfer(from, to, amount) { from.withdraw(amount); to.deposit(amount); }
}

class SavingsAccount extends BankAccount {
  static MIN_BALANCE = 1000;
  constructor(owner, opening, rate) {
    super(owner, opening);
    this.rate = rate;
  }
  withdraw(amount) {
    if (this.balance - amount < SavingsAccount.MIN_BALANCE) throw new Error(`Minimum balance Rs
${SavingsAccount.MIN_BALANCE} required`);
    return super.withdraw(amount);
  }
  addInterest() { return this.deposit(+(this.balance * this.rate / 100).toFixed(2)); }
}

const cur = new BankAccount("Asha", 5000);
const sav = new SavingsAccount("Asha", 20000, 4);
BankAccount.transfer(cur, sav, 1500);
sav.addInterest();
console.log(String(cur));
console.log(String(sav));
console.log("formatted:", BankAccount.formatInr(sav.balance));
try { sav.withdraw(22000); } catch (e) { console.log("blocked:", e.message); }
console.log("private balance hidden:", Object.keys(sav));

// ---------- Version 2: constructor functions and prototypes ----------
function BankAccountFn(owner, opening = 0) {
  let balance = 0;                                  // privacy through a closure
  this.owner = owner;
  this.number = BankAccountFn.nextNumber++;
  this.getBalance = function () { return balance; };
  this._change = function (delta) { balance += delta; };
  if (opening) this.deposit(opening);
}
BankAccountFn.nextNumber = 2001;                    // "static" property
BankAccountFn.prototype.deposit = function (amount) {
  if (!(amount > 0)) throw new RangeError("Amount must be positive");
  this._change(amount);
  return this;
};
BankAccountFn.prototype.withdraw = function (amount) {
  if (amount > this.getBalance()) throw new Error(`Insufficient funds in ${this.number}`);
  this._change(-amount);
  return this;
};

function SavingsAccountFn(owner, opening, rate) {
  BankAccountFn.call(this, owner, opening);         // super(...)
  this.rate = rate;
}
SavingsAccountFn.prototype = Object.create(BankAccountFn.prototype);
```

```
SavingsAccountFn.prototype.constructor = SavingsAccountFn;
SavingsAccountFn.prototype.withdraw = function (amount) {
  if (this.getBalance() - amount < 1000) throw new Error("Minimum balance Rs 1000 required");
  return BankAccountFn.prototype.withdraw.call(this, amount);   // super.withdraw(...)
};
SavingsAccountFn.prototype.addInterest = function () {
  return this.deposit(+(this.getBalance() * this.rate / 100).toFixed(2));
};

const sav2 = new SavingsAccountFn("Asha", 21500, 4).addInterest();
console.log(`SavingsAccountFn #${sav2.number}: Rs ${sav2.getBalance().toFixed(2)}`);
try { sav2.withdraw(22000); } catch (e) { console.log("blocked:", e.message); }
console.log("instanceof parent:", sav2 instanceof BankAccountFn, "| keys:", Object.keys(sav2));
```

OUTPUT

```
BankAccount #1001 (Asha): Rs 3500.00
SavingsAccount #1002 (Asha): Rs 22360.00
formatted: ₹22,360.00
blocked: Minimum balance Rs 1000 required
private balance hidden: [ 'owner', 'number', 'rate' ]
SavingsAccountFn #2001: Rs 22360.00
blocked: Minimum balance Rs 1000 required
instanceof parent: true | keys: [ 'owner', 'number', 'getBalance', '_change', 'rate' ]
```

NOTE

Comparing the two versions: the class version is shorter, keeps everything in one block, and has real private state shared through prototype methods. The prototype version needs manual linking (`Object.create` , restoring `constructor` , `Parent.call(this)`) and its privacy through closures forces per-object functions such as `getBalance` , which also appear in `Object.keys` . Both produce the same prototype chain underneath.

MODULE 14 · INTERMEDIATE

Error handling and debugging

Minimum duration **1 day** | Level **Intermediate**

Errors are unavoidable: users type unexpected input, networks fail and every developer writes bugs. This module covers the built-in error types and the Error object, handling errors with try, catch and finally, throwing your own errors and custom error classes, a first look at errors in asynchronous code, practical debugging with console techniques, the debugger statement, Chrome DevTools and VS Code, and defensive programming habits that make bugs fail fast with clear messages.

IN THIS MODULE

14.1 Error types
14.2 Handling errors
14.3 Debugging
14.4 Defensive programming

14.1 Error types

When JavaScript cannot continue, it **throws** an error object. If nothing catches it, the current script stops and the error is reported in the console (with the file, line and call stack). Every built-in error is an instance of `Error` or one of its subclasses, and the class name tells you what kind of problem occurred.

14.1.1 SyntaxError, ReferenceError, TypeError, RangeError, URIError

A **SyntaxError** means the code is not valid JavaScript; it is detected when the file is parsed, so **none** of the file runs (it can also be thrown at run time by `JSON.parse` or `eval`). A **ReferenceError** means a name that does not exist (a typo, or a `let` in its TDZ). A **TypeError**, the most common, means an operation on the wrong kind of value: calling something that is not a function, or reading a property of `undefined` / `null`. A **RangeError** means a number outside the allowed range. A **URIError** comes from malformed URI encoding functions.

TRIGGERING EACH BUILT-IN ERROR TYPE

main.js

```
const attempts = [
  ["SyntaxError (JSON)", () => JSON.parse("{name: 'Asha'}")],
  ["ReferenceError", () => totl + 1],
  ["TypeError (not a function)", () => { const user = {}; user.greet(); }],
  ["TypeError (undefined)", () => { const order = undefined; return order.id; }],
  ["TypeError (const)", () => { const PI = 3.14; PI = 3; }],
  ["RangeError (array length)", () => new Array(-1)],
  ["RangeError (toFixed)", () => (1.5).toFixed(101)],
  ["RangeError (recursion)", () => { const f = () => f(); f(); }],
  ["URIError", () => decodeURIComponent("%")],
];

for (const [label, fn] of attempts) {
  try {
    fn();
  } catch (e) {
    console.log(`${label.padEnd(27)} -> ${e.constructor.name}: ${e.message}`);
  }
}
```

OUTPUT

```
SyntaxError (JSON)          -> SyntaxError: Expected property name or '}' in JSON at position 1 (line
1 column 2)
ReferenceError              -> ReferenceError: totl is not defined
TypeError (not a function)  -> TypeError: user.greet is not a function
TypeError (undefined)       -> TypeError: Cannot read properties of undefined (reading 'id')
TypeError (const)           -> TypeError: Assignment to constant variable.
RangeError (array length)   -> RangeError: Invalid array length
RangeError (toFixed)        -> RangeError: toFixed() digits argument must be between 0 and 100
RangeError (recursion)      -> RangeError: Maximum call stack size exceeded
URIError                    -> URIError: URI malformed
```

A SYNTAX ERROR STOPS THE WHOLE FILE

main.js

```
console.log("this line never runs");
let total = 10;
if (total > 5 {
  console.log("big");
}
```

ERROR

```
main.js:3
if (total > 5 {
             ^

SyntaxError: Unexpected token '{'
```

Even the first line does not print: the file is rejected before execution starts.

Error	Typical cause	Example
SyntaxError	invalid code, or invalid JSON passed to JSON.parse	`if (x > 5 {`
ReferenceError	undeclared or misspelled name; TDZ access	`totl + 1`
TypeError	wrong type: calling a non-function, property of undefined, assigning to const	`undefined.id`
RangeError	number out of range; too much recursion	`new Array(-1)`

Error	Typical cause	Example
URIError	malformed % escape in a URI function	`decodeURIComponent("%")`
EvalError	historical; not thrown by modern engines	-
AggregateError	several errors at once (Promise.any)	Module 17

14.1.2 The Error object: name, message, stack, cause

Every error object has a `name` (the type, such as "TypeError"), a `message` (the human description), and in all engines a non-standard but universal `stack` string showing the chain of calls that led to the error, newest first. Since ES2022, an error can also carry a `cause`: the original, lower-level error that led to it, passed with `new Error(message, { cause })`. This keeps the full story when an error is caught and re-thrown with a more meaningful message.

INSPECTING AN ERROR AND ITS CAUSE main.js

```
function loadConfig(text) {
  try {
    return JSON.parse(text);
  } catch (err) {
    throw new Error("Config file is not valid JSON", { cause: err });   // wrap, keep the original
  }
}

try {
  loadConfig("{ port: 3000 }");
} catch (e) {
  console.log("name:   ", e.name);
  console.log("message:", e.message);
  console.log("cause:  ", e.cause.name, "-", e.cause.message);
  console.log("stack (first 3 lines):");
  console.log(e.stack.split("\n").slice(0, 3).map((l) => "    " + l.trim().replace(/\(.*[\\/]/, "
(")).join("\n"));
}

const e = new RangeError("Too big");
console.log(e instanceof RangeError, e instanceof Error, String(e));
```

OUTPUT

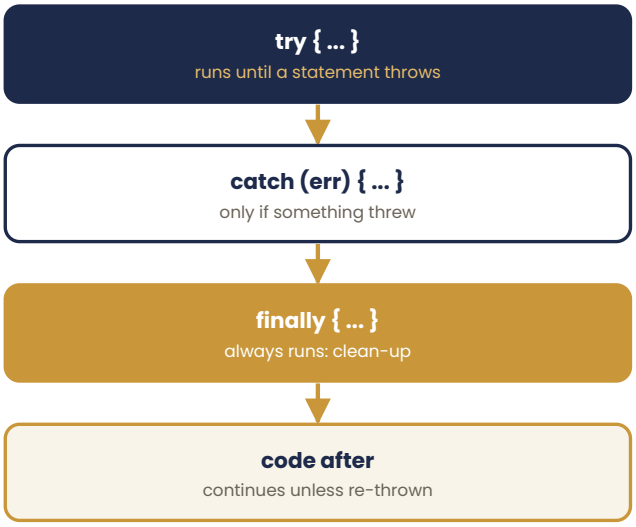
```
name:    Error
message: Config file is not valid JSON
cause:   SyntaxError - Expected property name or '}' in JSON at position 2 (line 1 column 3)
stack (first 3 lines):
   Error: Config file is not valid JSON
   at loadConfig (main.js:5:11)
   at Object.<anonymous> (main.js:10:3)
true true RangeError: Too big
```

14.2 Handling errors

14.2.1 try, catch, finally

Code that might fail goes in a `try` block. If any statement in it throws, the rest of the `try` block is skipped and control jumps to `catch (error)`, which receives the thrown value. The `finally` block runs **afterwards in every case**: after success, after a caught error, and even if `try` or `catch` executes `return`

or throws again. It is the place for clean-up such as hiding a loading spinner, closing a file or releasing a lock.



THE ORDER OF TRY, CATCH AND FINALLY

TRY, CATCH AND FINALLY IN ACTION

main.js

```
function divideMarks(total, students) {
  console.log(`dividing ${total} by ${students}`);
  try {
    if (students === 0) throw new RangeError("No students in the class");
    const avg = total / students;
    console.log("  average:", avg);
    return avg;
  } catch (err) {
    console.log("  caught:", err.message);
    return null;
  } finally {
    console.log("  finally: log closed");          // runs even after return
  }
}

console.log("result:", divideMarks(450, 5));
console.log("result:", divideMarks(450, 0));

// try...finally without catch: the error still propagates, but clean-up happens
function withSpinner(task) {
  console.log("spinner ON");
  try { return task(); } finally { console.log("spinner OFF"); }
}
try {
  withSpinner(() => { throw new Error("upload failed"); });
} catch (e) {
  console.log("outer caught:", e.message);
}
```

OUTPUT

```
dividing 450 by 5
  average: 90
  finally: log closed
result: 90
dividing 450 by 0
  caught: No students in the class
  finally: log closed
result: null
spinner ON
spinner OFF
outer caught: upload failed
```

Only catch errors you can actually handle. Catching everything and ignoring it (`catch (e) {}`) hides bugs. A good pattern is to handle the errors you expect and **re-throw** anything else so it is not swallowed:

HANDLE WHAT YOU EXPECT, RE-THROW THE REST main.js

```
function parseAge(text) {
  try {
    const data = JSON.parse(text);
    return data.age.toFixed(0);                    // may throw TypeError if age missing
  } catch (err) {
    if (err instanceof SyntaxError) {              // expected: bad input
      return "invalid JSON";
    }
    throw err;                                     // unexpected: let it surface
  }
}

console.log(parseAge('{"age": 21.6}'));
console.log(parseAge("not json"));
try {
  parseAge('{"name": "Asha"}');
} catch (e) {
  console.log("re-thrown to caller:", e.constructor.name, "-", e.message);
}
```

OUTPUT

```
22
invalid JSON
re-thrown to caller: TypeError - Cannot read properties of undefined (reading 'toFixed')
```

WATCH OUT

`try...catch` only catches errors thrown **synchronously** inside it. An error thrown later inside a `setTimeout` callback or an un-awaited promise escapes it (14.2.5 and Module 17).

14.2.2 Optional catch binding

Since ES2019, the `(error)` after `catch` can be omitted when the error object is not needed, for example when any failure simply means "use a default".

CATCH WITHOUT A PARAMETER main.js

```
function readSetting(json, key, fallback) {
  try {
    return JSON.parse(json)[key] ?? fallback;
  } catch {                          // no (err): we only care that it failed
    return fallback;
  }
}
console.log(readSetting('{"theme":"dark"}', "theme", "light"));
console.log(readSetting("corrupted!!", "theme", "light"));

function isValidUrl(text) {
  try { new URL(text); return true; } catch { return false; }
}
console.log(isValidUrl("https://learnbeingforward.in"), isValidUrl("not a url"));
```

OUTPUT

```
dark
light
true false
```

14.2.3 throw

`throw value` stops the current function immediately and starts searching up the call stack for the nearest enclosing `catch`. JavaScript lets you throw any value, but always throw an **Error object** (or a subclass): only Error objects record a stack trace and have the standard `name` and `message` that tools and other developers expect.

THROWING MEANINGFUL ERRORS main.js

```
function withdraw(balance, amount) {
  if (typeof amount !== "number" || Number.isNaN(amount)) throw new TypeError(`Amount must be a
number, got ${typeof amount}`);
  if (amount <= 0) throw new RangeError("Amount must be greater than zero");
  if (amount > balance) throw new Error(`Insufficient balance: ${balance} < ${amount}`);
  return balance - amount;
}

for (const amt of [300, "300", -5, 5000]) {
  try {
    console.log(`withdraw ${JSON.stringify(amt)} -> new balance`, withdraw(1000, amt));
  } catch (e) {
    console.log(`withdraw ${JSON.stringify(amt)} -> ${e.name}: ${e.message}`);
  }
}

// throwing a string works, but loses the stack and the name
try { throw "something bad"; } catch (e) { console.log("thrown string:", typeof e, "| stack:",
e.stack); }
```

OUTPUT

```
withdraw 300 -> new balance 700
withdraw "300" -> TypeError: Amount must be a number, got string
withdraw -5 -> RangeError: Amount must be greater than zero
withdraw 5000 -> Error: Insufficient balance: 1000 < 5000
thrown string: string | stack: undefined
```

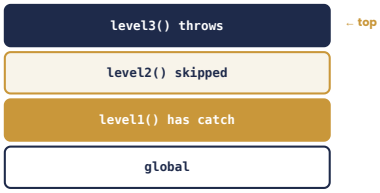
An error travels up the call stack until something catches it; every function in between stops immediately. This is why errors can be handled at a sensible, higher level (for example, one place that shows a message to the user) instead of in every function.

ERRORS PROPAGATE UP THE CALL STACK main.js

```
function level3() { console.log("  level3 throws"); throw new Error("disk full"); }
function level2() { level3(); console.log("  level2 never continues"); }
function level1() {
  try { level2(); }
  catch (e) { console.log("  level1 caught:", e.message); }
}
level1();
console.log("program continues");
```

OUTPUT

```
  level3 throws
  level1 caught: disk full
program continues
```



THE ERROR UNWINDS LEVEL3 AND LEVEL2 UNTIL LEVEL1 CATCHES IT

14.2.4 Custom error classes

Extending `Error` (Module 13) lets you create your own error types, which callers can tell apart with `instanceof` and which can carry extra data such as the field that failed or an HTTP status code. Set `this.name` (or use `this.constructor.name` in a base class) so that logs show the right type. A small hierarchy with a common base class lets callers catch a whole family of errors at once.

A SMALL HIERARCHY OF CUSTOM ERRORS main.js

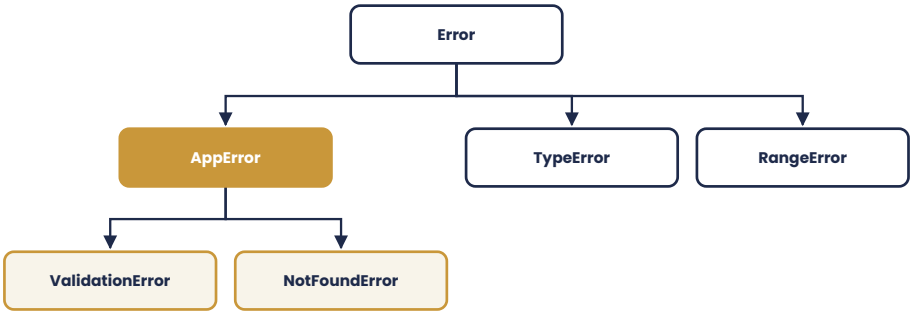
```
class AppError extends Error {
  constructor(message, options) {
    super(message, options);
    this.name = this.constructor.name;        // every subclass gets its own name
  }
}
class ValidationError extends AppError {
  constructor(field, message) {
    super(`${field}: ${message}`);
    this.field = field;
  }
}
class NotFoundError extends AppError {
  constructor(resource, id) {
    super(`${resource} ${id} not found`);
    this.status = 404;
  }
}

const users = new Map([[1, "Asha"]]);
function getUser(id) {
  if (!Number.isInteger(id)) throw new ValidationError("id", "must be an integer");
  if (!users.has(id)) throw new NotFoundError("User", id);
  return users.get(id);
}

for (const id of [1, "x", 7]) {
  try {
    console.log("found:", getUser(id));
  } catch (e) {
    if (e instanceof ValidationError) console.log(`400 Bad Request (${e.name}, field "${e.field}"):
${e.message}`);
    else if (e instanceof NotFoundError) console.log(`${e.status} (${e.name}): ${e.message}`);
    else throw e;
    console.log("   is an AppError:", e instanceof AppError);
  }
}
```

OUTPUT

```
found: Asha
400 Bad Request (ValidationError, field "id"): id: must be an integer
   is an AppError: true
404 (NotFoundError): User 7 not found
   is an AppError: true
```



CUSTOM ERRORS FIT INTO THE BUILT-IN HIERARCHY

14.2.5 Error handling in async code (detailed in Module 17)

Asynchronous code needs its own techniques, covered fully in Module 17. In brief: errors inside callbacks such as `setTimeout` must be caught **inside** the callback; a rejected promise is handled with `.catch()`; and inside an `async` function, `await` turns a rejection back into a normal thrown error, so ordinary `try...catch` works again.

THREE ASYNC ERROR PATTERNS

main.js

```
// 1. try/catch around setTimeout does NOT catch the callback's error
try {
  setTimeout(() => {
    try { null.length; }                                  // must catch in here
    catch (e) { console.log("1. caught inside the callback:", e.constructor.name); }
  }, 0);
} catch {
  console.log("never printed");
}

// 2. promises: .catch()
fetchScore("bad").then((s) => console.log(s)).catch((e) => console.log("2. .catch():", e.message));

// 3. async/await: plain try/catch works again
async function showScore(id) {
  try {
    const s = await fetchScore(id);
    console.log("3. score:", s);
  } catch (e) {
    console.log("3. try/catch with await:", e.message);
  } finally {
    console.log("3. finally for", id);
  }
}
showScore("ok");
showScore("bad");

function fetchScore(id) {
  return new Promise((resolve, reject) =>
    setTimeout(() => (id === "ok" ? resolve(91) : reject(new Error(`no score for ${id}`))), 10));
}
```

OUTPUT

```
1. caught inside the callback: TypeError
2. .catch(): no score for bad
3. score: 91
3. finally for ok
3. try/catch with await: no score for bad
3. finally for bad
```

LAST-RESORT GLOBAL HANDLERS IN THE BROWSER

main.js

```
window.addEventListener("error", (e) => console.log("global error handler:", e.message));
window.addEventListener("unhandledrejection", (e) => {
  console.log("unhandled rejection:", e.reason.message);
  e.preventDefault();                    // we logged it ourselves
});
setTimeout(() => { throw new Error("thrown in a timer"); }, 0);
Promise.reject(new Error("nobody caught this promise"));
```

BROWSER CONSOLE

```
global error handler: Uncaught Error: thrown in a timer
Uncaught Error: thrown in a timer
unhandled rejection: nobody caught this promise
```

NOTE

Global handlers (`window.onerror` , `unhandledrejection` , and in Node.js `process.on("uncaughtException")`) are a safety net for logging and reporting (for example to a monitoring service), not a replacement for handling errors where they occur.

14.3 Debugging

Debugging is the process of finding out why code does not behave as expected. A reliable method beats random changes: (1) reproduce the bug consistently; (2) read the error message and stack trace carefully: they name the type, message, file and line; (3) form a hypothesis about the cause; (4) inspect values with console output or breakpoints to confirm or reject it; (5) fix the cause, not the symptom; (6) re-test, ideally with an automated test (Module 24).



A SYSTEMATIC DEBUGGING LOOP

14.3.1 console techniques

Beyond `console.log` (Module 2), a few habits make logging much more useful. Log **labelled objects** (`console.log({ total, items })`) so you can see the variable names. Use `console.table` for arrays of records, `console.group` to nest output, `console.count` to see how often a line runs, `console.time` to measure, `console.assert` to print only when a condition fails, and `console.trace` to print the call stack at that point.

USEFUL CONSOLE TECHNIQUES

main.js

```
const cart = [{ item: "pen", price: 10, qty: 3 }, { item: "book", price: 250, qty: 1 }];
let total = 0;

console.group("calculating total");
for (const line of cart) {
  const lineTotal = line.price * line.qty;
  console.log({ item: line.item, lineTotal });        // labelled values
  total += lineTotal;
}
console.groupEnd();
console.log({ total });

console.table(cart);

function render() { console.count("render called"); }
render(); render(); render();

console.assert(total === 280, "total should be 280");        // silent: passes
console.assert(cart.length === 3, "expected 3 lines, got", cart.length);

function inner() { console.trace("how did we get here?"); }
function outer() { inner(); }
outer();
```

OUTPUT

```
calculating total
  { item: 'pen', lineTotal: 30 }
  { item: 'book', lineTotal: 250 }
{ total: 280 }
┌─────────┬────────┬───────┬─────┐
│ (index) │ item   │ price │ qty │
├─────────┼────────┼───────┼─────┤
│ 0       │ 'pen'  │ 10    │ 3   │
│ 1       │ 'book' │ 250   │ 1   │
└─────────┴────────┴───────┴─────┘
render called: 1
render called: 2
render called: 3
Assertion failed: expected 3 lines, got 2
Trace: how did we get here?
    at inner (main.js:21:28)
    at outer (main.js:22:20)
    at Object.<anonymous> (main.js:23:1)
```

NOTE

`console.assert`, `console.trace` and `console.error` write to the error stream (stderr) in Node.js; in the browser they appear with a red or expandable marker. The trace lines above are shortened by Node.js to the relevant frames.

14.3.2 The debugger statement

The `debugger;` statement acts as a breakpoint written in code: when the browser DevTools (or a Node.js debugger) are open, execution pauses on that line so you can inspect every variable in scope. When no debugger is attached, the statement does nothing. Remove it before committing code; linters (Module 19) flag it.

PAUSING WITH DEBUGGER

main.js

```
function applyDiscount(cart, code) {
  const rate = code === "SAVE10" ? 0.1 : 0;
  debugger;                        // pauses here when DevTools is open
  return cart.total * (1 - rate);
}
applyDiscount({ total: 500 }, "save10");
```

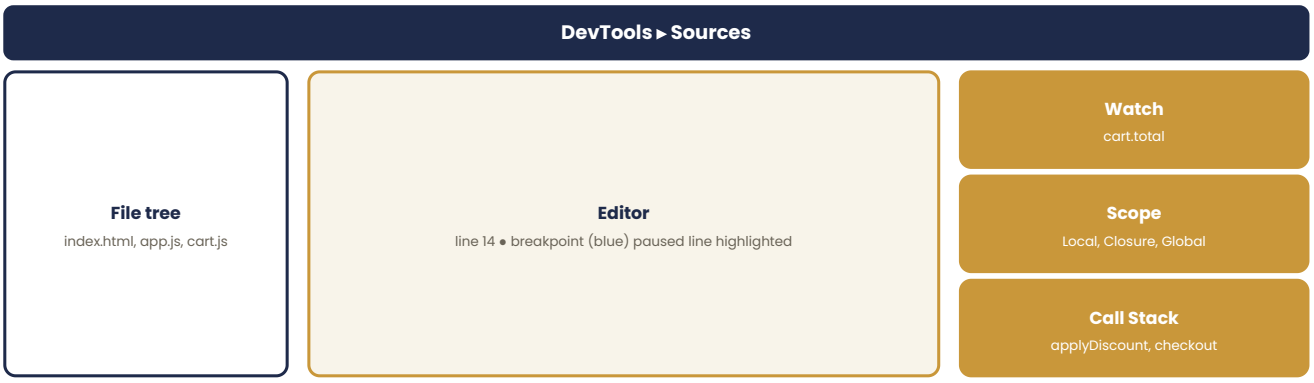
WHAT YOU SEE

```
Paused on debugger statement
Scope ▸ Local:  cart: {total: 500}   code: "save10"   rate: 0
-> the bug is visible at once: the code is lower-case, so rate is 0
```

14.3.3 Chrome DevTools Sources panel: breakpoints, stepping, watch, call stack

Open DevTools with **F12** (or **Ctrl+Shift+I**, **Cmd+Option+I** on Mac) and choose the **Sources** panel. Open your file from the file tree (or press Ctrl+P), then click a line number to set a **breakpoint**. When the code reaches it, execution pauses and the right-hand pane shows everything you need to understand the program at that moment.

Tool	What it does
Line breakpoint	click a line number; execution pauses before that line runs
Conditional breakpoint	right-click a line number: pause only when an expression is true (e.g. `i === 57`)
Logpoint	right-click: print a message without editing the code or pausing
DOM / event / XHR breakpoints	pause when an element changes, an event fires or a request is sent
Resume (F8)	continue until the next breakpoint
Step over (F10)	run the current line, including any function it calls, and stop on the next line
Step into (F11)	go inside the function called on this line
Step out (Shift+F11)	finish the current function and stop in its caller
Scope pane	local, closure and global variables with their current values
Watch pane	expressions re-evaluated at every pause (e.g. `cart.items.length`)
Call Stack pane	the chain of calls that led here; click a frame to inspect its variables
Console while paused	evaluate any expression in the paused scope, or even change variables



THE LAYOUT OF THE SOURCES PANEL WHILE PAUSED

A breakpoint session for a typical bug, a cart total that is wrong only sometimes, could go: set a breakpoint inside the loop that adds line totals; add `line.price` and `total` to Watch; step with F10 through each iteration; notice that one price is the string `"250"`, so `total + price` concatenates instead of adding; fix the data conversion where the price is read from the form (`Number(input.value)`); remove the breakpoint and re-test.

THE KIND OF BUG STEPPING EXPOSES

main.js

```
// The bug a breakpoint would reveal: one price came from a form field as a string
const lines = [{ price: 10, qty: 3 }, { price: "250", qty: 1 }, { price: 99, qty: 2 }];
let total = 0;
for (const line of lines) total += line.qty === 1 ? line.price : line.price * line.qty;
console.log("buggy total:", total);

// Fixed at the source: convert once when reading input
const fixed = lines.map((l) => ({ ...l, price: Number(l.price) }));
console.log("fixed total:", fixed.reduce((t, l) => t + l.price * l.qty, 0));
```

BROWSER CONSOLE

```
buggy total: 30250198
fixed total: 478
```

14.3.4 Debugging Node.js in VS Code

VS Code has a built-in debugger for Node.js. The quickest way: open a **JavaScript Debug Terminal** (Command Palette, Ctrl+Shift+P, then "Debug: JavaScript Debug Terminal") and run `node app.js` in it; breakpoints set by clicking in the editor gutter are hit automatically. For repeatable set-ups, create a `.vscode/launch.json` through the Run and Debug view (Ctrl+Shift+D). The same panes as in Chrome appear: Variables, Watch, Call Stack and Breakpoints, with F5 to continue and F10/F11 to step. From the command line, `node --inspect-brk app.js` lets Chrome attach via `chrome://inspect`.

.VSCODE/LAUNCH.JSON

json

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "node",
      "request": "launch",
      "name": "Debug app.js",
      "program": "${workspaceFolder}/app.js",
      "skipFiles": ["<node_internals>/**"]
    }
  ]
}
```

TIP

Turn on "Pause on caught/uncaught exceptions" in the Breakpoints pane (Chrome and VS Code). The debugger then stops exactly where an error is thrown, with all variables still available, which is often the fastest way to find a crash.

14.4 Defensive programming

14.4.1 Validating input

Data that comes from outside your code (form fields, URL parameters, API responses, files, `localStorage`) cannot be trusted: it may be missing, of the wrong type, out of range or malicious. Validate it at the **boundary**, where it enters your program, and convert it to the right types there; the rest of the code can then rely on clean data. Remember that browser-side validation is for user experience only; a server must validate again (Module 24).

VALIDATING AND NORMALISING INPUT AT THE BOUNDARY

main.js

```
function validateRegistration(input) {
  const errors = [];
  const name = String(input.name ?? "").trim();
  const age = Number(input.age);
  const email = String(input.email ?? "").trim().toLowerCase();

  if (name.length < 2) errors.push("name must have at least 2 characters");
  if (!Number.isInteger(age) || age < 16 || age > 100) errors.push("age must be a whole number between
16 and 100");
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) errors.push("email is not valid");

  return errors.length ? { ok: false, errors } : { ok: true, value: { name, age, email } };
}

console.log(validateRegistration({ name: "  Asha ", age: "21", email: "ASHA@LBF.IN" }));
console.log(validateRegistration({ name: "A", age: "twenty", email: "asha@" }));
console.log(validateRegistration({}));
```

OUTPUT

```
{ ok: true, value: { name: 'Asha', age: 21, email: 'asha@lbf.in' } }
{
  ok: false,
  errors: [
    'name must have at least 2 characters',
    'age must be a whole number between 16 and 100',
    'email is not valid'
  ]
}
{
  ok: false,
  errors: [
    'name must have at least 2 characters',
    'age must be a whole number between 16 and 100',
    'email is not valid'
  ]
}
```

14.4.2 Fail fast and meaningful error messages

Fail fast means detecting a problem as early as possible and stopping with a clear error, instead of letting a bad value travel through the program and cause a confusing failure far away (the classic "Cannot read properties of undefined" ten functions later). Check preconditions at the top of a function with **guard clauses**. A good error message says **what** was wrong, **which** value caused it, and ideally **what was expected**: "quantity must be a positive integer, got -2" is far more useful than "invalid input".

GUARD CLAUSES AND CLEAR MESSAGES

main.js

```
// Without fail-fast: the problem surfaces far from its cause
function priceWithTaxBad(item) { return item.price * 1.18; }
console.log("bad:", priceWithTaxBad({ cost: 100 }));        // NaN, silently

// With guard clauses and meaningful messages
function priceWithTax(item, rate = 0.18) {
  if (item == null) throw new TypeError("priceWithTax: item is required");
  if (typeof item.price !== "number" || !Number.isFinite(item.price))
    throw new TypeError(`priceWithTax: item.price must be a finite number, got
${JSON.stringify(item.price)}`);
  if (item.price < 0) throw new RangeError(`priceWithTax: price cannot be negative, got
${item.price}`);
  if (rate < 0 || rate > 1) throw new RangeError(`priceWithTax: rate must be between 0 and 1, got
${rate}`);
  return Math.round(item.price * (1 + rate) * 100) / 100;
}

console.log("good:", priceWithTax({ price: 100 }));
for (const bad of [null, { cost: 100 }, { price: -5 }]) {
  try { priceWithTax(bad); } catch (e) { console.log(`${e.name}: ${e.message}`); }
}

// a tiny assert helper used during development
function assert(condition, message) { if (!condition) throw new Error(`Assertion failed: ${message}`);
}
try { assert([1, 2].length === 3, "expected 3 items"); } catch (e) { console.log(e.message); }
```

OUTPUT

```
bad: NaN
good: 118
TypeError: priceWithTax: item is required
TypeError: priceWithTax: item.price must be a finite number, got undefined
RangeError: priceWithTax: price cannot be negative, got -5
Assertion failed: expected 3 items
```

Validate at boundaries

Check and convert external data where it enters; trust it inside.

Guard clauses

Check preconditions first and return or throw early, keeping the main path un-nested.

Specific errors

Use TypeError, RangeError or custom classes, with messages that include the bad value.

Never swallow

An empty catch hides bugs; log, handle or re-throw.

Use defaults wisely

?? for missing values, but do not hide invalid ones.

Freeze constants

Object.freeze configuration so it cannot be changed by accident.

Built-in errors tell you the kind of problem: SyntaxError (invalid code or JSON), ReferenceError (unknown name), TypeError (wrong kind of value), RangeError (out of range) and URIError. Error objects have name, message, stack and an optional cause for wrapping. try runs risky code, catch handles a thrown error (the binding is optional), finally always runs for clean-up; handle what you expect and re-throw the rest. throw stops execution and unwinds the call stack to the nearest catch; always throw Error objects, and extend Error for custom types with extra data. Async errors need callbacks to catch internally, .catch() for promises or try/catch around await. Debug systematically with labelled console output, console.table/count/assert/trace, the debugger statement, DevTools breakpoints and stepping, and the VS Code debugger. Defensive code validates input at the boundary and fails fast with specific, meaningful messages.

PRACTICE CHECKPOINT

Build a **form-input validator** that throws custom **ValidationError**s, and fix a **buggy program** using what DevTools breakpoints would reveal.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<form id="signup">
  <input name="name" placeholder="Name">
  <input name="email" placeholder="Email">
  <input name="age" placeholder="Age">
  <button>Register</button>
</form>
<p id="messages"></p>
```

SAMPLE SOLUTION main.js

```
class ValidationError extends Error {
  constructor(field, message) {
    super(message);
    this.name = "ValidationError";
    this.field = field;
  }
}

const rules = {
  name: (v) => { if (v.trim().length < 2) throw new ValidationError("name", "Name needs at least 2
letters"); },
  email: (v) => { if (!/^[^\s@]+@[^\s@]+\.[a-z]{2,}$/i.test(v)) throw new ValidationError("email",
`"${v}" is not a valid email`); },
  age: (v) => {
    const n = Number(v);
    if (!Number.isInteger(n)) throw new ValidationError("age", "Age must be a whole number");
    if (n < 16) throw new ValidationError("age", `Minimum age is 16, got ${n}`);
  },
};

function validateForm(form) {
  const problems = [];
  for (const [field, check] of Object.entries(rules)) {
    try {
      check(form.elements[field].value);
      form.elements[field].classList.remove("invalid");
    } catch (e) {
      if (!(e instanceof ValidationError)) throw e;        // a real bug: do not hide it
      form.elements[field].classList.add("invalid");
      problems.push(e);
```

```
    }
  }
  return problems;
}

const form = document.querySelector("#signup");
form.addEventListener("submit", (event) => {
  event.preventDefault();
  const problems = validateForm(form);
  const out = document.querySelector("#messages");
  out.textContent = problems.length ? problems.map((p) => `${p.field}: ${p.message}`).join(" | ") :
"Registered!";
  console.log(problems.length ? `${problems.length} error(s): ` + problems.map((p) => p.field).join(",
") : "form OK");
});

// Fixing the bug: average marks came out as "NaN" for some classes.
// A breakpoint inside average() with marks in Watch showed the empty-array case (0 / 0).
function averageBuggy(marks) { return marks.reduce((t, m) => t + m, 0) / marks.length; }
function average(marks) {
  if (!Array.isArray(marks)) throw new TypeError("average: marks must be an array");
  if (marks.length === 0) return null;                    // the fix: handle the empty class
  return marks.reduce((t, m) => t + m, 0) / marks.length;
}
console.log("buggy:", averageBuggy([]), "| fixed:", average([]), average([80, 90]));
```

BROWSER CONSOLE

```
buggy: NaN | fixed: null 85
3 error(s): name, email, age
form OK
--- page shows ---
   Register

Registered!
```

Simulated user actions, in order: types "A" into `[name=name]`; types "asha@lbf" into `[name=email]`; types "15" into `[name=age]`; clicks `button`; types "Asha" into `[name=name]`; types "asha@lbf.in" into `[name=email]`; types "21" into `[name=age]`; clicks `button`.

MODULE 15 · INTERMEDIATE

The DOM (Document Object Model)

Minimum duration **4 days** Level **Intermediate**

The DOM is the browser's live, tree-shaped representation of a web page, and the bridge between JavaScript and what the user sees. This module explains the DOM tree, the window and document objects, and nodes versus elements; then covers selecting and traversing elements, changing their content (and the XSS risk of innerHTML), attributes, data attributes, styles and classes, creating and removing elements efficiently, working with forms and the Constraint Validation API, the Browser Object Model, and how to keep DOM updates fast.

IN THIS MODULE

15.1 DOM basics
15.2 Selecting elements
15.3 Traversing the DOM
15.4 Changing content
15.5 Changing attributes and styles
15.6 Creating and removing elements
15.7 Forms
15.8 Browser Object Model (BOM)
15.9 DOM performance

NOTE

Every example in this module really runs in a browser (headless Chromium). The HTML shown above each example is the page the script runs in; "page shows" lists the visible text of the page after the script has finished, and "HTML" shows the resulting markup of an element.

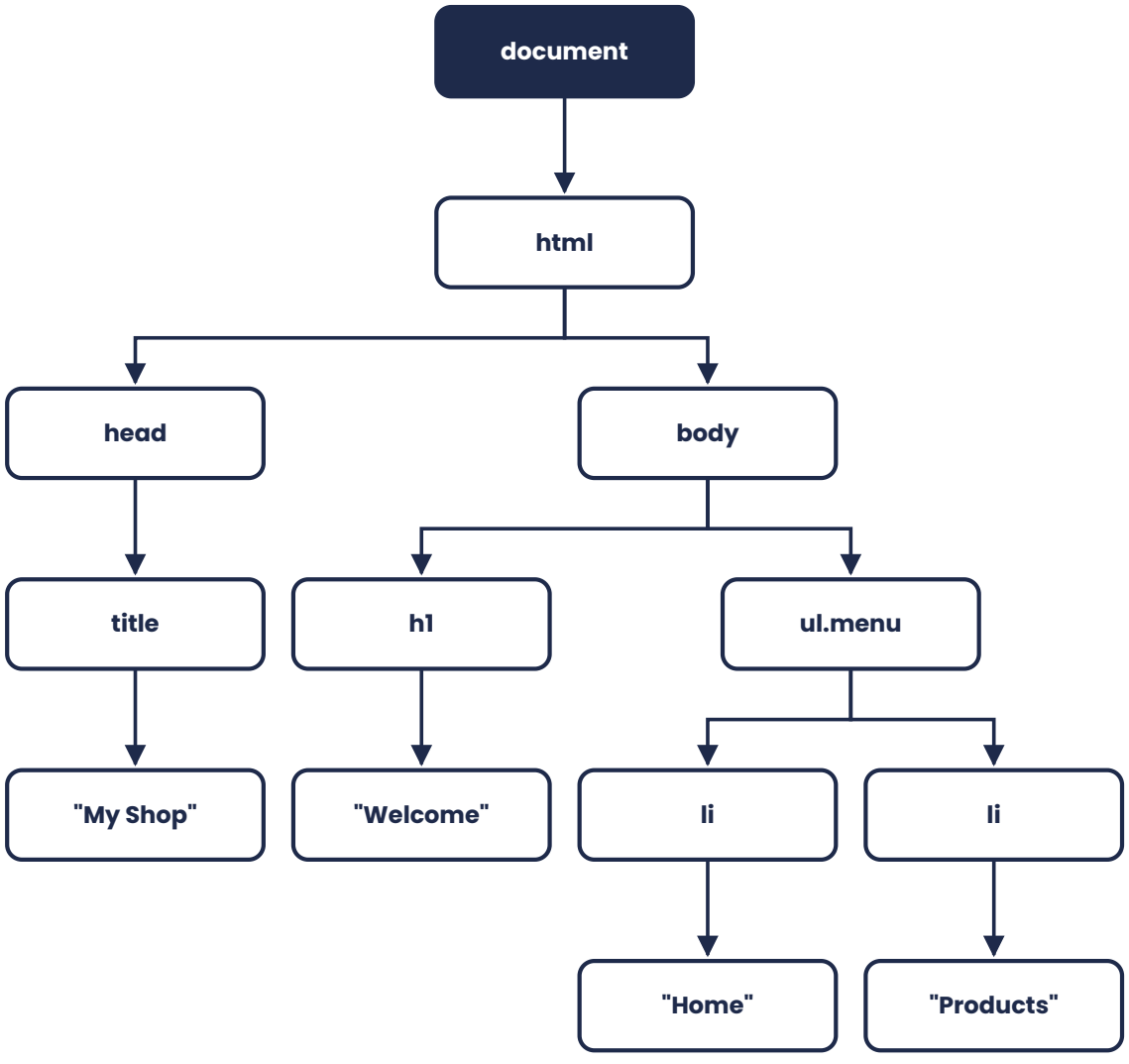
15.1 DOM basics

15.1.1 What the DOM is and the DOM tree

When a browser loads an HTML page, it parses the text and builds a tree of objects in memory: the **Document Object Model**. Each tag becomes an **element** object, the text inside tags becomes **text nodes**, and nesting becomes parent-child relationships. The page you see is drawn from this tree, not from the HTML file. When JavaScript changes the tree (adds an element, changes text, sets a class), the browser updates the screen automatically. The DOM is a standard API defined by WHATWG, so the same code works in every browser.

A SMALL PAGE HTML

```
<!doctype html>
<html>
  <head>
    <title>My Shop</title>
  </head>
  <body>
    <h1 id="title">Welcome</h1>
    <ul class="menu">
      <li>Home</li>
      <li>Products</li>
    </ul>
  </body>
</html>
```



THE DOM TREE BUILT FROM IT (TEXT NODES IN LIGHT BOXES)

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<h1 id="title">Welcome</h1>
<ul class="menu">
  <li>Home</li>
  <li>Products</li>
</ul>
```

EXPLORING THE TREE FROM JAVASCRIPT main.js

```
console.log("document.title:", document.title);
console.log("root element:", document.documentElement.tagName);
console.log("body children:", [...document.body.children].map((el) => el.tagName));
const ul = document.querySelector("ul");
console.log("ul has", ul.children.length, "li elements and", ul.childNodes.length, "child nodes in
total");
console.log("first li text:", ul.firstElementChild.textContent);
```

BROWSER CONSOLE

```
document.title: Demo
root element: HTML
body children: [H1, UL, SCRIPT]
ul has 2 li elements and 5 child nodes in total
first li text: Home
```

The extra child nodes counted above are **whitespace text nodes**: the line breaks and spaces between the `<li>` tags in the HTML. They are why the element-only properties (`children`, `firstElementChild`) are usually what you want (15.1.3).

15.1.2 window and document objects

In a browser, `window` is the global object: it represents the browser tab and holds global variables, timers (`setTimeout`), dialogs (`alert`), `location`, `history`, `localStorage` and much more (15.8). `window.document`, usually written simply `document`, is the entry point to the DOM tree of the page loaded in that window, with properties for its main parts and methods for finding and creating elements.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<script>var score = 91;</script><p>Hello</p>
```

WINDOW AND DOCUMENT main.js

```
console.log("window is the global object:", window === globalThis);
console.log("document belongs to window:", window.document === document);
console.log("var becomes a window property:", (function () { return window.score; })());
console.log("document.head / body:", document.head.tagName, document.body.tagName);
console.log("document.URL:", document.URL);
console.log("readyState:", document.readyState);
console.log("viewport (innerWidth x innerHeight):", window.innerWidth, "x", window.innerHeight);
```

BROWSER CONSOLE

```
window is the global object: true
document belongs to window: true
var becomes a window property: 91
document.head / body: HEAD BODY
document.URL: http://lbf.local/index.html
readyState: loading
viewport (innerWidth x innerHeight): 800 x 600
```

document property / method	Gives you
`document.documentElement`	the <html> element
`document.head`, `document.body`	the <head> and <body> elements
`document.title`	the page title (readable and writable)

document property / method	Gives you
`document.forms`, `images`, `links`	collections of those elements
`document.URL`, `readyState`	the address and loading state (loading, interactive, complete)
`getElementById`, `querySelector`…	finding elements (15.2)
`createElement`, `createTextNode`	making new nodes (15.6)

NOTE

Your script can only find elements that already exist when it runs. Put `<script>` tags at the end of `<body>`, or (better) in `<head>` with the `defer` attribute, or as `type="module"` (deferred automatically). Module 16 covers the DOMContentLoaded event.

15.1.3 Nodes vs elements

Everything in the tree is a **node** (the `Node` interface): the document, elements, text, comments. An **element** is the kind of node that represents a tag (the `Element` interface, with HTML-specific subclasses such as `HTMLInputElement`). Each node has a `nodeType` number and a `nodeName`. Many DOM properties come in two flavours: one that includes all nodes (`childNodes`, `firstChild`, `nextSibling`) and one that includes only elements (`children`, `firstElementChild`, `nextElementSibling`).

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

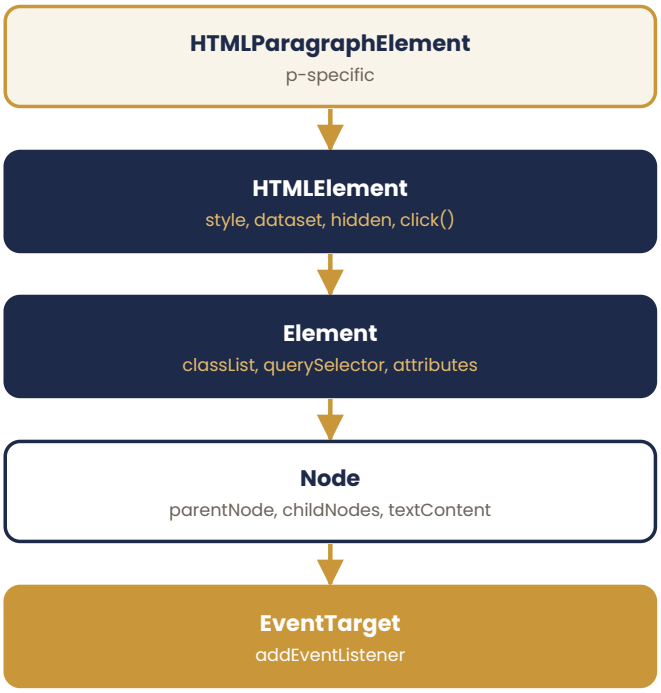
```html
<div id="box">Intro text<!-- a comment --><p>Paragraph</p><span>Note</span></div>
```

DIFFERENT KINDS OF NODES main.js

```
const box = document.querySelector("#box");
for (const node of box.childNodes) {
  const kind = { 1: "ELEMENT", 3: "TEXT", 8: "COMMENT" }[node.nodeType];
  console.log(`${kind.padEnd(8)} nodeName=${node.nodeName.padEnd(9)}
${JSON.stringify(node.textContent)}`);
}
console.log("children (elements only):", [...box.children].map((e) => e.tagName));
const p = box.querySelector("p");
console.log("p is a Node:", p instanceof Node, "| Element:", p instanceof Element, "|
HTMLParagraphElement:", p instanceof HTMLParagraphElement);
console.log("prototype chain:", (function () { const names = []; for (let o =
Object.getPrototypeOf(p); o; o = Object.getPrototypeOf(o)) names.push(o.constructor.name); return
names.join(" -> "); })());
```

BROWSER CONSOLE

```
TEXT     nodeName=#text     "Intro text"
COMMENT  nodeName=#comment  " a comment "
ELEMENT  nodeName=P         "Paragraph"
ELEMENT  nodeName=SPAN      "Note"
children (elements only): [P, SPAN]
p is a Node: true | Element: true | HTMLParagraphElement: true
prototype chain: HTMLParagraphElement -> HTMLElement -> Element -> Node -> EventTarget -> Object
```



DOM OBJECTS USE PROTOTYPE INHERITANCE (MODULE 13)

15.2 Selecting elements

15.2.1 getElementById()

`document.getElementById("id")` returns the single element with that `id`, or `null` if there is none. Pass the id **without** a `#`. It is the fastest lookup and ids should be unique in a page. Always consider the `null` case: calling a method on the result when the element is missing causes a TypeError.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) — HTML

```html
<h1 id="title">Welcome</h1>
```

FINDING ONE ELEMENT BY ID — main.js

```js
const title = document.getElementById("title");
console.log(title.tagName, "-", title.textContent);
title.textContent = "Welcome back, Asha!";

const missing = document.getElementById("doesNotExist");
console.log("missing element:", missing);
console.log("safe access:", missing?.textContent ?? "(no element)");
```

BROWSER CONSOLE

```
H1 - Welcome
missing element: null
safe access: (no element)
--- page shows ---
Welcome back, Asha!
```

15.2.2 getElementsByClassName(), getElementsByTagName()

These older methods return an **HTMLCollection** of all matching elements (possibly empty, never `null`). They can be called on `document` or on any element to search only inside it. `getElementsByClassName`

accepts several space-separated classes, matching elements that have all of them.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<ul id="cart">
  <li class="item">Pen</li>
  <li class="item sale">Book</li>
  <li class="item sale">Bag</li>
</ul>
<ul><li>Other list</li></ul>
```

COLLECTIONS BY CLASS AND TAG main.js

```
const items = document.getElementsByClassName("item");
console.log("items:", items.length, [...items].map((li) => li.textContent));

const sale = document.getElementsByClassName("item sale");       // both classes
console.log("on sale:", [...sale].map((li) => li.textContent));

const lis = document.getElementById("cart").getElementsByTagName("li");   // search inside one element
console.log("li inside #cart:", lis.length);
console.log("first by index:", items[0].textContent, "| by item():", items.item(1).textContent);
```

BROWSER CONSOLE

```
items: 3 [Pen, Book, Bag]
on sale: [Book, Bag]
li inside #cart: 3
first by index: Pen | by item(): Book
```

15.2.3 querySelector() and querySelectorAll()

The modern, flexible way to select is with **CSS selectors**, the same syntax you use in stylesheets. `querySelector(selector)` returns the **first** matching element or `null`; `querySelectorAll(selector)` returns a **NodeList** of all matches, which has `forEach` and can be spread into an array. Both can be called on an element to search only its descendants. Any selector works: ids, classes, attributes, pseudo-classes and combinators.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<main id="main">
  <h2>Courses</h2>
  <div class="card" data-id="js" data-price="4999"><h3>JavaScript</h3><span class="price">Rs
4,999</span></div>
  <div class="card featured" data-id="py" data-price="5999"><h3>Python</h3><span class="price">Rs
5,999</span></div>
  <div class="card featured" data-id="java"><h3>Java</h3><span class="price">Rs 6,499</span></div>
  <ul><li>Live classes</li><li>Projects</li><li>Placement help</li></ul>
  <input type="email" name="newsletter">
</main>
```

SELECTING WITH CSS SELECTORS

main.js

```
const $ = (sel, root = document) => root.querySelector(sel);
const $$ = (sel, root = document) => [...root.querySelectorAll(sel)];

console.log("#main h2:          ", $("#main h2").textContent);
console.log(".card (first):     ", $(".card").dataset.id);
console.log(".card (all):       ", $$(".card").map((c) => c.dataset.id));
console.log(".card.featured:    ", $$(".card.featured").map((c) => c.dataset.id));
console.log("[data-price]:      ", $$("[data-price]").length);
console.log("input[type=email]: ", $("input[type=email]").name);
console.log("li:nth-child(2):   ", $("li:nth-child(2)").textContent);
console.log("ul > li:last-child:", $("ul > li:last-child").textContent);
console.log("not .featured:     ", $$(".card:not(.featured)").map((c) => c.dataset.id));
console.log("inside one card:   ", $(".price", $$(".card")[1]).textContent);

document.querySelectorAll(".card").forEach((card, i) => console.log(`  card ${i}:
${card.querySelector("h3").textContent}`));
```

BROWSER CONSOLE

```
#main h2:           Courses
.card (first):      js
.card (all):        [js, py, java]
.card.featured:     [py, java]
[data-price]:       2
input[type=email]:  newsletter
li:nth-child(2):    Projects
ul > li:last-child: Placement help
not .featured:      [js]
inside one card:    Rs 5,999
  card 0: JavaScript
  card 1: Python
  card 2: Java
```

Selector	Matches
`#id`	the element with that id
`.class`, `.a.b`	elements with the class (or with both classes)
`tag`	all elements of that type
`A B` / `A > B`	B anywhere inside A / B directly inside A
`A + B` / `A ~ B`	B right after A / any B after A (siblings)
`[attr]`, `[attr=value]`	elements with the attribute (and value)
`:first-child`, `:nth-child(n)`	position among siblings
`:not(sel)`, `:checked`, `:disabled`	negation and state pseudo-classes
`A, B`	elements matching either selector

TIP

Use `getElementById` or `querySelector("#id")` for single elements and `querySelectorAll` for groups. The difference in speed is irrelevant for normal pages; the readability of CSS selectors usually wins.

15.2.4 NodeList vs HTMLCollection (live vs static)

An **HTMLCollection** (from `getElementsBy...` and `element.children`) is **live**: it updates automatically when the document changes. The **NodeList** from `querySelectorAll` is **static**: a snapshot taken when it was called. (`childNodes` is a live NodeList.) Live collections can cause surprising bugs, such as infinite loops when you add matching elements while looping over them. Neither is a real array: NodeList has `forEach`, HTMLCollection does not, so convert with `[...list]` or `Array.from(list)` to use `map`, `filter` and the rest.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<ul><li class="task">Task 1</li><li class="task">Task 2</li></ul>
```

LIVE AND STATIC COLLECTIONS main.js

```
const live = document.getElementsByClassName("task");     // HTMLCollection (live)
const snapshot = document.querySelectorAll(".task");      // NodeList (static)
console.log("before:", live.length, snapshot.length);

const li = document.createElement("li");
li.className = "task";
li.textContent = "Task 3";
document.querySelector("ul").append(li);
console.log("after adding one:", live.length, "(live)", snapshot.length, "(static)");

console.log("NodeList has forEach:", typeof snapshot.forEach, "| HTMLCollection has forEach:", typeof
live.forEach);
console.log("as array:", Array.from(live, (el) => el.textContent));
console.log("types:", live.constructor.name, snapshot.constructor.name);
```

BROWSER CONSOLE

```
before: 2 2
after adding one: 3 (live) 2 (static)
NodeList has forEach: function | HTMLCollection has forEach: undefined
as array: [Task 1, Task 2, Task 3]
types: HTMLCollection NodeList
```

	HTMLCollection	NodeList
Returned by	getElementsByClassName/TagName, children, forms	querySelectorAll (static), childNodes (live)
Contains	elements only	any nodes (elements, text…)
Updates automatically	yes (live)	no for querySelectorAll
forEach	no	yes
Convert to array	`[...c]` / Array.from	`[...n]` / Array.from

15.3 Traversing the DOM

15.3.1 parentElement, children, firstElementChild, nextElementSibling

From any element you can move around the tree: up to its `parentElement`; down to its `children` (an HTMLCollection), `firstElementChild` and `lastElementChild`; and sideways to `previousElementSibling` and

`nextElementSibling`. Each returns `null` when there is nothing in that direction. The node-level versions (`parentNode`, `childNodes`, `firstChild`, `nextSibling`) include text and comment nodes.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<nav id="nav">
  <ol id="steps">
    <li>Cart</li>
    <li>Address</li>
    <li class="current">Payment</li>
    <li>Confirm</li>
  </ol>
</nav>
```

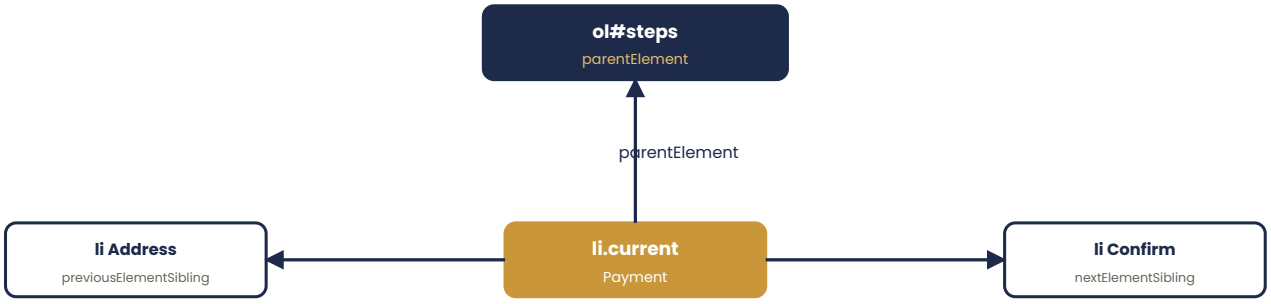
MOVING AROUND THE TREE main.js

```
const current = document.querySelector(".current");
console.log("current:          ", current.textContent);
console.log("parentElement:    ", current.parentElement.id);
console.log("previous sibling:", current.previousElementSibling.textContent);
console.log("next sibling:     ", current.nextElementSibling.textContent);
const list = current.parentElement;
console.log("first / last:     ", list.firstElementChild.textContent, "/",
list.lastElementChild.textContent);
console.log("children count:  ", list.childElementCount);
console.log("nextSibling (any node):", JSON.stringify(current.nextSibling.textContent));
console.log("beyond the last: ", list.lastElementChild.nextElementSibling);

// walk all ancestors
const names = [];
for (let el = current; el; el = el.parentElement) names.push(el.tagName.toLowerCase() + (el.id ? "#" +
el.id : ""));
console.log("ancestors:", names.join(" < "));
```

BROWSER CONSOLE

```
current:          Payment
parentElement:    steps
previous sibling: Address
next sibling:     Confirm
first / last:     Cart / Confirm
children count:   4
nextSibling (any node): "\n    "
beyond the last:  null
ancestors: li < ol#steps < nav#nav < body < html
```



NAVIGATION PROPERTIES RELATIVE TO LI.CURRENT

15.3.2 closest()

`element.closest(selector)` walks **up** from the element itself through its ancestors and returns the first one that matches the selector, or `null`. It is perfect for finding the "card" or "row" that contains a clicked button, and it is the key tool for event delegation (Module 16). The opposite direction, finding descendants, is `querySelector`; checking whether an element itself matches is `element.matches(selector)`.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<table id="orders">
  <tr data-id="101"><td class="name">Laptop</td><td><button data-action="edit">Edit</button></td></tr>
  <tr data-id="102" class="odd"><td class="name">Mouse</td><td><button data-
action="delete">Delete</button></td></tr>
</table>
```

FINDING THE ENCLOSING ROW main.js

```
const button = document.querySelector("button[data-action=delete]");
const row = button.closest("tr");
console.log("row of the button:", row.dataset.id);
console.log("table:", button.closest("table").id);
console.log("closest checks itself first:", button.closest("button") === button);
console.log("no match:", button.closest(".modal"));
console.log("matches:", button.matches("[data-action]"), row.matches(".odd"));
console.log("product in that row:", row.querySelector(".name").textContent);
```

BROWSER CONSOLE

```
row of the button: 102
table: orders
closest checks itself first: true
no match: null
matches: true true
product in that row: Mouse
```

15.4 Changing content

15.4.1 textContent vs innerText vs innerHTML

Three properties read and write an element's content. `textContent` gets the raw text of the element and all descendants, including hidden elements, and setting it replaces everything with plain text (fast and safe). `innerText` gets the text **as rendered**: it respects CSS (hidden elements are skipped, line breaks follow the layout), which makes it slower because it must calculate styles. `innerHTML` gets or sets the HTML markup inside the element; setting it makes the browser parse the string as HTML. `outerHTML` includes the element itself.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<div id="box">Visible <span style="display:none">hidden</span><b>bold</b></div>
<p id="a"></p>
<p id="b"></p>
```

READING AND WRITING CONTENT

main.js

```
const box = document.querySelector("#box");
console.log("textContent:", JSON.stringify(box.textContent));
console.log("innerText:  ", JSON.stringify(box.innerText));
console.log("innerHTML:  ", box.innerHTML);

document.querySelector("#a").textContent = "<b>bold?</b>";     // shown literally
document.querySelector("#b").innerHTML = "<b>bold!</b> and <i>italic</i>";   // parsed as HTML
console.log("outerHTML of #b:", document.querySelector("#b").outerHTML);
```

BROWSER CONSOLE

```
textContent: "Visible hiddenbold"
innerText:   "Visible bold"
innerHTML:   Visible <span style="display:none">hidden</span><b>bold</b>
outerHTML of #b: <p id="b"><b>bold!</b> and <i>italic</i></p>
--- page shows ---
Visible bold

<b>bold?</b>

bold! and italic
```

Property	Reads	Setting it	Use for
`textContent`	all text, including hidden	replaces content with plain text	most text updates (fast, safe)
`innerText`	visible text as rendered	plain text, \n becomes 	copying what the user sees
`innerHTML`	the HTML markup	parses a string as HTML	trusted markup only
`outerHTML`	markup including the element	replaces the element	rarely

15.4.2 Security risk of innerHTML (XSS)

Inserting **untrusted** text (anything typed by a user, read from a URL, or received from an API) with `innerHTML` creates a **Cross-Site Scripting (XSS)** vulnerability: an attacker can supply HTML that runs JavaScript in your page, for example an image whose `onerror` handler steals cookies or session tokens. `<script>` tags inserted via innerHTML do not run, but event-handler attributes do. The fix is simple: use `textContent` for text, create elements with DOM methods, or escape the text before building HTML. When you genuinely must insert user-provided HTML (a rich-text comment), sanitise it with a library such as DOMPurify.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY)

HTML

```
<div id="unsafe"></div>
<div id="safe"></div>
<div id="escaped"></div>
```

INNERHTML RUNS INJECTED CODE; TEXTCONTENT DOES NOT

main.js

```
const comment = '<img src="x" onerror="console.log(\'XSS! attacker code runs, could read\',
document.cookie || \'cookies\')">';

document.querySelector("#unsafe").innerHTML = comment;          // DANGEROUS
document.querySelector("#safe").textContent = comment;          // shown as text

function escapeHtml(s) {
  return s.replace(/[&<>"']/g, (c) => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'":
"&#39;" })[c]);
}
document.querySelector("#escaped").innerHTML = `<b>Comment:</b> ${escapeHtml(comment)}`;
console.log("escaped markup:", document.querySelector("#escaped").innerHTML.slice(0, 60) + "...");
```

BROWSER CONSOLE

```
escaped markup: <b>Comment:</b> &lt;img src="x" onerror="console.log('XSS! a...
XSS! attacker code runs, could read cookies
```

WATCH OUT

Never pass user input to `innerHTML`, `outerHTML`, `insertAdjacentHTML`, `document.write`, or to `eval` / `new Function` / `setTimeout(string)`. Module 24 covers XSS, Content Security Policy and the Trusted Types API in depth.

15.5 Changing attributes and styles

15.5.1 getAttribute(), setAttribute(), removeAttribute()

HTML attributes are read and changed with `getAttribute(name)`, `setAttribute(name, value)`, `removeAttribute(name)`, `hasAttribute(name)` and `toggleAttribute(name)`. Most standard attributes are also available as **properties** (`img.src`, `a.href`, `input.value`, `el.id`). Attributes and properties are not always the same: the property may be normalised (a full URL for `href`), typed (a boolean for `checked`), or represent the **current** state while the attribute keeps the **initial** value (`input.value` vs `getAttribute("value")`).

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY)

HTML

```
<a href="/courses">Courses</a>
<img src="logo.png" width="120">
<input value="initial">
<button>Pay</button>
```

ATTRIBUTES AND PROPERTIES

main.js

```
const link = document.querySelector("a");
const img = document.querySelector("img");
const input = document.querySelector("input");

console.log("attribute href:", link.getAttribute("href"), "| property href:", link.href);
link.setAttribute("target", "_blank");
link.setAttribute("rel", "noopener");
img.setAttribute("alt", "LBF logo");
img.removeAttribute("width");
console.log("has width?", img.hasAttribute("width"));

input.value = "typed by user";                // current value (property)
console.log("value property:", input.value, "| value attribute:", input.getAttribute("value"));

const btn = document.querySelector("button");
btn.toggleAttribute("disabled");
console.log("disabled property:", btn.disabled);
console.log("all attributes of a:", [...link.attributes].map((a) => `${a.name}="${a.value}"`).join("
"));
```

BROWSER CONSOLE

```
attribute href: /courses | property href: http://lbf.local/courses
has width? false
value property: typed by user | value attribute: initial
disabled property: true
all attributes of a: href="/courses" target="_blank" rel="noopener"
--- element HTML ---
<a href="/courses" target="_blank" rel="noopener">Courses</a>
```

15.5.2 dataset and data attributes

Custom attributes whose names start with `data-` are the standard place to store extra information on an element, such as an id or a price that your code needs later. They are read and written through the `dataset` object: `data-product-id` becomes `dataset.productId` (kebab-case to camelCase). Values are always **strings**, so convert numbers with `Number()`. CSS can also select by them (`[data-status="paid"]`).

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY)

HTML

```
<div class="product" data-product-id="LBF-101" data-price="4999" data-discount="10">JavaScript
course</div>
```

READING AND WRITING DATA-* ATTRIBUTES main.js

```
const card = document.querySelector(".product");
console.log("dataset:", JSON.stringify({ ...card.dataset }));     // a DOMStringMap, copied for
printing
console.log("id:", card.dataset.productId, "| price + 1:", Number(card.dataset.price) + 1, "| as
string:", card.dataset.price + 1);

card.dataset.inCart = "true";                              // creates data-in-cart
delete card.dataset.discount;                              // removes data-discount
console.log("data-in-cart attribute:", card.getAttribute("data-in-cart"));
console.log("entries:", JSON.stringify(Object.entries(card.dataset)));
```

BROWSER CONSOLE

```
dataset: {"productId":"LBF-101","price":"4999","discount":"10"}
id: LBF-101 | price + 1: 5000 | as string: 49991
data-in-cart attribute: true
entries: [["productId","LBF-101"],["price","4999"],["inCart","true"]]
--- element HTML ---
<div class="product" data-product-id="LBF-101" data-price="4999" data-in-cart="true">JavaScript
course</div>
```

15.5.3 style property

`element.style` reads and writes **inline** styles (the `style` attribute). CSS property names are written in camelCase (`backgroundColor` for `background-color`), values are strings with units (`"20px"`), and assigning `""` removes a property. `style` only shows inline styles; to read the **final** computed value from all stylesheets, use `getComputedStyle(element)`. CSS custom properties (variables) are set with `style.setProperty("--name", value)`.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<style>.box { font-size: 18px; width: 200px; }</style>
<div class="box">Styled box</div>
```

INLINE AND COMPUTED STYLES main.js

```
const box = document.querySelector(".box");
box.style.backgroundColor = "#C9A227";
box.style.padding = "12px";
box.style.borderRadius = "8px";
box.style.cssText += "; color: #1B2A4A";                    // several at once
console.log("inline style attribute:", box.getAttribute("style"));

console.log("style.fontSize (inline only):", JSON.stringify(box.style.fontSize));
const computed = getComputedStyle(box);
console.log("computed font-size:", computed.fontSize, "| computed width:", computed.width);

document.documentElement.style.setProperty("--brand", "#1B2A4A");
console.log("CSS variable:", getComputedStyle(document.documentElement).getPropertyValue("--brand"));
box.style.padding = "";                                     // remove
console.log("after removing padding:", box.getAttribute("style"));
```

BROWSER CONSOLE

```
inline style attribute: background-color: rgb(201, 162, 39); padding: 12px; border-radius: 8px; color:
rgb(27, 42, 74);
style.fontSize (inline only): ""
computed font-size: 18px | computed width: 200px
CSS variable: #1B2A4A
after removing padding: background-color: rgb(201, 162, 39); border-radius: 8px; color: rgb(27, 42,
74);
```

TIP

Prefer changing **classes** (15.5.4) over setting many inline styles: keep the visual design in CSS and let JavaScript only switch states. Use `style` for values that are truly dynamic, such as positions or progress-bar widths calculated at run time.

15.5.4 classList: add, remove, toggle, contains

`element.classList` manages the element's classes without string manipulation. `add` and `remove` accept several classes; `toggle(cls)` adds the class if missing and removes it if present, returning the new state, and `toggle(cls, force)` adds or removes according to a boolean; `contains` tests; `replace(old, new)` swaps. `className` is the whole class string, rarely needed now.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<div id="panel" class="card hidden">Panel</div>
```

WORKING WITH CLASSES

main.js

```
const panel = document.querySelector("#panel");
panel.classList.add("open", "highlight");
console.log("after add:     ", panel.className);
panel.classList.remove("hidden");
console.log("after remove:  ", panel.className);
console.log("toggle dark ->", panel.classList.toggle("dark"), "| again ->",
panel.classList.toggle("dark"));
const isMobile = true;
panel.classList.toggle("compact", isMobile);              // force on
console.log("contains compact?", panel.classList.contains("compact"));
panel.classList.replace("highlight", "selected");
console.log("final classes: ", [...panel.classList]);
```

BROWSER CONSOLE

```
after add:      card hidden open highlight
after remove:   card open highlight
toggle dark -> true | again -> false
contains compact? true
final classes:  [card, open, selected, compact]
--- element HTML ---
<div id="panel" class="card open selected compact">Panel</div>
```

15.6 Creating and removing elements

15.6.1 createElement(), append(), prepend(), before(), after()

`document.createElement(tag)` creates a new, detached element; set its text, attributes and classes, then insert it. The modern insertion methods accept any number of nodes **or strings** (strings become text nodes, never HTML): `parent.append(...)` adds at the end inside, `prepend(...)` at the start inside, `el.before(...)` and `el.after(...)` as siblings. The older `appendChild` and `insertBefore` take a single node. Inserting a node that is already in the document **moves** it.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY)

HTML

```
<ul id="list"><li id="css">CSS</li><li>JavaScript</li></ul>
```

CREATING AND INSERTING ELEMENTS

main.js

```
const list = document.querySelector("#list");

const li = document.createElement("li");
li.textContent = "Node.js";
li.className = "new";
list.append(li);                              // end, inside

const first = document.createElement("li");
first.textContent = "HTML";
list.prepend(first);                          // start, inside

list.before("Course list:");                  // a string becomes a text node
const note = document.createElement("p");
note.textContent = "3 more modules coming soon";
list.after(note);                             // sibling after the list

const css = document.querySelector("#css");
css.after(Object.assign(document.createElement("li"), { textContent: "Sass" }));   // after a specific
item
list.append(first);                           // moving an existing element
```

BROWSER CONSOLE

```
--- page shows ---
Course list:
CSS
Sass
JavaScript
Node.js
HTML

3 more modules coming soon
--- element HTML ---
<ul id="list"><li id="css">CSS</li><li>Sass</li><li>JavaScript</li><li class="new">Node.js</li>
<li>HTML</li></ul>
```



WHERE EACH INSERTION METHOD PUTS THE NEW CONTENT

15.6.2 remove() and replaceWith()

`element.remove()` removes an element (with everything inside it) from the document. `element.replaceWith(...nodes)` swaps it for new content. `parent.replaceChildren(...nodes)` replaces all children at once, and with no arguments empties the element, which is clearer than `innerHTML = ""`. A removed element still exists in memory as long as a variable refers to it, so it can be re-inserted.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```html
<ul><li>Apple</li><li>Banana</li><li>Cherry</li><li>Date</li></ul>
<div id="other">Moved here: </div>
<div id="old"><p>a</p><p>b</p></div>
```

REMOVING AND REPLACING main.js

```javascript
const items = document.querySelectorAll("li");
items[1].remove();                                  // remove "Banana"

const strong = document.createElement("strong");
strong.textContent = "Cherry (replaced)";
items[2].replaceWith(strong);                       // swap an element

const saved = items[0];
saved.remove();
console.log("removed element still usable:", saved.textContent, "| in document?",
document.contains(saved));
document.querySelector("#other").append(saved);     // re-insert elsewhere

document.querySelector("#old").replaceChildren();   // empty it
console.log("#old children:", document.querySelector("#old").children.length);
```

BROWSER CONSOLE

```
removed element still usable: Apple | in document? false
#old children: 0
--- page shows ---
Cherry (replaced)
Date
Moved here:
Apple
```

15.6.3 insertAdjacentHTML()

`element.insertAdjacentHTML(position, html)` parses an HTML string and inserts the result at one of four positions, **without** destroying the existing content (unlike `innerHTML +=`, which re-parses and rebuilds everything, losing event listeners and input values). The positions are `"beforebegin"`, `"afterbegin"`, `"beforeend"` and `"afterend"`. There are also `insertAdjacentElement` and `insertAdjacentText`. The XSS warning of 15.4.2 applies: use it only with trusted or escaped strings.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```html
<ul><li>existing item</li></ul>
<div id="form"><input></div>
```

INSERTING HTML AT FOUR POSITIONS main.js

```
const list = document.querySelector("ul");
list.insertAdjacentHTML("beforebegin", "<h3>Before begin</h3>");
list.insertAdjacentHTML("afterbegin", "<li><em>after begin</em></li>");
list.insertAdjacentHTML("beforeend", "<li><em>before end</em></li>");
list.insertAdjacentHTML("afterend", "<p>after end</p>");

// innerHTML += destroys existing elements (and their state)
const input = document.querySelector("input");
input.value = "typed text";
document.querySelector("#form").innerHTML += "<span> (added)</span>";
console.log("value after innerHTML +=:", JSON.stringify(document.querySelector("input").value), "|
same element?", document.querySelector("input") === input);
```

BROWSER CONSOLE

```
value after innerHTML +=: "" | same element? false
--- page shows ---
Before begin
after begin
existing item
before end

after end

 (added)
```

15.6.4 DocumentFragment for performance

Every insertion into the live document can make the browser recalculate layout (15.9). When adding many elements, build them inside a **DocumentFragment**, a lightweight container that is not part of the page, and then insert the fragment once: its children move into the document in a single operation and the fragment becomes empty. Today, `parent.append(...arrayOfNodes)` achieves the same single insertion.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<ol id="students"></ol>
<ul id="marks"></ul>
```

BUILDING MANY ELEMENTS OFF-PAGE main.js

```
const students = ["Asha", "Ravi", "Meera", "John", "Zoya", "Kiran"];
const list = document.querySelector("#students");

const fragment = document.createDocumentFragment();
students.forEach((name, i) => {
  const li = document.createElement("li");
  li.textContent = `${i + 1}. ${name}`;
  fragment.append(li);                      // not in the page yet: no layout work
});
console.log("fragment children before insert:", fragment.childNodes.length);
list.append(fragment);                      // one insertion
console.log("fragment children after insert:", fragment.childNodes.length, "| list items:",
list.children.length);

// the same with an array and spread
const marks = [91, 78, 85].map((m) => Object.assign(document.createElement("li"), { textContent: m
}));
document.querySelector("#marks").append(...marks);
```

BROWSER CONSOLE

```
fragment children before insert: 6
fragment children after insert: 0 | list items: 6
--- page shows ---
1. Asha
2. Ravi
3. Meera
4. John
5. Zoya
6. Kiran
91
78
85
```

15.6.5 <template> element

A `<template>` element holds HTML that the browser parses but does not display or run. Its `content` property is a DocumentFragment. To use it, **clone** the content with `template.content.cloneNode(true)`, fill in the clone (with `textContent`, which is safe), and insert it. Templates keep markup in HTML instead of long strings in JavaScript, and are the basis of Web Components (Module 21).

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<template id="course-card">
  <div class="card"><h3></h3><p class="meta"></p><button>Enrol</button></div>
</template>
<div id="grid"></div>
```

STAMPING OUT CARDS FROM A TEMPLATE main.js

```
const template = document.querySelector("#course-card");
const courses = [
  { name: "JavaScript", weeks: 12, level: "Beginner" },
  { name: "React", weeks: 8, level: "Intermediate" },
];

const grid = document.querySelector("#grid");
for (const c of courses) {
  const card = template.content.cloneNode(true);       // deep copy of the fragment
  card.querySelector("h3").textContent = c.name;
  card.querySelector(".meta").textContent = `${c.weeks} weeks · ${c.level}`;
  card.querySelector(".card").dataset.level = c.level.toLowerCase();
  grid.append(card);
}
console.log("cards:", grid.children.length, "| template still intact:",
template.content.children.length);
```

BROWSER CONSOLE

```
cards: 2 | template still intact: 1
--- page shows ---
JavaScript

12 weeks · Beginner

Enrol
React

8 weeks · Intermediate

Enrol
--- element HTML ---
<div id="grid">
  <div class="card" data-level="beginner"><h3>JavaScript</h3><p class="meta">12 weeks · Beginner</p>
<button>Enrol</button></div>

  <div class="card" data-level="intermediate"><h3>React</h3><p class="meta">8 weeks · Intermediate</p>
<button>Enrol</button></div>
</div>
```

15.7 Forms

15.7.1 Reading input values, checkboxes, radios and selects

Form controls expose their current state through properties. Text inputs, textareas and selects have `value` (always a **string**; use `valueAsNumber` or `Number()` for numbers and `valueAsDate` for dates). Checkboxes and radios have a boolean `checked`. A select has `value`, `selectedIndex` and, for multiple selection, `selectedOptions`. `form.elements` gives access to controls by name, and `new FormData(form)` collects every named value at once, which is ideal for sending to a server.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<form id="enrol">
  <input name="fullName" value="Asha Rao">
  <input name="age" type="number" value="21">
  <label><input name="newsletter" type="checkbox" checked> Newsletter</label>
  <label><input name="batch" type="radio" value="morning"> Morning</label>
  <label><input name="batch" type="radio" value="evening" checked> Evening</label>
  <select name="course"><option value="js">JavaScript</option><option value="py"
selected>Python</option></select>
  <select name="skills" multiple><option selected>HTML</option><option>CSS</option><option
selected>Git</option></select>
  <input name="start" type="date" value="2026-04-01">
</form>
```

READING EVERY KIND OF CONTROL main.js

```
const form = document.querySelector("#enrol");
const els = form.elements;

console.log("name:     ", els.fullName.value);
console.log("age:      ", els.age.value, typeof els.age.value, "| valueAsNumber:",
els.age.valueAsNumber);
console.log("newsletter checked:", els.newsletter.checked);
console.log("batch (radio):", form.querySelector("input[name=batch]:checked").value, "| via
RadioNodeList:", els.batch.value);
console.log("course (select):", els.course.value, "| text:", els.course.selectedOptions[0].text);
console.log("skills (multi):", [...els.skills.selectedOptions].map((o) => o.value));
console.log("start date:", els.start.valueAsDate.toDateString());

const data = new FormData(form);
console.log("FormData skills:", data.getAll("skills"));
console.log("as object:", Object.fromEntries(data));
```

BROWSER CONSOLE

```
name:      Asha Rao
age:       21 string | valueAsNumber: 21
newsletter checked: true
batch (radio): evening | via RadioNodeList: evening
course (select): py | text: Python
skills (multi): [HTML, Git]
start date: Wed Apr 01 2026
FormData skills: [HTML, Git]
as object: {fullName: Asha Rao, age: 21, newsletter: on, batch: evening, course: py}
```

Values can be set the same way: assign `value`, `checked` or `selectedIndex`. Changing them from code does not fire `input` or `change` events (Module 16).

15.7.2 Form validation with JavaScript and the Constraint Validation API

HTML has built-in validation attributes: `required`, `type="email"`, `min`/`max`, `minlength`/`maxlength` and `pattern`. The **Constraint Validation API** lets JavaScript use the same engine: `input.checkValidity()` returns a boolean, `input.validity` tells you exactly which rule failed (`valueMissing`, `typeMismatch`, `tooShort`, `rangeUnderflow`, `patternMismatch` …), `validationMessage` gives the browser's message, and `setCustomValidity(msg)` adds your own rule (an empty string clears it). `form.checkValidity()` checks all controls; `reportValidity()` also shows the browser's bubbles. CSS can style `:invalid` and `:user-invalid` fields.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<form id="signup" novalidate>
  <input name="email" type="email" required>
  <input name="password" type="password" required minlength="8">
  <input name="confirm" type="password" required>
  <input name="age" type="number" min="16" max="100">
</form>
```

CHECKING VALIDITY FROM JAVASCRIPT main.js

```
const form = document.querySelector("#signup");
const { email, password, confirm, age } = form.elements;

function check() {
  confirm.setCustomValidity(confirm.value === password.value ? "" : "Passwords do not match");   //
custom rule
  for (const input of [email, password, confirm, age]) {
    const failed = Object.keys(ValidityState.prototype).filter((k) => k !== "valid" &&
input.validity[k]);
    console.log(`${input.name.padEnd(8)} valid=${String(input.validity.valid).padEnd(5)}
${failed.join(",")} ${input.validationMessage}`);
  }
  console.log("whole form valid:", form.checkValidity());
}

email.value = "asha@"; password.value = "abc"; confirm.value = "abd"; age.value = "12";
check();
console.log("--- after fixing ---");
email.value = "asha@lbf.in"; password.value = "s3cretpass"; confirm.value = "s3cretpass"; age.value =
"21";
check();
```

BROWSER CONSOLE

```
email    valid=false typeMismatch Please enter a part following '@'. 'asha@' is incomplete.
password valid=true
confirm  valid=false customError Passwords do not match
age      valid=false rangeUnderflow Value must be greater than or equal to 16.
whole form valid: false
--- after fixing ---
email    valid=true
password valid=true
confirm  valid=true
age      valid=true
whole form valid: true
```

NOTE

`novalidate` on the form turns off the browser's automatic bubbles on submit so that your code decides how to show errors, while the validation API still works. Client-side validation improves the user experience but is never a security measure: always validate on the server too.

15.8 Browser Object Model (BOM)

15.8.1 window, location, history, navigator, screen

The **Browser Object Model** is the set of objects for the browser itself rather than the page content. It is not one formal standard, but its parts are standardised individually. `location` represents the current URL and can navigate; `history` moves back and forward and lets single-page apps change the URL without

reloading (`pushState`); `navigator` describes the browser (language, online status, clipboard, geolocation, user agent); `screen` describes the display; and `window` itself provides size, scrolling, timers, dialogs and opening windows.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY)

HTML

```
<p>BOM demo</p>
<script>history.replaceState(null, "", "/index.html?course=js&batch=evening#reviews");</script>
```

EXPLORING THE BOM

main.js

```
console.log("location.href:    ", location.href);
console.log("protocol / host:  ", location.protocol, location.host);
console.log("pathname:          ", location.pathname);
console.log("search params:    ", Object.fromEntries(new URLSearchParams(location.search)));
console.log("hash:              ", location.hash);

history.pushState({ page: 2 }, "", "/courses/js?tab=syllabus");      // change URL, no reload
console.log("after pushState:  ", location.pathname + location.search, "| history.length:",
history.length, "| state:", history.state);

console.log("navigator.language:", navigator.language, "| online:", navigator.onLine, "| cookies:",
navigator.cookieEnabled);
console.log("screen:", screen.width + "x" + screen.height, "| window inner:", innerWidth + "x" +
innerHeight, "| pixel ratio:", devicePixelRatio);
console.log("scroll position:", scrollX, scrollY);
```

BROWSER CONSOLE

```
location.href:     http://lbf.local/index.html?course=js&batch=evening#reviews
protocol / host:   http: lbf.local
pathname:          /index.html
search params:     {course: js, batch: evening}
hash:              #reviews
after pushState:   /courses/js?tab=syllabus | history.length: 3 | state: {page: 2}
navigator.language: en-US | online: true | cookies: true
screen: 800x600 | window inner: 800x600 | pixel ratio: 1
scroll position: 0 0
```

Object	Useful members
`window`	innerWidth/innerHeight, scrollTo(), scrollY, open(), alert/confirm/prompt, setTimeout, matchMedia()
`location`	href, protocol, host, pathname, search, hash; assign(url), replace(url), reload()
`history`	back(), forward(), go(n), pushState(state, "", url), replaceState(), length, state
`navigator`	language(s), onLine, userAgent, clipboard, geolocation, share(), serviceWorker
`screen`	width, height, availWidth, orientation

NAVIGATION METHODS main.js

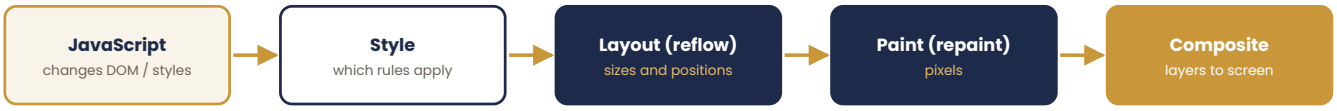
```
location.href = "https://learnbeingforward.in";   // navigate (adds a history entry)
location.replace("/login");                         // navigate without a history entry
location.reload();                                  // reload the page
history.back();                                     // like the Back button
window.open("/help", "_blank", "noopener");         // new tab
```

NOTE
These leave or reload the current page, so they are shown without output.

15.9 DOM performance

15.9.1 Reflow and repaint

After the DOM or styles change, the browser runs a **rendering pipeline**: recalculate styles, compute **layout** (the size and position of every affected box, also called **reflow**), **paint** pixels (repaint), and **composite** layers onto the screen. Changes to geometry (width, height, font size, adding elements) trigger layout, which is the most expensive step; colour changes only trigger repaint; `transform` and `opacity` can often be handled by the compositor alone, which is why they are preferred for animations.



THE RENDERING PIPELINE

Browsers normally batch changes and run layout once before the next frame. But **reading** a layout property (`offsetHeight`, `getBoundingClientRect()`, `scrollTop`, `getComputedStyle`) right after a **write** forces the browser to do the layout immediately, a **forced synchronous layout**. Alternating reads and writes in a loop causes **layout thrashing**, which can make a page stutter.

15.9.2 Batching DOM updates

To keep pages fast: **batch reads, then writes** instead of alternating them; build many elements in a fragment or array and insert once; change a class instead of many individual styles; avoid touching the DOM inside tight loops; animate with `transform`/`opacity`; and schedule visual updates with `requestAnimationFrame`, which runs your callback just before the browser paints the next frame. For very long lists, render only the visible rows (virtualisation).

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<style>.b { display:inline-block; padding:2px; }</style><div id="boxes"></div>
```

LAYOUT THRASHING VERSUS BATCHING

main.js

```
const box = document.querySelector("#boxes");
box.append(...Array.from({ length: 300 }, () => Object.assign(document.createElement("div"), {
className: "b", textContent: "item" })));
const items = [...box.children];

// Thrashing: read, write, read, write... forces a layout on every iteration
let t = performance.now();
for (const el of items) el.style.width = (el.offsetWidth + 1) + "px";
const thrash = performance.now() - t;

// Batched: all reads first, then all writes -> one layout
t = performance.now();
const widths = items.map((el) => el.offsetWidth);          // reads
items.forEach((el, i) => { el.style.width = (widths[i] + 1) + "px"; });   // writes
const batched = performance.now() - t;
console.log("batched version was faster:", batched < thrash);

// requestAnimationFrame: run visual updates right before the next paint
let frames = 0;
function step() {
  box.style.transform = `translateX(${++frames * 2}px)`;
  if (frames < 5) requestAnimationFrame(step);
  else console.log("animated over", frames, "frames, transform:", box.style.transform);
}
requestAnimationFrame(step);
```

BROWSER CONSOLE

```
batched version was faster: true
animated over 5 frames, transform: translateX(10px)
```

NOTE

Timings vary between machines and runs, so the example prints only whether batching was faster (on typical machines thrashing is tens of times slower here). In DevTools, the **Performance** panel shows layout work as purple "Layout" blocks, and warns about forced reflows.

The DOM is the browser's live tree of node objects built from HTML; window is the global object and document the entry to the tree. Elements are the tag nodes; children/firstElementChild skip text nodes. Select with getElementById, querySelector and querySelectorAll (CSS selectors); getElementsBy… return live HTMLCollections, querySelectorAll a static NodeList. Traverse with parentElement, children, siblings and closest(). Change text with textContent (safe), and never insert untrusted strings with innerHTML (XSS). Use get/set/removeAttribute, dataset for data-* values, style for inline styles (getComputedStyle for final values) and classList for classes. Create with createElement, insert with append/prepend/before/after, remove with remove/replaceWith, add HTML with insertAdjacentHTML, batch with DocumentFragment, and stamp copies from <template>. Read forms with value/checked/selectedOptions/FormData and validate with the Constraint Validation API. The BOM provides location, history, navigator and screen. Avoid layout thrashing by batching reads and writes.

PRACTICE CHECKPOINT

Build a **to-do list** where users add, complete, edit and delete tasks, entirely with DOM methods.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY)

HTML

```
<h2>My tasks</h2>
<form id="add-form"><input id="new-task" placeholder="New task"><button>Add</button></form>
<ul id="tasks"></ul>
<p id="counter"></p>
```

SAMPLE SOLUTION

main.js

```
const form = document.querySelector("#add-form");
const input = document.querySelector("#new-task");
const list = document.querySelector("#tasks");
const counter = document.querySelector("#counter");
let nextId = 1;

function updateCounter() {
  const total = list.children.length;
  const done = list.querySelectorAll("li.done").length;
  counter.textContent = `${done} of ${total} completed`;
}

function createTask(text) {
  const li = document.createElement("li");
  li.dataset.id = nextId++;
  const check = Object.assign(document.createElement("input"), { type: "checkbox" });
  const label = Object.assign(document.createElement("span"), { className: "text", textContent: text
});   // safe text
  const edit = Object.assign(document.createElement("button"), { textContent: "Edit", className:
"edit" });
  const del = Object.assign(document.createElement("button"), { textContent: "Delete", className:
"delete" });
  li.append(check, label, edit, del);
  return li;
}

form.addEventListener("submit", (e) => {
  e.preventDefault();
  const text = input.value.trim();
  if (!text) return;
  list.append(createTask(text));
  input.value = "";
  updateCounter();
});

list.addEventListener("click", (e) => {                 // one listener for all tasks (Module 16)
  const li = e.target.closest("li");
  if (!li) return;
  if (e.target.matches("input[type=checkbox]")) li.classList.toggle("done", e.target.checked);
  if (e.target.matches(".delete")) li.remove();
  if (e.target.matches(".edit")) {
    const span = li.querySelector(".text");
    const newText = prompt("Edit task", span.textContent);
    if (newText?.trim()) span.textContent = newText.trim();
  }
  updateCounter();
  console.log("tasks:", [...list.children].map((t) => (t.classList.contains("done") ? "[x] " : "[ ] ")
+ t.querySelector(".text").textContent).join(" | "));
});
updateCounter();
```

```
CONSOLE (user input in gold)
tasks: [x] Revise closures | [ ] Build the DOM project | [ ] Read MDN on events
[prompt] Edit task Build the to-do project
tasks: [x] Revise closures | [ ] Build the to-do project | [ ] Read MDN on events
tasks: [x] Revise closures | [ ] Build the to-do project
--- page shows ---
My tasks
Add
Revise closuresEditDelete
Build the to-do projectEditDelete

1 of 2 completed
```

Simulated user actions, in order: types "Revise closures" into `#new-task`; clicks `#add-form button`; types "Build the DOM project" into `#new-task`; clicks `#add-form button`; types "Read MDN on events" into `#new-task`; clicks `#add-form button`; clicks `li[data-id="1"] input`; clicks `li[data-id="2"] .edit`; clicks `li[data-id="3"] .delete`.

MODULE 16 · INTERMEDIATE

Events

Minimum duration **3 days** | Level **Intermediate**

Events are how a web page reacts: to clicks, key presses, form input, page loading, scrolling and much more. This module explains the ways to attach and remove handlers, the event object, the main families of events (mouse, keyboard, form, window, pointer and touch, clipboard and drag-and-drop), how events travel through the DOM in the capturing and bubbling phases, preventing default actions, listener options, event delegation, custom events, and keeping event-heavy pages fast with debouncing and throttling.

IN THIS MODULE

16.1 Event basics
16.2 The event object
16.3 Event types
16.4 Event flow
16.5 Event delegation
16.6 Custom events
16.7 Performance

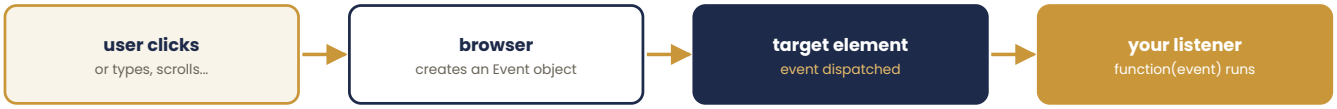
NOTE

The examples run in a real browser. User actions such as clicks and typing are performed by the test browser and listed under each example; some examples also create events from code with `dispatchEvent` or `element.click()` to simulate actions such as double-clicks, mouse moves and drag-and-drop.

16.1 Event basics

16.1.1 What events are

An **event** is a signal that something has happened: the user clicked a button, pressed a key, typed in a field or scrolled; the page finished loading; a network request completed; a video ended. The browser creates an **event object** describing what happened and delivers it to the **target** element. Your code registers **event listeners** (also called handlers): functions that the browser calls when a particular event happens on a particular element. This style of programming, where code runs in response to events rather than from top to bottom, is called **event-driven programming**.



FROM USER ACTION TO YOUR CODE

16.1.2 Inline handlers vs onclick property vs addEventListener()

There are three ways to attach a handler. An **inline attribute** (`onclick="..."` in HTML) mixes JavaScript into markup, runs code from a string and is blocked by strict Content Security Policies: avoid it. The **on-property** (`button.onclick = fn`) is simple but holds only **one** handler per event: assigning again replaces

the previous one. `addEventListener(type, listener, options)` is the modern standard: any number of listeners per event, control over the phase and options (16.4.4), and the ability to remove them. Note that the event type is written without "on": `"click"`, not `"onclick"`.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```html
<button id="pay" onclick="console.log('inline attribute handler')">Pay now</button>
```

THREE WAYS TO HANDLE A CLICK main.js

```js
const btn = document.querySelector("#pay");

// 2. on-property: only one handler; the second assignment replaces the first
btn.onclick = () => console.log("onclick handler A");
btn.onclick = () => console.log("onclick handler B (replaced A)");

// 3. addEventListener: any number of handlers, run in the order they were added
btn.addEventListener("click", () => console.log("listener 1"));
btn.addEventListener("click", (event) => console.log("listener 2, event type:", event.type));
```

BROWSER CONSOLE

```
onclick handler B (replaced A)
listener 1
listener 2, event type: click
```

Simulated user actions, in order: clicks `#pay`.

Approach	Example	Verdict
Inline attribute	`<button onclick="pay()">`	avoid: mixes HTML and JS, global functions only, blocked by CSP
On-property	`btn.onclick = pay;`	acceptable for quick scripts; one handler only
addEventListener	`btn.addEventListener("click", pay)`	recommended: many handlers, options, removable

16.1.3 Removing listeners

`removeEventListener(type, listener)` removes a listener, but only if you pass the **same function reference** (and the same `capture` setting) that was added. An anonymous function written inline cannot be removed, because a new identical-looking function is a different object. Other options: `{ once: true }` removes the listener automatically after the first call, and an `AbortController` signal removes many listeners at once.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```html
<button>Click me</button>
```

REMOVEEVENTLISTENER NEEDS THE SAME FUNCTION

main.js

```
const btn = document.querySelector("button");
let clicks = 0;

function countClick() {
  clicks++;
  console.log("click", clicks);
  if (clicks === 2) {
    btn.removeEventListener("click", countClick);        // same reference: works
    console.log("listener removed after 2 clicks");
  }
}
btn.addEventListener("click", countClick);

btn.addEventListener("click", () => console.log("anonymous listener"));
btn.removeEventListener("click", () => console.log("anonymous listener"));   // different function: no
effect

const controller = new AbortController();
btn.addEventListener("click", () => console.log("  signal listener A"), { signal: controller.signal
});
btn.addEventListener("click", () => console.log("  signal listener B"), { signal: controller.signal
});

for (let i = 1; i <= 3; i++) {
  if (i === 3) { controller.abort(); console.log("controller.abort() removed A and B"); }
  btn.click();                                           // simulate a click from code
}
```

BROWSER CONSOLE

```
click 1
anonymous listener
  signal listener A
  signal listener B
click 2
listener removed after 2 clicks
anonymous listener
  signal listener A
  signal listener B
controller.abort() removed A and B
anonymous listener
```

16.2 The event object

A listener receives one argument, the **event object**, conventionally named `event`, `evt` or `e`. Its class depends on the event (`MouseEvent`, `KeyboardEvent`, `InputEvent`, `SubmitEvent`…), and all inherit common properties from `Event`: `type`, `target`, `currentTarget`, `timeStamp`, `isTrusted` (true for real user actions), `bubbles`, and the methods `preventDefault()` and `stopPropagation()`.

16.2.1 target vs currentTarget

`event.target` is the element where the event actually **happened** (the innermost element clicked, for example a `<span>` inside a button). `event.currentTarget` is the element whose listener is **currently running**, the one you called `addEventListener` on. They differ whenever an event bubbles up from a child (16.4), which is the basis of event delegation (16.5). Inside a regular-function listener, `this` equals `currentTarget`; in arrow functions use `currentTarget` explicitly.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```html
<div class="card"><h3>JavaScript</h3><button class="btn"><span class="icon">★</span> Save</button>
</div>
```

CLICKING THE ICON INSIDE A CARD main.js

```
const card = document.querySelector(".card");
card.addEventListener("click", function (event) {
  console.log("target:       ", event.target.tagName, event.target.className || "");
  console.log("currentTarget:", event.currentTarget.tagName, event.currentTarget.className);
  console.log("this === currentTarget:", this === event.currentTarget);
});
```

BROWSER CONSOLE

```
target:        SPAN icon
currentTarget: DIV card
this === currentTarget: true
```

Simulated user actions, in order: clicks `.icon`.



TARGET IS WHERE THE CLICK HAPPENED; CURRENTTARGET IS WHERE THE LISTENER IS

16.2.2 type, key, clientX, clientY

Specific event classes add their own information. Mouse and pointer events carry coordinates: `clientX` / `clientY` relative to the visible viewport, `pageX` / `pageY` relative to the whole document, `offsetX` / `offsetY` relative to the target element, plus `button` (0 left, 1 middle, 2 right) and modifier flags `ctrlKey`, `shiftKey`, `altKey`, `metaKey`. Keyboard events carry `key` (the character or name produced, such as `"a"`, `"A"`, `"Enter"`, `"ArrowLeft"`) and `code` (the physical key, such as `"KeyA"`, independent of layout), plus `repeat` for held keys.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```html
<style>#area { margin: 20px; width: 200px; height: 80px; background: #eee; }</style>
<div id="area">Click area</div>
<input>
```

MOUSE COORDINATES AND KEYS main.js

```
const area = document.querySelector("#area");
area.addEventListener("click", (e) => {
  console.log(`${e.type}: clientX=${e.clientX} clientY=${e.clientY} offsetX=${e.offsetX}
offsetY=${e.offsetY} button=${e.button} shift=${e.shiftKey}`);
  console.log("class:", e.constructor.name, "| isTrusted:", e.isTrusted);
});

const input = document.querySelector("input");
input.addEventListener("keydown", (e) => {
  console.log(`keydown key=${JSON.stringify(e.key).padEnd(12)} code=${e.code.padEnd(10)}
shift=${e.shiftKey}`);
});
```

BROWSER CONSOLE

```
click: clientX=128 clientY=60 offsetX=100 offsetY=40 button=0 shift=false
class: PointerEvent | isTrusted: true
keydown key="a"           code=KeyA       shift=false
keydown key="Shift"       code=ShiftLeft  shift=true
keydown key="A"           code=KeyA       shift=true
keydown key="Enter"       code=Enter      shift=false
keydown key="ArrowLeft"   code=ArrowLeft  shift=false
```

Simulated user actions, in order: clicks `#area`; presses a in `input`; presses Shift+A in `input`; presses Enter in `input`; presses ArrowLeft in `input`.

16.3 Event types

16.3.1 Mouse events: click, dblclick, mouseover, mouseout, mouseenter, mousemove

A single click produces `mousedown`, `mouseup` and then `click`; two quick clicks also produce `dblclick`. A right-click fires `contextmenu`. When the pointer moves onto an element, `mouseover` fires (it **bubbles**, so it also fires again when moving between child elements) and `mouseenter` fires once (it does **not** bubble and ignores children); leaving fires `mouseout` and `mouseleave`. `mousemove` fires continuously while the pointer moves, many times per second.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<style>#box { width: 220px; height: 90px; padding: 20px; background: #f3ead2; } #inner { background:
#1B2A4A; color: white; }</style>
<div id="box">Box <div id="inner">inner</div></div>
<p id="outside">outside</p>
```

THE SEQUENCE OF MOUSE EVENTS main.js

```
const box = document.querySelector("#box");
const log = (e) => console.log(e.type.padEnd(11), "on", e.target.id || e.target.tagName);
["mousedown", "mouseup", "click", "dblclick", "mouseenter", "mouseover", "mouseleave", "mouseout",
"contextmenu"]
  .forEach((type) => box.addEventListener(type, log));

let moves = 0;
box.addEventListener("mousemove", () => moves++);
box.addEventListener("mouseleave", () => console.log(`(mousemove fired ${moves} times; the test
browser jumps straight to each point, a real mouse produces dozens)`));
```

BROWSER CONSOLE

```
mouseover   on box
mouseenter  on box
mouseout    on box
mouseover   on inner
mousedown   on inner
mouseup     on inner
click       on inner
mouseout    on inner
mouseleave  on box
(mousemove fired 3 times; the test browser jumps straight to each point, a real mouse produces dozens)
```

Simulated user actions, in order: hovers over `#box`; hovers over `#inner`; clicks `#inner`; hovers over `#outside`.

NOTE

Moving from the box into its child `#inner` fired `mouseout` on the box and `mouseover` again (bubbling from the child), but no extra `mouseenter` / `mouseleave`. Use `mouseenter` / `mouseleave` for hover effects to avoid this flicker.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<button>report.pdf</button>
```

DBLCLICK AND CONTEXTMENU main.js

```
const btn = document.querySelector("button");
btn.addEventListener("click", () => console.log("click"));
btn.addEventListener("dblclick", () => console.log("dblclick: open the file"));
btn.addEventListener("contextmenu", (e) => { e.preventDefault(); console.log("contextmenu: custom menu
instead of the browser's"); });

// simulate a double-click and a right-click from code
btn.dispatchEvent(new MouseEvent("click", { bubbles: true }));
btn.dispatchEvent(new MouseEvent("click", { bubbles: true }));
btn.dispatchEvent(new MouseEvent("dblclick", { bubbles: true }));
btn.dispatchEvent(new MouseEvent("contextmenu", { bubbles: true, cancelable: true, button: 2 }));
```

BROWSER CONSOLE

```
click
click
dblclick: open the file
contextmenu: custom menu instead of the browser's
```

16.3.2 Keyboard events: keydown, keyup

`keydown` fires when a key is pressed (and repeatedly while it is held, with `repeat: true`); `keyup` fires when it is released. Keyboard events go to the **focused** element, or to `document.body` when nothing is focused, so global shortcuts are usually registered on `document` or `window`. The old `keypress` event and the `keyCode` / `which` properties are deprecated: use `key` and `code`.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY)

HTML

```
<p>Press keys anywhere on the page</p>
```

GLOBAL KEYBOARD SHORTCUTS

main.js

```
document.addEventListener("keydown", (e) => {
  if ((e.ctrlKey || e.metaKey) && e.key === "s") {
    e.preventDefault();                                  // stop the browser's "Save page" dialog
    console.log("shortcut: save draft");
    return;
  }
  if (e.key === "Escape") console.log("Escape: close the dialog");
  else if (e.key.startsWith("Arrow")) console.log("navigate", e.key.replace("Arrow",
"").toLowerCase());
});
document.addEventListener("keyup", (e) => { if (e.key === "Shift") console.log("keyup: Shift
released"); });
```

BROWSER CONSOLE

```
shortcut: save draft
navigate right
navigate up
Escape: close the dialog
keyup: Shift released
```

Simulated user actions, in order: presses Control+s in `body`; presses ArrowRight in `body`; presses ArrowUp in `body`; presses Escape in `body`; presses Shift in `body`.

16.3.3 Form events: submit, input, change, focus, blur

`submit` fires on the **form** when it is submitted (by a submit button or Enter in a field); call `preventDefault()` to handle it with JavaScript instead of reloading the page. `input` fires on every change to a field's value (each keystroke, paste or cut). `change` fires when a change is **committed**: for text fields when they lose focus after being changed, for checkboxes, radios and selects immediately. `focus` and `blur` fire when a field gains and loses focus; they do not bubble, but their bubbling versions `focusin` and `focusout` do. `reset` fires when a form is reset.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY)

HTML

```
<form>
  <input name="name">
  <select name="plan"><option>basic</option><option>pro</option></select>
  <button>Join</button>
</form>
```

FORM EVENTS IN ORDER main.js

```
const form = document.querySelector("form");
const name = form.elements.name;
const plan = form.elements.plan;

name.addEventListener("focus", () => console.log("focus: name"));
name.addEventListener("input", (e) => console.log(`input: "${e.target.value}"`));
name.addEventListener("change", (e) => console.log(`change: committed "${e.target.value}"`));
name.addEventListener("blur", () => console.log("blur: name"));
plan.addEventListener("change", (e) => console.log("change: plan =", e.target.value));

form.addEventListener("submit", (e) => {
  e.preventDefault();                       // no page reload
  const data = Object.fromEntries(new FormData(form));
  console.log("submit:", JSON.stringify(data), "| submitter:", e.submitter.textContent);
});
```

BROWSER CONSOLE

```
focus: name
input: "Asha"
change: committed "Asha"
blur: name
change: plan = pro
submit: {"name":"Asha","plan":"pro"} | submitter: Join
```

Simulated user actions, in order: types "Asha" into `[name=name]`; clicks `select`; presses ArrowDown in `select`; clicks `button`.

NOTE

The test browser filled the field in one step, so a single `input` event appears; a person typing "Asha" would produce four `input` events, one per character.

16.3.4 Window events: load, DOMContentLoaded, resize, scroll, beforeunload

`DOMContentLoaded` fires on `document` when the HTML has been fully parsed and the DOM is ready, without waiting for images and stylesheets; it is the usual moment to start your code (scripts with `defer` or `type="module"` already run just before it). `load` fires on `window` after **everything**, including images, has loaded. `resize` and `scroll` fire on `window` (scroll also on scrollable elements), many times per second, so they are usually throttled (16.7). `beforeunload` lets a page ask for confirmation before the user leaves with unsaved changes; `visibilitychange` on `document` tells you when the tab is hidden or shown.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<textarea>unsaved draft</textarea>
<div style="height: 2000px"></div>
```

PAGE LIFE-CYCLE AND WINDOW EVENTS main.js

```
console.log("1. script runs, readyState =", document.readyState);
document.addEventListener("DOMContentLoaded", () => console.log("2. DOMContentLoaded: DOM ready,
readyState =", document.readyState));
window.addEventListener("load", () => console.log("3. load: images and styles loaded too, readyState
=", document.readyState));

window.addEventListener("resize", () => console.log("resize:", innerWidth + "x" + innerHeight));
window.addEventListener("scroll", () => console.log("scroll: scrollY =", Math.round(scrollY)), { once:
true });
window.addEventListener("beforeunload", (e) => {
  if (document.querySelector("textarea").value) e.preventDefault();   // browser shows "Leave site?"
});
setTimeout(() => {
  window.scrollTo(0, 300);
  window.resizeTo?.(600, 400);                                        // ignored for normal tabs
  window.dispatchEvent(new Event("resize"));                          // simulate a resize
}, 100);
```

BROWSER CONSOLE

```
1. script runs, readyState = loading
2. DOMContentLoaded: DOM ready, readyState = interactive
3. load: images and styles loaded too, readyState = complete
resize: 800x600
scroll: scrollY = 300
```

16.3.5 Pointer and touch events

Pointer events unify mouse, pen and touch input in one API: `pointerdown`, `pointermove`, `pointerup`, `pointercancel`, `pointerenter`/`pointerleave`. Their event object adds `pointerType` (`"mouse"`, `"pen"` or `"touch"`), `pointerId` (to track several fingers), `pressure` and size. `setPointerCapture` keeps sending events to an element even when the pointer leaves it, which is essential for dragging sliders. The older **touch events** (`touchstart`, `touchmove`, `touchend` with a `touches` list) are still supported, but pointer events are recommended for new code. The CSS property `touch-action` controls whether the browser scrolls or zooms on touch.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<style>#slider { position: relative; width: 300px; height: 20px; background: #ddd; touch-action: none;
} .knob { position: absolute; width: 10px; height: 20px; background: #C9A227; }</style>
<div id="slider"><div class="knob"></div></div>
```

A SLIDER DRIVEN BY POINTER EVENTS main.js

```
const slider = document.querySelector("#slider");
const knob = slider.querySelector(".knob");
let dragging = false;

slider.addEventListener("pointerdown", (e) => {
  dragging = true;
  console.log(`pointerdown: type=${e.pointerType} id=${e.pointerId}`);
  try {
    slider.setPointerCapture(e.pointerId);              // keep receiving moves outside the element
  } catch (err) {
    console.log("  (capture needs a real pointer:", err.name + ")");
  }
});
slider.addEventListener("pointermove", (e) => {
  if (!dragging) return;
  const pct = Math.round(Math.min(100, Math.max(0, (e.clientX - slider.getBoundingClientRect().left) /
slider.offsetWidth * 100)));
  knob.style.left = pct + "%";
  slider.dataset.value = pct;
});
slider.addEventListener("pointerup", () => {
  dragging = false;
  console.log("pointerup: value =", slider.dataset.value);
});

// simulate a touch drag from 10% to 75%
const r = slider.getBoundingClientRect();
const at = (type, pct) => slider.dispatchEvent(new PointerEvent(type, { bubbles: true, pointerId: 7,
pointerType: "touch", clientX: r.left + r.width * pct, clientY: r.top + 5 }));
at("pointerdown", 0.1); at("pointermove", 0.4); at("pointermove", 0.75); at("pointerup", 0.75);
```

BROWSER CONSOLE

```
pointerdown: type=touch id=7
  (capture needs a real pointer: NotFoundError)
pointerup: value = 75
```

NOTE

`setPointerCapture` only accepts a pointer that is really pressed on the device, so it refuses the simulated pointer created from code (the try/catch reports this). On a real device, the same code captures the mouse, pen or finger and works unchanged.

16.3.6 Clipboard and drag-and-drop events

Clipboard events `copy`, `cut` and `paste` fire when the user uses the clipboard; `event.clipboardData` lets you read pasted data or replace what gets copied (after `preventDefault()`). The asynchronous `navigator.clipboard` API (Module 21) copies text from code, for example for a "Copy" button. **Drag and drop** uses `draggable="true"` on the source element and a sequence of events: `dragstart` on the source (store data in `event.dataTransfer`), `dragenter` / `dragover` on possible targets (call `preventDefault()` in `dragover` to allow dropping), `drop` on the target (read the data), and `dragend` on the source. Dropping files from the desktop delivers them in `dataTransfer.files`.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<input id="phone" placeholder="Phone">
<p id="code">Your coupon: LBF-2026-WELCOME</p>
```

CLEANING PASTED INPUT AND CONTROLLING COPY main.js

```
const field = document.querySelector("#phone");
field.addEventListener("paste", (e) => {
  e.preventDefault();
  const pasted = e.clipboardData.getData("text/plain");
  field.value = pasted.replace(/\D/g, "").slice(-10);          // keep the last 10 digits only
  console.log(`paste: "${pasted}" -> cleaned to "${field.value}"`);
});
document.querySelector("#code").addEventListener("copy", (e) => {
  e.preventDefault();
  e.clipboardData.setData("text/plain", "LBF-2026-WELCOME");     // control what is copied
  console.log("copy: replaced the copied text with the coupon code");
});

// simulate a paste and a copy (real ones come from Ctrl+V / Ctrl+C)
const paste = new DataTransfer();
paste.setData("text/plain", "+91 98450-12345");
field.dispatchEvent(new ClipboardEvent("paste", { clipboardData: paste, bubbles: true, cancelable:
true }));
document.querySelector("#code").dispatchEvent(new ClipboardEvent("copy", { clipboardData: new
DataTransfer(), bubbles: true, cancelable: true }));
```

BROWSER CONSOLE

```
paste: "+91 98450-12345" -> cleaned to "9845012345"
copy: replaced the copied text with the coupon code
```

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<div class="column" data-name="To do"><b>To do</b>
  <p class="task" id="t1" draggable="true">Write notes</p>
  <p class="task" id="t2" draggable="true">Record video</p>
</div>
<div class="column" data-name="Done"><b>Done</b></div>
```

A SMALL DRAG-AND-DROP BOARD

main.js

```
const items = document.querySelectorAll(".task");
const columns = document.querySelectorAll(".column");

items.forEach((item) => item.addEventListener("dragstart", (e) => {
  e.dataTransfer.setData("text/plain", item.id);
  console.log("dragstart:", item.textContent);
}));
columns.forEach((col) => {
  col.addEventListener("dragover", (e) => e.preventDefault());       // allow dropping here
  col.addEventListener("drop", (e) => {
    e.preventDefault();
    const id = e.dataTransfer.getData("text/plain");
    col.append(document.getElementById(id));
    console.log(`drop: ${id} -> ${col.dataset.name}`);
  });
});

// simulate dragging task t1 into the "Done" column
const dt = new DataTransfer();
document.querySelector("#t1").dispatchEvent(new DragEvent("dragstart", { dataTransfer: dt, bubbles:
true }));
const done = document.querySelector("[data-name=Done]");
done.dispatchEvent(new DragEvent("dragover", { dataTransfer: dt, bubbles: true, cancelable: true }));
done.dispatchEvent(new DragEvent("drop", { dataTransfer: dt, bubbles: true, cancelable: true }));
```

BROWSER CONSOLE

```
dragstart: Write notes
drop: t1 -> Done
--- page shows ---
To do

Record video

Done

Write notes
```

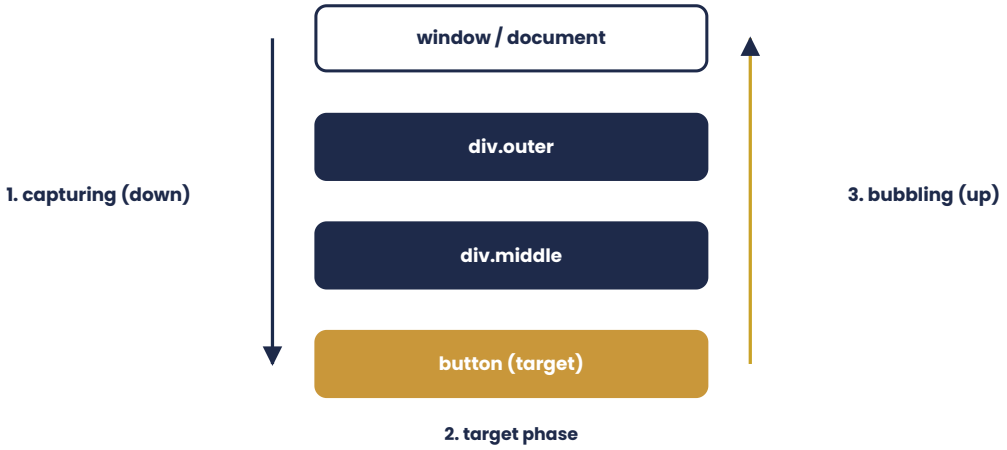
Drag event	Fires on	Typical use
`dragstart`	the dragged element	store data with dataTransfer.setData
`dragenter` / `dragleave`	a potential drop target	highlight / un-highlight the target
`dragover`	the target, continuously	call preventDefault() to allow a drop
`drop`	the target	read dataTransfer.getData or .files
`dragend`	the dragged element	clean up styles

16.4 Event flow

16.4.1 Capturing and bubbling phases

When an event happens on a nested element, it does not just fire there. It travels in three **phases**. In the **capturing phase**, it travels down from `window` through every ancestor to the target. In the **target phase**, it reaches the element itself. In the **bubbling phase**, it travels back up through every ancestor to `window`. Listeners run in the bubbling phase by default; pass `{ capture: true }` (or `true` as the third argument)

to run in the capturing phase. Most events bubble; exceptions include `focus`, `blur`, `mouseenter`, `mouseleave`, `load` and `scroll` on elements.



THE THREE PHASES OF EVENT PROPAGATION

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<div id="outer">outer <div id="middle">middle <button id="btn">Click</button></div></div>
```

WATCHING AN EVENT TRAVEL DOWN AND UP main.js

```
const ids = ["outer", "middle", "btn"];
for (const id of ids) {
  const el = document.getElementById(id);
  el.addEventListener("click", (e) => console.log(`capture  ${id.padEnd(6)} phase=${e.eventPhase}`), {
capture: true });
  el.addEventListener("click", (e) => console.log(`bubble   ${id.padEnd(6)} phase=${e.eventPhase}`));
}
document.addEventListener("click", () => console.log("bubble   document"));
```

BROWSER CONSOLE

```
capture  outer  phase=1
capture  middle phase=1
capture  btn    phase=2
bubble   btn    phase=2
bubble   middle phase=3
bubble   outer  phase=3
bubble   document
```

Simulated user actions, in order: clicks `#btn`.

NOTE

`eventPhase` is 1 for capturing, 2 at the target and 3 for bubbling. At the target itself, capture listeners run before bubble listeners in modern browsers.

16.4.2 stopPropagation()

`event.stopPropagation()` stops the event from travelling any further (to ancestors when bubbling, or to descendants when capturing); other listeners on the current element still run. `stopImmediatePropagation()` also stops the remaining listeners on the same element. Use these sparingly: stopping propagation breaks event delegation, analytics and "click outside to close" logic that rely on events reaching the document.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<div class="card"><h3>Headphones</h3><button class="like">♥ Like</button><button
class="delete">Delete</button></div>
```

STOPPING A CLICK FROM REACHING THE CARD main.js

```
const card = document.querySelector(".card");
const like = document.querySelector(".like");

card.addEventListener("click", () => console.log("card clicked: open the product page"));
like.addEventListener("click", (e) => {
  e.stopPropagation();                          // liking should not also open the page
  console.log("like clicked: added to wishlist");
});
like.addEventListener("click", () => console.log("second listener on the like button still runs"));

const del = document.querySelector(".delete");
del.addEventListener("click", (e) => { e.stopImmediatePropagation(); console.log("delete: confirm
first"); });
del.addEventListener("click", () => console.log("never runs: stopped immediately"));
```

BROWSER CONSOLE

```
card clicked: open the product page
like clicked: added to wishlist
second listener on the like button still runs
delete: confirm first
```

Simulated user actions, in order: clicks `h3`; clicks `.like`; clicks `.delete`.

16.4.3 preventDefault()

Many events have a **default action** performed by the browser: following a link, submitting a form, checking a checkbox, typing a character, opening the context menu, scrolling on wheel or touch. `event.preventDefault()` cancels that action (if the event is `cancelable`) while the event keeps propagating normally. `event.defaultPrevented` tells later listeners that it was cancelled. This is a different job from `stopPropagation`: one stops the browser's behaviour, the other stops other listeners.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<a href="/next-page">Next page</a>
<label><input type="checkbox" id="terms"> I agree</label>
<input id="digits" placeholder="digits only">
```

CANCELLING DEFAULT ACTIONS main.js

```
document.querySelector("a").addEventListener("click", (e) => {
  e.preventDefault();                                  // do not navigate
  console.log("link click handled by JS, navigation prevented; href was",
e.currentTarget.getAttribute("href"));
});

document.querySelector("#terms").addEventListener("click", (e) => {
  e.preventDefault();                                  // the box cannot be ticked
  setTimeout(() => console.log("checkbox click prevented, checked is", e.target.checked));   // read
after the browser undoes it
});

document.querySelector("#digits").addEventListener("keydown", (e) => {
  if (e.key.length === 1 && !/\d/.test(e.key)) {
    e.preventDefault();                                // block letters
    console.log("blocked key:", e.key);
  }
});
document.querySelector("#digits").addEventListener("keyup", (e) => console.log("  field value now:",
JSON.stringify(e.target.value)));
document.addEventListener("click", (e) => console.log("document still hears it; defaultPrevented =",
e.defaultPrevented));
```

BROWSER CONSOLE

```
link click handled by JS, navigation prevented; href was /next-page
document still hears it; defaultPrevented = true
document still hears it; defaultPrevented = true
checkbox click prevented, checked is false
  field value now: "4"
blocked key: x
  field value now: "4"
  field value now: "42"
```

Simulated user actions, in order: clicks `a`; clicks `#terms`; presses 4 in `#digits`; presses x in `#digits`; presses 2 in `#digits`.

16.4.4 Listener options: once, passive, capture

The third argument of `addEventListener` can be an options object. `once: true` removes the listener after its first call. `capture: true` runs it in the capturing phase. `passive: true` promises that the listener will never call `preventDefault()`, which lets the browser start scrolling immediately without waiting for your code; it is important for `touchstart`, `touchmove` and `wheel` listeners, and browsers make those passive by default on `window` and `document`. `signal` attaches an AbortSignal for removal (16.1.3).

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<button>Start</button>
```

ONCE AND PASSIVE

main.js

```
const btn = document.querySelector("button");
btn.addEventListener("click", () => console.log("welcome tour starts (once only)"), { once: true });
btn.addEventListener("click", () => console.log("normal click listener"));
btn.click(); btn.click();

document.addEventListener("wheel", (e) => {
  e.preventDefault();                                  // ignored in a passive listener
  console.log("passive wheel listener: defaultPrevented =", e.defaultPrevented);
}, { passive: true });
document.dispatchEvent(new WheelEvent("wheel", { deltaY: 100, cancelable: true }));
```

BROWSER CONSOLE

```
welcome tour starts (once only)
normal click listener
normal click listener
✖ Unable to preventDefault inside passive event listener invocation.
passive wheel listener: defaultPrevented = false
```

NOTE

Calling `preventDefault()` inside a passive listener has no effect (Chrome prints the warning shown in the console output), which is exactly the promise that lets scrolling stay smooth.

16.5 Event delegation

16.5.1 Handling events for many or dynamic elements with one listener

Instead of attaching a listener to each of 500 list items, attach **one** listener to their common ancestor and let events bubble up to it. Inside, use `event.target.closest(selector)` to find which item (and which button) was involved, and check that it is inside the container. This **event delegation** uses less memory, needs no set-up when items are added or removed later (new items work automatically), and keeps the logic in one place. It works for every bubbling event; for `focus` / `blur` use `focusin` / `focusout`.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY)

HTML

```
<ul id="cart">
  <li data-item="pen">Pen <span class="qty">2</span> <button data-action="inc">+</button><button data-
action="dec">-</button><button data-action="remove">x</button></li>
  <li data-item="book">Book <span class="qty">1</span> <button data-action="inc">+</button><button
data-action="dec">-</button><button data-action="remove">x</button></li>
</ul>
```

ONE LISTENER FOR A WHOLE CART

main.js

```
const list = document.querySelector("#cart");

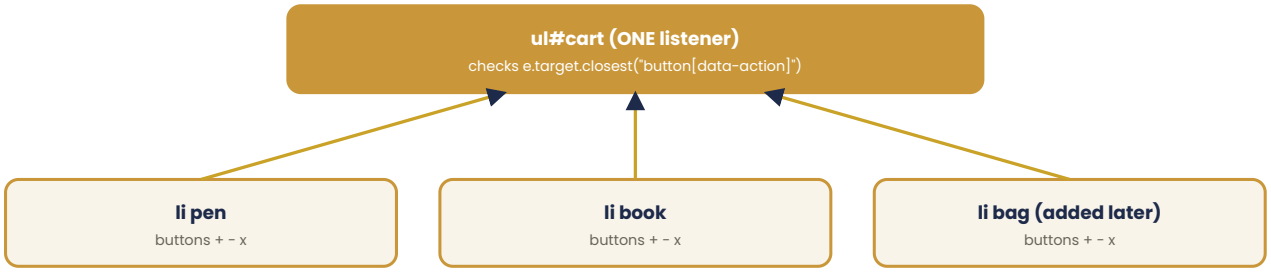
list.addEventListener("click", (e) => {
  const button = e.target.closest("button[data-action]");
  if (!button || !list.contains(button)) return;         // click was not on an action button
  const row = button.closest("li");
  const qty = row.querySelector(".qty");
  switch (button.dataset.action) {
    case "inc": qty.textContent = Number(qty.textContent) + 1; break;
    case "dec": qty.textContent = Math.max(1, Number(qty.textContent) - 1); break;
    case "remove": row.remove(); break;
  }
  console.log(`${button.dataset.action} on ${row.dataset.item} ->`, [...list.children].map((li) =>
`${li.dataset.item}x${li.querySelector(".qty").textContent}`).join(", ") || "(empty)");
});

// an item added later works with no extra listener
list.insertAdjacentHTML("beforeend", '<li data-item="bag">Bag <span class="qty">1</span> <button data-
action="inc">+</button><button data-action="dec">-</button><button data-action="remove">x</button>
</li>');
```

BROWSER CONSOLE

```
inc on pen -> penx3, bookx1, bagx1
remove on book -> penx3, bagx1
inc on bag -> penx3, bagx2
```

Simulated user actions, in order: clicks `li[data-item=pen] [data-action=inc]`; clicks `li[data-item=book] [data-action=remove]`; clicks `li[data-item=bag] [data-action=inc]`; clicks `li[data-item=bag] .qty`.



CLICKS BUBBLE FROM ANY ROW TO THE SINGLE LISTENER

16.6 Custom events

16.6.1 CustomEvent and dispatchEvent()

You can create and fire your own events to let parts of an application communicate without depending on each other directly. `new CustomEvent(type, { detail, bubbles, cancelable })` creates an event that carries any data in `detail`, and `element.dispatchEvent(event)` fires it synchronously, running all listeners before it returns. `dispatchEvent` returns `false` if a cancelable event was cancelled with `preventDefault()`, allowing listeners to veto an action. Use lower-case names with a prefix or colon, such as `"cart:add"`, to avoid clashes with built-in events. Any object can become an event source by extending `EventTarget`.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<header>Cart: <span id="cart-count">0</span></header>
<button data-product="Mouse" data-price="799">Add mouse</button>
<button data-product="Laptop Pro" data-price="189999">Add laptop</button>
```

COMMUNICATING WITH CUSTOM EVENTS main.js

```
const cartIcon = document.querySelector("#cart-count");
const buttons = document.querySelectorAll("[data-product]");

// the product cards announce what happened; they do not know who listens
buttons.forEach((b) => b.addEventListener("click", () => {
  const ok = b.dispatchEvent(new CustomEvent("cart:add", {
    bubbles: true, cancelable: true,
    detail: { product: b.dataset.product, price: Number(b.dataset.price) },
  }));
  console.log(`dispatch returned ${ok} for ${b.dataset.product}`);
}));

// independent listeners higher up react to the custom event
let count = 0;
document.addEventListener("cart:add", (e) => {
  if (e.detail.price > 50000) { e.preventDefault(); console.log("  veto: price above limit"); return;
}
  cartIcon.textContent = ++count;
  console.log(`  header: cart has ${count} item(s)`);
});
document.addEventListener("cart:add", (e) => {
  if (!e.defaultPrevented) console.log(`  analytics: add_to_cart ${e.detail.product} Rs
${e.detail.price}`);
});

// an EventTarget subclass: events without the DOM
class Timer extends EventTarget {
  start(n) { for (let i = n; i >= 0; i--) this.dispatchEvent(new CustomEvent(i ? "tick" : "done", {
detail: i })); }
}
const t = new Timer();
t.addEventListener("tick", (e) => console.log("tick", e.detail));
t.addEventListener("done", () => console.log("timer done"));
t.start(2);
```

BROWSER CONSOLE

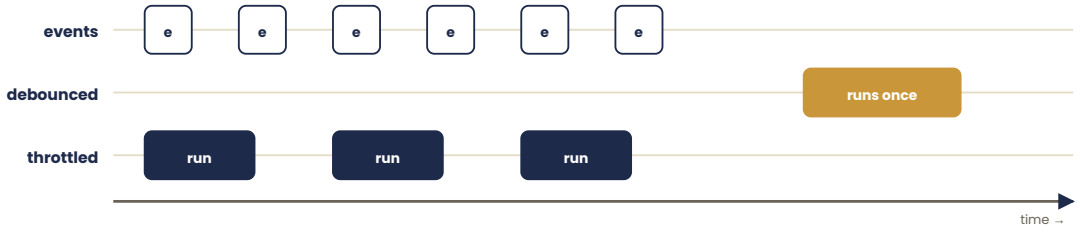
```
tick 2
tick 1
timer done
  header: cart has 1 item(s)
  analytics: add_to_cart Mouse Rs 799
dispatch returned true for Mouse
  veto: price above limit
dispatch returned false for Laptop Pro
```

Simulated user actions, in order: clicks `[data-product=Mouse]`; clicks `[data-product="Laptop Pro"]`.

16.7 Performance

16.7.1 Debouncing input and throttling scroll events

Some events fire very rapidly: `input` on every keystroke, `scroll`, `resize`, `mousemove` and `pointermove` dozens of times per second. Running expensive work (a network search, layout calculations) on each one wastes resources and makes the page sluggish. **Debouncing** (Module 11) waits until the events **stop** for a given time and then runs once: ideal for search-as-you-type and auto-save. **Throttling** runs at most once per interval while events continue: ideal for scroll position, infinite scrolling and resize. For visual updates, `requestAnimationFrame` is a natural throttle to the screen's frame rate, and passive listeners keep scrolling smooth.



SIX RAPID EVENTS: DEBOUNCE RUNS ONCE AT THE END; THROTTLE RUNS AT A STEADY RATE

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY)

HTML

```
<input id="search" placeholder="Search courses">
<div style="height: 5000px"></div>
```

DEBOUNCED SEARCH AND THROTTLED SCROLL

main.js

```
function debounce(fn, ms) {
  let id;
  return (...args) => { clearTimeout(id); id = setTimeout(() => fn(...args), ms); };
}
function throttle(fn, ms) {
  let last = 0;
  return (...args) => {
    const now = performance.now();
    if (now - last >= ms) { last = now; fn(...args); }
  };
}

const search = document.querySelector("#search");
let inputEvents = 0;
search.addEventListener("input", () => inputEvents++);
search.addEventListener("input", debounce((e) => {
  console.log(`debounced search for "${e.target.value}" after ${inputEvents} input events`);
}, 200));

let scrollEvents = 0, handled = 0;
window.addEventListener("scroll", () => scrollEvents++, { passive: true });
window.addEventListener("scroll", throttle(() => handled++, 100), { passive: true });

// simulate typing one character every 40 ms, then scrolling for about 0.6 s
const text = "closures";
[...text].forEach((ch, i) => setTimeout(() => {
  search.value += ch;
  search.dispatchEvent(new Event("input", { bubbles: true }));
}, i * 40));
let y = 0;
const scroller = setInterval(() => { window.scrollTo(0, (y += 40)); }, 16);
setTimeout(() => {
  clearInterval(scroller);
  console.log(`scroll: ${scrollEvents > handled * 2 ? "many" : scrollEvents} events, throttled handler
ran ${handled <= 8 ? "only a few" : handled} times`);
}, 650);
```

BROWSER CONSOLE

```
debounced search for "closures" after 8 input events
scroll: many events, throttled handler ran only a few times
```

NOTE

The exact counts of scroll events depend on the machine, so the example reports them qualitatively. The debounced search ran exactly once, after the last keystroke, instead of eight times.

Events signal that something happened; listeners attached with addEventListener (preferred over inline attributes and on-properties) receive an event object, and are removed with removeEventListener (same function), once or an AbortSignal. target is where the event happened, currentTarget where the listener is. Mouse events carry coordinates and buttons; keyboard events carry key and code; form events include submit (preventDefault to stay on the page), input, change, focus and blur; window events include DOMContentLoaded, load, resize, scroll and beforeunload; pointer events unify mouse, pen and touch; clipboard and drag-and-drop events use clipboardData and dataTransfer. Events travel down in the capturing phase and up in the bubbling phase; stopPropagation stops the travel, preventDefault cancels the browser's default action, and options set once, passive and capture. Event delegation handles many or future elements with one listener and closest(). CustomEvent with detail and dispatchEvent decouples components. Debounce and throttle keep rapid events cheap.

PRACTICE CHECKPOINT

Build an **image gallery** with a **modal**, **keyboard navigation** and **event delegation**, plus a **live search box** that is debounced.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<input id="search" placeholder="Search photos">
<div id="gallery"></div>
<div id="modal" hidden><button class="close">×</button><p class="caption"></p></div>
```

SAMPLE SOLUTION main.js

```
const photos = [
  { src: "beach.jpg", title: "Goa beach", tags: "sea sand travel" },
  { src: "hills.jpg", title: "Ooty hills", tags: "green tea mountain" },
  { src: "fort.jpg", title: "Mysuru palace", tags: "history night lights" },
  { src: "lake.jpg", title: "Dal lake", tags: "water boat mountain" },
];
const gallery = document.querySelector("#gallery");
const modal = document.querySelector("#modal");
const caption = modal.querySelector(".caption");
let visible = photos.map((_, i) => i);
let current = -1;

function render() {
  gallery.replaceChildren(...visible.map((i) => {
    const fig = document.createElement("figure");
    fig.dataset.index = i;
    fig.innerHTML = `<img alt=""><figcaption></figcaption>`;
    fig.querySelector("img").alt = photos[i].title;          // text set safely
    fig.querySelector("figcaption").textContent = photos[i].title;
    return fig;
  }));
}

function openModal(i) {
  current = i;
  caption.textContent = `${photos[i].title} (${visible.indexOf(i) + 1}/${visible.length})`;
  modal.hidden = false;
  console.log("modal:", caption.textContent);
}
function closeModal() { modal.hidden = true; current = -1; console.log("modal closed"); }
function step(dir) {
  const pos = (visible.indexOf(current) + dir + visible.length) % visible.length;
  openModal(visible[pos]);
}

gallery.addEventListener("click", (e) => {                    // delegation: one listener
  const fig = e.target.closest("figure");
  if (fig) openModal(Number(fig.dataset.index));
});
modal.addEventListener("click", (e) => { if (e.target === modal || e.target.matches(".close"))
closeModal(); });
document.addEventListener("keydown", (e) => {                 // keyboard navigation
  if (modal.hidden) return;
  if (e.key === "ArrowRight") step(1);
  else if (e.key === "ArrowLeft") step(-1);
  else if (e.key === "Escape") closeModal();
});

function debounce(fn, ms) { let id; return (...a) => { clearTimeout(id); id = setTimeout(() =>
```

```
fn(...a), ms); }; }
document.querySelector("#search").addEventListener("input", debounce((e) => {
  const q = e.target.value.trim().toLowerCase();
  visible = photos.map((_, i) => i).filter((i) => (photos[i].title + " " +
photos[i].tags).toLowerCase().includes(q));
  render();
  console.log(`search "${q}": ${visible.map((i) => photos[i].title).join(", ") || "no results"}`);
}, 250));

render();
```

BROWSER CONSOLE

```
modal: Ooty hills (2/4)
modal: Mysuru palace (3/4)
modal: Dal lake (4/4)
modal: Goa beach (1/4)
modal closed
search "mountain": Ooty hills, Dal lake
modal: Dal lake (2/2)
modal: Ooty hills (1/2)
modal closed
```

Simulated user actions, in order: clicks `figure[data-index="1"]`; presses ArrowRight in `body`; presses ArrowRight in `body`; presses ArrowRight in `body`; presses Escape in `body`; types "mountain" into `#search`; waits 400 ms; clicks `figure[data-index="3"]`; presses ArrowLeft in `body`; clicks `.close`.

MODULE 17 · INTERMEDIATE

Asynchronous JavaScript

Minimum duration **4 days** | Level **Intermediate**

JavaScript runs on a single thread, yet a web page can wait for a network response, a timer and a user click at the same time without freezing. This module explains how: blocking versus non-blocking code, the event loop with its call stack, Web APIs, task queue and microtask queue, then the three generations of asynchronous code (callbacks, promises and async/await), the promise combinators, Promise.withResolvers and Promise.try, promisifying old APIs, top-level await, async iteration, and cancelling work with AbortController and timeouts.

IN THIS MODULE

17.1 Synchronous vs asynchronous code
17.2 The event loop
17.3 Callbacks
17.4 Promises
17.5 async and await
17.6 Cancellation and timeouts

NOTE

Most examples run in Node.js 22, which uses the same event loop model and promise behaviour as browsers. Delays are kept short (tens of milliseconds) so that the examples run quickly; real network requests take far longer.

17.1 Synchronous vs asynchronous code

17.1.1 Blocking vs non-blocking

Synchronous code runs one statement after another; each must finish before the next starts. That is fine for calculations, but some operations take a long time while the computer simply **waits**: a network request (hundreds of milliseconds), reading a file, a timer, waiting for the user. If JavaScript waited synchronously, the single thread would be **blocked**: no clicks handled, no animation, a frozen page. **Asynchronous** code starts the slow operation, hands it to the environment (the browser or Node.js), and continues immediately; when the result is ready, a function you supplied is called. This is **non-blocking**.

BLOCKING VERSUS NON-BLOCKING WAITING main.js

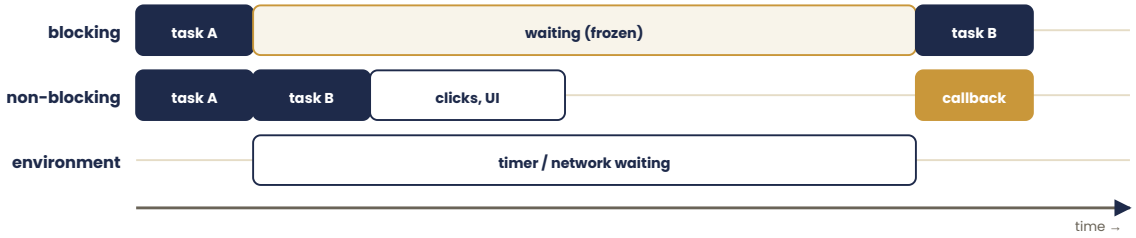
```
function blockFor(ms) {                    // simulates a slow synchronous task
  const end = Date.now() + ms;
  while (Date.now() < end) {}               // the thread can do nothing else
}

console.log("1. order placed");
let t = Date.now();
blockFor(200);
console.log(`2. blocking wait finished after ~${Math.round((Date.now() - t) / 100) * 100} ms; nothing
else could run meanwhile`);

console.log("3. start a non-blocking wait");
t = Date.now();
setTimeout(() => console.log(`5. timer callback after ~${Math.round((Date.now() - t) / 100) * 100}
ms`), 200);
console.log("4. ...the program carried on immediately");
```

OUTPUT

```
1. order placed
2. blocking wait finished after ~200 ms; nothing else could run meanwhile
3. start a non-blocking wait
4. ...the program carried on immediately
5. timer callback after ~200 ms
```



NON-BLOCKING CODE HANDS THE WAIT TO THE ENVIRONMENT AND STAYS RESPONSIVE

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<button id="sync">Sync work</button> <button id="async">Async work</button>
<p id="status"></p>
```

WHY BLOCKING IS UNACCEPTABLE IN A BROWSER main.js

```
const status = document.querySelector("#status");
document.querySelector("#sync").addEventListener("click", () => {
  const t = performance.now();
  while (performance.now() - t < 300) {}       // freezes the whole tab for 300 ms
  status.textContent = "sync done";
  console.log("sync button: page was frozen, no repaint or input possible for 300 ms");
});
document.querySelector("#async").addEventListener("click", () => {
  status.textContent = "loading...";            // the page can repaint at once
  setTimeout(() => { status.textContent = "async done"; console.log("async button: page stayed
responsive while waiting"); }, 300);
});
```

BROWSER CONSOLE

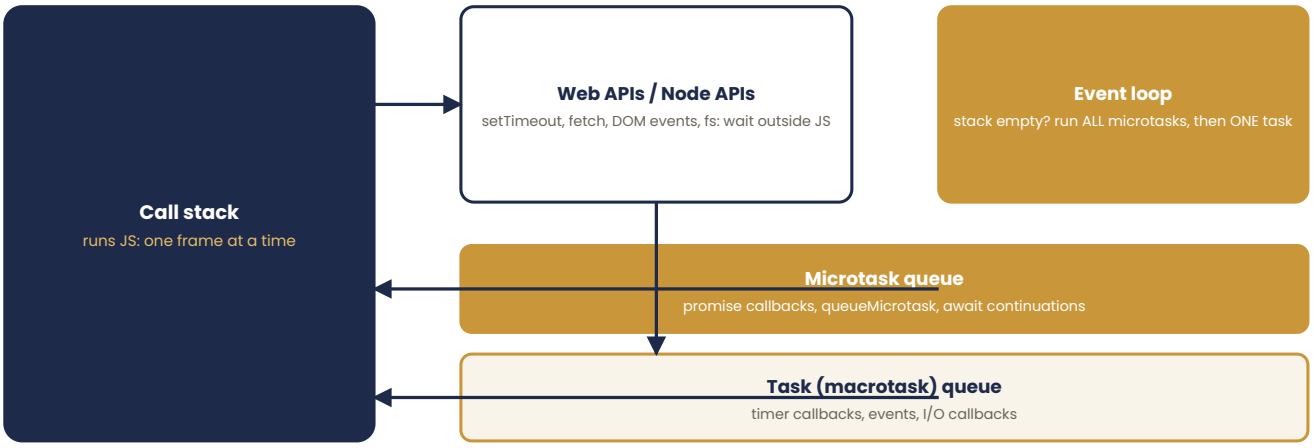
```
sync button: page was frozen, no repaint or input possible for 300 ms
async button: page stayed responsive while waiting
```

Simulated user actions, in order: clicks `#sync`; clicks `#async`; waits 400 ms.

17.2 The event loop

17.2.1 Call stack, Web APIs, callback (task) queue

Four parts work together. The **call stack** (Module 11) runs JavaScript, one frame at a time. **Web APIs** (in Node.js, the libuv library and C++ APIs) are provided by the environment, not by the JavaScript engine: timers, network, DOM events, file system. They do the waiting **outside** the JavaScript thread. When an operation completes, its callback is placed in a **task queue** (also called the callback queue or macrotask queue). The **event loop** is a simple, endless cycle: whenever the call stack is empty, take the next task from the queue and run it to completion.



THE PIECES OF THE JAVASCRIPT CONCURRENCY MODEL

A CALLBACK WAITS FOR THE STACK TO BE EMPTY

main.js

```
console.log("A: script starts (call stack)");

setTimeout(() => console.log("D: timer callback (from the task queue)"), 0);

console.log("B: setTimeout returned at once; the timer runs in the environment");
for (let i = 0; i < 3; i++) console.log(`C${i}: still synchronous work on the stack`);
```

OUTPUT

```
A: script starts (call stack)
B: setTimeout returned at once; the timer runs in the environment
C0: still synchronous work on the stack
C1: still synchronous work on the stack
C2: still synchronous work on the stack
D: timer callback (from the task queue)
```

17.2.2 Microtask queue vs macrotask queue

There are two kinds of queue. **Macrotasks** (tasks) are timer callbacks, I/O callbacks, user events and `MessageChannel` messages. **Microtasks** are promise reactions (`then` / `catch` / `finally` callbacks), the continuation of an `async` function after `await`, `queueMicrotask(fn)` and `MutationObserver` callbacks. The crucial rule: after each macrotask (including the initial script), the event loop runs **all** microtasks, including any new microtasks added while doing so, before taking the next macrotask. In browsers, rendering may happen between macrotasks, never during microtask processing.

Macrotasks (task queue)	Microtasks (microtask queue)
`setTimeout`, `setInterval` callbacks	`promise.then / catch / finally` callbacks
DOM events (click, input…)	code after `await` in an async function
network and file I/O callbacks	`queueMicrotask(fn)`
`MessageChannel`, `setImmediate` (Node.js)	`MutationObserver` callbacks
ONE is run per loop iteration	ALL are run until the queue is empty

17.2.3 Order of execution: sync code, microtasks, macrotasks

Putting the rules together gives a reliable way to predict output: (1) run all synchronous code to the end; (2) run every microtask, in the order queued; (3) run one macrotask; (4) again run all microtasks; repeat. The famous puzzle below is a classic interview question.

THE CLASSIC ORDERING PUZZLE

main.js

```
console.log("1 sync start");

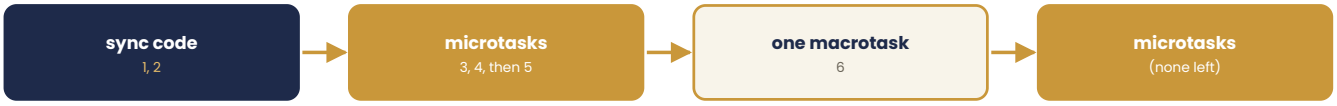
setTimeout(() => console.log("6 timeout (macrotask)"), 0);

Promise.resolve()
  .then(() => console.log("3 promise then (microtask)"))
  .then(() => console.log("5 second then (microtask queued by the first)"));

queueMicrotask(() => console.log("4 queueMicrotask (microtask)"));

console.log("2 sync end");
```

```
OUTPUT
1 sync start
2 sync end
3 promise then (microtask)
4 queueMicrotask (microtask)
5 second then (microtask queued by the first)
6 timeout (macrotask)
```



SYNCHRONOUS CODE, THEN ALL MICROTASKS, THEN ONE MACROTASK

MICROTASKS RUN BETWEEN MACROTASKS main.js

```
setTimeout(() => {
  console.log("timeout 1");
  Promise.resolve().then(() => console.log("  microtask inside timeout 1"));
}, 0);
setTimeout(() => console.log("timeout 2"), 0);

async function run() {
  console.log("async fn: before await (synchronous)");
  await null;
  console.log("async fn: after await (microtask)");
}
run();
console.log("end of script");
```

OUTPUT

```
async fn: before await (synchronous)
end of script
async fn: after await (microtask)
timeout 1
  microtask inside timeout 1
timeout 2
```

WATCH OUT

Because all microtasks run before the next macrotask, an endless chain of microtasks (a promise callback that keeps scheduling another) starves the task queue: timers never fire and the page never repaints, just like an infinite loop.

17.2.4 setTimeout(fn, 0) explained

`setTimeout(fn, 0)` does **not** run `fn` immediately. It asks the environment to queue `fn` as a macrotask after **at least** 0 ms, so it runs only after the current script and all pending microtasks have finished. The delay is a minimum, never a guarantee: if the stack is busy for 500 ms, a 10 ms timer fires after 500 ms. Browsers also clamp nested timers to at least 4 ms and throttle timers in background tabs. Developers use `setTimeout(fn, 0)` to defer work until after the current code (and, in browsers, to let the page repaint first).

THE DELAY IS A MINIMUM, NOT A PROMISE main.js

```
const start = Date.now();
setTimeout(() => console.log(`timer asked for 10 ms, ran after about ${Math.round((Date.now() - start)
/ 100) * 100} ms`), 10);

const end = Date.now() + 300;
while (Date.now() < end) {}                    // the stack is busy for 300 ms
console.log("busy loop finished");

setTimeout(() => console.log("0 ms timer: after the script"), 0);
Promise.resolve().then(() => console.log("promise: before any timer"));
console.log("last line of the script");
```

OUTPUT

```
busy loop finished
last line of the script
promise: before any timer
timer asked for 10 ms, ran after about 300 ms
0 ms timer: after the script
```

17.3 Callbacks

17.3.1 Asynchronous callbacks

The first generation of asynchronous code passed a **callback** function to be called when the work finishes. Event listeners and timers still work this way, and Node.js APIs traditionally use the **error-first callback** convention: the callback receives `(error, result)`, where `error` is `null` on success. The callback runs later, so its result cannot be `return`ed to the original caller; any code that needs the result must go **inside** the callback.

ERROR-FIRST CALLBACKS

main.js

```
// simulated database calls with error-first callbacks
const db = {
  users: { 1: { id: 1, name: "Asha" } },
  posts: { 1: [{ id: 10, title: "Closures explained" }, { id: 11, title: "Event loop notes" }] },
};
function getUser(id, callback) {
  setTimeout(() => {
    const user = db.users[id];
    if (user) callback(null, user);
    else callback(new Error(`User ${id} not found`));
  }, 20);
}

console.log("requesting user 1...");
getUser(1, (err, user) => {
  if (err) return console.log("error:", err.message);
  console.log("got user:", user.name);
});
getUser(9, (err, user) => {
  if (err) return console.log("error:", err.message);
  console.log("got user:", user.name);
});
const result = getUser(1, () => {});
console.log("the function itself returns:", result, "(the data is not available yet)");
```

OUTPUT

```
requesting user 1...
the function itself returns: undefined (the data is not available yet)
got user: Asha
error: User 9 not found
```

17.3.2 Callback hell and inversion of control

When each step depends on the previous one (load the user, then their posts, then the comments on the first post), callbacks nest inside callbacks, drifting to the right in a shape nicknamed the **pyramid of doom** or **callback hell**. Error handling must be repeated at every level, and the flow is hard to read and change. A deeper problem is **inversion of control**: you hand your continuation to someone else's function and must trust it to call the callback exactly once, with the right arguments, and not synchronously one time and asynchronously another. A buggy or malicious library might call it twice or never. Promises were designed to solve both problems.

THE PYRAMID OF DOOM main.js

```
const data = {
  users: { 1: { id: 1, name: "Asha" } },
  posts: { 1: [{ id: 10, title: "Closures explained" }] },
  comments: { 10: ["Great post!", "Very clear, thanks"] },
};
const later = (fn) => setTimeout(fn, 10);
function getUser(id, cb) { later(() => (data.users[id] ? cb(null, data.users[id]) : cb(new Error("no
user")))); }
function getPosts(userId, cb) { later(() => cb(null, data.posts[userId] ?? [])); }
function getComments(postId, cb) { later(() => (data.comments[postId] ? cb(null,
data.comments[postId]) : cb(new Error("no comments")))); }

getUser(1, (err, user) => {
  if (err) return console.log("error:", err.message);
  getPosts(user.id, (err, posts) => {
    if (err) return console.log("error:", err.message);
    getComments(posts[0].id, (err, comments) => {
      if (err) return console.log("error:", err.message);
      console.log(`${user.name} -> "${posts[0].title}" -> ${comments.length} comments:
${comments.join(" | ")}`);
    });
  });
});
```

```
OUTPUT
Asha -> "Closures explained" -> 2 comments: Great post! | Very clear, thanks
```

TRUSTING A CALLBACK TO BE CALLED ONCE main.js

```
// inversion of control: a third-party function we do not control
function buggyPayment(amount, onSuccess) {
  onSuccess(`txn-${amount}`);
  onSuccess(`txn-${amount}`);            // bug: calls our callback twice!
}
let charged = 0;
buggyPayment(499, (txn) => { charged += 499; console.log("charged customer for", txn); });
console.log("total charged:", charged, "(customer billed twice)");

// a promise can only settle once, so the same bug is harmless
const once = new Promise((resolve) => { resolve("txn-499"); resolve("txn-499"); });
once.then((txn) => console.log("promise version charged once for", txn));
```

```
OUTPUT
charged customer for txn-499
charged customer for txn-499
total charged: 998 (customer billed twice)
promise version charged once for txn-499
```

17.4 Promises

A **promise** is an object representing the eventual result of an asynchronous operation: a placeholder for a value that is not available yet. Instead of passing a callback **into** a function, the function **returns** a promise, and you attach reactions to it. This gives control back to you, guarantees that each reaction runs at most once and always asynchronously, lets results be chained in a flat sequence, and lets one handler catch errors from many steps. `fetch`, most modern browser APIs and the `fs/promises` module of Node.js all return promises.

17.4.1 Creating promises: new Promise((resolve, reject) => ...)

The `Promise` constructor takes an **executor** function, which runs **immediately** (synchronously) and receives two functions: call `resolve(value)` when the operation succeeds and `reject(error)` when it fails (always with an Error object). Only the first call counts. If the executor throws, the promise is rejected. `Promise.resolve(v)` and `Promise.reject(err)` create already-settled promises.

WRAPPING ASYNCHRONOUS WORK IN A PROMISE main.js

```
function delay(ms, value) {
  return new Promise((resolve) => setTimeout(() => resolve(value), ms));
}

function rollDice() {
  return new Promise((resolve, reject) => {
    console.log("  executor runs immediately");
    setTimeout(() => {
      const n = [4, 6, 1][rollDice.calls++ % 3];          // fixed sequence for repeatable output
      if (n === 1) reject(new Error("rolled a 1, you lose"));
      else resolve(n);
    }, 20);
  });
}
rollDice.calls = 0;

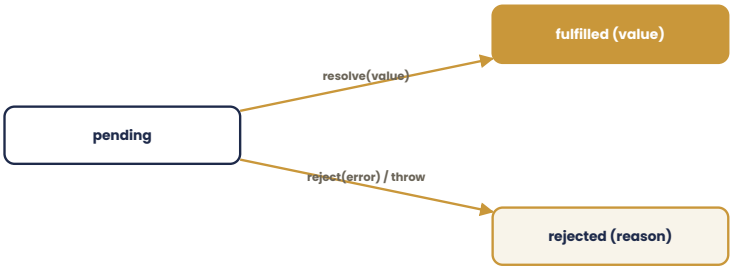
console.log("before");
const p = rollDice();
console.log("after: p is", p);                            // still pending
p.then((n) => console.log("resolved with", n));
rollDice().then((n) => console.log("resolved with", n));
rollDice().then((n) => console.log("resolved with", n), (err) => console.log("rejected:",
err.message));
delay(50, "done").then((v) => console.log("delay:", v));
```

OUTPUT

```
before
  executor runs immediately
after: p is Promise { <pending> }
  executor runs immediately
  executor runs immediately
resolved with 4
resolved with 6
rejected: rolled a 1, you lose
delay: done
```

17.4.2 States: pending, fulfilled, rejected

A promise is always in one of three states. It starts **pending**. It becomes **fulfilled** with a value when resolved, or **rejected** with a reason (an error) when rejected. Fulfilled and rejected are together called **settled**, and a settled promise never changes state again. Reactions attached **after** a promise has settled still run (asynchronously) with its stored result, so it does not matter whether you attach them before or after the work completes.



A PROMISE SETTLES ONCE AND THEN NEVER CHANGES

OBSERVING THE THREE STATES main.js

```
const fast = Promise.resolve(42);
const slow = new Promise((r) => setTimeout(() => r("slow value"), 30));
const failed = Promise.reject(new Error("broken"));
failed.catch(() => {});                                  // handled, so no warning

console.log(fast, slow, failed);                         // Node.js shows the state

setTimeout(() => {
  console.log("later:", slow);
  slow.then((v) => console.log("attached after settling still gets:", v));
}, 50);
```

OUTPUT

```
Promise { 42 } Promise { <pending> } Promise {
  <rejected> Error: broken
      at Object.<anonymous> (main.js:3:31)
}
later: Promise { 'slow value' }
attached after settling still gets: slow value
```

17.4.3 then(), catch(), finally()

`promise.then(onFulfilled, onRejected?)` registers what to do with the value (and optionally with an error). `promise.catch(onRejected)` is shorthand for `then(undefined, onRejected)`. `promise.finally(onFinally)` runs in both cases, receives no argument, and passes the original result or error through unchanged, which is ideal for clean-up such as hiding a spinner. Each of these methods returns a **new** promise, which is what makes chaining possible.

REACTING TO SUCCESS AND FAILURE main.js

```
function fetchCourse(id) {
  return new Promise((resolve, reject) =>
    setTimeout(() => (id === "js" ? resolve({ id, title: "JavaScript", seats: 12 }) : reject(new
Error(`course ${id} not found`))), 20));
}

console.log("loading...");
fetchCourse("js")
  .then((course) => console.log("then:", course.title, "-", course.seats, "seats"))
  .catch((err) => console.log("catch:", err.message))
  .finally(() => console.log("finally: hide spinner (js)"));

fetchCourse("cobol")
  .then((course) => console.log("then:", course.title))
  .catch((err) => console.log("catch:", err.message))
  .finally(() => console.log("finally: hide spinner (cobol)"));
```

OUTPUT

```
loading...
then: JavaScript - 12 seats
finally: hide spinner (js)
catch: course cobol not found
finally: hide spinner (cobol)
```

17.4.4 Promise chaining

Whatever a `then` callback returns becomes the value of the promise that `then` returned. If it returns a plain value, the next `then` receives it; if it returns a **promise**, the chain waits for that promise and passes on its result. This turns nested callbacks into a flat sequence of steps. If any step throws or returns a rejected promise, the following `then` callbacks are skipped until the next `catch`, which can recover by returning a value. Always `return` the inner promise: forgetting it breaks the chain.

A FLAT CHAIN INSTEAD OF A PYRAMID

main.js

```
const later = (v, ms = 10) => new Promise((r) => setTimeout(() => r(v), ms));
const getUser = (id) => later({ id, name: "Asha" });
const getPosts = (user) => later([{ id: 10, title: "Closures explained" }, { id: 11, title: "Event
loop notes" }]);
const getComments = (post) => (post ? later(["Great post!", "Very clear"]) : Promise.reject(new
Error("no post")));

let userName;
getUser(1)
  .then((user) => { userName = user.name; return getPosts(user); })   // return the promise
  .then((posts) => getComments(posts[0]))
  .then((comments) => console.log(`${userName}: first post has ${comments.length} comments`))
  .catch((err) => console.log("error anywhere above:", err.message));

// values, errors and recovery travelling through a chain
Promise.resolve(2)
  .then((n) => n * 10)                          // plain value -> 20
  .then((n) => { throw new Error(`failed at ${n}`); })
  .then(() => console.log("skipped"))
  .catch((err) => { console.log("caught:", err.message); return 0; })   // recover
  .then((n) => console.log("chain continues with", n));
```

OUTPUT

```
caught: failed at 20
chain continues with 0
Asha: first post has 2 comments
```

WATCH OUT

A promise rejection that nobody handles produces an **unhandled rejection**: a console error in browsers, and in Node.js the process exits with an error. Every chain should end with a `catch` (or be awaited inside `try`).

17.4.5 Promise.all(), allSettled(), race(), any()

Four static methods combine several promises into one. `Promise.all` waits for **all** to fulfil and gives an array of results in the original order, but rejects as soon as **any one** rejects (fail-fast). `Promise.allSettled` always waits for all and reports each outcome as `{ status, value }` or `{ status, reason }`. `Promise.race` settles like the **first** promise to settle, whether success or failure (useful for timeouts). `Promise.any` fulfils with the first **success**, ignoring failures, and rejects with an `AggregateError` only if all fail.

THE FOUR COMBINATORS

main.js

```
const ok = (v, ms) => new Promise((r) => setTimeout(() => r(v), ms));
const fail = (msg, ms) => new Promise((_, j) => setTimeout(() => j(new Error(msg)), ms));

Promise.all([ok("user", 30), ok("posts", 50), ok("ads", 10)])
  .then((r) => console.log("all:        ", r, "(after the slowest, 50 ms)"));

Promise.all([ok("user", 30), fail("posts API down", 20), ok("ads", 10)])
  .catch((e) => console.log("all (fail): ", e.message, "(at 20 ms, without waiting for the others)"));

Promise.allSettled([ok("user", 30), fail("posts API down", 20)])
  .then((r) => console.log("allSettled: ", JSON.stringify(r.map((x) => x.status === "fulfilled" ?
x.value : x.reason.message))));

Promise.race([ok("mirror A", 40), ok("mirror B", 15)])
  .then((v) => console.log("race:       ", v));

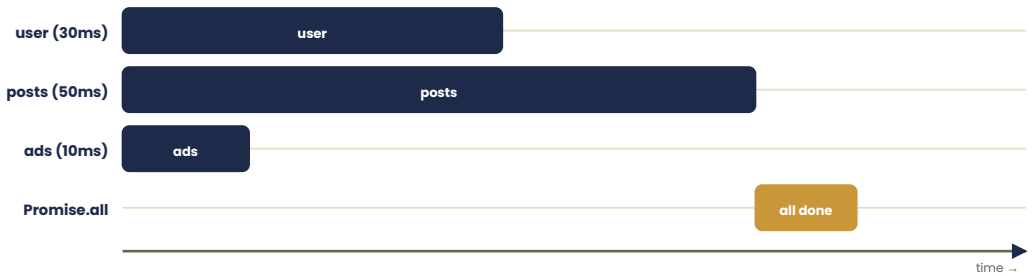
Promise.any([fail("server 1", 10), ok("server 2", 25), ok("server 3", 35)])
  .then((v) => console.log("any:        ", v));

Promise.any([fail("x", 5), fail("y", 5)])
  .catch((e) => console.log("any (all failed):", e.constructor.name, e.errors.map((x) => x.message)));
```

OUTPUT

```
any (all failed): AggregateError [ 'x', 'y' ]
race:        mirror B
all (fail):  posts API down (at 20 ms, without waiting for the others)
any:         server 2
allSettled:  ["user","posts API down"]
all:         [ 'user', 'posts', 'ads' ] (after the slowest, 50 ms)
```

Method	Fulfils when	Rejects when	Typical use
`Promise.all`	all fulfil (array of values, in order)	the first one rejects	load several things that are all required
`Promise.allSettled`	all settle (array of outcomes)	never	show what succeeded and what failed
`Promise.race`	the first to settle fulfils	the first to settle rejects	timeouts, fastest response
`Promise.any`	the first fulfils	all reject (AggregateError)	try several mirrors or sources



PROMISE.ALL RUNS REQUESTS IN PARALLEL: TOTAL TIME IS THE SLOWEST ONE, NOT THE SUM

17.4.6 Promise.withResolvers() and Promise.try()

`Promise.withResolvers()` (ES2024) returns `{ promise, resolve, reject }`: a promise together with the functions that settle it, so it can be resolved from outside the executor, for example from an event handler

or a queue. `Promise.try(fn, ...args)` (ES2025) calls `fn` and always returns a promise: if `fn` returns a value it is wrapped, if it returns a promise that promise is followed, and if it **throws synchronously** the error becomes a rejection. That lets one `catch` handle both synchronous and asynchronous failures. Promise.try is available in current browsers and Node.js 24; the example below runs in the browser.

PROMISE.WITHRESOLVERS IN NODE.JS main.js

```
// A "one-time event" helper built with withResolvers
function nextMessage(emitter) {
  const { promise, resolve } = Promise.withResolvers();
  emitter.onmessage = resolve;                // settled later, from outside
  return promise;
}
const chat = {};
nextMessage(chat).then((msg) => console.log("first message received:", msg));
setTimeout(() => chat.onmessage("Hello from Ravi"), 20);

// A simple async queue: resolve waiting consumers when items arrive
const waiting = [];
function take() { const r = Promise.withResolvers(); waiting.push(r.resolve); return r.promise; }
function put(item) { waiting.shift()?.(item); }
take().then((x) => console.log("consumer 1 got", x));
take().then((x) => console.log("consumer 2 got", x));
setTimeout(() => { put("order #1"); put("order #2"); }, 30);
```

OUTPUT

```
first message received: Hello from Ravi
consumer 1 got order #1
consumer 2 got order #2
```

PROMISE.TRY IN THE BROWSER main.js

```
function parseSettings(text) {
  return JSON.parse(text);                    // throws synchronously on bad input
}

// Without Promise.try, a synchronous throw escapes the .catch()
try {
  Promise.resolve(parseSettings("{bad json")).catch(() => console.log("never reached"));
} catch (e) {
  console.log("plain call: threw synchronously ->", e.name);
}

// With Promise.try, every failure arrives in .catch()
Promise.try(parseSettings, "{bad json")
  .then((s) => console.log("settings:", s))
  .catch((e) => console.log("Promise.try: rejection ->", e.name));

Promise.try(parseSettings, '{"theme":"dark"}')
  .then((s) => console.log("Promise.try: value ->", JSON.stringify(s)));
```

BROWSER CONSOLE

```
plain call: threw synchronously -> SyntaxError
Promise.try: value -> {"theme":"dark"}
Promise.try: rejection -> SyntaxError
```

17.4.7 Promisifying callback functions

Older APIs that take error-first callbacks can be wrapped in functions that return promises; this is called **promisifying**. Write a function that returns `new Promise` and, inside, calls the old API with a callback that

calls `reject(err)` or `resolve(result)`. Node.js provides `util.promisify` to do this automatically, and most of its modules now also offer promise versions directly (`fs/promises`, `timers/promises`).

TURNING CALLBACKS INTO PROMISES main.js

```
// an old-style API we cannot change
function readSensor(id, callback) {
  setTimeout(() => (id > 0 ? callback(null, { id, temp: 21 + id }) : callback(new Error("invalid
sensor id"))), 10);
}

// 1. promisify by hand
function readSensorAsync(id) {
  return new Promise((resolve, reject) => {
    readSensor(id, (err, data) => (err ? reject(err) : resolve(data)));
  });
}
readSensorAsync(3).then((d) => console.log("by hand:", d));
readSensorAsync(-1).catch((e) => console.log("by hand error:", e.message));

// 2. a generic promisify
const promisify = (fn) => (...args) => new Promise((res, rej) => fn(...args, (err, val) => (err ?
rej(err) : res(val))));
promisify(readSensor)(5).then((d) => console.log("generic:", d));

// 3. Node.js built-ins
const util = require("node:util");
util.promisify(readSensor)(7).then((d) => console.log("util.promisify:", d));
const { setTimeout: sleep } = require("node:timers/promises");
sleep(40, "awake").then((v) => console.log("timers/promises:", v));
```

OUTPUT

```
by hand: { id: 3, temp: 24 }
by hand error: invalid sensor id
generic: { id: 5, temp: 26 }
util.promisify: { id: 7, temp: 28 }
timers/promises: awake
```

17.5 async and await

`async` / `await` (ES2017) is syntax on top of promises that lets asynchronous code be written in the same top-to-bottom style as synchronous code, with ordinary `try` / `catch`, loops and variables. It does not block the thread: `await` pauses only the current async function, letting everything else run, and resumes it (as a microtask) when the promise settles.

17.5.1 Async functions always return promises

Putting `async` before a function (declaration, expression, arrow or method) changes it in one way: it **always returns a promise**. A returned value becomes the fulfilment value, a thrown error becomes a rejection, and a returned promise is followed. Callers therefore use `await` or `.then()` to get the result.

ASYNC WRAPS RESULTS IN PROMISES

main.js

```
async function getNumber() { return 42; }
async function failSoon() { throw new Error("async failure"); }
const getName = async () => "Asha";
class Api { async status() { return "ok"; } }

const p = getNumber();
console.log("returns:", p, "| is a Promise:", p instanceof Promise);
p.then((v) => console.log("value:", v));
failSoon().catch((e) => console.log("rejection:", e.message));
getName().then((n) => console.log("arrow:", n));
new Api().status().then((s) => console.log("method:", s));
```

OUTPUT

```
returns: Promise { 42 } | is a Promise: true
value: 42
rejection: async failure
arrow: Asha
method: ok
```

17.5.2 await and sequential vs parallel awaiting

`await promise` waits for the promise and evaluates to its value, or throws its error. It can be used inside async functions (and at the top level of modules, 17.5.4). Writing several `await`s one after another runs the operations **sequentially**, each starting only after the previous has finished: correct when each step needs the previous result, but needlessly slow for independent operations. For independent work, start all the promises first and then await them together with `Promise.all`, so they run **in parallel**.

SEQUENTIAL VERSUS PARALLEL AWAITING

main.js

```
const later = (v, ms) => new Promise((r) => setTimeout(() => r(v), ms));
const getUser = () => later({ id: 1, name: "Asha" }, 30);
const getCourses = () => later(["JavaScript", "React"], 30);
const getNotices = () => later(["Holiday on Friday"], 30);

async function dashboardSequential() {
  const t = Date.now();
  const user = await getUser();          // 30 ms
  const courses = await getCourses();    // +30 ms
  const notices = await getNotices();    // +30 ms
  console.log(`sequential: ${user.name}, ${courses.length} courses, ${notices.length} notice
(~${Math.round((Date.now() - t) / 30) * 30} ms)`);
}

async function dashboardParallel() {
  const t = Date.now();
  const [user, courses, notices] = await Promise.all([getUser(), getCourses(), getNotices()]);
  console.log(`parallel:   ${user.name}, ${courses.length} courses, ${notices.length} notice
(~${Math.round((Date.now() - t) / 30) * 30} ms)`);
}

async function main() {
  await dashboardSequential();
  await dashboardParallel();

  // dependent steps must be sequential: each needs the previous result
  const user = await getUser();
  const posts = await later([`${user.name}'s first post`], 10);
  console.log("dependent steps:", posts[0]);
}
main();
```

OUTPUT

```
sequential: Asha, 2 courses, 1 notice  (~90 ms)
parallel:   Asha, 2 courses, 1 notice  (~30 ms)
dependent steps: Asha's first post
```



INDEPENDENT REQUESTS: AWAIT THEM TOGETHER

WATCH OUT

`await` inside `forEach` does not wait: `forEach` ignores the promises its callback returns. Use `for...of` with `await` for sequential processing, or `Promise.all(items.map(async ...))` for parallel processing.

LOOPS AND AWAIT

main.js

```
const later = (v, ms) => new Promise((r) => setTimeout(() => r(v), ms));
const ids = [3, 1, 2];

async function withForEach() {
  ids.forEach(async (id) => { await later(null, id * 10); console.log("  forEach item", id); });
  console.log("forEach 'finished' before any item!");
}
async function withForOf() {
  for (const id of ids) { await later(null, id * 10); console.log("  for...of item", id); }
  console.log("for...of finished after all items, in order");
}
async function withMap() {
  const results = await Promise.all(ids.map(async (id) => { await later(null, id * 10); return id *
100; }));
  console.log("Promise.all + map (parallel, results in order):", results);
}
(async () => { await withForEach(); await later(null, 50); await withForOf(); await withMap(); })();
```

OUTPUT

```
forEach 'finished' before any item!
  forEach item 1
  forEach item 2
  forEach item 3
  for...of item 3
  for...of item 1
  for...of item 2
for...of finished after all items, in order
Promise.all + map (parallel, results in order): [ 300, 100, 200 ]
```

17.5.3 Error handling with try/catch

Because `await` throws when a promise rejects, async functions handle errors with ordinary `try` / `catch` / `finally`, covering several awaits in one block. Errors that are not caught make the async function's own promise reject, so the caller can handle them. For a single call where you want a value even on failure, attaching `.catch()` to the awaited promise is a concise alternative.

TRY/CATCH WITH AWAIT

main.js

```
const later = (v, ms = 10) => new Promise((r) => setTimeout(() => r(v), ms));
async function getOrder(id) {
  await later();
  if (id === 0) throw new Error("Order id 0 is invalid");
  if (id > 100) throw new RangeError(`Order ${id} does not exist`);
  return { id, total: 1499 };
}

async function showOrder(id) {
  console.log(`loading order ${id}...`);
  try {
    const order = await getOrder(id);
    console.log("  total:", order.total);
  } catch (err) {
    console.log(`  could not load: ${err.name}: ${err.message}`);
  } finally {
    console.log("  spinner hidden");
  }
}

async function main() {
  await showOrder(7);
  await showOrder(500);
  const order = await getOrder(0).catch(() => ({ id: 0, total: 0 }));   // fallback value
  console.log("with .catch fallback:", order);
  await getOrder(0);                                                    // not caught here...
}
main().catch((err) => console.log("...so the caller catches it:", err.message));
```

OUTPUT

```
loading order 7...
  total: 1499
  spinner hidden
loading order 500...
  could not load: RangeError: Order 500 does not exist
  spinner hidden
with .catch fallback: { id: 0, total: 0 }
...so the caller catches it: Order id 0 is invalid
```

17.5.4 Top-level await in modules

In **ES modules** (files loaded with `type="module"` in browsers, or `.mjs` files and `"type": "module"` packages in Node.js), `await` can be used at the top level, outside any function (ES2022). The module pauses there, and any module that imports it waits until it has finished. It is useful for loading configuration, initialising a database connection or importing a module conditionally. It is not available in classic scripts or CommonJS files.

TOP-LEVEL AWAIT IN AN ES MODULE (MAIN.MJS) main.js

```
const later = (v, ms) => new Promise((r) => setTimeout(() => r(v), ms));

console.log("module starts");
const config = await later({ apiUrl: "https://api.lbf.in", retries: 3 }, 20);    // no async function
needed
console.log("config loaded:", config.apiUrl);

const { default: os } = await import("node:os");                                 // dynamic import,
awaited
console.log("platform module loaded:", typeof os.platform);

const [a, b] = await Promise.all([later("A", 10), later("B", 10)]);
console.log("parallel at top level:", a, b);
```

OUTPUT

```
module starts
config loaded: https://api.lbf.in
platform module loaded: function
parallel at top level: A B
```

17.5.5 Async iteration with for await...of

Some data arrives over time in pieces: lines of a large file, pages of API results, chunks of a network stream. An **async iterable** has a `[Symbol.asyncIterator]()` method whose `next()` returns a **promise** of `{ value, done }`. `for await (const x of source)` consumes it, awaiting each item. The easiest way to create one is an **async generator** (`async function*`), which can both `await` and `yield`. `for await` also works over an array of promises, and `Array.fromAsync` collects an async iterable into an array.

PAGINATED DATA WITH AN ASYNC GENERATOR main.js

```
const later = (v, ms = 15) => new Promise((r) => setTimeout(() => r(v), ms));

// an async generator that fetches pages until there are no more
async function* fetchPages() {
  const pages = { 1: ["Asha", "Ravi"], 2: ["Meera", "John"], 3: ["Zoya"] };
  for (let page = 1; pages[page]; page++) {
    console.log(`  (requesting page ${page})`);
    yield await later(pages[page]);
  }
}

async function main() {
  for await (const users of fetchPages()) {
    console.log("received", users.join(", "));
  }

  for await (const result of [later("first", 30), later("second", 10)]) {
    console.log("array of promises, in array order:", result);
  }

  const all = await Array.fromAsync(fetchPages());
  console.log("Array.fromAsync:", all.flat());
}
main();
```

OUTPUT

```
  (requesting page 1)
received Asha, Ravi
  (requesting page 2)
received Meera, John
  (requesting page 3)
received Zoya
array of promises, in array order: first
array of promises, in array order: second
  (requesting page 1)
  (requesting page 2)
  (requesting page 3)
Array.fromAsync: [ 'Asha', 'Ravi', 'Meera', 'John', 'Zoya' ]
```

Node.js streams and the browser's `ReadableStream` (for example `response.body` from `fetch`) are async iterables, so a large download or file can be processed chunk by chunk with `for await` without loading it all into memory.

17.6 Cancellation and timeouts

17.6.1 AbortController and AbortSignal.timeout()

Promises cannot be cancelled by themselves, but many APIs accept an **AbortSignal**. Create an `AbortController`, pass its `signal` to `fetch`, event listeners, Node.js timers or your own functions, and call `controller.abort(reason)` to cancel: the operation stops and its promise rejects with an `AbortError` (or the given reason). `AbortSignal.timeout(ms)` creates a signal that aborts automatically after a time limit, with a `TimeoutError`, and `AbortSignal.any([...])` combines several signals. Typical uses: cancel a search request when the user types again, cancel downloads when leaving a page, and stop waiting for a slow server.

CANCELLING AND TIMING OUT WORK

main.js

```
// Your own cancellable function: listen to the signal
function slowTask(ms, signal) {
  return new Promise((resolve, reject) => {
    if (signal?.aborted) return reject(signal.reason);
    const id = setTimeout(() => resolve(`finished after ${ms} ms`), ms);
    signal?.addEventListener("abort", () => { clearTimeout(id); reject(signal.reason); }, { once: true
});
  });
}

async function main() {
  // 1. manual cancellation
  const controller = new AbortController();
  setTimeout(() => controller.abort(), 20);
  try { await slowTask(200, controller.signal); }
  catch (e) { console.log("1. cancelled:", e.name, "-", e.message); }

  // 2. automatic timeout
  try { await slowTask(200, AbortSignal.timeout(30)); }
  catch (e) { console.log("2. timed out:", e.name, "-", e.message); }

  // 3. fast enough: completes normally
  console.log("3.", await slowTask(10, AbortSignal.timeout(100)));

  // 4. combine "user cancelled" with a timeout
  const user = new AbortController();
  const signal = AbortSignal.any([user.signal, AbortSignal.timeout(1000)]);
  setTimeout(() => user.abort(new Error("user pressed Cancel")), 15);
  try { await slowTask(500, signal); }
  catch (e) { console.log("4. combined signal:", e.message); }

  // 5. a timeout with Promise.race (works with any promise)
  const timeout = (ms) => new Promise((_, j) => setTimeout(() => j(new Error(`no answer within ${ms}
ms`)), ms));
  try { await Promise.race([slowTask(300), timeout(25)]); }
  catch (e) { console.log("5. race timeout:", e.message); }
}
main();
```

OUTPUT

```
1. cancelled: AbortError - This operation was aborted
2. timed out: TimeoutError - The operation was aborted due to timeout
3. finished after 10 ms
4. combined signal: user pressed Cancel
5. race timeout: no answer within 25 ms
```

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY)

HTML

```
<input id="q" placeholder="Search">
```

CANCELLING STALE FETCH REQUESTS WHILE THE USER TYPES

main.js

```
const input = document.querySelector("#q");
let controller;

input.addEventListener("input", async () => {
  controller?.abort();                                  // cancel the previous request
  controller = new AbortController();
  const q = input.value;
  try {
    const res = await fetch(`/api/search.json?q=${encodeURIComponent(q)}`, { signal: controller.signal
});
    const data = await res.json();
    console.log(`results for "${q}":`, data.results.join(", "));
  } catch (e) {
    if (e.name === "AbortError") console.log(`request for "${q}" cancelled (user kept typing)`);
    else console.log("error:", e.message);
  }
});
for (const q of ["ja", "jav", "java"]) {
  input.value = q;
  input.dispatchEvent(new Event("input"));
}
```

BROWSER CONSOLE

```
request for "ja" cancelled (user kept typing)
request for "jav" cancelled (user kept typing)
results for "java": Java, JavaScript, Java Spring
```

JavaScript is single-threaded; asynchronous, non-blocking APIs hand waiting to the environment. The event loop runs synchronous code, then all microtasks (promise callbacks, await continuations, queueMicrotask), then one macrotask (timers, events, I/O), repeatedly; setTimeout(fn, 0) runs after the current code and pending microtasks. Callbacks (often error-first) lead to nesting and inversion of control. Promises are pending, then fulfilled or rejected once; then, catch and finally return new promises, so steps chain flatly and one catch handles errors. Promise.all (all or fail-fast), allSettled (every outcome), race (first to settle) and any (first success) combine promises; withResolvers exposes resolve/reject and Promise.try catches synchronous throws; promisify wraps callback APIs. async functions always return promises; await pauses only the function; run independent work in parallel with Promise.all; handle errors with try/catch; modules allow top-level await; for await...of consumes async iterables and async generators. AbortController, AbortSignal.timeout and AbortSignal.any cancel work.

PRACTICE CHECKPOINT

Predict the output order of **ten event-loop puzzles**, and rewrite a callback-based "load user, then posts, then comments" flow with **promises** and then with **async/await**.

SAMPLE SOLUTION

main.js

```
const out = [];
const log = (s) => out.push(s);

// ---- Ten puzzles in one run (expected order: 1..19 as numbered) ----
log("1");                                                  // P1 sync first
setTimeout(() => log("15"), 0);                            // P2 macrotask
Promise.resolve().then(() => log("4"));                    // P3 microtask
queueMicrotask(() => log("5"));                            // P4 microtask in queue order
new Promise((res) => { log("2"); res(); }).then(() => log("6")); // P5 executor is synchronous
```

```
(async () => { log("3"); await null; log("7"); })();             // P6 code before await is sync
Promise.resolve().then(() => log("8")).then(() => log("11"));    // P7 chained then waits a turn
setTimeout(() => { log("16"); Promise.resolve().then(() => log("17")); }, 0);   // P8 microtask inside
a task
setTimeout(() => log("19"), 5);                                  // P9 longer delay comes later
Promise.reject(new Error("x")).catch(() => log("9")).finally(() => log("12"));  // P10 catch, then
finally
Promise.resolve().then(() => log("10")).then(() => log("13")).then(() => log("14"));
setTimeout(() => log("18"), 0);
setTimeout(() => { console.log("puzzle order:", out.join(" ")); console.log("all in order:",
out.every((v, i) => Number(v) === i + 1)); }, 20);

// ---- The same flow three ways ----
const db = {
  users: { 1: { id: 1, name: "Asha" } },
  posts: { 1: [{ id: 10, title: "Closures explained" }] },
  comments: { 10: ["Great post!", "Very clear"] },
};
// callback versions
const cbUser = (id, cb) => setTimeout(() => (db.users[id] ? cb(null, db.users[id]) : cb(new Error("no
user"))), 5);
const cbPosts = (uid, cb) => setTimeout(() => cb(null, db.posts[uid] ?? []), 5);
const cbComments = (pid, cb) => setTimeout(() => cb(null, db.comments[pid] ?? []), 5);

cbUser(1, (e, user) => {
  if (e) return console.log(e.message);
  cbPosts(user.id, (e, posts) => {
    if (e) return console.log(e.message);
    cbComments(posts[0].id, (e, comments) => {
      if (e) return console.log(e.message);
      console.log(`callbacks:   ${user.name} / ${posts[0].title} / ${comments.length} comments`);
    });
  });
});

// promise versions (promisified)
const promisify = (fn) => (...a) => new Promise((res, rej) => fn(...a, (e, v) => (e ? rej(e) :
res(v))));
const getUser = promisify(cbUser), getPosts = promisify(cbPosts), getComments = promisify(cbComments);

let u, p0;
getUser(1)
  .then((user) => { u = user; return getPosts(user.id); })
  .then((posts) => { p0 = posts[0]; return getComments(p0.id); })
  .then((comments) => console.log(`promises:    ${u.name} / ${p0.title} / ${comments.length}
comments`))
  .catch((e) => console.log("promise error:", e.message));

// async/await version
async function loadFeed(id) {
  try {
    const user = await getUser(id);
    const [first] = await getPosts(user.id);
    const comments = await getComments(first.id);
    console.log(`async/await: ${user.name} / ${first.title} / ${comments.length} comments`);
  } catch (e) {
    console.log("async/await error:", e.message);
  }
}
loadFeed(1);
loadFeed(2);
```

OUTPUT

```
async/await error: no user
puzzle order: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
all in order: true
callbacks:   Asha / Closures explained / 2 comments
promises:    Asha / Closures explained / 2 comments
async/await: Asha / Closures explained / 2 comments
```

MODULE 18 · INTERMEDIATE

Working with APIs and browser storage

Minimum duration **3 days** | Level **Intermediate**

Almost every real application talks to a server. This module explains the HTTP basics a front-end developer needs (the client-server model, methods, status codes, headers, REST and JSON), then shows how to use the Fetch API for every kind of request, including error handling, loading states and file uploads; it introduces XMLHttpRequest and Axios, explains the same-origin policy and CORS, covers sending API keys and tokens, and finishes with the ways to store data in the browser: localStorage, sessionStorage, cookies and IndexedDB.

IN THIS MODULE

18.1 HTTP basics
18.2 Fetch API
18.3 Other request tools
18.4 CORS
18.5 Authentication basics
18.6 Browser storage

NOTE

The browser examples run against a small simulated server inside the test browser at `http://lbf.local`. Lines starting with `[server]` show what the server received, so you can see exactly what each request sent. Real public APIs behave the same way, only slower.

18.1 HTTP basics

18.1.1 Client-server model

On the web, a **client** (a browser, a mobile app, a Node.js script) sends a **request** to a **server**, which processes it and sends back a **response**. The language they speak is **HTTP** (HyperText Transfer Protocol), and with HTTPS the conversation is encrypted with TLS. Each request is independent: HTTP is **stateless**, so the server does not remember previous requests unless the client sends something that identifies it, such as a cookie or a token (18.5). A page load makes many requests: the HTML, then CSS, JavaScript, images and API calls for data.



ONE HTTP REQUEST AND RESPONSE

WHAT A REQUEST AND ITS RESPONSE LOOK LIKE text

```
GET /api/courses?level=beginner HTTP/1.1
Host: api.learnbeingforward.in
Accept: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Cache-Control: max-age=60

[{"id":1,"title":"JavaScript","weeks":12},{"id":2,"title":"Python","weeks":10}]
```

A request consists of a **method**, a **URL** (path and query string), **headers** (metadata such as the accepted format or credentials) and, for some methods, a **body**. A response consists of a **status code**, headers and a body. You can watch every request of a page in the DevTools **Network** panel, including timings, headers and bodies.

18.1.2 HTTP methods: GET, POST, PUT, PATCH, DELETE

The method says what the client wants to do with the resource identified by the URL. **Safe** methods only read; **idempotent** methods give the same result whether sent once or several times (important when a request is retried after a network error).

Method	Meaning	Body?	Safe / idempotent	Example
GET	read a resource or list	no	safe, idempotent	`GET /api/courses/7`
POST	create a resource, or trigger an action	yes	neither	`POST /api/courses`
PUT	replace a resource completely	yes	idempotent	`PUT /api/courses/7`
PATCH	change part of a resource	yes	not guaranteed	`PATCH /api/courses/7`
DELETE	remove a resource	usually no	idempotent	`DELETE /api/courses/7`
HEAD / OPTIONS	headers only / ask what is allowed (CORS preflight)	no	safe	-

18.1.3 Status codes and headers

The three-digit **status code** summarises the result, grouped by the first digit. Your code should check it: a 404 or 500 response is still a successful HTTP exchange, so `fetch` does not treat it as an error by itself (18.2.2). **Headers** are name-value pairs that describe the request or response; names are case-insensitive.

Code	Meaning	When you see it
200 OK	success	GET, PUT, PATCH succeeded
201 Created	a new resource was created	after POST (often with a Location header)
204 No Content	success, no body	after DELETE
301 / 302 / 304	moved / found elsewhere / not modified	redirects and caching

Code	Meaning	When you see it
400 Bad Request	invalid input	validation failed
401 Unauthorized	not logged in, or bad token	missing or expired credentials
403 Forbidden	logged in but not allowed	insufficient permissions
404 Not Found	no such resource	wrong URL or id
409 / 422	conflict / unprocessable data	duplicate email, semantic errors
429 Too Many Requests	rate limit exceeded	calling an API too often
500 / 502 / 503	server error / bad gateway / unavailable	bug or outage on the server

Header	Direction	Purpose
`Content-Type`	both	format of the body, e.g. `application/json`
`Accept`	request	formats the client can handle
`Authorization`	request	credentials, e.g. `Bearer <token>`
`Cookie` / `Set-Cookie`	request / response	send / store cookies
`Cache-Control`, `ETag`	response	caching rules and versions
`Location`	response	URL of a created or moved resource
`Access-Control-Allow-Origin`	response	CORS permission (18.4)

18.1.4 REST APIs and JSON

REST (Representational State Transfer) is the most common style for web APIs. Data is organised as **resources** with URLs (nouns, usually plural: `/courses`, `/courses/7`, `/courses/7/reviews`), and the HTTP method expresses the action, so the same URL serves several purposes. Query parameters filter, sort and paginate (`/courses?level=beginner&page=2`). Request and response bodies are almost always **JSON** (Module 9): text that `JSON.stringify` produces and `JSON.parse` (or `response.json()`) reads. Other styles you may meet are GraphQL (one endpoint, the client describes the data it wants) and WebSockets for real-time data (Module 21).

Action	Method and URL	Success response
list courses	`GET /api/courses`	200 + array
get one course	`GET /api/courses/7`	200 + object (404 if missing)
create a course	`POST /api/courses` + JSON body	201 + created object
replace a course	`PUT /api/courses/7` + full JSON	200 + updated object
update some fields	`PATCH /api/courses/7` + partial JSON	200 + updated object
delete a course	`DELETE /api/courses/7`	204, no body
nested resource	`GET /api/courses/7/reviews`	200 + reviews of course 7

18.2 Fetch API

18.2.1 GET requests with fetch()

`fetch(url, options?)` is the modern, promise-based way to make HTTP requests, available in browsers and in Node.js 18+. It returns a promise that fulfils with a `Response` object as soon as the response **headers** arrive. Reading the body is a second asynchronous step, using a method that matches the format: `response.json()` parses JSON, `text()` gives a string, `blob()` binary data (images, files), `formData()` and `arrayBuffer()`. The body can be read only once. With no options, fetch sends a GET request.

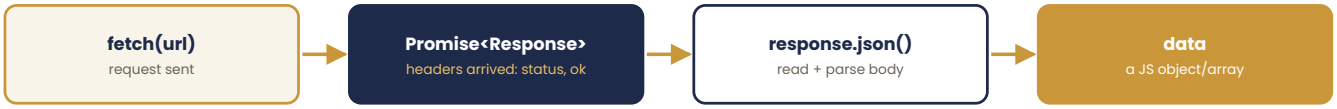
FETCHING JSON FROM AN API main.js

```
// promise chain
fetch("/api/courses")
  .then((response) => {
    console.log("status:", response.status, response.statusText, "| ok:", response.ok);
    console.log("content-type:", response.headers.get("content-type"));
    return response.json();                    // a second promise: read and parse the body
  })
  .then((courses) => {
    console.log("received", courses.length, "courses:", courses.map((c) => c.title).join(", "));
  });

// the same with async/await
async function loadCourse(id) {
  const response = await fetch(`/api/courses/${id}`);
  const course = await response.json();
  console.log(`course ${id}: ${course.title}, ${course.weeks} weeks`);
}
loadCourse(2);
```

BROWSER CONSOLE

```
[server] GET /api/courses
[server] GET /api/courses/2
status: 200 OK | ok: true
content-type: application/json
received 3 courses: JavaScript, React, Node.js
course 2: React, 8 weeks
```



TWO AWAITS: ONE FOR THE RESPONSE, ONE FOR THE BODY

18.2.2 Handling response.ok and errors

This is the most important detail of fetch: the promise only **rejects** when the request cannot be completed at all (no network, DNS failure, CORS block, or an abort). An HTTP **error status** such as 404 or 500 still **fulfils** the promise, with `response.ok` set to `false` (ok is true only for statuses 200-299). You must check `ok` (or `status`) yourself and throw. Many APIs also send an error description as JSON in the body, which is worth reading for the message. Wrapping this in a small helper avoids repeating it.

FETCH DOES NOT REJECT ON 404 OR 500

main.js

```
async function getJSON(url) {
  let response;
  try {
    response = await fetch(url);
  } catch (networkError) {                          // the request could not be made at all
    throw new Error(`Network problem: ${networkError.message}`);
  }
  if (!response.ok) {                               // HTTP error: fetch did NOT reject
    const body = await response.json().catch(() => ({}));
    const error = new Error(body.message || `HTTP ${response.status}`);
    error.status = response.status;
    throw error;
  }
  return response.json();
}

async function show(url) {
  try {
    const data = await getJSON(url);
    console.log(`${url} ->`, JSON.stringify(data));
  } catch (e) {
    console.log(`${url} -> failed: ${e.message}${e.status ? ` (status ${e.status})` : ""}`);
  }
}

(async () => {
  await show("/api/courses/1");
  await show("/api/courses/99");
  await show("/api/broken");
  await show("http://offline.invalid/api");
  const raw = await fetch("/api/courses/99");
  console.log("note: fetch fulfilled for the 404, ok =", raw.ok);
})();
```

BROWSER CONSOLE

```
/api/courses/1 -> {"id":1,"title":"JavaScript","weeks":12,"level":"beginner"}
/api/courses/99 -> failed: Course 99 not found (status 404)
/api/broken -> failed: Database unavailable (status 500)
http://offline.invalid/api -> failed: Network problem: Failed to fetch
note: fetch fulfilled for the 404, ok = false
```

Situation	fetch promise	response.ok	What to do
200-299 success	fulfils	true	read the body
404, 401, 500...	fulfils	false	check ok/status, throw or show a message
no network, DNS, CORS blocked	rejects (TypeError)	-	catch and show "check your connection"
aborted / timed out	rejects (AbortError / TimeoutError)	-	usually ignore or show "timed out"

18.2.3 POST, PUT and DELETE with JSON bodies

To send data, pass an options object: `method`, `headers` and `body`. For JSON, convert the object with `JSON.stringify` and set the `Content-Type: application/json` header so the server knows how to parse it.

The server usually replies with the created or updated object (201 or 200) or with no body for a DELETE (204, so do not call `json()` on it).

CREATING, UPDATING AND DELETING

main.js

```
async function api(method, url, data) {
  const response = await fetch(url, {
    method,
    headers: { "Content-Type": "application/json", Accept: "application/json" },
    body: data === undefined ? undefined : JSON.stringify(data),
  });
  if (!response.ok) throw new Error(`${method} ${url} failed: ${response.status}`);
  return response.status === 204 ? null : response.json();
}

(async () => {
  const created = await api("POST", "/api/courses", { title: "TypeScript", weeks: 6, level:
"intermediate" });
  console.log("created:", JSON.stringify(created));

  const replaced = await api("PUT", `/api/courses/${created.id}`, { title: "TypeScript", weeks: 8,
level: "advanced" });
  console.log("replaced:", JSON.stringify(replaced));

  const patched = await api("PATCH", `/api/courses/${created.id}`, { weeks: 9 });
  console.log("patched:", JSON.stringify(patched));

  const deleted = await api("DELETE", `/api/courses/${created.id}`);
  console.log("deleted, response body:", deleted);
})();
```

BROWSER CONSOLE

```
[server] POST /api/courses body={"title":"TypeScript","weeks":6,"level":"intermediate"}
created: {"title":"TypeScript","weeks":6,"level":"intermediate","id":4}
[server] PUT /api/courses/4 body={"title":"TypeScript","weeks":8,"level":"advanced"}
replaced: {"title":"TypeScript","weeks":8,"level":"advanced","id":4}
[server] PATCH /api/courses/4 body={"weeks":9}
patched: {"weeks":9,"id":4,"title":"TypeScript","level":"advanced"}
[server] DELETE /api/courses/4
deleted, response body: null
```

WATCH OUT

Forgetting `JSON.stringify` sends the text "[object Object]"; forgetting the Content-Type header makes many servers ignore the body. Both are common causes of "the server received nothing" bugs.

18.2.4 Headers, query parameters and URLSearchParams

Request headers are passed as a plain object or a `Headers` instance; response headers are read with `response.headers.get(name)`. Query strings must be **encoded** (spaces, &, non-ASCII characters): build them with `URLSearchParams`, or with the `URL` object, rather than by concatenating strings. `URLSearchParams` also parses query strings and can be sent directly as a form-encoded body.

BUILDING URLS AND SENDING HEADERS

main.js

```
const params = new URLSearchParams({ q: "java & spring", level: "beginner", page: 2 });
params.append("tag", "web");
params.append("tag", "backend");                 // repeated keys are allowed
console.log("query string:", params.toString());

const url = new URL("/api/search", location.origin);
url.search = params;
url.searchParams.set("sort", "-rating");
console.log("full URL:", url.href);

const headers = new Headers({ Accept: "application/json" });
headers.set("X-Client-Version", "2.4.0");

(async () => {
  const res = await fetch(url, { headers });
  console.log("rate limit left:", res.headers.get("X-RateLimit-Remaining"), "| total results:",
res.headers.get("X-Total-Count"));
  const parsed = new URLSearchParams("?q=react&page=3&tag=ui&tag=css");
  console.log("parsing:", parsed.get("q"), parsed.get("page"), parsed.getAll("tag"),
parsed.has("sort"));

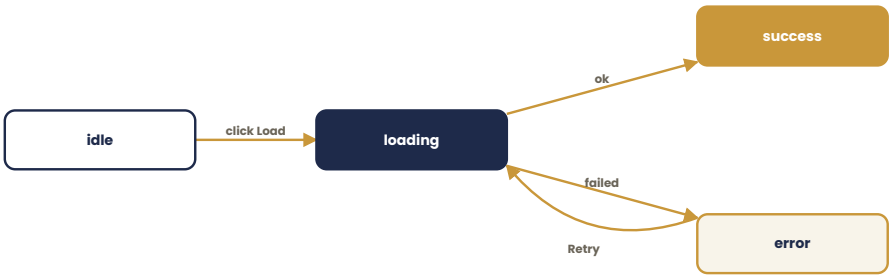
  await fetch("/api/login", { method: "POST", body: new URLSearchParams({ user: "asha", remember:
"yes" }) });   // form-encoded body
})();
```

BROWSER CONSOLE

```
query string: q=java+%26+spring&level=beginner&page=2&tag=web&tag=backend
full URL: http://lbf.local/api/search?
q=java+%26+spring&level=beginner&page=2&tag=web&tag=backend&sort=-rating
[server] GET /api/search?q=java+%26+spring&level=beginner&page=2&tag=web&tag=backend&sort=-rating X-
Client-Version=2.4.0 Accept=application/json
rate limit left: 58 | total results: 23
parsing: react 3 [ui, css] false
[server] POST /api/login Content-Type=application/x-www-form-urlencoded;charset=UTF-8
body=user=asha&remember=yes
```

18.2.5 Loading states and error messages in the UI

Users must always know what is happening. Every request in a user interface has at least three states: **loading** (show a spinner or skeleton, disable the button to prevent double submission), **success** (show the data, or an "empty" message if there is none), and **error** (show a friendly message and, where sensible, a Retry button). `finally` is the natural place to leave the loading state.



THE STATES OF A REQUEST IN THE UI

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<button id="load">Load courses</button>
<p id="status"></p>
<ul id="list"></ul>
```

LOADING, ERROR AND RETRY main.js

```
const button = document.querySelector("#load");
const status = document.querySelector("#status");
const list = document.querySelector("#list");
let attempt = 0;

async function loadCourses() {
  button.disabled = true;
  status.textContent = "Loading courses...";
  list.replaceChildren();
  try {
    const res = await fetch(++attempt === 1 ? "/api/courses-flaky" : "/api/courses");   // the first
call fails
    if (!res.ok) throw new Error(`Server error (${res.status})`);
    const courses = await res.json();
    if (courses.length === 0) { status.textContent = "No courses yet."; return; }
    list.append(...courses.map((c) => Object.assign(document.createElement("li"), { textContent:
`${c.title} (${c.weeks} weeks)` })));
    status.textContent = `${courses.length} courses loaded`;
  } catch (e) {
    status.textContent = `Could not load courses: ${e.message}. Please try again.`;
    button.textContent = "Retry";
  } finally {
    button.disabled = false;
    console.log(`attempt ${attempt}: "${status.textContent}"`);
  }
}
button.addEventListener("click", loadCourses);
```

BROWSER CONSOLE

```
attempt 1: "Could not load courses: Server error (503). Please try again."
attempt 2: "3 courses loaded"
--- page shows ---
Retry

3 courses loaded

JavaScript (12 weeks)
React (8 weeks)
Node.js (8 weeks)
```

Simulated user actions, in order: clicks `#load`; waits 300 ms; clicks `#load`; waits 300 ms.

18.2.6 Uploading files with FormData

To upload files, or to submit a form exactly as the browser would, send a `FormData` object as the body. Build it from a form element (`new FormData(form)`, which includes the selected files of `<input type="file">`) or append fields and files yourself. Do **not** set the `Content-Type` header: the browser sets `multipart/form-data` with the correct boundary automatically. Files are `File` objects (a kind of `Blob`) with `name`, `size` and `type`, which you can check before uploading.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```html
<form id="upload">
  <input name="name" value="Asha Rao">
  <input name="resume" type="file" accept="application/pdf">
  <button>Upload</button>
</form>
```

UPLOADING A FILE WITH FORMDATA main.js

```
const form = document.querySelector("#upload");
form.addEventListener("submit", async (e) => {
  e.preventDefault();
  const data = new FormData(form);                          // includes the file input
  const file = data.get("resume");
  if (file.size > 2 * 1024 * 1024) return console.log("file too large");
  if (file.type !== "application/pdf") return console.log("only PDF files are allowed");
  data.append("uploadedAt", "2026-03-09");
  console.log(`uploading ${file.name} (${file.size} bytes, ${file.type})`);
  const res = await fetch("/api/upload", { method: "POST", body: data });   // no Content-Type header!
  console.log("server replied:", res.status, JSON.stringify(await res.json()));
});

// simulate the user choosing a file (real users pick one in the file dialog)
const fakePdf = new File(["%PDF-1.7 sample resume content"], "asha_resume.pdf", { type:
"application/pdf" });
const dt = new DataTransfer();
dt.items.add(fakePdf);
form.elements.resume.files = dt.files;
form.requestSubmit();
```

BROWSER CONSOLE

```
uploading asha_resume.pdf (30 bytes, application/pdf)
[server] POST /api/upload Content-Type=multipart/form-data; boundary=----(random) body=
<multipart/form-data, 432 bytes>
server replied: 201 {"stored":"uploads/asha_resume.pdf"}
```

18.3 Other request tools

18.3.1 XMLHttpRequest (legacy)

Before fetch, requests were made with `XMLHttpRequest` (XHR), the API behind the original "AJAX" technique (Asynchronous JavaScript and XML, though JSON soon replaced XML). It is event-based rather than promise-based and more verbose, but you will still meet it in older code and libraries. It has one feature fetch lacks natively: **upload progress** events.

XMLHTTPREQUEST

main.js

```
const xhr = new XMLHttpRequest();
xhr.open("GET", "/api/courses");
xhr.responseType = "json";
xhr.onload = () => {
  if (xhr.status >= 200 && xhr.status < 300) console.log("XHR loaded", xhr.response.length, "courses,
status", xhr.status);
  else console.log("XHR HTTP error", xhr.status);
};
xhr.onerror = () => console.log("XHR network error");
xhr.send();

// wrapping XHR in a promise, as libraries used to do
function xhrJSON(url) {
  return new Promise((resolve, reject) => {
    const r = new XMLHttpRequest();
    r.open("GET", url);
    r.responseType = "json";
    r.onload = () => (r.status < 300 ? resolve(r.response) : reject(new Error(`HTTP ${r.status}`)));
    r.onerror = () => reject(new Error("network error"));
    r.send();
  });
}
xhrJSON("/api/courses/2").then((c) => console.log("promisified XHR:", c.title));
```

BROWSER CONSOLE

```
XHR loaded 3 courses, status 200
promisified XHR: React
```

18.3.2 Axios (overview)

Axios is a popular HTTP client library for browsers and Node.js (installed with `npm install axios`). Compared with fetch it automatically converts JSON in both directions, **rejects on HTTP error statuses**, supports **interceptors** (code that runs on every request or response, for example to add a token or refresh it), default settings per instance (base URL, headers, timeout), and upload/download progress. Fetch needs no dependency and is the web standard; small wrapper functions like the ones in this module give most of the convenience.

AXIOS IN AN ES MODULE main.js

```
import axios from "axios";

const api = axios.create({
  baseURL: "https://api.learnbeingforward.in",
  timeout: 5000,
  headers: { Accept: "application/json" },
});

api.interceptors.request.use((config) => {          // runs before every request
  config.headers.Authorization = `Bearer ${localStorage.getItem("token")}`;
  return config;
});

try {
  const { data } = await api.get("/courses", { params: { level: "beginner" } });   // JSON parsed
already
  console.log(data.length, "courses");
  const { data: created } = await api.post("/courses", { title: "Vue", weeks: 6 }); // object sent as
JSON
  console.log("created", created.id);
} catch (err) {
  console.log("failed:", err.response?.status, err.message);   // 4xx/5xx reject automatically
}
```

OUTPUT (with a running API)

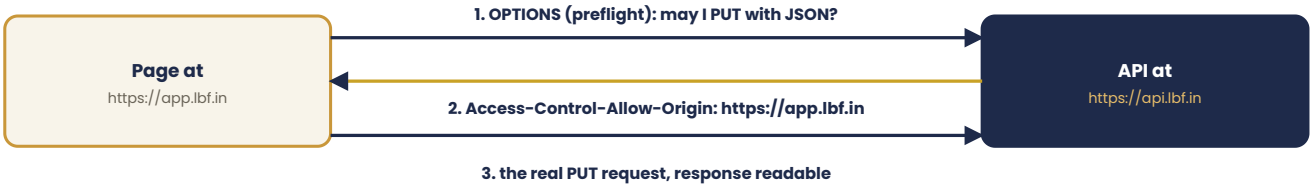
```
3 courses
created 4
```

	fetch	Axios
Installation	built in	npm package (about 13 kB gzipped)
JSON	JSON.stringify / response.json() by hand	automatic
HTTP errors (4xx/5xx)	fulfils; check response.ok	rejects
Timeouts	AbortSignal.timeout()	`timeout` option
Interceptors / defaults	write your own wrapper	built in
Upload progress	not directly (streams)	onUploadProgress

18.4 CORS

18.4.1 Same-origin policy and CORS errors

An **origin** is the combination of scheme, host and port: `https://app.lbf.in` and `https://api.lbf.in` are different origins, as are `http://` and `https://` versions or different ports. The browser's **same-origin policy** stops JavaScript on one origin from reading responses from another origin, so a malicious site cannot read your bank data using your logged-in cookies. **CORS** (Cross-Origin Resource Sharing) is how a server opts in: it sends `Access-Control-Allow-Origin` (and related) headers naming the origins allowed to read its responses. For "non-simple" requests (methods like PUT/DELETE, JSON content type, custom headers), the browser first sends an automatic **preflight** `OPTIONS` request to ask permission.



Without the header in step 2, the browser blocks the response: a CORS error

A CROSS-ORIGIN REQUEST WITH A PREFLIGHT

WHAT A CORS ERROR LOOKS LIKE main.js

```
// page served from http://localhost:5173, API without CORS headers
fetch("https://api.example.com/data")
  .then((r) => r.json())
  .catch((e) => console.log(e.name, e.message));
```

BROWSER CONSOLE

```
✖ Access to fetch at 'https://api.example.com/data' from origin 'http://localhost:5173'
  has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present
  on the requested resource.
TypeError Failed to fetch
```

The JavaScript only sees a generic TypeError; the detailed reason appears in the console. The request may even have reached the server: CORS blocks reading the response, not sending it.

It is a server setting

CORS errors cannot be fixed in front-end code. The API must send Access-Control-Allow-Origin (Express: the cors package, Module 22).

During development

Use the dev server's proxy (Vite, webpack) so the browser sees one origin, or enable CORS for localhost on your own API.

Not for security of the API

CORS protects users in browsers; tools like curl or a server ignore it, so APIs still need authentication.

RESPONSE HEADERS THAT ALLOW A CROSS-ORIGIN CLIENT text

```
Access-Control-Allow-Origin: https://app.lbf.in
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Credentials: true
Access-Control-Max-Age: 600
```

18.5 Authentication basics

18.5.1 API keys, tokens (JWT) and sending Authorization headers

Authentication proves who is making a request; **authorisation** decides what they may do. Three mechanisms are common. An **API key** identifies an application to a third-party service (a weather or maps API); it is sent as a header or query parameter. Because anything shipped to the browser is visible to users, secret keys must stay on a server, which calls the third-party API on the browser's behalf. A **session cookie**: after login the server sets a cookie, and the browser sends it automatically with every

request to that site (18.6.3). A **token**, most often a **JWT** (JSON Web Token): after login the server returns a signed token, and the client sends it in the `Authorization: Bearer <token>` header. A JWT has three Base64URL parts, header.payload.signature; the payload is **readable by anyone** (it is only encoded, not encrypted), and the signature lets the server detect tampering.

LOGGING IN AND SENDING A BEARER TOKEN

main.js

```
async function login(email, password) {
  const res = await fetch("/api/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email, password }),
  });
  if (!res.ok) throw new Error("Invalid email or password");
  return (await res.json()).token;
}

function decodeJwtPayload(token) {                    // reading is possible; verifying needs the
server's key
  const base64 = token.split(".")[1].replace(/-/g, "+").replace(/_/g, "/");
  return JSON.parse(atob(base64));
}

async function authFetch(url, token) {
  const res = await fetch(url, { headers: { Authorization: `Bearer ${token}` } });
  if (res.status === 401) throw new Error("Session expired: please log in again");
  return res.json();
}

(async () => {
  const token = await login("asha@lbf.in", "correct-horse");
  console.log("token parts:", token.split(".").length);
  const payload = decodeJwtPayload(token);
  console.log("payload:", JSON.stringify(payload), "| expires:", new Date(payload.exp *
1000).toISOString());
  const me = await authFetch("/api/me", token);
  console.log("profile:", JSON.stringify(me));
  try { await authFetch("/api/me", "expired.token.value"); } catch (e) { console.log(e.message); }
})();
```

BROWSER CONSOLE

```
[server] POST /api/login
token parts: 3
payload: {"sub":"7","name":"Asha","role":"student","exp":1773054000} | expires: 2026-03-
09T11:00:00.000Z
[server] GET /api/me Authorization=Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI3IiwibmFtZSI6IkFzaGEiLCJyb2xlIjoic3R1ZGVudCIsImV4cCI6M
Tc3MzA1NDAwMH0.Xh1sQm3kq9oX2pZtW7yV5nL0aB4cD6eF8gH1iJ2kL3m
profile: {"id":7,"name":"Asha","courses":["JavaScript","React"]}
[server] GET /api/me Authorization=Bearer expired.token.value
Session expired: please log in again
```

Mechanism	Sent how	Where it lives in the browser	Watch out for
API key	header (e.g. `X-API-Key`) or query param	should not be in the browser if secret	anyone can read front-end code
Session cookie	automatically with each request	HttpOnly cookie (JS cannot read it)	CSRF: use SameSite and CSRF tokens

Mechanism	Sent how	Where it lives in the browser	Watch out for
JWT bearer token	`Authorization: Bearer ...`	memory, or storage (XSS risk)	payload is readable; keep tokens short-lived

WATCH OUT

A token in `localStorage` can be stolen by any XSS bug (Module 15). Many applications therefore keep tokens in memory and use an **HttpOnly, Secure, SameSite** cookie for the long-lived refresh token. Never put passwords or secrets in a JWT payload. Module 24 covers web security in depth.

18.6 Browser storage

18.6.1 localStorage and sessionStorage

The **Web Storage API** offers two simple key-value stores per origin, with the same methods: `setItem(key, value)`, `getItem(key)` (`null` if missing), `removeItem(key)`, `clear()`, `key(i)` and `length`. Keys and values are always **strings**. `localStorage` persists until it is cleared, surviving browser restarts, and is shared by all tabs of the origin. `sessionStorage` lasts only as long as the tab, and is separate for each tab. Both are **synchronous** and limited to about 5 MB per origin, so they suit small data: preferences, a draft, recent searches. Another tab is notified of `localStorage` changes through the `storage` event.

USING LOCALSTORAGE AND SESSIONSTORAGE main.js

```
localStorage.setItem("theme", "dark");
localStorage.setItem("fontSize", 18);                    // stored as the string "18"
console.log("theme:", localStorage.getItem("theme"));
console.log("fontSize:", localStorage.getItem("fontSize"), typeof localStorage.getItem("fontSize"));
console.log("missing key:", localStorage.getItem("language"));
console.log("number of keys:", localStorage.length, "| first key:", localStorage.key(0));

localStorage.removeItem("fontSize");
console.log("after removeItem:", Object.keys(localStorage));

sessionStorage.setItem("checkoutStep", "2");
console.log("sessionStorage (this tab only):", sessionStorage.getItem("checkoutStep"));

const visits = Number(localStorage.getItem("visits") ?? 0) + 1;   // a visit counter
localStorage.setItem("visits", visits);
console.log("visits on this browser:", visits);

try {
  localStorage.setItem("huge", "x".repeat(6 * 1024 * 1024));
} catch (e) {
  console.log("storage full:", e.name);
}
```

BROWSER CONSOLE

```
theme: dark
fontSize: 18 string
missing key: null
number of keys: 2 | first key: theme
after removeItem: [theme]
sessionStorage (this tab only): 2
visits on this browser: 1
storage full: QuotaExceededError
```

18.6.2 Storing objects with JSON

To store objects or arrays, convert them with `JSON.stringify` when saving and `JSON.parse` when loading. Storing an object directly saves the useless string `"[object Object]"`. Because stored data may be missing, outdated or corrupted (or edited by the user in DevTools), read it defensively with a default value and a `try/catch`. Remember that JSON loses dates (they become strings), `undefined`, functions, Maps and Sets.

SAVING AND LOADING OBJECTS SAFELY main.js

```
const store = {
  get(key, fallback) {
    try {
      const raw = localStorage.getItem(key);
      return raw === null ? fallback : JSON.parse(raw);
    } catch {
      return fallback;                              // corrupted value
    }
  },
  set(key, value) { localStorage.setItem(key, JSON.stringify(value)); },
};

const cart = [{ id: 1, name: "Pen", qty: 3 }, { id: 2, name: "Book", qty: 1 }];
localStorage.setItem("wrong", cart);
console.log("without JSON:", localStorage.getItem("wrong"));

store.set("cart", cart);
console.log("raw string:", localStorage.getItem("cart"));
const loaded = store.get("cart", []);
console.log("loaded back:", loaded.map((i) => `${i.name}x${i.qty}`).join(", "), "| is array:",
Array.isArray(loaded));

localStorage.setItem("prefs", "{broken json");
console.log("corrupted -> fallback:", JSON.stringify(store.get("prefs", { theme: "light" })));

store.set("lastLogin", new Date(Date.UTC(2026, 2, 9, 4, 30)));
const back = store.get("lastLogin");
console.log("date comes back as", typeof back, back, "-> revive with new Date():", new
Date(back).getUTCHours());
```

BROWSER CONSOLE

```
without JSON: [object Object],[object Object]
raw string: [{"id":1,"name":"Pen","qty":3},{"id":2,"name":"Book","qty":1}]
loaded back: Penx3, Bookx1 | is array: true
corrupted -> fallback: {"theme":"light"}
date comes back as string 2026-03-09T04:30:00.000Z -> revive with new Date(): 4
```

18.6.3 Cookies

Cookies are small pieces of data (about 4 KB each) that the browser stores per site and **sends to the server automatically** in the `Cookie` header of every matching request. That makes them the classic mechanism for sessions. Servers set them with the `Set-Cookie` header; JavaScript can read and write non-HttpOnly cookies through the awkward `document.cookie` string: assigning sets **one** cookie, and reading returns all of them as `"a=1; b=2"`. Attributes control the lifetime and security: `Expires` / `Max-Age`, `Path`, `Domain`, `Secure` (HTTPS only), `HttpOnly` (hidden from JavaScript, set by servers only) and `SameSite` (Strict, Lax or None) against cross-site request forgery.

READING AND WRITING COOKIES main.js

```
document.cookie = "theme=dark; path=/; max-age=" + 60 * 60 * 24 * 30;      // 30 days
document.cookie = "lang=kn; path=/";                                       // session cookie
document.cookie = `city=${encodeURIComponent("Bengaluru Urban")}; path=/; SameSite=Lax`;
console.log("document.cookie:", document.cookie);

function getCookie(name) {
  const pair = document.cookie.split("; ").find((c) => c.startsWith(name + "="));
  return pair ? decodeURIComponent(pair.split("=").slice(1).join("=")) : null;
}
console.log("city:", getCookie("city"), "| missing:", getCookie("token"));

document.cookie = "lang=; path=/; max-age=0";                              // delete by expiring
console.log("after deleting lang:", document.cookie);

(async () => {
  await fetch("/api/session");                                          // cookies are sent
automatically
  console.log("after the server's Set-Cookie, JS sees:", document.cookie, "(the HttpOnly session
cookie is hidden)");
  await fetch("/api/session");
})();
```

BROWSER CONSOLE

```
document.cookie: theme=dark; lang=kn; city=Bengaluru%20Urban
city: Bengaluru Urban | missing: null
after deleting lang: theme=dark; city=Bengaluru%20Urban
[server] GET /api/session Cookie=theme=dark; city=Bengaluru%20Urban
after the server's Set-Cookie, JS sees: theme=dark; city=Bengaluru%20Urban (the HttpOnly session
cookie is hidden)
[server] GET /api/session Cookie=theme=dark; city=Bengaluru%20Urban; sid=abc123
```

NOTE

The second request shows `sid=abc123` in the Cookie header sent to the server even though JavaScript cannot read it: exactly how HttpOnly session cookies protect against token theft. The modern asynchronous `cookieStore` API (Chromium) is a friendlier alternative to `document.cookie`.

18.6.4 IndexedDB (overview)

IndexedDB is a full transactional database built into the browser, for large amounts of structured data (hundreds of megabytes or more, depending on disk space), including objects, dates, Blobs and files, without converting to strings. It stores objects in **object stores** with keys and **indexes** for searching, works **asynchronously** so it never blocks the page, and is available in web workers. It is the storage behind offline-capable apps (with service workers, Module 21). Its native API is event-based and verbose, so most developers use a small promise wrapper such as **idb** or a library such as **Dexie.js**. The example below wraps the native requests in promises.

STORING AND QUERYING OBJECTS IN INDEXEDDB

main.js

```
function promisify(request) {
  return new Promise((resolve, reject) => {
    request.onsuccess = () => resolve(request.result);
    request.onerror = () => reject(request.error);
  });
}

async function main() {
  const open = indexedDB.open("lbf-notes", 1);
  open.onupgradeneeded = () => {                              // create the schema (version 1)
    const store = open.result.createObjectStore("notes", { keyPath: "id", autoIncrement: true });
    store.createIndex("byCourse", "course");
    console.log("database created with store 'notes' and index 'byCourse'");
  };
  const db = await promisify(open);

  const tx = db.transaction("notes", "readwrite");
  const notes = tx.objectStore("notes");
  for (const n of [
    { course: "js", text: "Closures capture variables", created: new Date(2026, 2, 9) },
    { course: "js", text: "Microtasks run before timers", created: new Date(2026, 2, 10) },
    { course: "react", text: "State updates are batched", created: new Date(2026, 2, 11) },
  ]) await promisify(notes.add(n));

  const all = await promisify(notes.getAll());
  console.log("stored:", all.length, "notes; first created is a real Date:", all[0].created instanceof
Date);
  const jsNotes = await promisify(notes.index("byCourse").getAll("js"));
  console.log("js notes via index:", jsNotes.map((n) => n.text).join(" | "));
  await promisify(notes.delete(3));
  console.log("count after delete:", await promisify(notes.count()));
}
main().catch((e) => console.log("IndexedDB error:", e.message));
```

BROWSER CONSOLE

```
database created with store 'notes' and index 'byCourse'
stored: 3 notes; first created is a real Date: true
js notes via index: Closures capture variables | Microtasks run before timers
count after delete: 2
```

18.6.5 Comparing storage options

	localStorage	sessionStorage	Cookies	IndexedDB
Capacity	~5 MB	~5 MB	~4 KB each	large (share of disk)
Lifetime	until cleared	until the tab closes	Expires / Max-Age or session	until cleared
Sent to server	no	no	yes, with every request	no
Data type	strings	strings	strings	objects, Blobs, files
API	synchronous, simple	synchronous, simple	document.cookie string	asynchronous, complex
Shared between tabs	yes	no	yes	yes

	localStorage	sessionStorage	Cookies	IndexedDB
Readable by JS	yes (XSS risk)	yes	unless HttpOnly	yes
Good for	preferences, recent items, drafts	one-tab wizard state	sessions, server-side preferences	offline data, large caches, files

TIP

All browser storage can be inspected and edited in DevTools (Application panel in Chrome, Storage in Firefox) and can be wiped by the user or by the browser under storage pressure. Never store passwords or secrets there, and treat what you read back as untrusted input.

HTTP requests have a method (GET reads, POST creates, PUT replaces, PATCH updates, DELETE removes), a URL, headers and an optional body; responses have a status code (2xx success, 4xx client errors, 5xx server errors), headers and a body, usually JSON in REST APIs. fetch returns a promise for a Response; read the body with json()/text()/blob(); fetch rejects only on network failure, so check response.ok. Send JSON with method, a Content-Type header and JSON.stringify; build query strings with URLSearchParams; show loading, success, empty and error states; upload files with FormData (no Content-Type header). XMLHttpRequest is the legacy API; Axios adds automatic JSON, error rejection and interceptors. The same-origin policy blocks reading cross-origin responses unless the server sends CORS headers. APIs authenticate with API keys (keep secret keys on servers), session cookies or bearer tokens such as JWTs. Store small string data in localStorage (persistent) or sessionStorage (per tab) with JSON for objects; cookies travel to the server and support HttpOnly and SameSite; IndexedDB stores large structured data asynchronously.

PRACTICE CHECKPOINT

Build a **weather app** that fetches data from a public API with **loading and error states**, and saves the **last five searches** in localStorage.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<form id="search"><input name="city" placeholder="City"><button>Search</button></form>
<p id="status"></p>
<p id="result"></p>
<div id="recent"></div>
```

SAMPLE SOLUTION main.js

```
// In a real project, API_URL would be a public API such as https://api.open-meteo.com
// (with a geocoding step), or your own server that hides a secret API key.
const API_URL = "/api/weather";
const form = document.querySelector("#search");
const input = form.elements.city;
const status = document.querySelector("#status");
const result = document.querySelector("#result");
const recentList = document.querySelector("#recent");

const recent = {
  load() { try { return JSON.parse(localStorage.getItem("recentCities")) ?? []; } catch { return []; }
},
  add(city) {
    const list = [city, ...this.load().filter((c) => c.toLowerCase() !== city.toLowerCase())].slice(0,
5);
```

```
    localStorage.setItem("recentCities", JSON.stringify(list));
    return list;
  },
};

function renderRecent(list = recent.load()) {
  recentList.replaceChildren(...list.map((c) => Object.assign(document.createElement("button"), {
textContent: c, type: "button" })));
}

async function search(city) {
  status.textContent = `Loading weather for ${city}...`;
  result.textContent = "";
  form.querySelector("button").disabled = true;
  try {
    const res = await fetch(`${API_URL}?${new URLSearchParams({ city })}`, { signal:
AbortSignal.timeout(5000) });
    if (res.status === 404) throw new Error(`City "${city}" was not found`);
    if (!res.ok) throw new Error(`Weather service error (${res.status})`);
    const w = await res.json();
    result.textContent = `${w.city}: ${w.temp}°C, ${w.description}, humidity ${w.humidity}%`;
    status.textContent = "Updated";
    renderRecent(recent.add(w.city));
  } catch (e) {
    status.textContent = e.name === "TimeoutError" ? "The weather service is too slow, try again" :
e.message;
  } finally {
    form.querySelector("button").disabled = false;
    console.log(`${city}: ${status.textContent} | ${result.textContent || "-"} | recent:
${recent.load().join(", ")}`);
  }
}

form.addEventListener("submit", (e) => {
  e.preventDefault();
  const city = input.value.trim();
  if (city) search(city);
});
recentList.addEventListener("click", (e) => { if (e.target.matches("button"))
search(e.target.textContent); });
localStorage.setItem("recentCities", JSON.stringify(["Pune", "Delhi", "Mumbai", "Chennai", "Kochi"]));
// earlier visits
renderRecent();
```

BROWSER CONSOLE

```
Mysuru: Updated | Mysuru: 27°C, partly cloudy, humidity 61% | recent: Mysuru, Pune, Delhi, Mumbai,
Chennai
Delhi: Updated | Delhi: 34°C, haze, humidity 38% | recent: Delhi, Mysuru, Pune, Mumbai, Chennai
Atlantis: City "Atlantis" was not found | - | recent: Delhi, Mysuru, Pune, Mumbai, Chennai
--- page shows ---
Search

City "Atlantis" was not found

DelhiMysuruPuneMumbaiChennai
```

Simulated user actions, in order: types "Mysuru" into `[name=city]`; clicks `#search button`; waits 300 ms; clicks `#recent button:nth-child(3)`; waits 300 ms; types "Atlantis" into `[name=city]`; clicks `#search button`; waits 300 ms.

LEVEL 3

Advanced level

Outcome: modules and tooling, engine internals and patterns, advanced browser APIs, Node.js with Express, TypeScript, testing, frameworks and interview readiness.

7 MODULES · 25 DAYS MINIMUM

Module 19 · Modules and the tooling ecosystem	**2 days**
Module 20 · Advanced JavaScript internals and patterns	**3 days**
Module 21 · Advanced browser APIs	**3 days**
Module 22 · Back-end JavaScript with Node.js and Express	**5 days**
Module 23 · TypeScript essentials	**4 days**
Module 24 · Testing, security and best practices	**3 days**
Module 25 · Frameworks, DSA in JavaScript and interview preparation	**5 days**

MODULE 19 · ADVANCED

Modules and the tooling ecosystem

Minimum duration **2 days** | Level **Advanced**

Real projects are split into many files and built with tools. This module explains modules (ES modules with named and default exports, dynamic import, module scripts in the browser, JSON modules, and CommonJS), the npm package manager (package.json, dependencies, semantic versioning, lock files, scripts and npx), build tools (why bundlers exist, Vite, webpack, Babel and environment variables), code-quality tools (ESLint and Prettier), and version control with Git and GitHub, ending with deploying a static site.

IN THIS MODULE

19.1 Modules
19.2 npm
19.3 Build tools
19.4 Code quality tools
19.5 Version control

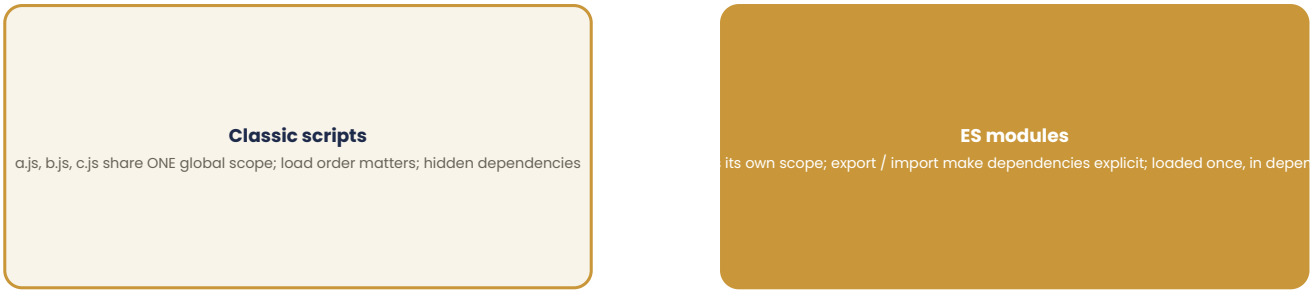
NOTE

Examples marked TERMINAL were really run in a fresh project folder called `my-app`: the files shown above each one were created first, then each `$` command was executed. Commands that need to download packages from the internet are shown with typical output instead.

19.1 Modules

19.1.1 Why modules

Early web pages loaded several `<script>` files that all shared one global scope: a variable named `total` in one file could silently overwrite `total` in another, the files had to be loaded in exactly the right order, and nothing said which file depended on which. A **module** is a file with its own private scope that explicitly **exports** what it offers and **imports** what it needs. Modules give: no global pollution; clear, checkable dependencies; reuse across projects; easier testing; and the ability for tools to remove unused code (**tree shaking**) and split code into chunks loaded on demand.



FROM SHARED GLOBALS TO EXPLICIT MODULES

19.1.2 ES Modules: export, export default, import

ES modules (ESM), standardised in ES2015, are the official module system of JavaScript, supported natively by browsers and Node.js. Put `export` before a declaration (or list names in `export { a, b }`) to make it available; use `import { a } from "./file.js"` to use it. Module code always runs in **strict mode**, top-level variables are private to the module, `this` at the top level is `undefined`, and each module is evaluated **only once** no matter how many files import it. Imports are **live bindings**: read-only views of the exporting module's variables, not copies. In Node.js, ES modules use the `.mjs` extension or `"type": "module"` in package.json; import paths must include the file extension.

UTILS/MATH.MJS JavaScript

```
export const PI = 3.14159;                    // named export (declaration)

export function circleArea(r) {
  return PI * r * r;
}

function square(x) { return x * x; }          // private: not exported
const cube = (x) => x * x * x;

export { square as sq, cube };                // export a list, with renaming

console.log("math.mjs evaluated (only once)");
```

UTILS/COUNTER.MJS JavaScript

```
export let count = 0;                         // a live binding
export function increment() { count++; }
```

APP.MJS JavaScript

```
import { PI, circleArea, sq, cube } from "./utils/math.mjs";
import * as math from "./utils/math.mjs";      // namespace object, same module instance
import { count, increment } from "./utils/counter.mjs";

console.log("PI:", PI, "| area r=2:", circleArea(2).toFixed(2));
console.log("sq(4):", sq(4), "| cube(3):", cube(3));
console.log("namespace keys:", Object.keys(math));

console.log("count before:", count);
increment(); increment();
console.log("count after (live binding):", count);
try { count = 99; } catch (e) { console.log("imports are read-only:", e.name); }
console.log("top-level this in a module:", this);
```

SPLITTING CODE INTO ES MODULES

main.js

```
node app.mjs
```

TERMINAL

```
$ node app.mjs
math.mjs evaluated (only once)
PI: 3.14159 | area r=2: 12.57
sq(4): 16 | cube(3): 27
namespace keys: [ 'PI', 'circleArea', 'cube', 'sq' ]
count before: 0
count after (live binding): 2
imports are read-only: TypeError
top-level this in a module: undefined
```

Syntax	Meaning
`export const x = 1;` / `export function f() {}`	named export of a declaration
`export { a, b as bee };`	export a list, optionally renamed
`export default value;`	the module's default export (19.1.3)
`export { x } from "./other.js";` / `export * from ...`	re-export from another module (barrel files)
`import { a, b as bee } from "./m.js";`	import named exports
`import thing from "./m.js";`	import the default export
`import * as m from "./m.js";`	namespace object with every export
`import "./polyfills.js";`	run a module for its side effects only
`const m = await import("./m.js");`	dynamic import (19.1.4)

19.1.3 Named vs default exports

A module can have any number of **named exports** and at most one **default export**. Named exports must be imported with their exact names (in braces, optionally renamed with `as`), which keeps naming consistent and lets editors auto-import and tools tree-shake reliably. A default export is imported **without braces under any name** the importer chooses; it suits modules with one main thing, such as a class or a React component. Many teams prefer named exports everywhere for consistency; frameworks often require a default export for pages and components.

CART.MJS

JavaScript

```
export default class Cart {                    // the main thing: default export
  items = [];
  add(item) { this.items.push(item); return this; }
  get total() { return this.items.reduce((t, i) => t + i.price, 0); }
}

export const TAX_RATE = 0.18;                  // helpers: named exports
export const withTax = (amount) => Math.round(amount * (1 + TAX_RATE));
```

MAIN.MJS JavaScript

```
import ShoppingCart, { TAX_RATE, withTax as addTax } from "./Cart.mjs";   // default name chosen
freely

const cart = new ShoppingCart().add({ name: "Pen", price: 100 }).add({ name: "Book", price: 400 });
console.log("total:", cart.total, "| with", TAX_RATE * 100 + "% tax:", addTax(cart.total));

const mod = await import("./Cart.mjs");
console.log("the default export is just an export named 'default':", mod.default === ShoppingCart);
```

MIXING A DEFAULT AND NAMED EXPORTS main.js

```
node main.mjs
```

TERMINAL

```
$ node main.mjs
total: 500 | with 18% tax: 590
the default export is just an export named 'default': true
```

	Named export	Default export
Per module	any number	at most one
Import syntax	`import { name } from`	`import anyName from`
Name at import	must match (or rename with `as`)	chosen by the importer
Best for	utilities, constants, several related things	one main class, function or component
Tooling	reliable auto-import and refactoring	names can drift between files

19.1.4 Dynamic import() and code splitting

Static `import` statements must be at the top level and are resolved before any code runs. The `import(specifier)` **expression** loads a module at run time and returns a **promise** for its namespace object. It can be used anywhere (inside functions, conditions, event handlers), with computed paths. Bundlers turn every dynamic import into a separate file (a **chunk**) that is downloaded only when needed: this is **code splitting**, used to keep the initial download small, for example by loading a chart library only when the user opens the reports page, or a language pack only for the chosen language.

LANG/EN.MJS JavaScript

```
export default { greet: (n) => `Hello, ${n}!` };
```

LANG/KN.MJS JavaScript

```
export default { greet: (n) => `Namaskara, ${n}!` };
```

HEAVY-REPORT.MJS JavaScript

```
console.log("  (heavy-report.mjs loaded now)");
export function buildReport(rows) { return `report with ${rows.length} rows`; }
```

APP.MJS

JavaScript

```
async function greetIn(lang, name) {
  const { default: pack } = await import(`./lang/${lang}.mjs`);   // computed path
  console.log(pack.greet(name));
}
await greetIn("en", "Asha");
await greetIn("kn", "Asha");

console.log("app started; report code not loaded yet");
async function onReportsClick() {
  const { buildReport } = await import("./heavy-report.mjs");     // loaded on first use only
  console.log(buildReport([1, 2, 3]));
}
await onReportsClick();
await onReportsClick();                                           // second time: already cached

try { await import("./lang/fr.mjs"); } catch (e) { console.log("missing module:", e.code); }
```

LOADING MODULES ON DEMAND

main.js

```
node app.mjs
```

TERMINAL

```
$ node app.mjs
Hello, Asha!
Namaskara, Asha!
app started; report code not loaded yet
  (heavy-report.mjs loaded now)
report with 3 rows
report with 3 rows
missing module: ERR_MODULE_NOT_FOUND
```

19.1.5 <script type="module">

Browsers load ES modules with `<script type="module" src="app.js">` (or inline module code). Module scripts are **deferred** automatically (they run after the HTML is parsed, in order), run in strict mode with their own scope, are fetched with CORS (so they do not work from `file://` URLs: use a local server such as VS Code Live Server or Vite), and are executed once even if included twice. Browsers require full relative paths with extensions (`"./utils.js"`); bare names like `"lodash"` only work with an **import map** or a bundler. `nomodule` scripts were the fallback for very old browsers.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY)

HTML

```
<ul id="courses"></ul>
```

A MODULE SCRIPT IMPORTING OTHER MODULES main.js

```
import { formatInr } from "./js/format.js";
import { courses } from "./js/data.js";

const list = document.querySelector("#courses");
list.append(...courses.map((c) => Object.assign(document.createElement("li"), { textContent:
`${c.title}: ${formatInr(c.fee)}` })));
console.log("module script: this =", this, "| top-level variable on window?", "list" in window);
console.log("readyState when module ran:", document.readyState);
```

BROWSER CONSOLE

```
module script: this = undefined | top-level variable on window? false
readyState when module ran: interactive
--- page shows ---
JavaScript: ₹14,999
React: ₹11,999
```

IMPORT MAPS LET BROWSERS RESOLVE BARE PACKAGE NAMES HTML

```
<!-- index.html -->
<script type="importmap">
  { "imports": { "dayjs": "https://cdn.jsdelivr.net/npm/dayjs@1/+esm" } }
</script>
<script type="module" src="./js/app.js"></script>
<!-- inside app.js you can now write: import dayjs from "dayjs"; -->
```

19.1.6 Import attributes and JSON modules

ES2025 standardised **import attributes**: extra information written after `with` in an import, telling the host what kind of module is expected. The first use is **JSON modules**: `import config from "./config.json" with { type: "json" }` imports a JSON file as a default-exported object. The attribute is required for security, so a server cannot trick the page into running JavaScript when JSON was expected. JSON modules work in current browsers and Node.js; CSS modules (`type: "css"`) are supported in Chromium. Dynamic imports take the attributes as a second argument.

CONFIG.JSON JavaScript

```
{ "appName": "LBF Portal", "version": "2.4.0", "features": { "darkMode": true, "chat": false } }
```

APP.MJS JavaScript

```
import config from "./config.json" with { type: "json" };

console.log(`${config.appName} v${config.version}`);
console.log("enabled features:", Object.entries(config.features).filter(([, on]) => on).map(([f]) =>
f));

const { default: again } = await import("./config.json", { with: { type: "json" } });
console.log("same object (modules are cached):", again === config);
```

IMPORTING JSON AS A MODULE main.js

```
node app.mjs
```

```
TERMINAL
$ node app.mjs
LBF Portal v2.4.0
enabled features: [ 'darkMode' ]
same object (modules are cached): true
```

19.1.7 CommonJS (require, module.exports) and interoperability

Before ES modules existed, Node.js adopted **CommonJS** (CJS): `require(path)` loads a module **synchronously** and returns its `module.exports` object; files assign to `module.exports` or add properties to `exports`. CommonJS is still the default for `.js` files in Node.js packages without `"type": "module"`, and a huge part of npm uses it, so you need to read both. Interoperability rules: ES modules can `import` CommonJS modules (the `module.exports` object becomes the default export); CommonJS can load ES modules with dynamic `import()`, and recent Node.js versions (20.19+, 22.12+) also allow `require()` of synchronous ES modules. Prefer ES modules for new code.

LEGACY/MATH.CJS JavaScript

```
function add(a, b) { return a + b; }
const VERSION = "1.0";
module.exports = { add, VERSION };            // what require() returns
console.log("math.cjs loaded, __filename ends with:", __filename.split("/").slice(-2).join("/"));
```

MODERN/GREET.MJS JavaScript

```
export const greet = (n) => `Hi ${n}`;
export default "greet module";
```

APP.CJS JavaScript

```
const { add, VERSION } = require("./legacy/math.cjs");     // synchronous
const path = require("node:path");                         // built-in modules too
console.log("add:", add(2, 3), "| version:", VERSION, "| ext:", path.extname("a/b.cjs"));
console.log("cached:", require("./legacy/math.cjs") === require("./legacy/math.cjs"));

import("./modern/greet.mjs").then((m) => console.log("CJS -> ESM via import():", m.greet("Asha"), "|",
m.default));
const viaRequire = require("./modern/greet.mjs");          // Node.js 22.12+: require of ESM
console.log("CJS -> ESM via require():", viaRequire.greet("Ravi"));
```

MIXED.MJS JavaScript

```
import math from "./legacy/math.cjs";                  // module.exports becomes the default
import { add } from "./legacy/math.cjs";                   // named import works for detectable
properties
console.log("ESM -> CJS:", math.add(10, 5), add(1, 1));
console.log("no __dirname in ESM; use import.meta:", typeof __dirname, "|",
import.meta.filename.endsWith("mixed.mjs"), import.meta.dirname.endsWith("my-app"));
```

COMMONJS AND ESM SIDE BY SIDE

main.js

```
node app.cjs
node mixed.mjs
```

TERMINAL

```
$ node app.cjs
math.cjs loaded, __filename ends with: legacy/math.cjs
add: 5 | version: 1.0 | ext: .cjs
cached: true
CJS -> ESM via require(): Hi Ravi
CJS -> ESM via import(): Hi Asha | greet module
$ node mixed.mjs
math.cjs loaded, __filename ends with: legacy/math.cjs
ESM -> CJS: 15 2
no __dirname in ESM; use import.meta: undefined | true true
```

	ES modules (ESM)	CommonJS (CJS)
Syntax	`import` / `export`	`require()` / `module.exports`
Loading	asynchronous, analysed before running	synchronous, at run time
Where	browsers, Node.js, Deno, Bun, bundlers	Node.js (and bundlers)
Node.js file types	.mjs, or .js with "type": "module"	.cjs, or .js by default
Top-level await	yes	no
File info	`import.meta.url`, `.filename`, `.dirname`	`__filename`, `__dirname`
Tree shaking	yes (static structure)	limited

19.2 npm

npm (Node Package Manager) comes with Node.js. It is three things: the public **registry** at npmjs.com with over three million open-source packages, the `npm` **command-line tool** that installs and manages them, and the `package.json` **format** that describes a project. Front-end projects use npm too, for frameworks, build tools and libraries, even though the final code runs in the browser.

19.2.1 package.json and npm init

Every JavaScript project has a `package.json` at its root: its name and version, the entry file, its dependencies, the scripts used to run, build and test it, and settings such as `"type": "module"`. `npm init` creates one by asking questions; `npm init -y` accepts the defaults. `npm pkg set` edits fields from the command line.

CREATING PACKAGE.JSON main.js

```
npm init -y
npm pkg set type=module description="LBF weather app" author="Asha Rao"
npm pkg get name type author
```

TERMINAL

```
$ npm init -y
Wrote to package.json:

{
  "name": "my-app",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
$ npm pkg set type=module description="LBF weather app" author="Asha Rao"
$ npm pkg get name type author
{
  "name": "my-app",
  "type": "module",
  "author": "Asha Rao"
}
```

A TYPICAL PACKAGE.JSON FOR A FRONT-END PROJECT json

```
{
  "name": "weather-app",
  "version": "1.0.0",
  "description": "LBF weather app",
  "type": "module",
  "main": "src/main.js",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "format": "prettier --write .",
    "test": "node --test"
  },
  "dependencies": {
    "dayjs": "^1.11.13"
  },
  "devDependencies": {
    "eslint": "^10.1.0",
    "prettier": "^3.8.1",
    "vite": "^7.1.0"
  },
  "engines": { "node": ">=22" }
}
```

19.2.2 Installing dependencies and dev dependencies

`npm install <package>` (short: `npm i`) downloads a package and everything it depends on into the `node_modules` folder and records it in package.json. **Dependencies** are needed when the application runs (a date library, a framework). **Dev dependencies**, added with `-D` (`--save-dev`), are needed only during

development: build tools, linters, test frameworks. Running `npm install` with no name installs everything listed, which is how a teammate sets up a cloned project. `node_modules` is never committed to Git: it can always be recreated.

COMMON INSTALL COMMANDS main.js

```
npm install dayjs                # dependency
npm install -D vite eslint prettier   # dev dependencies
npm uninstall dayjs              # remove
npm install                      # install everything in package.json (after cloning)
npm outdated                     # which packages have newer versions
npm update                       # update within the allowed ranges
npm ls --depth=0                 # what is installed
```

TYPICAL OUTPUT

```
added 1 package, and audited 2 packages in 1s
found 0 vulnerabilities

added 156 packages, and audited 158 packages in 9s
found 0 vulnerabilities
```

TIP

`npm install -g` installs a tool globally (for the command line everywhere), but project tools should be dev dependencies so that every developer and the CI server use the same version. Run them with npm scripts or npx.

19.2.3 Semantic versioning and lock files

npm packages follow **semantic versioning** (semver): `MAJOR.MINOR.PATCH`, for example 3.8.1. A **patch** release fixes bugs, a **minor** release adds features in a backward-compatible way, and a **major** release may break existing code. In package.json, a **range** says which versions are acceptable: `^3.8.1` (the default) allows any 3.x.y from 3.8.1 up, but not 4.0.0; `~3.8.1` allows only patch updates (3.8.x); `3.8.1` pins one exact version. Because ranges could resolve to different versions on different days, npm writes `package-lock.json`, recording the **exact** version of every installed package, including the dependencies of dependencies. Commit the lock file; `npm ci` installs exactly what it lists (ideal for CI servers and deployments).

Range	Allows	Example matches for 3.8.1
`^3.8.1`	same major, >= given	3.8.1, 3.8.9, 3.9.0, 3.15.2 (not 4.0.0)
`~3.8.1`	same major.minor	3.8.1, 3.8.7 (not 3.9.0)
`3.8.1`	exactly this	3.8.1 only
`>=3.8.1 <4`	explicit range	like ^3.8.1
`*` / `latest`	anything / newest	avoid in real projects
`^0.4.2`	below 1.0, ^ allows patches only	0.4.2, 0.4.9 (not 0.5.0)

WHICH VERSIONS DOES A RANGE ACCEPT? main.js

```
// A simplified check of how npm interprets ^ and ~ ranges
function satisfies(version, range) {
  const [M, m, p] = version.split(".").map(Number);
  const op = range[0];
  const [RM, Rm, Rp] = range.replace(/^[\^~]/, "").split(".").map(Number);
  const gte = M > RM || (M === RM && (m > Rm || (m === Rm && p >= Rp)));
  if (op === "^") return gte && (RM > 0 ? M === RM : M === 0 && m === Rm);
  if (op === "~") return gte && M === RM && m === Rm;
  return M === RM && m === Rm && p === Rp;
}

const versions = ["3.8.0", "3.8.1", "3.8.9", "3.9.0", "3.15.2", "4.0.0"];
for (const range of ["^3.8.1", "~3.8.1", "3.8.1"]) {
  console.log(range.padEnd(7), "->", versions.filter((v) => satisfies(v, range)).join(", "));
}
console.log("^0.4.2  ->", ["0.4.1", "0.4.2", "0.4.9", "0.5.0"].filter((v) => satisfies(v,
"^0.4.2")).join(", "));
```

OUTPUT

```
^3.8.1  -> 3.8.1, 3.8.9, 3.9.0, 3.15.2
~3.8.1  -> 3.8.1, 3.8.9
3.8.1   -> 3.8.1
^0.4.2  -> 0.4.2, 0.4.9
```

19.2.4 npm scripts

The `"scripts"` section of package.json defines named commands for the project: `npm run <name>` runs one (`npm test` and `npm start` work without "run"). Scripts can call tools installed in `node_modules/.bin` by name, chain commands with `&&`, pass extra arguments after `--`, and have automatic `pre` / `post` hooks (a `prebuild` script runs before `build`). They document how to work with a project: a new developer only needs to read them.

PACKAGE.JSON JavaScript

```
{
  "name": "my-app",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "greet": "node greet.js",
    "test": "node --test --test-reporter=spec",
    "precheck": "echo running checks...",
    "check": "npm run greet --silent && npm test --silent"
  }
}
```

GREET.JS JavaScript

```
const arg = process.argv.find((a) => a.startsWith("--name="));
console.log(`Hello, ${arg ? arg.split("=")[1] : "Asha"}! (Node ${process.version})`);
```

DEFINING AND RUNNING SCRIPTS

main.js

```
npm run
npm run greet
npm run greet -- --name=Ravi
npm test
npm run check
```

TERMINAL

```
$ npm run
Lifecycle scripts included in my-app@1.0.0:
  test
    node --test --test-reporter=spec
available via `npm run-script`:
  greet
    node greet.js
  precheck
    echo running checks...
  check
    npm run greet --silent && npm test --silent
$ npm run greet

> my-app@1.0.0 greet
> node greet.js

Hello, Asha! (Node v22.22.2)
$ npm run greet -- --name=Ravi

> my-app@1.0.0 greet
> node greet.js --name=Ravi

Hello, Ravi! (Node v22.22.2)
$ npm test

> my-app@1.0.0 test
> node --test --test-reporter=spec

✔ adds numbers (0.784176ms)
✔ joins strings (0.172488ms)
ℹ tests 2
ℹ suites 0
ℹ pass 2
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 110.773926
$ npm run check

> my-app@1.0.0 precheck
> echo running checks...

running checks...

> my-app@1.0.0 check
> npm run greet --silent && npm test --silent

Hello, Asha! (Node v22.22.2)
✔ adds numbers (0.821896ms)
✔ joins strings (0.220145ms)
ℹ tests 2
ℹ suites 0
ℹ pass 2
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 105.406907
```

19.2.5 npx

`npx <command>` runs a package's command-line tool without installing it globally: it uses the project's local copy in `node_modules/.bin` if there is one, and otherwise downloads the package temporarily and runs it. It is the standard way to run project generators and one-off tools, such as `npm create vite@latest` (which uses npx behind the scenes), `npx eslint .` or `npx prettier --check .`.

RUNNING TOOLS WITH NPX

main.js

```
npx prettier --version            # local copy if installed, otherwise temporary download
npx eslint src/                   # lint with the project's ESLint
npx serve dist                    # serve a folder over HTTP, no install
npx cowsay "Learn Being Forward"  # any package with a CLI
```

TYPICAL OUTPUT

```
3.8.1
 ┌──────────────────────────────────────┐
 │   Serving!                           │
 │   - Local:    http://localhost:3000  │
 └──────────────────────────────────────┘
```

19.2.6 Alternatives: pnpm, Yarn, Bun

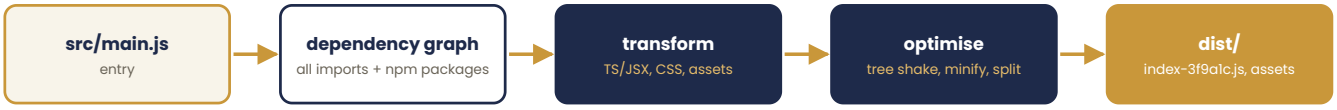
Several tools read the same package.json and npm registry. **pnpm** stores each package version once on disk in a global store and links it into projects, saving space and time, and is strict about undeclared dependencies; it is popular for monorepos. **Yarn** introduced lock files and workspaces early; Yarn 1 is still common in older projects, and modern Yarn (Berry) offers Plug'n'Play installs without node_modules. **Bun** is a fast all-in-one JavaScript runtime that includes a package manager, bundler and test runner. Use whichever the project already uses (its lock file tells you: `package-lock.json`, `pnpm-lock.yaml`, `yarn.lock`, `bun.lock`).

Task	npm	pnpm	Yarn	Bun
install all	`npm install`	`pnpm install`	`yarn`	`bun install`
add a dependency	`npm i dayjs`	`pnpm add dayjs`	`yarn add dayjs`	`bun add dayjs`
add a dev dependency	`npm i -D vite`	`pnpm add -D vite`	`yarn add -D vite`	`bun add -d vite`
run a script	`npm run dev`	`pnpm dev`	`yarn dev`	`bun run dev`
run a tool once	`npx tool`	`pnpm dlx tool`	`yarn dlx tool`	`bunx tool`

19.3 Build tools

19.3.1 Why bundlers

Browsers can load ES modules directly, so why build at all? Because a real application has hundreds of modules plus npm packages (with bare imports such as `"react"` that browsers cannot resolve), and shipping them unprocessed means many requests and much unused code. A **bundler** starts from the entry file, follows every import to build a **dependency graph**, and produces a few optimised files: it resolves npm packages, removes unused exports (**tree shaking**), **minifies** (removes whitespace, shortens names), splits code into chunks for lazy loading, processes CSS, images and fonts, adds content **hashes** to file names for long-term caching, and can transpile TypeScript, JSX and newer syntax. In development, build tools also provide a local server with instant reloading.



WHAT A BUNDLER DOES FOR A PRODUCTION BUILD

19.3.2 Vite

Vite (French for "fast") is the most popular build tool for new front-end projects and the default for Vue, Svelte and many React set-ups. In development it does not bundle at all: it serves your source files as native ES modules and transforms them on request, so the dev server starts instantly and **Hot Module Replacement** (HMR) updates the page within milliseconds of saving a file, keeping the application state. For production, `vite build` creates an optimised bundle in `dist/` (using Rollup/Rolldown). It supports TypeScript, JSX, CSS modules, Sass and environment variables out of the box.

CREATING AND RUNNING A VITE PROJECT main.js

```
npm create vite@latest weather-app -- --template vanilla
cd weather-app
npm install
npm run dev          # development server with HMR
npm run build        # production build in dist/
npm run preview      # serve the production build locally
```

TYPICAL OUTPUT

```
  VITE v7.1.0  ready in 212 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose

vite v7.1.0 building for production...
✓ 9 modules transformed.
dist/index.html                  0.46 kB │ gzip: 0.30 kB
dist/assets/index-BH2fTqZs.css   1.24 kB │ gzip: 0.64 kB
dist/assets/index-DkQ1wE9x.js    3.87 kB │ gzip: 1.71 kB
✓ built in 148ms
```

STRUCTURE OF A VITE PROJECT text

```
weather-app/
├── index.html            <- entry HTML: <script type="module" src="/src/main.js">
├── package.json
├── vite.config.js        <- optional configuration
├── public/               <- copied as-is (favicon, robots.txt)
└── src/
    ├── main.js           <- entry module
    ├── api.js
    ├── storage.js
    └── style.css         <- imported from JS: import "./style.css"
```

19.3.3 webpack (overview)

webpack (2012) made bundling mainstream and still powers many existing applications, including older Create React App and Angular projects. Everything is configured in `webpack.config.js`: an **entry**, an **output**, **loaders** that transform files (babel-loader for JavaScript, css-loader for CSS) and **plugins** for everything else (generating HTML, extracting CSS). It is extremely flexible but slower and more complex to configure than Vite. Newer Rust-based tools such as Rspack (webpack-compatible), Turbopack (Next.js) and esbuild offer much faster builds.

A MINIMAL WEBPACK CONFIGURATION JavaScript

```
// webpack.config.js
import HtmlWebpackPlugin from "html-webpack-plugin";

export default {
  mode: "production",
  entry: "./src/main.js",
  output: { filename: "[name].[contenthash].js", path: new URL("./dist", import.meta.url).pathname,
clean: true },
  module: {
    rules: [
      { test: /\.css$/, use: ["style-loader", "css-loader"] },
      { test: /\.js$/, exclude: /node_modules/, use: "babel-loader" },
    ],
  },
  plugins: [new HtmlWebpackPlugin({ template: "./index.html" })],
};
```

19.3.4 Transpiling with Babel (overview)

A **transpiler** converts source code into equivalent code in another version or dialect. **Babel** converts modern JavaScript into older syntax for browsers that need it, and also strips TypeScript types and converts JSX. With `@babel/preset-env` and a **browserslist** setting, it transforms only what your target browsers do not support. New **APIs** (such as `Promise.withResolvers`) cannot be added by syntax transformation; they need **polyfills** (core-js, Module 20). Today, many projects use faster transpilers built into their tools (esbuild in Vite, SWC in Next.js), but the idea is the same.

WHAT BABEL DOES TO MODERN SYNTAX main.js

```
// input (modern)
const total = (items) => items?.reduce((t, { price = 0 }) => t + price, 0) ?? 0;

// output for old browsers (simplified Babel output)
"use strict";
var total = function total(items) {
  var _items$reduce;
  return (_items$reduce = items === null || items === void 0 ? void 0 : items.reduce(function (t,
_ref) {
    var _ref$price = _ref.price, price = _ref$price === void 0 ? 0 : _ref$price;
    return t + price;
  }, 0)) !== null && _items$reduce !== void 0 ? _items$reduce : 0;
};
```

NOTE
The output behaves identically but uses only ES5 syntax: arrow functions, destructuring defaults, ?. and ?? have been rewritten.

19.3.5 Environment variables

Settings that differ between environments (API URLs, feature flags, keys) should not be hard-coded. They are supplied as **environment variables**, usually kept in `.env` files that are **not committed** to Git (commit a `.env.example` listing the names instead). In Node.js, read them from `process.env`; Node.js 20.6+ can load a file with `--env-file`, older projects use the dotenv package. In Vite, variables prefixed with `VITE_` are exposed to browser code as `import.meta.env.VITE_NAME`, along with `import.meta.env.MODE`, `DEV` and `PROD`.

.ENV

text

```
# never commit this file
PORT=3000
API_URL=https://api.open-meteo.com/v1
FEATURE_CHAT=false
```

SERVER.JS

JavaScript

```
const port = Number(process.env.PORT ?? 3000);
const apiUrl = process.env.API_URL;
const chatEnabled = process.env.FEATURE_CHAT === "true";       // values are always strings
console.log({ port, apiUrl, chatEnabled, mode: process.env.NODE_ENV ?? "development" });
```

READING A .ENV FILE IN NODE.JS

main.js

```
node --env-file=.env server.js
PORT=8080 node --env-file=.env server.js
```

TERMINAL

```
$ node --env-file=.env server.js
{
  port: 3000,
  apiUrl: 'https://api.open-meteo.com/v1',
  chatEnabled: false,
  mode: 'development'
}
$ PORT=8080 node --env-file=.env server.js
{
  port: 8080,
  apiUrl: 'https://api.open-meteo.com/v1',
  chatEnabled: false,
  mode: 'development'
}
```

NOTE

A variable set in the shell (here `PORT=8080`) takes priority over the value in the file. Anything shipped to the browser, including `VITE_` variables, is visible to users: never put secret keys there.

19.4 Code quality tools

19.4.1 ESLint

ESLint is a **linter**: it analyses source code without running it and reports likely bugs (unused variables, use of undeclared names, unreachable code, `==` comparisons) and style problems, according to configurable **rules**. Many problems can be fixed automatically with `--fix`. ESLint 9+ uses a **flat config** file, `eslint.config.js`, exporting an array of configuration objects; typical projects start from the recommended rules of `@eslint/js` and add plugins for frameworks or TypeScript. Editors show ESLint problems as you type.

ESLINT.CONFIG.JS JavaScript

```
export default [
  {
    files: ["**/*.js"],
    languageOptions: { ecmaVersion: "latest", sourceType: "module", globals: { console: "readonly" }
},
    rules: {
      "no-unused-vars": "warn",
      "no-undef": "error",
      "eqeqeq": "error",
      "prefer-const": "error",
      "no-unreachable": "error",
      "no-var": "error",
    },
  },
];
```

APP.JS JavaScript

```
var city = "Mysuru";
let temp = 27;
const unused = 1;

function describe(t) {
  if (t == "27") {
    return `${city}: warm`;
    console.log("never runs");
  }
  return totl;
}
console.log(describe(temp));
```

LINTING A FILE WITH PROBLEMS main.js

```
eslint app.js
eslint app.js --fix
cat app.js
```

TERMINAL

```
$ eslint app.js

app.js
   1:1   error    Unexpected var, use let or const instead         no-var
   2:5   error    'temp' is never reassigned. Use 'const' instead  prefer-const
   3:7   warning  'unused' is assigned a value but never used      no-unused-vars
   6:9   error    Expected '===' and instead saw '=='              eqeqeq
   8:5   error    Unreachable code                                 no-unreachable
  10:10  error    'totl' is not defined                            no-undef

✖ 6 problems (5 errors, 1 warning)
  2 errors and 0 warnings potentially fixable with the `--fix` option.
$ eslint app.js --fix

app.js
   3:7   warning  'unused' is assigned a value but never used  no-unused-vars
   6:9   error    Expected '===' and instead saw '=='          eqeqeq
   8:5   error    Unreachable code                             no-unreachable
  10:10  error    'totl' is not defined                        no-undef

✖ 4 problems (3 errors, 1 warning)
$ cat app.js
const city = "Mysuru";
const temp = 27;
const unused = 1;

function describe(t) {
  if (t == "27") {
    return `${city}: warm`;
    console.log("never runs");
  }
  return totl;
}
console.log(describe(temp));
```

NOTE

`--fix` automatically changed `var` and `let` to `const` because those variables are never reassigned; the remaining errors (the misspelt `totl`, `==`, unreachable code) need a human decision.

19.4.2 Prettier

Prettier is an opinionated **code formatter**: it reprints code in one consistent style (indentation, quotes, semicolons, line length, trailing commas), so teams stop arguing about style and diffs only show real changes. It has very few options, set in `.prettierrc`. Use ESLint for code quality and Prettier for formatting (disable ESLint's formatting rules with eslint-config-prettier if a shared config includes any), and run both automatically: format on save in the editor, and check in CI or a pre-commit hook (husky with lint-staged).

.PRETTIERRC text

```
{ "singleQuote": false, "semi": true, "printWidth": 80, "trailingComma": "all" }
```

MESSY.JS JavaScript

```
const   courses=[{title:'JavaScript',weeks:12},{title:"React",weeks:8}]
function total(list){return list.reduce((t,c)=>t+c.weeks,0)}
if(total(courses)>10){console.log('long programme: '+total(courses)+' weeks')}else{
console.log("short") }
```

FORMATTING WITH PRETTIER main.js

```
prettier --check messy.js
prettier messy.js
```

TERMINAL

```
$ prettier --check messy.js
Checking formatting...
[warn] messy.js
[warn] Code style issues found in the above file. Run Prettier with --write to fix.
$ prettier messy.js
const courses = [
  { title: "JavaScript", weeks: 12 },
  { title: "React", weeks: 8 },
];
function total(list) {
  return list.reduce((t, c) => t + c.weeks, 0);
}
if (total(courses) > 10) {
  console.log("long programme: " + total(courses) + " weeks");
} else {
  console.log("short");
}
```

FORMAT AND FIX ON EVERY SAVE IN VS CODE main.js

```
// .vscode/settings.json
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.codeActionsOnSave": { "source.fixAll.eslint": "explicit" }
}
```

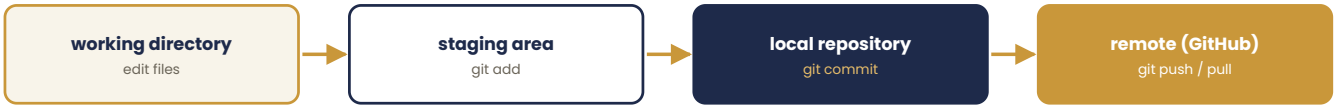
NOTE

With these settings VS Code runs Prettier and ESLint's automatic fixes each time a file is saved.

19.5 Version control

19.5.1 Git and GitHub basics

Git is a distributed version control system: it records snapshots (**commits**) of a project over time, so you can see what changed, when and why, return to any earlier version, work on features in parallel **branches** and merge them. Every developer has the full history locally. **GitHub** (like GitLab and Bitbucket) hosts Git repositories online and adds collaboration: **pull requests** for code review, issues, and Actions for automated testing and deployment. The everyday loop is: change files, `git add` them to the staging area, `git commit` with a message, and `git push` to the remote.



WHERE CHANGES TRAVEL IN GIT

.GITIGNORE text

```
node_modules/
dist/
.env
.DS_Store
```

A FIRST GIT WORKFLOW main.js

```
git init -q -b main
git config user.name "Asha Rao" && git config user.email "asha@lbf.in"
git status --short
git add .
git commit -q -m "Initial weather app" && git log --oneline | sed 's/^[0-9a-f]* /<hash> /'
git switch -q -c feature/recent-searches
echo 'export const MAX_RECENT = 5;' > src/config.js
git add src/config.js && git commit -q -m "Add recent searches limit"
git switch -q main && git merge -q feature/recent-searches && git log --oneline | sed 's/^[0-9a-f]*
/<hash> /'
git branch
cat .gitignore
```

TERMINAL

```
$ git init -q -b main
$ git config user.name "Asha Rao" && git config user.email "asha@lbf.in"
$ git status --short
?? .gitignore
?? index.html
?? src/
$ git add .
$ git commit -q -m "Initial weather app" && git log --oneline | sed 's/^[0-9a-f]* /<hash> /'
<hash> Initial weather app
$ git switch -q -c feature/recent-searches
$ echo 'export const MAX_RECENT = 5;' > src/config.js
$ git add src/config.js && git commit -q -m "Add recent searches limit"
$ git switch -q main && git merge -q feature/recent-searches && git log --oneline | sed 's/^[0-9a-f]*
/<hash> /'
<hash> Add recent searches limit
<hash> Initial weather app
$ git branch
  feature/recent-searches
* main
$ cat .gitignore
node_modules/
dist/
.env
.DS_Store
```

NOTE

`.env` was never committed because `.gitignore` lists it; `git status` does not even show it. Commit hashes are replaced by <hash> here because they differ on every machine.

Command	Purpose
`git clone <url>`	copy a remote repository to your machine
`git status` / `git diff`	what changed and is staged
`git add <files>` / `git commit -m "msg"`	stage and record a snapshot
`git log --oneline`	history
`git switch -c <branch>` / `git merge`	create a branch / bring its commits in
`git remote add origin <url>`	connect to GitHub
`git push -u origin main` / `git pull`	upload / download commits
`git restore <file>` / `git revert <commit>`	discard changes / undo a commit safely

19.5.2 Deploying static sites (GitHub Pages, Netlify, Vercel)

A front-end application without its own server is a **static site**: after `npm run build`, the `dist/` folder contains only HTML, CSS, JavaScript and assets, which any web server or CDN can host. Free hosting platforms connect to your GitHub repository and rebuild and redeploy automatically on every push, give each pull request its own preview URL, and provide HTTPS and custom domains. **GitHub Pages** serves a branch or the output of a GitHub Actions workflow; **Netlify** and **Vercel** detect Vite projects automatically (build command `npm run build`, output folder `dist`) and also offer serverless functions.

Platform	How to deploy	Notes
GitHub Pages	repository Settings > Pages, source "GitHub Actions" with a Vite workflow	set `base: "/repo-name/"` in vite.config.js for project sites
Netlify	import the repo, or drag the dist folder onto app.netlify.com/drop	forms, redirects, functions
Vercel	import the repo at vercel.com/new, or run `npx vercel`	made by the Next.js team; preview deployments
Cloudflare Pages	connect the repo	global CDN, Workers for server code

AUTOMATIC DEPLOYMENT ON EVERY PUSH

text

```
# .github/workflows/deploy.yml  (GitHub Pages for a Vite project)
name: Deploy
on:
  push:
    branches: [main]
permissions: { contents: read, pages: write, id-token: write }
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: github-pages
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npm run lint && npm run build
      - uses: actions/upload-pages-artifact@v3
        with: { path: dist }
      - uses: actions/deploy-pages@v4
```

Modules give each file its own scope with explicit dependencies. ES modules use export (named, default, re-exports) and import (named, default, namespace, side-effect); imports are live, read-only bindings, modules run once in strict mode, and dynamic import() returns a promise and enables code splitting. Browsers load modules with <script type="module"> (deferred, CORS, import maps for bare names); JSON modules use import attributes. CommonJS (require, module.exports) remains common in Node.js and interoperates with ESM. npm manages packages via package.json (dependencies vs devDependencies), semver ranges (^, ~, exact) and lock files (npm ci), scripts (npm run) and npx; pnpm, Yarn and Bun are alternatives. Bundlers build a dependency graph and produce optimised, tree-shaken, minified, hashed output; Vite offers instant dev servers with HMR, webpack is the configurable veteran, Babel transpiles syntax, and .env files supply environment variables. ESLint finds bugs, Prettier formats, Git tracks history in commits and branches, and platforms such as GitHub Pages, Netlify and Vercel deploy static builds automatically.

PRACTICE CHECKPOINT

Convert the **weather app** into **ES modules** inside a **Vite project** with **ESLint** and **Prettier**, and **deploy** it.

A suggested module structure keeps each responsibility in its own file. The modules below are the weather app from Module 18, split up; the commands after them lint the project, format it with Prettier and confirm the formatting, and run the unit tests for the storage module.

SRC/CONFIG.JS

JavaScript

```
export const API_URL = import.meta.env?.VITE_API_URL ?? "/api/weather";
export const MAX_RECENT = 5;
```

SRC/API.JS JavaScript

```
import { API_URL } from "./config.js";

export async function fetchWeather(city) {
  const res = await fetch(`${API_URL}?${new URLSearchParams({ city })}`, {
    signal: AbortSignal.timeout(5000),
  });
  if (res.status === 404) throw new Error(`City "${city}" was not found`);
  if (!res.ok) throw new Error(`Weather service error (${res.status})`);
  return res.json();
}
```

SRC/RECENT.JS JavaScript

```
import { MAX_RECENT } from "./config.js";

const KEY = "recentCities";

export function loadRecent(storage = localStorage) {
  try {
    return JSON.parse(storage.getItem(KEY)) ?? [];
  } catch {
    return [];
  }
}

export function addRecent(city, storage = localStorage) {
  const list = [
    city,
    ...loadRecent(storage).filter((c) => c.toLowerCase() !== city.toLowerCase()),
  ].slice(0, MAX_RECENT);
  storage.setItem(KEY, JSON.stringify(list));
  return list;
}
```

SRC/MAIN.JS JavaScript

```
import { fetchWeather } from "./api.js";
import { loadRecent, addRecent } from "./recent.js";

const form = document.querySelector("#search");
const status = document.querySelector("#status");
const result = document.querySelector("#result");

async function search(city) {
  status.textContent = `Loading weather for ${city}...`;
  try {
    const w = await fetchWeather(city);
    result.textContent = `${w.city}: ${w.temp}°C, ${w.description}`;
    status.textContent = `Recent: ${addRecent(w.city).join(", ")}`;
  } catch (e) {
    status.textContent = e.message;
  }
}

form.addEventListener("submit", (e) => {
  e.preventDefault();
  const city = form.elements.city.value.trim();
  if (city) search(city);
});
status.textContent = `Recent: ${loadRecent().join(", ") || "none"}`;
```

CHECKING THE MODULAR WEATHER APP

main.js

```
npm run lint --silent && echo "lint: no problems"
npm run format --silent > /dev/null && npm run format:check --silent
node --test --test-reporter=spec
```

TERMINAL

```
$ npm run lint --silent && echo "lint: no problems"
lint: no problems
$ npm run format --silent > /dev/null && npm run format:check --silent
Checking formatting...
All matched files use Prettier code style!
$ node --test --test-reporter=spec
✔ keeps the five most recent unique cities (3.319444ms)
✔ survives corrupted storage (0.358885ms)
ℹ tests 2
ℹ suites 0
ℹ pass 2
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 149.313863
```

To finish: create the project with `npm create vite@latest weather-app -- --template vanilla`, copy these files into `src/`, install the tools with `npm install -D eslint prettier`, push the repository to GitHub, and import it into Netlify or Vercel (build command `npm run build`, output folder `dist`), or add the GitHub Pages workflow from 19.5.2. Set `VITE_API_URL` in the platform's environment settings.

MODULE 20 · ADVANCED

Advanced JavaScript internals and patterns

Minimum duration **3 days** | Level **Advanced**

This module looks under the hood and then upward to design. It explains how engines such as V8 parse, interpret and optimise code (Ignition, TurboFan, hidden classes and inline caches), how memory is allocated and garbage-collected and how leaks happen and are found, metaprogramming with Proxy and Reflect, functional programming principles, the classic design patterns in JavaScript form, performance measurement and Web Vitals, and finally how to write polyfills for familiar built-ins.

IN THIS MODULE

20.1 How engines run JavaScript
20.2 Memory management
20.3 Metaprogramming
20.4 Functional programming
20.5 Design patterns in JavaScript
20.6 Performance
20.7 Writing polyfills

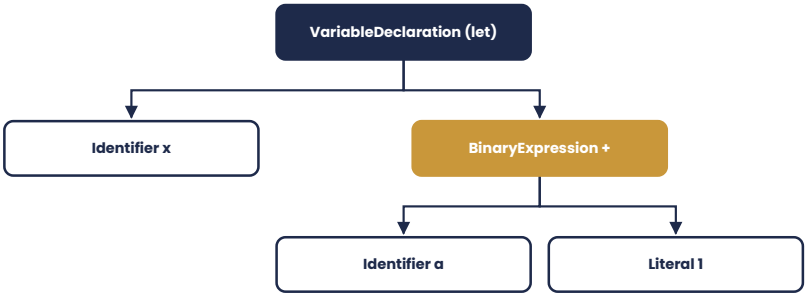
20.1 How engines run JavaScript

20.1.1 Parsing, the AST and bytecode

A JavaScript engine (V8 in Chrome and Node.js, SpiderMonkey in Firefox, JavaScriptCore in Safari) turns source text into running machine instructions in stages. The **scanner** (tokenizer) splits the text into tokens: keywords, names, numbers, operators. The **parser** checks the grammar (this is where SyntaxErrors come from) and builds an **Abstract Syntax Tree** (AST), a tree of nodes describing the program's structure. To start quickly, V8 parses functions **lazily**: only a fast pre-parse until a function is first called. The **interpreter** then compiles the AST into compact **bytecode**, instructions for a virtual machine, and executes it. Tools such as Babel, ESLint and Prettier work on ASTs too.



FROM SOURCE TEXT TO EXECUTION



THE AST OF LET X = A + 1;

PEEKING AT V8 BYTECODE

main.js

```
// Node.js can show the bytecode V8 generates for a function
const { execFileSync } = require("node:child_process");
const fs = require("node:fs");
fs.writeFileSync("add.js", "function addOne(a) { return a + 1; }\naddOne(5);\n");
const out = execFileSync(process.execPath, ["--print-bytecode", "--print-bytecode-filter=addOne",
"add.js"]).toString();
const lines = out.split("\n").filter((l) => /^\s+\d+ (S|E)?>?\s*0x/.test(l) ||
/Ldar|AddSmi|Return|Add|Star/.test(l));
console.log("V8 bytecode for addOne(a) { return a + 1; }:");
for (const l of lines.slice(0, 6)) console.log("  " + l.replace(/^.*?@\s*\d+ : [0-9a-f ]+/,
"").trim());
```

OUTPUT

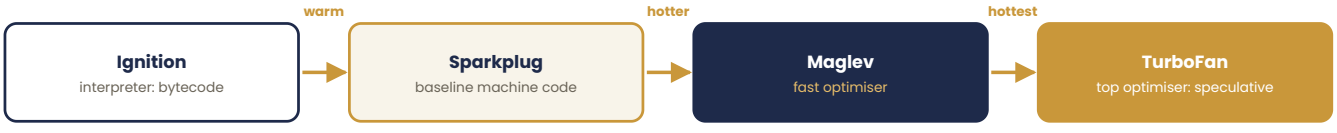
```
V8 bytecode for addOne(a) { return a + 1; }:
  Ldar a0
  AddSmi [1], [0]
  Return
```

NOTE

`Ldar a0` loads the first argument into the accumulator register, `AddSmi [1]` adds the small integer 1, and `Return` returns the accumulator. You will never write bytecode, but seeing it shows that the engine does not run your source text directly.

20.1.2 JIT compilation in V8 (Ignition and TurboFan, overview)

Interpreting bytecode starts fast but runs slowly. Modern engines therefore use **Just-In-Time (JIT) compilation**: while code runs, the engine profiles it, and functions that become **hot** (called often, or with long loops) are compiled to optimised **machine code**. In V8, **Ignition** is the interpreter, **Sparkplug** a very fast baseline compiler, **Maglev** a mid-tier optimiser, and **TurboFan** the top-tier optimising compiler. Optimising compilers make **speculative** assumptions based on what they have seen ("`a` has always been a small integer"); if an assumption fails later (a string arrives), the engine **deoptimises**, discarding the machine code and falling back to bytecode. Code whose types stay consistent therefore runs fastest.



V8 TIERS: HOTTER CODE GETS MORE OPTIMISATION (WITH DEOPTIMISATION BACK ON FAILED ASSUMPTIONS)

HOT CODE GETS FASTER AS THE JIT OPTIMISES IT

main.js

```
function sum(arr) {
  let total = 0;
  for (let i = 0; i < arr.length; i++) total += arr[i];
  return total;
}

const numbers = Array.from({ length: 1e6 }, (_, i) => i % 100);
const time = (label, fn) => { const t = performance.now(); const r = fn(); return [label, r,
performance.now() - t]; };

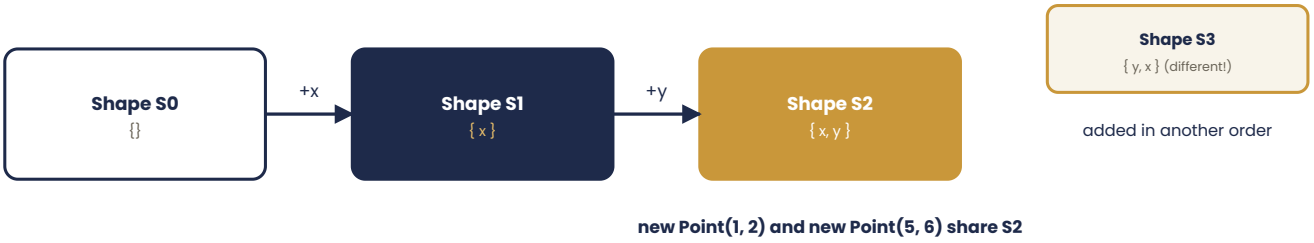
const runs = [];
for (let round = 1; round <= 5; round++) runs.push(time(`round ${round}`, () => sum(numbers)));
const first = runs[0][2], last = runs[4][2];
console.log("result each time:", runs[0][1]);
console.log(`later rounds faster than the first (JIT warm-up): ${last < first}`);
```

OUTPUT

```
result each time: 49500000
later rounds faster than the first (JIT warm-up): true
```

20.1.3 Hidden classes and inline caching (overview)

JavaScript objects can gain and lose properties at any time, which would make property access slow if every lookup searched a dictionary. V8 instead assigns each object a hidden **shape** (also called a hidden class or map) describing its property layout; objects created the same way, with properties added in the same order, **share** a shape. At each property access in the code, an **inline cache** remembers the shape it saw and where the property was, so the next access with the same shape is a direct memory read. Accesses that see one shape are **monomorphic** (fastest), a few shapes **polymorphic**, many shapes **megamorphic** (slow). Practical advice: initialise all properties in the constructor, in the same order; avoid adding properties later or using `delete`; keep arrays of one element type.



OBJECTS BUILT IN THE SAME ORDER SHARE A HIDDEN SHAPE

CHECKING HIDDEN CLASSES WITH A V8 INTRINSIC main.js

```
// V8 can tell us whether two objects share a hidden class (needs a special flag)
const v8 = require("node:v8");
v8.setFlagsFromString("--allow-natives-syntax");
const sameShape = new Function("a", "b", "return %HaveSameMap(a, b)");

class Point { constructor(x, y) { this.x = x; this.y = y; } }
const p1 = new Point(1, 2), p2 = new Point(5, 6);
console.log("two Points:", sameShape(p1, p2));

const a = { x: 1, y: 2 }, b = { y: 2, x: 1 };
console.log("same properties, different order:", sameShape(a, b));

const c = { x: 1, y: 2 }, d = { x: 1, y: 2 };
d.z = 3;                                        // added later: d moves to a new shape
console.log("one object got an extra property:", sameShape(c, d));

const e = { x: 1, y: 2 }, f = { x: 1, y: 2 };
delete f.y;                                     // delete changes the shape too
console.log("after delete:", sameShape(e, f));

function sumX(points) { let t = 0; for (const p of points) t += p.x; return t; }
console.log("sumX sees one shape (monomorphic) for", [p1, p2].length, "Points:", sumX([p1, p2]));
```

OUTPUT

```
two Points: true
same properties, different order: false
one object got an extra property: false
after delete: false
sumX sees one shape (monomorphic) for 2 Points: 6
```

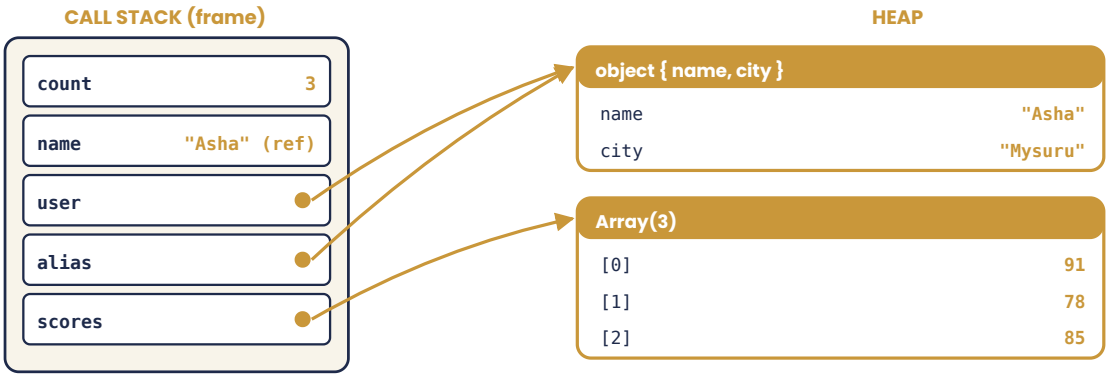
NOTE

`%HaveSameMap` is an internal V8 function, available only with `--allow-natives-syntax` and used here purely for illustration; never use it in real code. These optimisations matter in hot loops over many objects; in ordinary code, clarity matters far more.

20.2 Memory management

20.2.1 Stack and heap

JavaScript manages memory automatically, but knowing where values live explains copying, sharing and leaks. The **call stack** holds each function call's frame: its local variables, small primitive values and **references**. It is fast, ordered and freed automatically when a function returns. The **heap** is a large, unordered region for objects, arrays, functions and closures, anything whose size or lifetime is not fixed. A variable holding an object holds only a reference (address) to the heap object; copying the variable copies the reference, so both variables see the same object (Module 3).



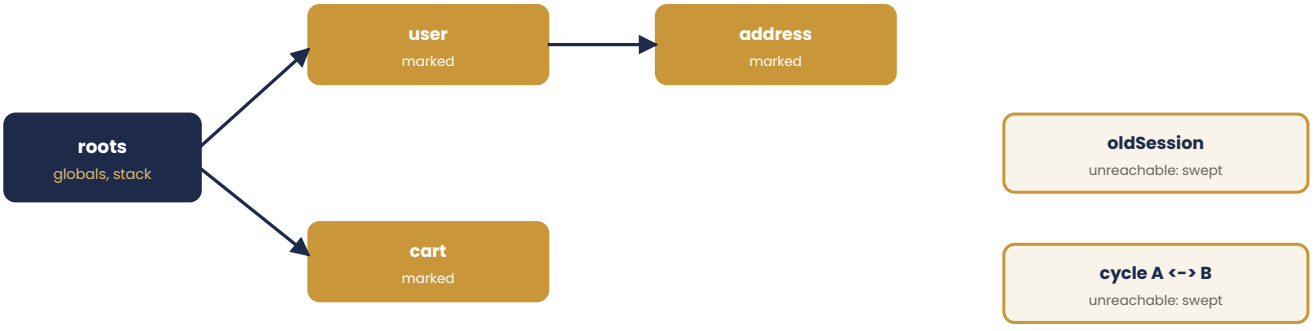
TWO VARIABLES CAN REFERENCE THE SAME HEAP OBJECT

NOTE

Engines are free to optimise: V8 stores small integers directly ("Smis") and keeps strings on the heap, and short-lived objects may never leave the fast young generation. The model above is the right mental picture, not an exact memory map.

20.2.2 Garbage collection: reachability and mark-and-sweep

The **garbage collector** (GC) frees heap memory that the program can no longer use. The criterion is **reachability**: an object is kept if it can be reached from a **root** (global variables, the current call stack, active closures and timers) by following references; everything else is garbage, even if objects still reference each other in a cycle. The basic algorithm is **mark-and-sweep**: starting from the roots, mark every reachable object, then sweep away all unmarked ones. V8 is **generational**: most objects die young, so new objects go into a small young generation that is collected very often and quickly (scavenging), and survivors are promoted to the old generation, collected less often with incremental and concurrent marking to avoid long pauses.



MARK FROM THE ROOTS; SWEEP EVERYTHING UNMARKED (EVEN CYCLES)

UNREACHABLE OBJECTS ARE COLLECTED, CYCLES INCLUDED main.js

```
const v8 = require("node:v8");
const vm = require("node:vm");
v8.setFlagsFromString("--expose-gc");
const gc = vm.runInNewContext("gc");                    // allows forcing a collection for this demo
const heapMB = () => (process.memoryUsage().heapUsed / 1048576).toFixed(0);

gc();
console.log("baseline heap: ~" + Math.round(heapMB() / 5) * 5 + " MB");
let big = Array.from({ length: 1e6 }, (_, i) => ({ id: i }));   // about 1 million objects
console.log("after allocating:", Number(heapMB()) > 30 ? "much larger" : heapMB());

let a = { name: "A" }, b = { name: "B" };
a.partner = b; b.partner = a;                           // a reference cycle
big = null; a = null; b = null;                         // drop the roots
gc();
console.log("after dropping references and collecting:", Number(heapMB()) < 20 ? "back near the
baseline" : heapMB() + " MB");
```

OUTPUT

```
baseline heap: ~5 MB
after allocating: much larger
after dropping references and collecting: back near the baseline
```

20.2.3 Memory leaks: globals, timers, detached DOM nodes, closures

A **memory leak** is memory that is still reachable, and therefore kept, although the program will never use it again. Over time the page slows down or crashes. The common causes are: accidental **globals** (assigning to an undeclared variable in sloppy mode, or growing global caches and arrays forever); **timers and listeners** never cleared (a `setInterval` or a listener on `window` keeps its callback, and everything that callback references, alive); **detached DOM nodes** (elements removed from the page but still referenced from JavaScript, together with their whole subtree); and **closures** that capture large objects and outlive their purpose. Single-page applications are especially prone, because the page is never reloaded.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<ul id="list"></ul>
```

FOUR LEAK PATTERNS AND THEIR FIXES

main.js

```
// 1. Unbounded cache: grows forever
const cache = {};
function remember(key, value) { cache[key] = value; }            // never evicted
for (let i = 0; i < 1000; i++) remember("search-" + i, new Array(100).fill(i));
console.log("1. cache entries kept forever:", Object.keys(cache).length);

// 2. Timer that is never cleared keeps its closure alive
function startWidget() {
  const hugeData = new Array(100000).fill("x");                  // captured by the interval
  const id = setInterval(() => hugeData.length, 1000);
  return () => clearInterval(id);                                 // the fix: return a cleanup
function
}
const stopWidget = startWidget();
stopWidget();
console.log("2. interval cleared, its closure can now be collected");

// 3. Detached DOM node still referenced
const detached = [];
const list = document.querySelector("#list");
for (let i = 0; i < 3; i++) {
  const li = document.createElement("li");
  li.textContent = "row " + i;
  list.append(li);
  detached.push(li);                                              // reference kept in JS
}
list.replaceChildren();                                           // removed from the page...
console.log("3. rows on page:", list.children.length, "| still in memory via array:", detached.length,
"| connected:", detached[0].isConnected);
detached.length = 0;                                              // the fix: drop references

// 4. Listener on window keeps a component alive
function mountComponent() {
  const state = { bigBuffer: new Array(50000).fill(0) };
  const onResize = () => state.bigBuffer.length;
  window.addEventListener("resize", onResize);
  return () => window.removeEventListener("resize", onResize);    // cleanup on unmount
}
const unmount = mountComponent();
unmount();
console.log("4. listener removed on unmount");
```

BROWSER CONSOLE

```
1. cache entries kept forever: 1000
2. interval cleared, its closure can now be collected
3. rows on page: 0 | still in memory via array: 3 | connected: false
4. listener removed on unmount
```

Leak	Symptom	Fix
Accidental or growing globals	memory rises steadily	strict mode / modules; bounded caches (LRU), WeakMap keyed by objects
Forgotten timers	callbacks keep running after a view is gone	clearInterval / clearTimeout in cleanup
Forgotten listeners	handlers run for removed components	removeEventListener, { once }, AbortController

Leak	Symptom	Fix
Detached DOM nodes	"Detached" elements in heap snapshots	drop references to removed elements
Closures capturing large data	big arrays retained by small functions	capture only what is needed; null out references

20.2.4 WeakRef and FinalizationRegistry (overview)

ES2021 added two low-level tools. A **WeakRef** holds a reference to an object **without** keeping it alive: `ref.deref()` returns the object, or `undefined` once it has been collected. A **FinalizationRegistry** lets you register a callback that may run some time **after** an object has been collected, for example to release an associated external resource. Garbage collection timing is unpredictable, callbacks may never run, and engines differ, so these should be used rarely: mainly for caches of expensive, re-creatable objects. Prefer WeakMap/WeakSet (Module 12) for attaching data to objects.

WEAKREF AND FINALIZATIONREGISTRY (WITH A FORCED GC FOR THE DEMO) main.js

```
const v8 = require("node:v8"), vm = require("node:vm");
v8.setFlagsFromString("--expose-gc");
const gc = vm.runInNewContext("gc");

const registry = new FinalizationRegistry((label) => console.log(`finalizer: "${label}" was garbage-
collected`));

let image = { pixels: new Uint8Array(5_000_000) };          // an expensive object
const ref = new WeakRef(image);
registry.register(image, "thumbnail-42");

console.log("deref while referenced:", ref.deref() === image);
image = null;                                               // drop the only strong reference

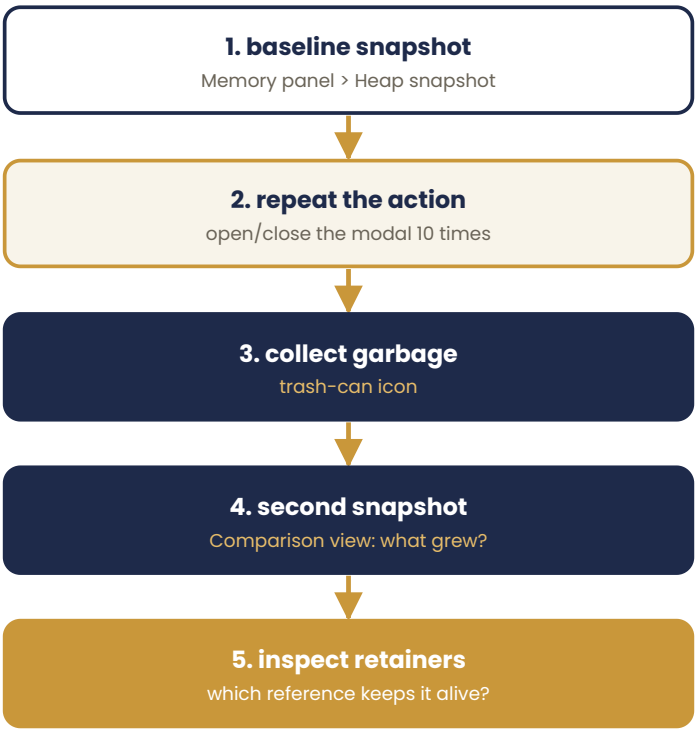
setTimeout(() => {
  gc();                                                     // forced here; normally it happens
whenever V8 decides
  console.log("deref after collection:", ref.deref());
  setTimeout(() => console.log("(finalizers run some time after a collection, or possibly never)"),
50);
}, 10);
```

OUTPUT

```
deref while referenced: true
deref after collection: undefined
(finalizers run some time after a collection, or possibly never)
```

20.2.5 Finding leaks with the DevTools Memory panel

Chrome DevTools has the tools to confirm and locate leaks. The **Performance** panel with the "Memory" checkbox shows the JS heap over time: a sawtooth that returns to the same level is healthy; a staircase that keeps rising after repeated actions suggests a leak. The **Memory** panel takes **heap snapshots**: take one, perform the suspect action several times (open and close a dialog), force GC (the trash-can icon), take another, and use the **Comparison** view to see which object types grew. Filtering for "Detached" finds removed DOM nodes still in memory, and the **Retainers** pane shows the chain of references keeping an object alive. The "Allocation instrumentation on timeline" option shows which code allocated memory that stayed.



A HEAP-SNAPSHOT WORKFLOW FOR FINDING A LEAK

WHAT A LEAK LOOKS LIKE IN A HEAP SNAPSHOT main.js

```
Summary filter: "Detached"
Constructor                    Distance  Shallow Size  Retained Size
▼ Detached HTMLLIElement   ×300      6         27 600       182 400
  ▼ Retainers
      [12] in Array @84211
      detached in system / Context @84105
      startList() in app.js:41
```

DEVTOOLS (MEMORY PANEL)

300 detached <li> elements are retained by an array named detached inside startList(): exactly the bug in pattern 3 above.

20.3 Metaprogramming

Metaprogramming is code that inspects or changes the behaviour of other code: intercepting property access, function calls or object creation. JavaScript provides `Proxy` and `Reflect` for this, alongside the property descriptors and well-known symbols already met. Frameworks rely on it heavily: Vue 3's reactivity, MobX, Immer and many validation and ORM libraries are built on proxies.

20.3.1 Proxy and traps

`new Proxy(target, handler)` creates an object that stands in front of `target`. Every fundamental operation on the proxy (reading a property, writing one, checking `in`, deleting, listing keys, calling a function, using `new`) is first passed to the matching **trap** method of the handler, if it has one; operations without a trap go straight to the target. Common uses are validation, default values, logging and access control, negative array indexes, and change detection.

PROXIES WITH GET, SET, HAS, DELETEPROPERTY AND APPLY TRAPS

main.js

```
// 1. Validation and logging
const user = new Proxy({ name: "Asha", age: 21 }, {
  get(target, prop, receiver) {
    console.log(`  get ${String(prop)}`);
    return Reflect.get(target, prop, receiver);
  },
  set(target, prop, value) {
    if (prop === "age" && (!Number.isInteger(value) || value < 0)) throw new TypeError(`Invalid age:
${value}`);
    console.log(`  set ${prop} = ${JSON.stringify(value)}`);
    target[prop] = value;
    return true;                                  // signal success
  },
  has(target, prop) { return !String(prop).startsWith("_") && prop in target; },
  deleteProperty(target, prop) { throw new Error(`Cannot delete ${prop}`); },
});
console.log(user.name);
user.age = 22;
try { user.age = -5; } catch (e) { console.log(e.message); }
user._secret = "x";
console.log("'_secret' in user:", "_secret" in user, "| 'name' in user:", "name" in user);
try { delete user.name; } catch (e) { console.log(e.message); }

// 2. Default values instead of undefined
const withDefault = (obj, fallback) => new Proxy(obj, { get: (t, p) => (p in t ? t[p] : fallback) });
const counts = withDefault({ apple: 2 }, 0);
console.log("counts:", counts.apple, counts.banana);

// 3. Negative indexes for arrays
const arr = new Proxy([10, 20, 30, 40], { get: (t, p, r) => Reflect.get(t, typeof p === "string" &&
/^-\d+$/.test(p) ? t.length + Number(p) : p, r) });
console.log("arr[-1]:", arr[-1], "| arr[-2]:", arr[-2], "| arr.length:", arr.length);

// 4. Intercepting function calls
const timed = new Proxy(function slowSquare(n) { return n * n; }, {
  apply(target, thisArg, args) { console.log(`  calling ${target.name}(${args})`); return
Reflect.apply(target, thisArg, args); },
});
console.log("result:", timed(12));
```

OUTPUT

```
  get name
Asha
  set age = 22
Invalid age: -5
  set _secret = "x"
'_secret' in user: false | 'name' in user: true
Cannot delete name
counts: 2 0
arr[-1]: 40 | arr[-2]: 30 | arr.length: 4
  calling slowSquare(12)
result: 144
```

Trap	Intercepts
`get(target, prop, receiver)`	reading `obj.prop`
`set(target, prop, value, receiver)`	writing; return true for success
`has(target, prop)`	the `in` operator

Trap	Intercepts
`deleteProperty(target, prop)`	`delete obj.prop`
`ownKeys(target)`	Object.keys, for…in, Reflect.ownKeys
`getOwnPropertyDescriptor`, `defineProperty`	descriptor operations
`apply(target, thisArg, args)`	calling a function proxy
`construct(target, args, newTarget)`	`new` on a function proxy
`getPrototypeOf`, `setPrototypeOf`	prototype operations

20.3.2 Reflect API

`Reflect` is a namespace object (like `Math`) with one method for each proxy trap: `Reflect.get`, `set`, `has`, `deleteProperty`, `ownKeys`, `apply`, `construct`, `defineProperty`, `getPrototypeOf` and more. Each performs the **default** behaviour of that operation, which makes it the natural way to forward an operation from inside a trap (and passing `receiver` keeps getters and inheritance working correctly). Reflect methods also offer cleaner versions of older operations: they return booleans instead of throwing (`Reflect.defineProperty`), and work as functions (`Reflect.has(obj, "x")` instead of the `in` operator).

REFLECT: THE DEFAULT BEHAVIOUR OF EVERY OPERATION

main.js

```
const course = { title: "JavaScript", weeks: 12, [Symbol("id")]: 7 };
Object.defineProperty(course, "secret", { value: "x", enumerable: false });

console.log("Reflect.has:", Reflect.has(course, "weeks"));
console.log("Reflect.ownKeys (all keys, incl. hidden and symbols):", Reflect.ownKeys(course));
console.log("Object.keys (enumerable strings only):", Object.keys(course));
console.log("Reflect.get:", Reflect.get(course, "title"));
console.log("Reflect.set returns a boolean:", Reflect.set(course, "weeks", 14), course.weeks);

const frozen = Object.freeze({ a: 1 });
console.log("Reflect.defineProperty on frozen (no throw):", Reflect.defineProperty(frozen, "b", {
value: 2 }));
console.log("Reflect.apply:", Reflect.apply(Math.max, null, [3, 9, 4]));

class Point { constructor(x, y) { this.x = x; this.y = y; } }
console.log("Reflect.construct:", Reflect.construct(Point, [1, 2]));

// why receiver matters: a getter that uses "this" through a proxy
const base = { first: "Asha", last: "Rao", get full() { return `${this.first} ${this.last}`; } };
const upper = new Proxy(base, { get: (t, p, r) => { const v = Reflect.get(t, p, r); return typeof v
=== "string" ? v.toUpperCase() : v; } });
console.log("getter via proxy with receiver:", upper.full);
```

OUTPUT

```
Reflect.has: true
Reflect.ownKeys (all keys, incl. hidden and symbols): [ 'title', 'weeks', 'secret', Symbol(id) ]
Object.keys (enumerable strings only): [ 'title', 'weeks' ]
Reflect.get: JavaScript
Reflect.set returns a boolean: true 14
Reflect.defineProperty on frozen (no throw): false
Reflect.apply: 9
Reflect.construct: Point { x: 1, y: 2 }
getter via proxy with receiver: ASHA RAO
```

20.3.3 Getters and setters for reactive data

Reactivity means that when data changes, everything that depends on it updates automatically, the core idea of Vue, Svelte, Solid and MobX. The mechanism is interception: wrap the state so that reading a property records "this effect depends on this property" (tracking) and writing one re-runs the dependent effects (triggering). Vue 2 used getters and setters defined with `Object.defineProperty`; Vue 3 uses Proxy, which also detects added and deleted properties. The example builds a tiny version of both.

TWO WAYS TO BUILD REACTIVE DATA main.js

```
// --- Version 1: getters/setters with defineProperty (Vue 2 style) ---
function observable(obj, onChange) {
  for (const key of Object.keys(obj)) {
    let value = obj[key];
    Object.defineProperty(obj, key, {
      get() { return value; },
      set(v) { const old = value; value = v; onChange(key, old, v); },
      enumerable: true,
    });
  }
  return obj;
}
const settings = observable({ theme: "light", fontSize: 14 }, (k, o, n) =>
console.log(`[defineProperty] ${k}: ${o} -> ${n}`));
settings.theme = "dark";
settings.newProp = 1;                                   // not detected: a known Vue 2 limitation

// --- Version 2: Proxy-based reactivity with automatic dependency tracking (Vue 3 style) ---
let activeEffect = null;
const deps = new Map();                                 // key -> Set of effects
function reactive(target) {
  return new Proxy(target, {
    get(t, key, r) {
      if (activeEffect) { if (!deps.has(key)) deps.set(key, new Set());
deps.get(key).add(activeEffect); }   // track
      return Reflect.get(t, key, r);
    },
    set(t, key, value, r) {
      const ok = Reflect.set(t, key, value, r);
      deps.get(key)?.forEach((fn) => fn());             // trigger
      return ok;
    },
  });
}
function effect(fn) { activeEffect = fn; fn(); activeEffect = null; }

const cart = reactive({ price: 499, qty: 2, coupon: "" });
effect(() => console.log(`[effect] total = Rs ${cart.price * cart.qty}`));
effect(() => console.log(`[effect] coupon label = "${cart.coupon || "none"}"`));
cart.qty = 3;                                           // re-runs only the total effect
cart.coupon = "LBF10";                                  // re-runs only the coupon effect
```

OUTPUT

```
[defineProperty] theme: light -> dark
[effect] total = Rs 998
[effect] coupon label = "none"
[effect] total = Rs 1497
[effect] coupon label = "LBF10"
```

20.4 Functional programming

20.4.1 Immutability and pure functions

Functional programming (FP) builds programs from functions that transform data rather than from steps that modify it. Two ideas are central. A **pure function** always returns the same output for the same input and has no **side effects**: it does not modify its arguments or anything outside itself, perform I/O, or depend on mutable external state. Pure functions are easy to test, reason about, cache (memoise) and run in parallel. **Immutability** means not changing data after creation: instead of modifying an object, create a new one with the change (spread, `map`, `filter`, `toSorted`, `with`, `structuredClone`). `Object.freeze` enforces it shallowly. React state updates and Redux reducers are built on these ideas.

PURE FUNCTIONS AND IMMUTABLE UPDATES

main.js

```
// Impure: depends on and changes outside state, mutates its argument
let taxRate = 0.18;
function addTaxImpure(cart) {
  cart.total = cart.total * (1 + taxRate);       // mutates the input
  return cart;
}
const c1 = { total: 1000 };
addTaxImpure(c1); addTaxImpure(c1);               // calling twice changes the result
console.log("impure, called twice:", c1.total);

// Pure: output depends only on inputs; returns a new object
const addTax = (cart, rate) => ({ ...cart, total: Math.round(cart.total * (1 + rate)) });
const c2 = { total: 1000 };
console.log("pure:", addTax(c2, 0.18), addTax(c2, 0.18), "| original unchanged:", c2);

// Immutable array updates (ES2023 copying methods)
const marks = Object.freeze([72, 95, 88]);
console.log("toSorted:", marks.toSorted((a, b) => a - b), "| with(1, 99):", marks.with(1, 99), "|
original:", marks);
try { "use strict"; marks.push(100); } catch (e) { console.log("frozen array push ->",
e.constructor.name); }

// Immutable nested update
const state = { user: { name: "Asha", prefs: { theme: "light" } }, cart: [] };
const next = { ...state, user: { ...state.user, prefs: { ...state.user.prefs, theme: "dark" } } };
console.log("new theme:", next.user.prefs.theme, "| old theme:", state.user.prefs.theme, "| untouched
parts shared:", next.cart === state.cart);
```

OUTPUT

```
impure, called twice: 1392.3999999999999
pure: { total: 1180 } { total: 1180 } | original unchanged: { total: 1000 }
toSorted: [ 72, 88, 95 ] | with(1, 99): [ 72, 99, 88 ] | original: [ 72, 95, 88 ]
frozen array push -> TypeError
new theme: dark | old theme: light | untouched parts shared: true
```

20.4.2 Higher-order functions, currying, composition

Functions are values, so they can be passed to and returned from other functions (**higher-order functions**, Module 6): `map`, `filter`, `reduce`, event handlers and middleware are all examples. **Currying** (Module 11) turns `f(a, b, c)` into `f(a)(b)(c)`, making it easy to create specialised functions by supplying some arguments early (**partial application**). **Composition** builds a new function from small ones: `compose(f, g)(x)` is `f(g(x))`, and `pipe` applies them left to right, which reads like a data pipeline. Point-free style writes such pipelines without mentioning the data.

BUILDING PIPELINES FROM SMALL FUNCTIONS main.js

```
const curry = (fn) => function curried(...args) {
  return args.length >= fn.length ? fn.apply(this, args) : (...more) => curried.apply(this, [...args,
...more]);
};
const pipe = (...fns) => (x) => fns.reduce((v, f) => f(v), x);
const compose = (...fns) => (x) => fns.reduceRight((v, f) => f(v), x);

// small, reusable, curried building blocks
const map = curry((fn, arr) => arr.map(fn));
const filter = curry((pred, arr) => arr.filter(pred));
const sortBy = curry((key, arr) => arr.toSorted((a, b) => b[key] - a[key]));
const take = curry((n, arr) => arr.slice(0, n));
const prop = curry((key, obj) => obj[key]);

const students = [
  { name: "Asha", marks: 91, batch: "A" }, { name: "Ravi", marks: 64, batch: "B" },
  { name: "Meera", marks: 85, batch: "A" }, { name: "John", marks: 77, batch: "A" }, { name: "Zoya",
marks: 95, batch: "B" },
];

const topTwoOfBatchA = pipe(
  filter((s) => s.batch === "A"),
  sortBy("marks"),
  take(2),
  map(prop("name")),
);
console.log("top two of batch A:", topTwoOfBatchA(students));

const shout = compose((s) => s + "!", (s) => s.toUpperCase(), (s) => s.trim());
console.log(shout("  well done  "));

const add = curry((a, b, c) => a + b + c);
console.log("curried:", add(1)(2)(3), add(1, 2)(3), add(1)(2, 3), add(1, 2, 3));
```

```
OUTPUT
top two of batch A: [ 'Asha', 'Meera' ]
WELL DONE!
curried: 6 6 6 6
```

20.4.3 Avoiding shared mutable state

Many hard bugs come from **shared mutable state**: several parts of a program holding references to the same object, one of them changes it, and the others are surprised. With asynchronous code the order of those changes is unpredictable, leading to race conditions. Defences: pass and return values instead of modifying shared ones; copy at boundaries (spread, `structuredClone`); freeze configuration; keep state in one place and change it only through clearly defined functions (the idea behind Redux and other stores); and prefer local variables to module-level ones.

SHARED REFERENCES AND AN ASYNC RACE main.js

```
// The bug: two "independent" carts share one default array
const DEFAULT_ITEMS = [];
function createCartBuggy(owner) { return { owner, items: DEFAULT_ITEMS }; }   // shared reference!
const a = createCartBuggy("Asha"), r = createCartBuggy("Ravi");
a.items.push("laptop");
console.log("Ravi's cart (buggy):", r.items);

// The fix: every cart gets its own array; updates return new objects
function createCart(owner) { return Object.freeze({ owner, items: Object.freeze([]) }); }
const addItem = (cart, item) => Object.freeze({ ...cart, items: Object.freeze([...cart.items, item])
});

let asha = createCart("Asha"), ravi = createCart("Ravi");
asha = addItem(asha, "laptop");
console.log("Ravi's cart (fixed):", ravi.items, "| Asha's:", asha.items);

// A race on shared state with async code
let balance = 1000;
const wait = (ms) => new Promise((res) => setTimeout(res, ms));
async function withdrawUnsafe(amount) {
  const current = balance;            // read
  await wait(10);                     // something async (a network call)
  balance = current - amount;         // write based on a stale read
}
Promise.all([withdrawUnsafe(300), withdrawUnsafe(200)]).then(() => console.log("balance after two
withdrawals (should be 500):", balance));
```

```
OUTPUT
Ravi's cart (buggy): [ 'laptop' ]
Ravi's cart (fixed): [] | Asha's: [ 'laptop' ]
balance after two withdrawals (should be 500): 800
```

20.5 Design patterns in JavaScript

A **design pattern** is a proven, named solution to a recurring design problem. Many classic patterns from the "Gang of Four" book look simpler in JavaScript because functions are first-class values and objects can be created directly; some are built into the language (iterators, modules, prototypes). Knowing the names helps you read framework code and communicate designs.

20.5.1 Module and revealing module patterns

Before ES modules, the **module pattern** used an **IIFE** (immediately invoked function expression) to create a private scope and return an object exposing only the public API; private variables lived in the closure. The **revealing module** variant defines all functions privately and returns an object literal that "reveals" selected ones by name. ES modules have replaced both for files, but the same idea (closure for privacy, explicit public API) is still used for objects created at run time.

MODULE AND REVEALING MODULE PATTERNS main.js

```
// Module pattern (IIFE + closure)
const counterModule = (function () {
  let count = 0;                                    // private
  function log(msg) { console.log(`  [counter] ${msg}`); }   // private helper
  return {                                          // public API
    increment() { count++; log(`now ${count}`); return count; },
    reset() { count = 0; log("reset"); },
  };
})();
counterModule.increment(); counterModule.increment(); counterModule.reset();
console.log("private count visible?", counterModule.count);

// Revealing module pattern
const cartModule = (() => {
  const items = [];
  const add = (name, price) => items.push({ name, price });
  const total = () => items.reduce((t, i) => t + i.price, 0);
  const describe = () => `${items.length} items, Rs ${total()}`;
  return { add, total, describe };                  // reveal by name
})();
cartModule.add("Pen", 10); cartModule.add("Book", 250);
console.log(cartModule.describe());
```

OUTPUT

```
  [counter] now 1
  [counter] now 2
  [counter] reset
private count visible? undefined
2 items, Rs 260
```

20.5.2 Singleton

A **singleton** ensures that only one instance of something exists and provides a global access point to it: a configuration object, a logger, a database connection pool, an application store. In JavaScript, the simplest singleton is just an object exported from an ES module, because a module is evaluated once and every importer receives the same instance. A class can enforce it with a static instance. Use singletons sparingly: global state makes testing and reasoning harder.

A LOGGER SINGLETON main.js

```
class Logger {
  static #instance;
  #lines = [];
  constructor() {
    if (Logger.#instance) return Logger.#instance;     // return the existing instance
    Logger.#instance = this;
  }
  static getInstance() { return Logger.#instance ?? new Logger(); }
  log(msg) { this.#lines.push(`${String(this.#lines.length + 1).padStart(2, "0")} ${msg}`); }
  dump() { return this.#lines.join("\n"); }
}

const a = new Logger();
const b = new Logger();
const c = Logger.getInstance();
a.log("app started"); b.log("user logged in"); c.log("order placed");
console.log("same instance:", a === b && b === c);
console.log(c.dump());

// The module-level equivalent (config.js): export const config = Object.freeze({ apiUrl: "...",
retries: 3 });
```

OUTPUT

```
same instance: true
01 app started
02 user logged in
03 order placed
```

20.5.3 Factory

A **factory** is a function (or static method) that creates objects for you, hiding which concrete class or shape is created and how. Callers ask for "a notification for this channel" and get the right object without knowing the classes. Factory functions are also the class-free way to create objects with private state through closures (Module 13).

FACTORY METHOD AND FACTORY FUNCTION

main.js

```
class EmailNotifier { constructor(to) { this.to = to; } send(m) { return `email to ${this.to}: ${m}`;
} }
class SmsNotifier { constructor(to) { this.to = to; } send(m) { return `SMS to ${this.to}:
${m.slice(0, 25)}`; } }
class PushNotifier { constructor(to) { this.to = to; } send(m) { return `push to ${this.to}: ${m}`; }
}

const notifiers = { email: EmailNotifier, sms: SmsNotifier, push: PushNotifier };
function createNotifier({ channel, to }) {                // the factory
  const Notifier = notifiers[channel];
  if (!Notifier) throw new Error(`Unsupported channel: ${channel}`);
  return new Notifier(to);
}

const users = [{ channel: "email", to: "asha@lbf.in" }, { channel: "sms", to: "+91 98450 12345" }, {
channel: "push", to: "device-7" }];
for (const u of users) console.log(createNotifier(u).send("Your batch starts on Monday at 10 am"));
try { createNotifier({ channel: "fax", to: "x" }); } catch (e) { console.log(e.message); }

// A factory function with private state
function createIdGenerator(prefix) {
  let n = 0;
  return { next: () => `${prefix}-${String(++n).padStart(3, "0")}` };
}
const orderIds = createIdGenerator("ORD");
console.log(orderIds.next(), orderIds.next());
```

OUTPUT

```
email to asha@lbf.in: Your batch starts on Monday at 10 am
SMS to +91 98450 12345: Your batch starts on Mond
push to device-7: Your batch starts on Monday at 10 am
Unsupported channel: fax
ORD-001 ORD-002
```

20.5.4 Observer and pub/sub

In the **observer** pattern, a **subject** keeps a list of **observers** (subscribers) and notifies them all when something changes; observers can subscribe and unsubscribe at any time. DOM events, Node.js `EventEmitter`, RxJS and every state-management library are built on it. **Publish/subscribe** (pub/sub) adds a middleman, an **event bus** or broker, with named **topics**: publishers and subscribers know only the bus and the topic name, never each other, which decouples parts of an application completely.

OBSERVER, PUB/SUB AND EVENTEMITTER

main.js

```
// Observer: a subject with subscribers
class Observable {
  #observers = new Set();
  subscribe(fn) { this.#observers.add(fn); return () => this.#observers.delete(fn); }   // returns
unsubscribe
  notify(data) { this.#observers.forEach((fn) => fn(data)); }
}
const stockPrice = new Observable();
const unsubChart = stockPrice.subscribe((p) => console.log(`  chart redraws at ${p}`));
stockPrice.subscribe((p) => { if (p > 2500) console.log(`  alert: price above 2500 (${p})`); });
stockPrice.notify(2480);
stockPrice.notify(2510);
unsubChart();
stockPrice.notify(2530);

// Pub/sub: an event bus with topics
const bus = {
  topics: new Map(),
  subscribe(topic, fn) { (this.topics.get(topic) ?? this.topics.set(topic, []).get(topic)).push(fn);
},
  publish(topic, payload) { (this.topics.get(topic) ?? []).forEach((fn) => fn(payload)); },
};
bus.subscribe("order:placed", (o) => console.log(`[email]     confirmation for order ${o.id}`));
bus.subscribe("order:placed", (o) => console.log(`[inventory] reserve ${o.items.length} items`));
bus.subscribe("order:shipped", (o) => console.log(`[sms]       order ${o.id} is on the way`));
bus.publish("order:placed", { id: 501, items: ["pen", "book"] });
bus.publish("order:shipped", { id: 501 });

// Node.js has the pattern built in
const { EventEmitter } = require("node:events");
const uploader = new EventEmitter();
uploader.on("progress", (pct) => console.log(`EventEmitter progress ${pct}%`));
uploader.once("done", () => console.log("EventEmitter done (once)"));
[50, 100].forEach((p) => uploader.emit("progress", p));
uploader.emit("done"); uploader.emit("done");
```

OUTPUT

```
  chart redraws at 2480
  chart redraws at 2510
  alert: price above 2500 (2510)
  alert: price above 2500 (2530)
[email]     confirmation for order 501
[inventory] reserve 2 items
[sms]       order 501 is on the way
EventEmitter progress 50%
EventEmitter progress 100%
EventEmitter done (once)
```



PUB/SUB: PUBLISHERS AND SUBSCRIBERS KNOW ONLY THE BUS

20.5.5 Decorator

A **decorator** adds behaviour to an object or function **without changing its code**, by wrapping it. In JavaScript the most common form is a higher-order function that takes a function and returns an enhanced one: logging, timing, caching (memoisation), retrying, access checks, debouncing. Wrappers can be stacked. A TC39 proposal (stage 3) adds `@decorator` syntax for classes and their members, already used through TypeScript and Babel in frameworks like Angular and NestJS.

FUNCTION DECORATORS: LOGGING, MEMOISATION, RETRY main.js

```
const withLogging = (fn) => function (...args) {
  const result = fn.apply(this, args);
  console.log(`  ${fn.name}(${args.join(", ")}) -> ${result}`);
  return result;
};
const withMemo = (fn) => {
  const cache = new Map();
  return function (...args) {
    const key = JSON.stringify(args);
    if (!cache.has(key)) cache.set(key, fn.apply(this, args));
    else console.log(`  (cache hit for ${key})`);
    return cache.get(key);
  };
};
const withRetry = (fn, attempts = 3) => async (...args) => {
  for (let i = 1; ; i++) {
    try { return await fn(...args); }
    catch (e) { console.log(`  attempt ${i} failed: ${e.message}`); if (i === attempts) throw e; }
  }
};

function shippingCost(km, kg) { return 40 + km * 2 + kg * 5; }
const cost = withMemo(withLogging(shippingCost));        // stacked decorators
cost(10, 2); cost(10, 2); cost(25, 1);

let calls = 0;
const flakyFetch = async (id) => { if (++calls < 3) throw new Error("network timeout"); return `data
for ${id}`; };
withRetry(flakyFetch)(42).then((d) => console.log("  retry succeeded:", d));
```

```
OUTPUT
  shippingCost(10, 2) -> 70
  (cache hit for [10,2])
  shippingCost(25, 1) -> 95
  attempt 1 failed: network timeout
  attempt 2 failed: network timeout
  retry succeeded: data for 42
```

20.5.6 Strategy

The **strategy** pattern defines a family of interchangeable algorithms behind a common interface, and lets the caller choose one at run time instead of hard-coding a chain of `if`/`switch` statements. In JavaScript, strategies are usually just functions stored in an object or passed as arguments: sorting comparators, validation rules, pricing and discount rules, payment methods. Adding a new strategy means adding a function, not editing existing code (the open/closed principle).

INTERCHANGEABLE DISCOUNT STRATEGIES

main.js

```
// Discount strategies: each is a function (cart) => discount amount
const discountStrategies = {
  none: () => 0,
  festival: (cart) => cart.total * 0.10,
  student: (cart) => Math.min(cart.total * 0.15, 1500),
  bulk: (cart) => (cart.items >= 5 ? cart.total * 0.20 : 0),
};

function checkout(cart, strategyName) {
  const strategy = discountStrategies[strategyName] ?? discountStrategies.none;
  const discount = Math.round(strategy(cart));
  return `${strategyName.padEnd(8)} discount Rs ${String(discount).padStart(5)}  pay Rs ${cart.total -
discount}`;
}

const cart = { total: 12000, items: 6 };
for (const s of ["none", "festival", "student", "bulk"]) console.log(checkout(cart, s));

discountStrategies.firstOrder = (c) => 500;              // new behaviour without editing checkout()
console.log(checkout(cart, "firstOrder"));

// strategies as arguments: sorting
const courses = [{ t: "React", fee: 11999, rating: 4.8 }, { t: "Java", fee: 14999, rating: 4.6 }, { t:
"Python", fee: 9999, rating: 4.9 }];
const sorters = { cheapest: (a, b) => a.fee - b.fee, bestRated: (a, b) => b.rating - a.rating };
console.log("cheapest:", courses.toSorted(sorters.cheapest).map((c) => c.t).join(", "), "| best
rated:", courses.toSorted(sorters.bestRated).map((c) => c.t).join(", "));
```

OUTPUT

```
none     discount Rs     0  pay Rs 12000
festival discount Rs  1200  pay Rs 10800
student  discount Rs  1500  pay Rs 10500
bulk     discount Rs  2400  pay Rs 9600
firstOrder discount Rs   500  pay Rs 11500
cheapest: Python, React, Java | best rated: Python, React, Java
```

20.6 Performance

20.6.1 Measuring with performance.now() and the Performance panel

Never guess about performance: **measure**. `performance.now()` returns a high-resolution timestamp in milliseconds (with fractions), relative to the page or process start and unaffected by changes to the system clock, which makes it the right tool for timing code (`Date.now()` is for calendar time). `performance.mark()` and `performance.measure()` create named timings that also appear in the DevTools **Performance** panel. Measure several runs and look at the median or minimum, because timings vary. For whole pages, record a profile in the Performance panel: it shows a flame chart of JavaScript calls, layout, paint, long tasks (over 50 ms, which block input) and screenshots over time.

TIMING CODE PROPERLY

main.js

```
const { performance } = require("node:perf_hooks");

function timeIt(label, fn, runs = 7) {
  const times = [];
  for (let i = 0; i < runs; i++) { const t = performance.now(); fn(); times.push(performance.now() -
t); }
  times.sort((a, b) => a - b);
  return { label, median: times[Math.floor(runs / 2)] };
}

const data = Array.from({ length: 200_000 }, (_, i) => ({ id: i, active: i % 3 === 0 }));
const ids = Array.from({ length: 2000 }, (_, i) => i * 50);

const viaFind = timeIt("array.find in a loop (O(n·m))", () => ids.map((id) => data.find((d) => d.id
=== id)));
const viaMap = timeIt("Map lookup (O(n + m))", () => { const byId = new Map(data.map((d) => [d.id,
d])); return ids.map((id) => byId.get(id)); });
console.log(`Map lookup faster than repeated find: ${viaMap.median < viaFind.median}`);

performance.mark("parse-start");
JSON.parse(JSON.stringify(data));
performance.mark("parse-end");
const m = performance.measure("JSON round trip", "parse-start", "parse-end");
console.log(`performance.measure: "${m.name}" took ${m.duration > 0 ? "a measurable time" : 0} (entry
type: ${m.entryType})`);
```

OUTPUT

```
Map lookup faster than repeated find: true
performance.measure: "JSON round trip" took a measurable time (entry type: measure)
```

NOTE

The exact speed-up depends on the machine; the point is the algorithmic difference. Optimising algorithms and data structures (Module 25) usually beats micro-optimising syntax.

20.6.2 Lazy loading and code splitting

The fastest code is code that is not downloaded until needed. **Code splitting** uses dynamic `import()` (Module 19) so that bundlers create separate chunks for routes, dialogs and heavy features, loaded on demand or prefetched when the browser is idle. Images and iframes can be lazy-loaded with the `loading="lazy"` attribute, or with IntersectionObserver (Module 21) for custom behaviour. Scripts that are not needed for the first render should use `defer` / `type="module"` or be loaded after interaction. Frameworks provide helpers such as `React.lazy` and route-based splitting.

LAZY-LOADING IMAGES, IFRAMES AND CODE

HTML

```
<!-- images below the fold load only when the user scrolls near them -->
<img src="hero.webp" alt="Students in class" width="1200" height="600" fetchpriority="high">
<img src="gallery-1.webp" alt="Lab" width="400" height="300" loading="lazy">
<iframe src="https://www.youtube.com/embed/..." loading="lazy" title="Course intro"></iframe>

<script type="module">
  // load the chart library only when the user opens the reports tab
  document.querySelector("#reports-tab").addEventListener("click", async () => {
    const { renderCharts } = await import("./reports.js");
    renderCharts();
  }, { once: true });
</script>
```

20.6.3 Avoiding layout thrashing

As Module 15 showed, reading layout properties (`offsetWidth`, `getBoundingClientRect`) after writing styles forces the browser to recalculate layout synchronously; doing this in a loop causes **layout thrashing**. Batch all reads first, then all writes; schedule visual writes in `requestAnimationFrame`; animate `transform` and `opacity` instead of `top`/`width`; and use CSS `contain` and `content-visibility: auto` to limit how much of the page each change affects. The Performance panel marks forced reflows with a red warning triangle.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) — HTML

```
<div class="card">Short</div>
<div class="card">A much longer card text that wraps onto several lines in a narrow column</div>
<style>.card { width: 160px; float: left; margin: 4px; border: 1px solid #ccc; }</style>
```

BATCHING READS AND WRITES IN A FRAME — main.js

```
const cards = [...document.querySelectorAll(".card")];

// Bad: read and write interleaved, forcing layout each iteration
function equaliseBad() { cards.forEach((c) => { c.style.height = "auto"; c.style.height =
c.offsetHeight + 20 + "px"; }); }

// Good: one write phase, one read phase, one write phase, inside a frame
function equaliseGood() {
  requestAnimationFrame(() => {
    cards.forEach((c) => (c.style.height = "auto"));              // writes
    const max = Math.max(...cards.map((c) => c.offsetHeight));    // reads (one layout)
    cards.forEach((c) => (c.style.height = max + "px"));          // writes
    console.log("all cards set to the tallest height:", new Set(cards.map((c) => c.style.height)).size
=== 1);
  });
}
equaliseBad();
equaliseGood();
```

BROWSER CONSOLE

```
all cards set to the tallest height: true
```

20.6.4 Web Vitals (overview)

Core Web Vitals are Google's user-centred metrics for page experience, measured from real users and used in search ranking. **LCP** (Largest Contentful Paint) measures loading: when the largest image or text block becomes visible; good is 2.5 s or less. **INP** (Interaction to Next Paint) measures responsiveness: the delay between user interactions and the next frame, across the visit; good is 200 ms or less. **CLS** (Cumulative Layout Shift) measures visual stability: how much content jumps around unexpectedly; good is 0.1 or less. Measure them with Lighthouse and the Performance panel in DevTools, PageSpeed Insights, or in your own code with the `web-vitals` library or `PerformanceObserver`.

Metric	Measures	Good	Common fixes
LCP	loading of the main content	<= 2.5 s	optimise and preload the hero image, fast server/CDN, less render-blocking CSS/JS
INP	responsiveness to input	<= 200 ms	break up long tasks, less JS on the main thread, web workers, debounce

Metric	Measures	Good	Common fixes
CLS	unexpected layout shifts	<= 0.1	width/height on images and ads, reserve space, avoid inserting content above
(TTFB, FCP)	server response, first paint	<= 0.8 s / 1.8 s	caching, CDN, smaller HTML

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<h1 id="title">Learn Being Forward</h1>
<p id="news">Admissions open for the March batch. Book a free demo class today.</p>
```

OBSERVING LCP AND LAYOUT SHIFTS main.js

```
new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    console.log(`LCP candidate: <${entry.element?.tagName.toLowerCase()}> rendered (${entry.renderTime
> 0 || entry.startTime > 0 ? "time recorded" : "-"})`);
  }
}).observe({ type: "largest-contentful-paint", buffered: true });

new PerformanceObserver((list) => {
  const shift = list.getEntries().reduce((t, e) => t + (e.hadRecentInput ? 0 : e.value), 0);
  console.log("layout shift detected, score:", shift.toFixed(3), shift <= 0.1 ? "(good)" : "(poor)");
}).observe({ type: "layout-shift", buffered: true });

requestAnimationFrame(() => requestAnimationFrame(() => {     // after the first paint, a late banner
pushes everything down
  document.body.insertAdjacentHTML("afterbegin", '<div style="height:120px;background:#eee">Late
banner</div>');
}));
```

BROWSER CONSOLE

```
layout shift detected, score: 0.057 (good)
LCP candidate: <h1> rendered (time recorded)
```

20.7 Writing polyfills

20.7.1 Implement map, filter, reduce, bind and Promise.all yourself

A **polyfill** is code that implements a standard feature for environments that lack it, so the rest of the code can use the feature normally. A correct polyfill first checks whether the feature already exists, follows the specification closely (argument handling, holes in arrays, `this`), and defines methods as non-enumerable. Production projects use maintained polyfill libraries such as core-js; writing your own is one of the most common interview exercises, because it proves you understand how the built-ins work. The versions below use names like `myMap` so they do not replace the real methods.

POLYFILLS FOR MAP, FILTER AND REDUCE

main.js

```
// --- Array.prototype.map / filter / reduce ---
function define(obj, name, fn) {
  if (!Object.hasOwn(obj, name)) Object.defineProperty(obj, name, { value: fn, writable: true,
configurable: true, enumerable: false });
}

define(Array.prototype, "myMap", function (callback, thisArg) {
  if (typeof callback !== "function") throw new TypeError(callback + " is not a function");
  const result = new Array(this.length);
  for (let i = 0; i < this.length; i++) {
    if (i in this) result[i] = callback.call(thisArg, this[i], i, this);   // skip holes like the real
map
  }
  return result;
});

define(Array.prototype, "myFilter", function (callback, thisArg) {
  const result = [];
  for (let i = 0; i < this.length; i++) {
    if (i in this && callback.call(thisArg, this[i], i, this)) result.push(this[i]);
  }
  return result;
});

define(Array.prototype, "myReduce", function (callback, ...init) {
  let i = 0, acc;
  if (init.length) acc = init[0];
  else {
    while (i < this.length && !(i in this)) i++;
    if (i >= this.length) throw new TypeError("Reduce of empty array with no initial value");
    acc = this[i++];
  }
  for (; i < this.length; i++) if (i in this) acc = callback(acc, this[i], i, this);
  return acc;
});

const nums = [1, 2, , 4];                                    // note the hole at index 2
console.log("myMap:    ", nums.myMap((n) => n * 10), "| real:", nums.map((n) => n * 10));
console.log("myFilter:", nums.myFilter((n) => n > 1), "| real:", nums.filter((n) => n > 1));
console.log("myReduce:", nums.myReduce((a, b) => a + b), nums.myReduce((a, b) => a + b, 100), "|
real:", nums.reduce((a, b) => a + b));
try { [].myReduce((a, b) => a + b); } catch (e) { console.log("empty reduce:", e.message); }
console.log("myMap appears in for...in / Object.keys?",
Object.keys(Array.prototype).includes("myMap"));
```

OUTPUT

```
myMap:    [ 10, 20, <1 empty item>, 40 ] | real: [ 10, 20, <1 empty item>, 40 ]
myFilter: [ 2, 4 ] | real: [ 2, 4 ]
myReduce: 7 107 | real: 7
empty reduce: Reduce of empty array with no initial value
myMap appears in for...in / Object.keys? false
```

POLYFILLS FOR BIND AND PROMISE.ALL

main.js

```
// --- Function.prototype.bind ---
Function.prototype.myBind = function (thisArg, ...preset) {
  const original = this;
  if (typeof original !== "function") throw new TypeError("myBind must be called on a function");
  function bound(...args) {
    // when called with new, ignore thisArg and construct normally
    return new.target ? new original(...preset, ...args) : original.apply(thisArg, [...preset,
```

```
...args]);
  }
  if (original.prototype) bound.prototype = Object.create(original.prototype);
  return bound;
};

const asha = { name: "Asha" };
function greet(greeting, punctuation) { return `${greeting}, ${this.name}${punctuation}`; }
const hello = greet.myBind(asha, "Hello");
console.log(hello("!"), "|", greet.bind(asha, "Hello")("!"));

const button = { label: "Pay", click() { return `clicked ${this.label}`; } };
const handler = button.click.myBind(button);
console.log(handler(), "(this survives being detached)");

function Point(x, y) { this.x = x; this.y = y; }
const PointAtOrigin = Point.myBind(null, 0);
const p = new PointAtOrigin(5);
console.log("with new:", p, p instanceof Point);

// --- Promise.all ---
function promiseAll(iterable) {
  return new Promise((resolve, reject) => {
    const items = Array.from(iterable);
    const results = new Array(items.length);
    let remaining = items.length;
    if (remaining === 0) return resolve(results);
    items.forEach((item, i) => {
      Promise.resolve(item).then((value) => {          // accepts plain values too
        results[i] = value;                             // keep the original order
        if (--remaining === 0) resolve(results);
      }, reject);                                       // first rejection wins
    });
  });
}

const later = (v, ms, fail) => new Promise((res, rej) => setTimeout(() => (fail ? rej(new Error(v)) :
res(v)), ms));
promiseAll([later("slow", 30), later("fast", 10), 42]).then((r) => console.log("promiseAll:", r));
promiseAll([later("ok", 20), later("boom", 5, true)]).catch((e) => console.log("promiseAll rejected:",
e.message));
promiseAll([]).then((r) => console.log("empty input:", r));
```

OUTPUT

```
Hello, Asha! | Hello, Asha!
clicked Pay (this survives being detached)
with new: Point { x: 0, y: 5 } true
empty input: []
promiseAll rejected: boom
promiseAll: [ 'slow', 'fast', 42 ]
```

Engines tokenize and parse source into an AST, compile it to bytecode (V8 Ignition) and JIT-compile hot code to optimised machine code (Sparkplug, Maglev, TurboFan), deoptimising when speculative type assumptions fail; hidden classes (shapes) and inline caches make property access fast when objects keep consistent shapes. Primitives and references live in stack frames, objects on the heap; the garbage collector frees unreachable objects with generational mark-and-sweep. Leaks come from growing globals, forgotten timers and listeners, detached DOM nodes and closures; heap snapshots in DevTools find them; WeakRef and FinalizationRegistry are rarely needed. Proxy traps intercept operations, Reflect performs their defaults, and both power reactivity. Functional programming favours pure functions, immutability, higher-order functions, currying and composition, and avoids shared mutable state. Patterns: module/revealing module, singleton, factory, observer and pub/sub, decorator, strategy. Measure performance with performance.now, marks and the Performance panel; lazy-load and code-split; batch DOM reads and writes; track Core Web Vitals (LCP, INP, CLS). Polyfills implement standard features, as in map, filter, reduce, bind and Promise.all.

PRACTICE CHECKPOINT

Write polyfills for **Array.prototype.map**, **Function.prototype.bind** and **Promise.all** (see 20.7.1), and build a simple **reactive store** using **Proxy** and the **Observer** pattern.

SAMPLE SOLUTION

main.js

```
function createStore(initialState) {
  const listeners = new Map();                // key -> Set of callbacks ("*" for any change)
  const history = [];

  const notify = (key, value, old) => {
    for (const fn of [...(listeners.get(key) ?? []), ...(listeners.get("*") ?? [])]) fn(value, old,
key);
  };

  const state = new Proxy(structuredClone(initialState), {
    set(target, key, value) {
      const old = target[key];
      if (Object.is(old, value)) return true;  // no change, no notification
      if (key === "qty" && (!Number.isInteger(value) || value < 0)) throw new RangeError(`qty must be
a non-negative integer, got ${value}`);
      target[key] = value;
      history.push(`${String(key)}: ${JSON.stringify(old)} -> ${JSON.stringify(value)}`);
      notify(key, value, old);
      return true;
    },
    deleteProperty() { throw new Error("Properties cannot be deleted from the store"); },
  });

  return {
    state,
    subscribe(key, fn) {                      // Observer pattern: returns an unsubscribe function
      if (!listeners.has(key)) listeners.set(key, new Set());
      listeners.get(key).add(fn);
      return () => listeners.get(key).delete(fn);
    },
    get history() { return [...history]; },
  };
}

const store = createStore({ product: "Headphones", price: 2499, qty: 1, coupon: null });
const { state } = store;
```

```
const total = () => state.price * state.qty * (state.coupon === "LBF10" ? 0.9 : 1);
const render = () => console.log(`  UI: ${state.qty} x ${state.product} = Rs ${total().toFixed(0)}`);

store.subscribe("qty", render);
store.subscribe("coupon", (c) => { console.log(`  coupon applied: ${c}`); render(); });
const stopLogging = store.subscribe("*", (v, old, key) => console.log(`[log] ${key} changed`));

state.qty = 2;
state.qty = 2;                                  // same value: ignored
state.coupon = "LBF10";
stopLogging();
state.qty = 3;
try { state.qty = -1; } catch (e) { console.log("validation:", e.message); }
try { delete state.price; } catch (e) { console.log("protection:", e.message); }
console.log("history:", store.history);
```

OUTPUT

```
  UI: 2 x Headphones = Rs 4998
[log] qty changed
  coupon applied: LBF10
  UI: 2 x Headphones = Rs 4498
[log] coupon changed
  UI: 3 x Headphones = Rs 6747
validation: qty must be a non-negative integer, got -1
protection: Properties cannot be deleted from the store
history: [ 'qty: 1 -> 2', 'coupon: null -> "LBF10"', 'qty: 2 -> 3' ]
```

MODULE 21 · ADVANCED

Advanced browser APIs

Minimum duration **3 days** | Level **Advanced**

Modern browsers are full application platforms. This module covers the observer APIs (IntersectionObserver for infinite scroll and lazy images, MutationObserver, ResizeObserver), graphics and media (the Canvas API, requestAnimationFrame animation, audio and video), background work with Web Workers and Service Workers, Progressive Web Apps, device and system APIs (geolocation, notifications, clipboard, files, sharing), real-time communication with WebSockets and Server-Sent Events, and Web Components with custom elements, Shadow DOM, templates and slots.

IN THIS MODULE

21.1 Observers
21.2 Graphics and media
21.3 Background work
21.4 Progressive Web Apps (PWA)
21.5 Device and system APIs
21.6 Real-time communication
21.7 Web Components

NOTE

Examples run in headless Chromium. APIs that require a secure context (HTTPS) ran on `https://lbf.local` with permissions such as location already granted, and the simulated location is Bengaluru. Images of canvases and components are real screenshots taken from the test browser.

21.1 Observers

Polling ("check every 100 ms whether the element is visible / has changed / has resized") wastes battery and causes layout thrashing. The **observer APIs** invert this: you describe what to watch, and the browser calls your callback efficiently, in batches, when it happens. All three share the same shape: `new XObserver(callback, options)`, then `observe(target)`, `unobserve(target)` and `disconnect()`; the callback receives an array of **entries**.

21.1.1 IntersectionObserver (infinite scroll, lazy images)

An **IntersectionObserver** reports when target elements enter or leave the viewport (or another scrolling `root` element). Options: `rootMargin` grows or shrinks the root box (`"200px"` fires 200 px **before** the element becomes visible, ideal for preloading), and `threshold` sets which visible fractions trigger the callback (0 = any pixel, 1 = fully visible, or an array). Each entry has `isIntersecting`, `intersectionRatio`, `target` and bounding rectangles. Uses: lazy-loading images, **infinite scroll** (watch a "sentinel" element at the end of a list), analytics for ads or sections seen, and starting animations when they scroll into view.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY)

HTML

```
<style>.post { height: 150px; margin: 8px; background: #f3ead2; }</style>
<div id="feed"></div>
<div id="sentinel">Loading more...</div>
```

INFINITE SCROLL WITH A SENTINEL ELEMENT

main.js

```
const feed = document.querySelector("#feed");
const sentinel = document.querySelector("#sentinel");
let page = 0, loading = false;

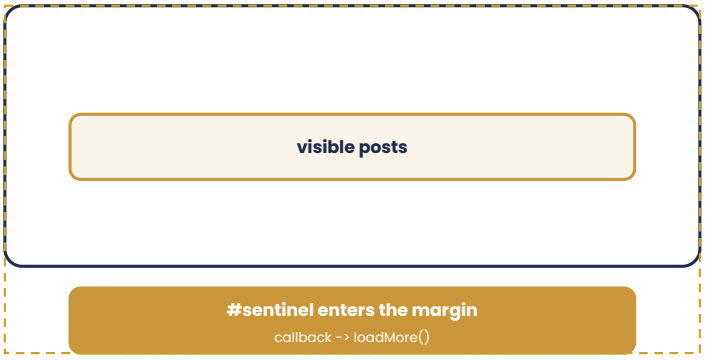
async function loadMore() {
  if (loading || page >= 3) return;
  loading = true;
  page++;
  await new Promise((r) => setTimeout(r, 50));            // pretend to fetch the next page
  for (let i = 1; i <= 5; i++) {
    const card = document.createElement("div");
    card.className = "post";
    card.textContent = `Post ${(page - 1) * 5 + i}`;
    feed.append(card);
  }
  console.log(`loaded page ${page}: ${feed.children.length} posts in the feed`);
  if (page === 3) { observer.disconnect(); console.log("no more pages: observer disconnected"); }
  loading = false;
}

const observer = new IntersectionObserver((entries) => {
  if (entries[0].isIntersecting) loadMore();              // sentinel reached: load the next page
}, { rootMargin: "100px" });
observer.observe(sentinel);
```

BROWSER CONSOLE

```
loaded page 1: 5 posts in the feed
loaded page 2: 10 posts in the feed
loaded page 3: 15 posts in the feed
no more pages: observer disconnected
```

Simulated user actions, in order: waits 300 ms; scrolls down 800 pixels with the mouse wheel; waits 300 ms; scrolls down 800 pixels with the mouse wheel; waits 300 ms.



THE SENTINEL TRIGGERS LOADING BEFORE THE USER REACHES THE END

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY)

HTML

```
<style>img { display: block; width: 300px; height: 220px; margin-bottom: 400px; background: #ddd; }
</style>
<img data-src="photo-1.webp" alt="Campus">
<img data-src="photo-2.webp" alt="Lab">
<img data-src="photo-3.webp" alt="Library">
```

LAZY-LOADING IMAGES AS THEY APPROACH THE VIEWPORT main.js

```
const io = new IntersectionObserver((entries, obs) => {
  for (const entry of entries) {
    if (!entry.isIntersecting) continue;
    const img = entry.target;
    img.src = img.dataset.src;                              // start the real download now
    img.classList.add("loaded");
    obs.unobserve(img);                                     // each image only once
    console.log(`lazy-loaded ${img.dataset.src} (ratio ${entry.intersectionRatio > 0 ? "> 0" : 0})`);
  }
}, { rootMargin: "50px 0px" });

document.querySelectorAll("img[data-src]").forEach((img) => io.observe(img));
setTimeout(() => console.log("images still waiting:",
[...document.querySelectorAll("img:not(.loaded)")].map((i) => i.dataset.src).join(", ")), 200);
```

BROWSER CONSOLE

```
lazy-loaded photo-1.webp (ratio > 0)
lazy-loaded photo-2.webp (ratio > 0)
images still waiting: photo-3.webp
lazy-loaded photo-3.webp (ratio > 0)
```

Simulated user actions, in order: waits 100 ms; scrolls down 700 pixels with the mouse wheel; waits 200 ms.

NOTE

For simple cases the native attribute `<img loading="lazy">` does the same with no JavaScript. IntersectionObserver is for custom behaviour: placeholders, fade-in effects, or deciding when to load other content.

21.1.2 MutationObserver

A **MutationObserver** reports changes to the DOM: children added or removed (`childList`), attribute changes (`attributes`, optionally filtered and with `attributeOldValue`), text changes (`characterData`), for one node or its whole tree (`subtree`). Callbacks run as **microtasks** with a batch of `MutationRecord`s, so many quick changes produce one call. Uses: reacting to content inserted by third-party scripts or frameworks, building browser extensions, auto-initialising widgets, undo history, and detecting changes in a rich-text editor.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<p>Messages: <span id="badge">0</span></p>
<div id="chat" data-status="away"></div>
```

WATCHING A CHAT BOX CHANGE main.js

```
const chat = document.querySelector("#chat");
const badge = document.querySelector("#badge");

const observer = new MutationObserver((records) => {
  for (const r of records) {
    if (r.type === "childList" && r.addedNodes.length) console.log(`childList: +${r.addedNodes.length}
node(s), -${r.removedNodes.length}`);
    if (r.type === "childList" && r.removedNodes.length && !r.addedNodes.length)
console.log(`childList: removed "${r.removedNodes[0].textContent}"`);
    if (r.type === "attributes") console.log(`attribute "${r.attributeName}": "${r.oldValue}" ->
"${r.target.getAttribute(r.attributeName)}"`);
    if (r.type === "characterData") console.log(`text changed: "${r.oldValue}" ->
"${r.target.data}"`);
  }
  badge.textContent = chat.children.length;                  // keep a counter in sync
});
observer.observe(chat, { childList: true, subtree: true, attributes: true, attributeOldValue: true,
characterData: true, characterDataOldValue: true });

// changes made by "someone else" (another script, a framework, the user)
chat.insertAdjacentHTML("beforeend", "<p>Asha: Hi!</p><p>Ravi: Hello</p>");   // one record, 2 nodes
chat.dataset.status = "online";
setTimeout(() => {
  chat.firstElementChild.firstChild.data = "Asha: Hi everyone!";
  chat.lastElementChild.remove();
  setTimeout(() => { console.log("messages now:", badge.textContent); observer.disconnect(); }, 0);
}, 0);
```

BROWSER CONSOLE

```
childList: +2 node(s), -0
attribute "data-status": "away" -> "online"
text changed: "Asha: Hi!" -> "Asha: Hi everyone!"
childList: removed "Ravi: Hello"
messages now: 1
```

21.1.3 ResizeObserver

A **ResizeObserver** reports when an **element's** size changes, for any reason: the window resized, a sidebar collapsed, content grew, CSS changed. That is more precise than the window `resize` event, which knows nothing about individual elements. Entries provide `contentRect` and `contentBoxSize` / `borderBoxSize`. Uses: responsive components that adapt to their container (when CSS container queries are not enough), redrawing charts and canvases at the right size, and truncating text.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<div id="card" style="width:180px;padding:10px;border:1px solid #ccc">Course card</div>
```

A COMPONENT THAT ADAPTS TO ITS OWN WIDTH

main.js

```
const card = document.querySelector("#card");
const ro = new ResizeObserver((entries) => {
  for (const entry of entries) {
    const width = Math.round(entry.contentRect.width);
    const layout = width < 300 ? "compact" : width < 500 ? "regular" : "wide";
    entry.target.dataset.layout = layout;                    // CSS can style [data-layout=...]
    console.log(`card is ${width}px wide -> ${layout} layout`);
  }
});
ro.observe(card);

const widths = ["250px", "420px", "640px"];
widths.forEach((w, i) => setTimeout(() => { card.style.width = w; }, 80 * (i + 1)));   // e.g. a sidebar opening and closing
```

BROWSER CONSOLE

```
card is 180px wide -> compact layout
card is 250px wide -> compact layout
card is 420px wide -> regular layout
card is 640px wide -> wide layout
```

21.2 Graphics and media

21.2.1 Canvas API: drawing shapes, text and images

The `<canvas>` element is a bitmap drawing surface. Get a **2D rendering context** with `canvas.getContext("2d")` and draw with commands: rectangles (`fillRect`, `strokeRect`, `clearRect`), **paths** (`beginPath`, `moveTo`, `lineTo`, `arc`, `quadraticCurveTo`, then `fill` or `stroke`), text (`fillText` with `font`, `textAlign`), images (`drawImage`), with styles (`fillStyle`, `strokeStyle`, `lineWidth`, gradients, shadows) and transformations (`translate`, `rotate`, `scale`, `save`/`restore`). Canvas is **immediate mode**: it remembers pixels, not objects, so to move something you clear and redraw. It suits charts, games, image editing and visualisations; SVG is better for scalable, accessible, interactive diagrams with few elements, and WebGL/WebGPU for 3D.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY)

HTML

```
<canvas id="chart" width="500" height="250"></canvas>
```

DRAWING A BAR CHART ON A CANVAS

main.js

```
const canvas = document.querySelector("#chart");
const ctx = canvas.getContext("2d");
const data = [["Jan", 42], ["Feb", 58], ["Mar", 75], ["Apr", 64], ["May", 90]];

// background with a gradient
const bg = ctx.createLinearGradient(0, 0, 0, canvas.height);
bg.addColorStop(0, "#FBF8F1"); bg.addColorStop(1, "#F1E7CF");
ctx.fillStyle = bg;
ctx.fillRect(0, 0, canvas.width, canvas.height);

// title text
ctx.fillStyle = "#1B2A4A";
ctx.font = "bold 16px sans-serif";
ctx.fillText("Monthly enrolments", 20, 28);

// axes as a path
ctx.strokeStyle = "#1B2A4A"; ctx.lineWidth = 2;
ctx.beginPath(); ctx.moveTo(40, 50); ctx.lineTo(40, 220); ctx.lineTo(380, 220); ctx.stroke();

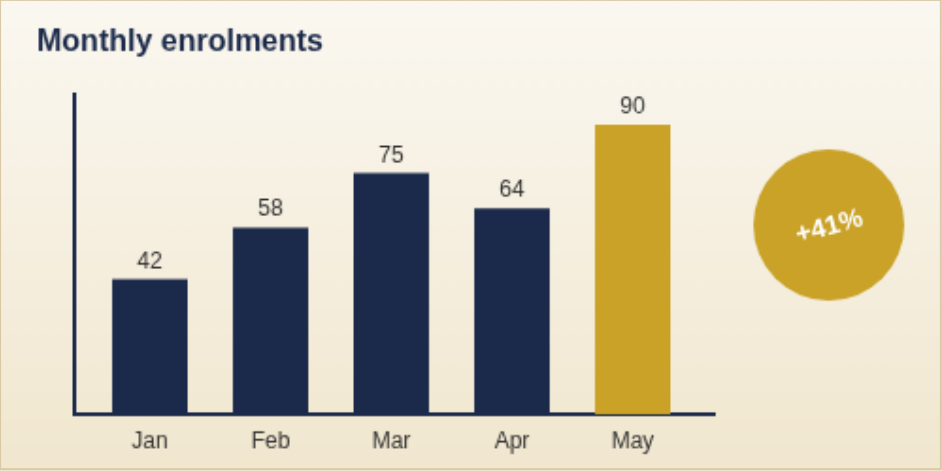
// bars and labels
data.forEach(([month, value], i) => {
  const x = 60 + i * 64, h = value * 1.7;
  ctx.fillStyle = i === 4 ? "#C9A227" : "#1B2A4A";
  ctx.fillRect(x, 220 - h, 40, h);
  ctx.fillStyle = "#333"; ctx.font = "12px sans-serif"; ctx.textAlign = "center";
  ctx.fillText(month, x + 20, 238);
  ctx.fillText(value, x + 20, 214 - h);
});

// a circle and a rotated badge using transforms
ctx.beginPath(); ctx.arc(440, 120, 40, 0, Math.PI * 2);
ctx.fillStyle = "#C9A227"; ctx.fill();
ctx.save();
ctx.translate(440, 120); ctx.rotate(-Math.PI / 12);
ctx.fillStyle = "#fff"; ctx.font = "bold 14px sans-serif"; ctx.fillText("+41%", 0, 5);
ctx.restore();

const px = ctx.getImageData(445, 100, 1, 1).data;                 // read a pixel back
console.log("canvas size:", canvas.width + "x" + canvas.height, "| pixel inside the circle rgb:",
px.slice(0, 3).join(","));
console.log("export as PNG data URL starts with:", canvas.toDataURL("image/png").slice(0, 22));
```

BROWSER CONSOLE

```
canvas size: 500x250 | pixel inside the circle rgb: 201,162,39
export as PNG data URL starts with: 
```



THE CANVAS PRODUCED BY THE CODE ABOVE (SCREENSHOT)

NOTE

A canvas has two sizes: the drawing-buffer size from the `width` / `height` attributes, and the displayed size from CSS. For sharp drawings on high-density screens, multiply the buffer size by `devicePixelRatio` and scale the context.

21.2.2 Animations with requestAnimationFrame

`requestAnimationFrame(callback)` asks the browser to call your function just **before the next repaint**, typically 60 times per second (or the display's refresh rate), passing a high-resolution timestamp. Unlike `setInterval`, it stays in sync with the screen, pauses automatically in background tabs (saving battery), and avoids drawing frames that would never be shown. An animation loop calls `requestAnimationFrame` again from inside the callback; `cancelAnimationFrame(id)` stops it. Base movement on the **elapsed time**, not the number of frames, so the speed is the same on 60 Hz and 120 Hz screens. For simple transitions of DOM elements, CSS transitions or the Web Animations API (`element.animate()`) are easier.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<canvas id="game" width="400" height="140" style="border:1px solid #ccc"></canvas>
<div id="box" style="width:40px;height:20px;background:#C9A227"></div>
```

A TIME-BASED ANIMATION LOOP

main.js

```
const canvas = document.querySelector("#game");
const ctx = canvas.getContext("2d");
const ball = { x: 30, y: 60, vx: 240, vy: 160, r: 12 };     // velocity in pixels per SECOND
let last = null, frames = 0, bounces = 0, id;

function frame(time) {
  if (last !== null) {
    const dt = Math.min((time - last) / 1000, 0.05);         // seconds since the previous frame
    ball.x += ball.vx * dt;
    ball.y += ball.vy * dt;
    if (ball.x < ball.r || ball.x > canvas.width - ball.r) { ball.vx *= -1; bounces++; }
    if (ball.y < ball.r || ball.y > canvas.height - ball.r) { ball.vy *= -1; bounces++; }
  }
  last = time;
  frames++;
  ctx.fillStyle = "rgba(251, 248, 241, 0.35)";               // translucent clear leaves a motion
trail
  ctx.fillRect(0, 0, canvas.width, canvas.height);
  ctx.beginPath(); ctx.arc(ball.x, ball.y, ball.r, 0, Math.PI * 2);
  ctx.fillStyle = "#1B2A4A"; ctx.fill();
  id = requestAnimationFrame(frame);
}
id = requestAnimationFrame(frame);

setTimeout(() => {
  cancelAnimationFrame(id);
  console.log(`stopped after ~1 s: ${frames > 30 ? "dozens of" : frames} frames drawn, ball bounced
${bounces > 0 ? "off the walls" : "0 times"}`);
}, 1000);

// the Web Animations API for DOM elements
const box = document.querySelector("#box");
const anim = box.animate([{ transform: "translateX(0)" }, { transform: "translateX(200px)" }], {
duration: 300, easing: "ease-out", fill: "forwards" });
anim.finished.then(() => console.log("element.animate finished, transform =",
getComputedStyle(box).transform));
```

BROWSER CONSOLE

```
element.animate finished, transform = matrix(1, 0, 0, 1, 200, 0)
stopped after ~1 s: dozens of frames drawn, ball bounced off the walls
```



THE LAST FRAME OF THE ANIMATION, WITH ITS MOTION TRAIL (SCREENSHOT)

21.2.3 Audio and video APIs

`<audio>` and `<video>` elements are controlled from JavaScript through the **HTMLMediaElement** API: `play()` (returns a promise; browsers block autoplay with sound until the user interacts), `pause()`, `currentTime`, `duration`, `volume`, `muted`, `playbackRate`, `loop`, and events such as `loadedmetadata`, `timeupdate`, `ended` and `error`. Custom player controls are built on these. For generating and processing

sound (synthesizers, visualisers, effects), the **Web Audio API** builds a graph of audio nodes. `getUserMedia` captures the camera and microphone, and `MediaRecorder` records them.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<audio controls></audio> <button id="start">Play tone</button>
```

MEDIA ELEMENT EVENTS AND THE WEB AUDIO API main.js

```
// Build a short WAV file in memory (a 440 Hz tone) and play it through an <audio> element
function makeTone(seconds = 1, freq = 440, rate = 8000) {
  const n = seconds * rate, buffer = new ArrayBuffer(44 + n * 2), v = new DataView(buffer);
  const str = (o, s) => [...s].forEach((c, i) => v.setUint8(o + i, c.charCodeAt(0)));
  str(0, "RIFF"); v.setUint32(4, 36 + n * 2, true); str(8, "WAVEfmt "); v.setUint32(16, 16, true);
  v.setUint16(20, 1, true); v.setUint16(22, 1, true); v.setUint32(24, rate, true); v.setUint32(28,
rate * 2, true);
  v.setUint16(32, 2, true); v.setUint16(34, 16, true); str(36, "data"); v.setUint32(40, n * 2, true);
  for (let i = 0; i < n; i++) v.setInt16(44 + i * 2, Math.sin(2 * Math.PI * freq * i / rate) * 8000,
true);
  return new Blob([buffer], { type: "audio/wav" });
}

const audio = document.querySelector("audio");
audio.src = URL.createObjectURL(makeTone(1));
audio.addEventListener("loadedmetadata", () => console.log("duration:", audio.duration.toFixed(2),
"s"));
audio.addEventListener("play", () => console.log("event: play"));
audio.addEventListener("ended", () => console.log("event: ended at", audio.currentTime.toFixed(2),
"s"));

audio.play().catch((e) => console.log("play() without a user gesture:", e.name));   // autoplay policy

document.querySelector("#start").addEventListener("click", () => {   // a user gesture allows playback
  audio.playbackRate = 2;                 // double speed: ends after about 0.5 s
  audio.play().then(() => console.log("play() after a click: promise resolved"));
});

console.log("can play mp4/h264?", document.createElement("video").canPlayType('video/mp4;
codecs="avc1.42E01E"') || "no", "| webm/vp9?",
document.createElement("video").canPlayType('video/webm; codecs="vp9"') || "no");

// Web Audio API: an oscillator analysed without playing anything aloud
const offline = new OfflineAudioContext(1, 4410, 44100);
const osc = new OscillatorNode(offline, { frequency: 1000, type: "square" });
osc.connect(offline.destination); osc.start();
offline.startRendering().then((buf) => {
  const samples = buf.getChannelData(0);
  console.log("Web Audio rendered", samples.length, "samples; square wave values:", [...new
Set(samples.slice(0, 100).map((s) => Math.round(s)))].join(" / "));
});
```

BROWSER CONSOLE

```
can play mp4/h264? no | webm/vp9? probably
play() without a user gesture: NotAllowedError
Web Audio rendered 4410 samples; square wave values: 0 / 1 / -1
duration: 1.00 s
event: play
play() after a click: promise resolved
event: ended at 1.00 s
```

Simulated user actions, in order: waits 200 ms; clicks `#start`; waits 900 ms.

A MINIMAL CUSTOM VIDEO PLAYER HTML

```
<video id="player" src="intro.mp4" poster="poster.jpg" width="640" preload="metadata" playsinline>
</video>
<button id="toggle">Play</button> <input id="seek" type="range" min="0" max="100" value="0">
<script>
  const v = document.querySelector("#player"), btn = document.querySelector("#toggle"), seek =
document.querySelector("#seek");
  btn.onclick = () => (v.paused ? v.play() : v.pause());
  v.onplay = () => (btn.textContent = "Pause");
  v.onpause = () => (btn.textContent = "Play");
  v.ontimeupdate = () => (seek.value = (v.currentTime / v.duration) * 100);
  seek.oninput = () => (v.currentTime = (seek.value / 100) * v.duration);
</script>
```

21.3 Background work

21.3.1 Web Workers

All page JavaScript shares the **main thread** with rendering and input handling, so a long computation (parsing a large file, image processing, encryption, complex calculations) freezes the page. A **Web Worker** runs a script in a **separate background thread**. The worker has no access to the DOM or `window`; the page and the worker communicate by **message passing**: `postMessage(data)` sends a copy of the data (structured clone), and the other side receives it in a `message` event. Large binary data can be **transferred** instead of copied. `new Worker("worker.js", { type: "module" })` creates a module worker. Shared workers and `SharedArrayBuffer` allow more advanced sharing.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<p id="status">idle</p>
```

MOVING HEAVY WORK OFF THE MAIN THREAD main.js

```
const worker = new Worker("prime-worker.js");
const status = document.querySelector("#status");

worker.addEventListener("message", (e) => {
  if (e.data.type === "progress") return;                    // could update a progress bar
  console.log(`worker found ${e.data.count} primes below ${e.data.limit} (largest
${e.data.largest})`);
  status.textContent = "done";
  worker.terminate();
});
worker.addEventListener("error", (e) => console.log("worker error:", e.message));

worker.postMessage({ limit: 2_000_000 });
console.log("message sent; the main thread stays free while the worker computes");

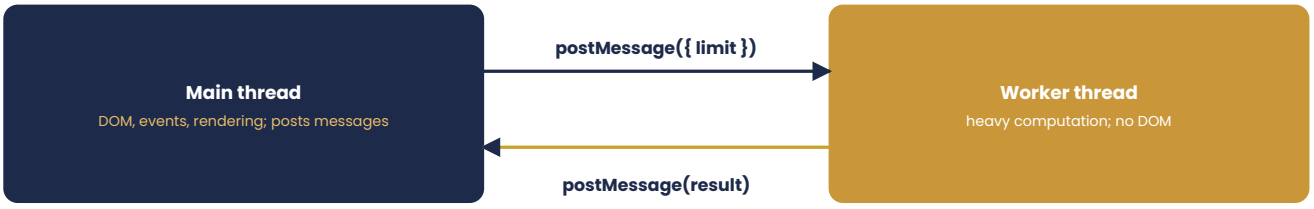
let ticks = 0;
const ui = setInterval(() => { ticks++; status.textContent = `working... (${ticks})`; }, 20);   // UI
keeps updating
const check = setInterval(() => {
  if (status.textContent === "done") { clearInterval(ui); clearInterval(check); console.log("UI timer
kept running during the work:", ticks > 1); }
}, 10);
```

BROWSER CONSOLE

```
message sent; the main thread stays free while the worker computes
worker found 148933 primes below 2000000 (largest 1999993)
UI timer kept running during the work: true
```

PRIME-WORKER.JS JavaScript

```
// prime-worker.js: runs in a background thread; no DOM access here
self.addEventListener("message", (e) => {
  const { limit } = e.data;
  const sieve = new Uint8Array(limit);
  let count = 0, largest = 0;
  for (let i = 2; i < limit; i++) {
    if (sieve[i]) continue;
    count++; largest = i;
    for (let j = i * i; j < limit; j += i) sieve[j] = 1;
  }
  self.postMessage({ type: "result", limit, count, largest });
});
```

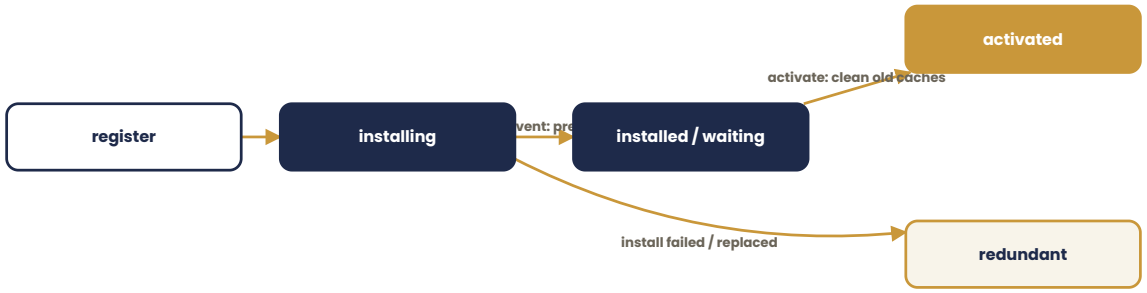


PAGE AND WORKER COMMUNICATE ONLY BY MESSAGES

21.3.2 Service Workers

A **Service Worker** is a special worker that sits **between your web app and the network**, acting as a programmable proxy. Once registered, it can intercept every request the pages of its scope make

(`fetch` event) and decide how to answer: from a **cache**, from the network, or by generating a response. This enables offline support, faster repeat visits, background sync and **push notifications** (it can run even when no page is open). Service workers require HTTPS (or localhost), have no DOM access, are event-driven (installed, activated, then woken for events), and are updated when the browser finds a changed script. They are the foundation of Progressive Web Apps.



THE SERVICE WORKER LIFE CYCLE

REGISTERING A SERVICE WORKER HTML

```
<!-- index.html (or main.js) -->
<script>
  if ("serviceWorker" in navigator) {
    window.addEventListener("load", async () => {
      try {
        const reg = await navigator.serviceWorker.register("/sw.js");
        console.log("service worker registered for scope", reg.scope);
      } catch (e) {
        console.log("registration failed:", e);
      }
    });
  }
</script>
```

21.4 Progressive Web Apps (PWA)

A **Progressive Web App** is a web application that uses modern APIs to behave like an installed app: it can be installed to the home screen or desktop, opens in its own window without browser bars, works offline or on poor connections, loads fast from cache, and can receive push notifications, while still being an ordinary website that works in any browser ("progressive enhancement"). Three ingredients are needed: HTTPS, a **web app manifest**, and a **service worker**. Twitter/X Lite, Starbucks, Pinterest and many Indian apps such as Flipkart Lite use PWAs to reach users on low-end phones and slow networks.

21.4.1 Web app manifest

The **manifest** is a JSON file, linked from the HTML with `<link rel="manifest">`, that tells the browser how the app should appear when installed: its name and short name, icons in several sizes (at least 192 and 512 px, plus a "maskable" icon for Android), start URL, display mode (`standalone` hides the browser interface), theme and background colours for the splash screen and title bar, and optional screenshots and shortcuts.

MANIFEST.WEBMANIFEST json

```
{
  "name": "LBF Sketchpad",
  "short_name": "Sketchpad",
  "description": "A drawing app that works offline",
  "start_url": "/?source=pwa",
  "scope": "/",
  "display": "standalone",
  "background_color": "#FBF8F1",
  "theme_color": "#1B2A4A",
  "icons": [
    { "src": "/icons/icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "/icons/icon-512.png", "sizes": "512x512", "type": "image/png" },
    { "src": "/icons/maskable-512.png", "sizes": "512x512", "type": "image/png", "purpose": "maskable"
}
  ],
  "shortcuts": [{ "name": "New sketch", "url": "/new" }]
}
```

LINKING THE MANIFEST HTML

```
<head>
  <link rel="manifest" href="/manifest.webmanifest">
  <meta name="theme-color" content="#1B2A4A">
  <link rel="apple-touch-icon" href="/icons/icon-192.png">
</head>
```

21.4.2 Offline caching with service workers

The service worker uses the **Cache Storage API** (`caches.open`, `cache.addAll`, `cache.match`, `cache.put`) to store responses. During the `install` event, it pre-caches the **app shell** (HTML, CSS, JS, icons); during `activate`, it deletes caches from old versions; and in the `fetch` event it applies a **caching strategy** per type of request. Libraries such as Google's Workbox (and Vite's PWA plugin) generate this code automatically.

A SERVICE WORKER WITH TWO CACHING STRATEGIES

JavaScript

```
// sw.js
const CACHE = "sketchpad-v3";
const APP_SHELL = ["/", "/index.html", "/app.js", "/style.css", "/icons/icon-192.png"];

self.addEventListener("install", (event) => {
  event.waitUntil(caches.open(CACHE).then((cache) => cache.addAll(APP_SHELL)));   // pre-cache
  self.skipWaiting();
});

self.addEventListener("activate", (event) => {
  event.waitUntil(caches.keys().then((keys) =>
    Promise.all(keys.filter((k) => k !== CACHE).map((k) => caches.delete(k)))));  // remove old
versions
  self.clients.claim();
});

self.addEventListener("fetch", (event) => {
  const { request } = event;
  if (request.method !== "GET") return;
  if (request.url.includes("/api/")) {
    // network first for data, fall back to the last cached copy when offline
    event.respondWith(fetch(request).then((res) => {
      const copy = res.clone();
      caches.open(CACHE).then((c) => c.put(request, copy));
      return res;
    }).catch(() => caches.match(request)));
  } else {
    // cache first for the app shell and static assets
    event.respondWith(caches.match(request).then((hit) => hit ?? fetch(request)));
  }
});
```

Strategy	How it works	Use for
Cache first	answer from cache; network only if missing	app shell, fonts, versioned assets
Network first	try network; cache as fallback when offline	API data, frequently updated pages
Stale-while-revalidate	answer from cache immediately, update cache from network in background	avatars, non-critical data
Network only / cache only	never cache / never fetch	payments, analytics / pre-cached offline page

USING CACHE STORAGE DIRECTLY main.js

```
// The Cache Storage API is also available to pages (secure contexts)
(async () => {
  const cache = await caches.open("demo-v1");
  await cache.put("/api/notes", new Response(JSON.stringify([{ id: 1, text: "Offline note" }]), {
headers: { "Content-Type": "application/json" } }));
  const hit = await cache.match("/api/notes");
  console.log("from cache:", JSON.stringify(await hit.json()));
  console.log("cache names:", await caches.keys());
  console.log("deleted:", await caches.delete("demo-v1"), "| remaining:", await caches.keys());
})();
```

BROWSER CONSOLE

```
from cache: [{"id":1,"text":"Offline note"}]
cache names: [demo-v1]
deleted: true | remaining: []
```

21.4.3 Installability

When a site is served over HTTPS and has a valid manifest (name, icons, start URL, display mode), browsers offer installation: Chrome and Edge show an install icon in the address bar, and Android shows an "Add to Home screen" prompt. A page can provide its own Install button by listening for the `beforeinstallprompt` event (Chromium), saving it, and calling `prompt()` when the user clicks. `appinstalled` fires after installation, and `matchMedia("(display-mode: standalone)")` tells the app whether it is running installed. Lighthouse and the DevTools **Application** panel check installability and show the manifest, service workers and caches.

A CUSTOM INSTALL BUTTON JavaScript

```
let deferredPrompt = null;
const installBtn = document.querySelector("#install");

window.addEventListener("beforeinstallprompt", (e) => {
  e.preventDefault();                // stop the mini-infobar; show our own button instead
  deferredPrompt = e;
  installBtn.hidden = false;
});

installBtn.addEventListener("click", async () => {
  deferredPrompt.prompt();
  const { outcome } = await deferredPrompt.userChoice;   // "accepted" or "dismissed"
  console.log("install", outcome);
  deferredPrompt = null;
  installBtn.hidden = true;
});

window.addEventListener("appinstalled", () => console.log("installed!"));
const isInstalled = matchMedia("(display-mode: standalone)").matches;
```

21.5 Device and system APIs

Browsers expose many device features, each guarded by the **permissions** model: powerful APIs work only in secure contexts (HTTPS), usually need a user gesture, and ask the user for permission the first time. Always **feature-detect** (`"geolocation" in navigator`), explain why you need the permission before asking, and handle refusal gracefully. `navigator.permissions.query({ name })` reports the current state (`granted`, `denied` or `prompt`).

21.5.1 Geolocation

`navigator.geolocation.getCurrentPosition(success, error, options)` asks for the device's location once; `watchPosition` keeps reporting changes (for navigation or fitness apps) until `clearWatch`. The position has `coords.latitude`, `longitude`, `accuracy` in metres, and sometimes `altitude`, `speed` and `heading`. Options: `enableHighAccuracy` (GPS, slower and more battery), `timeout`, `maximumAge` (accept a cached position). Errors have codes: 1 permission denied, 2 position unavailable, 3 timeout. The API uses callbacks; wrapping it in a promise is common.

FINDING THE NEAREST TRAINING CENTRE main.js

```
function getPosition(options) {
  return new Promise((resolve, reject) => navigator.geolocation.getCurrentPosition(resolve, reject,
options));
}
function distanceKm(a, b) {                                  // haversine formula
  const R = 6371, rad = (d) => (d * Math.PI) / 180;
  const dLat = rad(b.lat - a.lat), dLon = rad(b.lon - a.lon);
  const h = Math.sin(dLat / 2) ** 2 + Math.cos(rad(a.lat)) * Math.cos(rad(b.lat)) * Math.sin(dLon / 2)
** 2;
  return 2 * R * Math.asin(Math.sqrt(h));
}

(async () => {
  const state = (await navigator.permissions.query({ name: "geolocation" })).state;
  console.log("permission state:", state);
  try {
    const pos = await getPosition({ enableHighAccuracy: false, timeout: 5000, maximumAge: 60000 });
    const me = { lat: pos.coords.latitude, lon: pos.coords.longitude };
    console.log(`you are at ${me.lat.toFixed(4)}, ${me.lon.toFixed(4)} (accuracy
${pos.coords.accuracy} m)`);
    const centres = [{ name: "LBF Mysuru", lat: 12.2958, lon: 76.6394 }, { name: "LBF Chennai", lat:
13.0827, lon: 80.2707 }, { name: "LBF Pune", lat: 18.5204, lon: 73.8567 }];
    const nearest = centres.map((c) => ({ ...c, km: distanceKm(me, c) })).sort((a, b) => a.km - b.km);
    nearest.forEach((c) => console.log(`  ${c.name.padEnd(12)} ${c.km.toFixed(0)} km`));
  } catch (e) {
    console.log(["", "permission denied", "position unavailable", "timeout"][e.code] ?? e.message);
  }
})();
```

BROWSER CONSOLE

```
permission state: granted
you are at 12.9716, 77.5946 (accuracy 20 m)
  LBF Mysuru   128 km
  LBF Chennai  290 km
  LBF Pune     735 km
```

21.5.2 Notifications

The **Notifications API** shows system notifications outside the page. First request permission with `Notification.requestPermission()` (from a user action, such as clicking "Enable reminders"); then `new Notification(title, { body, icon, tag })` displays one while the page is open. A notification has `click` and `close` events. Notifications that arrive when the site is closed (**push notifications**) require a service worker with the Push API and a push server. Use them sparingly: unwanted notifications are the fastest way to lose users' trust.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<button id="remind">Enable class reminders</button>
```

REQUESTING PERMISSION AND SHOWING A NOTIFICATION main.js

```
async function enableReminders() {
  if (!("Notification" in window)) return console.log("notifications not supported");
  let permission = Notification.permission;
  if (permission === "default") permission = await Notification.requestPermission();
  console.log("permission:", permission);
  if (permission !== "granted") return;
  const n = new Notification("Class starts in 10 minutes", {
    body: "JavaScript: Async and the event loop (Room 2 / Zoom)",
    tag: "class-reminder",                  // replaces an older notification with the same tag
    requireInteraction: false,
  });
  n.addEventListener("show", () => console.log("notification shown:", n.title));
  n.addEventListener("error", () => console.log("the system could not display it (headless
browser)"));
  n.addEventListener("click", () => { window.focus(); n.close(); });
  console.log("created notification with tag:", n.tag);
}
document.querySelector("#remind").addEventListener("click", enableReminders);
```

BROWSER CONSOLE

```
permission: denied
```

Simulated user actions, in order: clicks `#remind`.

NOTE

The headless test browser always refuses notification permission, so the example took the "not granted" path and returned quietly, exactly what your code must do when a real user clicks Block. With permission granted, the notification appears in the operating system's notification area and the `show` event fires.

21.5.3 Clipboard API

The asynchronous **Clipboard API** reads and writes the system clipboard: `navigator.clipboard.writeText(text)` for "Copy" buttons, `readText()` to paste (which requires permission), and `write` / `read` with `ClipboardItem` for images and rich content. Both return promises, need a secure context, and writing normally needs a user gesture. Clipboard **events** (`copy`, `paste`) were covered in Module 16.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<p>Your coupon: <code id="coupon">LBF-WELCOME-2026</code> <button id="copy">Copy</button></p>
```

A COPY BUTTON

main.js

```
document.querySelector("#copy").addEventListener("click", async () => {
  const code = document.querySelector("#coupon").textContent;
  try {
    await navigator.clipboard.writeText(code);
    document.querySelector("#copy").textContent = "Copied!";
    console.log("copied to clipboard:", code);
    const back = await navigator.clipboard.readText();          // needs clipboard-read permission
    console.log("read back from clipboard:", back);
  } catch (e) {
    console.log("clipboard error:", e.name);
  }
});
```

BROWSER CONSOLE

```
copied to clipboard: LBF-WELCOME-2026
read back from clipboard: LBF-WELCOME-2026
--- page shows ---
Your coupon: LBF-WELCOME-2026 Copied!
```

Simulated user actions, in order: clicks `#copy`.

21.5.4 File API and drag-and-drop uploads

Files chosen with `<input type="file">` (optionally `multiple` and `accept="image/*"`) or dropped onto the page (`event.dataTransfer.files`, Module 16) arrive as `File` objects with `name`, `size`, `type` and `lastModified`. Their content can be read without uploading: `file.text()`, `file.arrayBuffer()` and `file.stream()` return promises (the older `FileReader` uses events), and `URL.createObjectURL(file)` gives a temporary URL for instant image previews (release it with `revokeObjectURL`). Upload with `FormData` (Module 18). The File System Access API (Chromium) can even open and save files on disk with the user's permission.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY)

HTML

```
<div id="drop" style="border:2px dashed #C9A227;padding:20px">Drop files here or <input id="picker"
type="file" multiple></div>
<ul id="files"></ul>
```

READING DROPPED FILES IN THE BROWSER

main.js

```
const zone = document.querySelector("#drop");
const list = document.querySelector("#files");

async function handleFiles(files) {
  for (const file of files) {
    const li = document.createElement("li");
    li.textContent = `${file.name} (${(file.size / 1024).toFixed(1)} KB, ${file.type || "unknown
type"})`;
    list.append(li);
    if (file.type === "text/csv") {
      const rows = (await file.text()).trim().split("\n").map((r) => r.split(","));
      console.log(`${file.name}: ${rows.length - 1} data rows, columns = ${rows[0].join(" | ")}`);
    } else if (file.type.startsWith("image/")) {
      const url = URL.createObjectURL(file);                // instant preview without uploading
      console.log(`${file.name}: preview URL ${url.slice(0, 18)}..., first bytes ${[...new
Uint8Array(await file.slice(0, 4).arrayBuffer())].map((b) => b.toString(16)).join(" ")}`);
      URL.revokeObjectURL(url);
    }
  }
}

zone.addEventListener("dragover", (e) => { e.preventDefault(); zone.classList.add("over"); });
zone.addEventListener("dragleave", () => zone.classList.remove("over"));
zone.addEventListener("drop", (e) => { e.preventDefault(); zone.classList.remove("over");
handleFiles(e.dataTransfer.files); });
document.querySelector("#picker").addEventListener("change", (e) => handleFiles(e.target.files));

// simulate the user dropping two files from the desktop
const dt = new DataTransfer();
dt.items.add(new File(["name,course,marks\nAsha,JS,91\nRavi,JS,78\n"], "marks.csv", { type: "text/csv"
}));
dt.items.add(new File([new Uint8Array([0x89, 0x50, 0x4e, 0x47, 1, 2, 3])], "photo.png", { type:
"image/png" }));
zone.dispatchEvent(new DragEvent("drop", { dataTransfer: dt, bubbles: true, cancelable: true }));
```

BROWSER CONSOLE

```
marks.csv: 2 data rows, columns = name | course | marks
photo.png: preview URL blob:http://lbf.lo..., first bytes 89 50 4e 47
--- page shows ---
Drop files here or
marks.csv (0.0 KB, text/csv)
photo.png (0.0 KB, image/png)
```

21.5.5 Web Share API

`navigator.share({ title, text, url, files })` opens the device's native share sheet (WhatsApp, email, messages…) on phones and on some desktop systems. It needs a secure context and a user gesture, returns a promise (rejected with `AbortError` if the user cancels), and `navigator.canShare(data)` checks whether particular data, such as files, can be shared. Where it is unavailable, fall back to copying the link or showing share buttons.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY)

HTML

```
<button id="share">Share this course</button>
```

SHARING WITH A FALLBACK main.js

```
async function shareCourse() {
  const data = { title: "JavaScript at LBF", text: "Join me in the JavaScript batch!", url:
"https://learnbeingforward.in/courses/js" };
  if (navigator.share && navigator.canShare?.(data)) {
    try {
      await navigator.share(data);
      console.log("shared via the native share sheet");
    } catch (e) {
      console.log(e.name === "AbortError" ? "user cancelled sharing" : "share failed: " + e.message);
    }
  } else {
    await navigator.clipboard.writeText(data.url);           // graceful fallback
    console.log("Web Share not available here: link copied instead ->", await
navigator.clipboard.readText());
  }
}
document.querySelector("#share").addEventListener("click", shareCourse);
```

BROWSER CONSOLE

```
Web Share not available here: link copied instead -> https://learnbeingforward.in/courses/js
```

Simulated user actions, in order: clicks `#share`.

21.6 Real-time communication

HTTP is request/response: the client asks, the server answers. For live data (chat, notifications, stock prices, collaborative editing, multiplayer games, delivery tracking), the server must be able to **push** updates. Polling (asking every few seconds) is wasteful and slow. Browsers offer two better options.

	Polling	Server-Sent Events	WebSockets
Direction	client asks repeatedly	server to client	both ways
Protocol	HTTP	HTTP (text/event-stream)	ws:// or wss:// after an HTTP upgrade
Data	any	text (UTF-8) events	text or binary
Reconnection	n/a	automatic, with Last-Event-ID	write your own
Use for	rare updates	notifications, feeds, live scores, AI token streaming	chat, games, collaboration

21.6.1 WebSockets

A **WebSocket** is a persistent, full-duplex connection: after an initial HTTP "upgrade" handshake, client and server can send messages to each other at any time with very little overhead. In the browser: `new WebSocket("wss://host/path")`, then `send(data)` and the events `open`, `message` (data in `event.data`), `error` and `close`; check `readyState` before sending. Messages are usually JSON with a `type` field. Production apps add reconnection with backoff, heartbeats and authentication; libraries such as Socket.IO add rooms and fallbacks, and Node.js servers commonly use the `ws` package.

A CHAT CLIENT TALKING TO A WEBSOCKET SERVER

main.js

```
const log = (m) => console.log(m);
const socket = new WebSocket("wss://lbf.local/ws/chat");

socket.addEventListener("open", () => {
  log("connected, readyState = " + socket.readyState);
  socket.send(JSON.stringify({ type: "join", room: "js-batch", user: "Asha" }));
  socket.send(JSON.stringify({ type: "message", text: "Is the class at 10?" }));
});

socket.addEventListener("message", (event) => {
  const msg = JSON.parse(event.data);
  if (msg.type === "system") log(`[system] ${msg.text}`);
  if (msg.type === "message") log(`${msg.from}: ${msg.text}`);
  if (msg.type === "message" && msg.from === "Ravi") socket.close(1000, "done");
});

socket.addEventListener("close", (e) => log(`closed (code ${e.code})`));
socket.addEventListener("error", () => log("connection error"));
```

BROWSER CONSOLE

```
[server] WebSocket connection opened
connected, readyState = 1
[server] received: {"type":"join","room":"js-batch","user":"Asha"}
[server] received: {"type":"message","text":"Is the class at 10?"}
[system] Welcome to the chat
[system] Asha joined js-batch (12 online)
Meera: Yes, 10 am sharp
Ravi: See you there!
closed (code 1000)
```

NOTE

The `[server]` lines come from the simulated server: it greeted the client on connection and answered each message. A real server (for example Node.js with the `ws` package, Module 22) would broadcast messages to everyone in the room.

21.6.2 Server-Sent Events

Server-Sent Events (SSE) stream events from the server to the browser over a normal HTTP response with the content type `text/event-stream`. The browser's `EventSource` object connects, parses the stream, dispatches `message` events (or named events when the server sends `event: name`), and **reconnects automatically** if the connection drops, sending the last received `id`. Because it is plain HTTP, SSE works through proxies and with HTTP/2, and is simpler than WebSockets when data flows only one way. It is how many AI chat interfaces stream their answers.

WHAT A TEXT/EVENT-STREAM RESPONSE LOOKS LIKE

text

```
id: 1
event: score
data: {"team":"IND","runs":182,"overs":"18.2"}

id: 2
event: score
data: {"team":"IND","runs":188,"overs":"18.4"}

event: wicket
data: {"batter":"Player 7","bowler":"Player 22"}

data: Commentary: what a finish this is turning into!
```

RECEIVING A LIVE SCORE STREAM

main.js

```
const source = new EventSource("/live/score");
let count = 0;

source.addEventListener("open", () => console.log("stream opened"));
source.addEventListener("score", (e) => {
  const s = JSON.parse(e.data);
  console.log(`score update #${e.lastEventId}: ${s.team} ${s.runs} (${s.overs} ov)`);
});
source.addEventListener("wicket", (e) => console.log("WICKET!", JSON.parse(e.data).batter));
source.addEventListener("message", (e) => {                // events without a name
  console.log(e.data);
  if (++count === 1) { source.close(); console.log("closed, readyState =", source.readyState); }
});
source.addEventListener("error", () => console.log("stream error, the browser will retry"));
```

BROWSER CONSOLE

```
[server] GET /live/score
stream opened
score update #1: IND 182 (18.2 ov)
score update #2: IND 188 (18.4 ov)
WICKET! Player 7
Commentary: what a finish this is turning into!
closed, readyState = 2
```

PRODUCING AN EVENT STREAM ON THE SERVER

main.js

```
// Node.js: a minimal SSE endpoint (Module 22)
import http from "node:http";
http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "text/event-stream", "Cache-Control": "no-cache", Connection:
"keep-alive" });
  let id = 0;
  const timer = setInterval(() => res.write(`id: ${++id}\ndata: ${JSON.stringify({ time: new
Date().toISOString() })}\n\n`), 1000);
  req.on("close", () => clearInterval(timer));
}).listen(3000);
```

NOTE

Each write sends one event; the connection stays open until the client disconnects.

21.7 Web Components

Web Components are a set of browser standards for creating your own reusable HTML elements, with encapsulated structure, style and behaviour, that work in any page and alongside any framework (or none). They consist of three technologies: **custom elements** (define a new tag and its behaviour), the **Shadow DOM** (a private DOM tree with scoped styles), and **templates and slots** (markup to stamp out, with placeholders for the user's content). Design systems at companies such as Adobe, Salesforce, GitHub and Microsoft are built with them.

21.7.1 Custom elements

A **custom element** is a class extending `HTMLElement`, registered with `customElements.define("tag-name", Class)`; the name must contain a hyphen. **Life-cycle callbacks** run at key moments: `constructor` (created; set up state only), `connectedCallback` (inserted into the document; render and add listeners), `disconnectedCallback` (removed; clean up), and `attributeChangedCallback(name, old, value)` for the attributes listed in `static observedAttributes`. Exposing attributes and properties, and dispatching custom events, gives components a clean public API.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<p>JavaScript course: <star-rating id="course" value="3"></star-rating></p>
<p>Old widget: <star-rating id="old" value="2" max="3"></star-rating></p>
```

A <STAR-RATING> CUSTOM ELEMENT

main.js

```
class StarRating extends HTMLElement {
  static observedAttributes = ["value", "max"];

  connectedCallback() {
    this.addEventListener("click", this.#onClick);
    this.render();
    console.log(`<star-rating> connected with value ${this.value}`);
  }
  disconnectedCallback() { this.removeEventListener("click", this.#onClick); console.log("<star-
rating> removed"); }
  attributeChangedCallback(name, oldValue, newValue) {
    if (this.isConnected) { this.render(); console.log(`attribute ${name}: ${oldValue} ->
${newValue}`); }
  }

  get value() { return Number(this.getAttribute("value") ?? 0); }
  set value(v) { this.setAttribute("value", v); }                // property reflects to the
attribute
  get max() { return Number(this.getAttribute("max") ?? 5); }

  #onClick = (e) => {
    const star = e.target.closest("[data-n]");
    if (!star) return;
    this.value = Number(star.dataset.n);
    this.dispatchEvent(new CustomEvent("rating-change", { detail: this.value, bubbles: true }));
  };
  render() {
    this.innerHTML = Array.from({ length: this.max }, (_, i) =>
      `<span data-n="${i + 1}" style="cursor:pointer;color:${i < this.value ? "#C9A227" :
"#ccc"};font-size:24px">★</span>`).join("");
  }
}
customElements.define("star-rating", StarRating);

document.addEventListener("rating-change", (e) => console.log("rating-change event, new rating:",
e.detail));
document.querySelector("#course").value = 4;                    // set from code
setTimeout(() => document.querySelector("#old").remove(), 0);
```

BROWSER CONSOLE

```
attribute value: null -> 3
<star-rating> connected with value 3
attribute value: null -> 2
attribute max: null -> 3
<star-rating> connected with value 2
attribute value: 3 -> 4
<star-rating> removed
attribute value: 4 -> 5
rating-change event, new rating: 5
```

Simulated user actions, in order: clicks `#course [data-n="5"]`.



THE COMPONENT AFTER THE CLICK ON THE FIFTH STAR (SCREENSHOT)

21.7.2 Shadow DOM

`this.attachShadow({ mode: "open" })` gives an element a **shadow root**: a separate, hidden DOM tree rendered in place of (or around) the element's normal children. Styles inside the shadow root apply **only**

there, and page styles do not leak in (except inherited properties such as `color` and `font`, and CSS custom properties, which are the intended way to theme a component). `document.querySelector` does not see inside it, so a component's internals cannot be broken accidentally. `:host` styles the element itself, and `::part()` lets the page style parts the component exposes. Native elements such as `<video>` and `<input type="range">` use the Shadow DOM internally.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<style>p { text-transform: uppercase; color: crimson; } .title { font-size: 40px; } alert-box.warn { --alert-bg: #FFF4D6; --alert-accent: #C9A227; }</style>
<p class="title">A page paragraph (uppercase, crimson)</p>
<alert-box title="Info:" message="Shadow DOM keeps component styles private."></alert-box>
<alert-box class="warn" title="Reminder:" message="Themed with CSS custom properties."></alert-box>
```

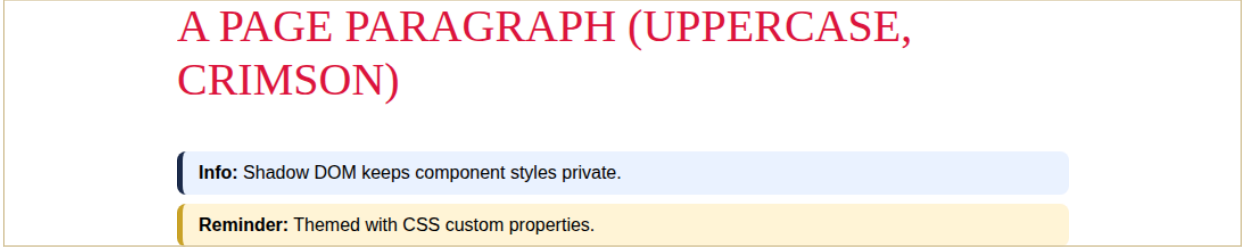
ENCAPSULATED STYLES WITH SHADOW DOM main.js

```
class AlertBox extends HTMLElement {
  constructor() {
    super();
    const root = this.attachShadow({ mode: "open" });
    root.innerHTML = `
      <style>
        :host { display: block; margin: 8px 0; }
        p { margin: 0; padding: 10px 14px; border-radius: 8px; font-family: sans-serif;
            background: var(--alert-bg, #EAF2FF); border-left: 5px solid var(--alert-accent, #1B2A4A);
}
        .title { font-weight: bold; }
      </style>
      <p><span class="title" part="title"></span> <span class="msg"></span></p>`;
  }
  connectedCallback() {
    this.shadowRoot.querySelector(".title").textContent = this.getAttribute("title") ?? "Note";
    this.shadowRoot.querySelector(".msg").textContent = this.getAttribute("message") ?? "";
  }
}
customElements.define("alert-box", AlertBox);

const box = document.querySelector("alert-box");
console.log("page selector sees inside?", document.querySelector(".title") !== null &&
document.querySelector(".title").closest("alert-box") !== null);
console.log("page <p> styles leaked in?",
getComputedStyle(box.shadowRoot.querySelector("p")).textTransform);
console.log("inner title via shadowRoot:", box.shadowRoot.querySelector(".title").textContent);
```

BROWSER CONSOLE

```
page selector sees inside? false
page <p> styles leaked in? none
inner title via shadowRoot: Info:
```



PAGE STYLES (UPPERCASE, CRIMSON, 40PX TITLES) DO NOT AFFECT THE COMPONENTS; CUSTOM PROPERTIES THEME THE SECOND ONE

21.7.3 Templates and slots

A `<template>` (Module 15) holds inert markup to clone into the shadow root, keeping HTML out of JavaScript strings. **Slots** are placeholders in the shadow tree where the element's own children (the "light DOM") are displayed. A default `<slot>` receives all unassigned children; **named slots** (`<slot name="price">`) receive children with a matching `slot="price"` attribute. Slotted content stays in the page's DOM, so it remains accessible and styled by the page (and by `::slotted()` from inside). The `slotchange` event fires when slotted content changes.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<template id="course-card-tpl">
  <style>
    .card { width: 240px; padding: 12px; border: 1px solid #D8C9A4; border-radius: 10px; font-family:
sans-serif; }
    ::slotted(h3) { margin: 0 0 6px; color: #1B2A4A; }
    .price { color: #C9A227; font-weight: bold; }
  </style>
  <div class="card">
    <slot name="title"><h3>Untitled course</h3></slot>
    <slot></slot>
    <div class="price"><slot name="price">Free</slot></div>
    <slot name="badge">No badge</slot>
  </div>
</template>
<course-card>
  <h3 slot="title">JavaScript Mastery</h3>
  <p>69 days from basics to advanced.</p>
  <span slot="price">Rs 14,999</span>
</course-card>
```

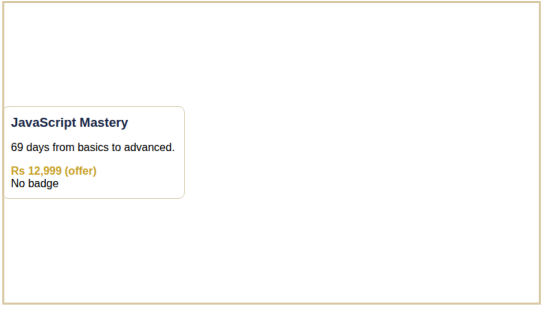
COMPOSING CONTENT WITH NAMED SLOTS main.js

```
class CourseCard extends HTMLElement {
  constructor() {
    super();
    const tpl = document.querySelector("#course-card-tpl");
    this.attachShadow({ mode: "open" }).append(tpl.content.cloneNode(true));
    this.shadowRoot.querySelector("slot[name=price]").addEventListener("slotchange", (e) =>
      console.log("slotchange: price is now", e.target.assignedElements()[0]?.textContent));
  }
}
customElements.define("course-card", CourseCard);

const card = document.querySelector("course-card");
const slots = card.shadowRoot.querySelectorAll("slot");
slots.forEach((s) => console.log(`slot "${s.name || "(default)"}" shows:`, s.assignedNodes({ flatten:
true }).map((n) => n.textContent.trim()).filter(Boolean).join(" + ") || "(fallback content)"));
setTimeout(() => {                                    // replacing the slotted element fires slotchange
  const offer = Object.assign(document.createElement("span"), { slot: "price", textContent: "Rs 12,999
(offer)" });
  card.querySelector("[slot=price]").replaceWith(offer);
}, 50);
```

BROWSER CONSOLE

```
slot "title" shows: JavaScript Mastery
slot "(default)" shows: 69 days from basics to advanced.
slot "price" shows: Rs 14,999
slot "badge" shows: No badge
slotchange: price is now Rs 14,999
slotchange: price is now Rs 12,999 (offer)
```



THE RENDERED <COURSE-CARD>: SLOTTED TITLE, DEFAULT CONTENT, UPDATED PRICE AND THE FALLBACK BADGE

Observers replace polling: IntersectionObserver (rootMargin, threshold) drives infinite scroll and lazy images, MutationObserver batches DOM change records, ResizeObserver reports element size changes. Canvas 2D draws shapes, paths, text and images in immediate mode; requestAnimationFrame runs time-based animation loops in step with the display; media elements expose play/pause/currentTime and events, and the Web Audio API generates and processes sound. Web Workers run heavy code on background threads via postMessage; Service Workers proxy network requests, enabling offline caching strategies and push. PWAs combine HTTPS, a web app manifest and a service worker to be installable and offline-capable. Device APIs (geolocation, notifications, clipboard, files, Web Share) need secure contexts, permissions and feature detection. WebSockets give two-way real-time messaging; Server-Sent Events stream server updates over HTTP with automatic reconnection. Web Components combine custom elements (life-cycle callbacks, observed attributes), Shadow DOM (encapsulated styles, theming with custom properties) and templates with slots.

PRACTICE CHECKPOINT

Build an **infinite-scroll image feed** with IntersectionObserver, and a simple **drawing app on Canvas** that works **offline as a PWA**.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```html
<canvas id="pad" width="300" height="140" style="border:1px solid #D8C9A4;touch-action:none"></canvas>
<input id="colour" type="color" value="#1b2a4a">
<div id="feed"></div>
<div id="more">Loading...</div>
```

SAMPLE SOLUTION main.js

```
// ---- Part 1: infinite feed with lazy images ----
const feed = document.querySelector("#feed");
const sentinel = document.querySelector("#more");
let page = 0;
const lazy = new IntersectionObserver((entries, obs) => {
  entries.filter((e) => e.isIntersecting).forEach((e) => { e.target.src = e.target.dataset.src;
obs.unobserve(e.target); });
}, { rootMargin: "200px" });
const pager = new IntersectionObserver(async ([entry]) => {
  if (!entry.isIntersecting) return;
  const res = await fetch(`/api/photos?page=${++page}`);
  const { photos, hasMore } = await res.json();
  for (const p of photos) {
    const img = Object.assign(document.createElement("img"), { alt: p.title, width: 160, height: 100
});
    img.dataset.src = p.url;
    feed.append(img);
    lazy.observe(img);
```

```
  }
  console.log(`feed page ${page}: ${feed.children.length} photos${hasMore ? "" : " (end of feed)"}`);
  if (!hasMore) return pager.disconnect();
  pager.unobserve(sentinel); pager.observe(sentinel);             // re-check: still visible? load
again
}, { rootMargin: "300px" });
pager.observe(sentinel);

// ---- Part 2: drawing app on a canvas ----
const canvas = document.querySelector("#pad");
const ctx = canvas.getContext("2d");
ctx.lineCap = "round"; ctx.lineJoin = "round"; ctx.lineWidth = 6; ctx.strokeStyle = "#1B2A4A";
let drawing = false;
const pos = (e) => { const r = canvas.getBoundingClientRect(); return [e.clientX - r.left, e.clientY -
r.top]; };
canvas.addEventListener("pointerdown", (e) => { drawing = true; ctx.beginPath();
ctx.moveTo(...pos(e)); });
canvas.addEventListener("pointermove", (e) => { if (drawing) { ctx.lineTo(...pos(e)); ctx.stroke(); }
});
canvas.addEventListener("pointerup", () => { drawing = false; localStorage.setItem("sketch",
canvas.toDataURL()); });   // autosave
document.querySelector("#colour").addEventListener("input", (e) => (ctx.strokeStyle =
e.target.value));

// simulate a user drawing the letters "JS" (a real user drags with mouse, pen or finger)
const r = canvas.getBoundingClientRect();
const stroke = (pts) => {
  const ev = (type, [x, y]) => canvas.dispatchEvent(new PointerEvent(type, { clientX: r.left + x,
clientY: r.top + y, bubbles: true }));
  ev("pointerdown", pts[0]); pts.slice(1).forEach((p) => ev("pointermove", p)); ev("pointerup",
pts.at(-1));
};
stroke([[60, 30], [60, 100], [55, 115], [40, 120], [28, 110]]);                          // J
ctx.strokeStyle = "#C9A227";
stroke([[150, 35], [120, 30], [100, 45], [110, 70], [140, 80], [150, 105], [130, 120], [100, 115]]);
// S
console.log("sketch autosaved to localStorage:", (localStorage.getItem("sketch") ??
"").startsWith("data:image/png"));

// ---- Part 3: make it an offline PWA ----
// manifest.webmanifest (name, icons, display: "standalone") is linked in <head>, and sw.js pre-caches
// "/", "/app.js", "/style.css" and the icons with a cache-first strategy (see 21.4.2).
if ("serviceWorker" in navigator && location.protocol === "https:") {
  console.log("service worker support available: register('/sw.js') on load to work offline");
}
```

BROWSER CONSOLE

```
sketch autosaved to localStorage: true
service worker support available: register('/sw.js') on load to work offline
feed page 1: 6 photos
feed page 2: 10 photos (end of feed)
```

Simulated user actions, in order: waits 300 ms; scrolls down 900 pixels with the mouse wheel; waits 300 ms.



THE CANVAS AFTER THE SIMULATED DRAWING (SCREENSHOT)

MODULE 22 · ADVANCED

Back-end JavaScript with Node.js and Express

Minimum duration **5 days** | Level **Advanced**

Node.js runs JavaScript on servers, so one language can power the whole application. This module covers Node.js fundamentals (how it differs from the browser, its event loop, globals, running scripts and the REPL), the core modules (fs, path, os, events, http, streams and buffers, the test runner and watch mode), the Express framework (routing, request and response, middleware, static files, error handling, project structure), building a complete REST API with validation and configuration, working with MongoDB and SQL databases, authentication and security (password hashing, JWT, protected routes, CORS, Helmet, rate limiting), and deployment.

IN THIS MODULE

22.1 Node.js fundamentals
22.2 Core modules
22.3 Express.js
22.4 Building a REST API
22.5 Databases
22.6 Authentication and security
22.7 Deployment

NOTE

TERMINAL examples were really run with Node.js 22: a server started with `&` runs in the background while the following `curl` commands send it requests; lines marked `[server log]` are what the server printed meanwhile. Examples that need npm packages (Express, Mongoose, bcrypt…) cannot be installed in the typesetting environment, so they show the output those programs produce.

22.1 Node.js fundamentals

22.1.1 What Node.js is and how it differs from the browser

Node.js (2009, Ryan Dahl) is a JavaScript **runtime** built on Chrome's V8 engine, designed for servers and command-line tools. It adds what a server needs and a browser must forbid: reading and writing files, opening network ports, running processes, accessing environment variables. Its I/O is **non-blocking**: one thread with an event loop can handle thousands of concurrent connections that are mostly waiting for databases and networks, which makes Node.js excellent for APIs, real-time services, proxies and tooling (npm, Vite, ESLint and TypeScript all run on Node.js). It is less suited to heavy CPU-bound computation on the main thread (use worker threads or another language for that). Netflix, PayPal, LinkedIn, Uber and Walmart run large Node.js services.

	Browser	Node.js
Global object	`window` (and `globalThis`)	`global` (and `globalThis`)

	Browser	Node.js
DOM, document, alert	yes	no
File system, processes, raw sockets	no (sandboxed)	yes: fs, child_process, net
Module systems	ES modules	ES modules and CommonJS
Environment	user's device, many browser versions	your server, one Node.js version you choose
Security model	untrusted code in a sandbox	trusted code with full access (permission model optional)
Shared APIs	fetch, URL, timers, console, crypto.subtle, streams, structuredClone, EventTarget	the same web-standard APIs

THE ENVIRONMENT NODE.JS PROVIDES

main.js

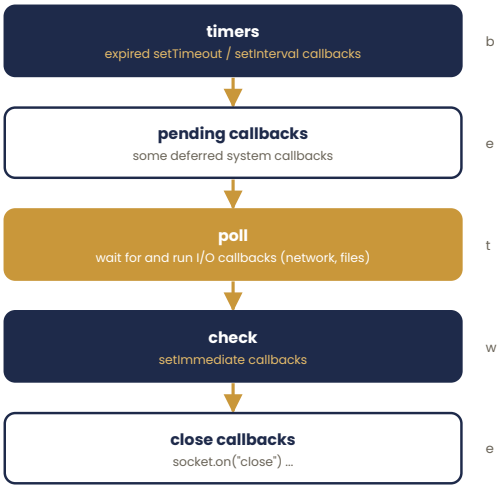
```
console.log("runtime:", typeof window === "undefined" ? "Node.js" : "browser", process.version);
console.log("platform:", process.platform, "| architecture:", process.arch);
console.log("globalThis === global:", globalThis === global);
console.log("web APIs also in Node.js:", ["fetch", "URL", "structuredClone", "TextEncoder",
"AbortController", "crypto"].filter((n) => n in globalThis).join(", "));
console.log("DOM in Node.js:", typeof document);
```

OUTPUT

```
runtime: Node.js v22.22.2
platform: linux | architecture: x64
globalThis === global: true
web APIs also in Node.js: fetch, URL, structuredClone, TextEncoder, AbortController, crypto
DOM in Node.js: undefined
```

22.1.2 The Node.js event loop and libuv (overview)

Node.js uses the same model as the browser (Module 17): one JavaScript thread, a call stack, microtasks and callback queues. The difference is underneath. **libuv**, a C library, implements the event loop and asynchronous I/O: network operations use the operating system's non-blocking mechanisms (epoll, kqueue, IOCP), while file-system operations, DNS lookups, compression and crypto run on a small **thread pool** (4 threads by default). Node's event loop goes through **phases** in each iteration: timers (`setTimeout` / `setInterval`), pending callbacks, poll (new I/O events), check (`setImmediate`), and close callbacks. Between every callback, Node.js drains `process.nextTick` callbacks first and then promise microtasks.



THE PHASES OF ONE NODE.JS EVENT-LOOP ITERATION

NEXTTICK, MICROTASKS, TIMERS AND SETIMMEDIATE

main.js

```
const fs = require("node:fs");

console.log("1 sync");
setTimeout(() => console.log("5/6 setTimeout 0 (timers phase)"), 0);
setImmediate(() => console.log("5/6 setImmediate (check phase)"));
Promise.resolve().then(() => console.log("4 promise microtask"));
process.nextTick(() => console.log("3 process.nextTick (before promises)"));
console.log("2 sync end");

fs.readFile(__filename, () => {                    // inside an I/O callback the order is fixed:
  setTimeout(() => console.log("8 timeout scheduled from I/O"), 0);
  setImmediate(() => console.log("7 immediate scheduled from I/O (check comes right after poll)"));
});
```

OUTPUT

```
1 sync
2 sync end
3 process.nextTick (before promises)
4 promise microtask
5/6 setTimeout 0 (timers phase)
5/6 setImmediate (check phase)
7 immediate scheduled from I/O (check comes right after poll)
8 timeout scheduled from I/O
```

NOTE

Outside an I/O callback, whether `setTimeout(fn, 0)` or `setImmediate` runs first depends on process timing, which is why both are labelled 5/6. Inside an I/O callback, `setImmediate` always runs first.

22.1.3 Global objects: process, __dirname, import.meta

Node.js provides a few globals of its own. `process` describes the running program: `process.argv` (command-line arguments), `process.env` (environment variables), `process.cwd()`, `process.exit(code)` and `process.exitCode`, `process.memoryUsage()`, `process.uptime()`, the standard streams `stdin` / `stdout` / `stderr`, and events such as `exit`, `uncaughtException` and the `SIGINT` signal (Ctrl+C). In **CommonJS** files, `__dirname` and `__filename` give the current file's folder and path, and `require` / `module` are available. In **ES modules**, use `import.meta.dirname`, `import.meta.filename` and `import.meta.url` instead. `Buffer`, timers, `console`, `URL` and `fetch` are global too.

INFO.MJS

JavaScript

```
const [, , command = "(none)", ...flags] = process.argv;          // skip node path and script path
console.log("command:", command, "| flags:", flags);
console.log("NODE_ENV:", process.env.NODE_ENV ?? "(not set)");
console.log("this file:", import.meta.filename.split("/").pop(), "| folder:",
import.meta.dirname.split("/").pop());
console.log("cwd is the project folder:", process.cwd() === import.meta.dirname);
console.log("pid is a number:", Number.isInteger(process.pid), "| uptime under 1 s:", process.uptime()
< 1);
process.on("exit", (code) => console.log(`exiting with code ${code}`));
process.exitCode = command === "(none)" ? 0 : 0;
```

PROCESS AND IMPORT.META IN AN ES MODULE

main.js

```
node info.mjs report --format=json
NODE_ENV=production node info.mjs
```

TERMINAL

```
$ node info.mjs report --format=json
command: report | flags: [ '--format=json' ]
NODE_ENV: (not set)
this file: info.mjs | folder: my-app
cwd is the project folder: true
pid is a number: true | uptime under 1 s: true
exiting with code 0
$ NODE_ENV=production node info.mjs
command: (none) | flags: []
NODE_ENV: production
this file: info.mjs | folder: my-app
cwd is the project folder: true
pid is a number: true | uptime under 1 s: true
exiting with code 0
```

22.1.4 Running scripts and the REPL

Run a file with `node app.js` (or `node app.mjs`); pass arguments after the file name. `node -e "code"` evaluates a snippet, `node -p "expr"` prints its value, `node --watch app.js` restarts on file changes (22.2.7), `node --env-file=.env` loads environment variables, and `node --inspect` enables the debugger (Module 14). Typing `node` alone starts the **REPL** (Read-Eval-Print Loop), an interactive prompt for experimenting: it prints the value of each expression, `_` holds the last result, Tab completes names, and dot-commands such as `.help`, `.editor` (multi-line input), `.load file.js` and `.exit` are available.

ONE-LINERS AND A REPL SESSION

main.js

```
node -p "[1, 2, 3].map((n) => n * 2)"
node -e "console.log(new Date(2026, 2, 9).toDateString())"
printf '1 + 2\n_ * 10\nconst s = new Set([1, 1, 2])\ns.size\n.exit\n' | node -i 2>&1 | tail -n +3 |
sed 's/> //g'
```

TERMINAL

```
$ node -p "[1, 2, 3].map((n) => n * 2)"
[ 2, 4, 6 ]
$ node -e "console.log(new Date(2026, 2, 9).toDateString())"
Mon Mar 09 2026
$ printf '1 + 2\n_ * 10\nconst s = new Set([1, 1, 2])\ns.size\n.exit\n' | node -i 2>&1 | tail -n +3 |
sed 's/> //g'
3
30
undefined
2
```

NOTE

In an interactive terminal the REPL shows a `>` prompt before each line you type; here the input was piped in, so only the printed results appear: 3, 30, undefined (for the declaration) and 2.

22.2 Core modules

Node.js ships with built-in modules, imported with the `node:` prefix (recommended, so they cannot be confused with npm packages): `import fs from "node:fs/promises"` or `require("node:fs")`. The most used are fs, path, os, events, http, stream, crypto, url, util, child_process and worker_threads.

22.2.1 fs and fs/promises (reading and writing files)

The **fs** module works with the file system in three styles: **promise-based** (`node:fs/promises`, recommended with `await`), **callback-based** (the original style), and **synchronous** (`readFileSync` etc., which block the event loop: fine for start-up scripts and CLIs, never inside a request handler). Common operations: `readFile` / `writeFile` (whole files; pass `"utf8"` for text), `appendFile`, `mkdir` (with `recursive: true`), `readdir`, `stat`, `rename`, `rm`, `copyFile`, and `access` / `existsSync`. Errors carry a `code` such as `ENOENT` (no such file) or `EACCES` (permission denied).

STUDENTS.JSON

JavaScript

```
[{"name":"Asha","marks":91},{"name":"Ravi","marks":64},{"name":"Meera","marks":85}]
```

FILES.MJS JavaScript

```
import { readFile, writeFile, appendFile, mkdir, readdir, stat, rm } from "node:fs/promises";
import { existsSync } from "node:fs";

const students = JSON.parse(await readFile("students.json", "utf8"));
console.log("read", students.length, "students");

await mkdir("data/archive", { recursive: true });              // no error if it exists
const lines = students.map((s) => `${s.name.padEnd(6)} ${s.marks} ${s.marks >= 75 ? "distinction" :
"pass"}`);
await writeFile("data/report.txt", "REPORT\n" + lines.join("\n") + "\n");
await appendFile("data/report.txt", `average: ${(students.reduce((t, s) => t + s.marks, 0) /
students.length).toFixed(1)}\n`);

const info = await stat("data/report.txt");
console.log("report size:", info.size, "bytes | is file:", info.isFile());
console.log("data folder:", await readdir("data"));

try {
  await readFile("missing.txt", "utf8");
} catch (err) {
  console.log("error code:", err.code);                          // ENOENT
}
await rm("data/archive", { recursive: true });
console.log("archive removed:", !existsSync("data/archive"));
```

READING, WRITING AND LISTING FILES main.js

```
node files.mjs
cat data/report.txt
```

TERMINAL

```
$ node files.mjs
read 3 students
report size: 80 bytes | is file: true
data folder: [ 'archive', 'report.txt' ]
error code: ENOENT
archive removed: true
$ cat data/report.txt
REPORT
Asha   91 distinction
Ravi   64 pass
Meera  85 distinction
average: 80.0
```

CALLBACK AND SYNCHRONOUS STYLES main.js

```
import fs from "node:fs";
fs.readFile("students.json", "utf8", (err, text) => {          // callback style (error-first)
  if (err) return console.error(err.code);
  console.log(JSON.parse(text).length);
});
const config = JSON.parse(fs.readFileSync("config.json", "utf8"));   // synchronous: only at start-up
```

NOTE

Same results; prefer fs/promises with await in application code.

22.2.2 path

File paths differ between operating systems (`/` on Linux and macOS, `\` on Windows). The **path** module builds and takes apart paths portably: `path.join` joins segments with the right separator and normalises `..`; `path.resolve` produces an absolute path (from the current directory); `basename`, `dirname`, `extname` and `parse` split a path; `relative` computes one path relative to another. Always build paths with path functions rather than string concatenation, and base them on `import.meta.dirname` / `__dirname` rather than the current working directory, which depends on where the program was started.

BUILDING AND SPLITTING PATHS main.js

```
const path = require("node:path");

const file = "/home/asha/projects/lbf-api/src/routes/students.js";
console.log("basename:", path.basename(file), "| without ext:", path.basename(file, ".js"));
console.log("dirname: ", path.dirname(file));
console.log("extname: ", path.extname(file));
console.log("parse:   ", path.parse(file));
console.log("join:    ", path.join("/home/asha", "projects", "..", "uploads", "photo.png"));
console.log("relative:", path.relative("/home/asha/projects/lbf-api", file));
console.log("resolve: ", path.resolve("/srv/app", "public", "./index.html"));
console.log("normalize:", path.normalize("src//utils/../index.js"));
console.log("separator here:", JSON.stringify(path.sep), "| Windows style:",
path.win32.join("C:\\app", "logs", "out.txt"));

// a safe way to serve a user-requested file inside one folder
const publicDir = "/srv/app/public";
function safeResolve(requested) {
  const full = path.resolve(publicDir, "." + path.sep + requested);
  if (!full.startsWith(publicDir + path.sep)) throw new Error(`blocked path traversal: ${requested}`);
  return full;
}
console.log(safeResolve("css/site.css"));
try { safeResolve("../../etc/passwd"); } catch (e) { console.log(e.message); }
```

```
OUTPUT
basename: students.js | without ext: students
dirname:  /home/asha/projects/lbf-api/src/routes
extname:  .js
parse:    {
  root: '/',
  dir: '/home/asha/projects/lbf-api/src/routes',
  base: 'students.js',
  ext: '.js',
  name: 'students'
}
join:     /home/asha/uploads/photo.png
relative: src/routes/students.js
resolve:  /srv/app/public/index.html
normalize: src/index.js
separator here: "/" | Windows style: C:\app\logs\out.txt
/srv/app/public/css/site.css
blocked path traversal: ../../etc/passwd
```

22.2.3 os

The **os** module reports on the machine: `os.platform()`, `os.type()`, `os.cpus()` (one entry per logical core, useful for deciding how many worker processes to start), `os.totalmem()` / `freemem()`, `os.homedir()`, `os.tmpdir()`, `os.hostname()`, `os.uptime()`, `os.networkInterfaces()` and `os.EOL` (the line ending). It is used in CLIs, monitoring and health-check endpoints.

INSPECTING THE MACHINE main.js

```
const os = require("node:os");
const gb = (b) => (b / 1024 ** 3).toFixed(1) + " GB";

console.log("platform / type:", os.platform(), "/", os.type());
console.log("logical CPU cores:", os.cpus().length > 0 ? "at least 1" : 0, "| model reported:", typeof
os.cpus()[0]?.model);
console.log("memory:", Number(gb(os.totalmem()).split(" ")[0]) > 0 ? "total and free memory available
in bytes" : "?");
console.log("temp folder:", os.tmpdir(), "| line ending:", JSON.stringify(os.EOL));

// a typical health-check payload
const health = {
  status: "ok",
  uptimeSeconds: Math.round(process.uptime()),
  loadAverage1m: os.loadavg()[0] >= 0,
  memoryUsedPercent: Math.round((1 - os.freemem() / os.totalmem()) * 100) >= 0,
  node: process.version,
};
console.log(health);
```

OUTPUT

```
platform / type: linux / Linux
logical CPU cores: at least 1 | model reported: string
memory: total and free memory available in bytes
temp folder: /tmp | line ending: "\n"
{
  status: 'ok',
  uptimeSeconds: 0,
  loadAverage1m: true,
  memoryUsedPercent: true,
  node: 'v22.22.2'
}
```

NOTE

The exact numbers (cores, memory, load) depend on the machine, so the example prints them as checks; on your computer, log the raw values with `console.log(os.cpus().length, gb(os.totalmem()))`.

22.2.4 events and EventEmitter

Much of Node.js is built on the **EventEmitter** class from the **events** module (Module 20): HTTP servers emit `request`, streams emit `data` and `end`, processes emit `exit`. Your own classes can extend it to announce what happens. `on` / `addListener` subscribes, `once` subscribes for a single event, `off` removes, `emit(name, ...args)` calls listeners **synchronously** in order, and the special `"error"` event throws if no listener handles it. `events.once(emitter, name)` returns a promise, and `events.on` an async iterator.

A SERVICE THAT EMITS EVENTS main.js

```
const { EventEmitter, once } = require("node:events");

class OrderService extends EventEmitter {
  #nextId = 100;
  place(customer, items) {
    if (!items.length) {
      this.emit("error", new Error(`Order for ${customer} has no items`));
      return;
    }
    const order = { id: ++this.#nextId, customer, items };
    this.emit("placed", order);                       // listeners run synchronously, in order
    return order;
  }
}

const orders = new OrderService();
orders.on("placed", (o) => console.log(`[email]     confirmation to ${o.customer} for order
${o.id}`));
orders.on("placed", (o) => console.log(`[inventory] reserve ${o.items.join(", ")}`));
orders.once("placed", (o) => console.log(`[promo]     first order of the day: ${o.id}`));
orders.on("error", (err) => console.log("[error]    ", err.message));   // without this, emit("error")
would throw

orders.place("Asha", ["pen", "book"]);
orders.place("Ravi", ["bag"]);
orders.place("Meera", []);
console.log("listeners for 'placed':", orders.listenerCount("placed"));

(async () => {
  setTimeout(() => orders.place("John", ["laptop"]), 10);
  const [order] = await once(orders, "placed");        // await the next event
  console.log("awaited event for order", order.id);
})();
```

OUTPUT

```
[email]     confirmation to Asha for order 101
[inventory] reserve pen, book
[promo]     first order of the day: 101
[email]     confirmation to Ravi for order 102
[inventory] reserve bag
[error]     Order for Meera has no items
listeners for 'placed': 2
[email]     confirmation to John for order 103
[inventory] reserve laptop
awaited event for order 103
```

22.2.5 http (building a server from scratch)

The **http** module creates web servers without any framework. `http.createServer((req, res) => ...)` calls your function for every request: `req` (an IncomingMessage, a readable stream) has `method`, `url` and `headers`, and its body arrives in chunks; `res` (a ServerResponse, a writable stream) sets `statusCode` and headers (`setHeader` or `writeHead`) and sends the body with `write`/`end`. `server.listen(port)` starts listening. Doing routing, body parsing and errors by hand quickly gets repetitive, which is exactly what Express solves, but seeing it once shows what every framework does underneath.

SERVER.JS JavaScript

```
const http = require("node:http");
```

```
const courses = [
  { id: 1, title: "JavaScript", level: "beginner" },
  { id: 2, title: "Node.js", level: "advanced" },
];

function sendJson(res, status, data) {
  res.writeHead(status, { "Content-Type": "application/json" });
  res.end(JSON.stringify(data));
}

async function readJsonBody(req) {
  let raw = "";
  for await (const chunk of req) raw += chunk;          // the body arrives as a stream of chunks
  return JSON.parse(raw);
}

const server = http.createServer(async (req, res) => {
  const url = new URL(req.url, `http://${req.headers.host}`);
  console.log(req.method, url.pathname + url.search);
  try {
    if (req.method === "GET" && url.pathname === "/") {
      res.setHeader("Content-Type", "text/plain; charset=utf-8");
      return res.end("Welcome to the LBF API\n");
    }
    if (req.method === "GET" && url.pathname === "/api/courses") {
      const level = url.searchParams.get("level");
      return sendJson(res, 200, level ? courses.filter((c) => c.level === level) : courses);
    }
    if (req.method === "POST" && url.pathname === "/api/courses") {
      const body = await readJsonBody(req);
      const course = { id: courses.length + 1, ...body };
      courses.push(course);
      return sendJson(res, 201, course);
    }
    sendJson(res, 404, { error: "Not found" });
  } catch (err) {
    sendJson(res, 400, { error: "Invalid JSON body" });
  }
});

server.listen(3000, () => console.log("Server listening on http://localhost:3000"));
```

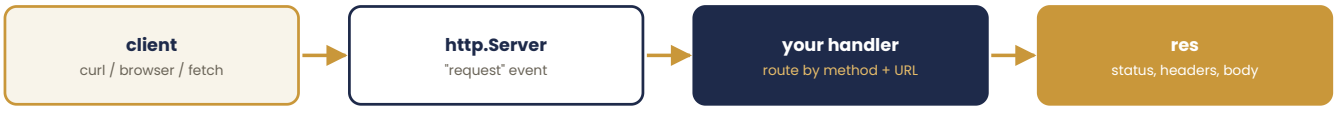
AN HTTP SERVER WITH NO FRAMEWORK

main.js

```
node server.js &
curl -s http://localhost:3000/
curl -s http://localhost:3000/api/courses
curl -s "http://localhost:3000/api/courses?level=advanced"
curl -s -X POST http://localhost:3000/api/courses -H "Content-Type: application/json" -d
'{"title":"TypeScript","level":"intermediate"}'
curl -s -X POST http://localhost:3000/api/courses -H "Content-Type: application/json" -d '{bad json'
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:3000/nothing-here
```

TERMINAL

```
$ node server.js &
Server listening on http://localhost:3000
$ curl -s http://localhost:3000/
Welcome to the LBF API
  [server log] GET /
$ curl -s http://localhost:3000/api/courses
[{"id":1,"title":"JavaScript","level":"beginner"},{"id":2,"title":"Node.js","level":"advanced"}]
  [server log] GET /api/courses
$ curl -s "http://localhost:3000/api/courses?level=advanced"
[{"id":2,"title":"Node.js","level":"advanced"}]
  [server log] GET /api/courses?level=advanced
$ curl -s -X POST http://localhost:3000/api/courses -H "Content-Type: application/json" -d
'{"title":"TypeScript","level":"intermediate"}'
{"id":3,"title":"TypeScript","level":"intermediate"}
  [server log] POST /api/courses
$ curl -s -X POST http://localhost:3000/api/courses -H "Content-Type: application/json" -d '{bad json'
{"error":"Invalid JSON body"}
  [server log] POST /api/courses
$ curl -s -o /dev/null -w "%{http_code}\n" http://localhost:3000/nothing-here
404
  [server log] GET /nothing-here
```



EVERY REQUEST BECOMES A CALL TO YOUR HANDLER

22.2.6 Streams and buffers

A **Buffer** is a fixed-length chunk of raw binary data (a subclass of `Uint8Array`), used for files, network packets, images and encodings: `Buffer.from("text")`, `buf.toString("base64")`, `Buffer.concat`. **Streams** process data **piece by piece** as it arrives instead of loading everything into memory: a 5 GB log file can be processed with a few kilobytes of memory, and the first bytes of a response can be sent before the last are read. There are four kinds: **Readable** (file reads, HTTP requests), **Writable** (file writes, HTTP responses), **Duplex** (sockets) and **Transform** (compression, encryption, parsing). Connect them with `pipeline()` from `node:stream/promises`, which also handles errors and back-pressure (slowing a fast producer down for a slow consumer). Readable streams are async iterables.

BUFFERS: RAW BYTES AND ENCODINGS

main.js

```
const buf = Buffer.from("Namaste 🙏", "utf8");
console.log("buffer:", buf, "| bytes:", buf.length, "| characters:", [..."Namaste 🙏"].length);
console.log("hex:", buf.toString("hex").slice(0, 14), "... | base64:", buf.toString("base64"));
console.log("back to text:", Buffer.from(buf.toString("base64"), "base64").toString("utf8"));
console.log("first 7 bytes as text:", buf.subarray(0, 7).toString());
```

OUTPUT

```
buffer: <Buffer 4e 61 6d 61 73 74 65 20 f0 9f 99 8f> | bytes: 12 | characters: 9
hex: 4e616d61737465 ... | base64: TmFtYXN0ZSDwn5mP
back to text: Namaste 🙏
first 7 bytes as text: Namaste
```

STREAM-REPORT.MJS

JavaScript

```
import { createReadStream, createWriteStream } from "node:fs";
import { createInterface } from "node:readline";
import { pipeline } from "node:stream/promises";
import { Transform } from "node:stream";
import { createGzip } from "node:zlib";

// 1. Read line by line: memory stays small however large the file is
const counts = {};
let lines = 0;
for await (const line of createInterface({ input: createReadStream("access.log") })) {
  const status = line.split(" ")[3];
  counts[status] = (counts[status] ?? 0) + 1;
  lines++;
}
console.log(`${lines} lines, status counts:`, counts);
console.log("heap used stays small:", process.memoryUsage().heapUsed < 60 * 1024 * 1024);

// 2. A pipeline: read -> keep only 500 errors -> gzip -> write
const onlyErrors = new Transform({
  transform(chunk, _enc, done) {
    const text = (this.rest ?? "") + chunk.toString();
    const parts = text.split("\n");
    this.rest = parts.pop();                              // keep a partial last line for the next
chunk
    done(null, parts.filter((l) => l.includes(" 500 ")).map((l) => l + "\n").join(""));
  },
});
await pipeline(createReadStream("access.log"), onlyErrors, createGzip(),
createWriteStream("errors.log.gz"));
console.log("errors.log.gz written by a streaming pipeline");
```

PROCESSING A LARGE FILE WITH STREAMS main.js

```
node make-log.js
ls -l access.log | awk '{print $5, $9}'
node stream-report.mjs
ls -l errors.log.gz | awk '{print $5, $9}'
```

TERMINAL

```
$ node make-log.js
access.log written
$ ls -l access.log | awk '{print $5, $9}'
8386640 access.log
$ node stream-report.mjs
200000 lines, status counts: { '200': 168000, '404': 28000, '500': 4000 }
heap used stays small: true
errors.log.gz written by a streaming pipeline
$ ls -l errors.log.gz | awk '{print $5, $9}'
654 errors.log.gz
```

Stream type	Examples	Key API
Readable	fs.createReadStream, HTTP request (server), process.stdin	for await…of, on("data"), pipe()
Writable	fs.createWriteStream, HTTP response, process.stdout	write(), end(), "drain" event
Duplex	TCP sockets, WebSocket connections	both readable and writable
Transform	zlib.createGzip, crypto ciphers, CSV parsers	transform(chunk, enc, callback)

22.2.7 Built-in test runner and watch mode (overview)

Modern Node.js includes tools that once required packages. The **test runner** (`node:test` with `node:assert`) provides `test` / `describe` / `it`, hooks (`before`, `afterEach`…), mocking (`mock.fn`, `mock.timers`), and runs with `node --test`, which finds `*.test.js` files automatically (Module 24 covers testing in depth, including Jest and Vitest). **Watch mode**, `node --watch server.js`, restarts the program whenever an imported file changes, replacing nodemon in many projects; `node --test --watch` re-runs tests on change.

GRADES.JS JavaScript

```
function grade(marks) {
  if (!Number.isFinite(marks) || marks < 0 || marks > 100) throw new RangeError(`invalid marks:
${marks}`);
  if (marks >= 90) return "A+";
  if (marks >= 75) return "A";
  if (marks >= 60) return "B";
  return marks >= 40 ? "C" : "F";
}
module.exports = { grade };
```

GRADES.TEST.JS JavaScript

```
const { describe, it } = require("node:test");
const assert = require("node:assert/strict");
const { grade } = require("./grades.js");

describe("grade()", () => {
  it("gives A+ from 90", () => assert.equal(grade(90), "A+"));
  it("handles boundaries", () => {
    assert.equal(grade(75), "A");
    assert.equal(grade(74.9), "B");
    assert.equal(grade(40), "C");
    assert.equal(grade(39), "F");
  });
  it("rejects invalid input", () => {
    assert.throws(() => grade(101), RangeError);
    assert.throws(() => grade(NaN), /invalid marks/);
  });
});
```

TESTING A MODULE WITH NODE:TEST main.js

```
node --test --test-reporter=spec
```

TERMINAL

```
$ node --test --test-reporter=spec
▶ grade()
  ✔ gives A+ from 90 (4.00865ms)
  ✔ handles boundaries (0.250218ms)
  ✔ rejects invalid input (0.478107ms)
✔ grade() (5.834222ms)
ℹ tests 3
ℹ suites 1
ℹ pass 3
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 353.745359
```

WATCH MODE RESTARTS ON EVERY SAVE main.js

```
$ node --watch server.js
Server listening on http://localhost:3000
Restarting 'server.js'
Server listening on http://localhost:3000
```

NOTE

"Restarting" appears each time a watched file is saved.

22.3 Express.js

Express is the most widely used Node.js web framework: minimal, unopinionated, and built around two ideas, **routing** (mapping a method and URL to a handler) and **middleware** (functions that process requests in a chain). It removes the boilerplate of the raw http module: URL and query parsing, JSON bodies, response helpers, static files and error handling. Express 5 (current) handles errors thrown in

async handlers automatically. Many other frameworks follow the same model (Fastify, Koa, Hono) or build on it (NestJS).

22.3.1 Setting up an Express app

CREATING THE PROJECT main.js

```
mkdir lbf-api && cd lbf-api
npm init -y
npm pkg set type=module scripts.dev="node --watch --env-file=.env src/server.js" scripts.start="node src/server.js"
npm install express
npm run dev
```

TYPICAL OUTPUT

```
added 66 packages, and audited 67 packages in 3s
found 0 vulnerabilities

> lbf-api@1.0.0 dev
> node --watch --env-file=.env src/server.js

API running on http://localhost:3000
```

A MINIMAL EXPRESS APPLICATION main.js

```
// src/server.js
import express from "express";

const app = express();
app.use(express.json());                        // parse JSON request bodies

app.get("/", (req, res) => {
  res.send("Welcome to the LBF API");
});

app.get("/api/health", (req, res) => {
  res.json({ status: "ok", time: new Date().toISOString() });
});

const PORT = process.env.PORT ?? 3000;
app.listen(PORT, () => console.log(`API running on http://localhost:${PORT}`));
```

REQUESTS

```
$ curl http://localhost:3000/
Welcome to the LBF API
$ curl http://localhost:3000/api/health
{"status":"ok","time":"2026-03-09T04:30:00.000Z"}
```

22.3.2 Routing and route parameters

Routes are defined with methods named after HTTP verbs: `app.get`, `post`, `put`, `patch`, `delete` and `all`. Paths can contain **route parameters** (`/students/:id`, available in `req.params.id`, always as strings), optional segments and wildcards. Routes are matched in the order they are defined, so specific routes must come before general ones. `app.route(path)` chains several methods for one path, and `express.Router()` creates a mini-application for a group of routes that can be mounted under a prefix with `app.use("/api/students", router)`.

ROUTES, PARAMETERS AND ROUTERS

main.js

```
import express from "express";
const app = express();

app.get("/api/courses", (req, res) => res.json({ route: "list courses" }));
app.get("/api/courses/:id", (req, res) => res.json({ route: "one course", id: req.params.id, type:
typeof req.params.id }));
app.get("/api/courses/:courseId/batches/:batchId", (req, res) => res.json(req.params));

app.route("/api/notes")                                    // several methods, one path
  .get((req, res) => res.json({ route: "list notes" }))
  .post((req, res) => res.status(201).json({ route: "create note" }));

const students = express.Router();                         // a group of routes
students.get("/", (req, res) => res.json({ route: "list students" }));
students.get("/:id", (req, res) => res.json({ route: "student", id: Number(req.params.id) }));
app.use("/api/students", students);                        // mounted under a prefix

app.listen(3000);
```

REQUESTS

```
$ curl localhost:3000/api/courses/7
{"route":"one course","id":"7","type":"string"}
$ curl localhost:3000/api/courses/7/batches/2026-mar
{"courseId":"7","batchId":"2026-mar"}
$ curl -X POST localhost:3000/api/notes
{"route":"create note"}
$ curl localhost:3000/api/students/42
{"route":"student","id":42}
```

22.3.3 Request and response objects

Express extends Node's request and response objects with conveniences. On `req`: `params` (route parameters), `query` (parsed query string), `body` (parsed by body middleware such as `express.json()`), `headers` and `get(name)`, `method`, `path`, `ip`, `cookies` (with cookie-parser). On `res`: `status(code)` (chainable), `json(data)`, `send(text)`, `sendStatus(code)`, `set(header, value)`, `cookie()`, `redirect(url)`, `sendFile(path)` and `download(path)`. Send exactly one response per request: forgetting to respond leaves the client waiting, and responding twice throws "Cannot set headers after they are sent".

READING THE REQUEST AND BUILDING THE RESPONSE

main.js

```
app.post("/api/search/:category", (req, res) => {
  const { category } = req.params;                 // from the URL path
  const { page = "1", sort = "name" } = req.query; // from ?page=2&sort=fee
  const { keywords = [] } = req.body;              // from the JSON body
  const client = req.get("User-Agent");            // a header

  res
    .status(200)
    .set("X-Total-Count", "42")
    .json({ category, page: Number(page), sort, keywords, client });
});

app.get("/old-courses", (req, res) => res.redirect(301, "/api/courses"));
app.delete("/api/cache", (req, res) => res.sendStatus(204));
```

REQUESTS

```
$ curl -s -i -X POST "localhost:3000/api/search/web?page=2&sort=fee" \
       -H "Content-Type: application/json" -H "User-Agent: curl" -d '{"keywords":["js","react"]}'
HTTP/1.1 200 OK
X-Total-Count: 42
Content-Type: application/json; charset=utf-8

{"category":"web","page":2,"sort":"fee","keywords":["js","react"],"client":"curl"}
```

22.3.4 Middleware (built-in, third-party, custom)

A **middleware** is a function `(req, res, next)` that runs during the request-response cycle. It can read or modify `req` and `res`, end the request by sending a response, or call `next()` to pass control to the next middleware or route. Middleware registered with `app.use()` runs for every request (or for every path under a prefix) in the order it was added; middleware can also be attached to individual routes. **Built-in**: `express.json()`, `express.urlencoded()`, `express.static()`. **Third-party**: cors, helmet, morgan (request logging), cookie-parser, express-rate-limit, multer (file uploads). **Custom**: anything you write, such as logging, authentication or adding a request id.



A REQUEST FLOWS THROUGH MIDDLEWARE IN ORDER UNTIL ONE SENDS THE RESPONSE

CUSTOM, BUILT-IN AND THIRD-PARTY MIDDLEWARE

main.js

```
import express from "express";
import morgan from "morgan";
const app = express();

// custom middleware: request id and timing
app.use((req, res, next) => {
  req.id = crypto.randomUUID().slice(0, 8);
  const start = performance.now();
  res.on("finish", () => console.log(`[${req.id}] ${req.method} ${req.originalUrl} ->
${res.statusCode} in ${(performance.now() - start).toFixed(1)} ms`));
  next();                                          // pass control on
});

app.use(express.json({ limit: "100kb" }));         // built-in
app.use(morgan("tiny"));                           // third-party logger

// middleware for one route only: a simple API-key check
function requireApiKey(req, res, next) {
  if (req.get("X-API-Key") !== process.env.API_KEY) return res.status(401).json({ error: "Missing or
invalid API key" });
  next();
}

app.get("/api/public", (req, res) => res.json({ ok: true, requestId: req.id }));
app.get("/api/reports", requireApiKey, (req, res) => res.json({ report: "confidential" }));
```

SERVER LOG

```
[a3f9c2d1] GET /api/public -> 200 in 1.2 ms
GET /api/public 200 35 - 1.412 ms
[5be01f77] GET /api/reports -> 401 in 0.4 ms
GET /api/reports 401 39 - 0.517 ms
```

22.3.5 Serving static files

`app.use(express.static("public"))` serves the files in a folder (HTML, CSS, JavaScript, images) at matching URLs: `public/css/site.css` becomes `/css/site.css`, and `public/index.html` is served for `/`. Options set caching headers (`maxAge`) and other behaviour, and a prefix can be added (`app.use("/assets", express.static(...))`). This lets one Express server deliver both a front end (for example a Vite build in `dist/`) and its API. For a single-page app, add a final catch-all route that returns `index.html` for unknown non-API paths so client-side routing works.

SERVING A FRONT END AND UPLOADS

main.js

```
import express from "express";
import path from "node:path";
const app = express();
const publicDir = path.join(import.meta.dirname, "..", "public");

app.use(express.static(publicDir, { maxAge: "1d" }));           // / -> public/index.html
app.use("/uploads", express.static(path.join(import.meta.dirname, "..", "uploads")));
app.use("/api", apiRouter);

app.get(/^(?!\/api).*/, (req, res) => res.sendFile(path.join(publicDir, "index.html")));   // SPA
fallback
```

```
REQUESTS
GET /                 -> public/index.html (200, Cache-Control: public, max-age=86400)
GET /css/site.css     -> public/css/site.css (200)
GET /uploads/a.png    -> uploads/a.png (200)
GET /dashboard/stats  -> public/index.html (SPA route handled in the browser)
```

22.3.6 Error-handling middleware

An **error-handling middleware** has **four** parameters, `(err, req, res, next)`, and is registered **last**, after all routes. Express sends control there whenever a handler throws, an async handler rejects (automatic in Express 5; in Express 4 call `next(err)`), or code calls `next(err)`. It centralises error responses: convert known errors (validation, not found, unauthorised) to the right status codes, log unexpected ones, and never leak stack traces to clients in production. A 404 handler, a normal middleware placed after the routes, catches requests that matched nothing.

CENTRAL ERROR HANDLING

main.js

```
class HttpError extends Error {
  constructor(status, message, details) { super(message); this.status = status; this.details =
details; }
}

app.get("/api/students/:id", async (req, res) => {
  const student = await Student.findById(req.params.id);       // may throw
  if (!student) throw new HttpError(404, `Student ${req.params.id} not found`);
  res.json(student);
});

// 404 for anything not matched above
app.use((req, res, next) => next(new HttpError(404, `No route for ${req.method}
${req.originalUrl}`)));

// the error handler: four parameters, registered last
app.use((err, req, res, next) => {
  const status = err.status ?? 500;
  if (status >= 500) console.error(err);                          // log real failures with their stack
  res.status(status).json({
    error: status >= 500 && process.env.NODE_ENV === "production" ? "Internal server error" :
err.message,
    ...(err.details && { details: err.details }),
  });
});
```

REQUESTS

```
$ curl localhost:3000/api/students/999
{"error":"Student 999 not found"}
$ curl localhost:3000/api/unknown
{"error":"No route for GET /api/unknown"}
(database down, production)  ->  500 {"error":"Internal server error"}
```

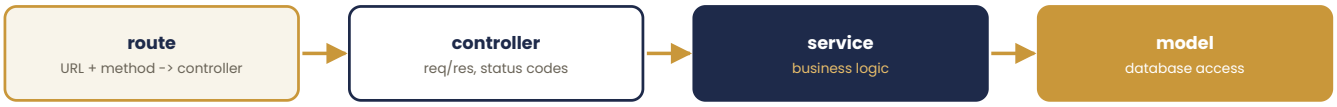
22.3.7 Structuring a project (routes, controllers, services)

Putting everything in one file works for a demo but not for a real API. A common **layered structure** separates responsibilities: **routes** map URLs and methods to controller functions (and attach middleware); **controllers** handle HTTP concerns: read the request, call a service, send the response; **services** contain the business logic and know nothing about HTTP; **models** or a data-access layer talk to the database; **middleware** holds cross-cutting code (auth, validation, errors); and **config** reads environment variables in one place. Keeping `app.js` (builds the app) separate from `server.js` (starts listening) lets tests import the app without opening a port.

A LAYERED EXPRESS PROJECT

text

```
lbf-api/
├── package.json
├── .env                    (not committed)    .env.example (committed)
└── src/
    ├── server.js           starts the HTTP server: app.listen(config.port)
    ├── app.js              creates the Express app, middleware, routers, error handler
    ├── config/index.js     reads and validates environment variables
    ├── routes/
    │   ├── auth.routes.js      POST /login, POST /register
    │   └── student.routes.js   GET/POST /students, GET/PUT/DELETE /students/:id
    ├── controllers/
    │   └── student.controller.js   req/res handling only
    ├── services/
    │   └── student.service.js      business rules, no HTTP
    ├── models/
    │   └── student.model.js        Mongoose schema / SQL queries
    ├── middleware/
    │   ├── auth.js  validate.js  error.js
    └── utils/HttpError.js
```



EACH LAYER HAS ONE JOB

22.4 Building a REST API

22.4.1 CRUD endpoints

A REST API for a resource offers the four **CRUD** operations (Create, Read, Update, Delete) on two URL patterns: the collection (`/api/students`) and an individual item (`/api/students/:id`), with the status codes from Module 18. Lists usually support filtering, sorting and **pagination** through query parameters, returning metadata such as the total count. The example below is a complete, runnable CRUD API for students written with only Node.js built-ins (the http module and the built-in SQLite database), so every request can be shown really running; the Express version of the same API, with validation and authentication, is the practice checkpoint at the end of the module.

A COMPLETE CRUD API WITH NODE.JS AND SQLITE

main.js

```
node --no-warnings api.mjs &
curl -s -X POST localhost:3000/api/students -H "Content-Type: application/json" -d '{"name":"Asha
Rao","email":"asha@lbf.in","course":"JavaScript","marks":91}'
curl -s -X POST localhost:3000/api/students -H "Content-Type: application/json" -d '{"name":"Ravi
Kumar","email":"ravi@lbf.in","course":"Python","marks":64}'
curl -s -X POST localhost:3000/api/students -H "Content-Type: application/json" -d '{"name":"Meera
N","email":"meera@lbf.in","course":"JavaScript","marks":85}'
curl -s "localhost:3000/api/students?course=JavaScript&sort=-marks"
curl -s -X PATCH localhost:3000/api/students/2 -H "Content-Type: application/json" -d '{"marks":72}'
curl -s -X DELETE -o /dev/null -w "%{http_code}\n" localhost:3000/api/students/3
curl -s localhost:3000/api/students/3
curl -s "localhost:3000/api/students?page=1&limit=1"
```

TERMINAL

```
$ node --no-warnings api.mjs &
Student API on http://localhost:3000
$ curl -s -X POST localhost:3000/api/students -H "Content-Type: application/json" -d '{"name":"Asha
Rao","email":"asha@lbf.in","course":"JavaScript","marks":91}'
{"id":1,"name":"Asha Rao","email":"asha@lbf.in","course":"JavaScript","marks":91}
  [server log] POST /api/students
$ curl -s -X POST localhost:3000/api/students -H "Content-Type: application/json" -d '{"name":"Ravi
Kumar","email":"ravi@lbf.in","course":"Python","marks":64}'
{"id":2,"name":"Ravi Kumar","email":"ravi@lbf.in","course":"Python","marks":64}
  [server log] POST /api/students
$ curl -s -X POST localhost:3000/api/students -H "Content-Type: application/json" -d '{"name":"Meera
N","email":"meera@lbf.in","course":"JavaScript","marks":85}'
{"id":3,"name":"Meera N","email":"meera@lbf.in","course":"JavaScript","marks":85}
  [server log] POST /api/students
$ curl -s "localhost:3000/api/students?course=JavaScript&sort=-marks"
{"total":2,"page":1,"data":[{"id":1,"name":"Asha
Rao","email":"asha@lbf.in","course":"JavaScript","marks":91},{"id":3,"name":"Meera
N","email":"meera@lbf.in","course":"JavaScript","marks":85}]}
  [server log] GET /api/students?course=JavaScript&sort=-marks
$ curl -s -X PATCH localhost:3000/api/students/2 -H "Content-Type: application/json" -d '{"marks":72}'
{"id":2,"name":"Ravi Kumar","email":"ravi@lbf.in","course":"Python","marks":72}
  [server log] PATCH /api/students/2
$ curl -s -X DELETE -o /dev/null -w "%{http_code}\n" localhost:3000/api/students/3
204
  [server log] DELETE /api/students/3
$ curl -s localhost:3000/api/students/3
{"error":"Student 3 not found"}
  [server log] GET /api/students/3
$ curl -s "localhost:3000/api/students?page=1&limit=1"
{"total":2,"page":1,"data":[{"id":1,"name":"Asha
Rao","email":"asha@lbf.in","course":"JavaScript","marks":91}]}
  [server log] GET /api/students?page=1&limit=1
```

Operation	Request	Success	Errors
List (filter, sort, page)	`GET /api/students?course=JS&sort=-marks&page=2`	200 + { total, page, data }	400 invalid query
Read one	`GET /api/students/7`	200 + student	404
Create	`POST /api/students` + JSON	201 + created student	400 invalid, 409 duplicate email
Update	`PUT` (all fields) / `PATCH` (some) `/api/students/7`	200 + updated student	400, 404

Operation	Request	Success	Errors
Delete	`DELETE /api/students/7`	204 (no body)	404

WATCH OUT

Never build SQL by concatenating user input (`"... WHERE name = '" + name + "'"`): that allows **SQL injection**. Use parameter placeholders (`?`) as above, and whitelist anything that cannot be a parameter, such as the column used for sorting.

22.4.2 Validation

Every request body, parameter and query string comes from outside and must be **validated** on the server, whatever the front end already checked (Module 14): required fields, types, formats, ranges and lengths, plus stripping unknown fields so that clients cannot set things like `role: "admin"`. Respond with **400** (or 422) and a list of clear, per-field messages. Hand-written checks work for small APIs; schema libraries such as **Zod**, Joi, Yup or express-validator describe the rules declaratively, validate and convert types, and (with Zod) also produce TypeScript types. Validation fits naturally into a reusable middleware.

A REUSABLE VALIDATION MIDDLEWARE

main.js

```
// A small schema validator and Express-style middleware (no libraries)
const rules = {
  required: (v) => v !== undefined && v !== null && v !== "" || "is required",
  string: (v) => v === undefined || typeof v === "string" || "must be a string",
  email: (v) => v === undefined || /^[^\s@]+@[^\s@]+\.[a-z]{2,}$/i.test(v) || "must be a valid email",
  int: (v) => v === undefined || Number.isInteger(v) || "must be a whole number",
  range: (min, max) => (v) => v === undefined || (v >= min && v <= max) || `must be between ${min} and
${max}`,
  oneOf: (...opts) => (v) => v === undefined || opts.includes(v) || `must be one of: ${opts.join(",
")}`,
  minLength: (n) => (v) => v === undefined || String(v).length >= n || `must have at least ${n}
characters`,
};
const studentSchema = {
  name: [rules.required, rules.string, rules.minLength(2)],
  email: [rules.required, rules.email],
  course: [rules.required, rules.oneOf("JavaScript", "Python", "Java")],
  marks: [rules.int, rules.range(0, 100)],
};

function validate(schema) {
  return (req, res, next) => {
    const errors = {};
    const clean = {};
    for (const [field, checks] of Object.entries(schema)) {
      for (const check of checks) {
        const result = check(req.body[field]);
        if (result !== true) { errors[field] = `${field} ${result}`; break; }
      }
      if (req.body[field] !== undefined) clean[field] = req.body[field];
    }
    if (Object.keys(errors).length) return res.status(400).json({ error: "Validation failed", details:
errors });
    req.body = clean;                                   // unknown fields (like role) are dropped
    next();
  };
}

// simulate Express calling the middleware
const fakeRes = () => ({ status(c) { this.code = c; return this; }, json(d) { console.log(this.code,
JSON.stringify(d, null, 1).replace(/\n\s*/g, " ")); } });
const mw = validate(studentSchema);
const req = { body: { name: "Asha Rao", email: "asha@lbf.in", course: "JavaScript", marks: 91, role:
"admin" } };
mw(req, fakeRes(), () => console.log("valid -> handler receives:", JSON.stringify(req.body)));
mw({ body: { name: "A", email: "asha@", course: "COBOL", marks: 140 } }, fakeRes(), () => {});
```

OUTPUT

```
valid -> handler receives: {"name":"Asha Rao","email":"asha@lbf.in","course":"JavaScript","marks":91}
400 { "error": "Validation failed", "details": { "name": "name must have at least 2 characters",
"email": "email must be a valid email", "course": "course must be one of: JavaScript, Python, Java",
"marks": "marks must be between 0 and 100" } }
```

THE SAME WITH ZOD

main.js

```
import { z } from "zod";

const StudentInput = z.object({
  name: z.string().trim().min(2),
  email: z.string().email().toLowerCase(),
  course: z.enum(["JavaScript", "Python", "Java"]),
  marks: z.coerce.number().int().min(0).max(100).optional(),
});

const validateBody = (schema) => (req, res, next) => {
  const result = schema.safeParse(req.body);
  if (!result.success) return res.status(400).json({ error: "Validation failed", details:
result.error.flatten().fieldErrors });
  req.body = result.data;                    // parsed, converted and stripped of unknown keys
  next();
};

router.post("/", validateBody(StudentInput), createStudent);
```

RESPONSE TO AN INVALID BODY

```
400 {"error":"Validation failed","details":{"name":["String must contain at least 2 character(s)"],
     "email":["Invalid email"],"course":["Invalid enum value. Expected 'JavaScript' | 'Python' |
'Java', received 'COBOL'"],
     "marks":["Number must be less than or equal to 100"]}}
```

22.4.3 Environment variables with .env

Configuration that varies between development, testing and production, and every secret (database URLs, JWT secrets, API keys), comes from **environment variables** (Module 19): a `.env` file locally (never committed; commit `.env.example`), and the hosting platform's settings in production. Load them with `node --env-file=.env` or the dotenv package, read and **validate them once** at start-up in a config module, convert types there, and **fail fast** with a clear message if something required is missing, rather than crashing later on the first request.

.ENV

text

```
PORT=4000
NODE_ENV=development
DATABASE_URL=mongodb://localhost:27017/lbf
JWT_SECRET=change-me-to-a-long-random-string
JWT_EXPIRES_IN=1h
CORS_ORIGIN=http://localhost:5173
```

CONFIG.JS

JavaScript

```
const required = ["DATABASE_URL", "JWT_SECRET"];
const missing = required.filter((name) => !process.env[name]);
if (missing.length) {
  console.error(`Missing required environment variables: ${missing.join(", ")}`);
  process.exit(1);                                    // fail fast at start-up
}

module.exports = Object.freeze({
  port: Number(process.env.PORT ?? 3000),
  env: process.env.NODE_ENV ?? "development",
  databaseUrl: process.env.DATABASE_URL,
  jwt: { secret: process.env.JWT_SECRET, expiresIn: process.env.JWT_EXPIRES_IN ?? "1h" },
  corsOrigin: (process.env.CORS_ORIGIN ?? "").split(",").filter(Boolean),
});
```

A CONFIG MODULE THAT FAILS FAST

main.js

```
node --env-file=.env server.js
JWT_SECRET= node --env-file=.env server.js
```

TERMINAL

```
$ node --env-file=.env server.js
starting in development on port 4000
CORS origins: [ 'http://localhost:5173' ] | JWT expires in 1h
$ JWT_SECRET= node --env-file=.env server.js
Missing required environment variables: JWT_SECRET
```

22.4.4 Testing APIs with Postman or Thunder Client

While building an API you need to send requests with any method, headers and body and inspect the responses. **curl** works everywhere from the terminal (as used throughout this module). **Postman** is a desktop application for building and saving requests in **collections**, with **environments** (variables such as `{{baseUrl}}` and `{{token}}`), scripts that save a token from the login response into a variable, automated test assertions, and documentation sharing. **Thunder Client** and the **REST Client** extension (`.http` files) do the same inside VS Code, and Bruno and Insomnia are open-source alternatives. Automated tests with Supertest (Module 24) then lock the behaviour in.

A SAVED COLLECTION OF REQUESTS

text

```
### requests.http (VS Code REST Client) - click "Send Request" above each block
@baseUrl = http://localhost:3000/api
@token = eyJhbGciOiJIUzI1NiIs...

POST {{baseUrl}}/auth/login
Content-Type: application/json

{ "email": "admin@lbf.in", "password": "S3cure!pass" }

###
GET {{baseUrl}}/students?course=JavaScript&page=1
Authorization: Bearer {{token}}

###
POST {{baseUrl}}/students
Authorization: Bearer {{token}}
Content-Type: application/json

{ "name": "Zoya Khan", "email": "zoya@lbf.in", "course": "Java", "marks": 88 }
```

Tool	Where	Strengths
curl	any terminal	scriptable, universal, great for docs and CI
Postman	desktop / web app	collections, environments, test scripts, team sharing, mock servers
Thunder Client	VS Code extension	lightweight Postman-like UI next to your code
REST Client (.http files)	VS Code extension	requests stored as plain text in the repository
Bruno / Insomnia	desktop	open-source, Git-friendly collections

22.5 Databases

Data must survive server restarts and be shared between server instances, so APIs store it in a database. The two main families are **document databases** such as **MongoDB**, which store flexible JSON-like documents (natural for JavaScript objects and fast to start with), and **relational (SQL) databases** such as **PostgreSQL** and **MySQL**, which store rows in tables with a fixed schema, relationships enforced by foreign keys, powerful joins and strong transactional guarantees. JavaScript code talks to them through drivers, or through an **ODM** (Object-Document Mapper, such as Mongoose) or **ORM** (Object-Relational Mapper, such as Prisma, Sequelize, Drizzle or TypeORM).

	MongoDB (document)	PostgreSQL / MySQL (relational)
Data model	collections of JSON-like documents	tables of rows and columns
Schema	flexible (enforced by Mongoose or validators)	fixed, defined with CREATE TABLE / migrations
Relationships	embedding or references (populate)	foreign keys and JOINs
Query language	MongoDB query API (find, aggregate)	SQL
Transactions	supported (multi-document since 4.0)	core strength (ACID)
Typical JS tools	mongodb driver, Mongoose	pg, mysql2, Prisma, Sequelize, Drizzle

	MongoDB (document)	PostgreSQL / MySQL (relational)
Good for	rapidly changing data, content, catalogs, logs	financial data, complex relations and reporting

22.5.1 MongoDB with Mongoose

MongoDB stores **documents** (BSON, a binary JSON with extra types such as ObjectId and Date) in **collections** inside a database; MongoDB Atlas provides a free cloud cluster. **Mongoose** adds structure: a **schema** defines fields, types, defaults and validation rules; a **model** compiled from it provides methods to create, query, update and delete documents (`create`, `find`, `findById`, `findByIdAndUpdate`, `deleteOne`), plus query helpers (`sort`, `limit`, `skip`, `select`), `populate` for references, middleware hooks (e.g. hash a password before saving) and timestamps.

A MONGOOSE SCHEMA, MODEL AND QUERIES main.js

```
// src/models/student.model.js
import mongoose from "mongoose";

const studentSchema = new mongoose.Schema(
  {
    name: { type: String, required: [true, "Name is required"], trim: true, minlength: 2 },
    email: { type: String, required: true, unique: true, lowercase: true, match: [/^\S+@\S+\.\S+$/,
"Invalid email"] },
    course: { type: String, enum: ["JavaScript", "Python", "Java"], required: true },
    marks: { type: Number, min: 0, max: 100, default: 0 },
    batch: { type: mongoose.Schema.Types.ObjectId, ref: "Batch" },   // reference to another
collection
  },
  { timestamps: true },                                          // createdAt, updatedAt
);
studentSchema.virtual("grade").get(function () { return this.marks >= 75 ? "A" : this.marks >= 60 ?
"B" : "C"; });

export const Student = mongoose.model("Student", studentSchema);

// usage (e.g. in a service)
await mongoose.connect(process.env.DATABASE_URL);
const asha = await Student.create({ name: "Asha Rao", email: "ASHA@lbf.in", course: "JavaScript",
marks: 91 });
const toppers = await Student.find({ course: "JavaScript", marks: { $gte: 80 } }).sort({ marks: -1
}).limit(10).select("name marks");
const updated = await Student.findByIdAndUpdate(asha._id, { $inc: { marks: 2 } }, { new: true,
runValidators: true });
await Student.deleteOne({ _id: asha._id });
const stats = await Student.aggregate([{ $group: { _id: "$course", avg: { $avg: "$marks" }, count: {
$sum: 1 } } }]);
```

TYPICAL RESULTS

```
asha     -> { _id: new ObjectId("65f0c1..."), name: 'Asha Rao', email: 'asha@lbf.in', course:
'JavaScript',
             marks: 91, createdAt: 2026-03-09T04:30:00.000Z, updatedAt: 2026-03-09T04:30:00.000Z }
toppers  -> [ { _id: ..., name: 'Asha Rao', marks: 91 }, { _id: ..., name: 'Meera N', marks: 85 } ]
updated  -> marks: 93
stats    -> [ { _id: 'JavaScript', avg: 88, count: 2 }, { _id: 'Python', avg: 64, count: 1 } ]
invalid  -> ValidationError: Student validation failed: course: `COBOL` is not a valid enum value
```

22.5.2 SQL databases (MySQL or PostgreSQL) with an ORM such as Prisma or Sequelize (overview)

With a relational database you define tables and relationships, and query with SQL. You can use a driver directly (`pg` for PostgreSQL, `mysql2` for MySQL) with parameterised queries, as in the SQLite example below, which runs with Node.js's built-in `node:sqlite` module and shows the same ideas: a schema, a foreign key, a JOIN and a transaction. An **ORM** maps tables to JavaScript objects and generates SQL for you. **Prisma** describes models in a `schema.prisma` file, generates **migrations** (versioned schema changes) and a fully typed client; **Sequelize** defines models in JavaScript with methods such as `findAll` and associations; **Drizzle** is a lightweight, SQL-like TypeScript ORM.

SQL-DEMO.MJS

JavaScript

```
import { DatabaseSync } from "node:sqlite";
const db = new DatabaseSync(":memory:");
db.exec(`
  PRAGMA foreign_keys = ON;
  CREATE TABLE courses  (id INTEGER PRIMARY KEY, title TEXT NOT NULL UNIQUE, fee INTEGER NOT NULL);
  CREATE TABLE students (id INTEGER PRIMARY KEY, name TEXT NOT NULL, course_id INTEGER REFERENCES
courses(id), paid INTEGER DEFAULT 0);
`);
const addCourse = db.prepare("INSERT INTO courses (title, fee) VALUES (?, ?)");
const addStudent = db.prepare("INSERT INTO students (name, course_id) VALUES (?, ?)");
[["JavaScript", 14999], ["Python", 12999]].forEach(([t, f]) => addCourse.run(t, f));
[["Asha", 1], ["Ravi", 2], ["Meera", 1]].forEach(([n, c]) => addStudent.run(n, c));

console.log("JOIN: students with their course");
for (const row of db.prepare(`SELECT s.name, c.title, c.fee FROM students s JOIN courses c ON c.id =
s.course_id ORDER BY s.name`).all())
  console.log(`  ${row.name.padEnd(6)} ${row.title.padEnd(11)} Rs ${row.fee}`);

console.log("GROUP BY:", db.prepare("SELECT c.title, COUNT(s.id) AS students FROM courses c LEFT JOIN
students s ON s.course_id = c.id GROUP BY c.id").all().map((r) => `${r.title}=${r.students}`).join(",
"));

try { addStudent.run("Ghost", 99); } catch (e) { console.log("foreign key protects data:", e.message);
}

// a transaction: all statements succeed, or none do
function recordPayment(studentId, amount) {
  db.exec("BEGIN");
  try {
    db.prepare("UPDATE students SET paid = paid + ? WHERE id = ?").run(amount, studentId);
    if (amount <= 0) throw new Error("invalid amount");
    db.exec("COMMIT");
  } catch (e) {
    db.exec("ROLLBACK");
    console.log("rolled back:", e.message);
  }
}
recordPayment(1, 5000);
recordPayment(2, -100);
console.log("paid:", db.prepare("SELECT name, paid FROM students ORDER BY id").all().map((r) =>
`${r.name}=${r.paid}`).join(", "));
```

TABLES, A JOIN AND A TRANSACTION IN SQL

main.js

```
node --no-warnings sql-demo.mjs
```

TERMINAL

```
$ node --no-warnings sql-demo.mjs
JOIN: students with their course
  Asha   JavaScript  Rs 14999
  Meera  JavaScript  Rs 14999
  Ravi   Python      Rs 12999
GROUP BY: JavaScript=2, Python=1
foreign key protects data: FOREIGN KEY constraint failed
rolled back: invalid amount
paid: Asha=5000, Ravi=0, Meera=0
```

A PRISMA SCHEMA

text

```
// prisma/schema.prisma
datasource db {
  provider = "postgresql"            // or "mysql", "sqlite"
  url      = env("DATABASE_URL")
}
generator client { provider = "prisma-client-js" }

model Course {
  id       Int       @id @default(autoincrement())
  title    String    @unique
  fee      Int
  students Student[]
}

model Student {
  id        Int      @id @default(autoincrement())
  name      String
  email     String   @unique
  course    Course   @relation(fields: [courseId], references: [id])
  courseId  Int
  marks     Int      @default(0)
  createdAt DateTime @default(now())
}
```

QUERYING WITH PRISMA

main.js

```
// npx prisma migrate dev --name init   (creates the tables and the typed client)
import { PrismaClient } from "@prisma/client";
const prisma = new PrismaClient();

const course = await prisma.course.create({ data: { title: "JavaScript", fee: 14999 } });
await prisma.student.create({ data: { name: "Asha", email: "asha@lbf.in", courseId: course.id, marks:
91 } });

const list = await prisma.student.findMany({
  where: { marks: { gte: 80 } },
  include: { course: true },          // JOIN
  orderBy: { marks: "desc" },
  take: 10,
});
```

TYPICAL RESULT

```
[ { id: 1, name: 'Asha', email: 'asha@lbf.in', courseId: 1, marks: 91, createdAt: 2026-03-
09T04:30:00.000Z,
    course: { id: 1, title: 'JavaScript', fee: 14999 } } ]
```

22.6 Authentication and security

22.6.1 Password hashing with bcrypt

Passwords must **never** be stored in plain text, or with fast hashes such as MD5 or SHA-256: if the database leaks, attackers can test billions of guesses per second. Use a deliberately **slow, salted** password-hashing function: **bcrypt** (most common in Node.js), **scrypt** (built into Node's crypto module) or **Argon2** (the modern recommendation). A random **salt** per user makes identical passwords produce different hashes and defeats precomputed tables; a **cost factor** makes each hash take tens of milliseconds, which is nothing for one login but enormous for an attacker. At login, hash the attempt with the stored salt and compare in constant time. The example uses Node's built-in scrypt; bcrypt usage is shown after it.

SALTED, SLOW PASSWORD HASHING WITH SCRYPT

main.js

```
const crypto = require("node:crypto");
const { promisify } = require("node:util");
const scrypt = promisify(crypto.scrypt);

async function hashPassword(password) {
  const salt = crypto.randomBytes(16).toString("hex");              // unique per user
  const key = await scrypt(password, salt, 64);                      // slow by design
  return `scrypt$${salt}$${key.toString("hex")}`;                    // store salt with the hash
}

async function verifyPassword(password, stored) {
  const [, salt, hashHex] = stored.split("$");
  const key = await scrypt(password, salt, 64);
  return crypto.timingSafeEqual(key, Buffer.from(hashHex, "hex"));   // constant-time comparison
}

(async () => {
  const h1 = await hashPassword("S3cure!pass");
  const h2 = await hashPassword("S3cure!pass");
  console.log("stored value format:", h1.split("$")[0], "+ 32-char salt + 128-char hash; total
length", h1.length);
  console.log("same password, different hashes (salted):", h1 !== h2);
  console.log("correct password:", await verifyPassword("S3cure!pass", h1));
  console.log("wrong password:  ", await verifyPassword("s3cure!pass", h1));

  const sha = crypto.createHash("sha256").update("S3cure!pass").digest("hex");
  console.log("why not plain SHA-256: always the same ->", sha ===
crypto.createHash("sha256").update("S3cure!pass").digest("hex"));
})();
```

OUTPUT

```
stored value format: scrypt + 32-char salt + 128-char hash; total length 168
same password, different hashes (salted): true
correct password: true
wrong password:   false
why not plain SHA-256: always the same -> true
```

THE SAME WITH BCRYPT

main.js

```
import bcrypt from "bcrypt";

const hash = await bcrypt.hash("S3cure!pass", 12);       // 12 = cost factor (2^12 rounds)
console.log(hash);
console.log(await bcrypt.compare("S3cure!pass", hash));   // true
console.log(await bcrypt.compare("wrong", hash));         // false
```

OUTPUT

```
$2b$12$8vJm1Z0q3kQ5mH2bPpXzUuB6tQf0wY7rN1cL4eD9sA2gK5hJ3iO6e
true
false
```

The hash string itself contains the algorithm ($2b$), the cost (12) and the salt, so only this one string needs storing.

22.6.2 JWT authentication

With **JWT authentication** (introduced in Module 18), the server checks the email and password at `POST /login` and returns a signed token containing claims such as the user's id and role (`sub`, `role`) and an expiry (`exp`). The client sends it with each request in `Authorization: Bearer <token>`, and the server **verifies the signature** with its secret, so no session storage is needed. The token is signed, not encrypted:

anyone can read the payload, nobody can change it without invalidating the signature. Keep access tokens short-lived (15 minutes to an hour), use a long random secret from the environment, and use refresh tokens (in HttpOnly cookies) for long sessions. The example implements HS256 signing with Node's crypto to show exactly what the jsonwebtoken library does.

SIGNING AND VERIFYING A JWT (HS256)

main.js

```
const crypto = require("node:crypto");
const SECRET = "a-long-random-secret-from-the-environment";
const b64url = (data) => Buffer.from(data).toString("base64url");

function sign(payload, secret, expiresInSec) {
  const header = { alg: "HS256", typ: "JWT" };
  const body = { ...payload, iat: Math.floor(Date.now() / 1000), exp: Math.floor(Date.now() / 1000) +
expiresInSec };
  const unsigned = `${b64url(JSON.stringify(header))}.${b64url(JSON.stringify(body))}`;
  const signature = crypto.createHmac("sha256", secret).update(unsigned).digest("base64url");
  return `${unsigned}.${signature}`;
}

function verify(token, secret) {
  const [h, p, s] = token.split(".");
  if (!s) throw new Error("malformed token");
  const expected = crypto.createHmac("sha256", secret).update(`${h}.${p}`).digest("base64url");
  if (s.length !== expected.length || !crypto.timingSafeEqual(Buffer.from(s), Buffer.from(expected)))
throw new Error("invalid signature");
  const payload = JSON.parse(Buffer.from(p, "base64url").toString());
  if (payload.exp < Date.now() / 1000) throw new Error("token expired");
  return payload;
}

const token = sign({ sub: "7", name: "Asha", role: "admin" }, SECRET, 3600);
console.log("token has 3 parts:", token.split(".").length, "| header:", Buffer.from(token.split(".")
[0], "base64url").toString());
const claims = verify(token, SECRET);
console.log("verified claims:", { sub: claims.sub, role: claims.role, validForSeconds: claims.exp -
claims.iat });

const [h, p, s] = token.split(".");
const forgedPayload = b64url(JSON.stringify({ ...claims, role: "superadmin" }));
for (const [label, t, key] of [["tampered payload", `${h}.${forgedPayload}.${s}`, SECRET], ["wrong
secret", token, "guess"], ["expired", sign({ sub: "7" }, SECRET, -10), SECRET]]) {
  try { verify(t, key); } catch (e) { console.log(`${label.padEnd(16)} -> rejected: ${e.message}`); }
}
```

OUTPUT

```
token has 3 parts: 3 | header: {"alg":"HS256","typ":"JWT"}
verified claims: { sub: '7', role: 'admin', validForSeconds: 3600 }
tampered payload -> rejected: invalid signature
wrong secret     -> rejected: invalid signature
expired          -> rejected: token expired
```

A LOGIN ROUTE WITH BCRYPT AND JSONWEBTOKEN

main.js

```
import jwt from "jsonwebtoken";
import bcrypt from "bcrypt";

router.post("/login", async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email }).select("+passwordHash");
  if (!user || !(await bcrypt.compare(password, user.passwordHash))) {
    return res.status(401).json({ error: "Invalid email or password" });   // same message for both
cases
  }
  const token = jwt.sign({ sub: user.id, role: user.role }, process.env.JWT_SECRET, { expiresIn: "1h"
});
  res.json({ token, user: { id: user.id, name: user.name, role: user.role } });
});
```

REQUESTS

```
$ curl -s -X POST localhost:3000/api/auth/login -H "Content-Type: application/json" -d
'{"email":"admin@lbf.in","password":"S3cure!pass"}'
{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxIiwicm9sZSI6ImFkbWluIiwiaWF0IjoxNzczMDMwNjA
wLCJleHAiOjE3NzMwMzQyMDB9.k3Xq...","user":{"id":"1","name":"Admin","role":"admin"}}
$ curl -s -X POST localhost:3000/api/auth/login -H "Content-Type: application/json" -d
'{"email":"admin@lbf.in","password":"nope"}'
{"error":"Invalid email or password"}
```

22.6.3 Protecting routes

A **protected route** requires a valid token. An authentication middleware reads the `Authorization` header, verifies the token, attaches the user's claims to `req.user` and calls `next()`, or responds with **401 Unauthorized** when the token is missing, invalid or expired. A second, **authorisation** middleware checks roles or ownership and responds with **403 Forbidden** when the authenticated user is not allowed. Middleware can protect a single route, a router, or everything after it.

AUTHENTICATION AND ROLE-BASED AUTHORISATION MIDDLEWARE main.js

```
// src/middleware/auth.js
import jwt from "jsonwebtoken";

export function requireAuth(req, res, next) {
  const [scheme, token] = (req.get("Authorization") ?? "").split(" ");
  if (scheme !== "Bearer" || !token) return res.status(401).json({ error: "Authentication required"
});
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);     // { sub, role, iat, exp }
    next();
  } catch (err) {
    res.status(401).json({ error: err.name === "TokenExpiredError" ? "Token expired" : "Invalid token"
});
  }
}

export const requireRole = (...roles) => (req, res, next) =>
  roles.includes(req.user.role) ? next() : res.status(403).json({ error: "You do not have permission
to do this" });

// src/routes/student.routes.js
router.get("/", requireAuth, listStudents);                                  // any logged-in user
router.post("/", requireAuth, requireRole("admin", "staff"), validateBody(StudentInput),
createStudent);
router.delete("/:id", requireAuth, requireRole("admin"), deleteStudent);    // admins only
```

```
REQUESTS
GET    /api/students   (no token)                -> 401 {"error":"Authentication required"}
GET    /api/students   (expired token)           -> 401 {"error":"Token expired"}
GET    /api/students   (valid student token)     -> 200 {...}
DELETE /api/students/3 (valid student token)     -> 403 {"error":"You do not have permission to do
this"}
DELETE /api/students/3 (valid admin token)       -> 204
```

22.6.4 CORS, Helmet and rate limiting

Three middleware packages cover common protections. **cors** sends the CORS headers (Module 18) so that your front end's origin, and only that origin, may call the API from a browser. **Helmet** sets security-related HTTP headers in one line: a Content-Security-Policy, `X-Content-Type-Options: nosniff`, `Strict-Transport-Security`, frame protection (clickjacking) and more, and removes `X-Powered-By`. **express-rate-limit** limits how many requests a client can make in a time window (stricter on login and sign-up routes to slow down password guessing), answering **429 Too Many Requests**. Add input validation, parameterised queries, HTTPS, request size limits, dependency audits (`npm audit`) and logging, and you have the basics of a secure API (Module 24 covers OWASP risks).

SECURITY MIDDLEWARE

main.js

```
import express from "express";
import cors from "cors";
import helmet from "helmet";
import rateLimit from "express-rate-limit";

const app = express();
app.use(helmet());
app.use(cors({ origin: ["https://app.learnbeingforward.in", "http://localhost:5173"], credentials:
true }));
app.use(express.json({ limit: "100kb" }));
app.use("/api", rateLimit({ windowMs: 15 * 60 * 1000, limit: 300, standardHeaders: "draft-8",
legacyHeaders: false }));
app.use("/api/auth/login", rateLimit({ windowMs: 15 * 60 * 1000, limit: 5, message: { error: "Too many
login attempts, try again in 15 minutes" } }));
```

RESPONSE HEADERS / REQUESTS

```
Content-Security-Policy: default-src 'self';base-uri 'self';font-src 'self' https: data:;...
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Access-Control-Allow-Origin: http://localhost:5173
RateLimit: "300-in-15min"; r=299; t=900

6th failed login within 15 minutes -> 429 {"error":"Too many login attempts, try again in 15 minutes"}
```

LIMITER.MJS

JavaScript

```
import http from "node:http";

// fixed-window limiter: at most LIMIT requests per client per WINDOW_MS
function rateLimit({ windowMs, limit }) {
  const hits = new Map();                          // ip -> { count, resetAt }
  return (req, res) => {
    const now = Date.now();
    const key = req.socket.remoteAddress;
    let entry = hits.get(key);
    if (!entry || entry.resetAt <= now) entry = { count: 0, resetAt: now + windowMs };
    entry.count++;
    hits.set(key, entry);
    res.setHeader("RateLimit-Remaining", Math.max(0, limit - entry.count));
    if (entry.count > limit) {
      res.setHeader("Retry-After", Math.ceil((entry.resetAt - now) / 1000));
      res.writeHead(429, { "Content-Type": "application/json" });
      res.end(JSON.stringify({ error: "Too many requests" }));
      return false;
    }
    return true;
  };
}

const loginLimiter = rateLimit({ windowMs: 60_000, limit: 3 });
http.createServer((req, res) => {
  if (req.url === "/api/login" && !loginLimiter(req, res)) return console.log("blocked", req.method,
req.url);
  res.writeHead(401, { "Content-Type": "application/json" });
  res.end(JSON.stringify({ error: "Invalid email or password" }));
}).listen(3000, () => console.log("listening on 3000"));
```

HOW A RATE LIMITER WORKS (BUILT FROM SCRATCH)

main.js

```
node --no-warnings limiter.mjs &
for i in 1 2 3 4; do curl -s -o /dev/null -w "request $i -> %{http_code}\n" localhost:3000/api/login -
X POST; done
curl -s -D - -o /dev/null localhost:3000/api/login -X POST | grep -i -E "retry-after|ratelimit"
```

TERMINAL

```
$ node --no-warnings limiter.mjs &
listening on 3000
$ for i in 1 2 3 4; do curl -s -o /dev/null -w "request $i -> %{http_code}\n" localhost:3000/api/login
-X POST; done
request 1 -> 401
request 2 -> 401
request 3 -> 401
request 4 -> 429
  [server log] blocked POST /api/login
$ curl -s -D - -o /dev/null localhost:3000/api/login -X POST | grep -i -E "retry-after|ratelimit"
RateLimit-Remaining: 0
Retry-After: 60
  [server log] blocked POST /api/login
```

22.7 Deployment

22.7.1 Deploying a Node app (Render, Railway, a VPS)

Deploying an API means running it on a server reachable from the internet, with its environment variables, a database and HTTPS. Before deploying: read the port from `process.env.PORT`; keep secrets in environment variables; add a `start` script and an `engines.node` field; set `NODE_ENV=production`; listen on `0.0.0.0` if required; add a health-check route; log to stdout; and handle shutdown signals (SIGTERM) to close the server and database connections gracefully. **Platform-as-a-service** providers (Render, Railway, Fly.io, Heroku, Koyeb) connect to a GitHub repository, install dependencies, run your start command, provide HTTPS and redeploy on every push; managed databases (MongoDB Atlas, Neon or Supabase for PostgreSQL) are added the same way. On a **VPS** (a virtual machine from AWS EC2, DigitalOcean, Linode, or an Indian provider) you install Node.js yourself, keep the app running with a process manager such as **PM2** (restarts on crash, cluster mode, logs), and put **Nginx** in front as a reverse proxy with a free Let's Encrypt certificate. **Docker** packages the app and its runtime into an image that runs the same everywhere.

Option	Effort	How	Good for
Render / Railway / Fly.io	low	connect the GitHub repo; set build (npm ci) and start (npm start) commands and env vars	most student and startup projects
Serverless (Vercel, Netlify, AWS Lambda)	low	export handlers as functions; no long-running server	APIs with spiky traffic; no WebSockets
VPS + PM2 + Nginx	medium	SSH in, install Node, clone, pm2 start, configure Nginx and certbot	full control, predictable cost
Docker / Kubernetes	medium to high	build an image; run on any container platform	teams, microservices, scaling

PACKAGING THE API AS A DOCKER IMAGE

text

```
# Dockerfile
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY src ./src
ENV NODE_ENV=production
EXPOSE 3000
USER node
CMD ["node", "src/server.js"]
```

RUNNING ON A VPS WITH PM2 AND NGINX

main.js

```
# on the VPS
npm ci --omit=dev
pm2 start src/server.js --name lbf-api -i max     # cluster mode: one process per CPU core
pm2 save && pm2 startup                           # restart on reboot
pm2 logs lbf-api

# /etc/nginx/sites-available/api.learnbeingforward.in
server {
  server_name api.learnbeingforward.in;
  location / {
    proxy_pass http://127.0.0.1:3000;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
  }
}
# sudo certbot --nginx -d api.learnbeingforward.in    (free HTTPS certificate)
```

TYPICAL OUTPUT

```
[PM2] Starting src/server.js in cluster_mode (4 instances)
┌────┬─────────┬─────────┬──────┬────────┬──────────┐
│ id │ name    │ mode    │ ↺    │ status │ memory   │
├────┼─────────┼─────────┼──────┼────────┼──────────┤
│ 0  │ lbf-api │ cluster │ 0    │ online │ 52.1mb   │
│ 1  │ lbf-api │ cluster │ 0    │ online │ 51.8mb   │
└────┴─────────┴─────────┴──────┴────────┴──────────┘
```

HANDLING SIGTERM GRACEFULLY

main.js

```
// Graceful shutdown: finish in-flight requests before exiting (platforms send SIGTERM on redeploy)
const http = require("node:http");
const server = http.createServer((req, res) => res.end("ok")).listen(0, () => {
  console.log("server started");
  process.kill(process.pid, "SIGTERM");          // simulate the platform stopping the app
});

process.on("SIGTERM", () => {
  console.log("SIGTERM received: closing the server (no new connections)");
  server.close(() => {
    console.log("all connections closed; database connections would be closed here");
    process.exit(0);
  });
  setTimeout(() => process.exit(1), 10_000).unref();   // force-exit if something hangs
});
```

OUTPUT

```
server started
SIGTERM received: closing the server (no new connections)
all connections closed; database connections would be closed here
```

Node.js runs JavaScript on servers with V8 and libuv: non-blocking I/O, an event loop with phases (timers, poll, check), process.nextTick and microtasks between callbacks. Globals include process (argv, env, exit), __dirname in CommonJS and import.meta.dirname in ES modules; the REPL and node -e/-p help experiment. Core modules: fs/promises for files, path for portable paths, os for machine info, EventEmitter for events, http for servers, Buffers and streams (pipeline, back-pressure) for data, node:test and --watch for testing and development. Express adds routing (params, query, routers), request/response helpers, middleware chains (built-in, third-party, custom), static files, four-argument error handlers, and a layered structure of routes, controllers, services and models. REST APIs expose CRUD with proper status codes, pagination and validation (hand-written or Zod), with configuration validated from .env, and are tried with curl, Postman or Thunder Client. MongoDB with Mongoose stores documents with schemas; SQL databases use parameterised queries, joins and transactions, often through Prisma or Sequelize. Hash passwords with bcrypt, scrypt or Argon2; issue short-lived JWTs and verify them in auth middleware (401) with role checks (403); add cors, helmet and rate limiting. Deploy to Render or Railway, a VPS with PM2 and Nginx, or Docker, with graceful shutdown.

PRACTICE CHECKPOINT

Build a **Student Management REST API** with Express and MongoDB or MySQL, including CRUD, validation, JWT login and protected routes, connected to a front end with fetch.

The sample solution uses the layered structure from 22.3.7 with Express, Mongoose, Zod, bcrypt and jsonwebtoken. Install the dependencies with `npm install express mongoose zod bcrypt jsonwebtoken cors helmet express-rate-limit` and create a `.env` as in 22.4.3.

APP.JS AND SERVER.JS

JavaScript

```
// src/app.js
import express from "express";
import cors from "cors";
import helmet from "helmet";
import rateLimit from "express-rate-limit";
import authRoutes from "./routes/auth.routes.js";
import studentRoutes from "./routes/student.routes.js";
import { errorHandler, notFound } from "./middleware/error.js";
import config from "./config/index.js";

export const app = express();
app.use(helmet());
app.use(cors({ origin: config.corsOrigin }));
app.use(express.json({ limit: "100kb" }));
app.use("/api/auth/login", rateLimit({ windowMs: 15 * 60_000, limit: 5 }));
app.get("/api/health", (req, res) => res.json({ status: "ok" }));
app.use("/api/auth", authRoutes);
app.use("/api/students", studentRoutes);
app.use(notFound);
app.use(errorHandler);

// src/server.js
import mongoose from "mongoose";
import { app } from "./app.js";
import config from "./config/index.js";
await mongoose.connect(config.databaseUrl);
const server = app.listen(config.port, () => console.log(`API on http://localhost:${config.port}`));
process.on("SIGTERM", () => server.close(() => mongoose.disconnect()));
```

USER MODEL AND LOGIN ROUTE

JavaScript

```
// src/models/user.model.js
import mongoose from "mongoose";
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  passwordHash: { type: String, required: true, select: false },
  role: { type: String, enum: ["admin", "staff", "student"], default: "student" },
});
export const User = mongoose.model("User", userSchema);

// src/routes/auth.routes.js
import { Router } from "express";
import bcrypt from "bcrypt";
import jwt from "jsonwebtoken";
import { z } from "zod";
import { User } from "../models/user.model.js";
import { validateBody } from "../middleware/validate.js";
import { HttpError } from "../utils/HttpError.js";
import config from "../config/index.js";

const router = Router();
const Login = z.object({ email: z.string().email().toLowerCase(), password: z.string().min(8) });

router.post("/login", validateBody(Login), async (req, res) => {
  const user = await User.findOne({ email: req.body.email }).select("+passwordHash");
  if (!user || !(await bcrypt.compare(req.body.password, user.passwordHash))) throw new HttpError(401,
"Invalid email or password");
  const token = jwt.sign({ sub: user.id, role: user.role }, config.jwt.secret, { expiresIn:
config.jwt.expiresIn });
  res.json({ token, user: { id: user.id, name: user.name, role: user.role } });
});
export default router;
```

ROUTES, CONTROLLER AND SERVICE

JavaScript

```
// src/routes/student.routes.js
import { Router } from "express";
import { requireAuth, requireRole } from "../middleware/auth.js";
import { validateBody } from "../middleware/validate.js";
import { StudentInput, StudentPatch } from "../models/student.model.js";
import * as ctrl from "../controllers/student.controller.js";

const router = Router();
router.use(requireAuth);                                    // everything below needs a valid
token
router.get("/", ctrl.list);
router.get("/:id", ctrl.getOne);
router.post("/", requireRole("admin", "staff"), validateBody(StudentInput), ctrl.create);
router.patch("/:id", requireRole("admin", "staff"), validateBody(StudentPatch), ctrl.update);
router.delete("/:id", requireRole("admin"), ctrl.remove);
export default router;

// src/controllers/student.controller.js
import * as service from "../services/student.service.js";
export const list = async (req, res) => res.json(await service.list(req.query));
export const getOne = async (req, res) => res.json(await service.getById(req.params.id));
export const create = async (req, res) => res.status(201).json(await service.create(req.body));
export const update = async (req, res) => res.json(await service.update(req.params.id, req.body));
export const remove = async (req, res) => { await service.remove(req.params.id); res.sendStatus(204);
};
```

```
// src/services/student.service.js
import { Student } from "../models/student.model.js";
import { HttpError } from "../utils/HttpError.js";

export async function list({ course, page = 1, limit = 20, sort = "-createdAt" }) {
  const filter = course ? { course } : {};
  const [total, data] = await Promise.all([
    Student.countDocuments(filter),
    Student.find(filter).sort(sort).skip((page - 1) * limit).limit(Math.min(limit, 100)),
  ]);
  return { total, page: Number(page), data };
}
export async function getById(id) {
  const s = await Student.findById(id);
  if (!s) throw new HttpError(404, `Student ${id} not found`);
  return s;
}
export async function create(data) {
  if (await Student.exists({ email: data.email })) throw new HttpError(409, "Email already
registered");
  return Student.create(data);
}
export async function update(id, changes) {
  const s = await Student.findByIdAndUpdate(id, changes, { new: true, runValidators: true });
  if (!s) throw new HttpError(404, `Student ${id} not found`);
  return s;
}
export async function remove(id) {
  const r = await Student.findByIdAndDelete(id);
  if (!r) throw new HttpError(404, `Student ${id} not found`);
}
```

CONNECTING A FRONT END WITH FETCH JavaScript

```
// front end: public/students.js (served by Vite or express.static)
const API = "http://localhost:3000/api";
let token = sessionStorage.getItem("token");

async function api(path, { method = "GET", body } = {}) {
  const res = await fetch(`${API}${path}`, {
    method,
    headers: { "Content-Type": "application/json", ...(token && { Authorization: `Bearer ${token}` })
},
    body: body && JSON.stringify(body),
  });
  if (res.status === 401) { sessionStorage.removeItem("token"); throw new Error("Please log in
again"); }
  const data = res.status === 204 ? null : await res.json();
  if (!res.ok) throw new Error(data?.details ? Object.values(data.details).flat().join("; ") :
data?.error);
  return data;
}

export async function login(email, password) {
  ({ token } = await api("/auth/login", { method: "POST", body: { email, password } }));
  sessionStorage.setItem("token", token);
}
export const listStudents = (course) => api(`/students?${new URLSearchParams(course ? { course } :
{})}`);
export const addStudent = (s) => api("/students", { method: "POST", body: s });
export const updateStudent = (id, changes) => api(`/students/${id}`, { method: "PATCH", body: changes
});
export const deleteStudent = (id) => api(`/students/${id}`, { method: "DELETE" });
```

TRYING THE FINISHED API

main.js

```
$ curl -s -X POST localhost:3000/api/auth/login -H "Content-Type: application/json" -d
'{"email":"admin@lbf.in","password":"S3cure!pass"}'
$ TOKEN=<token from the response>
$ curl -s -X POST localhost:3000/api/students -H "Authorization: Bearer $TOKEN" -H "Content-Type:
application/json" \
       -d '{"name":"Zoya Khan","email":"zoya@lbf.in","course":"Java","marks":88}'
$ curl -s "localhost:3000/api/students?course=Java" -H "Authorization: Bearer $TOKEN"
$ curl -s localhost:3000/api/students
```

RESPONSES

```
{"token":"eyJhbGciOiJIUzI1NiIs...","user":
{"id":"65f0c1a2e4b0c9d1a2b3c4d5","name":"Admin","role":"admin"}}
{"name":"Zoya
Khan","email":"zoya@lbf.in","course":"Java","marks":88,"_id":"65f0c1b7e4b0c9d1a2b3c4e1","createdAt":"2
026-03-09T04:31:12.402Z","updatedAt":"2026-03-09T04:31:12.402Z"}
{"total":1,"page":1,"data":[{"_id":"65f0c1b7e4b0c9d1a2b3c4e1","name":"Zoya
Khan","email":"zoya@lbf.in","course":"Java","marks":88,...}]}
{"error":"Authentication required"}
```

MODULE 23 · ADVANCED

TypeScript essentials

Minimum duration **4 days** Level **Advanced**

TypeScript adds a static type system to JavaScript and is now the default language of most professional front-end and Node.js projects. This module explains what TypeScript is and how to compile it, the tsconfig.json file, basic and special types, inference, object types with type aliases and interfaces, optional and readonly properties, unions, intersections, literal types and enums, typed functions, classes with access modifiers, generics, narrowing, the built-in utility types, and how to use TypeScript with Node.js and Vite.

IN THIS MODULE

23.1 Introduction
23.2 Basic types
23.3 Object types
23.4 Functions
23.5 Classes in TypeScript
23.6 Generics
23.7 Narrowing
23.8 Utility types
23.9 Using TypeScript with Node.js and Vite

NOTE

Every TypeScript example was compiled with the TypeScript compiler in strict mode (`tsc --strict`) and then run with Node.js. Examples labelled COMPILER ERRORS show the real messages tsc printed: those programs never ran, which is the whole point of a type checker.

23.1 Introduction

23.1.1 What TypeScript is and why it is used

TypeScript, created by Microsoft (2012, led by Anders Hejlsberg, the designer of C#), is a **superset** of JavaScript: every JavaScript program is valid TypeScript, and TypeScript adds optional **type annotations** (`let age: number`) plus a compiler that checks them **before the code runs**. The types are then **erased**: the compiler outputs plain JavaScript, so TypeScript adds no run-time cost and runs anywhere JavaScript runs. Its benefits: whole classes of bugs (typos in property names, calling a function with the wrong arguments, forgetting that a value may be `undefined`) are caught as you type; editors offer precise autocompletion, inline documentation and safe automatic refactoring; and types document the code for the next developer. Angular, Next.js, NestJS, VS Code itself, and most large codebases at companies such as Google, Microsoft, Airbnb and Slack are written in TypeScript.



TYPES ARE CHECKED AT COMPILE TIME AND REMOVED FROM THE OUTPUT

BUGS TYPESCRIPT CATCHES BEFORE RUNNING main.js

```
function totalPrice(items: { name: string; price: number; qty: number }[]) {
  return items.reduce((sum, item) => sum + item.price * item.quantity, 0);
}

const cart = [{ name: "Pen", price: "10", qty: 3 }];
totalPrice(cart);

const user = { name: "Asha", email: "asha@lbf.in" };
console.log(user.emial.toUpperCase());
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,62): error TS2339: Property 'quantity' does not exist on type '{ name: string; price:
number; qty: number; }'.
main.ts(6,12): error TS2345: Argument of type '{ name: string; price: string; qty: number; }[]' is not
assignable to parameter of type '{ name: string; price: number; qty: number; }[]'.
  Type '{ name: string; price: string; qty: number; }' is not assignable to type '{ name: string;
price: number; qty: number; }'.
    Types of property 'price' are incompatible.
      Type 'string' is not assignable to type 'number'.
main.ts(9,18): error TS2551: Property 'emial' does not exist on type '{ name: string; email: string;
}'. Did you mean 'email'?
```

Plain JavaScript would have run this code: `item.quantity` would be `undefined` (giving NaN), the string price would concatenate, and `user.emial` would crash at run time. TypeScript finds all three in the editor.

23.1.2 Installing and compiling with tsc

Install the compiler per project as a dev dependency: `npm install -D typescript`, then run it with `npx tsc`. `tsc file.ts` compiles one file to `file.js`; `tsc` with no arguments compiles the whole project described by `tsconfig.json`; `tsc --watch` recompiles on every change; `tsc --noEmit` only type-checks (common in CI when another tool does the compiling). Errors are printed with the file, line, column and an error code such as TS2322. By default tsc still writes the JavaScript even when there are type errors (use `noEmitOnError` to prevent that). Modern runtimes can also run TypeScript directly: Node.js 22.18+/23.6+ strips types natively (`node app.ts`), as do Deno, Bun, tsx and ts-node.

GREET.TS TypeScript

```
type Level = "beginner" | "advanced";

function greet(name: string, level: Level = "beginner"): string {
  return `Welcome ${name}, enjoy the ${level} track!`;
}

const students: string[] = ["Asha", "Ravi"];
for (const s of students) console.log(greet(s, "advanced"));
```

COMPILING A SINGLE FILE main.js

```
tsc --version
tsc greet.ts
cat greet.js
node greet.js
```

TERMINAL

```
$ tsc --version
Version 6.0.3
$ tsc greet.ts
$ cat greet.js
"use strict";
function greet(name, level = "beginner") {
    return `Welcome ${name}, enjoy the ${level} track!`;
}
const students = ["Asha", "Ravi"];
for (const s of students)
    console.log(greet(s, "advanced"));
$ node greet.js
Welcome Asha, enjoy the advanced track!
Welcome Ravi, enjoy the advanced track!
```

NOTE

Look at `greet.js`: the type alias, the annotations and the return type are all gone, and the remaining code is ordinary JavaScript.

23.1.3 tsconfig.json

`tsconfig.json` at the project root tells tsc which files to compile and how. `tsc --init` generates one with sensible defaults. The most important options: `strict` (turns on the whole family of strict checks, including `strictNullChecks` and `noImplicitAny`; always enable it in new projects); `target` (which JavaScript version to output); `module` and `moduleResolution` (the module system: `nodenext` for Node.js, `esnext` / `bundler` for Vite); `rootDir` / `outDir` (where sources are and where output goes); `include` / `exclude`; `lib` (which built-in declarations exist, such as DOM); `sourceMap` (debug the original .ts); `declaration` (emit .d.ts files for libraries); and extra checks such as `noUnusedLocals` and `noUncheckedIndexedAccess`.

TSCONFIG.JSON JavaScript

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "nodenext",
    "moduleResolution": "nodenext",
    "rootDir": "src",
    "outDir": "dist",
    "strict": true,
    "noUnusedLocals": true,
    "noUncheckedIndexedAccess": true,
    "sourceMap": true,
    "types": []
  },
  "include": ["src"]
}
```

SRC/INDEX.TS

TypeScript

```ts
import { average } from "./math.js";       // ESM in Node.js: import the .js output name

const marks: Record<string, number[]> = { Asha: [91, 88], Ravi: [64, 70] };
for (const [name, list] of Object.entries(marks)) {
  console.log(`${name}: ${average(list).toFixed(1)}`);
}
const first = marks["Meera"];                 // noUncheckedIndexedAccess: number[] | undefined
console.log("Meera's first mark:", first?.[0] ?? "no marks yet");
```

BUILDING A PROJECT WITH TSCONFIG.JSON

main.js

```
tsc
ls dist
node dist/index.js
tsc --showConfig | head -12
```

TERMINAL

```
$ tsc
$ ls dist
index.js
index.js.map
math.js
math.js.map
$ node dist/index.js
Asha: 89.5
Ravi: 67.0
Meera's first mark: no marks yet
$ tsc --showConfig | head -12
{
    "compilerOptions": {
        "target": "es2022",
        "module": "nodenext",
        "moduleResolution": "nodenext",
        "rootDir": "./src",
        "outDir": "./dist",
        "strict": true,
        "noUnusedLocals": true,
        "noUncheckedIndexedAccess": true,
        "sourceMap": true,
        "types": [],
```

23.2 Basic types

23.2.1 string, number, boolean, arrays, tuples

The primitive types are written in lower case: `string`, `number` (all numbers, integer or decimal), `boolean`, `bigint`, `symbol`, `null` and `undefined`. Annotate a variable with `: Type` after its name. Arrays are written `number[]` or `Array<number>`, and `readonly string[]` forbids mutation. A **tuple** is an array with a fixed length and a known type at each position, such as `[string, number]`; tuple elements can be named (`[name: string, marks: number]`), optional, or a rest (`[string, ...number[]]`). Use the lower-case primitives, never the wrapper types `String` or `Number`.

ANNOTATING PRIMITIVES, ARRAYS AND TUPLES main.js

```
let studentName: string = "Asha";
let marks: number = 91.5;
let passed: boolean = marks >= 40;
let population: bigint = 1_400_000_000n;

const scores: number[] = [91, 78, 85];
const tags: Array<string> = ["js", "ts"];
const fixed: readonly number[] = [1, 2, 3];

let entry: [string, number] = ["Asha", 91];                  // tuple: exactly a string then a number
const point: [x: number, y: number, z?: number] = [3, 4];    // named and optional elements
const row: [string, ...number[]] = ["Ravi", 64, 70, 72];      // a rest element

const [who, score] = entry;                                   // destructuring keeps the types
console.log(studentName, marks, passed, population);
console.log(scores.map((s) => s + 1), tags.join("/"), fixed.length);
console.log(`${who.toUpperCase()} scored ${score.toFixed(0)}`, point, row.length);

function minMax(values: number[]): [min: number, max: number] {  // tuples are handy return types
  return [Math.min(...values), Math.max(...values)];
}
const [lo, hi] = minMax(scores);
console.log("min/max:", lo, hi);
```

OUTPUT

```
Asha 91.5 true 1400000000n
[ 92, 79, 86 ] js/ts 3
ASHA scored 91 [ 3, 4 ] 4
min/max: 78 91
```

TYPE ERRORS WITH PRIMITIVES, ARRAYS AND TUPLES main.js

```
let marks: number = 91;
marks = "ninety";

const scores: number[] = [91, 78];
scores.push("85");

const fixed: readonly number[] = [1, 2];
fixed.push(3);

let entry: [string, number] = ["Asha", 91];
entry = [91, "Asha"];
const third = entry[2];
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,1): error TS2322: Type 'string' is not assignable to type 'number'.
main.ts(5,13): error TS2345: Argument of type 'string' is not assignable to parameter of type
'number'.
main.ts(8,7): error TS2339: Property 'push' does not exist on type 'readonly number[]'.
main.ts(11,10): error TS2322: Type 'number' is not assignable to type 'string'.
main.ts(11,14): error TS2322: Type 'string' is not assignable to type 'number'.
main.ts(12,21): error TS2493: Tuple type '[string, number]' of length '2' has no element at index '2'.
```

23.2.2 any, unknown, never, void

Four special types describe unusual situations. `any` switches type checking off for a value: anything can be assigned to it and done with it. It is an escape hatch that spreads silently (anything derived from `any` is `any`), so avoid it; `noImplicitAny` (part of strict) forbids `any` being inferred silently. `unknown` is the safe

alternative: any value can be assigned to it, but you must **narrow** it (check what it is) before using it; use it for JSON input, caught errors and user data. `void` is the return type of functions that return nothing useful. `never` is the type of values that can never exist: the return type of a function that always throws or loops forever, and the type left over when every possibility has been handled (used for exhaustiveness checks, 23.7).

UNKNOWN, VOID, NEVER AND ANY main.js

```
// unknown: must be checked before use
function describe(value: unknown): string {
  if (typeof value === "string") return `a string of length ${value.length}`;
  if (typeof value === "number") return `the number ${value.toFixed(1)}`;
  if (Array.isArray(value)) return `an array of ${value.length} items`;
  if (value instanceof Date) return `a date in ${value.getFullYear()}`;
  return `something else (${typeof value})`;
}
const parsed: unknown = JSON.parse('{"id": 7}');
for (const v of ["hi", 3.14159, [1, 2], new Date(2026, 2, 9), parsed, null]) console.log(describe(v));

// void: no useful return value
function logLine(message: string): void {
  console.log(`[log] ${message}`);
}
logLine("server started");

// never: a function that never returns normally
function fail(message: string): never {
  throw new Error(message);
}
try { fail("configuration missing"); } catch (e) { console.log("caught:", (e as Error).message); }

// any: everything allowed, nothing checked
let anything: any = "text";
anything = 42;
console.log("any allows nonsense until run time:", typeof anything.toFixed, "| would crash:", typeof
anything.notAFunction);
```

OUTPUT

```
a string of length 2
the number 3.1
an array of 2 items
a date in 2026
something else (object)
something else (object)
[log] server started
caught: configuration missing
any allows nonsense until run time: function | would crash: undefined
```

UNKNOWN MUST BE NARROWED; ANY IS NEVER CHECKED

main.js

```
const input: unknown = JSON.parse('"hello"');
console.log(input.toUpperCase());

function noReturn(): never {
  return 1;
}

let risky: any = "text";
const n: number = risky;
n.toFixed(2);
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,13): error TS18046: 'input' is of type 'unknown'.
main.ts(5,3): error TS2322: Type '1' is not assignable to type 'never'.
```

The first two are errors as expected. The last two lines compile without complaint even though `n` actually holds a string: `any` disabled the check, and the program would crash at run time. Prefer `unknown`.

23.2.3 Type inference

You rarely need to write every type: TypeScript **infers** types from values. `let count = 0` is a `number`; `const city = "Mysuru"` has the **literal** type `"Mysuru"` (a const can never change); arrays, object literals and function return types are inferred too, and callback parameters get their types from context (`scores.map((s) => ...)` knows `s` is a number). Good practice: annotate function **parameters** (which cannot be inferred), public API return types and empty initial values (`let list: string[] = []`), and let inference handle the rest. Hover over a name in VS Code to see what was inferred. `as const` asks for the narrowest possible type, and the `satisfies` operator checks a value against a type without widening it.

LETTING TYPESCRIPT INFER TYPES

main.js

```
let count = 0;                      // number
const city = "Mysuru";              // literal type "Mysuru"
const scores = [91, 78, 85];        // number[]
const student = { name: "Asha", marks: 91, tags: ["js"] };   // { name: string; marks: number; tags:
string[] }

const doubled = scores.map((s) => s * 2);       // s: number (contextual typing), result number[]
function average(values: number[]) {            // return type inferred as number
  return values.reduce((a, b) => a + b, 0) / values.length;
}

const directions = ["north", "south"] as const;  // readonly ["north", "south"]
type Direction = (typeof directions)[number];    // "north" | "south"
const go: Direction = "south";

type Theme = Record<string, string>;
const palette = { navy: "#1B2A4A", gold: "#C9A227" } satisfies Theme;   // checked, but keeps exact
keys
console.log(count + 1, city.length, doubled, average(scores).toFixed(2), student.tags[0], go,
palette.gold);
```

OUTPUT

```
1 6 [ 182, 156, 170 ] 84.67 js south #C9A227
```

INFERRED TYPES ARE ENFORCED TOO

main.js

```
let count = 0;
count = "zero";

const student = { name: "Asha", marks: 91 };
student.grade = "A";

const palette = { navy: "#1B2A4A" } satisfies Record<string, string>;
console.log(palette.gold);
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,1): error TS2322: Type 'string' is not assignable to type 'number'.
main.ts(5,9): error TS2339: Property 'grade' does not exist on type '{ name: string; marks: number;
}'.
main.ts(8,21): error TS2339: Property 'gold' does not exist on type '{ navy: string; }'.
```

23.3 Object types

23.3.1 Type aliases

An object type lists properties and their types: `{ name: string; marks: number }`. Writing it repeatedly is tedious, so give it a name with a **type alias**: `type Student = { ... }`. A type alias can name **any** type, not only objects: unions (`type Id = string | number`), tuples, functions (`type Comparator = (a: number, b: number) => number`) and combinations. TypeScript is **structurally typed**: an object is compatible with a type if it has the required properties with the right types, whatever it is called or where it came from ("duck typing" checked at compile time). Extra properties are rejected only in a fresh object literal, where they are probably typos.

NAMING TYPES WITH TYPE ALIASES

main.js

```
type Student = {
  id: number;
  name: string;
  marks: number;
};
type Id = string | number;
type Comparator<T> = (a: T, b: T) => number;

const byMarks: Comparator<Student> = (a, b) => b.marks - a.marks;

const students: Student[] = [
  { id: 1, name: "Asha", marks: 91 },
  { id: 2, name: "Ravi", marks: 64 },
  { id: 3, name: "Meera", marks: 85 },
];

function label(id: Id): string {
  return typeof id === "number" ? `#${id.toString().padStart(3, "0")}` : id.toUpperCase();
}

// structural typing: any object with the right shape is accepted
const fromApi = JSON.parse('{"id": 4, "name": "John", "marks": 77, "batch": "B"}') as Student;
const all = [...students, fromApi].toSorted(byMarks);
for (const s of all) console.log(label(s.id), s.name.padEnd(6), s.marks);
console.log(label("lbf-2026"));
```

OUTPUT

```
#001 Asha   91
#003 Meera  85
#004 John   77
#002 Ravi   64
LBF-2026
```

23.3.2 Interfaces

An **interface** is another way to name an object type: `interface Student { ... }`. Interfaces can **extend** one or more other interfaces, describe methods and callable or indexable objects, and be implemented by classes (23.5). Unlike type aliases, interfaces with the same name **merge** (declaration merging), which libraries use to let you add properties to, for example, Express's `Request`. In everyday code the two are nearly interchangeable; a common convention is interfaces for object shapes and public APIs, and type aliases for unions, tuples, functions and computed types.

INTERFACES, EXTENSION AND INDEX SIGNATURES

main.js

```
interface Person {
  name: string;
  email: string;
}
interface Student extends Person {            // inheritance between interfaces
  rollNo: number;
  courses: string[];
  enrol(course: string): void;                // a method
}
interface Dictionary {
  [key: string]: number;                      // index signature: any string key -> number
}

const asha: Student = {
  name: "Asha",
  email: "asha@lbf.in",
  rollNo: 101,
  courses: ["JavaScript"],
  enrol(course) { this.courses.push(course); }, // parameter type comes from the interface
};
asha.enrol("TypeScript");

const attendance: Dictionary = { Mon: 1, Tue: 1, Wed: 0 };
attendance["Thu"] = 1;

// declaration merging: a second declaration adds a property
interface Person { phone?: string }
const ravi: Person = { name: "Ravi", email: "ravi@lbf.in", phone: "+91 98450 12345" };

console.log(asha.courses, Object.values(attendance).reduce((a, b) => a + b, 0) + " days", ravi.phone);
```

OUTPUT

```
[ 'JavaScript', 'TypeScript' ] 3 days +91 98450 12345
```

	`interface`	`type` alias
Object shapes	yes	yes
Unions, tuples, primitives, mapped types	no	yes
Extending	`extends`	intersection `&`
Declaration merging	yes	no (duplicate name error)
Implemented by classes	yes	yes (if it is an object type)
Typical use	public object shapes, library APIs	unions, functions, utility and computed types

23.3.3 Optional and readonly properties

A `?` after a property name makes it **optional**: it may be missing, so its type includes `undefined`, and strict mode makes you handle that case (with `?.`, `??` or a check). `readonly` prevents a property from being reassigned after creation (a compile-time guarantee; use `Object.freeze` for run-time protection). `Readonly<T>` and `ReadonlyArray<T>` make whole objects and arrays read-only.

OPTIONAL AND READONLY PROPERTIES

main.js

```
interface Course {
  readonly id: number;
  title: string;
  fee: number;
  discountPercent?: number;            // optional: number | undefined
  mentor?: { name: string; email?: string };
}

function finalFee(c: Course): number {
  const discount = c.discountPercent ?? 0;          // must handle the missing case
  return Math.round(c.fee * (1 - discount / 100));
}

const js: Course = { id: 1, title: "JavaScript", fee: 14999, discountPercent: 10, mentor: { name:
"Ravi" } };
const py: Course = { id: 2, title: "Python", fee: 12999 };

for (const c of [js, py]) {
  console.log(`${c.title}: Rs ${finalFee(c)} | mentor email: ${c.mentor?.email ?? "not set"}`);
}
js.title = "JavaScript Pro";                        // allowed
console.log(js.title, js.id);
```

OUTPUT

```
JavaScript: Rs 13499 | mentor email: not set
Python: Rs 12999 | mentor email: not set
JavaScript Pro 1
```

ERRORS FOR READONLY AND OPTIONAL PROPERTIES

main.js

```
interface Course {
  readonly id: number;
  title: string;
  discountPercent?: number;
}
const c: Course = { id: 1, title: "JS" };
c.id = 2;
const percent: number = c.discountPercent;
console.log(c.discountPercent.toFixed(1));
const missing: Course = { title: "No id" };
```

COMPILER ERRORS (tsc --strict)

```
main.ts(7,3): error TS2540: Cannot assign to 'id' because it is a read-only property.
main.ts(8,7): error TS2322: Type 'number | undefined' is not assignable to type 'number'.
  Type 'undefined' is not assignable to type 'number'.
main.ts(9,13): error TS18048: 'c.discountPercent' is possibly 'undefined'.
main.ts(10,7): error TS2741: Property 'id' is missing in type '{ title: string; }' but required in
type 'Course'.
```

23.3.4 Union and intersection types

A **union** type `A | B` means "either A or B": `string | number`, `Student | null`, `"light" | "dark"`. You can only use members common to all options until you **narrow** the value (23.7). An **intersection** `A & B` means "both A and B at once": an object of type `Person & Employee` has every property of both. Unions model alternatives (results that can succeed or fail, different shapes of messages); intersections combine capabilities (mixing in timestamps or ids).

UNIONS FOR ALTERNATIVES, INTERSECTIONS FOR COMBINATIONS

main.js

```
type Id = string | number;
type Timestamps = { createdAt: Date; updatedAt: Date };
type WithId = { id: Id };
type Note = { text: string; pinned: boolean };
type StoredNote = Note & WithId & Timestamps;          // intersection: has everything

function formatId(id: Id): string {
  return typeof id === "number" ? `N-${id}` : id;       // narrowing a union
}

const note: StoredNote = {
  id: 42, text: "Revise generics", pinned: true,
  createdAt: new Date(2026, 2, 9), updatedAt: new Date(2026, 2, 10),
};
console.log(formatId(note.id), note.text, note.updatedAt > note.createdAt);

function findNote(notes: StoredNote[], id: Id): StoredNote | undefined {   // union with undefined
  return notes.find((n) => n.id === id);
}
const found = findNote([note], 7);
console.log(found ? found.text : "note 7 not found");
```

OUTPUT

```
N-42 Revise generics true
note 7 not found
```

23.3.5 Literal types and enums

A **literal type** is a type with exactly one value: `"admin"`, `404`, `true`. Unions of literals model a fixed set of allowed values, with autocompletion and typo checking: `type Role = "admin" | "staff" | "student"`. An **enum** (a TypeScript feature that also generates JavaScript code) gives names to a set of constants: numeric enums count from 0 by default, string enums map names to strings. Many teams prefer literal unions or `as const` objects, which add nothing at run time, but enums remain common, especially in Angular and NestJS code. (With Node.js's built-in type stripping or the `erasableSyntaxOnly` option, enums are not allowed because they are not just erasable types.)

LITERAL TYPES, ENUMS AND AS CONST

main.js

```
type Role = "admin" | "staff" | "student";
type HttpSuccess = 200 | 201 | 204;

function canDelete(role: Role): boolean {
  return role === "admin";
}

enum Level { Beginner, Intermediate, Advanced }            // 0, 1, 2
enum Status { Active = "ACTIVE", Paused = "PAUSED", Done = "DONE" }

const course = { title: "TypeScript", level: Level.Intermediate, status: Status.Active };
console.log("canDelete(staff):", canDelete("staff"), "| canDelete(admin):", canDelete("admin"));
console.log("level value:", course.level, "| name:", Level[course.level], "| status:", course.status);
console.log("enum objects exist at run time:", Level, Status);

// the "as const object" alternative: no generated enum code, same safety
const PAYMENT = { UPI: "upi", CARD: "card", CASH: "cash" } as const;
type Payment = (typeof PAYMENT)[keyof typeof PAYMENT];      // "upi" | "card" | "cash"
const method: Payment = PAYMENT.UPI;
const ok: HttpSuccess = 201;
console.log(method, ok);
```

OUTPUT

```
canDelete(staff): false | canDelete(admin): true
level value: 1 | name: Intermediate | status: ACTIVE
enum objects exist at run time: {
  '0': 'Beginner',
  '1': 'Intermediate',
  '2': 'Advanced',
  Beginner: 0,
  Intermediate: 1,
  Advanced: 2
} { Active: 'ACTIVE', Paused: 'PAUSED', Done: 'DONE' }
upi 201
```

LITERAL TYPES CATCH TYPOS

main.js

```
type Role = "admin" | "staff" | "student";
let r: Role = "teacher";

enum Level { Beginner, Intermediate, Advanced }
let l: Level = Level.Expert;

function setTheme(theme: "light" | "dark") {}
setTheme("Dark");
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,5): error TS2322: Type '"teacher"' is not assignable to type 'Role'.
main.ts(5,22): error TS2339: Property 'Expert' does not exist on type 'typeof Level'.
main.ts(8,10): error TS2345: Argument of type '"Dark"' is not assignable to parameter of type '"light"
| "dark"'.
```

23.4 Functions

23.4.1 Typing parameters and return values

Annotate each parameter (`(price: number, qty: number)`); the return type can be inferred or written after the parameter list (`: number`), which documents the contract and catches mistakes inside the function. Function **types** describe callbacks and variables holding functions: `(a: number, b: number) => number`. Rest parameters are typed as arrays (`...values: number[]`). **Overloads** describe a function that accepts several distinct argument combinations, and `this` can be typed as a fake first parameter.

TYPED FUNCTIONS, CALLBACKS AND OVERLOADS main.js

```
function lineTotal(price: number, qty: number): number {
  return price * qty;
}

const applyTax = (amount: number, rate = 0.18): number => Math.round(amount * (1 + rate));

function sum(...values: number[]): number {
  return values.reduce((t, v) => t + v, 0);
}

type Formatter = (value: number) => string;
const inr: Formatter = (v) => new Intl.NumberFormat("en-IN", { style: "currency", currency: "INR"
}).format(v);

function applyAll(values: number[], transform: (v: number, i: number) => number): number[] {
  return values.map(transform);
}

// overloads: two ways to call one function
function parseDate(text: string): Date;
function parseDate(day: number, month: number, year: number): Date;
function parseDate(a: string | number, m?: number, y?: number): Date {
  return typeof a === "string" ? new Date(a) : new Date(y!, m! - 1, a);
}

console.log(lineTotal(499, 3), applyTax(1000), sum(1, 2, 3, 4), inr(150000));
console.log(applyAll([1, 2, 3], (v, i) => v * 10 + i));
console.log(parseDate("2026-03-09").getUTCDate(), parseDate(9, 3, 2026).getMonth() + 1);
```

OUTPUT

```
1497 1180 10 ₹1,50,000.00
[ 10, 21, 32 ]
9 3
```

MISTAKES IN CALLS AND RETURN VALUES main.js

```
function lineTotal(price: number, qty: number): number {
  return `${price * qty}`;
}
lineTotal(499);
lineTotal(499, "3");
function greet(name) { return "Hi " + name; }
const total: string = lineTotal(1, 2);
```

COMPILER ERRORS (tsc --strict)

```
main.ts(2,3): error TS2322: Type 'string' is not assignable to type 'number'.
main.ts(4,1): error TS2554: Expected 2 arguments, but got 1.
main.ts(5,16): error TS2345: Argument of type 'string' is not assignable to parameter of type
'number'.
main.ts(6,16): error TS7006: Parameter 'name' implicitly has an 'any' type.
main.ts(7,7): error TS2322: Type 'number' is not assignable to type 'string'.
```

23.4.2 Optional and default parameters

A parameter followed by `?` is **optional**: callers may omit it, and its type includes `undefined`, so the function must handle that. A parameter with a **default value** is also optional, and its type is inferred from the default; inside the function it is never `undefined`. Optional and default parameters must come after required ones (unless you pass `undefined` explicitly). For many optional settings, a destructured options object with defaults (Module 12) is clearer, and it is easy to type.

OPTIONAL, DEFAULT AND OPTIONS-OBJECT PARAMETERS main.js

```
function greet(name: string, title?: string): string {
  return title ? `Hello, ${title} ${name}` : `Hello, ${name}`;   // title: string | undefined
}

function paginate<T>(items: T[], page = 1, size = 2): T[] {         // page and size inferred as
number
  return items.slice((page - 1) * size, page * size);
}

interface ButtonOptions { label: string; variant?: "primary" | "ghost"; disabled?: boolean }
function button({ label, variant = "primary", disabled = false }: ButtonOptions): string {
  return `<button class="${variant}"${disabled ? " disabled" : ""}>${label}</button>`;
}

console.log(greet("Rao"), "|", greet("Rao", "Dr."));
const names = ["Asha", "Ravi", "Meera", "John", "Zoya"];
console.log(paginate(names), paginate(names, 2), paginate(names, 2, 3));
console.log(button({ label: "Save" }), button({ label: "Cancel", variant: "ghost", disabled: true }));
```

OUTPUT

```
Hello, Rao | Hello, Dr. Rao
[ 'Asha', 'Ravi' ] [ 'Meera', 'John' ] [ 'John', 'Zoya' ]
<button class="primary">Save</button> <button class="ghost" disabled>Cancel</button>
```

23.5 Classes in TypeScript

23.5.1 Access modifiers and parameter properties

TypeScript classes add type annotations for fields, plus **access modifiers**: `public` (the default; accessible everywhere), `private` (only inside the class), `protected` (inside the class and its subclasses),

and `readonly` (assignable only in the declaration or constructor). These are checked at **compile time only**; JavaScript's own `#private` fields (Module 13) are enforced at run time, and both can be used. **Parameter properties** declare and initialise a field directly in the constructor's parameter list (`constructor(private balance: number)`), removing boilerplate. Classes can `implements` interfaces, and `abstract` classes and methods define bases that must be extended. Strict mode also requires every field to be initialised.

ACCESS MODIFIERS, PARAMETER PROPERTIES AND ABSTRACT CLASSES main.js

```
interface Account {
  readonly number: string;
  deposit(amount: number): void;
  balance: number;
}

abstract class BaseAccount implements Account {
  private static nextNo = 1001;
  readonly number: string;
  protected history: string[] = [];

  constructor(public readonly owner: string, private _balance = 0) {   // parameter properties
    this.number = `LBF${BaseAccount.nextNo++}`;
  }
  get balance(): number { return this._balance; }
  deposit(amount: number): void {
    if (amount <= 0) throw new RangeError("Amount must be positive");
    this._balance += amount;
    this.history.push(`+${amount}`);
  }
  protected take(amount: number): void { this._balance -= amount; this.history.push(`-${amount}`); }
  abstract withdraw(amount: number): void;                // subclasses must implement
  statement(): string { return `${this.number} ${this.owner}: Rs ${this._balance}
[${this.history.join(" ")}]`; }
}

class SavingsAccount extends BaseAccount {
  constructor(owner: string, opening: number, private readonly minBalance = 1000) { super(owner,
opening); }
  withdraw(amount: number): void {
    if (this.balance - amount < this.minBalance) throw new Error(`Minimum balance Rs
${this.minBalance} required`);
    this.take(amount);                                    // protected member from the base class
  }
}

const acc = new SavingsAccount("Asha", 5000);
acc.deposit(2000);
acc.withdraw(3000);
try { acc.withdraw(3500); } catch (e) { console.log((e as Error).message); }
console.log(acc.statement());
console.log("owner is public:", acc.owner, "| balance via getter:", acc.balance);
```

OUTPUT

```
Minimum balance Rs 1000 required
LBF1001 Asha: Rs 4000 [+2000 -3000]
owner is public: Asha | balance via getter: 4000
```

WHAT THE COMPILER PROTECTS

main.js

```
class Wallet {
  private balance = 0;
  protected log: string[] = [];
  readonly id: string;
  constructor(id: string) { this.id = id; }
}
class Child extends Wallet {
  peek() { return this.balance; }
}
const w = new Wallet("W1");
w.balance = 1000;
w.log.push("x");
w.id = "W2";
class Broken { name: string; }
```

COMPILER ERRORS (tsc --strict)

```
main.ts(8,24): error TS2341: Property 'balance' is private and only accessible within class 'Wallet'.
main.ts(11,3): error TS2341: Property 'balance' is private and only accessible within class 'Wallet'.
main.ts(12,3): error TS2445: Property 'log' is protected and only accessible within class 'Wallet' and
its subclasses.
main.ts(13,3): error TS2540: Cannot assign to 'id' because it is a read-only property.
main.ts(14,16): error TS2564: Property 'name' has no initializer and is not definitely assigned in the
constructor.
```

23.6 Generics

23.6.1 Generic functions, interfaces and constraints

Generics let you write code that works with many types while keeping type information, instead of losing it with `any`. A **type parameter** in angle brackets, conventionally `T`, stands for a type chosen by the caller: `function first<T>(items: T[]): T | undefined` returns a `number` for a `number[]` and a `Student` for a `Student[]`. TypeScript usually infers `T` from the arguments. Interfaces, type aliases and classes can be generic too (`ApiResponse<T>`, `Repository<T>`), and built-ins already are: `Array<T>`, `Promise<T>`, `Map<K, V>`. **Constraints** (`T extends { id: number }`) require the type to have certain members, and `K extends keyof T` restricts a parameter to the property names of another type.

GENERIC FUNCTIONS, INTERFACES, CONSTRAINTS AND CLASSES

main.js

```
function first<T>(items: T[]): T | undefined {
  return items[0];
}
const n = first([10, 20]);                // number | undefined
const s = first(["a", "b"]);              // string | undefined

interface ApiResponse<T> {
  ok: boolean;
  data: T;
  error?: string;
}
type Course = { id: number; title: string };
const res: ApiResponse<Course[]> = { ok: true, data: [{ id: 1, title: "TypeScript" }] };

// constraint: T must have an id
function findById<T extends { id: number }>(items: T[], id: number): T | undefined {
  return items.find((item) => item.id === id);
}

// keyof constraint: key must be a property name of T, and the result type follows it
function pluck<T, K extends keyof T>(items: T[], key: K): T[K][] {
  return items.map((item) => item[key]);
}

// a generic class
class Repository<T extends { id: number }> {
  private items = new Map<number, T>();
  save(item: T): T { this.items.set(item.id, item); return item; }
  get(id: number): T | undefined { return this.items.get(id); }
  all(): T[] { return [...this.items.values()]; }
}

const students = [{ id: 1, name: "Asha", marks: 91 }, { id: 2, name: "Ravi", marks: 64 }];
const repo = new Repository<(typeof students)[number]>();
students.forEach((st) => repo.save(st));

console.log(n, s, res.data[0]?.title);
console.log(findById(students, 2)?.name, pluck(students, "marks"), pluck(students, "name"));
console.log(repo.all().length, repo.get(1)?.name);
```

OUTPUT

```
10 a TypeScript
Ravi [ 91, 64 ] [ 'Asha', 'Ravi' ]
2 Asha
```

CONSTRAINTS IN ACTION main.js

```
function findById<T extends { id: number }>(items: T[], id: number): T | undefined {
  return items.find((i) => i.id === id);
}
findById([{ name: "no id" }], 1);

function pluck<T, K extends keyof T>(items: T[], key: K): T[K][] {
  return items.map((i) => i[key]);
}
pluck([{ id: 1, name: "Asha" }], "email");

const marks: number[] = pluck([{ id: 1, name: "Asha" }], "name");
```

COMPILER ERRORS (tsc --strict)

```
main.ts(4,13): error TS2353: Object literal may only specify known properties, and 'name' does not
exist in type '{ id: number; }'.
main.ts(9,34): error TS2345: Argument of type '"email"' is not assignable to parameter of type '"name"
| "id"'.
main.ts(11,7): error TS2322: Type 'string[]' is not assignable to type 'number[]'.
  Type 'string' is not assignable to type 'number'.
```

23.7 Narrowing

23.7.1 typeof, in, instanceof and discriminated unions

Narrowing is how TypeScript refines a broad type (a union, `unknown`) to a more specific one inside a block, by following your ordinary JavaScript checks. `typeof x === "string"` narrows primitives; truthiness and equality checks remove `null` and `undefined`; `"prop" in obj` narrows by property presence; `x instanceof Date` narrows to a class; `Array.isArray` to an array. A **discriminated union** is a union of object types that share a literal-typed property (the **discriminant**, such as `kind` or `status`); checking it in an `if` or `switch` narrows to exactly one member. Assigning the leftover value to `never` in the `default` case gives an **exhaustiveness check**: add a new member to the union and the compiler points to every switch that does not handle it. Custom **type guards** (`value is Type`) package a check into a reusable function.

NARROWING A UNION STEP BY STEP main.js

```
// typeof, truthiness, instanceof, in
function show(value: string | number | Date | null): string {
  if (value === null) return "nothing";
  if (typeof value === "string") return value.toUpperCase();
  if (typeof value === "number") return value.toFixed(2);
  return value.toISOString().slice(0, 10);                // only Date is left
}

type Admin = { name: string; permissions: string[] };
type Learner = { name: string; courses: string[] };
function access(user: Admin | Learner): string {
  return "permissions" in user ? `${user.name}: ${user.permissions.join(", ")}` : `${user.name}
studies ${user.courses.length} course(s)`;
}

// a discriminated union with an exhaustiveness check
type Payment =
  | { method: "upi"; vpa: string }
  | { method: "card"; last4: string; brand: "visa" | "rupay" }
  | { method: "cash" };
```

```
function describe(p: Payment): string {
  switch (p.method) {
    case "upi": return `UPI to ${p.vpa}`;                  // p is the upi member here
    case "card": return `${p.brand.toUpperCase()} card ending ${p.last4}`;
    case "cash": return "cash on delivery";
    default: {
      const unreachable: never = p;                       // compile error if a case is missing
      return unreachable;
    }
  }
}

// a custom type guard
type Course = { id: number; title: string };
function isCourse(x: unknown): x is Course {
  return typeof x === "object" && x !== null && typeof (x as Course).id === "number" && typeof (x as
Course).title === "string";
}

console.log(show("hi"), show(3.14159), show(new Date(Date.UTC(2026, 2, 9))), show(null));
console.log(access({ name: "Ravi", permissions: ["read", "delete"] }), "|", access({ name: "Asha",
courses: ["TS"] }));
for (const p of [{ method: "upi", vpa: "asha@okbank" }, { method: "card", last4: "4242", brand:
"rupay" }, { method: "cash" }] satisfies Payment[]) console.log(describe(p));
const data: unknown[] = JSON.parse('[{"id":1,"title":"TS"},{"id":"x"}]');
console.log("valid courses:", data.filter(isCourse).map((c) => c.title));
```

OUTPUT

```
HI 3.14 2026-03-09 nothing
Ravi: read, delete | Asha studies 1 course(s)
UPI to asha@okbank
RUPAY card ending 4242
cash on delivery
valid courses: [ 'TS' ]
```

THE EXHAUSTIVENESS CHECK FINDS THE NEW CASE

main.js

```
type Payment =
  | { method: "upi"; vpa: string }
  | { method: "card"; last4: string }
  | { method: "netbanking"; bank: string };

function describe(p: Payment): string {
  switch (p.method) {
    case "upi": return `UPI to ${p.vpa}`;
    case "card": return `card ${p.last4}`;
    default: {
      const unreachable: never = p;
      return unreachable;
    }
  }
}

function len(x: string | number) {
  return x.length;
}
```

COMPILER ERRORS (tsc --strict)

```
main.ts(11,13): error TS2322: Type '{ method: "netbanking"; bank: string; }' is not assignable to type
'never'.
main.ts(18,12): error TS2339: Property 'length' does not exist on type 'string | number'.
  Property 'length' does not exist on type 'number'.
```

23.8 Utility types

23.8.1 Partial, Required, Pick, Omit, Record, Readonly, ReturnType

TypeScript ships generic **utility types** that transform existing types, so you derive new types instead of repeating definitions (when the original changes, the derived types follow). `Partial<T>` makes every property optional (update payloads); `Required<T>` makes all required; `Readonly<T>` makes all read-only; `Pick<T, K>` keeps only some properties and `Omit<T, K>` removes some (a public view without the password); `Record<K, V>` builds an object type with keys K and values V; `ReturnType<F>` and `Parameters<F>` extract a function's return and parameter types; `Awaited<T>` unwraps promises; and `NonNullable`, `Exclude` and `Extract` filter unions.

DERIVING TYPES WITH THE UTILITY TYPES

main.js

```
interface User {
  id: number;
  name: string;
  email: string;
  passwordHash: string;
  role: "admin" | "student";
  phone?: string;
}

type UserUpdate = Partial<Omit<User, "id">>;           // any subset of fields except id
type PublicUser = Omit<User, "passwordHash">;           // safe to send to clients
type UserSummary = Pick<User, "id" | "name">;
type CompleteUser = Required<User>;                     // phone becomes required
type FrozenUser = Readonly<User>;
type RoleCounts = Record<User["role"], number>;         // { admin: number; student: number }

function updateUser(user: User, changes: UserUpdate): User {
  return { ...user, ...changes };
}
function toPublic({ passwordHash, ...rest }: User): PublicUser {
  return rest;
}
function createToken(user: UserSummary, ttlSeconds = 3600) {
  return { sub: user.id, name: user.name, exp: 1_773_054_000 + ttlSeconds };
}
type Token = ReturnType<typeof createToken>;            // derived from the function itself
type TokenArgs = Parameters<typeof createToken>;        // [user: UserSummary, ttlSeconds?: number]
type Loaded = Awaited<Promise<User[]>>;                 // User[]
type NonAdminRole = Exclude<User["role"], "admin">;     // "student"

const u: User = { id: 7, name: "Asha", email: "asha@lbf.in", passwordHash: "scrypt$...", role:
"student" };
const updated = updateUser(u, { phone: "+91 98450 12345" });
const token: Token = createToken(u);
const counts: RoleCounts = { admin: 1, student: 42 };
const args: TokenArgs = [{ id: 1, name: "Admin" }, 60];
const role: NonAdminRole = "student";
const list: Loaded = [u];
const frozen: FrozenUser = u;
console.log(toPublic(updated));
console.log(token, counts, args[1], role, list.length, frozen.name);
```

OUTPUT

```
{
  id: 7,
  name: 'Asha',
  email: 'asha@lbf.in',
  role: 'student',
  phone: '+91 98450 12345'
}
{ sub: 7, name: 'Asha', exp: 1773057600 } { admin: 1, student: 42 } 60 student 1 Asha
```

UTILITY TYPES ENFORCE THE DERIVED SHAPES

main.js

```
interface User { id: number; name: string; passwordHash: string; phone?: string }
type PublicUser = Omit<User, "passwordHash">;
const p: PublicUser = { id: 1, name: "Asha" };
console.log(p.passwordHash);

const frozen: Readonly<User> = { id: 1, name: "Asha", passwordHash: "x" };
frozen.name = "Ravi";

const full: Required<User> = { id: 1, name: "Asha", passwordHash: "x" };
```

COMPILER ERRORS (tsc --strict)

```
main.ts(4,15): error TS2339: Property 'passwordHash' does not exist on type 'PublicUser'.
main.ts(7,8): error TS2540: Cannot assign to 'name' because it is a read-only property.
main.ts(9,7): error TS2741: Property 'phone' is missing in type '{ id: number; name: string;
passwordHash: string; }' but required in type 'Required<User>'.
```

Utility type	Result	Typical use
`Partial<T>`	all properties optional	PATCH bodies, form drafts
`Required<T>`	all properties required	after defaults are applied
`Readonly<T>`	all properties read-only	state, configuration
`Pick<T, K>`	only properties K	summaries, list items
`Omit<T, K>`	all except K	public views, create inputs without id
`Record<K, V>`	object with keys K, values V	lookup tables, counts
`ReturnType<F>` / `Parameters<F>`	a function's return / parameter types	reuse types of existing functions
`Awaited<T>`	type inside a promise	results of async functions
`Exclude` / `Extract` / `NonNullable`	filtered unions	removing null or some members

23.9 Using TypeScript with Node.js and Vite

Node.js: install `typescript` and `@types/node` (type declarations for Node's APIs) as dev dependencies, use `"module": "nodenext"` in tsconfig, compile with `tsc` to `dist/` and run `node dist/server.js` in production. During development, run TypeScript directly: **tsx** (`npx tsx watch src/server.ts`) is popular, and recent Node.js versions strip types natively (`node src/server.ts`) as long as the code uses only erasable syntax. Packages written in JavaScript get types from **DefinitelyTyped** (`npm i -D @types/express`); many modern packages (Zod, Prisma, Axios) ship their own. **Vite**: `npm create vite@latest my-app -- --template vanilla-ts` (or `react-ts`) creates a ready project. Vite compiles .ts files very quickly with esbuild but does **not** type-check, so the build script runs `tsc --noEmit && vite build` and the editor shows errors as you type. Environment variables can be typed in `vite-env.d.ts`.

PACKAGE.JSON JavaScript

```
{
  "name": "lbf-api-ts",
  "type": "module",
  "scripts": {
    "dev": "node --watch src/server.ts",
    "typecheck": "tsc --noEmit",
    "build": "tsc",
    "start": "node dist/server.js"
  }
}
```

SRC/SERVER.TS TypeScript

```
import type { Student, NewStudent } from "./types.ts";

const students: Student[] = [];
let nextId = 1;

function addStudent(input: NewStudent): Student {
  const student: Student = { id: nextId++, ...input };
  students.push(student);
  return student;
}

addStudent({ name: "Asha", course: "TypeScript", marks: 91 });
addStudent({ name: "Ravi", course: "JavaScript", marks: 64 });
const top = students.toSorted((a, b) => b.marks - a.marks)[0];
console.log(`${students.length} students; top: ${top?.name} (${top?.marks})`);
```

RUNNING TYPESCRIPT DIRECTLY IN NODE.JS AND TYPE-CHECKING SEPARATELY main.js

```
node --experimental-strip-types --no-warnings src/server.ts
tsc --noEmit -p . && echo "type check passed"
```

TERMINAL

```
$ node --experimental-strip-types --no-warnings src/server.ts
2 students; top: Asha (91)
$ tsc --noEmit -p . && echo "type check passed"
type check passed
```

TYPING VITE ENVIRONMENT VARIABLES TypeScript

```
// src/vite-env.d.ts in a Vite + TypeScript project
/// <reference types="vite/client" />
interface ImportMetaEnv {
  readonly VITE_API_URL: string;
  readonly VITE_ENABLE_CHAT?: "true" | "false";
}
interface ImportMeta { readonly env: ImportMetaEnv }

// src/main.ts can now use import.meta.env.VITE_API_URL with full type checking
// package.json scripts: "dev": "vite", "build": "tsc --noEmit && vite build", "preview": "vite
preview"
```

TYPED EXPRESS HANDLERS TypeScript

```
// typing Express with @types/express
import express, { type Request, type Response, type NextFunction } from "express";

interface StudentParams { id: string }
interface StudentBody { name: string; course: string; marks?: number }

const app = express();
app.use(express.json());

app.post("/api/students/:id", (req: Request<StudentParams, unknown, StudentBody>, res: Response) => {
  const id = Number(req.params.id);          // params are strings
  const { name, marks = 0 } = req.body;      // body is typed
  res.status(201).json({ id, name, marks });
});

// augmenting Express's Request so req.user is known everywhere (declaration merging)
declare global {
  namespace Express { interface Request { user?: { sub: string; role: "admin" | "student" } } }
}
app.use((req: Request, res: Response, next: NextFunction) => { req.user = { sub: "7", role: "student"
}; next(); });
```

TypeScript is a typed superset of JavaScript: tsc checks types at compile time and emits plain JavaScript with the types erased; tsconfig.json (strict, target, module, outDir) configures it. Basic types include string, number, boolean, arrays and tuples; unknown is the safe alternative to any, void marks no return value, never marks impossible values; inference fills in most types, as const narrows them and satisfies checks without widening. Object types are named with type aliases or interfaces (extends, declaration merging, index signatures), with optional (?) and readonly properties; unions (|) model alternatives and intersections (&) combine shapes; literal unions and enums model fixed sets. Functions type parameters, returns, callbacks, overloads, optional and default parameters. Classes add public/private/protected/readonly, parameter properties, implements and abstract. Generics with constraints (extends, keyof) keep type information across reusable code. Narrowing uses typeof, truthiness, in, instanceof, discriminated unions with never-based exhaustiveness checks, and custom type guards. Utility types (Partial, Required, Readonly, Pick, Omit, Record, ReturnType, Parameters, Awaited) derive types. Use @types packages, tsx or native type stripping in Node.js, and tsc --noEmit alongside Vite.

PRACTICE CHECKPOINT

Convert the **Student Management API** or the **weather app** to TypeScript with **strict mode** enabled and **no any types**.

The sample solution converts the weather app's modules from Module 19. External data enters as `unknown` and is validated by a type guard before use, so no `any` appears anywhere; storage access is typed through a small interface so it can be tested with an in-memory fake.

THE WEATHER APP IN STRICT TYPESCRIPT (ALL MODULES IN ONE FILE FOR THE DEMO) main.js

```
// ---- types.ts ----
interface Weather { city: string; temp: number; description: string; humidity: number }
interface KeyValueStore { getItem(key: string): string | null; setItem(key: string, value: string):
void }
type Result<T> = { ok: true; value: T } | { ok: false; error: string };
```

```
// ---- validation.ts: unknown in, Weather out ----
function isWeather(x: unknown): x is Weather {
  if (typeof x !== "object" || x === null) return false;
  const w = x as Record<string, unknown>;
  return typeof w.city === "string" && typeof w.temp === "number" && typeof w.description === "string"
&& typeof w.humidity === "number";
}

// ---- api.ts ----
type Fetcher = (url: string) => Promise<{ ok: boolean; status: number; json(): Promise<unknown> }>;
async function fetchWeather(city: string, fetcher: Fetcher, baseUrl = "/api/weather"):
Promise<Result<Weather>> {
  try {
    const res = await fetcher(`${baseUrl}?${new URLSearchParams({ city })}`);
    if (res.status === 404) return { ok: false, error: `City "${city}" was not found` };
    if (!res.ok) return { ok: false, error: `Weather service error (${res.status})` };
    const data: unknown = await res.json();
    return isWeather(data) ? { ok: true, value: data } : { ok: false, error: "Unexpected response
format" };
  } catch (e: unknown) {
    return { ok: false, error: e instanceof Error ? e.message : String(e) };
  }
}

// ---- recent.ts ----
const KEY = "recentCities";
const MAX_RECENT = 5;
function loadRecent(store: KeyValueStore): string[] {
  try {
    const parsed: unknown = JSON.parse(store.getItem(KEY) ?? "[]");
    return Array.isArray(parsed) ? parsed.filter((c): c is string => typeof c === "string") : [];
  } catch {
    return [];
  }
}
function addRecent(city: string, store: KeyValueStore): string[] {
  const list = [city, ...loadRecent(store).filter((c) => c.toLowerCase() !==
city.toLowerCase())].slice(0, MAX_RECENT);
  store.setItem(KEY, JSON.stringify(list));
  return list;
}

// ---- demo with a fake server and an in-memory store ----
const fakeData: Record<string, unknown> = {
  Mysuru: { city: "Mysuru", temp: 27, description: "partly cloudy", humidity: 61 },
  Delhi: { city: "Delhi", temp: "34", description: "haze", humidity: 38 },        // wrong type from
the server
};
const fakeFetch: Fetcher = async (url) => {
  const city = new URL(url, "http://x").searchParams.get("city") ?? "";
  const body = fakeData[city];
  return { ok: body !== undefined, status: body === undefined ? 404 : 200, json: async () => body };
};
const memory = new Map<string, string>();
const store: KeyValueStore = { getItem: (k) => memory.get(k) ?? null, setItem: (k, v) => void
memory.set(k, v) };

async function search(city: string): Promise<void> {
  const result = await fetchWeather(city, fakeFetch);
  if (result.ok) {
    const w = result.value;                                    // narrowed to Weather
    console.log(`${w.city}: ${w.temp}°C, ${w.description} | recent: ${addRecent(w.city, store).join(",
")}`);
  } else {
```

```
    console.log(`${city}: ${result.error}`);
  }
}
(async () => { for (const c of ["Mysuru", "Delhi", "Atlantis", "Mysuru"]) await search(c); })();
```

OUTPUT

```
Mysuru: 27°C, partly cloudy | recent: Mysuru
Delhi: Unexpected response format
Atlantis: City "Atlantis" was not found
Mysuru: 27°C, partly cloudy | recent: Mysuru
```

MODULE 24 · ADVANCED

Testing, security and best practices

Minimum duration **3 days** Level **Advanced**

Professional JavaScript is tested, secure and written to shared standards. This module covers the kinds of automated tests, unit testing with Jest or Vitest (suites, matchers, mocks), testing asynchronous and DOM code, end-to-end testing with Playwright, coverage and test-driven development; the main web security risks (XSS, CSRF), Content Security Policy, secure token storage and dependency vulnerabilities; clean-code principles (naming, small functions, DRY, KISS, SOLID) and code reviews; and the basics of accessibility: keyboard access, ARIA and focus management.

IN THIS MODULE

24.1 Testing
24.2 Web security
24.3 Clean code
24.4 Accessibility basics

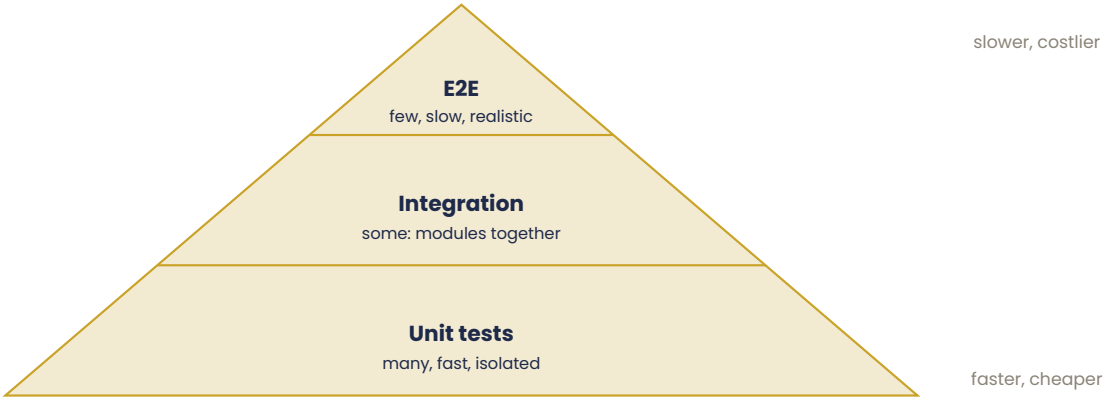
NOTE

Tests in this module were really run: unit tests with Node.js's built-in test runner (whose API is very close to Jest and Vitest), and end-to-end tests with Playwright in headless Chromium. Vitest and Jest themselves could not be installed in the typesetting environment, so their examples show the output they print for the same tests.

24.1 Testing

24.1.1 Unit, integration and end-to-end testing

An **automated test** is code that runs your code and checks the result, so that bugs are caught in seconds rather than by users, and so that you can change code confidently: if the tests still pass, you did not break existing behaviour (**regression** protection). There are three main levels. **Unit tests** check one small piece (a function, a class) in isolation, with dependencies replaced by fakes; they are fast and numerous. **Integration tests** check that several pieces work together: an API route with its database, a component with its store. **End-to-end (E2E) tests** drive the real application in a real browser like a user would: open the page, type, click, check what appears. They give the most confidence but are slower and more fragile. The **testing pyramid** recommends many unit tests, fewer integration tests and a few E2E tests for the critical journeys (sign-up, checkout).



THE TESTING PYRAMID

Level	Checks	Tools	Speed
Unit	one function or class, dependencies mocked	Vitest, Jest, node:test	milliseconds
Integration	several units together: API + DB, component + store	Vitest/Jest + Supertest, Testing Library	tens to hundreds of ms
End-to-end	the whole app in a browser, like a user	Playwright, Cypress	seconds per test

24.1.2 Jest or Vitest: test suites, matchers, mocks

Jest (from Meta) and **Vitest** (from the Vite team, Jest-compatible and much faster in Vite projects) are the most used JavaScript test frameworks; Node.js's built-in `node:test` follows the same ideas. Tests live in files named `*.test.js` or `*.spec.js`. `describe` groups related tests into a **suite**; `test` (or `it`) defines one test case; each test follows **Arrange, Act, Assert**. **Matchers** express the expectation: `expect(x).toBe(y)` (strict equality), `toEqual` (deep equality for objects and arrays), `toBeCloseTo` (floats), `toContain`, `toMatch` (regex), `toThrow`, `toHaveLength`, `toBeTruthy` and `.not`. Hooks (`beforeEach`, `afterEach`, `beforeAll`) set up and clean up. **Mocks** replace real dependencies (network, time, database, random numbers) with controllable fakes: `vi.fn()` / `jest.fn()` records calls, `mockReturnValue` sets results, `vi.mock("./api")` replaces a whole module, and fake timers control `setTimeout` and `Date`.

A VITEST TEST SUITE

main.js

```
// src/cart.js
export function cartTotal(items, { couponPercent = 0, taxRate = 0.18 } = {}) {
  if (!Array.isArray(items)) throw new TypeError("items must be an array");
  const subtotal = items.reduce((t, i) => t + i.price * i.qty, 0);
  const discounted = subtotal * (1 - couponPercent / 100);
  return Math.round(discounted * (1 + taxRate) * 100) / 100;
}

// src/cart.test.js
import { describe, test, expect } from "vitest";      // with Jest: globals, no import needed
import { cartTotal } from "./cart.js";

describe("cartTotal", () => {
  test("adds tax to the subtotal", () => {
    // Arrange
    const items = [{ price: 100, qty: 2 }, { price: 50, qty: 1 }];
    // Act
    const total = cartTotal(items);
    // Assert
    expect(total).toBe(295);
  });
  test("applies a coupon before tax", () => {
    expect(cartTotal([{ price: 1000, qty: 1 }], { couponPercent: 10 })).toBe(1062);
  });
  test("an empty cart costs nothing", () => expect(cartTotal([])).toBe(0));
  test("rejects invalid input", () => {
    expect(() => cartTotal(null)).toThrow(TypeError);
    expect(() => cartTotal("pen")).toThrow(/must be an array/);
  });
});
```

```
$ npx vitest run
 ✓ src/cart.test.js (4 tests) 3ms
   ✓ cartTotal > adds tax to the subtotal
   ✓ cartTotal > applies a coupon before tax
   ✓ cartTotal > an empty cart costs nothing
   ✓ cartTotal > rejects invalid input

 Test Files  1 passed (1)
      Tests  4 passed (4)
   Duration  212ms
```

The same tests, written with `node:test` and `node:assert`, really run below, together with a deliberately failing test to show what a failure report looks like.

CART.TEST.JS JavaScript

```
import { describe, test } from "node:test";
import assert from "node:assert/strict";
import { cartTotal } from "./cart.js";

describe("cartTotal", () => {
  test("adds tax to the subtotal", () => {
    assert.equal(cartTotal([{ price: 100, qty: 2 }, { price: 50, qty: 1 }]), 295);
  });
  test("applies a coupon before tax", () => {
    assert.equal(cartTotal([{ price: 1000, qty: 1 }], { couponPercent: 10 }), 1062);
  });
  test("returns line items unchanged (deep equality)", () => {
    const items = [{ price: 10, qty: 1 }];
    cartTotal(items);
    assert.deepEqual(items, [{ price: 10, qty: 1 }]);
  });
  test("rejects invalid input", () => {
    assert.throws(() => cartTotal(null), TypeError);
  });
  test("a wrong expectation fails loudly", () => {
    assert.equal(cartTotal([{ price: 99, qty: 1 }]), 99);       // forgot the tax: this test is wrong
  });
});
```

RUNNING A SUITE WITH NODE:TEST (INCLUDING ONE FAILURE) main.js

```
node --test --test-reporter=spec
```

TERMINAL

```
$ node --test --test-reporter=spec
▶ cartTotal
  ✔ adds tax to the subtotal (0.761477ms)
  ✔ applies a coupon before tax (0.246455ms)
  ✔ returns line items unchanged (deep equality) (1.400371ms)
  ✔ rejects invalid input (1.568791ms)
  ✖ a wrong expectation fails loudly (1.084235ms)
✖ cartTotal (6.249552ms)
ℹ tests 5
ℹ suites 1
ℹ pass 4
ℹ fail 1
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 104.890087

✖ failing tests:

test at cart.test.js:20:3
✖ a wrong expectation fails loudly (1.084235ms)
  AssertionError [ERR_ASSERTION]: Expected values to be strictly equal:

  116.82 !== 99

      at TestContext.<anonymous> (file://cart.test.js:21:12)
```

Jest / Vitest	node:test + assert	Checks
`expect(a).toBe(b)`	`assert.equal(a, b)` (strict)	same value (Object.is)

Jest / Vitest	node:test + assert	Checks
`expect(a).toEqual(b)`	`assert.deepEqual(a, b)`	same structure and contents
`expect(fn).toThrow(Err)`	`assert.throws(fn, Err)`	throws
`await expect(p).rejects.toThrow()`	`await assert.rejects(p)`	promise rejects
`expect(s).toMatch(/re/)`	`assert.match(s, /re/)`	regex match
`expect(a).toBeCloseTo(0.3)`	`assert.ok(Math.abs(a - 0.3) < 1e-9)`	floats
`vi.fn()` / `jest.fn()`	`mock.fn()`	a spy function
`vi.useFakeTimers()`	`mock.timers.enable()`	control time

Mocks isolate the unit under test. Below, a sign-up service depends on a user repository and an email sender; both are replaced with mock functions, so the test runs without a database or email server, and can check **how** the dependencies were called. Passing dependencies in as parameters (**dependency injection**) makes this easy.

SIGNUP.JS

JavaScript

```
export function createSignupService({ users, mailer, now = () => new Date() }) {
  return {
    async signUp(email) {
      if (await users.findByEmail(email)) throw new Error("Email already registered");
      const user = await users.insert({ email, createdAt: now().toISOString() });
      await mailer.send(email, "Welcome to LBF", `Hi! Your id is ${user.id}`);
      return user;
    },
  };
}
```

SIGNUP.TEST.JS

JavaScript

```
import { test, mock, beforeEach } from "node:test";
import assert from "node:assert/strict";
import { createSignupService } from "./signup.js";

let users, mailer, service;
beforeEach(() => {
  users = {
    findByEmail: mock.fn(async () => null),                    // a fake: no existing user
    insert: mock.fn(async (u) => ({ id: 7, ...u })),
  };
  mailer = { send: mock.fn(async () => true) };
  service = createSignupService({ users, mailer, now: () => new Date("2026-03-09T04:30:00Z") });
});

test("creates the user and sends one welcome email", async () => {
  const user = await service.signUp("asha@lbf.in");
  assert.deepEqual(user, { id: 7, email: "asha@lbf.in", createdAt: "2026-03-09T04:30:00.000Z" });
  assert.equal(mailer.send.mock.callCount(), 1);
  assert.deepEqual(mailer.send.mock.calls[0].arguments.slice(0, 2), ["asha@lbf.in", "Welcome to
LBF"]);
});

test("rejects duplicates without inserting or emailing", async () => {
  users.findByEmail.mock.mockImplementation(async () => ({ id: 1 }));
  await assert.rejects(service.signUp("asha@lbf.in"), /already registered/);
  assert.equal(users.insert.mock.callCount(), 0);
  assert.equal(mailer.send.mock.callCount(), 0);
});
```

MOCKS AND SPIES

main.js

```
node --test --test-reporter=spec
```

TERMINAL

```
$ node --test --test-reporter=spec
✔ creates the user and sends one welcome email (11.560259ms)
✔ rejects duplicates without inserting or emailing (0.761108ms)
ℹ tests 2
ℹ suites 0
ℹ pass 2
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 412.120304
```

24.1.3 Testing async code

Test functions can be `async`: the runner waits for the returned promise, so you simply `await` the code under test. Check rejections with `await expect(promise).rejects.toThrow()` (Jest/Vitest) or `await assert.rejects(promise)`. A classic bug is forgetting `await` or `return`: the test finishes before the assertion runs and passes even when the code is broken. Timers should not make tests slow or flaky: **fake timers** let a test advance time instantly (`vi.advanceTimersByTime(300)`, `mock.timers.tick(300)`). Network calls are mocked (a fake `fetch`, or a library such as MSW, Mock Service Worker).

WEATHER.TEST.JS

JavaScript

```
import { test, mock } from "node:test";
import assert from "node:assert/strict";
import { getTemperature, debounce } from "./weather.js";

const fakeFetch = (status, body) => mock.fn(async () => ({ ok: status < 400, status, json: async () =>
body }));

test("resolves with the temperature", async () => {
  const f = fakeFetch(200, { temp: 27 });
  assert.equal(await getTemperature("Mysuru", f), 27);
  assert.equal(f.mock.calls[0].arguments[0], "/api/weather?city=Mysuru");
});

test("rejects on HTTP errors", async () => {
  await assert.rejects(getTemperature("Atlantis", fakeFetch(404, {})), { message: "HTTP 404" });
});

test("debounce calls once after the pause (fake timers, no real waiting)", () => {
  mock.timers.enable({ apis: ["setTimeout"] });
  const spy = mock.fn();
  const search = debounce(spy, 300);
  search("j"); search("ja"); search("java");
  mock.timers.tick(299);
  assert.equal(spy.mock.callCount(), 0);
  mock.timers.tick(1);
  assert.equal(spy.mock.callCount(), 1);
  assert.deepEqual(spy.mock.calls[0].arguments, ["java"]);
  mock.timers.reset();
});
```

ASYNC FUNCTIONS, FAKE FETCH AND FAKE TIMERS

main.js

```
node --test --test-reporter=spec
```

TERMINAL

```
$ node --test --test-reporter=spec
✔ resolves with the temperature (1.179024ms)
✔ rejects on HTTP errors (13.700056ms)
✔ debounce calls once after the pause (fake timers, no real waiting) (1.277226ms)
ℹ tests 3
ℹ suites 0
ℹ pass 3
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 273.143082
```

24.1.4 Testing DOM code (Testing Library, overview)

Front-end unit and component tests usually run in Node.js with a simulated DOM (**jsdom** or happy-dom, enabled with `environment: "jsdom"` in Vitest). **Testing Library** (`@testing-library/dom`, and its React, Vue and Angular versions) encourages testing **what the user sees and does** rather than implementation details: find elements the way a user or screen reader would (`getByRole("button", { name: "Add" })`, `getByLabelText`, `getByText`), act with `user-event` (`await user.type(input, "Milk")`, `await user.click(button)`), and assert on the resulting DOM (`toBeInTheDocument`, `toHaveTextContent` from jest-dom). Tests written this way survive refactoring and push you towards accessible markup, because queries by role and label only work when roles and labels exist.

A USER-CENTRED DOM TEST WITH TESTING LIBRARY

main.js

```
// todo.test.js  (Vitest with environment "jsdom")
import { screen } from "@testing-library/dom";
import userEvent from "@testing-library/user-event";
import "@testing-library/jest-dom/vitest";
import { mountTodoApp } from "./todo.js";

test("adds, completes and deletes a task", async () => {
  document.body.innerHTML = '<div id="app"></div>';
  mountTodoApp(document.querySelector("#app"));
  const user = userEvent.setup();

  await user.type(screen.getByLabelText("New task"), "Revise closures");
  await user.click(screen.getByRole("button", { name: "Add" }));
  expect(screen.getByRole("listitem")).toHaveTextContent("Revise closures");

  await user.click(screen.getByRole("checkbox", { name: "Revise closures" }));
  expect(screen.getByText("1 of 1 completed")).toBeInTheDocument();

  await user.click(screen.getByRole("button", { name: "Delete Revise closures" }));
  expect(screen.queryByRole("listitem")).not.toBeInTheDocument();
});
```

```
$ npx vitest run todo
 ✓ todo.test.js (1 test) 48ms
   ✓ adds, completes and deletes a task

 Test Files  1 passed (1)
      Tests  1 passed (1)
```

24.1.5 End-to-end testing with Playwright or Cypress

E2E tests run the real application in real browsers. **Playwright** (Microsoft) drives Chromium, Firefox and WebKit, runs tests in parallel, **auto-waits** for elements to be ready (no manual sleeps), and records traces, screenshots and videos of failures; `npm init playwright@latest` sets it up, `npx playwright test` runs it, and `npx playwright codegen` records tests by clicking through the app. **Cypress** runs inside the browser with an excellent interactive runner and time-travel debugging. Both use user-facing locators (role, label, text) and web-first assertions such as `await expect(locator).toHaveText(...)` that retry until they pass or time out. The Playwright test below really ran against the to-do application from Module 15.

TESTS/TODO.SPEC.JS JavaScript

```
const { test, expect } = require("playwright/test");

test.beforeEach(async ({ page }) => {
  await page.goto("index.html");
});

test("adds tasks and shows the count", async ({ page }) => {
  for (const task of ["Revise closures", "Build the DOM project"]) {
    await page.getByLabel("New task").fill(task);
    await page.getByRole("button", { name: "Add" }).click();
  }
  await expect(page.getByRole("listitem")).toHaveCount(2);
  await expect(page.getByText("0 of 2 completed")).toBeVisible();
});

test("completes and deletes a task", async ({ page }) => {
  await page.getByLabel("New task").fill("Read MDN");
  await page.keyboard.press("Enter");
  await page.getByRole("checkbox", { name: "Read MDN" }).check();
  await expect(page.locator("#count")).toHaveText("1 of 1 completed");
  await page.getByRole("button", { name: "Delete Read MDN" }).click();
  await expect(page.getByRole("listitem")).toHaveCount(0);
});

test("ignores empty input", async ({ page }) => {
  await page.getByRole("button", { name: "Add" }).click();
  await expect(page.getByRole("listitem")).toHaveCount(0);
});
```

PLAYWRIGHT END-TO-END TEST OF THE TO-DO APP main.js

```
npx playwright test --reporter=list
```

TERMINAL

```
$ npx playwright test --reporter=list

Running 3 tests using 1 worker

  ✓  1 tests/todo.spec.js:7:1 › adds tasks and shows the count (348ms)
  ✓  2 tests/todo.spec.js:16:1 › completes and deletes a task (243ms)
  ✓  3 tests/todo.spec.js:25:1 › ignores empty input (146ms)

  3 passed (2.0s)
```

THE SAME IDEA IN CYPRESS

main.js

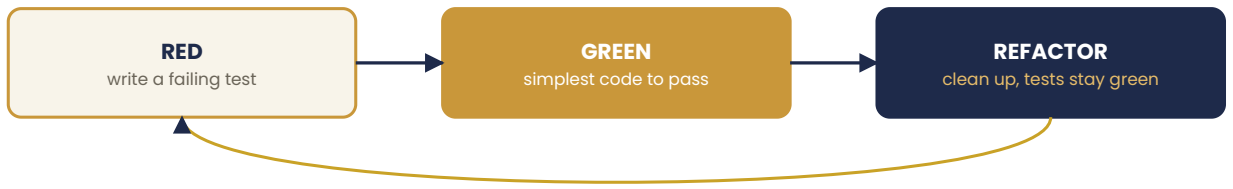
```
// cypress/e2e/todo.cy.js
describe("to-do app", () => {
  beforeEach(() => cy.visit("/"));
  it("adds a task", () => {
    cy.findByLabelText("New task").type("Revise closures{enter}");
    cy.get("li").should("have.length", 1).and("contain", "Revise closures");
  });
});
```

```
$ npx cypress run
  to-do app
    ✓ adds a task (412ms)

  1 passing (1s)
```

24.1.6 Code coverage and TDD

Code coverage measures which lines, branches and functions your tests executed (`vitest run --coverage`, `jest --coverage`, `node --test --experimental-test-coverage`). It reveals untested code, such as an error branch nobody exercised. Coverage shows what ran, not whether it was checked well, so 100% is not a goal in itself; around 70–90% for important logic is a common target, and a CI pipeline can fail when coverage drops. **Test-Driven Development** (TDD) writes the test **first**: **Red** (write a small failing test for the next bit of behaviour), **Green** (write the simplest code that passes), **Refactor** (clean up while the tests stay green), and repeat. TDD keeps designs simple and testable and gives you a full regression suite as a side effect.



THE TDD CYCLE

GRADE.TEST.JS

JavaScript

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { grade } from "./grade.js";

test("grade bands", () => {
  assert.equal(grade(95), "A+");
  assert.equal(grade(80), "A");
  assert.equal(grade(65), "B");
  assert.equal(grade(30), "C");
});
```

MEASURING COVERAGE: ONE BRANCH WAS NEVER TESTED main.js

```
node --test --experimental-test-coverage --test-reporter=spec 2>&1 | grep -E "^(ℹ|✔|✖)|file|grade|all files" | grep -v "duration_ms\|cancelled\|skipped\|todo\|suites"
```

TERMINAL

```
$ node --test --experimental-test-coverage --test-reporter=spec 2>&1 | grep -E "^(ℹ|✔|✖)|file|grade|all files" | grep -v "duration_ms\|cancelled\|skipped\|todo\|suites"
✔ grade bands (1.13017ms)
ℹ tests 1
ℹ pass 1
ℹ fail 0
ℹ start of coverage report
ℹ ------------------------------------------------------------
ℹ file          | line % | branch % | funcs % | uncovered lines
ℹ ------------------------------------------------------------
ℹ grade.js      | 100.00 |    87.50 |  100.00 |
ℹ grade.test.js | 100.00 |   100.00 |  100.00 |
ℹ ------------------------------------------------------------
ℹ all files     | 100.00 |    90.00 |  100.00 |
ℹ ------------------------------------------------------------
ℹ end of coverage report
```

NOTE

Every line ran, but branch coverage for grade.js is below 100%: the range-check branch (marks outside 0–100) was never taken, a gap that line coverage alone would hide. Adding `assert.throws(() => grade(101), RangeError)` covers it.

24.2 Web security

Security is part of every developer's job. The **OWASP Top 10** lists the most critical web application risks: broken access control, cryptographic failures, injection (SQL, XSS), insecure design, security misconfiguration, vulnerable components, authentication failures, integrity failures, logging gaps and server-side request forgery. Front-end developers deal most directly with XSS, CSRF, token storage and dependencies.

24.2.1 Cross-Site Scripting (XSS) and safe DOM updates

XSS happens when an attacker manages to run their JavaScript in your page, in your users' browsers, with your origin's privileges: reading tokens in storage, acting as the user, changing the page to phish for passwords. **Stored** XSS saves the payload on the server (a comment, a profile name) and serves it to every visitor; **reflected** XSS bounces it from the URL into the response; **DOM-based** XSS happens entirely in front-end code that writes untrusted data into a dangerous sink (Module 15). Defences: treat all external data as text (`textContent`, `setAttribute` for safe attributes, `createElement`); never pass it to `innerHTML`, `outerHTML`, `insertAdjacentHTML`, `document.write`, `eval`, `new Function` or string `setTimeout`; validate URLs before putting them in `href` / `src` (block `javascript:`); sanitise when HTML is genuinely required (DOMPurify, or the new `Element.setHTML()` Sanitizer API); rely on frameworks' automatic escaping (React escapes text, but `dangerouslySetInnerHTML` and `v-html` do not); and add a Content Security Policy as a second line of defence.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<div id="unsafe"></div>
<div id="safe"></div>
```

THE SAME REVIEW RENDERED UNSAFELY AND SAFELY

main.js

```
// Data from an API or a URL: treat it as hostile
const review = {
  author: 'Ravi<img src=x onerror="console.log(\'XSS via innerHTML: stole\',
localStorage.getItem(\'token\'))">',
  website: "javascript:console.log('XSS via a javascript: link')",
  text: "Great course! <b>5 stars</b>",
};
localStorage.setItem("token", "eyJhbGciOi...secret");

// UNSAFE: string-built HTML
document.querySelector("#unsafe").innerHTML = `<p><a href="${review.website}">${review.author}</a>:
${review.text}</p>`;
document.querySelector("#unsafe a").click();                    // simulate a user clicking the name

// SAFE: build elements and set text
function safeUrl(url) {
  try { const u = new URL(url, location.href); return ["http:", "https:"].includes(u.protocol) ?
u.href : "#"; }
  catch { return "#"; }
}
const p = document.createElement("p");
const a = Object.assign(document.createElement("a"), { href: safeUrl(review.website), textContent:
review.author });
p.append(a, ": ", review.text);
document.querySelector("#safe").append(p);
document.querySelector("#safe a").click();
console.log("safe version shows the payload as harmless text; link href =",
document.querySelector("#safe a").getAttribute("href"));
```

BROWSER CONSOLE

```
safe version shows the payload as harmless text; link href = #
XSS via a javascript: link
XSS via innerHTML: stole eyJhbGciOi...secret
```

24.2.2 Cross-Site Request Forgery (CSRF)

CSRF tricks a logged-in user's browser into sending an unwanted request to your site from **another** site: a hidden form on `evil.example` that posts to `bank.example/transfer`. If your site authenticates with **cookies**, the browser attaches them automatically, so the request looks legitimate. Defences: set session cookies with `SameSite=Lax` (the browser default today) or `Strict`, so they are not sent on cross-site POSTs; never change state with GET requests; require a **CSRF token** (a random value the server embeds in its own pages and checks on every state-changing request; attackers cannot read it from another origin); and check the `Origin` header on the server. APIs that authenticate with an `Authorization: Bearer` header are not exposed to classic CSRF, because browsers never add that header automatically.



HOW A CSRF ATTACK USES THE VICTIM'S COOKIES

SAMESITE COOKIES AND A CSRF TOKEN

main.js

```
// Express: SameSite cookies + a per-session CSRF token (double-submit pattern)
import crypto from "node:crypto";

app.post("/login", (req, res) => {
  const session = createSession(req.body);            // after checking the password
  res.cookie("sid", session.id, { httpOnly: true, secure: true, sameSite: "lax", maxAge: 3600_000 });
  res.cookie("csrf", crypto.randomBytes(32).toString("hex"), { secure: true, sameSite: "lax" });   //
readable by our JS
  res.sendStatus(204);
});

function requireCsrf(req, res, next) {
  const cookieToken = req.cookies.csrf, headerToken = req.get("X-CSRF-Token");
  if (!cookieToken || cookieToken !== headerToken) return res.status(403).json({ error: "CSRF check
failed" });
  next();
}
app.post("/api/transfer", requireCsrf, transferHandler);

// front end: copy the token into a header (another site cannot read our cookies to do this)
fetch("/api/transfer", { method: "POST", headers: { "X-CSRF-Token": getCookie("csrf"), "Content-Type":
"application/json" }, body });
```

RESULTS

```
our own page        -> 200 transfer done
forged form from evil.example (no header, no matching token) -> 403 {"error":"CSRF check failed"}
```

24.2.3 Content Security Policy (overview)

A **Content Security Policy** (CSP) is a response header (or `<meta http-equiv>` tag) that tells the browser which sources of scripts, styles, images, fonts and connections the page may use. A strict policy such as `script-src 'self'` blocks inline scripts, inline event handlers (`onerror="..."`), `eval` and scripts from other domains, so even if an attacker injects HTML, their script does not run. Real applications allow their own trusted scripts with **nonces** (a random value per response) or hashes. Start with `Content-Security-Policy-Report-Only` to see violations without breaking anything, then enforce. Helmet (Module 22) sets a sensible default.

A STRICT CSP STOPS INJECTED CODE main.js

```
console.log("app.js (allowed: same origin) is running");
document.querySelector("#comments").innerHTML = '<img src="x" onerror="console.log(\'injected handler ran!\')">';   // an injected payload
try {
  eval("console.log('eval ran')");
} catch (e) {
  console.log("eval blocked:", e.name);
}
```

BROWSER CONSOLE

```
✖ Refused to execute inline script because it violates the following Content Security Policy directive: "script-src 'self'". Either the 'unsafe-inline' keyword, a hash ('sha256-OdkwbFcQS7WIf1d5cCmnKzAw6no8X1rhv2Oyjt3MkAs='), or a nonce ('nonce-...') is required to enable inline execution.

app.js (allowed: same origin) is running
eval blocked: EvalError
✖ Refused to execute inline event handler because it violates the following Content Security Policy directive: "script-src 'self'". Either the 'unsafe-inline' keyword, a hash ('sha256-...'), or a nonce ('nonce-...') is required to enable inline execution. Note that hashes do not apply to event handlers, style attributes and javascript: navigations unless the 'unsafe-hashes' keyword is present.
```

A PRODUCTION-STYLE CSP HEADER text

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' 'nonce-r4nd0m2026';
  style-src 'self' https://fonts.googleapis.com;
  img-src 'self' data: https://images.lbf-cdn.in;
  connect-src 'self' https://api.learnbeingforward.in;
  frame-ancestors 'none';
  base-uri 'self';
  form-action 'self'

<!-- only scripts carrying the matching nonce may run inline -->
<script nonce="r4nd0m2026">initApp();</script>
```

24.2.4 Secure storage of tokens

Where a front end keeps credentials is a trade-off between XSS and CSRF exposure. `localStorage` and `sessionStorage` are readable by **any** script on the page, so one XSS bug lets an attacker steal a long-lived token and use it from anywhere. A cookie with `HttpOnly` cannot be read by JavaScript at all, `Secure` sends it only over HTTPS, and `SameSite` limits CSRF. A widely recommended pattern: keep the short-lived **access token in memory** (a variable; it disappears on reload), and keep a longer-lived **refresh token in an HttpOnly, Secure, SameSite cookie** scoped to the refresh endpoint; on page load or expiry, call `/auth/refresh` to get a new access token. Never store passwords, card data or secrets in the browser, and never put secrets in front-end code or `VITE_` variables.

Storage	Readable by injected JS?	Sent automatically?	Verdict for tokens
localStorage	yes	no	easy, but stolen by any XSS; avoid for long-lived tokens
sessionStorage	yes	no	same risk, per tab

Storage	Readable by injected JS?	Sent automatically?	Verdict for tokens
JS variable (memory)	only while the page runs	no	good for short-lived access tokens
HttpOnly Secure SameSite cookie	no	yes (same site)	best for refresh/session tokens; add CSRF protection
Non-HttpOnly cookie	yes	yes	worst of both

24.2.5 Dependency vulnerabilities and npm audit

A typical project installs hundreds of packages, most of them indirect dependencies, and any of them may contain a known vulnerability or, in a **supply-chain attack**, malicious code added by a compromised maintainer account or a look-alike package name (typosquatting). `npm audit` checks the installed versions against the GitHub Advisory Database and reports vulnerabilities by severity, with the path through which each is installed; `npm audit fix` upgrades within allowed ranges. Good habits: commit the lock file and install with `npm ci`; keep dependencies few and up to date (Dependabot or Renovate open update pull requests automatically); check a package's popularity, maintenance and exact name before installing; prefer built-in APIs over tiny packages; run audits in CI; and use `npm install --ignore-scripts` or allow-lists for install scripts where possible.

AUDITING DEPENDENCIES

main.js

```
npm audit
npm audit fix
npm audit --omit=dev --audit-level=high     # fail CI only for high/critical issues in production deps
```

TYPICAL OUTPUT

```
# npm audit report

semver  7.0.0 - 7.5.1
Severity: high
semver vulnerable to Regular Expression Denial of Service - https://github.com/advisories/GHSA-c2qf-
rxjj-qqgw
fix available via `npm audit fix`
node_modules/semver
  make-fetch-happen  depends on vulnerable versions of semver

2 vulnerabilities (1 moderate, 1 high)

To address all issues, run:
  npm audit fix
```

24.3 Clean code

24.3.1 Naming, small functions, avoiding deep nesting

Code is read far more often than it is written, so clarity is the main quality goal. **Names** should reveal intent: `daysUntilExpiry` rather than `d`, verbs for functions (`calculateTotal`, `isEligible`), nouns for values, `is/has/can` prefixes for booleans, plural names for collections, named constants instead of magic numbers (`MAX_LOGIN_ATTEMPTS` instead of `5`). **Functions** should be small and do one thing at one level of abstraction, with few parameters (use an options object beyond three). **Deep nesting** is hard to

follow: replace it with **guard clauses** (return early for invalid cases), extract helper functions, and use lookup tables or array methods instead of long if-else chains.

REFACTORING FOR READABILITY

main.js

```
// Before: unclear names, magic numbers, deep nesting, several jobs in one function
function calc(u, c) {
  let t = 0;
  if (u) {
    if (u.active) {
      if (c.length > 0) {
        for (let i = 0; i < c.length; i++) {
          if (c[i].q > 0) {
            t += c[i].p * c[i].q;
          }
        }
        if (u.type === "student") {
          t = t * 0.85;
        } else if (u.type === "staff") {
          t = t * 0.7;
        }
      }
    }
  }
  return t;
}

// After: intention-revealing names, constants, guard clauses, small focused functions
const DISCOUNT_BY_ROLE = { student: 0.15, staff: 0.30 };

const lineTotal = (item) => item.price * item.qty;
const subtotal = (items) => items.filter((item) => item.qty > 0).reduce((sum, item) => sum +
lineTotal(item), 0);
const discountFor = (role) => DISCOUNT_BY_ROLE[role] ?? 0;

function checkoutTotal(user, items) {
  if (!user?.active || items.length === 0) return 0;          // guard clause
  return subtotal(items) * (1 - discountFor(user.role));
}

const user = { active: true, role: "student", type: "student" };
const cart = [{ price: 1000, qty: 2, p: 1000, q: 2 }, { price: 500, qty: 0, p: 500, q: 0 }];
console.log("before:", calc(user, cart), "| after:", checkoutTotal(user, cart), "| inactive:",
checkoutTotal({ active: false }, cart));
```

OUTPUT

```
before: 1700 | after: 1700 | inactive: 0
```

24.3.2 DRY, KISS and SOLID in JavaScript

DRY (Don't Repeat Yourself): every piece of knowledge should live in one place; duplicated logic drifts apart when one copy is fixed and the other is not. But do not merge code that only looks similar ("the wrong abstraction is worse than duplication"). **KISS** (Keep It Simple): prefer the simplest solution that works; clever code is hard to debug. **YAGNI** (You Aren't Gonna Need It): do not build features or flexibility before they are needed. **SOLID** are five design principles from object-oriented programming that apply to JavaScript modules and functions too:

Principle	Meaning	In JavaScript
S - Single responsibility	a module or class has one reason to change	separate fetching, validation and rendering into different functions or modules
O - Open/closed	open for extension, closed for modification	add new behaviour by adding a strategy or plugin (Module 20), not by editing a big switch
L - Liskov substitution	subtypes must work wherever the base type is expected	a subclass must not break the parent's contract (e.g. throw where the parent did not)
I - Interface segregation	clients should not depend on methods they do not use	small, focused interfaces / objects; pass only what a function needs
D - Dependency inversion	depend on abstractions, not concrete details	inject dependencies (a repository, fetch, a clock) so they can be swapped and mocked

SOLID IDEAS WITH PLAIN FUNCTIONS main.js

```
// Single responsibility + dependency inversion + open/closed in a small report module
const formatters = {                                    // open for extension: add a format here
  text: (rows) => rows.map((r) => `${r.name}: ${r.marks}`).join("\n"),
  csv: (rows) => ["name,marks", ...rows.map((r) => `${r.name},${r.marks}`)].join("\n"),
};

function createReportService({ loadStudents, formatters }) {  // dependencies injected
  return {
    async build(format, minMarks = 0) {
      const format_ = formatters[format];
      if (!format_) throw new Error(`Unknown format: ${format}`);
      const rows = (await loadStudents()).filter((s) => s.marks >= minMarks);   // one job: assemble
      return format_(rows);
    },
  };
}

const fakeLoad = async () => [{ name: "Asha", marks: 91 }, { name: "Ravi", marks: 64 }, { name:
"Meera", marks: 85 }];
const reports = createReportService({ loadStudents: fakeLoad, formatters });

formatters.json = (rows) => JSON.stringify(rows);            // extended without editing the service
(async () => {
  console.log(await reports.build("text", 80));
  console.log(await reports.build("csv"));
  console.log(await reports.build("json", 90));
})();
```

OUTPUT

```
Asha: 91
Meera: 85
name,marks
Asha,91
Ravi,64
Meera,85
[{"name":"Asha","marks":91}]
```

24.3.3 Code reviews

In professional teams, every change is reviewed by another developer, usually as a **pull request** on GitHub or GitLab, before it is merged. Reviews catch bugs, spread knowledge of the codebase, keep the

style consistent and mentor newer developers. As an **author**: keep pull requests small and focused; write a clear description (what, why, how to test, screenshots for UI); run the linter, formatter and tests first; review your own diff before asking others. As a **reviewer**: check correctness, edge cases, security, tests, naming and readability, not personal style that tools should handle; ask questions rather than issue commands ("What happens if the list is empty?"); distinguish blocking issues from suggestions ("nit:"); and respond promptly. Automate what can be automated (CI runs lint, type checks and tests on every pull request), so humans can focus on design and logic.

Correctness
Does it do what the ticket asks? Edge cases: empty, null, huge, concurrent, failing network?

Security
Untrusted input validated? No innerHTML with user data, secrets, or SQL built from strings?

Tests
Are new behaviours and bug fixes covered? Do tests check results, not implementation details?

Readability
Clear names, small functions, no dead code or commented-out blocks, helpful comments for the "why".

Design
Right place for this code? Duplication? Will it be easy to change later?

Performance and a11y
Unnecessary work in loops or renders? Keyboard and screen-reader friendly UI?

24.4 Accessibility basics

24.4.1 Keyboard access, ARIA attributes and focus management

Accessibility (a11y) means people with disabilities can use your application: users who are blind (screen readers), have low vision (zoom, high contrast), cannot use a mouse (keyboard, switch devices, voice control), are deaf, or have cognitive disabilities. It is also a legal requirement in many countries (WCAG 2.2 is the standard) and benefits everyone. Three JavaScript-relevant basics: **Keyboard access**: everything clickable must be reachable with Tab and operable with Enter/Space; use real `<button>` and `<a href>` elements, which are focusable and keyboard-operable for free, instead of clickable `<div>`s, and never remove the focus outline without replacing it. **ARIA attributes** describe custom widgets to assistive technology: `aria-label` (a name for an icon button), `aria-expanded`, `aria-controls`, `aria-live="polite"` (announce dynamic updates such as "3 results found"), `role="dialog"` with `aria-modal`; the first rule of ARIA is to prefer native HTML elements when they exist. **Focus management**: when a dialog opens, move focus into it, keep Tab inside it (focus trap), close it on Escape and return focus to the button that opened it; after route changes or deletions, move focus somewhere sensible. The native `<dialog>` element with `showModal()` provides much of this automatically.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY)

HTML

```
<button id="open">Delete task</button>
<button id="more-btn" aria-expanded="false" aria-controls="more">More options</button>
<div id="more" hidden>Archive · Duplicate</div>
<button class="icon">🗑</button>
<p id="status" role="status" aria-live="polite"></p>
<dialog id="confirm" aria-labelledby="dlg-title">
  <form method="dialog">
    <h2 id="dlg-title">Delete this task?</h2>
    <button value="no" autofocus>Cancel</button>
    <button value="yes">Delete</button>
  </form>
</dialog>
```

AN ACCESSIBLE CONFIRM DIALOG AND DISCLOSURE BUTTON

main.js

```
const openBtn = document.querySelector("#open");
const dialog = document.querySelector("#confirm");
const status = document.querySelector("#status");
const describe = (el) => el.id ? "#" + el.id : el.tagName.toLowerCase() + (el.textContent ? `
"${el.textContent.trim()}"` : "");

// native <dialog>: modal, Escape closes it, focus moves inside and returns afterwards
openBtn.addEventListener("click", () => {
  dialog.showModal();
  console.log("dialog open, focus is on:", describe(document.activeElement));
});
dialog.addEventListener("close", () => {
  status.textContent = dialog.returnValue === "yes" ? "Task deleted" : "Deletion cancelled";   //
announced by aria-live
  console.log(`dialog closed (${dialog.returnValue || "escape"}), focus returned to:`,
describe(document.activeElement), "| live region says:", status.textContent);
});

// a custom disclosure widget made accessible with a real button + aria-expanded
const toggle = document.querySelector("#more-btn");
toggle.addEventListener("click", () => {
  const expanded = toggle.getAttribute("aria-expanded") === "true";
  toggle.setAttribute("aria-expanded", String(!expanded));
  document.querySelector("#more").hidden = expanded;
  console.log("aria-expanded is now", toggle.getAttribute("aria-expanded"));
});

// audit: icon-only buttons must have an accessible name
for (const b of document.querySelectorAll("button")) {
  const name = b.getAttribute("aria-label") || b.textContent.trim();
  if (!name || /^[^\w]+$/.test(name)) console.log("a11y warning: button without an accessible name:",
b.outerHTML);
}
```

BROWSER CONSOLE

```
a11y warning: button without an accessible name: <button class="icon">🗑</button>
dialog open, focus is on: button "Cancel"
dialog closed (escape), focus returned to: #open | live region says: Deletion cancelled
dialog open, focus is on: button "Cancel"
dialog closed (yes), focus returned to: #open | live region says: Task deleted
aria-expanded is now true
```

Simulated user actions, in order: presses Enter in `#open`; presses Escape in `body`; clicks `#open`; presses Enter in `button[value=yes]`; presses Enter in `#more-btn`.

Do	Instead of
`<button>` for actions, `<a href>` for navigation	`<div onclick>` (not focusable, no keyboard, no role)
`<label for>` on every input	placeholder text as the only label
`alt` text on meaningful images, `alt=""` on decorative ones	missing alt attributes
`aria-label` on icon-only buttons	a bare icon
visible focus styles (`:focus-visible`)	`outline: none` with no replacement
`aria-live` regions for dynamic messages	silent updates screen readers miss

Do	Instead of
colour contrast at least 4.5:1 for text	light grey on white
test with keyboard only and a screen reader; run Lighthouse / axe	assuming it works

Automated tests catch regressions: many fast unit tests, fewer integration tests and a few end-to-end tests. Jest, Vitest and node:test organise tests into suites with describe/test, assert with matchers (toBe, toEqual, toThrow), isolate code with mocks, spies and dependency injection, and test async code by awaiting promises and using fake timers and fake fetch. Testing Library tests DOM code the way users interact; Playwright and Cypress run end-to-end tests in real browsers with auto-waiting locators. Coverage shows untested code; TDD follows red-green-refactor. Prevent XSS by treating data as text, validating URLs, sanitising required HTML and adding a Content Security Policy; prevent CSRF with SameSite cookies, CSRF tokens and Origin checks; keep access tokens in memory and refresh tokens in HttpOnly Secure cookies; audit dependencies with npm audit and lock files. Clean code uses intention-revealing names, small functions, guard clauses, DRY, KISS, YAGNI and SOLID, and is reviewed through small, well-described pull requests. Accessible apps use native elements, keyboard access, ARIA where needed, live regions and careful focus management.

PRACTICE CHECKPOINT

Add **Vitest unit tests** to utility functions and the API, a **Playwright end-to-end test** for the to-do app, and **fix any XSS** issue found in earlier projects.

The Playwright test in 24.1.5 and the suites in 24.1.2 and 24.1.3 are the model answers for the testing parts: put the unit tests next to the modules as `*.test.js`, add `"test": "vitest"` to package.json, and add a Playwright project with `npm init playwright@latest`. For the API, Supertest calls the Express app directly without opening a port. The XSS fix below replaces string-built HTML in the to-do and gallery projects with DOM construction, and a regression test proves a malicious task title is displayed as text.

API TESTS WITH VITEST AND SUPERTEST

main.js

```
// tests/students.api.test.js  (Vitest + Supertest, MongoDB replaced by an in-memory server)
import { describe, test, expect, beforeAll, afterAll } from "vitest";
import request from "supertest";
import { MongoMemoryServer } from "mongodb-memory-server";
import mongoose from "mongoose";
import { app } from "../src/app.js";
import { tokenFor } from "./helpers.js";

let mongo;
beforeAll(async () => { mongo = await MongoMemoryServer.create(); await
mongoose.connect(mongo.getUri()); });
afterAll(async () => { await mongoose.disconnect(); await mongo.stop(); });

describe("/api/students", () => {
  test("rejects requests without a token", async () => {
    await request(app).get("/api/students").expect(401);
  });
  test("an admin can create and fetch a student", async () => {
    const admin = tokenFor({ role: "admin" });
    const created = await request(app).post("/api/students").set("Authorization", `Bearer ${admin}`)
      .send({ name: "Zoya Khan", email: "zoya@lbf.in", course: "Java", marks: 88 }).expect(201);
    const res = await request(app).get(`/api/students/${created.body._id}`).set("Authorization",
`Bearer ${admin}`).expect(200);
    expect(res.body).toMatchObject({ name: "Zoya Khan", course: "Java" });
  });
  test("validation errors return 400 with details", async () => {
    const res = await request(app).post("/api/students").set("Authorization", `Bearer ${tokenFor({
role: "admin" })}`)
      .send({ name: "Z", email: "nope", course: "COBOL" }).expect(400);
    expect(Object.keys(res.body.details)).toEqual(["name", "email", "course"]);
  });
  test("students cannot delete", async () => {
    await request(app).delete("/api/students/123").set("Authorization", `Bearer ${tokenFor({ role:
"student" })}`).expect(403);
  });
});
```

```
$ npx vitest run tests/students.api.test.js
 ✓ tests/students.api.test.js (4 tests) 640ms
   ✓ /api/students > rejects requests without a token
   ✓ /api/students > an admin can create and fetch a student
   ✓ /api/students > validation errors return 400 with details
   ✓ /api/students > students cannot delete

 Test Files  1 passed (1)
      Tests  4 passed (4)
```

TESTS/XSS.SPEC.JS

JavaScript

```
const { test, expect } = require("playwright/test");

test("a malicious task title is shown as text and never runs", async ({ page }) => {
  let attacked = false;
  page.on("dialog", async (d) => { attacked = true; await d.dismiss(); });
  await page.goto("index.html");
  const payload = '<img src=x onerror="alert(document.cookie)">';
  await page.getByLabel("New task").fill(payload);
  await page.getByRole("button", { name: "Add" }).click();
  await expect(page.getByRole("listitem")).toHaveText(payload);   // visible as literal text
  await expect(page.locator("#list img")).toHaveCount(0);         // no element was created
  expect(attacked).toBe(false);
});
```

REGRESSION TEST: TASK TITLES CANNOT INJECT HTML

main.js

```
npx playwright test --reporter=list
```

TERMINAL

```
$ npx playwright test --reporter=list

Running 1 test using 1 worker

  ✓  1 tests/xss.spec.js:3:1 › a malicious task title is shown as text and never runs (273ms)

  1 passed (1.6s)
```

MODULE 25 · ADVANCED

Frameworks, data structures, projects and interviews

Minimum duration **5 days** | Level **Advanced**

The final module connects everything to the work you will do as a developer. It introduces front-end frameworks with a practical first look at React (JSX, components, props, state and effects) and an overview of Angular, Vue, Svelte and Next.js; applies JavaScript to data structures and algorithms (Big-O, linked lists, stacks, queues, hash maps, trees, graphs, sorting and the main problem-solving patterns); describes six capstone projects; prepares you for interviews with checklists, output-prediction questions, machine-coding and polyfill rounds; and maps the career paths open to JavaScript developers.

IN THIS MODULE

25.1 Front-end frameworks (overview)
25.2 Data structures and algorithms in JavaScript
25.3 Capstone projects
25.4 Interview preparation
25.5 Career paths with JavaScript

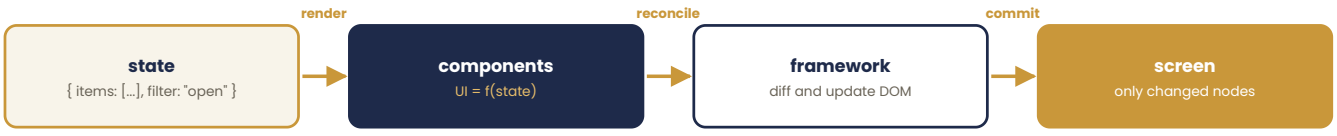
NOTE

The React examples were compiled with esbuild and run with React 19 in headless Chromium; the clicks and typing listed under each were performed by the test browser. The data-structure and algorithm programs run in Node.js.

25.1 Front-end frameworks (overview)

25.1.1 Why frameworks: components, state and declarative UI

With plain DOM code (Modules 15 and 16) you describe **how** to change the page step by step: find the element, update its text, add a class, append a row, remember to update the counter too. As an application grows, keeping the DOM in sync with the data becomes the main source of bugs. Frameworks change the model. You write **declarative** UI: describe **what** the page should look like for a given **state** (`UI = f(state)`), and the framework updates the real DOM efficiently whenever the state changes. Interfaces are built from **components**: reusable, self-contained pieces (a ProductCard, a SearchBox, a Cart) that combine markup, logic and style and receive data through **props**. Frameworks also bring routing, forms, data fetching, testing and a large ecosystem, so teams can build big applications consistently.



DECLARATIVE UI: CHANGE THE STATE, THE FRAMEWORK UPDATES THE PAGE

Imperative DOM code	Declarative framework code
find elements, set textContent, toggle classes by hand	describe the UI for the current state
every event handler must update every affected element	handlers only update state; the view follows automatically
structure spread across HTML and many scripts	components bundle markup, logic and style
easy for small pages	scales to large applications and teams

25.1.2 React basics: JSX, components, props, state with useState, effects with useEffect

React (Meta, 2013) is the most widely used UI library. A **component** is a JavaScript function that returns **JSX**: an HTML-like syntax inside JavaScript that a compiler turns into function calls (`<h1>Hi</h1>` becomes `jsx("h1", { children: "Hi" })`). JSX differences from HTML: `className` instead of `class`, `htmlFor`, camelCase event props (`onClick`), self-closing tags, a single root element (or a fragment `<>...</>`), and any JavaScript expression inside braces `{ }`: values, conditions (`{loggedIn && <Menu />}`, ternaries) and lists with `map` (each item needs a stable `key`). **Props** are the inputs passed from parent to child, read-only like function parameters. Text in JSX is escaped automatically, which protects against XSS.

COMPONENTS, JSX AND PROPS

main.js

```
import { createRoot } from "react-dom/client";

function CourseCard({ title, weeks, fee, tags = [], featured = false }) {     // props, with defaults
  return (
    <article className={featured ? "card featured" : "card"}>
      <h3>{title} {featured && <span>★ Featured</span>}</h3>
      <p>{weeks} weeks · {fee === 0 ? "Free" : `Rs ${fee.toLocaleString("en-IN")}`}</p>
      <ul>
        {tags.map((tag) => <li key={tag}>{tag}</li>)}
      </ul>
    </article>
  );
}

function CourseList({ courses }) {
  return (
    <>
      <h2>{courses.length} courses</h2>
      {courses.map((c) => <CourseCard key={c.id} {...c} />)}
    </>
  );
}

const courses = [
  { id: 1, title: "JavaScript", weeks: 12, fee: 14999, tags: ["DOM", "async"], featured: true },
  { id: 2, title: "Intro to Git", weeks: 1, fee: 0 },
  { id: 3, title: "<script>alert('x')</script>", weeks: 2, fee: 999 },        // text is escaped
];
createRoot(document.getElementById("root")).render(<CourseList courses={courses} />);
setTimeout(() => console.log("rendered", document.querySelectorAll("article").length, "cards; scripts
injected:", document.querySelectorAll("#root script").length), 50);
```

BROWSER CONSOLE

```
rendered 3 cards; scripts injected: 0
--- page shows ---
3 courses
JavaScript ★ Featured

12 weeks · Rs 14,999

DOM
async
Intro to Git

1 weeks · Free

<script>alert('x')</script>

2 weeks · Rs 999
--- element HTML ---
<div id="root"><h2>3 courses</h2><article class="card featured"><h3>JavaScript <span>★ Featured</span>
</h3><p>12 weeks · Rs 14,999</p><ul><li>DOM</li><li>async</li></ul></article><article class="card">
<h3>Intro to Git </h3><p>1 weeks · Free</p><ul></ul></article><article class="card">
<h3>&lt;script&gt;alert('x')&lt;/script&gt; </h3><p>2 weeks · Rs 999</p><ul></ul></article></div>
```

State is data that changes over time and should update the screen. `const [value, setValue] = useState(initial)` returns the current value and a setter; calling the setter schedules a **re-render** of the component with the new value. State must be treated as immutable: create new arrays and objects (`setItems([...items, item])`) instead of mutating. When the new state depends on the old, pass a function (`setCount((c) => c + 1)`). **Effects**, `useEffect(fn, deps)`, run code **after** rendering to synchronise

with things outside React: fetching data, timers, subscriptions, the document title. The dependency array controls when it re-runs (`[]` once after the first render, `[query]` whenever `query` changes), and the function it returns is the **clean-up**, run before the next effect and when the component is removed.

STATE WITH USESTATE AND EFFECTS WITH USEEFFECT

main.js

```
import { useState, useEffect } from "react";
import { createRoot } from "react-dom/client";

const ALL = ["JavaScript", "TypeScript", "Java", "Python", "React", "Node.js"];
function fakeSearch(q) {                                    // stands in for fetch(`/api/search?
q=${q}`)
  return new Promise((resolve) => setTimeout(() => resolve(ALL.filter((c) =>
c.toLowerCase().includes(q.toLowerCase()))), 30));
}

function TodoCounter() {
  const [count, setCount] = useState(0);
  return (
    <p>
      <button onClick={() => setCount((c) => c + 1)}>Add one</button> <output>{count}</output>
    </p>
  );
}

function Search() {
  const [query, setQuery] = useState("");
  const [results, setResults] = useState(ALL);
  const [loading, setLoading] = useState(false);

  useEffect(() => {                                         // runs after render whenever query changes
    let cancelled = false;
    setLoading(true);
    fakeSearch(query).then((r) => {
      if (!cancelled) { setResults(r); setLoading(false); console.log(`effect: "${query}" ->
${r.join(", ") || "none"}`); }
    });
    return () => { cancelled = true; };                     // clean-up: ignore stale responses
  }, [query]);

  useEffect(() => { document.title = `${results.length} results`; }, [results]);

  return (
    <section>
      <label>Search <input value={query} onChange={(e) => setQuery(e.target.value)} /></label>
      {loading ? <p>Loading...</p> : <p>{results.length} result(s): {results.join(", ")}</p>}
    </section>
  );
}

createRoot(document.getElementById("root")).render(<><TodoCounter /><Search /></>);
```

BROWSER CONSOLE

```
effect: "" -> JavaScript, TypeScript, Java, Python, React, Node.js
effect: "java" -> JavaScript, Java
--- page shows ---
Add one 3

Search

2 result(s): JavaScript, Java
```

Simulated user actions, in order: waits 100 ms; clicks `button`; clicks `button`; clicks `button`; types "java" into `input`; waits 200 ms.

> **NOTE**
> The first effect ran once after the initial render (empty query). Typing "java" changed the state, React re-rendered, and the effect ran again because its dependency changed. The counter shows 3 after three clicks.

Hook	Purpose	Example
`useState`	local state that triggers re-render	`const [open, setOpen] = useState(false)`
`useEffect`	side effects after render, with clean-up	fetching, timers, subscriptions
`useRef`	a mutable value or DOM reference that does not re-render	focus an input
`useMemo` / `useCallback`	cache an expensive value / a function between renders	filtered lists, stable callbacks
`useContext`	read shared data without passing props down	theme, logged-in user
`useReducer`	state with complex transitions	a cart or a form wizard

RENDER.JSX text

```
const React = require("react");
const { renderToStaticMarkup } = require("react-dom/server");

const Badge = ({ level }) => <span className={`badge ${level}`}>{level}</span>;
const element = <h2 id="title">TypeScript <Badge level="advanced" /></h2>;

console.log("a JSX element is a plain object:", element.type, Object.keys(element.props));
console.log(renderToStaticMarkup(element));            // server-side rendering, used by Next.js
```

JSX IS JUST JAVASCRIPT: RENDERING A COMPONENT TO HTML IN NODE.JS main.js

```
NODE_PATH=/home/claude/.npm-global/lib/node_modules tsx render.jsx
```

TERMINAL

```
$ NODE_PATH=/home/claude/.npm-global/lib/node_modules tsx render.jsx
a JSX element is a plain object: h2 [ 'id', 'children' ]
<h2 id="title">TypeScript <span class="badge advanced">advanced</span></h2>
```

25.1.3 Angular, Vue and Svelte at a glance

Angular (Google) is a complete, opinionated framework written in TypeScript: components with templates, dependency injection, a router, forms, an HTTP client and testing tools in one package, with signals for fine-grained reactivity; it is popular in large enterprises and service companies. **Vue** (Evan You) is approachable and progressive: single-file components (`.vue`) combine template, script and style, reactivity is built on proxies (Module 20), and the ecosystem (Vue Router, Pinia, Nuxt) is cohesive. **Svelte** moves work to compile time: components compile to small, efficient JavaScript that updates the DOM directly, with very little boilerplate (runes such as `$state` in Svelte 5). Solid, Qwik and Preact are further options. The core ideas you learn in any one of them (components, props, state, effects, routing) transfer to the others.

THE SAME COUNTER IN VUE, SVELTE AND ANGULAR

main.js

```
<!-- Counter.vue -->
<script setup>
import { ref } from "vue";
const count = ref(0);
</script>
<template>
  <button @click="count++">Clicked {{ count }} times</button>
</template>

<!-- Counter.svelte (Svelte 5) -->
<script>
  let count = $state(0);
</script>
<button onclick={() => count++}>Clicked {count} times</button>

// counter.component.ts (Angular)
@Component({
  selector: "app-counter",
  template: `<button (click)="count.set(count() + 1)">Clicked {{ count() }} times</button>`,
})
export class CounterComponent { count = signal(0); }
```

NOTE

All three render a button whose label updates on every click, like the React counter above.

	React	Angular	Vue	Svelte
Type	UI library + ecosystem	full framework	progressive framework	compiler framework
Language	JavaScript/TypeScript + JSX	TypeScript + templates	JS/TS + templates (SFC)	JS/TS + templates
Reactivity	state + re-render (virtual DOM)	signals, zone-less change detection	proxies (ref, reactive)	compiled runes
Learning curve	moderate	steepest	gentle	gentle
Full-stack framework	Next.js, Remix/React Router	Angular SSR / Analog	Nuxt	SvelteKit
Common in India	start-ups, product companies	enterprises, service companies	growing	small but growing

25.1.4 Full-stack frameworks: Next.js and similar

Meta-frameworks build on a UI framework to provide a complete application structure. **Next.js** (Vercel, on React) offers file-system routing (a file `app/courses/[id]/page.tsx` becomes the route `/courses/:id`), several rendering modes (server-side rendering for fresh data, static generation for speed, incremental regeneration, client-side rendering), **React Server Components** that fetch data on the server and send less JavaScript to the browser, server actions and API routes (back-end code in the same project), image and font optimisation, and one-command deployment. Similar frameworks: Remix/React Router 7, Nuxt (Vue), SvelteKit (Svelte), Astro (content sites with "islands" of interactivity) and Angular SSR. Rendering on the server improves first-load speed and SEO compared with a plain single-page app.

A SERVER-RENDERED PAGE IN NEXT.JS

main.js

```
// app/courses/[id]/page.tsx  (Next.js App Router: a React Server Component)
import { notFound } from "next/navigation";
import EnrolButton from "./EnrolButton";            // a client component ("use client") for
interactivity

export default async function CoursePage({ params }: { params: Promise<{ id: string }> }) {
  const { id } = await params;
  const res = await fetch(`${process.env.API_URL}/courses/${id}`, { next: { revalidate: 60 } });   //
cached for 60 s
  if (!res.ok) notFound();
  const course = await res.json();
  return (
    <main>
      <h1>{course.title}</h1>
      <p>{course.weeks} weeks · Rs {course.fee}</p>
      <EnrolButton courseId={course.id} />
    </main>
  );
}
```

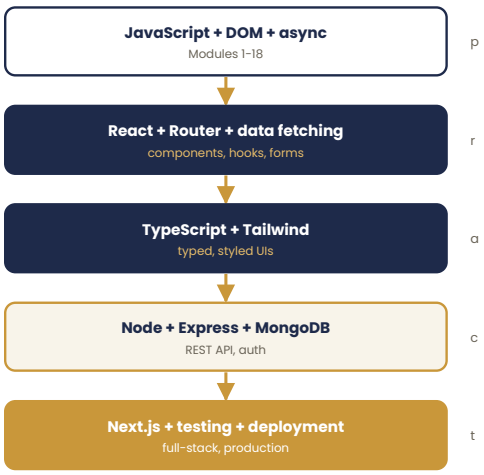
WHAT THE BROWSER RECEIVES

```
GET /courses/1 -> fully rendered HTML: <main><h1>JavaScript</h1><p>12 weeks · Rs 14999</p>
<button>Enrol</button></main>
(only the small EnrolButton component ships JavaScript to the browser)
```

Rendering mode	When HTML is produced	Good for
CSR (client-side)	in the browser after JS loads	dashboards behind a login
SSR (server-side)	on the server for every request	personalised or frequently changing pages
SSG (static)	at build time	marketing pages, blogs, documentation
ISR (incremental)	static, regenerated after N seconds	catalogues, course pages

25.1.5 Choosing a learning path (MERN stack)

After this course, a focused path beats trying everything. The most common route to a first JavaScript job in India is the **MERN** stack: **M**ongoDB (database), **E**xpress (API, Module 22), **R**eact (front end) and **N**ode.js (runtime). Variants include MEAN (Angular), MEVN (Vue), and PERN (PostgreSQL). A suggested order: (1) solid JavaScript and DOM (this course); (2) React fundamentals, React Router, forms and data fetching (TanStack Query); (3) styling with CSS, Tailwind CSS or a component library; (4) TypeScript in React; (5) Node.js, Express, MongoDB and authentication; (6) Next.js for full-stack and SSR; (7) testing, Git workflows and deployment; in parallel, (8) DSA practice for interviews. Build and deploy a project at every stage.



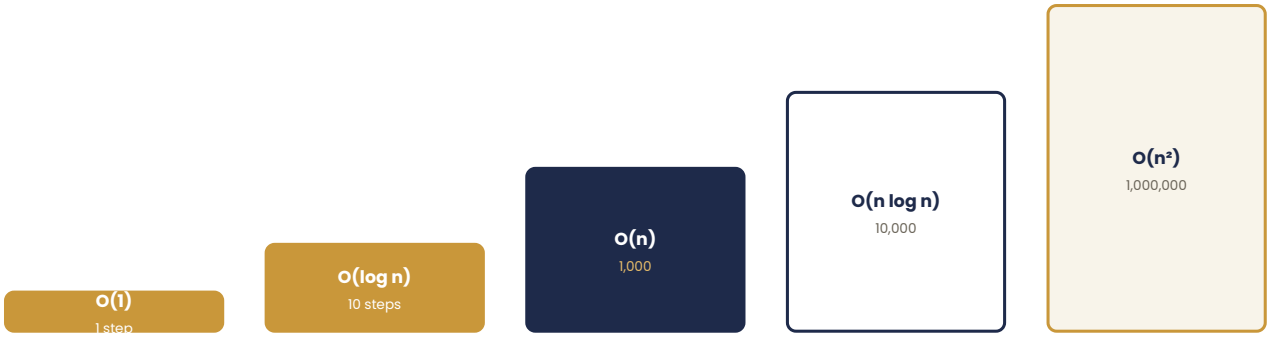
A MERN LEARNING PATH

25.2 Data structures and algorithms in JavaScript

Product companies test **data structures and algorithms** (DSA) in almost every interview, and choosing the right structure is what makes real code fast. JavaScript gives you arrays, objects, Map and Set built in; other structures are built from classes. Analyse every solution's **time** and **space** complexity.

25.2.1 Big-O analysis of array and object operations

Big-O notation describes how an algorithm's running time (or memory) grows as the input size *n* grows, ignoring constants and smaller terms: what matters is the shape of the curve. O(1) is constant (does not depend on n), O(log n) logarithmic (halving each step, like binary search), O(n) linear (one pass), O(n log n) (efficient sorting), O(n²) quadratic (nested loops over the data), O(2ⁿ) exponential (trying every subset). Knowing the cost of built-in operations prevents accidental quadratic code, such as calling `includes` or `indexOf` inside a loop, or `shift()` on a large array in a loop.



approximate steps for n = 1,000 (log scale)

GROWTH RATES FOR N = 1,000

Operation	Array	Object / Map / Set
access by index / key	O(1) `arr[i]`	O(1) average `map.get(k)`
search by value	O(n) `includes`, `indexOf`, `find`	O(1) `set.has(v)`
add / remove at the end	O(1) `push`, `pop`	O(1) `set`, `delete`
add / remove at the start	O(n) `unshift`, `shift` (re-indexing)	-

Operation	Array	Object / Map / Set
insert / delete in the middle	O(n) `splice`	-
copy, map, filter, reduce, spread	O(n)	O(n) to iterate
sort	O(n log n) `sort`	-
binary search (sorted array)	O(log n)	-

CHOOSING THE RIGHT STRUCTURE CHANGES THE COMPLEXITY

main.js

```
// Same task, two complexities: find which order ids are also in the refunds list
const orders = Array.from({ length: 20000 }, (_, i) => i);
const refunds = Array.from({ length: 20000 }, (_, i) => i * 3);

function commonQuadratic(a, b) {                        // O(n * m): includes scans b every time
  return a.filter((x) => b.includes(x));
}
function commonLinear(a, b) {                           // O(n + m): one pass to build a Set, one to
check
  const set = new Set(b);
  return a.filter((x) => set.has(x));
}

let t = performance.now();
const r1 = commonQuadratic(orders, refunds);
const slow = performance.now() - t;
t = performance.now();
const r2 = commonLinear(orders, refunds);
const fast = performance.now() - t;
console.log("same answer:", r1.length === r2.length, "| matches:", r2.length);
console.log("Set version faster:", fast < slow, `(roughly ${slow / fast > 50 ? "100x or more" :
"several times"} on this machine)`);

// O(log n): binary search halves the range each step
function binarySearch(sorted, target) {
  let lo = 0, hi = sorted.length - 1, steps = 0;
  while (lo <= hi) {
    steps++;
    const mid = (lo + hi) >> 1;
    if (sorted[mid] === target) return { index: mid, steps };
    if (sorted[mid] < target) lo = mid + 1; else hi = mid - 1;
  }
  return { index: -1, steps };
}
console.log("binary search in 1,000,000 items:", binarySearch(Array.from({ length: 1e6 }, (_, i) => i
* 2), 777776));
```

OUTPUT

```
same answer: true | matches: 6667
Set version faster: true (roughly 100x or more on this machine)
binary search in 1,000,000 items: { index: 388888, steps: 19 }
```

25.2.2 Linked lists, stacks and queues

A **linked list** stores items in nodes, each holding a value and a reference to the next node (and to the previous one in a doubly linked list). Inserting or removing at the head is O(1) with no shifting, but reaching the k-th item is O(k). A **stack** is last-in, first-out (LIFO): `push` and `pop` at one end, both O(1); it models undo history, the call stack, bracket matching and depth-first search. A JavaScript array is

already an efficient stack. A **queue** is first-in, first-out (FIFO): add at the back, remove from the front; it models task queues, print jobs and breadth-first search. Using `array.shift()` as dequeue is O(n), so for large queues use a linked list or a head index.

A LINKED LIST, A STACK AND A QUEUE main.js

```
class ListNode { constructor(value, next = null) { this.value = value; this.next = next; } }

class LinkedList {
  head = null; size = 0;
  prepend(v) { this.head = new ListNode(v, this.head); this.size++; return this; }        // O(1)
  append(v) {                                                                              // O(n)
    const node = new ListNode(v);
    if (!this.head) this.head = node;
    else { let cur = this.head; while (cur.next) cur = cur.next; cur.next = node; }
    this.size++; return this;
  }
  remove(v) {
    if (!this.head) return false;
    if (this.head.value === v) { this.head = this.head.next; this.size--; return true; }
    for (let cur = this.head; cur.next; cur = cur.next) {
      if (cur.next.value === v) { cur.next = cur.next.next; this.size--; return true; }
    }
    return false;
  }
  reverse() {                                                                              // classic
interview question
    let prev = null, cur = this.head;
    while (cur) { [cur.next, prev, cur] = [prev, cur, cur.next]; }
    this.head = prev; return this;
  }
  *[Symbol.iterator]() { for (let c = this.head; c; c = c.next) yield c.value; }
  toString() { return [...this].join(" -> ") + " -> null"; }
}

const list = new LinkedList().append("B").append("C").prepend("A").append("D");
console.log(String(list), "| size", list.size);
list.remove("C");
console.log("after remove C:", String(list));
console.log("reversed:     ", String(list.reverse()));

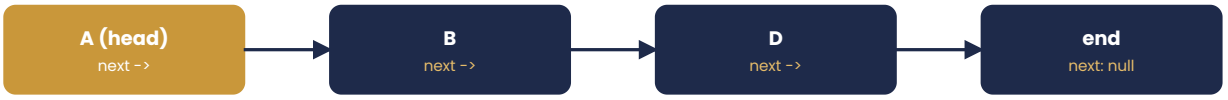
// Stack: balanced brackets
function isBalanced(text) {
  const pairs = { ")": "(", "]": "[", "}": "{" }, stack = [];
  for (const ch of text) {
    if ("([{".includes(ch)) stack.push(ch);
    else if (ch in pairs && stack.pop() !== pairs[ch]) return false;
  }
  return stack.length === 0;
}
console.log("balanced:", ["{[()()]}", "([)]", "((", "fn(a[0], {b: 1})"].map((s) => `${s}
${isBalanced(s)}`).join(" | "));

// Queue with O(1) dequeue using a head index
class Queue {
  #items = []; #head = 0;
  enqueue(x) { this.#items.push(x); }
  dequeue() { if (this.isEmpty) return undefined; const x = this.#items[this.#head++]; if (this.#head
> 1000 && this.#head * 2 > this.#items.length) { this.#items = this.#items.slice(this.#head);
this.#head = 0; } return x; }
  get isEmpty() { return this.#head >= this.#items.length; }
  get size() { return this.#items.length - this.#head; }
}
const printer = new Queue();
```

```
["report.pdf", "notes.docx", "photo.png"].forEach((j) => printer.enqueue(j));
console.log("printing in order:", printer.dequeue(), printer.dequeue(), "| still waiting:",
printer.size);
```

OUTPUT

```
A -> B -> C -> D -> null | size 4
after remove C: A -> B -> D -> null
reversed:       D -> B -> A -> null
balanced: {[()()]} true | ([)] false | (( false | fn(a[0], {b: 1}) true
printing in order: report.pdf notes.docx | still waiting: 1
```



A SINGLY LINKED LIST AFTER REMOVING C

25.2.3 Hash maps with Map and objects

A **hash map** stores key-value pairs and finds a key in O(1) average time by computing a hash of the key to locate its bucket. In JavaScript, use **Map** (any key type, insertion order, `size`) or a plain object for string keys, and **Set** for membership (Module 12). Hash maps solve a huge range of interview problems by trading memory for speed: counting frequencies, finding pairs or duplicates in one pass, grouping (anagrams), caching (memoisation), and indexing records by id. The example also builds a tiny hash table to show the idea of buckets and collisions.

HASH-MAP PATTERNS AND A TINY HASH TABLE — main.js

```
// 1. Two Sum in O(n): remember what we have seen
function twoSum(nums, target) {
  const seen = new Map();                                  // value -> index
  for (const [i, n] of nums.entries()) {
    if (seen.has(target - n)) return [seen.get(target - n), i];
    seen.set(n, i);
  }
  return null;
}
console.log("twoSum([2, 7, 11, 15], 9):", twoSum([2, 7, 11, 15], 9));

// 2. First non-repeating character
function firstUnique(s) {
  const count = new Map();
  for (const ch of s) count.set(ch, (count.get(ch) ?? 0) + 1);
  for (const ch of s) if (count.get(ch) === 1) return ch;
  return null;
}
console.log("firstUnique('learnbeingforward'):", firstUnique("learnbeingforward"));

// 3. Group anagrams
function groupAnagrams(words) {
  const groups = new Map();
  for (const w of words) {
    const key = [...w].sort().join("");
    groups.set(key, [...(groups.get(key) ?? []), w]);
  }
  return [...groups.values()];
}
console.log("groupAnagrams:", JSON.stringify(groupAnagrams(["eat", "tea", "tan", "ate", "nat",
"bat"])));
```

```
// 4. How a hash table works inside: buckets + collision chaining
class HashTable {
  constructor(size = 7) { this.buckets = Array.from({ length: size }, () => []); }
  #hash(key) { let h = 0; for (const ch of String(key)) h = (h * 31 + ch.charCodeAt(0)) %
this.buckets.length; return h; }
  set(key, value) {
    const bucket = this.buckets[this.#hash(key)];
    const entry = bucket.find((e) => e[0] === key);
    entry ? (entry[1] = value) : bucket.push([key, value]);
  }
  get(key) { return this.buckets[this.#hash(key)].find((e) => e[0] === key)?.[1]; }
}
const ht = new HashTable();
[["Asha", 91], ["Ravi", 64], ["Meera", 85], ["John", 77], ["Zoya", 95]].forEach(([k, v]) => ht.set(k,
v));
console.log("get('Meera'):", ht.get("Meera"), "| buckets:", ht.buckets.map((b) => b.map((e) =>
e[0]).join("+") || "-").join(" | "));
```

OUTPUT

```
twoSum([2, 7, 11, 15], 9): [ 0, 1 ]
firstUnique('learnbeingforward'): l
groupAnagrams: [["eat","tea","ate"],["tan","nat"],["bat"]]
get('Meera'): 85 | buckets: Asha | Meera | - | John | Ravi+Zoya | - | -
```

25.2.4 Trees, binary search trees and traversals

A **tree** is a hierarchy of nodes: one root, and each node has children (the DOM, folders, organisation charts and JSON are trees). In a **binary tree** each node has at most two children. A **binary search tree** (BST) keeps every value in a node's left subtree smaller and every value in its right subtree larger, so search, insertion and deletion take O(h) time, where h is the height: O(log n) when balanced, O(n) in the worst case (sorted input makes a "linked list"). **Traversals** visit every node: depth-first in-order (left, node, right; gives sorted output for a BST), pre-order (node first; copies a tree), post-order (children first; deletes a tree or computes sizes), and breadth-first level-order using a queue.

A BINARY SEARCH TREE WITH ALL FOUR TRAVERSALS — main.js

```
class Node { constructor(value) { this.value = value; this.left = null; this.right = null; } }

class BST {
  root = null;
  insert(value) {
    const node = new Node(value);
    if (!this.root) { this.root = node; return this; }
    let cur = this.root;
    while (true) {
      if (value < cur.value) { if (!cur.left) { cur.left = node; break; } cur = cur.left; }
      else { if (!cur.right) { cur.right = node; break; } cur = cur.right; }
    }
    return this;
  }
  has(value) {
    let cur = this.root, steps = 0;
    while (cur) { steps++; if (value === cur.value) return { found: true, steps }; cur = value <
cur.value ? cur.left : cur.right; }
    return { found: false, steps };
  }
  inOrder(n = this.root, out = []) { if (n) { this.inOrder(n.left, out); out.push(n.value);
this.inOrder(n.right, out); } return out; }
  preOrder(n = this.root, out = []) { if (n) { out.push(n.value); this.preOrder(n.left, out);
this.preOrder(n.right, out); } return out; }
```

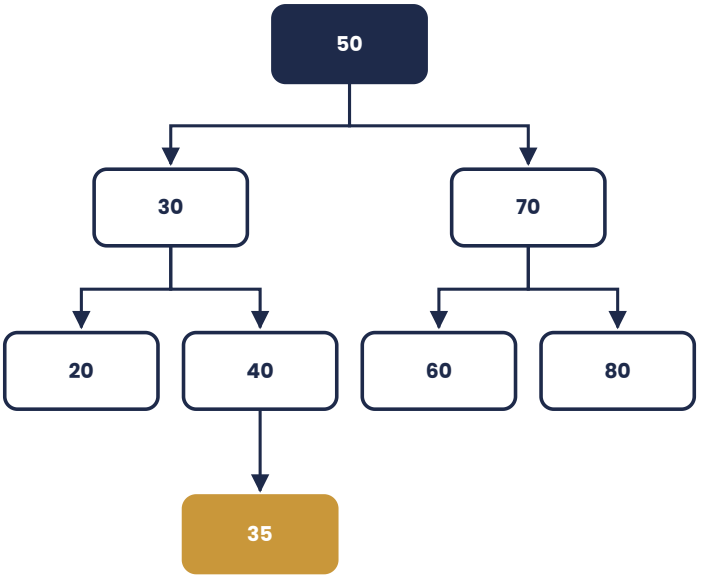
```
  postOrder(n = this.root, out = []) { if (n) { this.postOrder(n.left, out); this.postOrder(n.right,
out); out.push(n.value); } return out; }
  levelOrder() {
    const out = [], queue = this.root ? [this.root] : [];
    while (queue.length) {
      const level = [];
      for (let i = queue.length; i > 0; i--) {
        const n = queue.shift();
        level.push(n.value);
        if (n.left) queue.push(n.left);
        if (n.right) queue.push(n.right);
      }
      out.push(level);
    }
    return out;
  }
  height(n = this.root) { return n ? 1 + Math.max(this.height(n.left), this.height(n.right)) : 0; }
}

const tree = new BST();
[50, 30, 70, 20, 40, 60, 80, 35].forEach((v) => tree.insert(v));
console.log("in-order (sorted):", tree.inOrder().join(" "));
console.log("pre-order:        ", tree.preOrder().join(" "));
console.log("post-order:       ", tree.postOrder().join(" "));
console.log("level-order:      ", JSON.stringify(tree.levelOrder()));
console.log("height:", tree.height(), "| has(35):", tree.has(35), "| has(65):", tree.has(65));

const skewed = new BST();
[10, 20, 30, 40, 50, 60].forEach((v) => skewed.insert(v));
console.log("sorted inserts make a skewed tree of height", skewed.height(), "(like a linked list)");
```

OUTPUT

```
in-order (sorted): 20 30 35 40 50 60 70 80
pre-order:         50 30 20 40 35 70 60 80
post-order:        20 35 40 30 60 80 70 50
level-order:       [[50],[30,70],[20,40,60,80],[35]]
height: 4 | has(35): { found: true, steps: 4 } | has(65): { found: false, steps: 3 }
sorted inserts make a skewed tree of height 6 (like a linked list)
```



THE BST BUILT ABOVE; IN-ORDER TRAVERSAL VISITS 20 30 35 40 50 60 70 80

25.2.5 Graphs with BFS and DFS

A **graph** is a set of nodes (vertices) connected by edges, which may be directed (one-way links, like followers) or undirected (friendships, roads), and weighted (distances) or not. Graphs model networks, maps, dependencies (npm packages, course prerequisites) and recommendations. The usual representation in JavaScript is an **adjacency list**: a Map from each node to an array of neighbours. **Breadth-first search** (BFS) explores level by level with a queue and finds the **shortest path** (fewest edges) in an unweighted graph. **Depth-first search** (DFS) goes as deep as possible before backtracking, using recursion or a stack; it finds connected components, detects cycles and produces a **topological order** (an order respecting dependencies). Always track visited nodes to avoid infinite loops. Both run in O(V + E).

BFS, DFS AND TOPOLOGICAL SORT main.js

```
// An undirected graph of Indian cities (adjacency list)
const roads = [["Mysuru", "Bengaluru"], ["Bengaluru", "Chennai"], ["Bengaluru", "Hyderabad"],
["Chennai", "Hyderabad"],
  ["Hyderabad", "Pune"], ["Pune", "Mumbai"], ["Mysuru", "Mangaluru"], ["Mangaluru", "Goa"], ["Goa",
"Mumbai"]];
const graph = new Map();
for (const [a, b] of roads) {
  graph.set(a, [...(graph.get(a) ?? []), b]);
  graph.set(b, [...(graph.get(b) ?? []), a]);
}

function bfsPath(start, goal) {
  const queue = [[start]], visited = new Set([start]);
  while (queue.length) {
    const path = queue.shift();
    const node = path.at(-1);
    if (node === goal) return path;
    for (const next of graph.get(node) ?? []) {
      if (!visited.has(next)) { visited.add(next); queue.push([...path, next]); }
    }
  }
  return null;
}

function dfsOrder(start, visited = new Set(), order = []) {
  visited.add(start);
  order.push(start);
  for (const next of graph.get(start)) if (!visited.has(next)) dfsOrder(next, visited, order);
  return order;
}

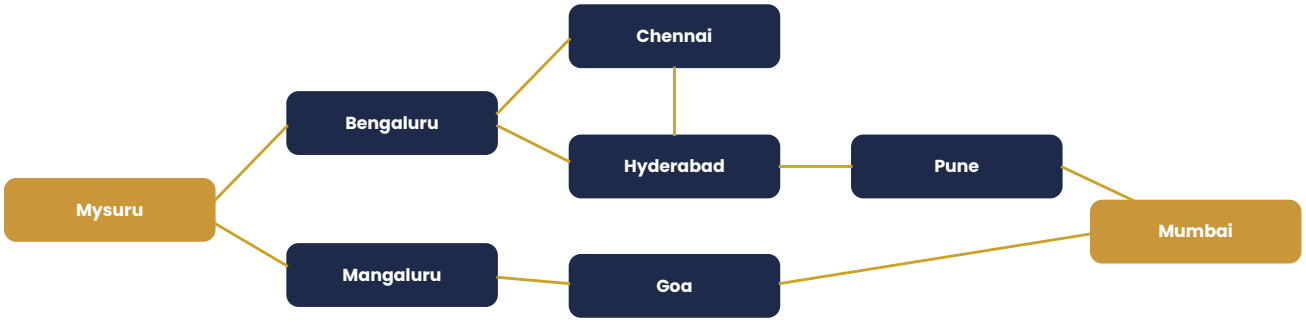
console.log("BFS shortest route Mysuru -> Mumbai:", bfsPath("Mysuru", "Mumbai").join(" -> "));
console.log("DFS visiting order from Mysuru:     ", dfsOrder("Mysuru").join(", "));

// Topological sort of course prerequisites (directed graph, DFS-based)
const prereqs = { "HTML": [], "CSS": ["HTML"], "JavaScript": ["HTML"], "React": ["JavaScript", "CSS"],
"Node.js": ["JavaScript"], "Next.js": ["React", "Node.js"] };
function topoSort(deps) {
  const done = new Set(), inProgress = new Set(), order = [];
  const visit = (course) => {
    if (done.has(course)) return;
    if (inProgress.has(course)) throw new Error(`cycle at ${course}`);
    inProgress.add(course);
    deps[course].forEach(visit);
    inProgress.delete(course); done.add(course); order.push(course);
  };
  Object.keys(deps).forEach(visit);
  return order;
```

```
}
console.log("study order:", topoSort(prereqs).join(" -> "));
```

OUTPUT

```
BFS shortest route Mysuru -> Mumbai: Mysuru -> Mangaluru -> Goa -> Mumbai
DFS visiting order from Mysuru:      Mysuru, Bengaluru, Chennai, Hyderabad, Pune, Mumbai, Goa,
Mangaluru
study order: HTML -> CSS -> JavaScript -> React -> Node.js -> Next.js
```



THE ROAD GRAPH (BFS FINDS MYSURU -> MANGALURU -> GOA -> MUMBAI)

25.2.6 Sorting: merge sort and quick sort

`Array.prototype.sort` (TimSort in V8, stable, O(n log n)) is what you use in practice, with a comparator for numbers (`(a, b) => a - b`; the default sorts as strings). Interviews ask you to implement the classic divide-and-conquer algorithms. **Merge sort** splits the array in halves recursively, sorts each half, and merges the two sorted halves: always O(n log n), stable, but needs O(n) extra memory. **Quick sort** picks a **pivot**, partitions the elements into those smaller and larger than it, and sorts each part recursively: O(n log n) on average and in place, but $O(n^2)$ in the worst case (consistently bad pivots, such as always the first element of a sorted array), which random or median-of-three pivots avoid. Simple $O(n^2)$ sorts (bubble, selection, insertion) are only for tiny or nearly sorted inputs.

MERGE SORT AND QUICK SORT

main.js

```
function mergeSort(arr) {
  if (arr.length <= 1) return arr;
  const mid = arr.length >> 1;
  const left = mergeSort(arr.slice(0, mid)), right = mergeSort(arr.slice(mid));
  const merged = [];
  let i = 0, j = 0;
  while (i < left.length && j < right.length) merged.push(left[i] <= right[j] ? left[i++] :
right[j++]);   // <= keeps it stable
  return merged.concat(left.slice(i), right.slice(j));
}

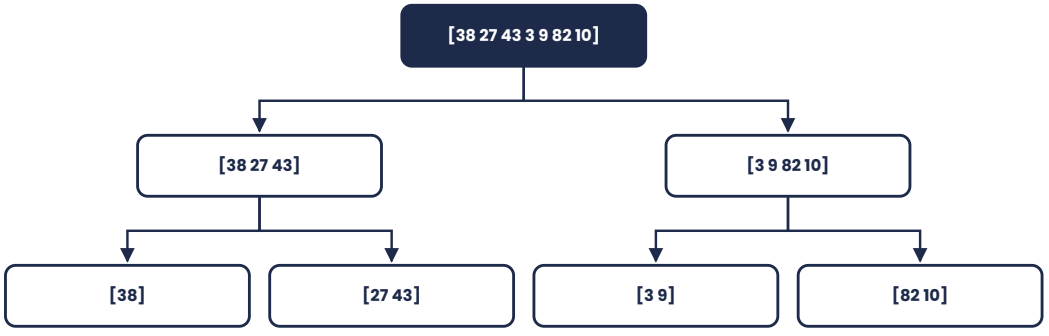
function quickSort(arr, lo = 0, hi = arr.length - 1) {     // in place, Lomuto partition
  if (lo >= hi) return arr;
  const pivotIndex = lo + ((hi - lo) >> 1);                  // middle element as pivot
  [arr[pivotIndex], arr[hi]] = [arr[hi], arr[pivotIndex]];
  const pivot = arr[hi];
  let store = lo;
  for (let i = lo; i < hi; i++) {
    if (arr[i] < pivot) { [arr[i], arr[store]] = [arr[store], arr[i]]; store++; }
  }
  [arr[store], arr[hi]] = [arr[hi], arr[store]];             // pivot to its final place
  quickSort(arr, lo, store - 1);
  quickSort(arr, store + 1, hi);
  return arr;
}

const data = [38, 27, 43, 3, 9, 82, 10, 27];
console.log("mergeSort:", mergeSort(data).join(" "), "| original untouched:", data.join(" "));
console.log("quickSort:", quickSort([...data]).join(" "));
console.log("default sort compares strings:", [10, 9, 100, 1].sort().join(" "), "| numeric:", [10, 9,
100, 1].sort((a, b) => a - b).join(" "));

const big = Array.from({ length: 100000 }, (_, i) => (i * 7919) % 100003);
const sortedCopy = big.toSorted((a, b) => a - b);
console.log("100k items: merge sort correct:", mergeSort(big).every((v, i) => v === sortedCopy[i]), "|
quick sort correct:", quickSort([...big]).every((v, i) => v === sortedCopy[i]));
```

OUTPUT

```
mergeSort: 3 9 10 27 27 38 43 82 | original untouched: 38 27 43 3 9 82 10 27
quickSort: 3 9 10 27 27 38 43 82
default sort compares strings: 1 10 100 9 | numeric: 1 9 10 100
100k items: merge sort correct: true | quick sort correct: true
```



MERGE SORT SPLITS UNTIL SINGLE ELEMENTS, THEN MERGES SORTED HALVES BACK UP

Algorithm	Best	Average	Worst	Extra space	Stable
Merge sort	n log n	n log n	n log n	O(n)	yes

Algorithm	Best	Average	Worst	Extra space	Stable
Quick sort	n log n	n log n	n²	O(log n)	no
Insertion sort	n	n²	n²	O(1)	yes
Array.prototype.sort (TimSort)	n	n log n	n log n	O(n)	yes

25.2.7 Two pointers, sliding window, recursion, dynamic programming

Most interview problems are variations of a handful of **patterns**. **Two pointers**: two indexes moving through an array, from both ends towards the middle (pair sums in a sorted array, palindromes, reversing) or at different speeds (removing duplicates in place, detecting a cycle). **Sliding window**: maintain a window over a contiguous range and slide it, updating the answer in O(1) per step instead of recomputing (maximum sum of k consecutive items, longest substring without repeating characters). **Recursion**: a function that solves a problem by calling itself on smaller inputs, with a base case; natural for trees, permutations and divide and conquer. **Dynamic programming** (DP): when recursive sub-problems overlap, store their answers (memoisation, top-down, or a table built bottom-up) to turn exponential time into polynomial time (Fibonacci, climbing stairs, coin change, longest common subsequence).

THE FOUR CORE PROBLEM-SOLVING PATTERNS main.js

```
// Two pointers: pair with a given sum in a sorted array, O(n)
function pairWithSum(sorted, target) {
  let i = 0, j = sorted.length - 1;
  while (i < j) {
    const sum = sorted[i] + sorted[j];
    if (sum === target) return [sorted[i], sorted[j]];
    sum < target ? i++ : j--;
  }
  return null;
}
const isPalindrome = (s) => { const t = s.toLowerCase().replace(/[^a-z0-9]/g, ""); for (let i = 0, j =
t.length - 1; i < j; i++, j--) if (t[i] !== t[j]) return false; return true; };
console.log("pairWithSum:", pairWithSum([1, 3, 4, 6, 8, 11], 10), "| palindrome:", isPalindrome("A
man, a plan, a canal: Panama"));

// Sliding window: best 3-day sales and longest substring without repeats
function maxWindowSum(nums, k) {
  let sum = nums.slice(0, k).reduce((a, b) => a + b, 0), best = sum;
  for (let i = k; i < nums.length; i++) { sum += nums[i] - nums[i - k]; best = Math.max(best, sum); }
  return best;
}
function longestUniqueSubstring(s) {
  const last = new Map(); let start = 0, best = "";
  for (let i = 0; i < s.length; i++) {
    if (last.has(s[i]) && last.get(s[i]) >= start) start = last.get(s[i]) + 1;
    last.set(s[i], i);
    if (i - start + 1 > best.length) best = s.slice(start, i + 1);
  }
  return best;
}
console.log("max 3-day sales:", maxWindowSum([120, 80, 200, 150, 90, 300, 60], 3), "| longest
unique:", longestUniqueSubstring("abcabcbbxyz"));

// Recursion: all permutations, and flattening nested data
const permutations = (arr) => arr.length <= 1 ? [arr] : arr.flatMap((x, i) =>
permutations([...arr.slice(0, i), ...arr.slice(i + 1)]).map((p) => [x, ...p]));
const deepSum = (v) => Array.isArray(v) ? v.reduce((t, x) => t + deepSum(x), 0) : v;
```

```
console.log("permutations of ABC:", permutations(["A", "B", "C"]).map((p) => p.join("")).join(" "), "|
deepSum:", deepSum([1, [2, [3, [4]], 5]]));

// Dynamic programming: naive vs memoised vs bottom-up
let calls = 0;
const fibNaive = (n) => { calls++; return n < 2 ? n : fibNaive(n - 1) + fibNaive(n - 2); };
fibNaive(25);
const memo = new Map();
const fibMemo = (n) => n < 2 ? n : memo.get(n) ?? (memo.set(n, fibMemo(n - 1) + fibMemo(n - 2)),
memo.get(n));
console.log(`fib(25) naive made ${calls} calls; memoised fib(78) = ${fibMemo(78)}`);

function coinChange(coins, amount) {                    // fewest coins, bottom-up table
  const dp = Array(amount + 1).fill(Infinity);
  dp[0] = 0;
  for (let a = 1; a <= amount; a++) for (const c of coins) if (c <= a) dp[a] = Math.min(dp[a], dp[a -
c] + 1);
  return dp[amount] === Infinity ? -1 : dp[amount];
}
const climbStairs = (n) => { let [a, b] = [1, 1]; for (let i = 2; i <= n; i++) [a, b] = [b, a + b];
return b; };
console.log("coinChange([1, 2, 5, 10], 27):", coinChange([1, 2, 5, 10], 27), "| ways to climb 10
stairs (1 or 2 steps):", climbStairs(10));
```

```
OUTPUT
pairWithSum: [ 4, 6 ] | palindrome: true
max 3-day sales: 540 | longest unique: bxyz
permutations of ABC: ABC ACB BAC BCA CAB CBA | deepSum: 15
fib(25) naive made 242785 calls; memoised fib(78) = 8944394323791464
coinChange([1, 2, 5, 10], 27): 4 | ways to climb 10 stairs (1 or 2 steps): 89
```

Pattern	Signals in the problem	Classic problems
Hash map / set	"find pair / duplicate / count / group", O(n) wanted	two sum, anagrams, first unique character
Two pointers	sorted array, pairs, in-place changes, palindromes	pair sum, remove duplicates, container with most water
Sliding window	contiguous subarray or substring, "longest/shortest/max k"	max sum of k items, longest substring without repeats
Stack	matching, "next greater", undo, nested structure	valid parentheses, next greater element
BFS / DFS	grids, trees, graphs, levels, shortest steps	number of islands, level order, shortest path
Binary search	sorted data, "minimum value that works"	search insert position, rotated array
Dynamic programming	count ways, min/max cost, overlapping sub-problems	climbing stairs, coin change, LCS, knapsack

25.3 Capstone projects

Projects prove your skills far better than certificates. Build at least three of the following, put each on GitHub with a clear README (what it does, screenshots, live link, how to run, what you learnt), deploy it, and be ready to explain every line in an interview. Each brief lists the core features, the concepts it exercises, and stretch goals for going further.

25.3.1 Personal portfolio website

A fast, responsive site that presents you to recruiters: who you are, your skills, your projects with live links and code, and a way to contact you. It is often the first thing an interviewer opens, so polish matters: performance, accessibility and mobile layout.

Core features	Concepts practised	Stretch goals
• hero section with name, role and call to action • projects grid loaded from a JSON file • filter projects by technology • dark/light theme toggle saved in localStorage • contact form with validation (Formspree/EmailJS)	• semantic HTML and responsive CSS (Flexbox, Grid) • DOM rendering from data, events and delegation • fetch for JSON, localStorage • accessibility and Lighthouse optimisation • deployment on Netlify/Vercel with a custom domain	• rebuild in React or Astro • blog with Markdown posts • animations with IntersectionObserver • Lighthouse score above 95 in all categories

25.3.2 Expense tracker with charts and localStorage

Record income and expenses by category and date, see the balance, filter by month, and visualise spending with charts. All data persists in the browser so the app works offline.

Core features	Concepts practised	Stretch goals
• add, edit and delete transactions • categories, dates and running balance • monthly filter and search • pie chart by category, bar chart by month • data saved in localStorage; export to CSV	• forms and validation, array methods (filter, reduce, groupBy) • Date and Intl formatting for rupees • Canvas or a chart library (Chart.js) • JSON with localStorage, modules • clean state management: render from one state object	• budgets with alerts • IndexedDB and PWA offline support • TypeScript conversion • sync to a back end with login

25.3.3 Quiz app with a timer and score tracking

A multiple-choice quiz that loads questions from an API or JSON file, times each question, gives instant feedback, and keeps a high-score table.

Core features	Concepts practised	Stretch goals
• questions from the Open Trivia DB API or a JSON file • countdown timer per question • instant correct/incorrect feedback and final score • shuffle questions and options • high scores saved in localStorage	• fetch with loading and error states • setInterval/clearInterval, timing logic • state machines (start, question, result) • keyboard navigation and ARIA live regions • Fisher-Yates shuffle	• categories and difficulty levels • leaderboard API with Express • multiplayer with WebSockets • React version with tests

25.3.4 E-commerce cart with product API

A shop front that lists products from an API (such as FakeStoreAPI or your own), with search, filters, a product detail page, and a cart with quantities and totals that persists between visits.

Core features	Concepts practised	Stretch goals
• product grid from an API, with skeleton loading • search, category filter and sort by price/rating • product detail view (hash or History API routing) • cart: add, change quantity, remove, totals with GST • cart persisted in localStorage	• async/await and error handling • event delegation, debounced search • routing with the History API • immutable state updates, derived totals • performance: lazy images, pagination	• checkout form with validation • wishlist and coupons • React + Context or Redux Toolkit • payment integration in test mode (Razorpay/Stripe)

25.3.5 Full-stack app (Node, Express, database, front end) with authentication

A complete application with users: for example a course-enrolment portal, a job board or a student management system (Module 22). This is the project that shows you can build and deploy a real product end to end.

Core features	Concepts practised	Stretch goals
• registration and login with hashed passwords and JWT • role-based access (admin, user) • CRUD for the main resource with validation • search, filters and pagination • front end (React or vanilla) consuming the API	• Express routing, middleware and error handling • MongoDB/Mongoose or SQL with an ORM • security: bcrypt, JWT, CORS, Helmet, rate limiting • environment variables and deployment (Render + Atlas) • API tests with Supertest	• refresh tokens in HttpOnly cookies • file uploads to cloud storage • email verification and password reset • Docker and a CI pipeline

25.3.6 Real-time chat app with WebSockets

A chat application with rooms, online presence and typing indicators, where messages appear instantly for everyone in the room.

Core features	Concepts practised	Stretch goals
• join with a name; list of rooms • real-time messages with Socket.IO or ws • online users list and "typing…" indicator • message history stored in a database • responsive UI with timestamps	• WebSockets and event-based protocols • Node.js server with rooms and broadcasting • reconnection and error handling • escaping user content (XSS) • deployment of a long-running server	• private messages and read receipts • authentication with JWT on the socket • file and image sharing • push notifications via service worker

25.4 Interview preparation

JavaScript interviews in India typically have three to five rounds: an online assessment (aptitude, MCQs, one or two coding problems), a JavaScript fundamentals round (concepts and output prediction), a machine-coding round (build a small feature in 60-90 minutes), a DSA round, and a managerial/HR round. Prepare for each deliberately.

25.4.1 Core topics checklist: hoisting, closures, this, prototypes, event loop, promises, == vs ===, var/let/const

Interviewers return to the same core concepts again and again. For each, you should be able to give a one-sentence definition, a small example, and a practical use or pitfall. The table summarises the expected answers and the module where each is covered in depth.

Topic	What to be able to explain	Module
var / let / const	function vs block scope, redeclaration, TDZ, const objects are still mutable	3, 11
Hoisting	creation phase: var as undefined, functions fully, let/const/class in TDZ	11
Closures	functions remember their lexical scope; private state, counters, once, memoize; the loop + var bug	11
this	determined by the call: method, plain call (undefined in strict), new, call/apply/bind, arrows inherit	11
Prototypes and classes	prototype chain, Object.create, class as syntax over prototypes, extends/super	13
== vs ===	coercion rules of ==; always use ===; Object.is for NaN and -0	4
Event loop	call stack, Web APIs, microtasks before macrotasks, setTimeout(fn, 0)	17
Promises and async/await	states, chaining, all/allSettled/race/any, error handling, sequential vs parallel	17
Scope and the scope chain	global, function, block, module; lexical scoping	11
Spread, rest, destructuring	copying (shallow), defaults, parameter patterns	12
Debounce and throttle	definitions, implementations and use cases	11, 16
Event delegation and propagation	bubbling/capturing, target vs currentTarget, stopPropagation vs preventDefault	16
Shallow vs deep copy	spread/Object.assign vs structuredClone	9, 12
Map/Set vs object/array	key types, order, performance, WeakMap	12
Modules	ESM vs CommonJS, named vs default exports, dynamic import	19

25.4.2 Output-prediction questions

"What does this print?" questions test whether you really understand the rules rather than having memorised definitions. Read each snippet and write down your prediction **before** looking at the output; then explain each line. The answers below come from actually running the code.

TEN OUTPUT-PREDICTION QUESTIONS main.js

```
// Q1: hoisting and the TDZ
console.log("Q1:", typeof a, typeof b);
var a = 1;
function b() {}

// Q2: closures in loops
const fs1 = [], fs2 = [];
for (var i = 0; i < 3; i++) fs1.push(() => i);
for (let j = 0; j < 3; j++) fs2.push(() => j);
```

```
console.log("Q2:", fs1.map((f) => f()).join(","), "|", fs2.map((f) => f()).join(","));

// Q3: this in methods and arrows
const obj = { name: "LBF", regular() { return this?.name; }, arrow: () => typeof this, nested() {
return [1].map(() => this.name)[0]; } };
const detached = obj.regular;
console.log("Q3:", obj.regular(), obj.arrow(), obj.nested(), detached.call({ name: "other" }));

// Q4: coercion
console.log("Q4:", [] + [], "|", [] + {}, "|", 1 + "2" - 1, "|", "5" * "2", "|", true + true, "|", []
== ![], "|", null == undefined, null === undefined);

// Q5: NaN and Object.is
console.log("Q5:", NaN === NaN, Object.is(NaN, NaN), [NaN].includes(NaN), [NaN].indexOf(NaN), 0 ===
-0, Object.is(0, -0));

// Q6: objects as keys and references
const k1 = {}, k2 = {}, store = {};
store[k1] = "first"; store[k2] = "second";
const x = { n: 1 }, y = x; y.n = 2;
console.log("Q6:", store[k1], "|", x.n, "|", { a: 1 } === { a: 1 });

// Q7: default sort and typeof surprises
console.log("Q7:", [10, 1, 5, 100].sort().join(" "), "|", typeof null, typeof [], typeof function ()
{}, typeof NaN);

// Q8: the event loop
const order = [];
setTimeout(() => order.push("timeout"), 0);
Promise.resolve().then(() => order.push("promise"));
queueMicrotask(() => order.push("microtask"));
(async () => { order.push("async-start"); await null; order.push("after-await"); })();
order.push("sync");
setTimeout(() => console.log("Q8:", order.join(" > ")), 10);

// Q9: prototypes
function Animal() {}
Animal.prototype.speak = function () { return "..."; };
const dog = new Animal();
Animal.prototype.speak = function () { return "woof"; };
dog.speak2 = () => "own";
console.log("Q9:", dog.speak(), Object.keys(dog), dog instanceof Animal, Object.getPrototypeOf(dog)
=== Animal.prototype);

// Q10: parseInt and map
console.log("Q10:", ["1", "2", "3"].map(parseInt), ["1", "2", "3"].map(Number), parseInt("08"),
parseInt("0x1F"), parseFloat("3.14abc"));
```

OUTPUT

```
Q1: undefined function
Q2: 3,3,3 | 0,1,2
Q3: LBF object LBF other
Q4:  | [object Object] | 11 | 10 | 2 | true | true false
Q5: false true true -1 true false
Q6: second | 2 | false
Q7: 1 10 100 5 | object object function number
Q9: woof [ 'speak2' ] true true
Q10: [ 1, NaN, NaN ] [ 1, 2, 3 ] 8 31 3.14
Q8: async-start > sync > promise > microtask > after-await > timeout
```

Q	Why
1	`var a` is hoisted as undefined; the function declaration is hoisted completely
2	one shared `var i` ends at 3; `let` creates a new binding per iteration
3	method call: this is obj; arrows take the outer this (module scope: undefined in a strict module, {} in CommonJS here); call sets this explicitly
4	[] + [] is "" ; [] + {} is "[object Object]"; "12" - 1 = 11; "5" * "2" = 10; true + true = 2; [] == ![] is true through coercion
5	NaN is not equal to itself; includes uses SameValueZero, indexOf uses ===
6	object keys become the string "[object Object]"; y and x reference one object; two literals are different objects
7	the default sort compares strings; typeof null is "object" (a historical bug)
8	sync code first (including the start of the async function), then microtasks in order, then the timeout
9	lookups go through the prototype at call time, so the replaced method is used; speak2 is the only own key
10	map passes (value, index): parseInt("2", 1) and parseInt("3", 2) are NaN

25.4.3 Machine-coding rounds (build a to-do, debounce, infinite scroll)

In a **machine-coding round** you build a small working feature, usually in plain JavaScript or React, within 60-90 minutes, while explaining your choices. Typical tasks: to-do list, autocomplete/search with debounce, infinite scroll, star rating, accordion, tabs, modal, image carousel, pagination, progress bar, OTP input, nested comments, file explorer tree, kanban board, Tic-Tac-Toe. Interviewers look for: **working** core functionality first; clean structure (state in one place, small functions, a render function); event delegation; edge cases (empty input, errors, fast typing, double clicks); accessibility (buttons, labels, keyboard); and communication. Strategy: clarify requirements (2-3 minutes), sketch the state and components, build the simplest end-to-end version, then add polish, and test as you go. A complete example follows: an autocomplete that fetches suggestions with debounce, cancels stale requests, caches results, highlights matches and supports keyboard navigation.

THE PAGE THIS EXAMPLE RUNS IN (HTML BODY) HTML

```
<label for="search">Course</label>
<input id="search" role="combobox" aria-controls="suggestions" aria-autocomplete="list"
autocomplete="off">
<ul id="suggestions" role="listbox"></ul>
```

MACHINE-CODING EXAMPLE: AN ACCESSIBLE, DEBOUNCED AUTOCOMPLETE main.js

```
const input = document.querySelector("#search");
const list = document.querySelector("#suggestions");
const cache = new Map();
let controller, active = -1, items = [];

const debounce = (fn, ms) => { let t; return (...a) => { clearTimeout(t); t = setTimeout(() =>
fn(...a), ms); }; };

async function getSuggestions(q) {
  if (cache.has(q)) return cache.get(q);
  controller?.abort();                                  // cancel the previous request
  controller = new AbortController();
  const res = await fetch(`/api/courses?q=${encodeURIComponent(q)}`, { signal: controller.signal });
  if (!res.ok) throw new Error("search failed");
  const data = await res.json();
```

```
  cache.set(q, data);
  return data;
}

function render(q) {
  list.replaceChildren(...items.map((name, i) => {
    const li = document.createElement("li");
    li.id = `opt-${i}`;
    li.setAttribute("role", "option");
    li.setAttribute("aria-selected", String(i === active));
    const at = name.toLowerCase().indexOf(q.toLowerCase());
    li.append(name.slice(0, at), Object.assign(document.createElement("mark"), { textContent:
name.slice(at, at + q.length) }), name.slice(at + q.length));
    return li;
  }));
  input.setAttribute("aria-activedescendant", active >= 0 ? `opt-${active}` : "");
}

input.addEventListener("input", debounce(async () => {
  const q = input.value.trim();
  active = -1;
  if (!q) { items = []; return render(""); }
  try {
    items = await getSuggestions(q);
    render(q);
    console.log(`suggestions for "${q}": ${items.join(", ") || "none"}`);
  } catch (e) {
    if (e.name !== "AbortError") console.log("error:", e.message);
  }
}, 250));

input.addEventListener("keydown", (e) => {
  if (!items.length) return;
  if (e.key === "ArrowDown") active = (active + 1) % items.length;
  else if (e.key === "ArrowUp") active = (active - 1 + items.length) % items.length;
  else if (e.key === "Enter" && active >= 0) { input.value = items[active]; items = [];
console.log("selected:", input.value); }
  else return;
  e.preventDefault();
  render(input.value);
});
list.addEventListener("click", (e) => { const li = e.target.closest("li"); if (li) { input.value =
li.textContent; items = []; render(""); console.log("clicked:", input.value); } });
```

BROWSER CONSOLE

```
[server] GET /api/courses?q=java
suggestions for "java": Java, JavaScript, JavaScript Advanced
selected: JavaScript
[server] GET /api/courses?q=react
suggestions for "react": React, React Native
clicked: React Native
--- element HTML ---
<input id="search" role="combobox" aria-controls="suggestions" aria-autocomplete="list"
autocomplete="off" aria-activedescendant="">
```

Simulated user actions, in order: types "java" into `#search`; waits 400 ms; presses ArrowDown in `#search`; presses ArrowDown in `#search`; presses Enter in `#search`; types "react" into `#search`; waits 400 ms; clicks `#suggestions li:nth-child(2)`.

Debounce, throttle and infinite scroll implementations were built in Modules 11, 16 and 21; practise writing each from memory in under ten minutes, then the classic React versions with `useState`, `useEffect` and a custom `useDebounce` hook.

25.4.4 Polyfill questions

"Implement X yourself" questions check that you understand how built-ins behave. Module 20 implemented `map`, `filter`, `reduce`, `bind` and `Promise.all`. Other favourites follow: `call`/`apply`, `Promise.allSettled`, `Promise.race`, `Array.prototype.flat`, `once`, `memoize`, a deep clone, `debounce` and `throttle`, `curry`, and a small `EventEmitter`. State the edge cases you handle (empty input, holes, non-function arguments, `this`).

MORE POLYFILL QUESTIONS

main.js

```
// call and apply: temporarily attach the function to the object
Function.prototype.myCall = function (ctx, ...args) {
  ctx = ctx ?? globalThis;
  const key = Symbol("fn");                       // unique key: no clash with existing properties
  ctx[key] = this;
  try { return ctx[key](...args); } finally { delete ctx[key]; }
};
Function.prototype.myApply = function (ctx, args = []) { return this.myCall(ctx, ...args); };
function intro(city, year) { return `${this.name} from ${city}, batch ${year}`; }
console.log(intro.myCall({ name: "Asha" }, "Mysuru", 2026), "|", intro.myApply({ name: "Ravi" },
["Pune", 2025]));

// flat with depth
function flat(arr, depth = 1) {
  return depth < 1 ? arr.slice() : arr.reduce((acc, v) => acc.concat(Array.isArray(v) ? flat(v, depth
- 1) : [v]), []);
}
console.log("flat:", JSON.stringify(flat([1, [2, [3, [4]]]])), JSON.stringify(flat([1, [2, [3, [4]]]],
Infinity)));

// Promise.allSettled and Promise.race
const allSettled = (ps) => Promise.all([...ps].map((p) => Promise.resolve(p).then((value) => ({
status: "fulfilled", value }), (reason) => ({ status: "rejected", reason: reason.message }))));
const race = (ps) => new Promise((res, rej) => [...ps].forEach((p) => Promise.resolve(p).then(res,
rej)));
const later = (v, ms, fail) => new Promise((r, j) => setTimeout(() => (fail ? j(new Error(v)) : r(v)),
ms));
allSettled([later("ok", 10), later("down", 5, true), 42]).then((r) => console.log("allSettled:",
JSON.stringify(r)));
race([later("slow", 30), later("fast", 5)]).then((v) => console.log("race:", v));

// once and memoize
const once = (fn) => { let done = false, result; return (...a) => (done ? result : ((done = true),
(result = fn(...a)))); };
const init = once(() => { console.log("  initialising..."); return "ready"; });
console.log("once:", init(), init());
const memoize = (fn) => { const c = new Map(); return (...a) => { const k = JSON.stringify(a); if
(!c.has(k)) c.set(k, fn(...a)); return c.get(k); }; };
let slowCalls = 0;
const square = memoize((n) => (slowCalls++, n * n));
square(9); square(9); square(4);
console.log("memoize: real calls =", slowCalls);

// deep clone handling dates, arrays, nested objects and cycles
function deepClone(value, seen = new WeakMap()) {
  if (value === null || typeof value !== "object") return value;
  if (value instanceof Date) return new Date(value);
  if (seen.has(value)) return seen.get(value);
  const copy = Array.isArray(value) ? [] : Object.create(Object.getPrototypeOf(value));
  seen.set(value, copy);
  for (const key of Reflect.ownKeys(value)) copy[key] = deepClone(value[key], seen);
  return copy;
}
```

```
const original = { name: "Asha", joined: new Date(2026, 2, 9), marks: [91, [88]] };
original.self = original;
const cloned = deepClone(original);
cloned.marks[1].push(100);
console.log("deepClone independent:", original.marks[1].length === 1, "| date copied:", cloned.joined
instanceof Date, "| cycle kept:", cloned.self === cloned);
```

```
OUTPUT
Asha from Mysuru, batch 2026 | Ravi from Pune, batch 2025
flat: [1,2,[3,[4]]] [1,2,3,4]
  initialising...
once: ready ready
memoize: real calls = 2
deepClone independent: true | date copied: true | cycle kept: true
race: fast
allSettled: [{"status":"fulfilled","value":"ok"},{"status":"rejected","reason":"down"},
{"status":"fulfilled","value":42}]
```

25.4.5 DSA coding rounds

In DSA rounds (on a shared editor, HackerRank or LeetCode-style platforms), follow a repeatable process and **talk through it**: (1) restate the problem and ask about input size, edge cases and constraints; (2) work through an example by hand; (3) describe a brute-force solution and its complexity; (4) improve it using a pattern (25.2.7); (5) code it cleanly with good names; (6) test with the example and edge cases (empty, one element, duplicates, negative numbers, very large input); (7) state the final time and space complexity. Practise consistently, a few problems a day, organised by pattern, rather than random problems. In JavaScript, know the useful built-ins: `Map`, `Set`, `sort` with comparators, `Array.from({ length: n }, ...)`, `Math.max(...arr)` (careful with very large arrays), `at(-1)`, string methods and `BigInt` for huge numbers.

Week	Topics	Target problems
1	arrays, strings, hash maps	15 (two sum, anagrams, frequency counts, prefix sums)
2	two pointers, sliding window, binary search	12
3	stacks, queues, linked lists	10
4	recursion, trees, BST	10
5	graphs (BFS/DFS), heaps	8
6	dynamic programming, greedy, revision	10 + timed mock interviews

TESTING YOUR OWN SOLUTION WITH EDGE CASES

main.js

```
// A worked DSA answer: "Find the length of the longest consecutive sequence in O(n)"
// 1. brute force: sort and scan -> O(n log n). 2. better: a Set, and only start counting at sequence
beginnings.
function longestConsecutive(nums) {
  const set = new Set(nums);
  let best = 0;
  for (const n of set) {
    if (set.has(n - 1)) continue;                // not the start of a run
    let length = 1;
    while (set.has(n + length)) length++;
    best = Math.max(best, length);
  }
  return best;                                   // each number is visited at most twice: O(n) time,
O(n) space
}

const tests = [
  [[100, 4, 200, 1, 3, 2], 4],
  [[0, 3, 7, 2, 5, 8, 4, 6, 0, 1], 9],
  [[], 0],
  [[5], 1],
  [[-2, -1, -3, 10], 3],
];
for (const [input, expected] of tests) {
  const got = longestConsecutive(input);
  console.log(`${JSON.stringify(input).padEnd(26)} -> ${got} ${got === expected ? "✓" : `✗ expected
${expected}`}`);
}
```

OUTPUT

```
[100,4,200,1,3,2]          -> 4 ✓
[0,3,7,2,5,8,4,6,0,1]      -> 9 ✓
[]                         -> 0 ✓
[5]                        -> 1 ✓
[-2,-1,-3,10]              -> 3 ✓
```

25.4.6 Presenting your portfolio and GitHub

Your GitHub profile and portfolio are your evidence. Pin your three to six best repositories; each needs a README with a one-line description, a screenshot or GIF, the live link, the tech stack, features, how to run it locally, and what you would improve next. Commit regularly with meaningful messages over weeks (a single "final commit" looks copied). Write a profile README introducing yourself. On your resume, describe projects by impact and technology ("Built a full-stack course portal with React, Express and MongoDB; JWT auth, role-based access, 40+ API tests, deployed on Render"), and link them. In the interview, prepare a two-minute walkthrough of one project: the problem, your architecture, one hard bug you solved and how, and what you learnt. Be honest about what you did yourself; interviewers will ask you to change something live.

README

Description, screenshot, live demo, stack, features, setup steps, future work.

Commit history

Small, regular, descriptive commits on a main branch with feature branches.

Live demos

Every project deployed and working; test the links before every application.

Code quality
ESLint/Prettier, no secrets in the repo (.env ignored), some tests, consistent structure.

LinkedIn and resume
Match your projects and skills; quantify results; link GitHub and portfolio.

Storytelling
Problem, approach, challenges, results, learning: practise saying it out loud.

25.5 Career paths with JavaScript

JavaScript is the most used programming language in the world and opens several career paths, from start-ups to large product and service companies. Most developers start in one area and broaden over time. Typical entry titles in India are Associate/Junior Software Engineer, Front-end Developer, Web Developer and Full-stack Developer Trainee.

25.5.1 Front-end developer

Builds the user interface of web applications: turning designs (Figma) into responsive, accessible, fast pages and components, managing client-side state, integrating APIs, and ensuring cross-browser quality. Core skills: HTML, CSS (Flexbox, Grid, Tailwind), JavaScript and TypeScript, React (or Angular/Vue), state management, testing (Vitest, Testing Library, Playwright), performance and accessibility, Git. Growth: senior front-end engineer, UI architect, design-systems engineer.

25.5.2 Back-end Node.js developer

Builds the server side: APIs, business logic, databases, authentication, integrations with payment and messaging services, background jobs, and the reliability, security and performance of services. Core skills: Node.js, Express/NestJS/Fastify, REST and GraphQL, SQL and MongoDB, caching (Redis), queues, authentication and security, testing, Docker, cloud basics (AWS/GCP/Azure), logging and monitoring. Growth: senior back-end engineer, platform/DevOps engineer, solutions architect.

25.5.3 Full-stack (MERN) developer

Works across the whole application, from the database to the UI, which makes them especially valuable in start-ups and small teams. Core skills: the MERN stack (or Next.js with a SQL database and an ORM), REST API design, authentication, deployment and CI/CD, plus enough design sense to build usable interfaces. Growth: senior full-stack engineer, tech lead, founding engineer, engineering manager.

25.5.4 Mobile developer with React Native

Builds iOS and Android apps from one JavaScript/TypeScript codebase with **React Native** (used by Meta, Microsoft, Shopify and many Indian apps), reusing React knowledge: components, props, state and hooks, with native components instead of HTML elements. Expo simplifies building, testing on devices and publishing to app stores. Core skills: React, React Native and Expo, navigation, device APIs (camera, location, notifications), offline storage, performance on low-end devices, app-store release. Growth: senior mobile engineer, cross-platform architect; alternatives include Ionic/Capacitor and native Kotlin/Swift.

Path	Must-have skills	Typical first projects	Tools
Front-end	HTML, CSS, JS, TS, React, accessibility, testing	portfolio, e-commerce UI, dashboard	Vite, Tailwind, React Router, TanStack Query, Playwright
Back-end (Node.js)	Node, Express/NestJS, SQL/MongoDB, auth, security	REST API with auth, URL shortener, job queue	Postman, Docker, Redis, Prisma, Jest

Path	Must-have skills	Typical first projects	Tools
Full-stack (MERN)	both of the above + deployment	course portal, job board, chat app	Next.js, MongoDB Atlas, Render/Vercel, GitHub Actions
Mobile (React Native)	React, React Native, device APIs, app publishing	expense tracker app, news reader, delivery tracker	Expo, React Navigation, AsyncStorage, Firebase

Frameworks provide declarative, component-based UIs: in React, components are functions returning JSX, props pass data down, useState holds state that re-renders the UI, and useEffect synchronises with the outside world; Angular, Vue and Svelte share the same ideas, and Next.js adds routing, server rendering and API routes; MERN is the common full-stack path. Big-O analysis guides the choice of structures: arrays (O(1) push/pop, O(n) search and shift), Map/Set (O(1) average lookups), linked lists, stacks and queues, binary search trees with in/pre/post/level-order traversals, graphs with BFS (shortest paths), DFS and topological sort, merge sort and quick sort, and the core patterns: hash maps, two pointers, sliding window, recursion and dynamic programming. Build and deploy capstone projects (portfolio, expense tracker, quiz, e-commerce cart, full-stack app with auth, real-time chat). Prepare interviews with the core concepts checklist, output prediction, machine coding, polyfills and patterned DSA practice, and present projects with strong READMEs. Careers include front-end, back-end Node.js, full-stack MERN and React Native mobile development.

PRACTICE CHECKPOINT

Build and deploy **one full-stack capstone project** with a GitHub repository, and solve **50 DSA problems** in JavaScript.

Use 25.3.5 as the project brief and Modules 19, 22 and 24 as the guide: a layered Express API with MongoDB or PostgreSQL, JWT authentication and validation; a React or vanilla front end using fetch with loading and error states; tests with Vitest/Supertest and one Playwright journey; ESLint and Prettier; a README with screenshots; and deployment on Render (API) plus Netlify or Vercel (front end), with environment variables configured on each platform. For the DSA target, follow the six-week plan in 25.4.5, keep each solution in a `dsa/` folder of your repository with a comment stating the pattern and complexity, and add a test file so the solutions can be re-checked with one command. The tracker below is a small starting point that verifies solutions automatically.

A DSA PROGRESS TRACKER

main.js

```
// dsa/tracker.js: run your solutions against test cases and track progress towards 50
const solutions = {
  "two-sum (hash map)": { fn: (nums, t) => { const m = new Map(); for (const [i, n] of nums.entries())
{ if (m.has(t - n)) return [m.get(t - n), i]; m.set(n, i); } return null; }, cases: [[[2, 7, 11, 15],
9], [0, 1]], [[3, 3], 6], [0, 1]]] },
  "valid-parentheses (stack)": { fn: (s) => { const st = [], p = { ")": "(", "]": "[", "}": "{" }; for
(const c of s) { if ("([{".includes(c)) st.push(c); else if (st.pop() !== p[c]) return false; } return
!st.length; }, cases: [["()[]{}"], true], [["(]"], false]] },
  "max-subarray (Kadane / DP)": { fn: (a) => { let best = a[0], cur = a[0]; for (const x of
a.slice(1)) { cur = Math.max(x, cur + x); best = Math.max(best, cur); } return best; }, cases: [[[-2,
1, -3, 4, -1, 2, 1, -5, 4]], 6], [[-1]], -1]] },
  "binary-search": { fn: (a, t) => { let lo = 0, hi = a.length - 1; while (lo <= hi) { const m = (lo +
hi) >> 1; if (a[m] === t) return m; a[m] < t ? (lo = m + 1) : (hi = m - 1); } return -1; }, cases:
[[[-1, 0, 3, 5, 9, 12], 9], 4], [[1, 2], 3], -1]] },
  "reverse-string (two pointers)": { fn: (s) => { const a = [...s]; for (let i = 0, j = a.length - 1;
i < j; i++, j--) [a[i], a[j]] = [a[j], a[i]]; return a.join(""); }, cases: [["hello"], "olleh"],
[[""], ""]] },
};

let solved = 0;
for (const [name, { fn, cases }] of Object.entries(solutions)) {
  const ok = cases.every(([args, expected]) => JSON.stringify(fn(...args)) ===
JSON.stringify(expected));
  if (ok) solved++;
  console.log(`${ok ? "✓" : "✗"} ${name}`);
}
const bar = "".repeat(solved) + "".repeat(50 - solved);
console.log(`progress: ${solved}/50 ${bar.slice(0, 25)}... keep going!`);
```

OUTPUT

```
✓ two-sum (hash map)
✓ valid-parentheses (stack)
✓ max-subarray (Kadane / DP)
✓ binary-search
✓ reverse-string (two pointers)
progress: 5/50 ...... keep going!
```